Skills Lab

Data Creators Programme 2026

Four Learning Days and Friday Capstone

Contents

Teaching programme 4
Monday — Designing data products 8
Monday's plan 8
Course kickoff and welcome 9
Introductions and product experience 10
The CG product-information case 12
How to use today's examples 13
What is a data product? 14
Designing data products 17
Interviewing producers and consumers 20
Group challenge: producer, consumer and creator interview 22
From interviews to a design brief 23
Group challenge: the design canvas 23
Cases: evaluating a product design 24
Data mesh and product ownership 27
Optional group challenge: discover your use cases 27
Quality and the data contract 29
SLAs, SLIs, SLOs and RAID 30
Prioritising promises: RAID, RICE and MoSCoW 33
Data product adoption 35
On screen: Data Marketplace 36
On screen: Enterprise architecture 39
Worked example: Morningstar exposures 41
How can a creator participate earlier in design decisions? 42
Annotated contract and stakeholder map 42
Tuesday — AI, Catalyst and the platform 44
Tuesday's plan 44
From a business need to a reviewed Catalyst draft 46
Choosing the harness settings for the task 50
Deliberate prompting and review 50
Catalyst readiness check 51
Worked Catalyst review: from interview notes to contract 52
Group challenge: a PRD with Catalyst 54
Exercise: generate a data contract with Catalyst 56
Refining the data contract with Catalyst 56
Exercise: interviewing Catalyst 63
Catalyst skills, phases and roles 63
From a reviewed specification to a generated model 65
The paved path: Capital Group's platform 66
Local ownership and the shared service 72
Dev container setup 72
Tuesday design and readiness review 74
On screen: Platform onboarding 75
On screen: The paved path 77
On screen: Data contracts 77
On screen: Catalyst skills 78
On screen: The DPLC skills map 80
Platform decisions: why the paved path works this way 83
Lifecycle, roles and warranty 95
Context use, retrieval and durable decisions 96
Why code gives useful but incomplete validation signals 96
How can a consumer use AI without Catalyst? 97
How should a team investigate an AI-assisted workflow? 97
Solutions: Tuesday — AI, Catalyst and the platform 97
Wednesday — Data systems and dbt 100
Wednesday's plan 100
How Wednesday and Thursday run 102
Data systems: database, warehouse, lake and lakehouse 103
On screen: S3 data lake 105
On screen: Databricks 106

Storage architecture: lakehouse, ACID and data mesh . . . 108
Data pipelines, the medallion system and grain . . . 114
Source-to-target mapping, transformation rules and traceability . . . 116
Worked example: one STTM row into code . . . 118
dbt question-led teaching — Portfolio Analytics . . . 120
Git and the paved path: a recap . . . 120
On screen: Repository anatomy . . . 122
On screen: The pull-request workflow . . . 124
Collaboration preparation: a reviewed Git change . . . 127
Docker: the problem containers solve . . . 129
Inside a container . . . 132
Anatomy of a Dockerfile . . . 134
Kubernetes: infrastructure as configuration . . . 136
Docker practice and diagnosis . . . 137
Portfolio Analytics setup . . . 138
`dbt build` and `--select` . . . 139
Review the model and its evidence . . . 140
Review challenge: find the defects in a generated model . . . 141
Optional enrichment: Catalyst's dbt skills . . . 143
Wednesday review: from rule to evidence . . . 145
Where should a transformation live? . . . 145
Solutions: Wednesday — Data systems and dbt . . . 146
Thursday — Orchestration and operations . . . 148
Thursday's plan . . . 148
On screen: Thursday preview . . . 149
Group challenge: drawing your data pipeline as a flow . . . 150
airflow question-led teaching — Portfolio Analytics . . . 153
The paved-path scheduling recipe on one page . . . 153
Airflow challenges: setup, known issues and walkthrough . . . 154
Guest presentation (optional): a production data product end to end . . . 159
After the Airflow practice . . . 162
Data operations: the tools around a product . . . 163
Guest demo: a real paved-path data product, end to end . . . 164
How does a data product get released? . . . 166
Group exercise: approve or request changes? . . . 172
Deployment environments at Capital Group . . . 173
Containers in deployment . . . 173
Kubernetes and the layers of orchestration . . . 174
Harness, indirection and Conway's law . . . 175
The cloud . . . 176
Reading a release pipeline . . . 177
The 3 a.m. failure sequence . . . 177
Operations reference . . . 178
Thursday review . . . 183
On screen: Pull requests in practice . . . 183
On screen: The notifications Lambda . . . 184
On screen: Amazon MWAA . . . 187
On screen: Finding DAGs . . . 188
On screen: DAG structure . . . 190
On screen: Runs & task instances . . . 192
On screen: DAGs that trigger DAGs . . . 196
On screen: Astronomer . . . 198
On screen: Signing in to AWS . . . 201
On screen: Projects & pipelines . . . 202
On screen: Importing a pipeline from Git . . . 204
On screen: Pipelines as code . . . 205
On screen: Stages & environment promotion . . . 207
On screen: Running pipelines . . . 209
On screen: Executions, logs and troubleshooting . . . 211
On screen: Terraform in the repository . . . 214
On screen: Terraform Cloud . . . 216
On screen: Helm charts . . . 219
On screen: Datadog and Retina . . . 221
On screen: Kubernetes with Lens . . . 223

On screen: Reading a pod spec . . . 226
Friday briefing and course review . . . 232
Friday schedule . . . 232
Work and evidence . . . 232
Course review . . . 233
How do you know when a data product is done? . . . 237

Projects and Capstone 240
Task index . . . 240
Git collaboration . . . 240
Docker and runtime boundaries . . . 241
dbt — Portfolio Analytics . . . 241
Databricks discovery . . . 241
Airflow — Portfolio Analytics . . . 242
Kafka optional clinic . . . 242
CG product-information capstone . . . 242
Route boundaries . . . 243
Git collaboration . . . 246
git challenges . . . 246
git: Phase 1 — foundations . . . 248
git: Phase 2 — advanced practice . . . 256
git optional stretch . . . 263
Docker and runtime boundaries . . . 268
docker challenges . . . 268
docker: Phase 1 — foundations . . . 269
docker: Phase 2 — advanced practice . . . 277
docker optional stretch . . . 284
dbt — Portfolio Analytics . . . 288
dbt — Portfolio Analytics . . . 288
dbt: Phase 1 — foundations . . . 292
dbt: Phase 2 — advanced practice . . . 301
Airflow — Portfolio Analytics . . . 312
airflow challenges . . . 312
airflow: Phase 1 — foundations . . . 315
airflow: Phase 2 — advanced practice . . . 324
Databricks discovery . . . 334
databricks-cli challenges . . . 334
databricks-cli: Phase 1 — foundations . . . 335
CG product-information capstone . . . 344
CG product-information business brief . . . 344
Three enquiries to answer . . . 345
Your source scope . . . 345
Evidence boundaries . . . 346
Phases and completion evidence . . . 346
Phase 0: Design and readiness . . . 346
Phase 1: Certified data creator . . . 346
Phase 2: Technical practitioner . . . 346
Phase 3: Advanced practitioner . . . 347
Report, presentation and submission . . . 347
Design workshops . . . 350
Write one from your build . . . 350
Optional activity: stakeholder interviews and a Catalyst contract . . . 350
Optional activity: interviewing with an empathy map . . . 352
Optional activity: planning stakeholder communication . . . 354
Further practice and optional clinics . . . 358
Version 2 challenge route . . . 358
Portfolio Analytics runtime . . . 359
Historical dbt extension . . . 359
Historical airflow extension . . . 359
kafka challenges . . . 359
kafka: Phase 1 — foundations . . . 361
CG capstone optional practice . . . 368
Optional reference examples . . . 368
SQL readiness check: five patterns . . . 369

Temporal-history clinic . . . 369

Reference appendices 372
Appendix A — Readiness . . . 374
Investment and organisational context . . . 374
Why the course uses a dev container . . . 377
Open the correct project . . . 377
Find files and previews . . . 378
Open a terminal and navigate . . . 378
Setup and readiness, in order . . . 379
Environment readiness check . . . 380
Assigned dataset and runtime readiness . . . 381
Platform conventions and vocabulary . . . 381
Organisational context . . . 384
Acronyms . . . 387
Appendix B — Data contracts and decision records, worked . . . 394
Recording decisions: the RFD . . . 394
Classification and the PIA gate . . . 395
One filled contract . . . 395
Change management worked through . . . 399
Appendix C — Capital Group conventions . . . 404
Reference catalogue . . . 404
Appendix D — dbt and tooling reference . . . 408
Slowly changing dimensions with `dbt snapshot` . . . 408
dbt and Airflow reference . . . 409
Reference recap: dbt concepts and evidence . . . 412
Connected dbt profile fields and targets . . . 414
Airflow: DAGs, schedules, retries and alerts . . . 414
Appendix E — Environment and troubleshooting . . . 424
Troubleshooting runbook . . . 424
Screen index . . . 425
Appendix F — Managing stakeholders . . . 428
Empathy maps . . . 428
Stakeholder roles, influence and adoption . . . 430
Appendix G — AI reference . . . 434
Model training and post-training . . . 434
Generated claims and the word “hallucination” . . . 434
Imitation and simulation as teaching analogies . . . 434
Repeated actions, correlated mistakes and responsibility . . . 434
The economics of AI: a dated framework . . . 435
Appendix H — Using VS Code and the training dev container . . . 438
What’s inside the environment . . . 438
Check your understanding . . . 439
First v2 command and everyday reference . . . 439
Appendix I — Historical data and bitemporal loading . . . 442
Three kinds of history . . . 442
Define time and grain before writing the merge . . . 442
A reference-data correction across both clocks . . . 443
Late-arriving dimensions . . . 444
Balance and positions history . . . 445
Source delivery policy and loading policy . . . 445
Prepared models and custom loading macros . . . 446
Corrections, replacements, and deletions . . . 447
Checkpoints, replay, and retained history . . . 447
Verify behavior with small histories . . . 448
Appendix J — Astro operations and runtime ownership . . . 450
Federation and the service hierarchy . . . 450
Provisioning and configuration authority . . . 450
Connections across hosted and classroom environments . . . 451
Runtime inventory and upgrade evidence . . . 451
Vulnerability ownership and shared templates . . . 452
Releases while work is active . . . 452
Capacity, remote compute and cost . . . 453
Availability, disaster recovery and support . . . 453

Review questions 454

Teaching programme

The programme runs from Monday to Friday. Use Appendix A — Readiness for preparation, business terms and environment setup. Practical instructions are in Projects and Capstone.

Begin with Appendix A — Readiness and the CG product brief. The internal analyst needs offering identities, expenses and performance evidence. For a real product, agree roles and approval decisions with the responsible people.

Monday — Designing data products

Monday's plan

Monday starts with a consumer and a decision. Your team works out who needs the product and what decision it supports, where the data comes from and what you're promising. You also write down what you don't know yet. The rest of the week builds on that design. On Tuesday Catalyst drafts from it, on Wednesday some of your transformation rules become dbt models and tests, and on Thursday Airflow runs them in order. On Friday you build and present a CG product-information data product.

The Policy Team joins us at 09:00–09:30. They'll talk about how the course fits the wider data creator initiative, and which parts of a training example are real organisational requirements. If they can't answer a question on the day, put it in your team's assumption register with the role that can answer it.

Working time	*Focus*	*What your team produces*
09:00–09:30	Policy Team guest	Questions for the assumption register
09:30–10:15	Product promises, assets and alignment	Named consumer and decision
10:15–10:30	Morning break	
10:30–11:30	Producer, consumer and creator interview	Interview notes and a requirements handoff
11:30–12:00	Reuse decision and design canvas; discovery optional	Reuse, compose or build decision; optional use-case list
12:00–13:00	Lunch	
13:00–14:00	Contract meanings and plain-English mapping	Product promises, glossary and mini-STTM
14:00–14:15	Afternoon break	
14:15–15:00	Service levels, prioritisation and adoption	SLAs, ranked commitments and adoption responsibilities
15:00–15:30	Review and handoff	Finished design brief

The day runs from 09:00 to 15:30, including the listed breaks and one-hour lunch. Keep the final review inside that boundary; optional discovery depth belongs after the core or in separate practice.

On screen today

System or sequence	*Why preview it*	*Teaching and screens*
Data Marketplace: search, owner, schema, dictionary and current/history views	Assess an existing candidate before choosing reuse, composition or a new build; distinguish a sample preview from coverage and quality evidence.	Reuse criteria; Marketplace sequence
Enterprise architecture: capability, application record and classification	Connect the product to a business capability and accountable application owner, then identify the field-level classification questions still to answer.	Access and approvals in the contract; Enterprise architecture sequence

Today's sessions

- Course kickoff and welcome
- Introductions and product experience
- The CG product-information case
- How to use today's examples
- What is a data product?

- Designing data products
- Interviewing producers and consumers
- Group challenge: producer, consumer and creator interview
- From interviews to a design brief
- Group challenge: the design canvas
- Cases: evaluating a product design
- Data mesh and product ownership
- Optional group challenge: discover your use cases
- Quality and the data contract
- SLAs, SLIs, SLOs and RAID
- Prioritising promises: RAID, RICE and MoSCoW
- Data product adoption
- Worked example: Morningstar exposures
- How can a creator participate earlier in design decisions?
- Annotated contract and stakeholder map

Course kickoff and welcome

> Begin by identifying the course responsibilities, your product experience and the support you need for practice.
>
> **THIS SESSION ANSWERS**
>
> - Why is Capital Group running this course?
> - How do the five days fit together?
> - What do you need to set up before Wednesday?
> - Which data products do you create or consume, and how can they fail?

> Knowledge check: The creator role
>
> What does a creator own when a product uses shared platform tools? What would you need to validate before a generated asset is usable?

Coordinator and leadership welcome

The programme develops product-design skills alongside the supported platform tools. Creators translate a business need into a reusable product, then validate its meaning, quality and delivery promises.

Participate in the exercises, ask for help when a prerequisite is missing, and give specific feedback on unclear material. Build a support network: identify the colleagues and platform roles who can resolve questions your team cannot answer.

Instructor and Decoded

Decoded develops and delivers the programme with Capital Group subject-matter experts. The delivery team supports the design exercises and technical practice. Use the current instructor brief for facilitator names, access support and session arrangements.

Course overview

The five days connect four skills: designing a product for a consumer, reviewing Catalyst output, using the paved-path tools, and presenting a checked product.

- **Monday–Tuesday:** interview consumers and producers, assess reuse, specify promises, and review a Catalyst-generated contract. Tuesday afternoon introduces the platform and prepares the development environment.

- **Wednesday–Thursday:** implement mappings as dbt models and tests, then schedule and inspect execution with Airflow. Git and containers support a reviewable, repeatable development workflow. Release and infrastructure demonstrations explain the wider operating context.
- **Friday:** build the assigned CG product-information slice, reconcile its outputs and present a consumer handoff.

Business and technical participants work together throughout. Experienced engineers can help with implementation while practising interviews and design; less technical participants learn enough to assess the models, tests and execution evidence.

The aim is to review and validate a product with the supported tools. Five days cannot cover every product or platform integration. Use current Confluence guidance and EDO support for policies, access and follow-on enablement.

One chain runs through the week: a contract clause becomes a model and test, the test runs in a DAG, and a failure requires diagnosis, recovery and communication. Tool choices involve trade-offs: a stricter schema is easier to enforce and harder to change; a finer-grained DAG is easier to inspect and costs more to operate.

Logistics

Use the day plans for working times and breaks. Raise access or setup issues before Wednesday's practice. Keep a shared assumption register and ask the relevant business or platform role to resolve questions that the training brief cannot answer. Instructor demonstrations use prepared examples; screenshots illustrate workflows and are not current policy or permission grants.

Introductions and product experience

Use your own product experience to identify the decisions, tools and responsibilities you want to practise. The activity and cases below connect that starting point to a consumer brief.

Prior knowledge and expectations

> Knowledge check: Your starting point
>
> Which programming or query tools have you used? What experience do you have with dbt, Git, Docker, Airflow or release pipelines? Which product-design responsibility is new to you? Use the answers to identify support needs before practice rather than to rank another learner's experience.

> Introduce your product context
>
> OUTPUTS A named business product, consumer decision and two pain points
>
> Introduce your role and expectations. Choose a data product your team creates or consumes and explain the decision it supports. Describe how it can fail: include one data or service problem and one ownership, access or communication problem. Keep this product as a reference example for interviews and contract design.

> Case study: Common product responsibilities
>
> **Cases: product and service pain points**
> Personal-trading, legal and ESG data can require entitlement decisions, consistent formats, ownership and an assessment of downstream impact. Decision: identify the owner and authorised consumers before interpreting a formatting fix as a complete solution. Reasoning: a usable file can still be inaccessible or expose information to the wrong audience. Lesson: treat access and accountability as product promises.
> Exposure data represents portfolio risk at a date, often with proportions between zero and one across geography or style. Missing or late vendor inputs and inconsistent definitions undermine downstream comparisons.

Decision: agree the unit, grain, definition and expected arrival, then test the output against them. Reasoning: technical completeness alone cannot resolve two different meanings of “exposure”. Lesson: the glossary and source commitment belong beside the model.

Investment-team reference data describes managers, covered funds and experience, sometimes combining HR-derived information with other sources. Global-results consumers may receive the same underlying source through files or APIs with different service expectations. Decision: record delivery formats and service commitments for each consumer class. Reasoning: one producer can support several products, and different delivery paths can fail differently. Lesson: distinguish a shared source from the promises made by each downstream product.

Climate, transactions, analytics and investment-operations data can cross privacy, compliance and vendor-contract boundaries. Decision: classify fields and intended uses before joining the sources. Reasoning: a useful combined analysis may still require restricted access or a permitted-use decision. Lesson: a platform such as Caspian hosts multiple data domains; naming the platform does not define a product or authorise every combination.

Case study: Integration problems and design responses

Cases: boundaries, timing and identifiers

A missed SLA may expose a pipeline problem or a mismatch between producer and consumer deadlines. Decision: record the end-to-end dependencies, arrival times and checks rather than assuming an identical dbt pipeline removes every discrepancy. Reasoning: sources can still have different commitments. Lesson: measure the consumer’s required outcome.

A positions-and-transactions mapping question may cross domain and engineering boundaries. A support ticket or unanswered shared inbox provides little value without someone who can route the business question. Decision: name a liaison and document meanings beyond the basic asset, position, return and fund vocabulary. Reasoning: source teams own transformations while consumers need an explanation of the resulting product. Lesson: an accountable contact and domain knowledge can be as necessary as correct SQL.

Different aggregations can produce different position values; different identifier-generation rules can create a portfolio UID collision, or leave CUSIP/ISIN-like security identifiers unmatched. Decision: agree a canonical identity, mapping rules and reconciliation checks. Reasoning: joining plausible keys can silently associate the wrong records. Lesson: cross-system unique identity must be specified and tested, even when names appear consistent.

Vendor products can change arrival times, restate values and restrict permitted use. API or warehouse connections also carry cost. Decision: record the licence, correction behaviour, procurement cost and approved delivery route before building a bespoke report. Reasoning: a technical connection does not settle rights to redistribute or repurpose data. Lesson: include vendor dependencies in feasibility and viability.

A creator translating between camelCase and underscore schemas can repeatedly absorb work from both producer and consumer. Decision: declare a stable interface, mapping and change policy. Reasoning: uncontrolled translations create discrepancies and support work. Lesson: make the boundary and responsibility visible rather than becoming an unbounded intermediary.

Key course terms

- Data creator: the person who turns source data and a consumer need into a specified, validated data product.
- Paved path: the supported tools and conventions for creating products in the Capital Group training context.
- Catalyst: the supporting AI tool used to draft and revise product artefacts; the creator reviews and owns the output.

Who creates, owns and consumes the product?

> Knowledge check: Map the responsibility
>
> For your example, who produces the source, owns its meaning, transforms it, approves access and uses the result? Which role would answer a question about an unexplained position or return? Identify a contact role for any missing responsibility.

The workshop examples mention EDO, CSG, GBS and Investment Operations, along with teams working on regulation, platforms and advisor-facing applications. Use these examples to ask who supplies information, who makes decisions about it and who needs the result. A department name alone does not answer those questions. For your own product, check the current responsibilities with the people who hold them. The course's "self-service, simplify, scout" approach encourages finding and reusing useful capability before building something new.

The CG product-information case

> A fictional internal Product Information Analyst supports sales and relationship colleagues answering financial-professional enquiries. The analyst needs traceable product identities, published fund expenses and performance context. Use the CG business brief as the authority for this classroom role and source scope.
>
> **THIS SESSION ANSWERS**
>
> - E1: What does the snapshot say about CGGR's vehicle and net expense ratio, and where did the answer come from?
> - E2: Which scoped fund offerings have published net expenses no greater than 50 bps, and which have unknown costs?
> - E3: Which three-year annualized NAV returns reported at September 30, 2026 meet the declared comparison rules?

> Knowledge check: Reading the CG product brief
>
> Who is the direct internal consumer, which enquiry are you answering, and which source limitation affects the answer?

Use three roles. The consumer explains the enquiry and their current research process. The producer explains the frozen public representations, dates and extracted fields. The creator connects those facts to a reviewed output. A fourth person can challenge quality assumptions. The role and enquiries are fictional classroom scenarios, rather than statements about an actual CG department.

Choose product lookup and one contrasting enquiry, such as the expense shortlist or performance evidence. Work backwards from each consumer need to its output, source selection, transformation and independent check. Keep the rules in plain English: select the declared representation, preserve the product ID and source link, and convert percentage points to basis points by multiplying by 100.

Element	*Proposed design*	*Question to resolve*
Consumer	Internal Product Information Analyst	Who validates the classroom handoff?
Outcome	Traceable answers to E1, E2 and E3	What remains a follow-up question?
Measure	Published net expense ratio and declared return figures	Are basis, period and reporting date compatible?
Grain	One row per scoped offering in the catalogue	How do catalogue/detail records and share classes affect identity?
Quality	Unique IDs, complete scoped membership and retained provenance	How will unknown expenses remain visible?
Access	Frozen public evidence and synthetic training enquiries	Which approved groups would use a real product?
Ownership	Classroom creator, producer and consumer roles	Who owns exceptions and proposed service promises?

The scope is 25 ETFs, 65 Class F-3 mutual-fund identifiers and 19 SMA strategy IDs. These are 109 offerings, rather than 109 underlying portfolios. All selected ETF fee reporting dates are unknown; SMA returns cover only 12 strategies. Do not replace missing costs or dates with zero or retrieval time. SMA composite results and individual account returns have different meanings.

For an empathy map, separate what the consumer said from your interpretation. A request for a traceable source link is evidence; an invented refresh commitment is an assumption. Record what would resolve each uncertainty before turning it into a product promise.

How to use today's examples

The motivations later today (power, control, competence, trust, esteem) help you ask better questions about what someone needs. When a consumer objects to a change, look at the objection itself first. It's often a real problem with the product. And someone with no formal influence can still know things you need.

Source-aligned, domain-aligned and consumer-aligned describe who the data serves. The CG product combines selected public representations, reusable offering/cost definitions and enquiry-specific outputs. Name the consumer and explain what each intermediate table is for.

Use CG product-information for today's required interview and design canvas. The other business cases illustrate design choices in different domains; use them to test your reasoning or choose an optional follow-up.

Today you frame the need, interview consumers, decide whether to reuse an existing product or build one, and agree its promises in a design brief. That brief is the first draft of the contract. Later in the week you connect the contract to tests, an execution graph and a reviewed handoff. Operational examples show where alerting, support ownership and a runbook fit.

What is a data product?

> Different teams may use different definitions of a data product. Establish a working definition, then test it through an interview about a consumer's decision.
>
> **THIS SESSION ANSWERS**
>
> - What is a data product, and how is it different from a data set?
> - Should we create a data product?
> - What data product should we create?
> - How do you find out what a consumer actually needs?

> Knowledge check: Product or data source?
>
> What promise turns a table into a useful product for your consumer? Can the same output be a product for one team and a source for another?

Consider several candidate properties: consumable, discoverable, trustworthy, accessible, observable and aligned with enterprise standards. Useful metadata names a business purpose, owner and lifecycle: planning, design, creation, governance and observation. A "single source of truth" is a claim to examine against the intended decision and source limitations.

Two teaching points follow:

- Wrong answers are welcome. Part of learning is *unlearning* what we've misunderstood.
- A data product is more than a data set, or a data set plus a view (a dashboard or report). It's that *plus something*, such as discoverability, quality properties and above all trust. "A data set we can trust" is a fair first pass.

There are three views of what a data product is. They are not equivalent, but they describe the same thing from different angles.

1. The promises view

The simplest formulation is that a data product is a data set plus a set of promises.

You could think of the promises as SLAs, but it is broader than SLAs:

> data set + SLAs + ownership + governance + quality + ...

All of these reduce to *someone promising something* about the data. A data product is not a data set, and not a property of a data set. It consists principally in the promises. When we ship a data product, we are releasing something with a set of promises attached: it will have a certain quality, it will arrive at a certain time, it has an owner who will manage it, respond to emails, and so on. Otherwise it's just a dump of data, and no one cares about that.

A promise creates an obligation to a consumer. State who is responsible, what will be delivered and how a failure will be handled. A dataset without those commitments leaves the consumer to discover its limitations.

2. The asset view

A data product is code + infrastructure + data:

- the code that produces the data,
- the infrastructure the code runs in,
- and the data itself (plus metadata).

Code automates the promises: building and serving the data, enforcing constraints, testing quality, monitoring delivery and recording failures. The infrastructure supplies the resources and interfaces that let that code run.

3. The alignment view

Alignment describes whose needs shape the vocabulary, structure and meaning of an output. A source is arranged for its producer; a product adapts that information for a consumer's decision.

A data product transforms data from a source-aligned state to a consumer-aligned state.

- *Source-aligned* means aligned with the interests of the outside party that produced it, some person or organisation that isn't us, who structured the data for their own purposes.
- *Consumer-aligned* means aligned with the interests of our consumer.
- The data product creator sits in the middle and does the realignment.

For an illustrative vendor-feed case, suppose a source supplies equity observations and a consumer needs fund-level exposure. Confirm the supplier's actual fields, identifiers, permitted use and provenance before designing that output. A provider name such as Morningstar does not establish fund ownership, manager assignments or an official book of record. The creator realigns verified source information to the agreed consumer question. These are relative terms: a consumer product for one team can become a source for another.

A second example: an Apple Watch could give you a thousand heart-rate readings a second. That's aligned to the sensor. The consumer wants to know whether their heart rate is healthy, whether they're improving, whether the exercise is working.

The heart of designing a data product, beneath the data, code, infrastructure and properties like trust, is taking something the consumer doesn't want (a data source) and transforming it into something they do want. The transformation from an uninteresting source to a useful product *is* the product. Alignment covers more than schema, values and rows. It includes why the vocabulary is the way it is, what a column name *means*.

Domain-aligned, and why these are relative terms

A third label is sometimes used. Domain-aligned data has been transformed away from the source into something reflecting a business or subject area, but not yet aligned to a particular consumer. In practice, if you merge multiple sources (an Apple Watch, a room thermostat and others) into "health data", the intermediate result reflects what the data is *about* rather than anyone's goals. Even more practically, there's an input folder, an output folder, and intermediate folders of transformed stuff in the middle. The middle ones are the "domain" ones.

These labels are relative to the product boundary. A consumer-ready output for one team can become source data for another.

- There is no such thing as "a data source" in the world. *Your product is someone else's source*. Labelling something a data source really says "this was produced by someone who isn't me, aligned to *their* interests"; consumer-aligned says "aligned to *my* interests" (or to the people we care about).
- In this course, the creator is accountable for a defined product boundary and its consumer requirements. That consumer may be another data team: a reusable domain-aligned product can be a legitimate deliverable. Alignment does not impose a universal CG restriction on which products a creator can build. Confirm the actual owner, consumers and responsibilities for the assigned product.

One possible interpretation is a ladder: source-aligned = data (raw temperatures per second), consumer-aligned = information (the trend), domain-aligned = insight (multiple data points combined). That interpretation is not a fixed definition; the ladder projects *your* goals onto the steps.

To the factory making the sensor (testing a resistor), temperature per second *is* the insight. That's their consumer-aligned product. Go a level below and another team dealing in voltages and resistors thinks *that* is the insight. From different positions in the chain, the three labels rotate. The labels change with the producer and consumer boundary.

Alignment, and whose needs the data serves

A source-aligned table reflects its producer. A domain-aligned table defines reusable offering identity or expense meaning; a consumer-aligned output answers the analyst's lookup, shortlist or comparison enquiry.

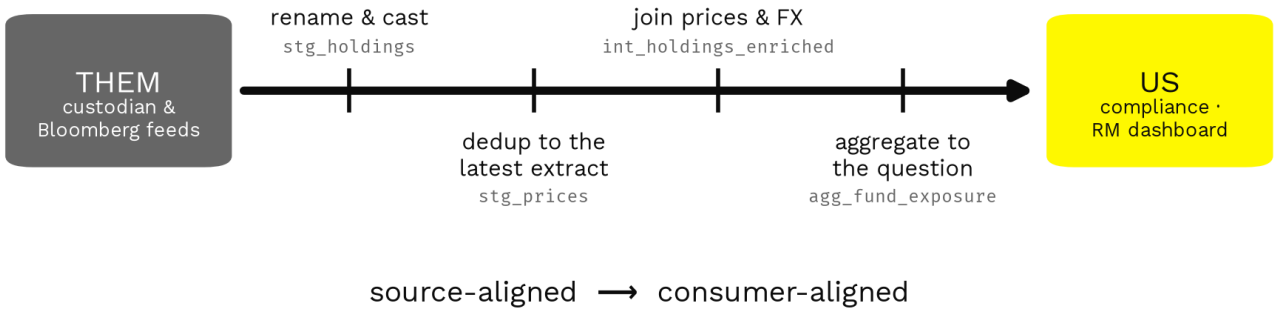


Figure 1: Alignment: whose needs the data serves

Reconciling the three views

- Most promises are automated by code. If you promise delivery at a certain time, code delivers on that promise. That connects the promises view to the asset view.
- Alignment is a specific, not-fully-specified kind of promise: a promise that the product will *meet your needs*. That connects the alignment view to the promises view.

Worked example: ESG data

- Source: MSCI provides environmental and social data, e.g. CO_2 data per company, plus lists of properties.
- Consumer product: an exposure or risk report, e.g. exposure to EU carbon taxes, perhaps 0.01 of risk per million tonnes of CO_2 produced by held companies.
- Promises we make to the customer: the report will be available at a given time (and if the promise itself says "95% of the time", then hitting 95% is keeping the promise, because the promise sets the expectation); there is a clear internal data owner who handles governance (who has access), quality validation, and so on.

Why do data products fail?

Consider both engineering and organisational failure conditions.

Engineering answers: lack of governance; quality; security/access control; non-functional requirements failing (performance, access, formats, where "non-functional" means requirements not associated with the operational functioning of the system).

Repeated departures from a documented process can become accepted practice. Record exceptions and their consequences so a workaround does not silently replace the agreed quality or change policy.

Human answers (the part engineers miss, issues that arise *between people*): the product is not needed, misaligned requirements, lack of adoption, loss of trust, lack of training or onboarding, and poor communication.

How do we solve those problems?

- Profile the consumer. Understand customer needs *before* building.
- Communication.
- Data lineage. Understand the source of the data and the events in its lifetime.
- Improve the technology stack (with the caveat that "improve" often just means "update").
- Ownership and accountability. A defined owner, and responsibility.
- Documentation. Note the link to trust. Accurate documentation records the thought process, requirements and decisions behind the product. Even if consumers only ever feed it into an AI model for a summary, having a sense of *why* decisions were made improves trust. Unexplained gaps can lead consumers to make incorrect assumptions about the data.
- Transparency, organisational as well as documentary. Be open with downstream consumers about what you're doing, release information early, bring people into meetings, offer access. It makes them feel secure.

Borderline questions

For this course, distinguish recording an event from transforming existing records for a consumer. An order-entry application creates transaction records; a reporting product reshapes those records under an agreed meaning and service promise. A larger system can contain both responsibilities.

Is the data product just the data asset, or the whole pipeline? It's the whole thing (code, infrastructure, data assets, promises), but the *emphasis* depends on your role. For data creators the emphasis is the data asset and the promises. Code and infrastructure are instrumental, the means of fulfilling the promises. Software engineers see it the other way around: code and infrastructure are their product, the data and promises are for other people.

So the smallest data product is an Excel file plus a promise to email it every day, and the largest is the lakehouse and everything in it.

Isn't ETL the same as what a data creator does? This is a real tension in the course. The person who does the consumer interview and the person who assembles lakehouse infrastructure have very different skill sets. So everyone creating data products should know something about the whole process, with role-dependent emphasis. Engineers benefit from understanding the design motivation so they can push back intelligently. Non-technical designers benefit from first-hand exposure to the systems so they can hold their own in engineering meetings.

Designing data products

> Knowledge check: Before choosing a product
>
> What would make reuse a better decision than a new build? Who can establish the value to the consumer?

Two questions concern us in the design of a data product:

1. Should we create a data product?
2. What data product should we create?

We bounce between these (today and tomorrow), because you need to know *what* the product is before deciding *whether* to build it, so there's a level of circularity.

Canvases. The general industry term for a grid of questions and prompts that helps you design something is a *canvas* (the fashion industry's equivalent is a mood board). We will use two:

- a design canvas, which answers the *should* question (covered later today, after the interviews)
- a product canvas, which answers the *what* question

The scoping questions

> Workflow
>
> The "should we build it?" decision can be framed as eight scoping questions:
>
> 1. Who needs it?
> 2. What's the value / ROI?
> 3. Can we build it (feasibility)?
> 4. Does it already exist?
> 5. How would it be adopted?
> 6. Marketplace and discoverability. How will consumers find it?
> 7. What SLAs would it carry?
> 8. Legal and compliance?
>
> Question 4 has three possible outcomes: subscribe to an exact match, compose from an extendable partial match, or build when no suitable product exists.

Reuse criterion	*Check*	*Red or yellow flag*
Freshness	Does the refresh cadence meet the target?	Monthly when the product needs daily data
Coverage	Are the required fields and entities present?	A key dimension is missing
Certification	Has the domain team certified it?	Uncertified or no owner listed
Lineage	Can we trace it to its sources?	No lineage documentation
Active consumers	Do other teams use it?	Zero consumers; certified with zero consumers is a yellow flag
Deprecation	Is it marked for retirement?	A deprecation tag or dated retirement plan

There's a worked reuse decision, including the benchmark-reuse judgement, in Appendix B.

You can also group the same questions into four themes. Is it desirable (what does the consumer need)? is it feasible for us to create? is it deliverable? is it viable? At this first stage you gather *just enough* information to decide whether to create the product at all, then make the decision.

Which questions need other roles? Separate the questions you can estimate from those that need consumer or business evidence:

- Hardest for the data creator to answer alone: desirability (who needs it, what value?), because *we are not the consumer*. Once we know who they are and what they want, we can estimate much of the rest.
- Easiest to answer ourselves: feasibility and delivery, meaning how we actually build and ship it.
- Most likely to be misunderstood: who needs it (requires business context) and metrics for success.
- Design implication: desirability and viability are the two ends with a human-behaviour element, hence hardest to predict; the central columns (feasibility, delivery) can largely be managed with

technology and tools. The outside columns are the unknowns, which sets up the next topic, interviews and requirement confirmation.

Four assessment themes on the design canvas

Desirable means the consumer needs it. Feasible means the data and the technology can produce it. Viable means it's worth what it costs, and deliverable means this organisation can build it and keep it running. A query that works can still fail on deliverability if nobody owns the data or you can't get access to it.

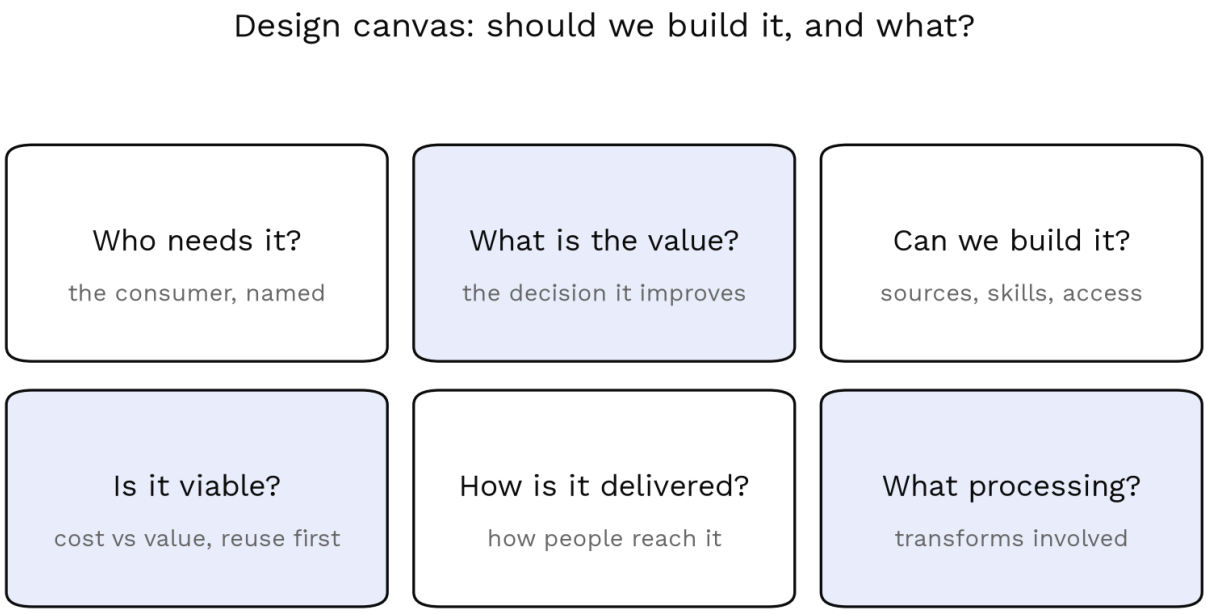


Figure 2: Six questions for assessing a product design

Finding out what data product to create

As the data creator, we are neither the consumer nor the source. We do the translation, so we don't automatically know what to build. Ways to find out from the consumer:

- Ask for a problem statement
- Survey them (email forms, questionnaires)
- Interview them (calls, meetings)
- Observe them (sit with the team, watch what they actually do)
- Research existing documentation
- Ask about pain points. What do you do now? what are the workarounds?

Strengths and weaknesses of each method

Method	*Useful for*	*Limitation to check*
Survey	Reaching many consumers quickly and comparing answers to known questions	Fixed questions can miss the real problem or reflect the designer's assumptions
Interview	Clarifying ambiguity through examples and follow-up questions	A small or unrepresentative sample can overstate one consumer's needs
Observation	Seeing actual workarounds, copying, reconciliation and overrides	Available tools can constrain behaviour; an observed action may not be the preferred workflow

Combine methods where the decision needs it. For example, an interview may reveal an approval delay, while observation shows that the available deployment process encourages an unauthorised shortcut. Validate the interpretation with the people doing the work.

Turning ambiguous accounts into requirements

A consumer's description may leave gaps, reflect competing priorities or omit steps that have become routine. A statement and an observed action can differ because of the environment, available tools, incentives or the person's understanding. Use multiple forms of evidence before concluding what should change.

A requirement that another organisation has solved still needs interpretation for this team: the available sources, decision rights and vocabulary may differ. Making a vague need precise is part of the creator's work. Like diagnosing a technical fault, start with the reported issue and investigate its cause. Check the resulting explanation with the consumer, then record the requirement and remaining uncertainty.

For difficult adoption or workflow concerns, Managing stakeholders expands evidence, observation and empathy methods. Continue with the core interview before the design canvas.

Interviewing producers and consumers

Interview the consumer about the decision and the producer about the source. Record what each person actually says, distinguish evidence from assumptions, and check your interpretation with them. A product can be technically correct and still miss the consumer's working deadline or meaning of a field.

> Knowledge check: Before the interview
>
> What would you ask a consumer who requests a dashboard? Which answers require a producer or a data owner? What observation would show whether the current workaround is meeting the need?

For a product with low adoption or a difficult change in working practice, consult Managing stakeholders. Its empathy map separates observations from hypotheses; it also includes an optional interview activity. Monday's core interview uses the brief and questions below.

Preparing for an interview

Prepare an evidence-led interview

DURATION 10 minutes within the interview session

OUTPUTS Role briefs, questions and an assumption register

1. Use the CG product-information brief, or a business product introduced in your opening discussion. Name the consumer, the decision and the producer. If roles overlap, identify which responsibility the person is answering for.
2. Write down what the brief already establishes: source tables, proposed row grain, output, working deadline and any quality or access constraints. Leave unspecified values as open questions.
3. Choose an interviewer/data creator, consumer and producer. The consumer prepares a description of a decision and current workaround; the producer prepares source meanings, availability and constraints. Role holders can add plausible training assumptions, labelled as assumptions.
4. Prepare open questions and follow-up probes. Start with a recent concrete example before asking for an ideal future state. Ask permission to inspect an example of the existing output or workflow.
5. Prepare a notes table with columns for statement, evidence/example, implication, unresolved question and owner. Record tensions between consumer needs and source constraints rather than resolving them silently.

Check: each role can explain their responsibility; the questions address the decision as well as data fields; assumptions have owners. No empathy-map output is required.

Interview questions

Questions and follow-up probes

OUTPUTS Decision, requirement and uncertainty notes

Use the questions as prompts, following the answer rather than reading a questionnaire mechanically.

Topic	*Open question*	*Follow-up probe*
Decision	What decision do you make with these data?	Walk me through the last time you made it. What changed as a result?
Current workaround	How do you make that decision today?	Show an example. Where do you copy, reconcile or override a value, and why?
Consumers	Who uses the result, directly or downstream?	Which roles have different needs? Who could confirm those differences?
Producer	How is the source generated and made available?	What does a row represent? Which fields change, arrive late or get corrected?
Definitions	What do these terms mean in your work?	What does “active”, “value” or “real time” mean in this example? Which dates, units and exclusions apply?
Quality	How do you recognise that the result is usable?	Which missing values or discrepancies stop the decision? What comparison or tolerance do you use?
Freshness	When do you need the result and why then?	Which business event sets that deadline? What happens on holidays or a late source delivery?
Access	Who needs access, and to what detail?	Which classifications, permissions or vendor terms constrain use? Who can approve them?
Failure impact	What happens when the data is late, wrong or missing?	What fallback is available? Who must be told, and how quickly?
Trade-offs	Which requirement matters most if time, cost or source coverage is limited?	What would be acceptable for a first release? What would make it unusable?

Delivery and adoption	How would the product fit into your existing workflow?	What format, support, training or evidence would help you adopt it?
Confirmation	What have I misunderstood or left out?	Who else should review this interpretation before we commit to it?

Check: capture the interviewee's example and your interpretation separately. Avoid suggesting a preferred answer, promising an unverified SLA or treating one consumer as the entire population.

Group challenge: producer, consumer and creator interview

Knowledge check: Listen for a requirement

How could the producer, consumer and creator describe different parts of the same problem? What would you ask to distinguish an observation from an assumption?

Interview for the CG product brief

DURATION 60 minutes (10:30 to 11:30)

OUTPUTS Interview notes, a requirements handoff and an uncertainty register

Work in threes using the CG product brief. The consumer asks E1, E2 or E3; the producer explains frozen products, fees and returns; the creator translates between them. Use the brief as authority and label added training assumptions.

1. Prepare role notes and select questions (10 minutes). The consumer describes how the result will be used; the producer explains what the source can provide. The creator initially sees only the shared brief.
2. Interview the consumer (15 minutes), asking for a concrete decision and current workaround. Probe timing, definitions, quality and failure impact. Read back two important requirements in the consumer's words.
3. Interview the producer (15 minutes). Check row grain, keys, join coverage, update and correction behaviour, access and upstream commitments. Ask how each consumer requirement maps to an available source or an unresolved gap.
4. Write a handoff (10 minutes): named consumer and decision; output grain and fields; glossary and proposed transformations; measurable promises; access assumptions; fallback; unresolved questions with responsible roles. Mark each statement as confirmed, proposed or unverified.
5. Review together against the original brief (10 minutes). Consumer and producer each identify a misinterpretation or missing condition. Revise the handoff and give the next decision an owner. A requirement that cannot yet be supported remains a question or risk.

Check: another creator can explain what to build and why; the producer recognises the source constraints; the consumer recognises the intended decision; conflicting deadlines and definitions are visible. The output feeds the design canvas. For a pair, interview the consumer first and use the source brief for producer evidence, recording the missing producer validation explicitly.

From interviews to a design brief

Should we build this data product at all, and if so, what is it? Your group answers that on the design canvas before lunch.

THIS SESSION ANSWERS

- Who needs it?
- What is the value of the data product?
- Can we build it?
- Is it viable?

> Knowledge check: Review the interview handoff
>
> Which statement is confirmed by the consumer, which by the producer, and which remains an assumption? What decision must the canvas resolve?

Use your interview notes to distinguish a requested feature from the decision and promises it supports. Review the handoff with another creator before completing the canvas.

For the rest of today:

- Design canvas. The questions that decide whether to build and what to build (group challenge, first thing).
- Prioritising promises. RAID, RICE and MoSCoW, for when not every promise can be kept.
- Data product adoption. A small topic on barriers to adoption and how to get past them, done while we're still in non-technical territory.
- Design documents. What you need to hand someone who is going to build a data product, including source-to-target mapping (STTM), transformation rules and glossaries, and the data contract they end up in.

Tomorrow we move on to working with AI and Catalyst, refine the data contract with it, and set up the dev containers, so the days get steadily more technical from here.

Group challenge: the design canvas

The canvas turns interview evidence into a decision about reuse, composition or a new product. Test desirability (a named consumer and decision), feasibility (sources, permissions and technical capability), viability (value against lifetime cost) and delivery (how the consumer obtains and adopts the result). Processing connects available source grain to the intended output.

Check your design vocabulary

> Knowledge check: From interview to specification
>
> Can you explain a source-to-target mapping rule for your product? Which SLA describes the promise, and which SLI measures it? Identify one feasibility question that the creator cannot answer without a producer or approver.

> Evaluate the product before building
>
> **DURATION** 30 minutes (11:30 to 12:00)
> **OUTPUTS** An evidence-labelled canvas and a reuse, compose or build decision
> **HANDOUT** The source prompts and checks below

Use the CG product interview handoff or the business product introduced earlier. Complete the ten canvas blocks below, recording evidence, assumptions and unresolved questions.

Theme	*Canvas block*	*Prompt*
Desirability	Audience	The consumers are ____, who decide ____ every ____.
Desirability	Actionable Insight	[Audience] can ____ because the product provides ____.
Feasibility	Data Sources	Fed by ____, owned by ____.
Feasibility	Data Processing	Transform ____ → ____ → ____.
Feasibility	Skills, Tools, and Methods	Needs ____ (skills/tools) and ____ (domain SME).
Delivery	Consumption Model	Consumers access it via ____.
Delivery	Adoption Plan	They discover and onboard via ____.
Viability	Lifetime Value	Saves / improves ____ worth ____.
Viability	Total Cost of Ownership	Build ____; run ____; support ____.
Viability	Risks	Top risks: ____.

For each answer, record its evidence, responsible role and implication for the decision. Check the coverage below with the consumer and producer rather than guessing missing facts.

Theme	*Coverage check*	*Evidence to retain*
Desirability	Named consumer population, decision, present workaround and improved outcome	Interview example and consumer confirmation
Feasibility	Source grain, keys, availability, access, skills and technical dependencies	Source brief, producer statement and unresolved approval
Viability	Benefit, build cost, recurring support/ procurement cost and reuse alternatives	Cost assumptions, reuse comparison and business justification
Delivery	Access method, format, deadline, onboarding and accountable contact	Consumer workflow, proposed service promise and adoption requirement
Processing	Required joins, filters, renames, casts, aggregation and quality rules	Proposed source-to-target mapping and glossary terms

Work for 20 minutes, compare your decision with another pair for 5 minutes, then revise for 5 minutes. Do not write every transformation rule in code yet; identify what must be specified. Joining Morningstar tables, filtering coverage, renaming or recasting fields and grouping into fund categories are examples of design choices that later become code.

Check: all ten blocks are complete, with an evidence-backed answer or an owned uncertainty; lifetime cost includes maintenance and communication; the output grain matches the decision; the reuse candidate has been assessed against freshness, coverage, ownership and access. Explain one difficult trade-off and the next decision required. “Reuse an existing product” is a valid outcome.

Cases: evaluating a product design

Knowledge check: Evaluate a design decision

What evidence would connect a proposed feature to a consumer benefit? Which cost, access or adoption constraint could change the decision?

Case study: Global results for Singapore: identifying the consumer

Global results for Singapore: identifying the consumer

FactSet data passes through global-results processing and a creator before reaching an investment team. The immediate customer and end decision-maker are different roles.

Decision: Define the interface and promises at the creator's own boundary, while obtaining enough upstream and downstream context to validate the end decision.

Reasoning: One downstream consumer may justify a tailored interface; many consumers require a stable standard so accommodating one team does not silently acquire every downstream commitment. Available access and time limit how far the creator can inspect the chain.

Lesson: Keep the end decision visible, record upstream dependencies and ask the accountable owner to settle responsibility boundaries. Local product ownership can help; it does not remove the need for explicit promises.

Case study: ESG risk results: connecting value to prioritisation

ESG risk results: connecting value to prioritisation

Third-party data flags issuers with material ESG risks for analysts and investors, while the creator may sit several steps away from their decision.

Decision: Interview an end consumer and describe the benefit over the product lifetime before claiming a prioritisation advantage.

Reasoning: The value materialises downstream; a local pipeline improvement is a means to that value and may not establish the business case on its own.

Lesson: Record both the causal explanation and any available measurement. An unquantified benefit is an assumption to investigate, not a fabricated return.

Case study: Morningstar exposures: counting recurring cost

Morningstar exposures: counting recurring cost

An exposure-ingestion product serves analysts, product managers and portfolio managers within CSG; other teams also want its output.

Decision: Declare the consumer population and interface, then include maintenance, troubleshooting and communication in the lifetime cost.

Reasoning: An illustrative estimate of four development sprints and about 20 support hours per week makes recurring effort visible. These are case assumptions, not forecasts for CG product-information.

Lesson: Support is part of viability. Transaction costs between teams remain real inside a large organisation and belong beside compute and procurement costs.

Case study: Investment research: designing for an AI consumer

Investment research: designing for an AI consumer

Research published in a portal is requested by an AI-agent workflow. A schema designed for a web application may not be the most useful representation for that consumer.

Decision: Compare authorised plain-text or Markdown extracts with other supported representations on a defined retrieval and interpretation task.

Reasoning: Text removes many presentation details, but tables, layout and richer media can still carry meaning. Compare outputs rather than assuming one format always preserves everything.

Lesson: Treat the agent as a consumer with explicit access, meaning and validation requirements. Structure and representation need evidence; retain provenance and review the result. Additional logging or observability is a later design consideration, not production work assigned here.

Case study: External counsel invoices: transparency and sensitivity

External counsel invoices: transparency and sensitivity

Invoice detail supports budgeting, authorisation and optimisation of external-counsel spend, but source matter details and named parties may be sensitive.

Decision: Design a permitted output that exposes the cost information needed for the decision and limits inappropriate disclosure.

Reasoning: Useful transparency does not require every source detail to be available to every consumer.

Lesson: Record classification, access and any disclosure or evidence-preservation questions for the authorised owner to resolve before committing to delivery.

Case study: Client engagement analytics: explaining the benefit

Client engagement analytics: explaining the benefit

Marketing, communications and related teams use engagement information to choose content and assess interest.

Decision: State how better behaviour information changes a content decision, then identify a way to evaluate that change.

Reasoning: “Improves marketing” is too broad to test without a decision, comparison and expected outcome; early discovery may not yet establish numerical benefit.

Lesson: Keep uncertain lifetime value visible and owned. The canvas should reveal the next evidence to collect.

Case study: Retirement client data: procurement and user experience

Retirement client data: procurement and user experience

External recordkeepers charge for large feeds and API calls. Investors need a usable portfolio experience, and an existing product must be replaced or migrated.

Decision: Assess procurement and recurring call costs alongside delivery needs, and plan how to compare the new experience with the old.

Reasoning: Cost and user experience have different measures. Retention, task completion or interaction patterns may provide evidence, but a monetary value is not automatic.

Lesson: Separate known costs from benefit assumptions and migration risk. When competitors offer similar core services, usability may still matter; validate its effect rather than attributing every improvement to the data product.

> Case study: Fund data: serving several consumer classes
>
> **Fund data: serving several consumer classes**
> A fund-data product may serve internal applications and external recipients, including a vendor, with different delivery expectations.
> Decision: Specify the consumer classes and supported formats, such as a file, API or approved database interface.
> Reasoning: One output shape may not fit every consumer, and each additional interface creates access, support and change obligations.
> Lesson: Delivery and adoption are linked. Offer a supported boundary with accountable promises rather than promising unlimited formats.

Data mesh and product ownership

> Knowledge check: Ownership and shared platforms
>
> Which responsibilities should stay close to the domain, and which standards or services can be shared?

Data mesh, briefly

A working definition (the term was coined around 2019 by Zhamak Dehghani). Data mesh is decentralised or localised data-product development, where "local" means *in the team most relevant to creating the data product*, as opposed to a centralised IT or engineering department building everything. There are two broad organisational models: one central team does everything, or teams self-serve. Data mesh is really about self-service with shared standards. You have access to Databricks, to the platform tools, to Catalyst, and Catalyst can support that pattern. Give everyone a tool that can create data products, and let them create their own.

The team closest to the data knows the data best, knows the SLA promises required, knows the customers. So local knowledge can reduce communication barriers, but ownership and downstream promises still need explicit agreements.

This isn't wholly new. The 90s had data marts and marketplaces with limited self-service, but mesh is more radical, because you do the *development* yourself. Local development can coexist with shared storage and platform services. A folder split alone does not establish domain ownership or product accountability.

The deeper point is that the hard part of a large business is making communication work across large numbers of people. As organisations grow, explicit ownership and communication become necessary. Data mesh is, in a sense, a self-organising management system.

For the relationship between domain ownership, shared storage and platform services, see Wednesday's data-mesh architecture explanation. Confirm current Capital Group responsibilities with the accountable platform or product owner.

Optional group challenge: discover your use cases

This optional activity broadens the examples available for later design work. Use it after completing your interview handoff and canvas; it is not a prerequisite for Tuesday or the practical route.

Knowledge check: A use case with a consumer

Which decision would a combined dataset support, and who would use it? What permission or grain mismatch could make the proposed combination unsuitable?

Optional use-case discovery

DURATION 15 minutes within canvas/application time if the core brief is complete

OUTPUTS A list of use cases, row grains and named consumers

In new mixed teams, introduce each other:

- Your team and what you work on; your data sets, pipelines and processes
- What SLAs (if any) are relevant to you, your team, your data
- What data pipelines, processes and source-to-target mappings are relevant
- Explore: if you combined data across the team's sources, what insights might that reveal? (There may be no useful or permitted combination; record that conclusion.)

Review two more questions: What is a *row* in one of your data sets, and who are your consumers?

Case study: Discovery across product boundaries

Cases: grains, joins and consumers

Advisor meetings, fund reference and equity-trading records can support an investment lifecycle view. Meetings may be weekly, fund reference monthly and trading daily. A trade authorisation, broker order and settlement are distinct row grains; an advisor/buying-unit meeting record and a fund record are different again. Decision: trace portfolio construction through authorisation, execution and settlement before joining it to advisor notes. Reasoning: the same proposed purchase may be constrained by a position limit or compliance rule. Lesson: a useful combined decision needs explicit keys, timing and ownership across the lifecycle.

An expense pipeline ingests bank spend and derives employee-expense metrics. A client view may combine CDM client data, TACO territory, transactions/redemptions, product reference and digital interactions. Decision: identify whether the consumer wants expense analysis, client/AUM coverage or fund-performance comparisons before combining them. Reasoning: a broad "single truth" label does not identify the decision or join grain. Lesson: comparisons of funds, managers, benchmarks or compensation need agreed definitions and authorised use.

An advisor-education platform produces progress records; an investment-data product transforms an external source into dashboards with a working delivery time and backup generations. Decision: distinguish producer responsibilities from downstream analytical promises, then map how advisor activity and investment information might support a business outcome. Reasoning: data can cross several teams and systems before a consumer sees it. Lesson: record lineage and source dependencies alongside proposed insight.

Advisor profiles, territory, fund transactions and web engagement can already form an advisor view. A profile row, a transaction and a page-visit event are different grains. Decision: aggregate and join deliberately for an analyst's opportunity assessment or a marketing journey. Reasoning: engagement does not itself establish intent or authorise use of every profile detail. Lesson: explain who acts on the result, through what channel, and under which access and purpose constraints.

Advisor–client interaction records, portfolio market values and end-of-day asset prices from vendors or the investment-operations ecosystem could support an analysis of interaction outcomes. Decision: define the outcome, valuation date and permitted links before claiming a return on an interaction. Reasoning: a possible join does not establish causation or consistent observation periods. Lesson: treat the combined insight as a hypothesis until consumer, source and measurement checks support it.

Quality and the data contract

> **Knowledge check: Quality as a promise**
>
> How would you test that the output is complete, fresh and accurate for its decision? Which is a schema check and which checks data values?

Quality dimensions

So what can you actually promise about quality? Most data-quality commitments fall under six dimensions:

Dimension	*What it measures*	*Fund-performance example*
Completeness	All expected records and required values are present.	Every active fund has the expected custodian rows for the business day.
Freshness	Data is current enough for its consumer.	Prior-close returns are available by 08:00 the next business day.
Accuracy	Values agree with an accepted source or calculation.	NAV matches the custodian statement within one basis point.
Validity	Values obey the declared type, range and format.	Exposure proportions remain between 0 and 1 and dates use the agreed format.
Uniqueness	The declared grain has no unintended duplicates.	One row exists per fund, valuation date and custodian.
Consistency	Equivalent values agree across sources or products.	The performance mart and executive summary report total return within two basis points.

On this course, every contract makes freshness, completeness and accuracy commitments. Add validity, uniqueness and consistency rules when the data and the consumer need them. Completeness itself splits three ways. Row completeness means all expected records arrive, column completeness means required fields have no unexpected nulls, and referential completeness means every foreign key points at a valid reference.

Pair each technical commitment with a business measure. A fund product can hit every freshness and accuracy target while someone still spends 20 hours a week on manual reconciliation, or consumers carry on using their spreadsheet. A business measure, such as reconciliation hours or the number of people still on the spreadsheet, will show you that.

The data contract: where it all lands

In the supplied course template, the design decisions are recorded in a human-readable data contract with eight sections. This template does not establish the current enterprise-wide CG standard; confirm the applicable version and approvals for a live product:

1. Identity — of the product, stakeholders, etc.
2. Schema — one row per column: type, whether PII is involved, etc. It can also hold the glossary, meaning what the columns mean.
3. Service Level Agreements
4. Data quality
5. Access (entitlements)
6. Change (change management) — a removed column breaks consumers that select it; narrowing `DECIMAL(10,6)` to `DECIMAL(10,2)` breaks precision; changing a return from TWR to MWR breaks meaning even though CI sees the same name and type. Semantic versioning gives breaking changes a major bump, additive changes a minor bump, and description or corrective changes a patch.

7. Team involvement / consumer registry — record one row for each consumer and its use case. Use a durable contact role such as `Consumer Representative — Portfolio Management`, because an individual's name may go stale. Derive the row's access level from the decision already recorded in §5.
8. Approvals

Together these sections are everything you can say about the product at the design stage. Dependencies (e.g. "we expect a certain amount of data coming in to be able to meet our SLA") ground the SLA section.

You'll find a worked §6 change table, versioning policy and §7 registry in Appendix B.

The phrase "data contract" unhelpfully refers to two different things:

1. A human-readable Markdown document with the eight sections above. It's the contract between you and whoever uses the product. It is really a *data product design document* (a service definition, if you prefer); calling it "the data contract" is the supplied course's terminology. Confirm the current internal vocabulary and authoritative format for the assigned product.
2. Automatic code enforcement of schemas, quality rules, and so on, in actual code. Wednesday's dbt tests are this sense: schema and data-content tests together check that the schema is correct and the values obey the rules. In other words, they're part of the data contract, enforced.

> Schema and data checks are different
>
> For a prepared or curated model, `contract: {enforced: true}` in its YAML makes dbt compare the declared columns and types with the model's `SELECT`; an added, removed, renamed or retyped column then fails the build before the table is written. Tests check the data values, and the enforced model contract checks the schema. A `contract/` directory with no dbt block can stay the same while the SQL drifts, so update both column lists in the same pull request.

So in conversation, someone designing a data product who says "we need to produce a data contract" almost certainly means the document. An engineer who says "I've just written a data contract" almost certainly means code.

SLAs, SLIs, SLOs and RAID

> Knowledge check: Service levels and failure
>
> What would you measure to know whether a promised working deadline was met? What upstream dependency could cause it to fail?

SLAs, SLIs and SLOs

- **SLI — service-level indicator:** the measured service behaviour, such as the proportion of deliveries received by a deadline.
- **SLO — service-level objective:** the target or range for that indicator over a stated window.
- **SLA — service-level agreement:** the promise to the consumer, including the response or consequence when a target is missed.

For example, measure the share of business-day deliveries available by 07:00. An SLO might require at least 95% over a quarter. The SLA adds who is notified after a miss, when the next update is due and any agreed remedy. A target need not mean perfect service. Google's SRE explanation distinguishes these terms.

The nines: a rule of thumb

90% is “one nine”, 99% two nines, 99.9% three nines, 99.99% four nines, and there is a five-nines. Applied to a year, roughly how much downtime does each allow?

Availability	*Downtime in a 365-day year*
90% (one nine)	36.50 days
99% (two nines)	3.65 days
99.9% (three nines)	8.76 hours
99.99% (four nines)	52.56 minutes
99.999% (five nines)	5.26 minutes

Calculate permitted downtime as `(1 - availability) × measurement period`. For four nines over a 365-day year, `(1 - 0.9999) × 365 × 24 = 0.876 hours`. A real agreement also states the measurement window, exclusions and response to a breach.

Four components and an error budget

Check an SLA for four components: an Indicator we can measure, an Objective with a threshold and measurement window, a Budget for the failures the objective permits, and a Consequence that says what happens when the budget or target is breached. If any of the four is missing, go back to the consumer and agree it before you call the statement an SLA.

Work out the error budget like this.

> Error budget = total measured events × (1 − reliability target), rounded down

Take a 95% freshness objective across 63 business days. The calculation gives `63 × (1 - 0.95) = 3.15`, rounded down to 3 missed days. Rounding up would allow a fourth miss that the objective did not permit. Keep the direction of the comparison straight as well. 28 successful refreshes out of 30 is about 93%, which misses a 95% objective.

The count says how much budget has gone; the burn rate says how quickly it is going. A three-miss budget spread over a 13-week quarter expects roughly 0.23 misses per week, so an early cluster of misses needs action before the budget is gone.

Observed position	*Reading*	*SRE response*
No misses in weeks 1–6	At or below the planned rate	Continue and keep measuring.
One miss in week 1	About four times the planned rate	Investigate the cause and watch the next run closely.
Two misses in week 1	About nine times the planned rate	Notify the DPM and pause risky changes while the cause is addressed.

The breach response also needs a short severity ladder:

Severity	*Typical trigger*	*Response*
Critical	Completeness or uniqueness failure blocks delivery	On-call response, named engineering escalation and a short resolution window.
High	Repeated freshness misses or an accuracy SLA breach	Engineering investigation, consumer notification and incident review.
Medium / Low	One tolerated miss, a quarantined batch, or a documentation defect	Steward-led follow-up on a business-day timescale.

> Error budgets at CG
>
> Error budgets and burn rates are site reliability engineering (SRE) practice. In SRE, a spent budget usually means holding back risky changes while the team works on reliability. For a CG product, agree the consequence with the DPM and the consumers and write it into the contract.

What do we make agreements about? Timeliness (daily, weekly, 8 AM, monthly, or batch completion, such as "the batch must complete before 10 AM every day"), schema / data contract (fields, values, typing, metrics on the data), quality, uptime, recovery time (if data is deleted or a system goes down, how many hours until it is available again), support. And importantly, not everything in an SLA is neatly measurable. There are qualitative agreements about what happens *when a promise is broken*, such as a duty to notify (messaging, emailing, updating people), a responsible person acting, a duty to explain (if uptime is missed, you at least get an email explaining why) and a duty to provide a root-cause analysis. (Duty to *fix* may belong to a broader business agreement rather than the data product's SLA.)

The duty to explain, with an example: during the pandemic, oil futures went negative, which had never happened in recorded history. An oil futures contract obliges you to take delivery; if you can't, you sell it on; if nobody is buying, you *pay* someone to take delivery, otherwise you bear the enormous expense yourself. A trader looking at a report with oil futures in the portfolio wants to know *why is this number negative?* It could equally be a data-quality issue, or something seriously broken. It's Tuesday's distinction between a data error and a real tail event, turning up in production. Part of support can be that a person exists whose duty is to explain such patterns, without waiting for an incident.

RAID

- Risks — possible future consequences.
- Assumptions — bets you're making; things you hope will be true.
- Issues — current and past risks (already materialised).
- Dependencies — upstream and downstream reliance.

Applied to a data product plan, assumptions exist at every layer. There are assumptions that enable the product to exist, assumptions behind the mapping and behind the glossary, and above all the assumptions that *ground the SLAs* (we only promise to meet our SLAs if upstream providers meet theirs). Dependencies follow directly, because upstream systems have their own SLAs we depend on. Issues are present-day problems with the things we're proposing. What problems exist now, and how are we looking to solve them? This profiling belongs in the proposal.

To score a risk, multiply probability (1–5) by impact (1–5), taking the higher of the delivery-delay and consumer-harm impact scores. A score of 16 or above needs immediate mitigation, and 4 or below can be monitored. The full scale is in Appendix C.

From contract package to backlog

> Workflow
>
> A complete contract package contains the data product contract, SLA definitions, non-functional requirements, a prioritised backlog and a feasibility review.
> The builder rates feasibility Green, Amber or Red across source availability, schema implementability, quality, SLAs and operational sustainability. Every Amber names the required modification, and every Red names the blocker.
> Derive the backlog from the contract sections: schema creates build and constraint work, quality rules create tests, SLAs create monitoring, access creates permissions, and consumers create onboarding. Then apply MoSCoW with Must work at no more than 60% of estimated effort, leaving room for delivery variation and Should work. The full worked package is in Appendix B.

Prioritising promises: RAID, RICE and MoSCoW

> Sometimes you can't keep every promise a product makes. This afternoon you rank them, then work out who has to back the product before anyone will use it.
>
> **THIS SESSION ANSWERS**
>
> - How do you decide which promise to break when you can't keep them all?
> - Why do good data products go unused?
> - Who needs to hear about your product, and how often?
> - What goes into a data contract?

> Knowledge check: Ranking commitments
>
> If two promises cannot both be delivered this iteration, what evidence would let you rank them? Which uncertainty belongs in RAID?

The promises you rank here are the service levels from the previous session.

RAID

Risks, Assumptions, Issues, Dependencies — assessing the degree to which you can keep your promises. Applied to a data product plan:

- Risks — of violating the promises.
- Assumptions exist at every layer. There are assumptions that enable the product to exist, assumptions behind the mapping, behind the glossary, and above all the assumptions that *ground the SLAs* (we only promise to meet our SLAs if upstream providers meet theirs).
- Issues are present-day problems with the things we're proposing. What problems exist now, and how are we looking to solve them? This profiling belongs in the proposal.
- Dependencies follow directly: upstream systems have their own SLAs we depend on, and note who depends on *us*.

In practice: assess RAID before starting a PI, reassess at leadership sync-ups as new risks occur, escalate blockers to leadership, and roll residual risks into the next PI at the end; other teams use it at quarterly planning.

MoSCoW

Must, Should, Could, Won't (Would) sorts things into what we must do, what we should do but don't have to, what we could do and what we'd like to do. Applied to SLIs, it asks which measures *must* hold, and what redress follows when a must fails. Generally speaking, SLAs live at the level of *should*,

which is why MoSCoW alone can't rank them. Most SLAs sit at the same MoSCoW level, so it gives no discrimination. You need an impact-based system like RICE. The frameworks can operate at different granularities: MoSCoW may support planning, while RAID is updated as risks and issues change.

RICE

When several SLAs are of similar importance and one has to give:

> (Reach × Impact × Confidence) ÷ Effort

- Reach — the blast radius of the breakage: five systems, five million pounds, five clients, five anything.
- Impact — what each affected thing is worth (e.g. revenue per system, dollars per client).
- Confidence — how likely that impact is to be realised across those things (e.g. 20%).
- Effort — divide by what it will cost you to honour or fix the obligation.

Worked example: three SLAs covering timing (deliver by 7:30), duty to explain, and quality. If breaking the timing SLA by ten minutes lets you deliver the explanation, RICE gives you a way to choose that you can justify afterwards ("here's the audit of my decision-making: reach × impact × confidence over effort said it was better to be ten minutes late with the explanation").

Practical guidance on scales and units:

- Design each factor to land on a 0–100 scale, then define a simple mapping from the real world onto the scale (e.g. effort: 10 = a day, 20 = a week; impact: each unit = $1M). A 0–100 result can be reported to different stakeholders as a percentage, a probability, a score.
- Alternatively, stay in raw units. Leave confidence between 0 and 1 and keep the rest in units, so the result reads as, say, "client-dollars per week". Either works. These formulas are *comparative*, and all that matters is that they rank the SLAs correctly.
- For reach, pick a granularity at which you can define impact. If you can't define impact per person, define it per team ("this team is delayed by this amount"); reach then becomes a number of teams. Ideally find common units across your SLAs so they're comparable. If you can't, that's the argument for normalising everything to 0–100.

Edge cases in promises

General promises run into edge cases:

- The two-phase refresh. A 7:30 AM delivery SLA covering three years of monthly plus one year of daily data was physically impossible to refresh in time. The fix was to ask the consumer what they care about most, which was yesterday's data. So refresh in two phases: yesterday's data (5–10 minutes of work) lands by 7:30, and the full history refreshes in the background during the day. It isn't the original promise, but it works. You've refined a general, universal SLA into a more specific one built around the customer's priorities.
- Force majeure. SLAs implicitly assume things like people being alive and available (deaths, holidays). Here you don't modify the SLA down. You accept that promises get broken when they can't be met; you just need a good reason.
- Breach handling. Agreements should specify what happens when a promise fails: breach codes, actions to take, notification.

What these acronym frameworks are actually for

Use a framework when the trade-off is difficult enough to need explicit reasoning. An obvious priority does not need a score, but the decision still needs a record of its impact and owner.

The real value is accountability and transparency. In a large organisation (especially in finance) you must be able to account to others for the decisions you made, with meeting minutes, audit trails, an email or PDF recording the impact assessment. Transparency builds trust, and "why did you decide that?" is much harder to answer well on the spot than the decision was to make. Do MoSCoW first; if everything is a must (it usually is), drop down to RICE to decide what breaks, and keep the notes.

Data product adoption

> Knowledge check: Adoption and responsibility
>
> A product meets its technical tests but consumers keep using a spreadsheet. What would you investigate before proposing more training?

We've covered designing data products (the scoping questions: should we create it, what would it be, cost and value). Now suppose we *have* a data product in mind. There are two situations:

1. First release, when we're concerned about achieving adoption.
2. Already released, when adoption is poor.

Barriers to adoption

- A rival or legacy alternative already in place (possibly the system being replaced)
- Discovery — people don't know it exists
- Effort to adopt
- Wrong design — doesn't meet business objectives; not properly consumer-aligned, or aligned to the wrong consumer
- Data issues — consistency, quality, freshness (or lack thereof)
- Competing initiatives — organisational competition for attention
- Timing — downstream isn't ready to consume (dependencies on other systems)
- Lack of training
- Documentation and trust — missing glossaries (what the columns and values mean), and trust deficits

Note that most of the ones that matter are *not* technical. And distinguish barriers to *adoption* from barriers to creation, which stop a product getting off the ground at all. These include sourcing (including the permitted-use and redistribution terms for each vendor product), ownership (who is the data owner?), compliance, gatekeeping, cost, operational support and resourcing. The technical issues (writing docs, odd values) have relatively obvious answers; the institutional ones don't.

Leadership, management and innovation

Distinguish three things:

- Management — given a policy, align and organise the business to it. Execution: "how do we carry this out?"
- Leadership — *set* the policy and the mission, and show people how to follow it. Leadership is a quality anyone can exercise (it isn't C-level; it can be within your team, often leadership by example).
- Innovation is different again. The innovator is out ahead of everyone: self-motivated, principled (sometimes wrongly, often stubbornly), holding themselves and the business to a high standard, inducting practices that drive the business forward. That's often because they have information others don't, or they're the first adopters of a system and know how good it is. (Catalyst, produced by the EDO, is an innovation in this sense.)

The catch with innovators is that they scare everyone else. They want change and they question the status quo. Their practices are unusual and so far ahead they're confusing. They act from information other people don't have, which creates friction ("isn't this obvious?" It's obvious only inside the ecosystem the innovator lives in). And they're often poor communicators. Change makes others feel unsafe.

The reason to draw these distinctions is that innovation and management don't solve adoption problems. "I'll lead by example" from an innovator just means demonstrating practices miles from where people are. That scares people more than it moves them. Management just tells people what to do. Leadership stays close to where people actually are, understands them, knows where things need to go, and actively, empathetically bridges the gap.

Often the real job of a leader is to pick an innovator they trust and say: the business's job is to follow that path. Then they bring people along it by selecting a credible direction and making the transition understandable.

Things a leader does for adoption

- Mandates and quotas (a management tool, but available)
- Champions, delegating to other leaders embedded in teams
- Selling the vision and getting buy-in, for alignment, via meetings and roadshows
- Pilot launches to demonstrate efficacy
- Training, run *with* your rollout. Publicly credit the early adopters and celebrate the people who do well in it, and the resistant start wanting the same credit. Recognition should reflect a real contribution and accompany useful support.

Using the paved-path tools during the course lets you assess their purpose and responsibilities through practice. Training, pilot evidence and a clear support route help a consumer adopt a product; they do not replace its quality or access commitments.

The product owner coordinates stakeholder communication and decisions. As creator, record who owns each unresolved requirement, who can approve it and who needs to review the result. For detailed roles, influence mapping, transition planning and cases, use Managing stakeholders, including why adoption needs explicit attention.

On screen: Data Marketplace

The Data Marketplace is where you look for an existing product before you build one. The overview tells you what the product is for and who owns it. The data details show where the tables live, the preview shows a few sample rows, and the schema and data dictionary give you the types and what each field means. Some products have current and historical tables, and those answer different questions.

> Knowledge check: Inspect a candidate product
>
> What do you need to know about an existing dataset before reusing it? Which questions can a preview answer, and which need a contract or owner?

Look at the data dictionary. It is marked "Not Validated", so those definitions need checking. For the CG design, compare grain, units, reporting dates, source provenance and ownership with the analyst's need. A preview cannot establish complete key coverage.

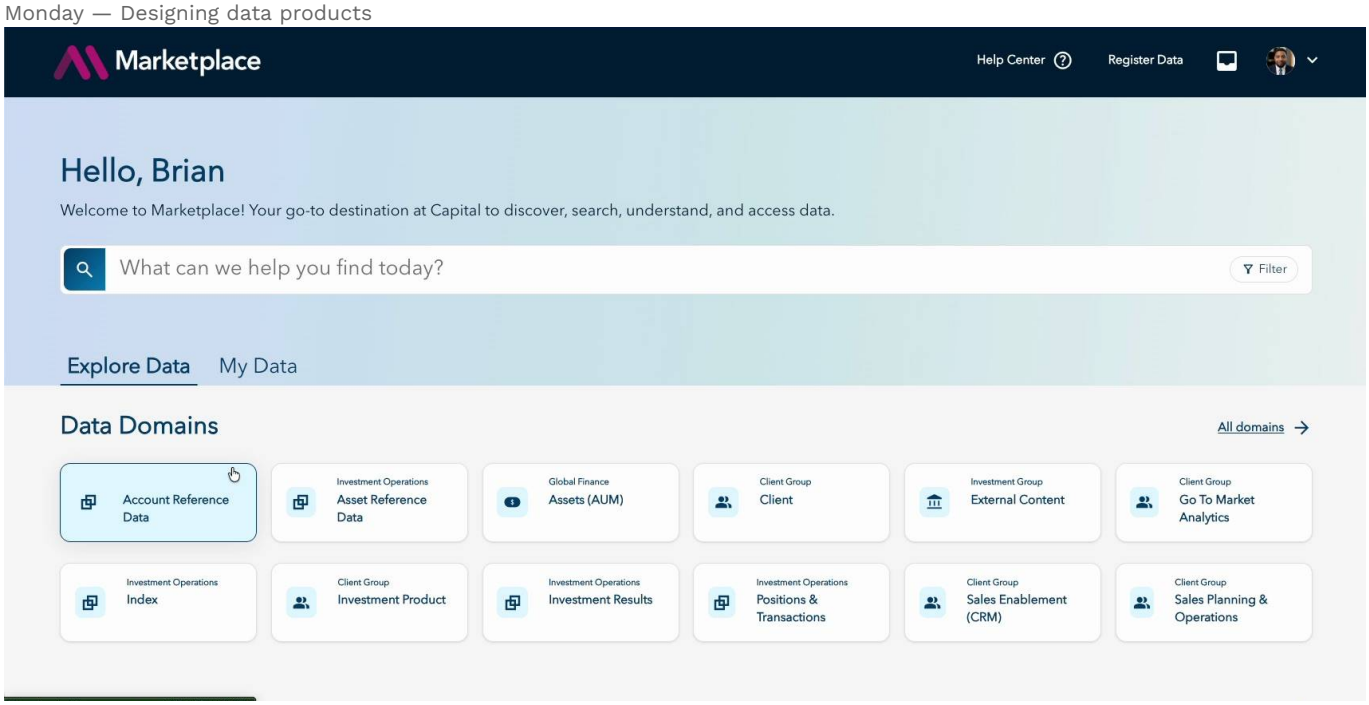


Figure 3: Marketplace home

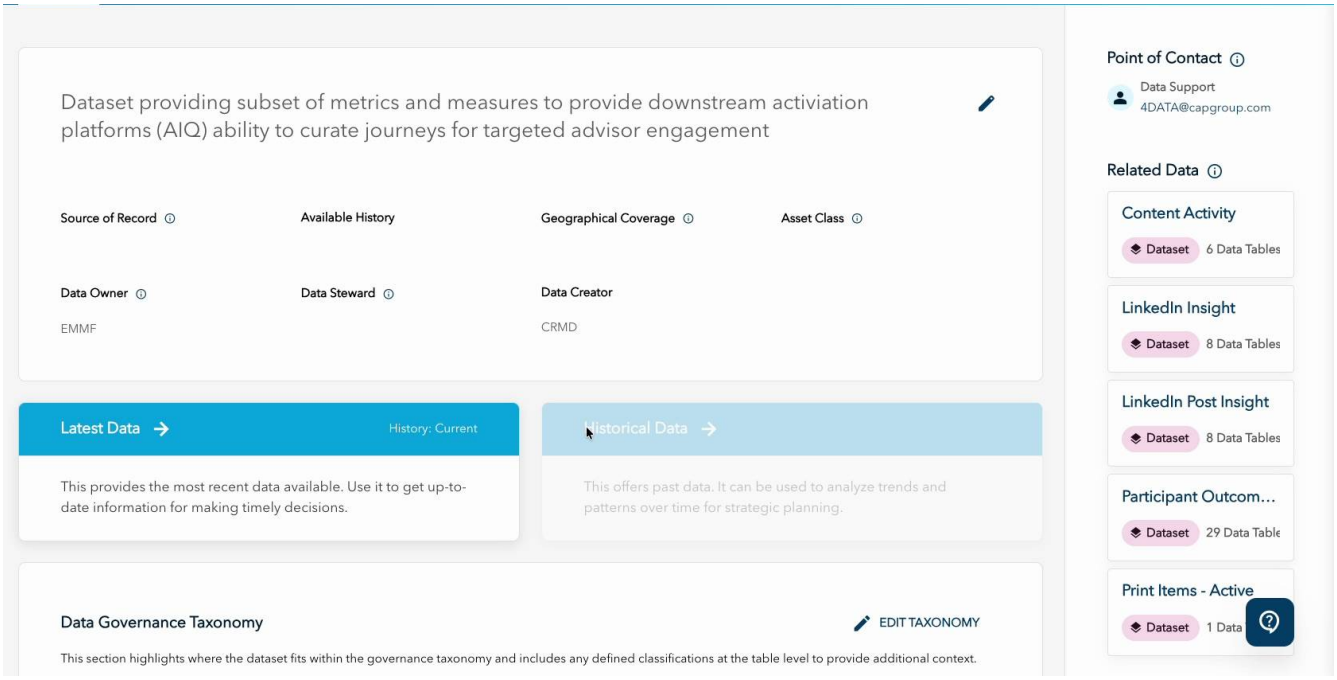
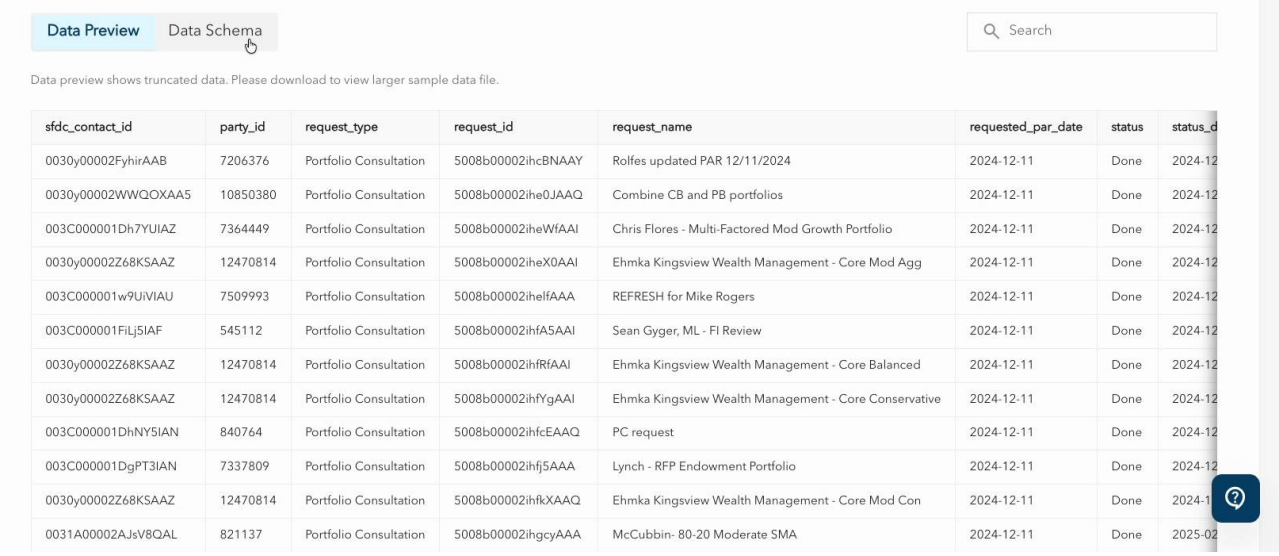
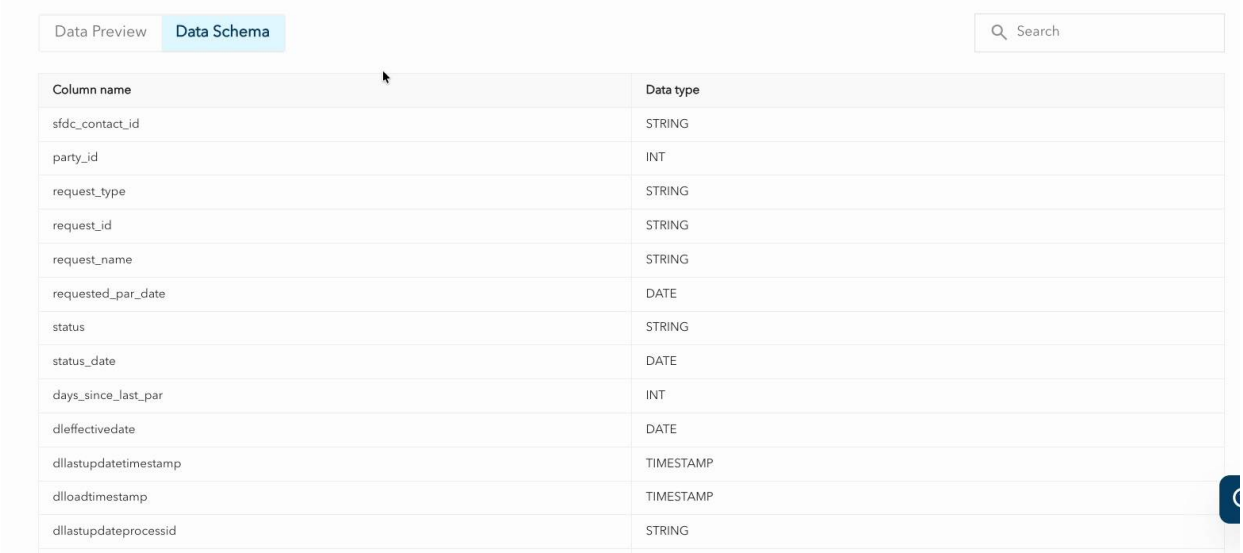


Figure 4: Dataset overview



sfdc_contact_id	party_id	request_type	request_id	request_name	requested_par_date	status	status_d
0030y00002FyhirAAB	7206376	Portfolio Consultation	5008b00002ihcBNAAY	Rolfes updated PAR 12/11/2024	2024-12-11	Done	2024-12
0030y00002WWQOXAA5	10850380	Portfolio Consultation	5008b00002ihe0JAAQ	Combine CB and PB portfolios	2024-12-11	Done	2024-12
003C000001Dh7YUIAZ	7364449	Portfolio Consultation	5008b00002iheWfAAI	Chris Flores - Multi-Factored Mod Growth Portfolio	2024-12-11	Done	2024-12
0030y00002Z68KSAAZ	12470814	Portfolio Consultation	5008b00002iheX0AAI	Ehmka Kingsview Wealth Management - Core Mod Agg	2024-12-11	Done	2024-12
003C000001w9UiVIAU	7509993	Portfolio Consultation	5008b00002ihelfAAA	REFRESH for Mike Rogers	2024-12-11	Done	2024-12
003C000001FiLj5IAF	545112	Portfolio Consultation	5008b00002ihfA5AAI	Sean Gyger, ML - FI Review	2024-12-11	Done	2024-12
0030y00002Z68KSAAZ	12470814	Portfolio Consultation	5008b00002ihfRfAAI	Ehmka Kingsview Wealth Management - Core Balanced	2024-12-11	Done	2024-12
0030y00002Z68KSAAZ	12470814	Portfolio Consultation	5008b00002ihfYgAAI	Ehmka Kingsview Wealth Management - Core Conservative	2024-12-11	Done	2024-12
003C000001DhNY5IAN	840764	Portfolio Consultation	5008b00002ihfcEAAQ	PC request	2024-12-11	Done	2024-12
003C000001DgPT3IAN	7337809	Portfolio Consultation	5008b00002ihfj5AAA	Lynch - RFP Endowment Portfolio	2024-12-11	Done	2024-12
0030y00002Z68KSAAZ	12470814	Portfolio Consultation	5008b00002ihfkXAAQ	Ehmka Kingsview Wealth Management - Core Mod Con	2024-12-11	Done	2024-1
0031A00002AJsV8QAL	821137	Portfolio Consultation	5008b00002ihgcyAAA	McCubbin- 80-20 Moderate SMA	2024-12-11	Done	2025-02

Figure 5: Data preview



Column name	Data type
sfdc_contact_id	STRING
party_id	INT
request_type	STRING
request_id	STRING
request_name	STRING
requested_par_date	DATE
status	STRING
status_date	DATE
days_since_last_par	INT
dleffectivedate	DATE
dllastupdatetimestamp	TIMESTAMP
dlloadtimestamp	TIMESTAMP
dllastupdateprocessid	STRING

Figure 6: Schema

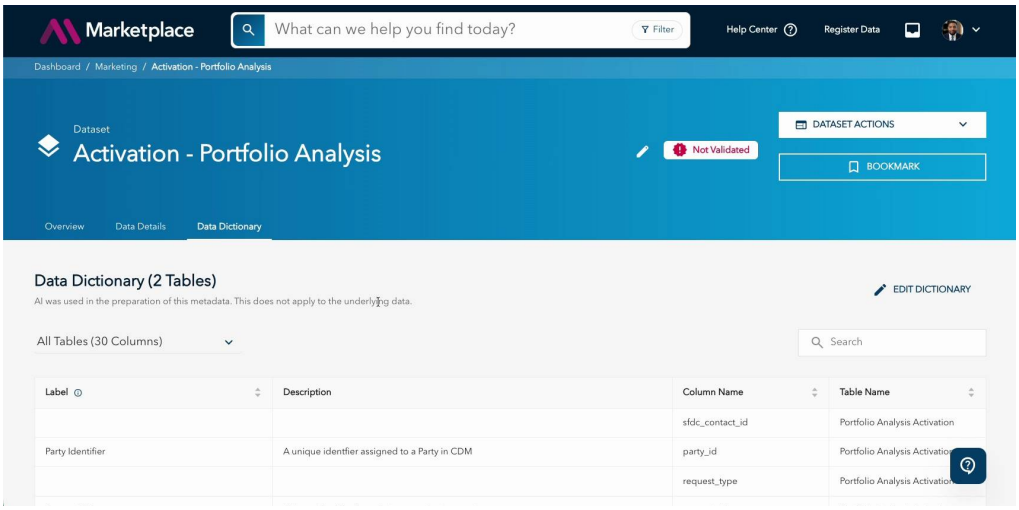


Figure 7: Data dictionary

Figure 8: About this data

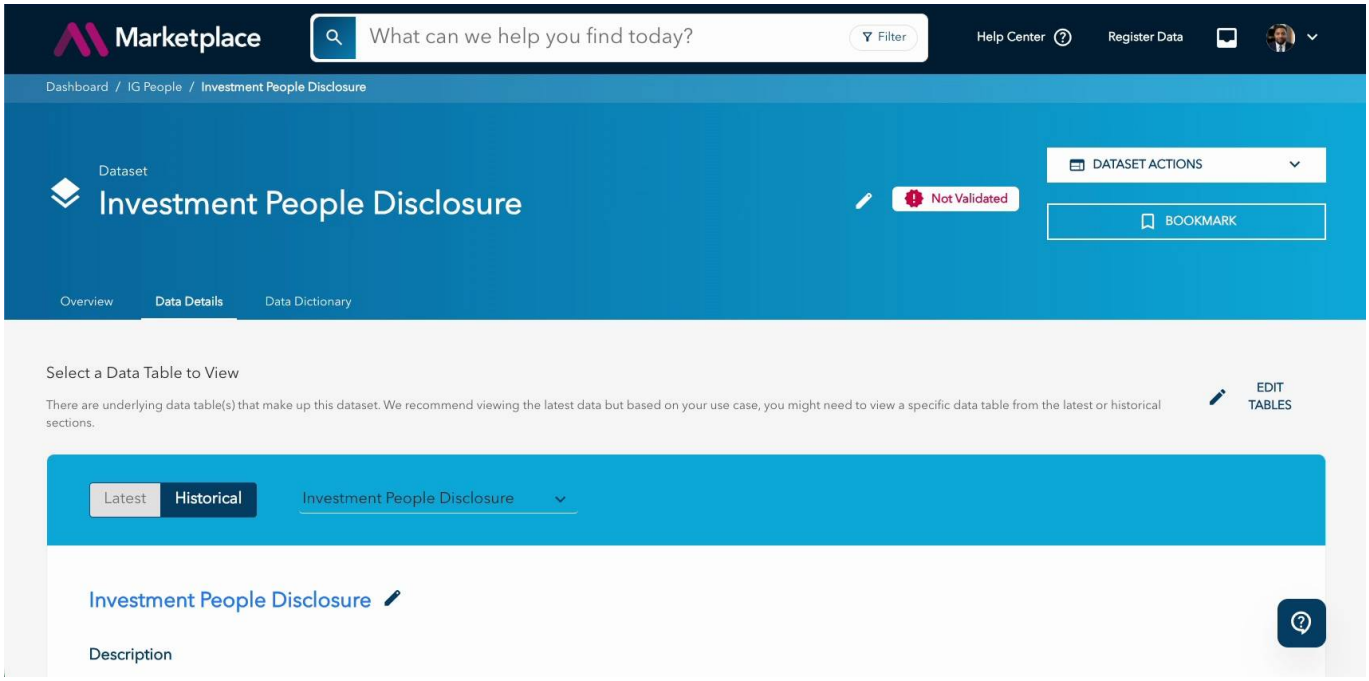


Figure 9: Historical tables

On screen: Enterprise architecture

The enterprise architecture tool links an application to the business capabilities it supports and to its data classification. That lets people outside engineering see who owns the application and what it's for. An application's classification can be broader than any one data output and the people allowed to use it, so you still need the product contract and a field-level sensitivity review.

Knowledge check: Business capability and classification

How does an application's business role differ from a field-level sensitivity decision? What evidence would you need before interpreting an architecture screenshot as an access rule?

For the CG product brief, ask which research capability the output supports and who decides its classification. A screenshot label is not policy.

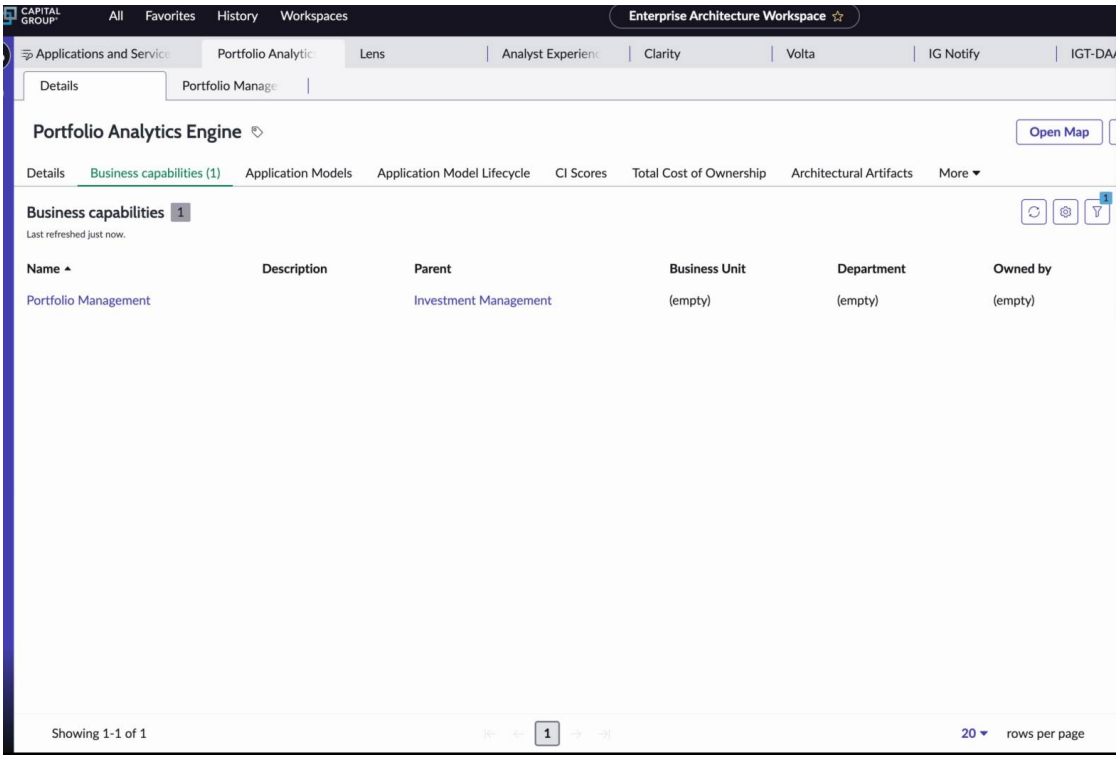


Figure 10: Business capabilities

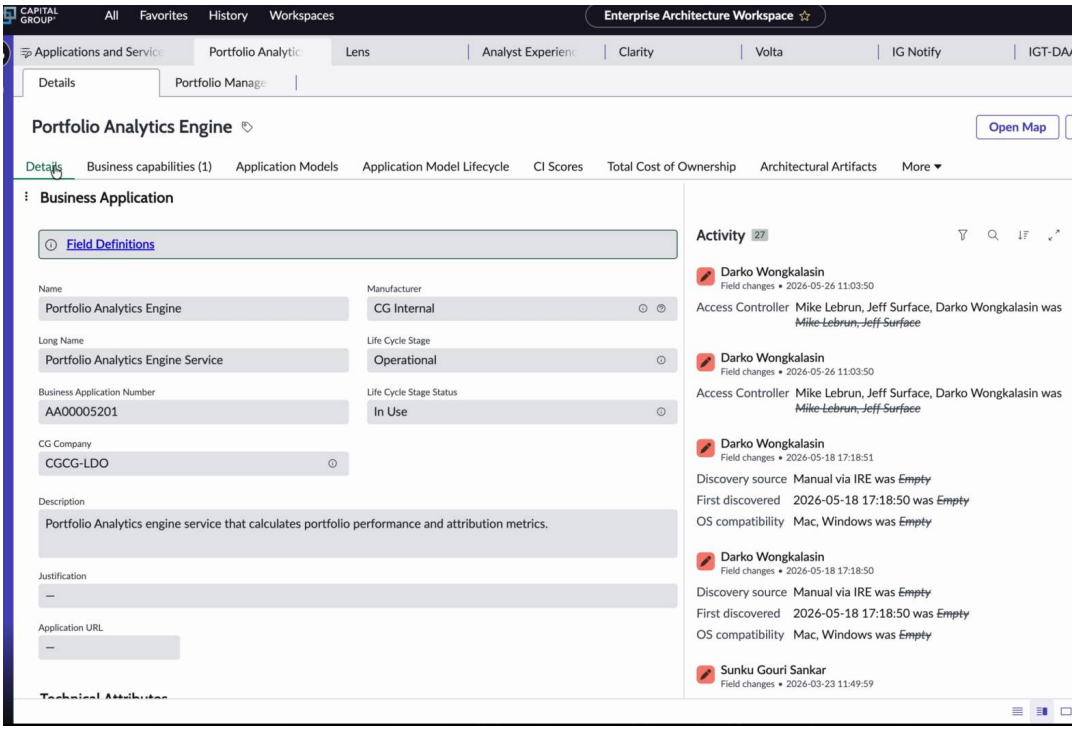


Figure 11: Application details

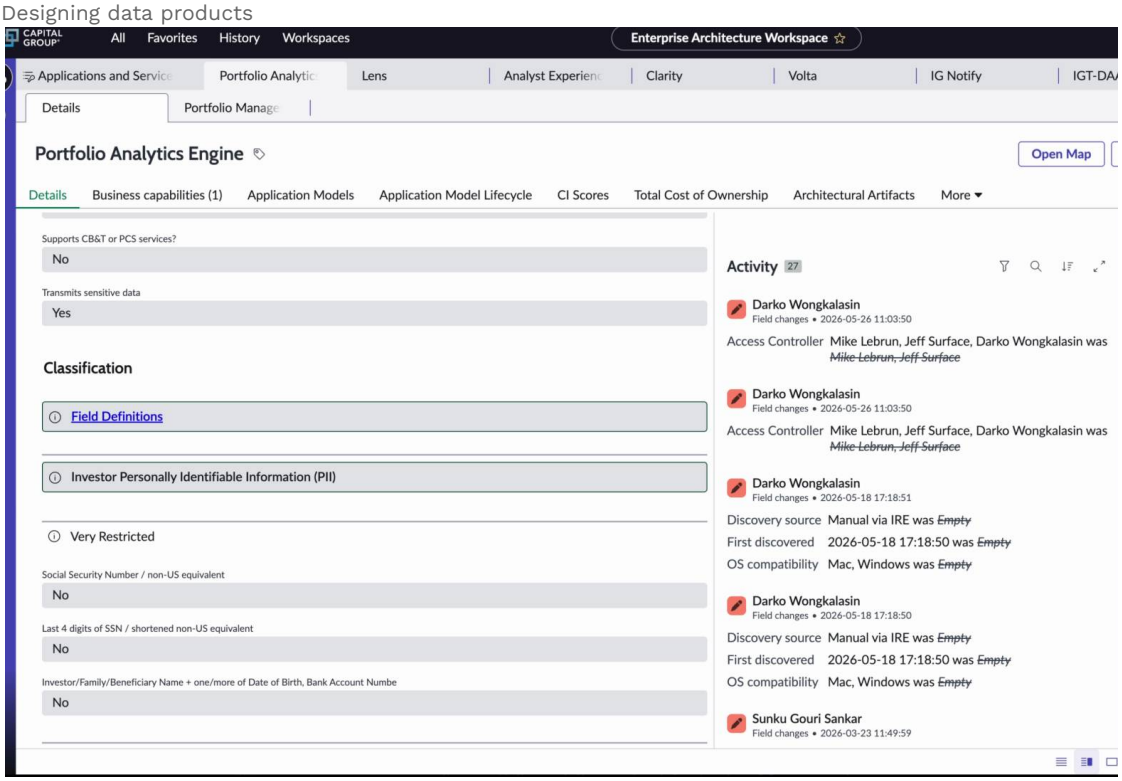


Figure 12: Data classification

> Learning Review
>
> Without looking back, explain the consumer decision, the source-to-consumer realignment and the promises in your product. Apply one quality dimension and one SLA measure to your own brief. Check that the requirement handoff, canvas and proposed contract agree; name an unresolved assumption and the role who can decide it.

Worked example: Morningstar exposures

Here's a short RFD for one choice from the week's Morningstar scenario.

RFD 0042: Publish fund exposures as a prepared star

Context. The product receives Morningstar exposures and fund reference data from separate sources. Consumers filter by fund, security classification and as-of date, and the contract grain is one row per fund, per security, per date. Client identifiers and other PII must not be copied into the exposure fact. The team already maintains dbt staging and integration models on the paved path.

Decision. We will publish `fact_fund_exposure` at the stated grain with foreign keys to `dim_fund`, `dim_security` and `dim_date`. Staging will retain each source's shape, integration will join and validate the sources, and the prepared model will expose the star. We will not publish the integration join as one wide consumer table.

Consequences. Consumer queries use stable dimensions and the same fund definition. The fact remains free of PII and can keep the reporting history required by its contract. The team must manage dimension keys and late-arriving security classifications; until a classification arrives, the contract's quarantine or agreed placeholder action applies. Adding a new dimension requires a contract and lineage review because existing consumers may depend on the present grain.

If a Consequences section lists only benefits, send it back. The author names what the team gives up, what may fail and which mitigation makes that cost acceptable. Without that, a later reader can't tell a weighed decision from a favourite option written up after the fact.

How can a creator participate earlier in design decisions?

Explain the decision you need to inform and the consequence of missing source or implementation knowledge. Ask which role represents that knowledge at the relevant meeting and when the creator should join. A concise question about a concrete dependency is easier to act on than a general request to attend every discussion.

If a request remains unresolved, follow the team's escalation route with the impact and earlier question visible. Match the technical detail to the decision: a budget discussion, consumer interview and implementation review need different levels of explanation. Build working relationships with product, business and engineering roles; ask for a delegate or documented decision when attendance is not useful or possible.

An approved AI assistant can help draft a clear memo, but the creator must check facts, recipients and wording. Discuss roles and decision rights directly rather than assuming exclusion is caused by personality or motives. The stakeholder map helps identify who can settle a stuck decision.

Annotated contract and stakeholder map

For the CG product brief, the internal analyst needs traceable offering information. Declare `product_id` as the offering key, the selected source representation, expense ratios in bps and the meaning of unknown values. Separate performance basis, period, return kind and reporting date. A contract must also state ownership, proposed freshness, allowed uses and exception handling.

Contract slice	*Annotation / decision*
Input: frozen products, fees, returns and source receipts	Catalogue/detail duplication and fee components require declared selection rules
Output: 25 ETFs in phase 1; 109 scoped offering IDs in phase 2	Preserve unknown costs and explain missing comparison evidence
Measure: net expense ratio in bps	Multiply percentage points by 100; gross and net expenses are alternatives
Performance: three-year annualized NAV return at September 30, 2026	Keep incompatible or absent figures visible with comparison reasons
Owner: classroom product creator; consumer: internal analyst	Unconfirmed access and service commitments remain proposed
Evidence: independent keys/values, failure/repair and reviewed handoff	Actual runtime receipts and consumer interpretation complement SQL tests

Map the internal analyst as consumer, sales/relationship colleagues as downstream users and the source collector as producer. Ask who follows up on missing extracted fields and what makes a source traceable. Optional communication planning is in Design workshops.

Tuesday — AI, Catalyst and the platform

The internal platform names, lifecycle phases and service arrangements in this chapter describe the supplied training version and illustrative cases. They are not a current enterprise policy register. Confirm current platform availability, ownership and mandatory controls with the product and platform owners; a screenshot or Catalyst-generated draft cannot establish approval.

Tuesday's plan

Today we take Monday's design into Catalyst and onto the platform. You'll use Catalyst to draft a contract, then check whether the draft matches what your consumer actually asked for. A tidy contract with a made-up schema or a made-up owner is worse than a rough one that lists what you still don't know. In the afternoon we go through the paved-path tools and get your dev container working. We close with a dedicated platform-decision session: why the paved path uses these tools, what your team owns, and how a dataset moves from its repository to checked, shared data.

The copy of Catalyst used for this course is version 1.13.0, with 106 skills. Your editor may have a newer release. The practical route uses the frozen v2 manifest and offered phases identified in the challenge reference.

Working time	*Focus*	*Outcome*
09:00–10:00	Models, tools, skills and reviewing AI output	A prompt you can check, and a list of what it may and may not do
10:00–10:15	Morning break	
10:15–11:15	Catalyst lifecycle and design documents	Corrections to the draft contract and mapping, each with its reason
11:15–12:00	DevHub, discovery and access	A reuse decision checked against what the platform shows
12:00–13:00	Lunch	
13:00–13:45	What each platform tool does, and the dev container	A map of where code, compute and data live
13:45–14:00	Afternoon break	
14:00–14:45	Readiness, OAuth and environment problems	A list of what works in your environment and what is still pending
14:45–15:00	Design and readiness review	Reviewed design artefacts and recorded setup results
15:00–15:30	Platform decisions: why the paved path works this way	An ownership map, a quality-gated publication sequence and questions for Wednesday

On screen today

System or sequence	*Why preview it*	*Teaching and screens*
Platform onboarding: DevHub, prerequisites and LaunchLake	Identify the service entry point, access prerequisites and provisioned scaffolding before mistaking repository creation for a validated product.	Readiness boundaries; onboarding sequence
Paved path: GitHub to Harness	Distinguish versioned code, release/promotion and scheduled data execution.	Platform roles; paved-path screen
Data contracts: contract folder and ODCS	Connect the human agreement with schema/quality definitions, migrations and the draft state shown on screen.	Contract meanings; contract sequence
Catalyst skills: repository, dependencies and NFR example	Inspect actual instructions and outputs rather than relying on a skill name or description.	Skill body and template; skills sequence
DPLC skills map: stages, phases and RFDs	Locate the next artefact and accountable role; use the published standard when a screenshot differs.	Lifecycle and roles; DPLC sequence

Today's sessions

- From a business need to a reviewed Catalyst draft
- Choosing the harness settings for the task
- Deliberate prompting and review
- Catalyst readiness check
- Worked Catalyst review: from interview notes to contract
- Group challenge: a PRD with Catalyst
- Exercise: generate a data contract with Catalyst
- Refining the data contract with Catalyst
- Exercise: interviewing Catalyst
- Catalyst skills, phases and roles
- From a reviewed specification to a generated model
- The paved path: Capital Group's platform
- Local ownership and the shared service
- Dev container setup
- Tuesday design and readiness review
- Platform decisions: why the paved path works this way
- Lifecycle, roles and warranty
- Context use, retrieval and durable decisions
- Why code gives useful but incomplete validation signals
- How can a consumer use AI without Catalyst?
- How should a team investigate an AI-assisted workflow?

From a business need to a reviewed Catalyst draft

Use Monday's CG product interview handoff to produce a contract draft. The business decision determines the required data; a Catalyst skill provides the document structure; you verify the result against consumer and producer evidence before handing it to the next creator.

THIS SESSION ANSWERS

- Which evidence does the skill need?
- What does the draft say that the brief does not support?
- Which checks establish correctness, and which decisions still require a person?

> Knowledge check: Before generating the contract
>
> Name the consumer decision, proposed output grain and one unresolved source constraint in your Monday brief. Which of those could an agent infer incorrectly? What would you inspect to test its interpretation?

Classifying and generating

A discriminative task assigns a label to existing content; a generative task produces new content. Catalyst's document skills use generation to turn source information into a draft. A well-formed table or fluent explanation is evidence that text was produced, while the facts and promises still need review.

Prompt, completion and model

A language model generates tokens from the context supplied to it. Tokens are pieces of text, rather than necessarily whole words. The prompt supplies instructions and evidence; the completion is generated output. Model weights encode patterns learned during training. A useful simplified description is `P(next token | supplied context; weights)`. This description helps explain why a plausible continuation can include an unsupported detail; it does not provide a test of correctness.[1]

Evidence and the context window

The context window limits what a model can use in a request. It may contain your conversation, skill instructions, retrieved excerpts and tool results, as well as generation capacity under the model's own limits. Keep the relevant consumer brief, producer schema, glossary, mapping and unresolved questions in durable files. A conversation summary can omit a condition, so ask the agent to reread the authoritative files when continuing work.

Context capacity and understanding

A larger window can hold more useful information and more irrelevant material. Give a focused evidence packet and ask a narrow review question. In research on particular long-context tasks, the location of relevant information affected performance; do not treat the window size as proof that every fact will be used.[2]

[1] Training and post-training approaches differ between models. The GPT-3 and InstructGPT research examples distinguish autoregressive generation from instruction and preference training; they do not establish the internal process of every model in your editor. See AI reference for the supplied explanation and research boundaries.

[2] Lost in the Middle evaluates sensitivity to the position of relevant information in the tested models and tasks. It supports checking context use empirically rather than assuming a universal position rule. Extended discussion is in Appendix G.

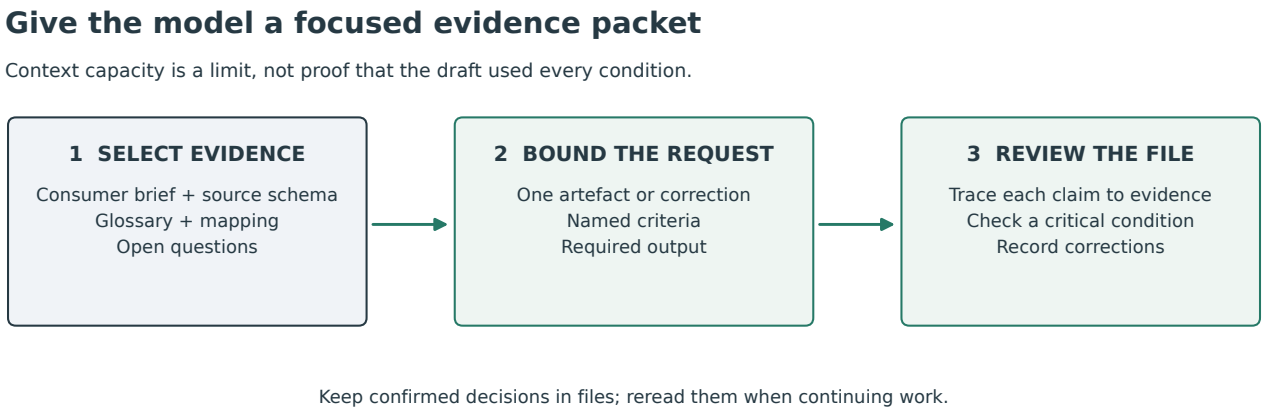


Figure 13: A focused brief, schema and mapping support a bounded request; review the generated file against that evidence.

What Catalyst provides

Catalyst 1.13.0 is a supplied collection of Capital Group skills, templates, standards and supporting client code. A skill's SKILL.md describes a task, its required context, instructions and output. Copilot is the editor/harness that makes model responses usable as file operations or tool calls. The model supplies generation. Keep the model, harness, skill and observed action distinct: asking for a document is different from reading the file that was written, and writing an approval table is different from obtaining an approval.

A document skill can provide a consistent structure and remind you of missing information. A phase orchestrator can call several other skills, including capabilities that contact services or write outside the current draft. Read the installed skill and its dependencies before invoking it. For the course, request a local draft and explicitly retain unresolved decisions; classroom text does not satisfy a live DPLC gate.

Data-contract instructions and required output

FROM CATALYST SKILLS

SOURCE `skills/data-contract/SKILL.md @ Catalyst 1.13.0`

The supplied source says:

```
## Instructions

1. Read the template at `templates/data-contract.md` to understand the
   exact document structure, sections, and field definitions.
2. Gather any required information from the user (skip what is already provided
   in the arguments or conversation context).
3. Produce a structured document that matches the template format exactly —
   same headings, same tables, same field names. Use `## ` (H2) headings
   with the exact text shown below — numbered prefixes are required.
   The output **must** contain these H2 sections in order:
   - `## 1. Identity` — Field/Value table with rows: Product Name, Domain
     Catalog, Owner, Steward, Version, Effective Date, Review Cadence,
     Classification
   - `## 2. Schema` — table with columns: Column, Type, Nullable,
     Description, PII, Sensitivity, Criticality, Example
   - `## 3. Service Level Agreements (SLAs)` — two sub-sections:
     - `### 3.1 SLA Targets` (columns: Metric, Target, Measurement Method,
       Measurement Window)
     - `### 3.2 Breach Response` (columns: Severity, Definition, Response
       Time, Resolution Time, Escalation Path)
   - `## 4. Data Quality Rules` — table with columns: Rule ID, Column(s),
     Rule Description, Severity, Action on Failure
   - `## 5. Access & Security` — table with columns: Field, Value
   - `## 6. Change Management` — three sub-sections:
     - `### 6.1 Breaking Changes` (columns: Change Type, Example, Minimum
```

```
      Notice Period, Migration Support)
    - `### 6.2 Non-Breaking Changes` (bullet list)
    - `### 6.3 Versioning Policy` (columns: Field, Value)
  - `## 7. Consumer Registry` — table with columns: Consumer, Team,
    Use Case, Access Level, Contact, Onboarded Date
  - `## 8. Approvals` — table with columns: Role, Name, Date, Status
  - `## Appendix: Revision History` — table with columns: Version, Date,
    Author, Summary of Changes (no numbering prefix on this heading)
4. Fill in every section with specific, concrete content based on what the user
   has provided. Do not leave placeholder text.
```

The body requires numbered sections, schema fields, SLA measurement and breach response, quality actions, change policy, registry and approval status. A missing source fact remains an owned decision in the training draft; it cannot be made concrete by inventing it. The draft is incomplete for live use until the required facts and approvals exist.

A model, a skill and an observed action

A model's assertion that a generated contract is correct is another generated statement. Open the file, compare the columns with the source, and calculate an expected example independently. For the CG product brief, independently select an ETF's net expense ratio and convert percentage points to bps before comparing it with the output. Consumer agreement requires an actual consumer decision, while a written approval row remains text until that decision exists.

The human role when working with AI agents

The creator remains responsible for the output, the evidence used to validate it and the decision to proceed. You can use Catalyst to write, critique and revise a document, while consumer agreement, source meaning and authorised access require evidence from the accountable roles. Open the actual output and trace its claims to that evidence.

Why this course exists

Understanding product design, dbt, orchestration and the platform enables you to review what an agent produces. You do not need to learn every implementation detail before drafting, but you need enough knowledge to identify a wrong grain, unsupported field, missing dependency or unapproved promise. The course builds that review capability through a connected product example.

Specialisation, purpose and domain evidence

You know why the product exists, how people use it, which terms have local meanings and who can answer an unresolved question. Put that knowledge into the evidence packet rather than expecting the model to possess it. Learning from use develops your ability to review the next output; a conversation does not by itself change the distributed Catalyst skill files. Improvements to those files require a maintained source change and review.

Judgment and taste in a product review

Judgment is an evaluative decision supported by reasons: remove an unsupported risk-score column, reject a currency conversion without an FX source, or leave a delivery target proposed until the producer confirms it. Record the evidence and consequence so another reviewer can assess the decision.

Taste concerns consistent choices where several acceptable forms exist: choose a clear vocabulary, a coherent document structure and the approved project conventions. Local experience helps you notice phrasing that a consumer may misunderstand. Apply that preference consistently while keeping it separate from a factual or contractual requirement. Both forms of review appear in the worked contract review; neither transfers accountability to the model.

Uneven capability and review evidence

> Expert
>
> Model performance can be uneven across superficially similar tasks. A correct SQL pattern or complete-looking contract does not establish correctness for a new source, grain or business rule. The model's account of its own work also needs checking: inspect the files, review the changed sections and compare results with independent expectations. Evaluate the actual task with representative cases.

Longer commentary on training analogies, specialist failure risks and the economics of AI is in AI reference.

Validate facts, meaning and actions

A useful draft can still be wrong in a consequential detail. Validate the source facts, the interpretation of the business requirement and the action the agent performed. An error in one category is not repaired by a successful check in another.

Unsupported generated claims

"Hallucination" commonly describes generated content that is unsupported or incorrect. Treat the practical problem directly: require a source for a column, value, owner, approval or result. If the source does not establish it, remove it or record an owned question. Plausibility and confidence are not substitutes for evidence.

Variation across runs

Outputs may vary with generation settings, model versions, context and tool results. Run-to-run variation means a previously good draft is not proof that the next one is correct. Keep a review procedure that survives different wording and organisation. Save the approved source artefacts and correction log so future work can use them.

When to add scrutiny

Add scrutiny when a long conversation may have lost context, a novel source or business rule is involved, or the agent reports a completed check without its result. Ask it to identify the evidence used, then read that evidence yourself. A second model or self-critique can surface gaps, but agreement can preserve a shared mistake. The independent CG product-information test rows and expected totals matter more than another assertion that the model is correct.

Review before continuing

Generated output often becomes input to later steps. Before a contract feeds a mapping or model, verify the facts that the next step will depend on. Choose deliberate review points: at minimum, check the specification before implementation, the model against independent expected rows, and the orchestration against the agreed dependencies. Preserve evidence for each checkpoint and correct the source document as well as downstream artefacts.

Before you rely on Catalyst

Ask what the output will be used for, who will own it and what evidence will establish that it is usable. Limit the request to the next reviewable artefact. If the draft is too broad to review, split it into meaningful sections rather than treating volume as progress. In this course the handoff remains a reviewed training draft; live approval, deployment and production responsibilities require their own authorised process.

Choosing the harness settings for the task

Read the installed tools' settings before starting an agent workflow. The names and available controls depend on the product and version. Evaluate the choices against the task you need to review rather than a vendor reputation.

> Knowledge check: Text, tool call or completed action?
>
> How would you tell whether the agent proposed a command, ran it, or checked its result? Which operation in a contract-generation task could change an external system?

The harness

A harness supplies context to a model, interprets structured tool requests, executes permitted operations and returns their results. Copilot provides this surrounding workflow in the course. A plan or proposal is a generated artefact; it is not evidence that a tool ran.

Review the available planning, execution and approval settings in your installed environment. Keep the course request bounded to local draft artefacts and use the organisation's supported settings. A stored project-facts file helps the agent find names and constraints but does not authorise an action. Do not include credentials in such a file. Treat unfamiliar retrieved content as evidence to inspect rather than new authority over the task.

Effort levels

An available reasoning-effort setting changes the model's allocation for a response. More effort is not a guarantee of a better result. Compare settings on a small representative task using the same evidence and review criteria, including correctness, completeness and the time needed to inspect the output. Record the setting used when comparing drafts.

Vendors and models

Use the models available and approved in your environment. Compare them empirically on your own contract or mapping task. A model may follow one template well and perform differently on an open-ended requirement or a source-specific rule. Model size, branding and apparent fluency do not establish suitability. Extended model-selection considerations are retained with their source boundaries in AI reference.

Deliberate prompting and review

Give Catalyst the business objective, authoritative evidence, required artefact and limits on the current action. Then plan, inspect and revise. A skill reduces the need to invent a structure; your task-specific context and review still determine whether the output is useful.

> Knowledge check: A prompt you can assess
>
> What would you add to "write a contract" so the draft stays faithful to the CG product-information brief? How would you distinguish missing evidence from a writing preference?

1. Use critique to find questions

Ask the model to critique a draft against named criteria, or compare it with another model's review. This can produce useful questions and omissions to inspect. It does not establish an independent proof: read the actual artefact, trace each proposed correction to evidence and keep tests that were not copied from the generated implementation.

2. Plan the next artefact

> Workflow
>
> 1. State the next output and how it will be used.
> 2. Ask for a plan based on the supplied brief, schema and unresolved questions.
> 3. Check that the plan respects the product grain, current scope and dependencies.
> 4. Generate or revise the local artefact, then open the file and inspect the diff.
> 5. Check the result against the acceptance criteria and record the remaining decisions.

Planning the plan can help when the task is poorly understood, but more planning layers are not automatically better. Use them to reveal uncertainty and choose a concrete next step.

3. Choose terms and evidence carefully

Define domain terms and distinguish requirements from examples. A word such as “equity” or “real time” can have several meanings; give the meaning from the product glossary and a concrete business event. Ask the agent to flag conflicting interpretations. Removing unrelated history can help more than adding another instruction to a confused conversation.

4. State the later use

Explain the next consumer of the artefact while keeping the current task bounded: “Draft the contract for later dbt model review; do not generate or deploy a model now.” That gives the model context for the necessary precision without authorising the next action.

5. Ask what may be missing

Ask for a review checklist or questions a specialist would investigate. Use the answer as a starting point: check it with source documentation and the relevant domain role. A question about keys, corrections, backfills or timing can reveal a missing condition even when the model cannot answer it.

6. Start fresh from durable evidence

If a conversation accumulates conflicting assumptions, save the confirmed decisions in the brief and start a focused request from those files. Keep a correction log. Do not depend on a summary to preserve every schema detail or unresolved question.

7. Capture a review aloud

A spoken or typed review can capture findings while you read the document. Record section, issue, supporting evidence, proposed change and confidence. Ask Catalyst to turn those notes into an action list, then check the list before revising the file. Use an approved transcription route for the data involved and reread the resulting transcript for errors. Voice notes can help capture a review; durable written decisions preserve what the next creator needs.

Catalyst readiness check

Check that the approved Catalyst plugin is available in your editor using the supplied onboarding instructions. Record the installed version and inspect the intended skill before selecting it.

> Knowledge check: Check the current installation
>
> Which plugin version is installed, and can you open a relevant SKILL.md and its template? Which operation would require access you have not yet established?

To test it, in VS Code, open the Copilot chat window and type:

```
run /data-contract
```

You should see text indicating a Catalyst *skill* called `data-contract` is running, and it should start asking you questions. If it does not start, record the editor, plugin version and error for the platform-support role.

A skill starting establishes availability, not correct output or remote access. Answer only the bounded training request; use the supplied evidence rather than improvising live product facts. A GitHub Enterprise sign-in failure needs its own support check even after an earlier successful session.

Worked Catalyst review: from interview notes to contract

> This is a made-up example with mistakes planted in it, so you can practise reviewing without a live catalogue, approval workflow or warehouse.
>
> **THIS SESSION ANSWERS**
>
> - What should you give Catalyst before it drafts a contract?
> - How do you find the mistakes in its draft?
> - What should the corrected draft say, and what should it leave open?

> Knowledge check: Trace the planted defects
>
> Which input would you inspect before accepting a risk-score column, currency conversion or approval?

If you have Catalyst running, compare your own draft against the same checks. Your wording will be different.

Prepare the inputs

Give the skill the smallest set of inputs that still explains the product: Monday's consumer brief, the producer list, the agreed glossary, the mini-STTM, the quality questions and a list of what you still don't know. Label each one. For example, the internal analyst needs a 50-bps shortlist, the source has multiple representations and expense components, and missing reporting dates must remain unknown.

Say what you want back and what Catalyst mustn't do. You want a draft file on your laptop. You don't want it approved, posted to Confluence or listed in the marketplace. Some skills call other skills that can do those things, so check what the skill depends on before you run it.

Here's an example prompt. Swap in your team's own filenames, and check which version of the skill you have first:

```
Use the supplied data-contract template to draft a CG product-information contract.
Read the consumer brief, glossary, mini-STTM and open-question register supplied
in this workspace. Produce a local Markdown draft only.

Keep observed facts, proposed commitments and unresolved decisions distinguishable.
Do not invent table columns, owner names, entitlements, approval dates or measured
quality results. If a mandatory field cannot be supported, describe the unresolved
decision and the responsible classroom role. Do not call external services or
publish an artifact. List the source evidence for every proposed quality rule.
```

The `data-contract` template has the eight sections you saw earlier, plus a revision history. The template asks for specific values in every field. If you don't have the value, the draft should say so and name who can supply it, instead of Catalyst making up a person, a threshold or an approval.

Review the first proposed draft

Suppose the draft adds `investor_eligibility_score`, says every missing expense is zero, promises "real-time updates", gives all staff access and marks the contract approved. Each of those sounds reasonable. Together they turn it into a different product.

Go through them one at a time and trace each back to your inputs.

Draft says	*Review finding*	*Correction*	*Why it matters*
Investor eligibility score included	Nothing in the mapping or sources provides it	Remove it, or raise it as a separate source requirement	The generated model will fail on a missing column or fill in a made-up default
Unknown expense ratios set to zero	Missing extracted costs do not mean no cost	Retain null and assign needs_review	Every SQL check still passes, and the consumer is misled
Real-time product information	Nobody agreed a refresh cadence	State the known reporting date and leave the delivery target as proposed	You end up with a schedule nobody can meet
All employees may read	Nothing in class tells you who is entitled in production	Note that access is for training only, and log the approval question	Access gets granted that nobody approved
Approved by owner	The owner hasn't approved anything	Keep it as a draft and record the approval as outstanding	An unfinished draft looks ready to release

Fix one kind of mistake at a time, then reread the file. If Catalyst says it removed a field, check the schema, the glossary, the acceptance criteria and any later model request, because agents often fix the one place you pointed at and leave the rest. Keep a short correction log: the source fact, the sections you changed and the question still open.

Then look hard at the SLA table. "Available by 07:30" needs a timezone, which days count, what "available" means, where you measure it and what happens on a breach. "Ninety-nine percent complete" needs a denominator. Ninety-nine percent of which offerings, from which registry? You can draft how each target will be measured today, and mark the targets themselves as proposed.

Separate the four artifact purposes

Catalyst produces four documents that overlap:

- The **PRD** explains the problem, the consumers, the requirements and how you'll know it worked. It starts in Discovery and gets updated in later phases.
- The **contract** is the agreement between producer and consumer, with its promises on quality, freshness and change.
- The **transformation specification** maps out the models and how you'll build and backfill them.
- The **quality-controls plan** says which controls you'll enforce and why, from the business requirements and the data profiling.

Because they overlap, a contradiction between two of them is easy to spot, and it usually means somebody hasn't made a decision yet.

The transformation-spec template puts the detailed logic in the dbt SQL and the quality rules in the contract. It has no separate section for test definitions. You still want the mini-STTM, because it explains the business rule the model implements and how you'll check it. When Catalyst generates a project, line the mapping up with where the current template puts things. Older templates had different headings.

The quality-controls template asks for scope, profiling results, proposed controls, business requirements, open questions, handover and approval. It treats the action (block, warn, quarantine) separately from the severity and wants a reason for each. A duplicate product key breaks the grain, so stopping publication is reasonable. A snapshot that's late but still usable can get a warning, but only if the contract says the consumer accepts that. "Warn, because stopping is a nuisance" is a bad reason.

The plan mentions Capital Group's internal rule patterns and quality system. You won't configure that platform in class. Use the business meaning to review each proposed rule and practise enforcing it in dbt.

Requirements that can be checked

FROM CATALYST SKILLS

SOURCE `skills/requirements-acceptance-criteria/SKILL.md @ Catalyst 1.13.0`

The supplied source says:

```
## Quality Checks

Before finalizing the document, verify:

- [ ] No placeholder text remains (no `[TBD]`, `[fill in]`, `[Name]` etc.)
- [ ] Each requirement has a unique ID and is categorized (functional / non-functional)
- [ ] Every requirement has at least one testable acceptance criterion
- [ ] Acceptance criteria use measurable language ("at least 99.5%") not vague ("high")
- [ ] Priority is assigned to each requirement (Must / Should / Could / Won't)
- [ ] Tables have data rows, not just headers
```

Turn the CG product-information decision into an identified requirement and measurable acceptance criterion before asking for code. A metric needs a denominator, source and business meaning; a template number is not an agreed target.

A corrected local handoff

The corrected draft now says what we know:

- The consumer is the internal Product Information Analyst.
- The catalogue grain is one row per scoped offering, keyed by `product_id`.
- Source representation and expense component selection are explicit.
- Duplicate product keys fail the quality check.
- Expense units, unknown reporting dates and return comparison dimensions are stated.
- Open service and access decisions are listed, each with someone to answer it.

It has no approval. An engineer can now tell which assumptions they can build on and which need a decision first.

Ask someone from another team to compare the handover with Monday's notes. They should find at least one constraint that survived generation, one decision that changed in review and one question still open. You're done when your team can explain every line of the contract and where it came from. If Catalyst wasn't available, do the same review by hand and label the output as hand-written.

Group challenge: a PRD with Catalyst

Use the product requirements document to capture the business problem, consumers, requirements and acceptance criteria before carrying them into the contract. Extend it with a sample mapping and glossary so the later review can trace the intended meaning.

Knowledge check: Requirements before implementation

What does the consumer need to decide, and what acceptance evidence would show that the output supports it?

Requirements, mapping and glossary

DURATION 25 minutes within design-document practice

OUTPUTS A grounded PRD, sample mapping, glossary and owned questions

Work in pairs using Monday's CG product-information brief or the business product already interviewed.

1. Use Catalyst's product requirements document skill to generate a PRD. Don't run it bare. Give it the brief, source schema and consumer evidence in the prompt. "Product requirements for the following. Here's a data product definition."
2. Once you have the PRD, iterate on it in the same conversation. Ask Catalyst to add a sample source-to-target map for your data, and a sample glossary defining all the key terms.

PRD instructions: preserve unknown decisions

FROM CATALYST SKILLS

SOURCE `skills/product-requirements-document/SKILL.md @ Catalyst 1.13.0`

The supplied source says:

```
## Instructions

1. Read the template below to understand the exact document structure, sections,
   and field definitions.
2. Gather any required information from the user (skip what is already provided
   in the arguments or conversation context).
3. Produce a structured document that matches the template format exactly —
   same headings, same tables, same field names.
4. Fill in every section with specific, concrete content based on what the user
   has provided. Do not leave bare placeholder text like "[TBD]" or "[fill in]".
   If a field is genuinely undecided in the source material, state what is
   unknown and who owns the decision — e.g., "Pending decision by [owner]
   (per [source])" rather than just "TBD".
```

The substantive instruction explicitly says that an undecided field must name the unknown and decision owner. Use the PRD to capture the problem, consumer and acceptance criteria; then carry supported requirements into the contract.

There are dedicated skills for some artefacts. Here the purpose is to practise a source-grounded iteration and verify that requirements, mapping and glossary agree. Keep unknown decisions explicit and confirm that no invented current-state facts or approvals have appeared.

Case study: A complete-looking PRD still needs source review

Cases: requirements and definitions

Mutual-fund monthly returns. A draft includes a summary, consumers, solution and data requirements, with a separate mapping. Decision: validate calculation meanings and source-to-target rules before accepting the mapping. Reasoning: an accurate schema does not establish the business rule or consumer goal. Lesson: cross-cutting rules such as refresh, referential integrity and allowed columns must be grounded, even when the document structure is complete.

Client analytics. A requirement mentions outliers without defining the comparison or calculation. Decision: ask the consumer which behaviour should be identified, then agree a model, baseline and threshold with evidence. Reasoning: "outlier" is relative to a method and population; a familiar numerical rule is not automatically appropriate. Lesson: a field name is not a requirement definition.

Internal monthly returns. A draft invents a current-state account instead of asking for missing facts. Decision: instruct the skill to name unsupported decisions and their owners, then verify the source used for each statement. Reasoning: a targeted template can reveal the missing section while the model still fills it incorrectly. Lesson: adapt the prompt and review method without treating a better-looking model response as proof.

Webinar attendance. An earlier contract feeds the PRD, which still needs review of each variable and rule, especially across several systems. Decision: trace the requirements through the source and use a written or spoken correction log section by section. Reasoning: fluent reuse of a previous document can preserve its errors. Lesson: inspect the diagrams, meanings and dependencies as well as the prose.

ESG data. A draft identifies sources, columns and proposed types. Decision: confirm those against source evidence and the desired consumer rule. Reasoning: a reviewer finding no obvious issue is bounded by the checks they performed. Lesson: useful drafting removes typing effort while the creator retains validation responsibility.

Case: listing reference migration

An existing listing-reference consumption moves from LASR to Caspian. Documentation provides column descriptions, a proposed mapping and possible risks, but the actual data has not been queried. Decision: treat document-derived risks as hypotheses, then validate the authorised source and affected consumer examples. Reasoning: a thorough documentation search can still miss value distributions, nulls, keys or correction behaviour. Lesson: record what was observed and what remains inferred before accepting the migration design.

Transfer checks

Ask which facts came from a source schema, which requirements came from a consumer, and which statements remain model proposals. Give the relevant authorised source pages or files for domain definitions. A targeted document skill and a phase orchestrator can differ in how they gather context; inspect the current body and dependencies instead of inferring behaviour from the skill's name.

Exercise: generate a data contract with Catalyst

Use Monday's brief and the reviewed requirements from today's worked example. The contract is a local training draft; it preserves unresolved facts and does not imply a live approval.

Draft the CG product-information producer–consumer contract

DURATION 15 minutes within design-document practice

OUTPUTS A local Markdown contract and source/uncertainty list

1. Inspect the installed data-contract skill, template and version. Give it the consumer decision, source schema, glossary, mapping and question register.
2. Request a local draft with the eight numbered sections. Ask it to distinguish confirmed source facts, proposed promises and unknown decisions with their responsible roles. Do not request service calls, publication or approvals.
3. Open the actual Markdown file and compare it with the template. Check identity, grain, columns, units, service measurements, quality actions, access, change policy and registry.
4. Trace one rule to the interview and source evidence, and identify one missing decision. Save the file and review notes for the next revision exercise.

Check: every schema field has a source; a promised target has evidence or is marked proposed; no generated approval is treated as real. A mandatory unanswered field makes the draft incomplete for live use. If Catalyst is unavailable, review the supplied planted-defect example and write the same correction log manually.

Refining the data contract with Catalyst

When the "should we, what should it be and how" questions are settled (or being settled), the design output at Capital Group is a **data contract**.

> Knowledge check: Review contract coverage
>
> Which promise belongs in the human agreement and how would code enforce it? Which required fact remains unanswered?

Two meanings of “data contract”

The document and the code that enforces it are both called the data contract. Monday’s data contract covers the two meanings.

The sections

The contract has eight sections, listed under Monday’s data contract. Use the section list to check coverage against the evidence rather than relying on memory of a previous draft.

Contract promise	*Example enforcement or review*
Schema	Check declared columns and types
Service levels and data quality	Measure freshness; test values, keys and relationships; apply the agreed failure action
Access and change	Inspect grants, release controls and consumer impact
Identity, consumers and approvals	Verify accountable roles, registered uses and actual decisions

Both halves state the same promises, so change them together.

A filled exemplar of all eight sections, including the Tier-1 and Tier-2 contrast, is in Appendix B. Response times and access breadth may loosen at Tier-2, while required keys, uniqueness and semantic-change notice remain fixed.

In §2 you classify every column separately. A column doesn’t simply take the product-level label from §1.

Field	*Values*	*Rule*
PII	`Yes` / `No`	Classify the column from its contents; an unresolved value is `Unknown`.
Sensitivity	`Public` / `Internal` / `Restricted` / `Confidential`	Classify each column, then raise §1 when the product label understates its columns.
Criticality	`Tier-1` to `Tier-4`	Record the consequence of failure, with Tier-1 highest.

In the supplied lifecycle standard, a PIA is routed when:

- any field has `PII = Yes`;
- any field has `PII = Unknown`, until the PIA resolves it to Yes or No;
- Sensitivity is `Confidential`;
- Sensitivity is `Restricted`, which also adds Phase-3 InfoSec review; or
- a named regulation applies, which routes through the Phase-2 PIA and the Phase-3 Security Design and InfoSec gate.

The supplied routing names SOX, DORA, GLBA, Reg S-P, GDPR and CCPA; the accountable privacy/security roles determine actual applicability and approvals.

Where do dependencies fit (e.g. “we expect a certain amount of data coming in to be able to meet our SLA”)? They go in the SLA section, and Appendix B has examples.

Data-contract Markdown template: schema

FROM CATALYST SKILLS

SOURCE skills/data-contract/SKILL.md @ Catalyst 1.13.0

The supplied source says:

```
## 2. Schema

| Column | Type | Nullable | Description | PII | Sensitivity | Criticality | Example |
|--------|------|----------|-------------|-----|-------------|-------------|---------|
| `id` | `STRING` | No | Primary identifier | No | General | High | `abc-123` |
| `email` | `STRING` | Yes | Customer email address | Yes | Restricted | Medium |
↪  `user@example.com` |
| `created_at` | `TIMESTAMP` | No | Record creation timestamp | No | General | Medium |
↪  `2026-01-15T08:30:00Z` |
| | | | | | | | |

> Add or remove rows as needed. Every column exposed to consumers must be documented here.

---
```

This is the actual linked template body, not the skill description. Its sample identifiers and classification labels illustrate the table shape; they are not CG product-information source fields or current approval evidence. Replace example rows with the verified source schema and reconcile labels with the supplied lifecycle standard before approval.

Lifecycle entry points

`dplc-concierge` works out what kind of work you're doing and sends you to the right skill. `dplc-preflight` checks your workspace, toolchain and access before a build, and `setup-workspace` fixes a missing tool. The stage and phase orchestrators call smaller skills. A document skill such as `data-contract` produces one document from a template. A capability such as `query-table` does one narrower thing. Before you run a skill, read what it depends on and what it produces, because some slash commands do a lot more than write text.

Triage context and reuse decision

FROM CATALYST SKILLS

SOURCE skills/dplc-triage/SKILL.md @ Catalyst 1.13.0

The supplied source says:

```
### Step 1 — Gather product context

Check the conversation and $ARGUMENTS for a product name or description. If
Phase 0 was just completed, pull the product name from that context. If not,
ask the user:

> "What data product are you triaging? Describe what it contains and the
> business need it serves."

Also ask (if not already known): Who is the requestor? Who is the Data Owner?

### Step 2 — Invoke `/marketplace-search` (Required)

**Delegate to `marketplace-search`.** Do this in two parts:

1. `Read skills/marketplace-search/SKILL.md` to load the sub-skill's
   instructions, exit-code handling, and output contract.
2. Invoke `/marketplace-search` with the product description / keywords
   derived from Step 1. Follow the sub-skill's workflow exactly — do not
   short-circuit by running an inline grep or a generic search.

Preserve the sub-skill's output (its match list, exit code, and recommended
action) verbatim in your reply. Then apply the conditional below:

- **Exact match found** — surface the existing product, recommend the
  requestor be routed to it, and **stop here**. Do **not** invoke
  `/demand-tracker-entry` or `/consumer-use-case-map`. Report the Phase 1
  gate as *Reject (duplicate)* and skip to Step 5.
- **Partial match found** — surface the candidate(s) and ask the user
  whether their need is meaningfully different. If yes, continue to Step 3.
```

```
  If no, route to the existing product and stop (as above).
- **No match found** — continue to Step 3.

Record the marketplace search outcome — it will appear in the demand tracker
entry Triage Notes.
```

This is a sequencing contract: gather the business context, preserve the search result and stop for an exact duplicate. The course reads and applies the decision logic to supplied evidence; it does not create a live demand-tracker entry.

The stage names vary. The lifecycle screen says Define, Deliver and Sustain. Some skill descriptions say Design, Build and Ship, and the skill IDs are `dplc-design-stage`, `dplc-build-stage` and `dplc-ship-stage`. They're the same three stages. RFD 0002 is the published standard, and it defines nine phases, numbered 0 to 8.

In class you draft documents on your laptop and review them. Nothing gets registered in the Demand Tracker, published to the catalogue or sent to an approver. A Markdown file in your repo doesn't meet the Phase 0 requirement for a real Demand Tracker entry, and an approval table that Catalyst wrote is just text.

The DPLC and the Concierge

Catalyst is built on a spine of nine phases, numbered 0 to 8, of designing and creating a data product. That's the **DPLC, the data product lifecycle**, the lifecycle represented by the supplied Catalyst version. Validate its current organisational scope and gates with the responsible product owner before live use. There is a skill called the **DPLC Concierge** (`dplc-concierge`) which runs through all nine phases end to end. It works out which phase you're at, asks for what it needs and guides you step by step. It can walk through several artefacts and capabilities; inspect the current phase and next action rather than starting an unrestricted end-to-end workflow for a local drafting task.

Here's the lifecycle on one page:

- **Define:** 0 Pre-requisite, 1 Triage, 2 Discovery, 3 Design.
- **Deliver:** 4 Build, 5 Validate, 6 Publish.
- **Sustain:** 7 Operate, 8 Evolve.
- **Phase 0 hard gate:** a real Demand Tracker entry and assigned Data Owner, DPM and Creator.
- **Phase 1 to 2 hard gate:** the Data Owner approves sensitivity, criticality and domain classification.

There are four roles you'll hear about all week: Data Owner, Data Product Manager, Data Steward and Data Product Creator. The Data Owner approves classification at the Phase 1 to 2 gate, so they decide whether the demand can go into Discovery. The Data Steward checks completeness in Discovery and signs off UAT in Validate.

In Phase 1, `marketplace-search` looks at the top `search_rank`. Above `0.5` means STOP, because a strong match already exists. Between `0.1` and `0.5` means INVESTIGATE, and `0.1` or below means PROCEED.

Marketplace decision rule

FROM CATALYST SKILLS

SOURCE skills/marketplace-search/SKILL.md @ Catalyst 1.13.0

The supplied source says:

> 4. **Apply the recommendation rule** based on the search rank of the top result:
>
> | Top result `search_rank` | Recommendation |
> |---|---|
> | `> 0.5` | **STOP** — strong match, point the requestor at it. |
> | `0.1` to `0.5` | **INVESTIGATE** — review existing before building. |
> | `≤ 0.1` or no items | **PROCEED** — no clear match; continue triage. |

The ranking rule recommends a next triage action. A candidate still needs scope, health, access, owner and contract review; the score is not proof of entitlement or fitness. Use supplied results unless approved live access is available.

Everything covered so far in the course corresponds to phases up to and including phase three, which ends with a data contract containing all the design work. Later phases turn it into code.

The data contract skill (`/data-contract`) is a mid-phase skill, so running it directly skips several phases. It sits at the end of the design phase, where everything produced so far (design canvas, product canvas, source-to-target mapping, glossary, traceability set) comes together, with the SLAs, into the contract. Run cold, it will ask for things you haven't produced, such as product name, schema, SLAs, quality and access details, or the path to an existing requirement or consumer use case. That's fine for exploration. Keep the result as a local draft and check which mandatory facts are still unsupported.

Use the supported slash command to select the intended installed skill and inspect the result. A natural-language request may also select a skill, but confirm which instructions and dependencies were used. Catalyst includes supporting client code, so selecting a workflow can involve operations beyond writing Markdown.

Contract revision from interview evidence

Revise the CG product-information contract from Monday's handoff

DURATION 20 minutes within contract review

OUTPUTS A revised draft, evidence-to-section mapping and correction log

1. Use the consumer/producer interview notes and brief from Monday. Identify one confirmed requirement, one constraint and one unresolved decision.
2. Draft a local contract with the data-contract skill and template. Specify that missing facts remain owned questions and that no approval or publication is implied.
3. Compare identity, schema, promises, quality actions, access, change policy and registry with the source notes. Mark any invented field, threshold, date or permission.
4. Give Catalyst the corrections and regenerate the affected sections. Open the file and inspect the actual change; verify the other sections still agree.
5. Hand the revised draft to a partner, who traces one requirement through the glossary, mapping and proposed check. Record the next decision and its owner.

Check: no unsupported commitment has become a confirmed fact; the output grain and consumer decision match Monday's brief; producer constraints remain visible. No empathy map is required. The optional stakeholder exercise adds adoption investigation when useful.

Reviewing the generated contracts

Use VS Code's Outline and Markdown preview to read the full artefact. Compare the draft against the source brief, template and consumer requirements section by section.

Review question	*What to inspect*	*Correction or next decision*
Does the schema come from the actual source?	Column names/types, row grain, keys, units and mapping	Remove unsupported fields or record a source requirement; check the catalogue path and accountable owner.
Can the source support the delivery promise?	Refresh event/cadence, producer SLA, timezone and calendar	Mark an unconfirmed target proposed and reconcile the dependency with the consumer deadline. An annual source cannot supply newly observed values every hour.
Does the quality metric have a meaning?	Denominator, comparison baseline, tolerance and population	Specify which records are expected and what source or calculation establishes correctness. A minimum respondent count is a use-case assumption to validate, not a universal rule.
What happens on a breach?	Severity, notification, response/resolution target and allowed output	Choose block, flag or quarantine according to the consumer agreement and fault scope.
Are decisions and permissions genuine?	Owner/steward roles, access groups, registry and approval status	Verify the roles and keep unapproved access or commitments outstanding.
Do the documents agree?	PRD, contract, glossary, mapping and quality plan	Correct the authoritative fact and all affected references; retain the evidence and reason.

Prefer precise measurement language. "Rolling 30 days", "each calendar day" and "each 30-day delivery" describe different windows, so reconcile them before using them to generate a check. Where priorities compete, use Monday's MoSCoW/RICE reasoning and retain the units and confidence behind the ranking. A structured draft helps reveal what remains to decide; it does not decide those facts for you.

Edge conditions and failure actions

These are illustrative training rules; the consumer and producer must agree the action for their product.

A quality rule in §4 should say what the pipeline does when the data is bad. Here are six conditions to cover.

Condition	*Example §4 rule for Morningstar exposures*	*Severity*
Late arrival	If the Morningstar file misses the market-close window, mark the output `STALE`, alert, and deliver it.	Medium
Missing source	If one expected Morningstar table is absent, publish with a `MISSING` status and do not impute it.	High
Null values	If `portfolio_id` or `as_of_date` is null, block delivery.	Critical
Schema mismatch	If a required column is absent or has the wrong type, block the batch. Don't coerce the value.	Critical
Duplicate records	If `portfolio_id` + `as_of_date` + `exposure_type` repeats, block delivery.	Critical
Out-of-range values	If an exposure falls outside 0 to 1, quarantine that row for review.	Medium

The action in each rule tells the pipeline how much usable data it may publish:

Workflow

- **Block delivery** stops the batch when publishing any part of it would break the contract, as with a missing required field or schema violation.
- **Flag and deliver** publishes usable data with a status and notification when consumers can work safely with a known limitation, such as a late source.
- **Quarantine row** publishes the unaffected rows and holds suspect values for review when the fault may be local to one record.

Why quarantine an out-of-range value? Because a range check can't tell a data error from a real but extreme market move. Quarantine keeps the row so a person can decide. The full quarantine discussion is in Appendix B.

The contract file itself follows the Open Data Contract Standard (ODCS), and Capital Group adds its SLAs under `customProperties.capitalGroupSla[]`.

Effective Catalyst practice

Catalyst can help you start an unfamiliar task, produce a structured draft, identify the next lifecycle artefact and preserve common conventions. Its templates make missing information visible; its capability skills can gather authorised evidence when the environment supports them. Use those benefits with a clear objective and a reviewable output.

Useful capability	*Creator responsibility*	*Evidence to keep*
Draft a contract or PRD	Supply consumer, source and scope context; identify unsupported facts	Brief, source/schema and owned question register
Apply a standard structure	Inspect the installed skill/version and reconcile source/template differences	Selected instructions/template and output file
Suggest checks or corrections	Validate business meaning, independent expected results and actual changes	Review criteria, test rows/results and correction log
Guide a lifecycle phase	Confirm the required role, artefact and gate; distinguish a local draft from an external action	Phase handoff and outstanding approvals
Gather or execute through tools	Verify authorisation, target and observed result	Tool/result evidence with credentials excluded

The creator owns the generated document or code, its validation and the decision to use it. The consumer and accountable producer/product roles still own their respective decisions and approvals. If a fact cannot be verified, record the uncertainty and stop that commitment from becoming an accepted premise. A useful model critique can suggest the next question; it cannot sign the contract on behalf of a person.

Exercise: interviewing Catalyst

You can teach yourself with Catalyst. You ask it about the lifecycle and about its own skills, then check what it tells you.

THIS SESSION ANSWERS

- What does Catalyst say the data product lifecycle at CG is?
- Which skills do you have, and which ones matter for design?
- How do you check an answer Catalyst gives you?

Knowledge check: Ask for the source

Where would you check the phase, role or skill capability the model reports? Which answer needs a published standard rather than another generated summary?

Catalyst's supplied skills and standards help you locate lifecycle guidance. Ask for the relevant SKILL.md or published RFD section and verify the answer against it. A generated explanation is a route to a source, rather than current policy authority on its own.

Locate lifecycle guidance in the supplied source

DURATION 15 minutes within design-document practice

OUTPUTS Seven answers with source paths and one owned question

Individually, for about 15 minutes, ask Catalyst:

1. What is the data product lifecycle at CG? What are the phases, and what are the requirements of each?
2. What is a data product?
3. What skills does Catalyst provide? Which are most relevant to the design phase?
4. Can Catalyst search Databricks, or other data sources / existing data products?
5. What parts of the data product lifecycle can Catalyst *not* help with?
6. What information do I need to create a data product using the DPLC?
7. What technologies does Catalyst use on the paved path, and what does each do?

Add one question about your product, then record the source path, version and section supporting each answer. Compare the installed plugin inventory with the skills actually available. Identify any answer that remains unverified.

Check the source behind the answer

Check the nine numbered phases, required artefacts and role responsibilities against the supplied published lifecycle reference. Then inspect a selected skill's inputs, outputs and dependencies. For a proposed new feature, identify the relevant evolution/redesign work rather than assuming a new build starts at the same phase as an existing product. For feedback, inspect the installed feedback skill and the authorised submission route; a classroom note is not a sent report.

Kafka is not assigned merely because a skill can mention it. Use the offered challenge routes and their explicit scope. Similarly, a skill's claim that it can help with both simple and complex products is something to test on a bounded example. Preserve an unresolved answer as a question with a responsible role.

Catalyst skills, phases and roles

The reviewed design must have a clear next artefact and accountable role before it becomes implementation input. Locate the relevant skill and its declared output, then keep the offered course route separate from the full lifecycle capability.

Knowledge check: Artefacts and accountable roles

Which phase owns your current draft, what must exist before the next phase and who can approve that decision?

Catalyst skills for Airflow

Catalyst's skill for generating Airflow is `generate-dag`. On Thursday you use it to generate a DAG for your drawn pipeline, then audit it against the drawn SLA (see the closing challenge).

DAG modes and output patterns

FROM CATALYST SKILLS

SOURCE skills/generate-dag/SKILL.md @ Catalyst 1.13.0

The supplied source says:

```
## Modes

| Mode | Schedule | MDS tags | Asset triggers | Validation |
|---|---|---|---|---|
| `prototype` | cron OK | optional | optional | lite |
| `production` | Asset-driven required (cron fallback with advisory) | all MDS required |
↪  required | full RFD 0011 |
| `lift-and-shift` | preserve existing | warn-and-add | preserve | full RFD 0011 |

See `references/mode-aware-emission.md` for the full mode matrix.

## Emission patterns

Three patterns. The skill auto-detects between the two YAML carriers; pure
Python is opt-in.

| Pattern | Detection | When chosen |
|---|---|---|
| **dag-factory** | Default | Repo has no `.blueprints/` directory OR user explicitly requests
↪  dag-factory |
| **astro-blueprints** | Repo has `.blueprints/*.py` + a blueprint loader | Team has the
↪  blueprint framework already in place |
| **python** | `pattern=python` explicit opt-in | Use when the DAG needs features YAML carriers
↪  can't express cleanly (multi-Asset triggers, dynamic task mapping, branching). |

If the YAML patterns are ambiguous, ask the user. See
`references/pattern-dag-factory.md`, `references/pattern-astro-blueprints.md`,
and `references/pattern-python.md` for the full templates.
```

The skill supports several patterns and production controls. Read these before the orchestration request so the offered capability is not mistaken for assigned course scope. The course follows its offered v2/Cosmos route; production callbacks, provisioning and alternative carriers remain outside that assignment.

Locate the next lifecycle phase

Use the nine-phase overview to identify the current phase, required artefact and next gate. Monday and Tuesday develop design evidence; Wednesday and Thursday encode and validate the assigned model and orchestration. Release and operation screens provide context, while the practical route states which actions are offered. Evolution can lead back into discovery and design when a requirement changes.

The four roles at phase 0

The supplied lifecycle identifies four responsibilities; one person may hold more than one. Its explicit Phase 0 go/no-go requires a real Demand Tracker entry, a Data Owner, at least one Creator and a DPM or equivalent representation. Identify stewardship responsibility as part of the product context:

- **Data owner** — identified, informed and willing to sponsor the request. They own the thing, and their role in the creation of a data product is sponsor and approver.
- **Data product manager** — gives you the requirements and owns the goals. Roughly what agile calls a **product owner**, the client or customer representative who tells you the requirements.

- **Data steward** — quality. In data-governance terms, a steward is usually someone in the domain team nominated to keep product quality high. The supplied material presents stewardship as a nominated responsibility; confirm the named role for the actual product. It's a nominated responsibility on top of someone's other tasks. It could be the same person as the creator.
- **Data product creator** — that's you. You specify, design and develop, use Catalyst and validate what Catalyst produces.

Asking Catalyst about a phase

Ask the installed Catalyst skill what a phase requires and produces, then verify the explanation against the published standard and the source instructions. The current delivery may offer more than one interface; confirm access and version before comparing answers.

For reusable questions about stakeholder involvement, completion criteria, transformation ownership and AI consumers, consult Questions and answers. Keep the current contract review focused on the consumer brief and its unresolved requirements.

From a reviewed specification to a generated model

Today is just a preview. You do the technical review on Wednesday, after the SQL and dbt sessions. `write-dbt-source` and `write-dbt-model` work from your project and source details, so give them the real table names. Don't let them guess a table from a familiar-sounding name. `setup-dbt-project` gives you a project skeleton, and you still have to check you can reach the warehouse. `dbt-unit-test` generates made-up test cases and a report, running on pandas by default or on PySpark. It's one kind of test among several in dbt.

> Knowledge check: From specification to model checks
>
> What expected example would reveal a wrong join, missing key or duplicate advisor? Which requirement establishes the expected result independently of the generated SQL?

A generated test report should say which SQL it tested, with what inputs, what it expected and which backend it ran on. So which test tells you what? A pandas test checks the logic of your transformation. A query on the warehouse checks the Spark setup and your Unity Catalog permissions. Neither one tests an edge case unless somebody wrote that case in. Keep the test cases and compare them with the reasons in the quality-controls plan.

For an advisor roll-up, a good set of edge cases is two portfolios with the same advisor, an advisor with no eligible portfolios, and a portfolio whose advisor key doesn't match anyone. Those check the sum, what happens when there are no rows, and referential integrity. Add a duplicate advisor key to see whether rows multiply. If the generated tests just rerun the model's own aggregation on its own output, ask for the expected result worked out separately from the test rows.

Model authoring produces SQL and YAML

FROM CATALYST SKILLS

SOURCE skills/write-dbt-model/SKILL.md @ Catalyst 1.13.0

The supplied source says:

```
## Outputs

- `<product>/dbt/<layer>/<model_name>.sql` — rendered model.
- `<product>/dbt/<layer>/_<layer>__models.yml` — created or updated
  with the model's column test stubs.
```

The skill produces both the model and its declared column-test stubs. Generating the files does not verify their business logic. Keep the source mapping and expected example rows with the handoff.

Validation evidence and gate findings

FROM CATALYST SKILLS

SOURCE skills/validate-dbt/SKILL.md @ Catalyst 1.13.0

The supplied source says:

```
### Step 4 — Emit the report

Render `templates/validation-report.md`. The report carries:

1. **Header** — data product / selector, resolved target, mode, timestamp,
   and which mode (inline vs on-disk) the gate ran in.
2. **Summary** — counts of `blocker | advisory | ok | n-a` and the one-line
   verdict. The verdict is `DEPLOY_READY` only if zero blockers remain.
3. **Findings by gate** — one block per gate (Compile / Schema diff / Test
   resolution) with the rule ID where one applies, severity, location, and a
   one-line remediation hint.
4. **Next steps** — ordered, blockers first; each names the skill to consult
   (`/troubleshoot-dbt-on-databricks` for failures, the authoring skill for
   missing coverage). If `DEPLOY_READY`, the final step notes the change is
   deploy-ready and to proceed with the bundle deploy — the paired
   `/deploy-dab` gate partner is planned (G-01), not yet built.

The gate never edits models. If the user wants the failures fixed, route them
to `/troubleshoot-dbt-on-databricks` or the relevant authoring skill.
```

The report distinguishes blockers, advisory findings, confirmed checks and checks not available in the selected mode. A source skill may describe deploy readiness; this course uses the findings to review the model and does not authorise a bundle deployment.

When we get to generate-dag on Thursday, look at the Databricks bundle, job and repository dependencies it declares. The skill can produce DAGs in several patterns, and the one it picks may not match the Cosmos challenge. In class we only run the Cosmos challenge. Don't deploy a bundle, set up a Kafka emitter or add Datadog callbacks just because the skill offers them.

By the end of the week you'll have a chain of documents: interview notes, proposed contract, corrected mapping, model, tests, the Airflow run and what the consumer makes of the result.

The paved path: Capital Group's platform

This afternoon goes technical. You meet each paved-path tool in plain words and get your dev container running.

THIS SESSION ANSWERS

- What is the paved path?
- Where does the data live, and what moves it and transforms it?
- Why do we work inside a dev container?

> Knowledge check: From code to data
>
> Which tool versions code, which schedules execution and which executes SQL? Does a successful release establish correct data?

The paved path is the supplied Capital Group collection of technologies and conventions for product creation. Catalyst supports parts of that work. This tour locates code, compute, storage, scheduling, governance and release responsibilities before Wednesday's systems and development practice.

What's on the paved path

The supplied platform reference includes:

- **Git and GitHub**
- **Docker** (and dev containers)
- **dbt**
- **Airflow** (with **Astronomer** as the vendor, and Cosmos for the dbt integration. Park the name Cosmos until Thursday, when it gets explained properly alongside Airflow.)
- **Harness** — build, release and infrastructure-pipeline context
- **Databricks** — including Unity Catalog, the Marketplace and the SQL warehouse
- **Catalyst** — the AI skills companion
- AWS S3 supplies storage and Datadog supplies observation in the platform context. Other ingestion or legacy tools may exist; confirm a product's actual dependencies rather than inferring a retirement or migration requirement from this overview.

The assigned transformation practice uses dbt. You may encounter existing Spark/PySpark products; their migration decisions belong to the accountable product and platform owners.

A data product is three folders

Strip the mystique before the tool tour. Fundamentally a data product is about three folders (sometimes four). Call them raw, transformed and report-ready. A file (say `data.csv`) moves through stages until it's `report.csv`, source-aligned in and consumer-aligned out. These are *relative* terms. Your final folder is somebody else's raw folder, because alignment is always alignment *to people*. (Industry and CG layer names, such as bronze, silver and gold or landing, ingest and curate, are part of the data-systems glossary covered on day 3.)

That framing yields the questions that map the whole architecture:

Question	*Answer in the supplied platform example*
Where is the data?	**S3** — Amazon's cloud files-and-folders system
How do we find it?	**Databricks** — the **Unity Catalog** (the record of what exists) plus the **Marketplace** (the search tool over it)
What moves it?	**Airflow** — triggers and orchestrates movement between the folders
What transforms it?	**dbt** — the proposed universal transformation tool
What set all of this up in the first place?	**Harness** — creates the folders and infrastructure; CI/CD

Git and GitHub

Git is version control, with three families of capability. There's versioning code (multiple versions of the entire piece of software existing in parallel, managed by one tool), code history and sharing.

GitHub is not Git. GitHub is a platform built on Git, a single source of truth, a **book of record for code**. The main thing GitHub adds for releasing software at CG is the **pull request (PR)**, a request to take

one version of the code and integrate it into or replace another, usually the *live* version. A PR requests code integration. Environment promotion separately selects a reviewed revision, target environment and required approvals.

At Capital Group, PRs are part of the permission structure. The pull request is often where acceptance for a release actually happens, which makes GitHub, a vendor holding the book of record for code, a key part of the permissions process. (More on why tomorrow.)

Docker

The problem Docker solves is portable software. The code on GitHub is not a full specification of the software, because code has dependencies. There are libraries, other code and an operating system.

A **container** is a bundling of the software with *all* of its dependencies (even, more or less, the operating system, with a caveat covered below). Think of it as a mini PC that runs standalone. Press play on a container and inside could be Airflow code, dbt code, SQL, all working together to produce your application. A **runtime** is the programme that runs a container. (Docker in depth after the dev-container setup.)

dbt

dbt is the data build tool. It does the data transformations, and it's basically SQL plus templates plus tests. This is the tool that performs the source-to-consumer alignment.

At the most basic level, dbt is a **SQL templating engine**. SQL is normally one query per statement, and dbt lets you turn a query into a template (string interpolation, placeholders) and run it across scenarios. Think of it as an SQL generator. The salesy framing is that it "brings software-engineering practices to SQL": versioning of queries, modularity, code generating SQL.

One distinctive feature is that it wraps your `SELECT` queries in `CREATE TABLE` statements automatically. A data analyst who only knows SELECT inside-out can write plain queries, and dbt treats them as instructions to create intermediate tables, handling persistence in the background. (And increasingly you don't hand-type the SQL anyway. You ask Catalyst, and it writes the dbt.)

What's a dbt test? A test of the expected output of a query, very often phrased so that the expected output is no rows. Example: prices should never be negative, so query for negative prices; the test passes when the query returns nothing. A core part of dbt testing is writing queries that should never produce data. (More kinds of test tomorrow.)

Airflow

Everyone says "orchestration", so what is orchestration? Think of a conductor. While the orchestra plays, the conductor is timing, cueing and coordinating. Violins in, drums in, timpani in. Airflow times, cues and coordinates **tasks**, where a task is just a programme that runs on a machine. (Scheduling is one part of Airflow; its dependency graph, execution records and retry/recovery behaviour provide additional coordination.)

So Airflow lets you schedule tasks daily, weekly or at a set time, chain them (task 1, then 2, then 3), branch conditionally (this task if it fails, that task if it succeeds) and monitor the whole pipeline.

A simple local sequence may need little coordination. Airflow is useful when dependencies, scheduling, execution state and recovery need an explicit managed workflow, whether the tasks execute locally or remotely.

The main task type we'll orchestrate is dbt transformation programmes. If Airflow is the conductor, dbt jobs are the orchestra, with the source section, domain section and consumer section cued in order. In the paved-path picture, Databricks is where the data lives and dbt is the transformation piece, the SQL that does the mapping. Airflow is the trigger. It supports scheduled execution and evidence about

runs; meeting the consumer SLA also depends on upstream data, correct transformations and the agreed failure response.

Astronomer

Two answers to “what is Astronomer?”:

1. **Managed Airflow**, an orchestrator run for us in the cloud. We rent the conductor. (Just as Databricks vends Spark, Astronomer vends Airflow.)
2. Just as valuable for the paved path, Astronomer packages Airflow in a Docker container. That gives you a cross-platform bundle that runs locally (Windows, Linux, Mac), and the cloud side runs those same bundles remotely. Astronomer is basically a bundling-up of Airflow. (Hands-on tomorrow and Thursday.)

An independently managed Airflow deployment can let a team plan changes at its own boundary while still coordinating shared dependencies. Astro also provides a local project runtime for development and tests. Independence does not remove cross-team compatibility, access or upgrade checks. Astronomer documents the local testing workflow.

Databricks

The problem Databricks set out to solve (~2014) was that querying large volumes of data was hard. Their solution was to put the data in files and folders in the cloud and run queries over them in parallel. It’s like keeping your data in a giant Google Drive, with **Spark** as the query system over it (the people who founded Databricks are the people who made Spark; earlier systems worked similarly, Spark was faster).

But files and folders don’t really work well on their own. What people actually want is a **transactional database**, where data isn’t lost and operations succeed or fail atomically. Those are ACID guarantees, and they’re very hard to maintain with data scattered across parallel files. Hence the **lakehouse**, a system of query and data management that *keeps* the files and folders but gives you the promises of a traditional database. It presents managed tables with transaction semantics. ACID refers to atomicity, consistency, isolation and durability; the exact behaviour depends on the table format and operation. Wednesday develops those distinctions. More on that tomorrow.

Here’s a more careful breakdown, useful in meetings. In the paved path, Databricks is the compute layer (it runs queries over your data), plus discoverability and governance (Unity Catalog, the Marketplace). Those last two are really the same thing, metadata management around your data.

It is *not* the storage. S3 is the storage, cloud files and folders. The supplied architecture places externally stored data in S3 and uses Databricks for SQL compute and table governance. This is an example architecture, not a claim about the proportion of enterprise storage or every vendor delivery route. When someone names a “data platform”, ask which storage, compute, governance and serving responsibilities they mean.

What lives in S3 is often **parquet files**, which are column-oriented storage. You *write* data a row at a time but *read and analyse* it a column at a time, so columnar layout is much faster for analytics. **Delta format** is parquet plus metadata, meaning version history, governance metadata and, most importantly, transaction support (you can `DELETE FROM` a Delta table, which you cannot do against a bare parquet file without rewriting it). Databricks then presents a “pretend table”, which is metadata over the S3-backed files.

This course uses the lakehouse for analytical products. Evaluate transactional latency, write patterns and application requirements separately rather than assuming an analytical design fits every workload.

Platform selection has technical and institutional dimensions. Compute capability is one consideration; supported vendor delivery, integration, access, contracts and the team’s operating knowledge are others. In the supplied example, sources deliver files into agreed storage locations and Databricks executes the

downstream analytical transformations. The delivery convention still needs an accountable source, a contract and arrival checks. A familiar enterprise platform can reduce some integration work without removing product-specific validation.

So what does Databricks do on the paved path? It runs our dbt. dbt holds the transformation rules; Databricks executes them over the files and folders (which live on Amazon S3).

Since the word will keep coming up, **Spark** is the parallel query engine underneath Databricks, the machinery that splits a query across many machines and reassembles the answer. **PySpark** is Spark's Python interface. Because PySpark sat on the paved path until recently, plenty of teams still work in it, so expect to hear the word from colleagues on older products. On the paved path a data creator writes dbt instead. The SQL still executes on Spark underneath, you just never drive the engine directly.

Why dbt for this transformation practice? dbt supplies SQL model, dependency, test and documentation conventions. It connects to supported data platforms through adapters, so supported targets and behaviour depend on the selected adapter and version. The training route uses its specified Databricks configuration; it is not an instruction to run dbt against any file or database. dbt documents its supported-platform/adapter relationship.

PySpark exposes Spark through Python and can support transformations with different requirements. Here SQL-based models keep the business mappings and tests explicit for the assigned product. Choose a supported tool according to the actual requirement and platform convention rather than assuming one interface is universally simpler.

Doesn't Databricks have its own job scheduler? Why Airflow? Yes, it does, and there's overlap. The course uses Airflow to express the wider dependency and scheduling workflow, with Cosmos connecting the assigned dbt project. Databricks jobs can also schedule work; the existence of overlapping capabilities does not change the offered course route.

Keep the interfaces and dependencies visible when considering portability. Open-source components, managed services, storage formats and organisation-specific conventions each have their own compatibility and operating boundaries. A named product alone does not determine the effort of moving a real pipeline.

Harness

Harness provides the demonstrated build/release pipeline context, including controlled infrastructure steps where configured. Airflow schedules and coordinates the data workflow. A successful release means the relevant code or configuration step completed; it does not establish that a later data run has correct values or meets its consumer promise.

Harness remains a demonstrated capability here; the offered learner tasks do not provision live infrastructure.

The paved path in one diagram

GitHub holds the versioned dbt code. Harness builds and promotes its release. Airflow, packaged and managed through Astronomer, schedules runs; Databricks executes dbt SQL over source-aligned S3/Delta tables and writes consumer-aligned tables. Unity Catalog provides governance and discovery; Datadog observes the runs.

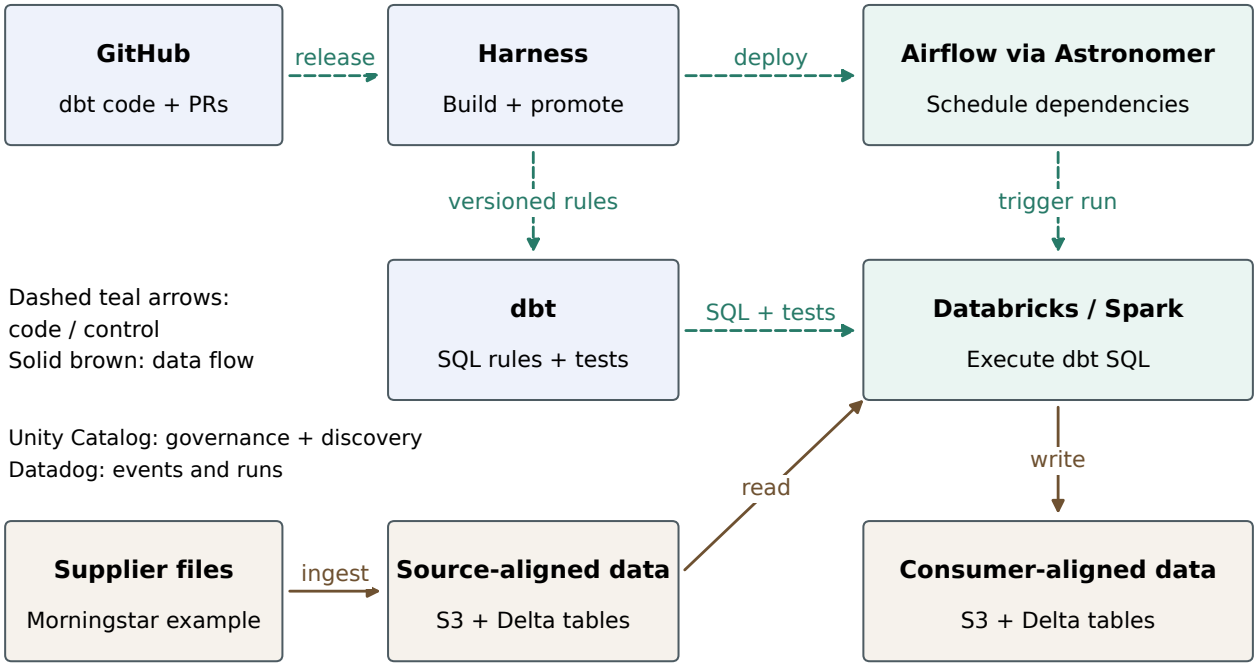


Figure 14: The paved path: dashed teal arrows show released code and control from GitHub through Harness, dbt and Airflow to Databricks; solid brown arrows show supplier files entering source-aligned S3/Delta tables, read by Databricks and written as consumer-aligned tables.

The arrow key separates the deployment/scheduling path from the data path. dbt supplies the SQL rules and tests; Databricks supplies their execution engine. Harness remains a demonstrated release capability in this course.

The three components of the course

1. **Designing data products** — interviewing, collecting requirements, the canvases, the questions to answer.
2. **Catalyst** — using it to support design and build.
3. **The tools** — the paved-path technologies above (the rest of the course).

Why the quick tour today? Partly for calibration. The course is designed so that completely non-technical people can leave after day 2 knowing roughly what these words mean, enough to follow engineering meetings. The rest of the course goes into detail.

> Case study: Common platform conventions support review
>
> **Paved-path product perspectives**
> Several engineering teams can develop similar products using different local environments and validation practices. Decision: maintain the product's code, configuration, tests and documentation in a coherent repository with supported tooling and controlled release steps. Reasoning: common conventions improve reproducibility and make affected dependencies easier to inspect. Lesson: a shared setup supports review while a successful local run remains separate from production acceptance.
> A legacy consumer needs a source team's entity relationships, types, keys, refresh cadence and definitions. Decision: use authorised source-repository documentation to draft the mapping, then verify it with the source and consumer evidence. Reasoning: self-service discovery can reduce repeated meetings, but incomplete

documentation can still produce a wrong interpretation. Lesson: keep documentation-derived conclusions distinct from data checks and owner decisions.
Teams may develop similar dashboard, testing or persona-oriented skills without knowing about each other. Decision: make reviewed skills, owners, versions and feedback routes discoverable. Reasoning: common lessons can be reused while different products still need their own context. Lesson: a catalogue of skills is useful only when the instructions are maintained and their behaviour is evaluated.
RFDs record standard proposals and their published status; ADRs record architectural decisions for a product. Keeping them with the code connects implementation changes to their rationale. Catalyst can support parts of the lifecycle, but capability coverage and installed versions must be inspected rather than assumed.

Local ownership and the shared service

Giving each team its own Airflow is one way to localise ownership, as we saw with Astro. Your team controls its own schedules, but another team may still own the ingestion product upstream of you, and you still depend on it. So write down four things: what the upstream team promises, how fresh your consumer needs the data, how you'll know the upstream data has arrived, and who to call when it hasn't. Your scheduler only runs your jobs.

> Knowledge check: Local ownership and dependencies
>
> What upstream promise must your product rely on, and what can your own scheduler control?

Search the marketplace before you decide to build, both in Triage and whenever someone asks whether a product already exists. The search gives you candidates. Before you reuse one, check that it fits your need, that you can get access and that it has been validated. If the search finds nothing, check whether you can see everything and whether you searched with the right words before you claim there's a gap.

Catalyst calls both the contract and the PRD a source of truth across the lifecycle. They cover different things. The PRD holds the problem and the requirements as they evolve, and the approved contract fixes the schema and the service promises. If the two disagree, take it back to whoever owns that business decision, correct both documents, and test the code against what was agreed.

Dev container setup

Check the development environment before practical execution. Use Using VS Code and the training dev container for the beginner walkthrough, terminals, commands, runtime boundaries and tool purposes. A successful container start is only one readiness check.

> Knowledge check: Readiness at each boundary
>
> What does opening the container establish? What still needs a separate authentication, data-access or orchestration check?

Readiness across your environments

There are four places your work runs. Your laptop runs the editor and Docker. The dev container holds the repository's tools. Astro starts a separate local Airflow with its own services. Databricks does the remote compute and controls access to the data. Getting the dev container open only tells you the first two work. You still have to check you can reach the warehouse and run a DAG.

The dev container is built from `.devcontainer/Dockerfile`. The Dockerfile at the root of the repo is the Astro runtime image for Airflow. If something is wrong with your dev container, edit the one in `.devcontainer`. Anything you save inside the mounted repository folder survives when the container is

rebuilt. Anything you save elsewhere in the container is lost when you delete or rebuild it, although simply stopping the container keeps it.

Before the technical sessions, use the setup and readiness walkthrough and the offered v2 challenge route. Run `dpc-check` in the dev-container terminal and read what each check reports. A failed local check, a registry you can't reach, an OAuth failure and a missing table each need a different person to fix them, so write down the symptom and which environment you were in. Leave credentials out of the support ticket. Ask for help early, while there's still time to sort out access.

Setup uses interactive OAuth by default. Run `mise run setup:databricks` before you start Astro, because Airflow won't pick up settings you create later in another terminal. Catalyst's `connect-databricks` skill logs in with OAuth machine credentials, which are separate from your own login, and it refuses personal access tokens. Use OAuth and don't create a PAT.

The checklist is short:

- Docker available in the approved host/container topology, with resources allocated according to the supplied training setup instructions.
- The Dev Containers extension in VS Code.
- The repo cloned to a local path (Documents or similar, never OneDrive).
- On Windows, `git config core.autocrlf input`, so pulls don't re-add the carriage returns that break shell scripts inside the Linux container.
- Above all, one successful Reopen in Container completed end to end.

Complete workspace/container preparation before timed practice. A successful Reopen in Container establishes the editor connection; the readiness checks then distinguish local tools, Docker runtime and authenticated remote access. Record a failed prerequisite with its error and ask the facilitator about the assigned support route.

Use the supplied training-repository instructions to open the EDO data-products folder in VS Code and select Reopen in Container. Allocate preparation time before the practical block: image pulls and setup duration depend on the image cache, network and host resources. Read the setup logs and retain the actual error if a step fails. Confirm the available support arrangement with the facilitator; the beginner walkthrough explains the sequence.

The container supplies a consistent toolchain and reduces differences between local installations. It does not establish remote permissions, warehouse availability or successful Airflow execution; check those boundaries separately.

It also mirrors reality. The dev container comes from the same kind of repo an EDO paved-path developer gets on day one, so it *is* the environment in which data products are developed. Databricks itself is a proprietary remote cloud environment, and the local container holds a *connection* to it. For a connected profile, the authenticated client submits work to the assigned remote workspace. The assigned v2 route uses dbt-databricks; an offline parse or file check does not establish remote execution.

Inside the supplied training workspace, follow the offered v2 route and its phase/task links. Ask Catalyst to explain a task against the authoritative README, then check its interpretation before changing a command. Record an unclear prerequisite or error with its runtime and ask the relevant support role.

dbt is the core skill of this course. It's what you'll actually use day to day as a data creator. Airflow is a technology to learn and understand, but in practice engineers set up the queues and schedules. dbt is where data products get created.

Known issues

- If startup fails, read the setup/build log before changing resources. Check the supplied environment requirements, available host memory, registry/network access and the reported error. Apply the relevant approved setup remedy, then rerun the specific readiness check; a successful image build does not establish authenticated remote access.
- The repository contains two Dockerfiles. The one in `.devcontainer` builds the dev container, and the one at the root is the Astro image for Airflow. Use the correct runtime boundary when diagnosing a failure; the beginner appendix explains which image each file builds.
- Stale cached container images can cause failed builds even on fresh clones. A clean rebuild without cache, or recloning the repo, fixes it. Preserve local work and record the failure before rebuilding or making a fresh clone; a merge conflict requires understanding and resolving the conflicting changes. Avoid cloning into OneDrive.
- Windows line endings have bitten before. Windows pulls can re-add carriage returns, which breaks shell scripts run inside the Linux container (shell scripts must be LF).

To set up the Databricks connection from inside the container, run the Mise task (`mise run setup:databricks`, which uses OAuth by default). It prints a workspace URL. Keep pressing enter until you reach the login page, authorise the permissions request, then select the workspace and warehouse. You won't use it today, but you need it from tomorrow, so set it up now.

How do the skills reach Databricks? There's no magic connection. Catalyst generates Python that uses the client scripts in the repo, and those scripts read the credentials that setup saved.

> **Credentials and packages**
>
> Use OAuth in class. The supplied setup uses interactive OAuth; Catalyst's connection skill has its own machine-credential requirements. Follow the approved setup route rather than creating a token to bypass it. Python packages come through Capital Group's internal Nexus (`cgrepo-aws.capgroup.com`, index `cgpypi`), so don't install straight from public PyPI.

> **Knowledge check: Ready for collaboration practice**
>
> What information would a reviewer need in a pull request for a contract or model change? If you have not opened one before, identify the walkthrough you will use in Wednesday's Git practice.

Tuesday design and readiness review

Keep the reviewed brief, PRD, contract, glossary, mapping and owned questions together for Wednesday's model work. Keep setup/readiness evidence separate so a source or access gap cannot be mistaken for a model defect.

> **Learning Review**
>
> Retrieve: explain model, harness, skill and observed action without looking back. Name the purpose of the PRD, contract, transformation specification and quality-controls plan, and identify the current lifecycle role/phase.
> Apply: choose one CG product-information draft defect and trace the correction through the consumer evidence, schema, mapping and proposed check. Explain why another model's agreement would not establish the missing fact. Identify a producer dependency and the platform tool that will execute or schedule the later work.
> Self-check: the draft contains no unsupported approved status, access right, currency conversion or source column; requirements and quality actions have evidence; unresolved decisions have owners; local container,

> remote access and Airflow readiness have distinct recorded results. If one check remains incomplete, write the next action and responsible role before handing the work on.

Keep this review to hand for the closing platform-decision session. Wednesday connects those requirements to source grain, dbt models/tests and collaborative repository work. Thursday adds scoped orchestration and recovery evidence. The offered v2 routes define what will run; screenshots and general skill capabilities do not add production tasks.

Demonstrations and challenge sources

Thursday's prepared demonstrations connect release, Airflow, Databricks and observation screens with the teaching. Use the day preview tables to locate each sequence and its purpose. Your instructor will confirm which prepared demonstrations are available.

The group activities develop interview and review decisions. For technical practice and follow-up, use the offered v2 routes and the complete challenge reference, with each solution at the end of its owning guide. A general skill capability or historical challenge label does not extend the assignment.

On screen: Platform onboarding

DevHub is where you start a new data product on the platform. The LaunchLake guide walks you through it, the prerequisites page lists the entitlements you'll need, and the vending wizard creates the repository and scaffolding for you. The wizard will happily create a repository before anyone has understood what the consumer needs, so keep Monday's design work to hand.

An entitlement is just permission to use a service. If you can't start the runtime, work out what has gone wrong first. Is a prerequisite missing, is the network down, or is your local toolchain broken? Each of those goes to a different person. In class you work from the training repository, so you won't be filling in the provisioning form on these screens.

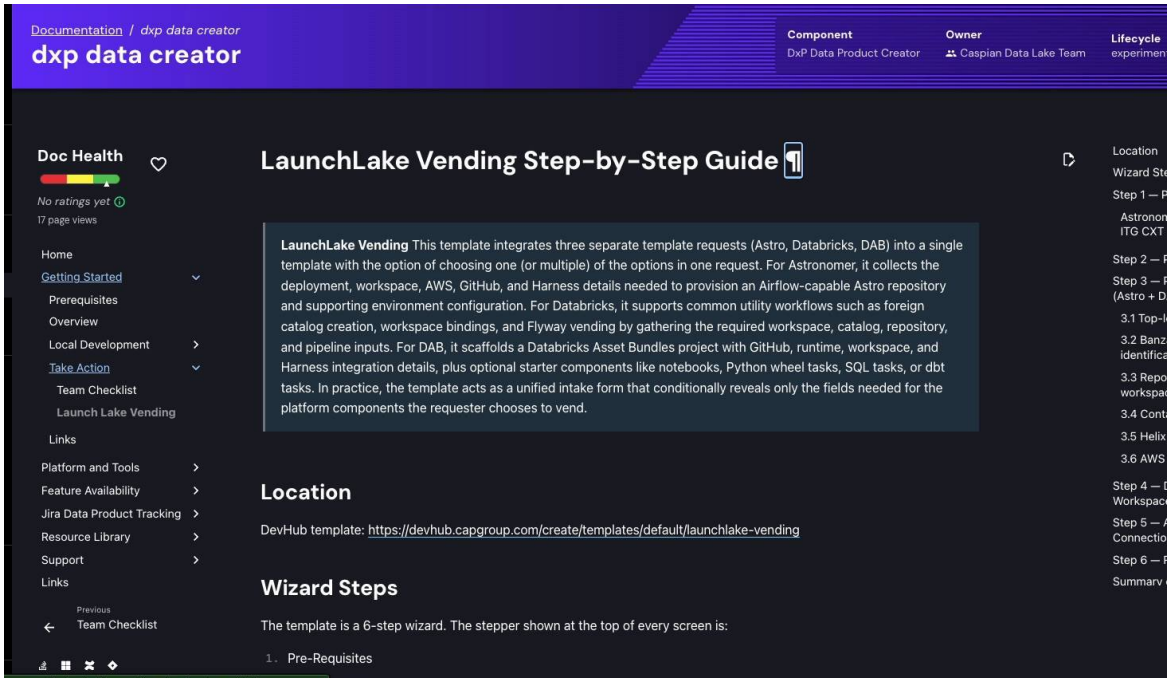


Figure 15: DevHub: LaunchLake guide

Step 1 — Pre-Requisites

Before opening the template, confirm the following (verbatim from the source document):

1. Helix account should be approved and ready.
2. Databricks Admin roles are added to the right role for entitlements.
3. Astronomer access is managed at the workspace level and aligned to your organization. To request access, go to **Access Central** and submit a request for the appropriate entitlements listed below.
4. Request AD groups for **astronomer** (if not exists).
5. Get access to harness and banzai project name.
6. individual responsible for vending must have access to Harness, GitHub, AWS Helix accounts and Astronomer workspace before vending process can begin.
7. Go the **Launchlake** vending template.

Astronomer Entitlements — ITG CXT

ITG CXT

Env	Entitlement	Astronomer Role	Access Central Role Name
DEV	OZ_DEV_AA00002933_ASTRONOMER_ITG_CXT_ENGINEER_R	Workspace Member	ITR - Astronomer ITG CXT - Development User R
DEV	OZ_DEV_AA00002933_ASTRONOMER_ITG_CXT_ENGINEER_RWX	Workspace Operator	ITR - Astronomer ITG CXT - OZ DEV User RWX
SQA	SNP_ATS_AA00002933_ASTRONOMER_ITG_CXT_ENGINEER_R	Workspace Member	ITR - Astronomer ITG CXT - SNP ATS User R
SQA	SNP_ATS_AA00002933_ASTRONOMER_ITG_CXT_ENGINEER_TSA_RWX	Workspace Operator	ITR - Astronomer ITG CXT TSA - SNP ATS User RWX
PRD	CPZ_PRD_AA00002933_ASTRONOMER_ITG_CXT_ENGINEER_R	Workspace Member	ITR - Astronomer ITG CXT - CPZ PRD User R
PRD	CPZ_PRD_AA00002933_ASTRONOMER_ITG_CXT_ENGINEER_TSA_RWX	Workspace Operator	ITR - Astronomer ITG CXT TSA - Production User RWX

Figure 16: Prerequisites & entitlements

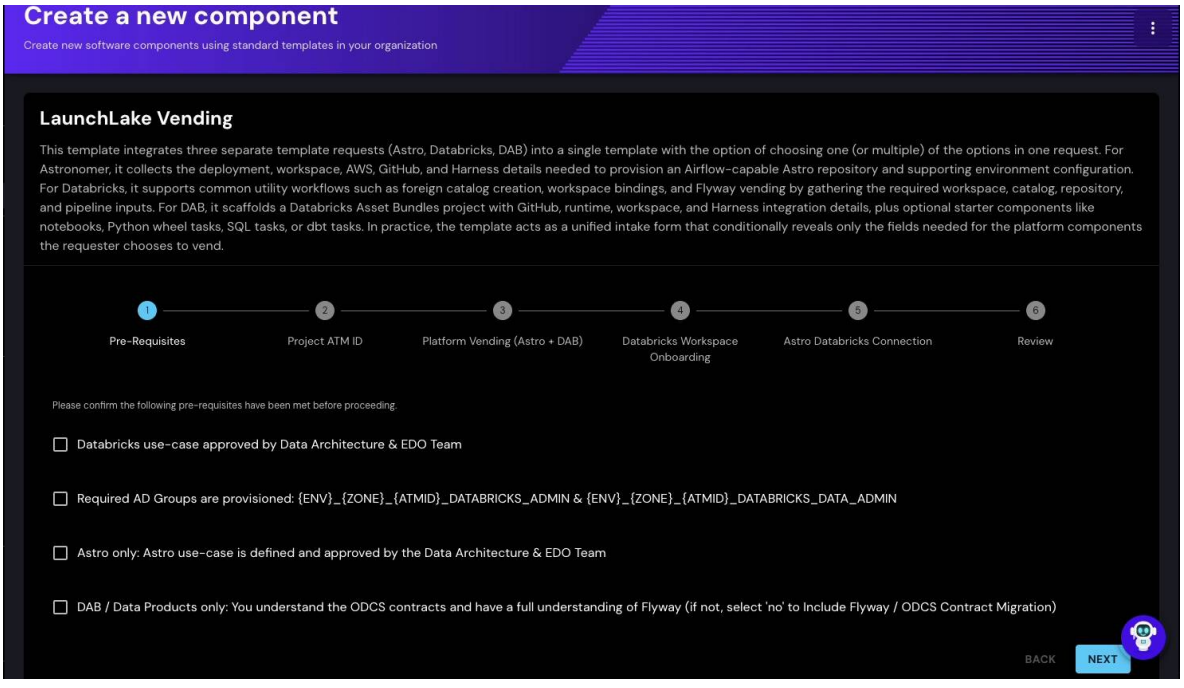


Figure 17: LaunchLake vending wizard

On screen: The paved path

So what's on the whiteboard? Code lives in GitHub, and Harness builds it and promotes it through the environments. That's deployment. Airflow runs the data later, on its own schedule. People often assume a green release means the data is right. It only means the code went live, and the nightly run can still fail.

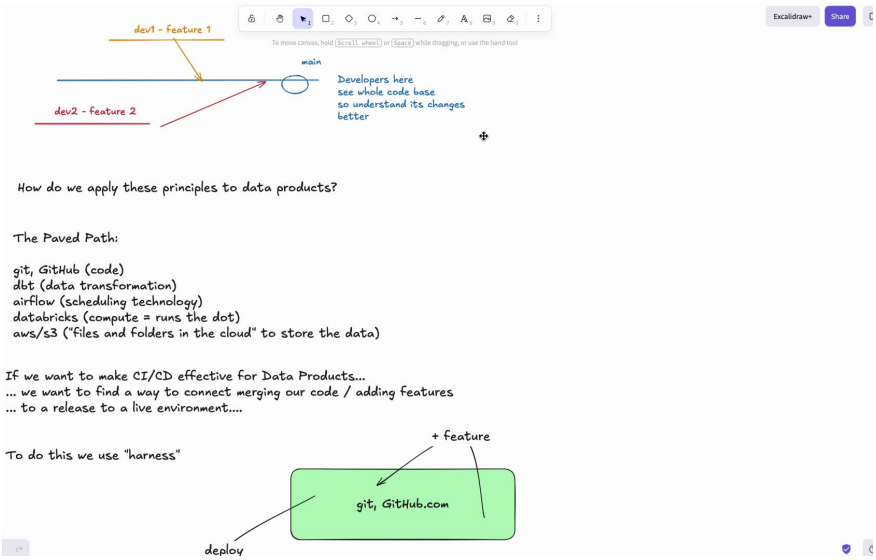


Figure 18: Paved path: from Git to Harness

On screen: Data contracts

So where does the contract live? In the repo, in a `contract` folder next to the dataset it describes. The screenshot of the YAML shows only the top of the ODCS file, which is the name, version, `status: draft` and the start of the schema. Ownership and quality rules sit further down, so open the whole file when you want to trace a promise to the test that enforces it.

You'll also see a `migrations` folder beside the contract. The client repository turns ODCS files into database migrations using `datacounsel` and Flyway, and the pipeline commits the generated SQL there. That step changes the table schema itself. It's separate from running dbt models and from getting consumers to sign off. You don't need to run migrations for the capstone.

```yaml
apiVersion: v3.0.0
kind: DataContract
id: urn:datacontract:bmr_admin:fibmr_account_ui_reverse_feed
name: fibmr_account_ui_reverse_feed
version: 1.0.0
domain: bmr
status: draft
schema:
- name: fibmr_account_ui_reverse_feed
  type: table
  format: delta
```

Figure 19: ODCS contract YAML

On screen: Catalyst skills

This is what a Catalyst skill looks like on disk: a folder of skills, each with a SKILL.md. Open one and you'll see the workflow, plus the skills it depends on, the documents it reads, and the templates and standards it follows. The version in GitHub may not match the one installed in your editor, so open the installed copy.

The NFR example drafts non-functional requirements, the operating constraints such as response times and availability. Whether its draft is right depends on your real requirements.

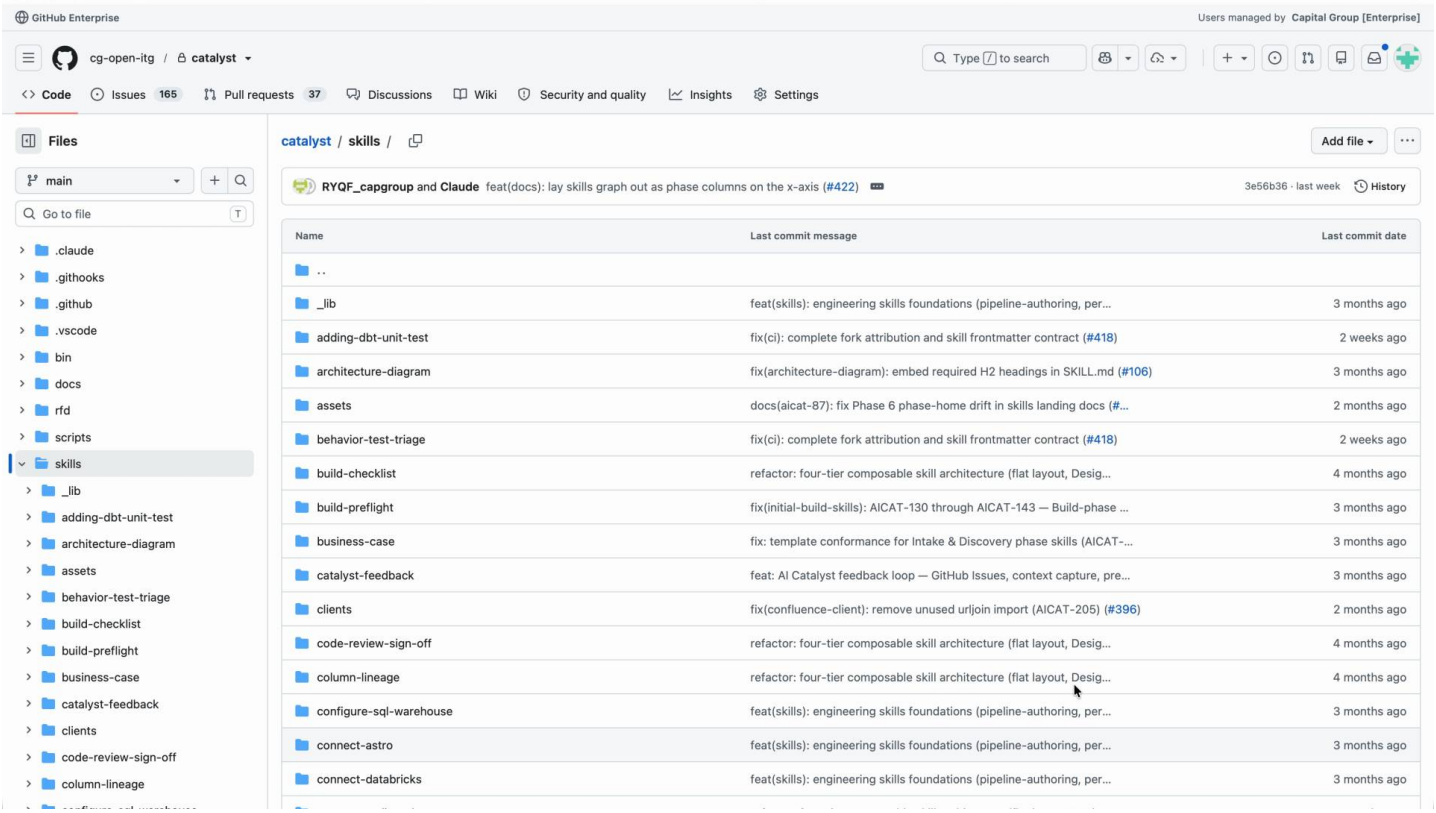


Figure 20: Skills in GitHub

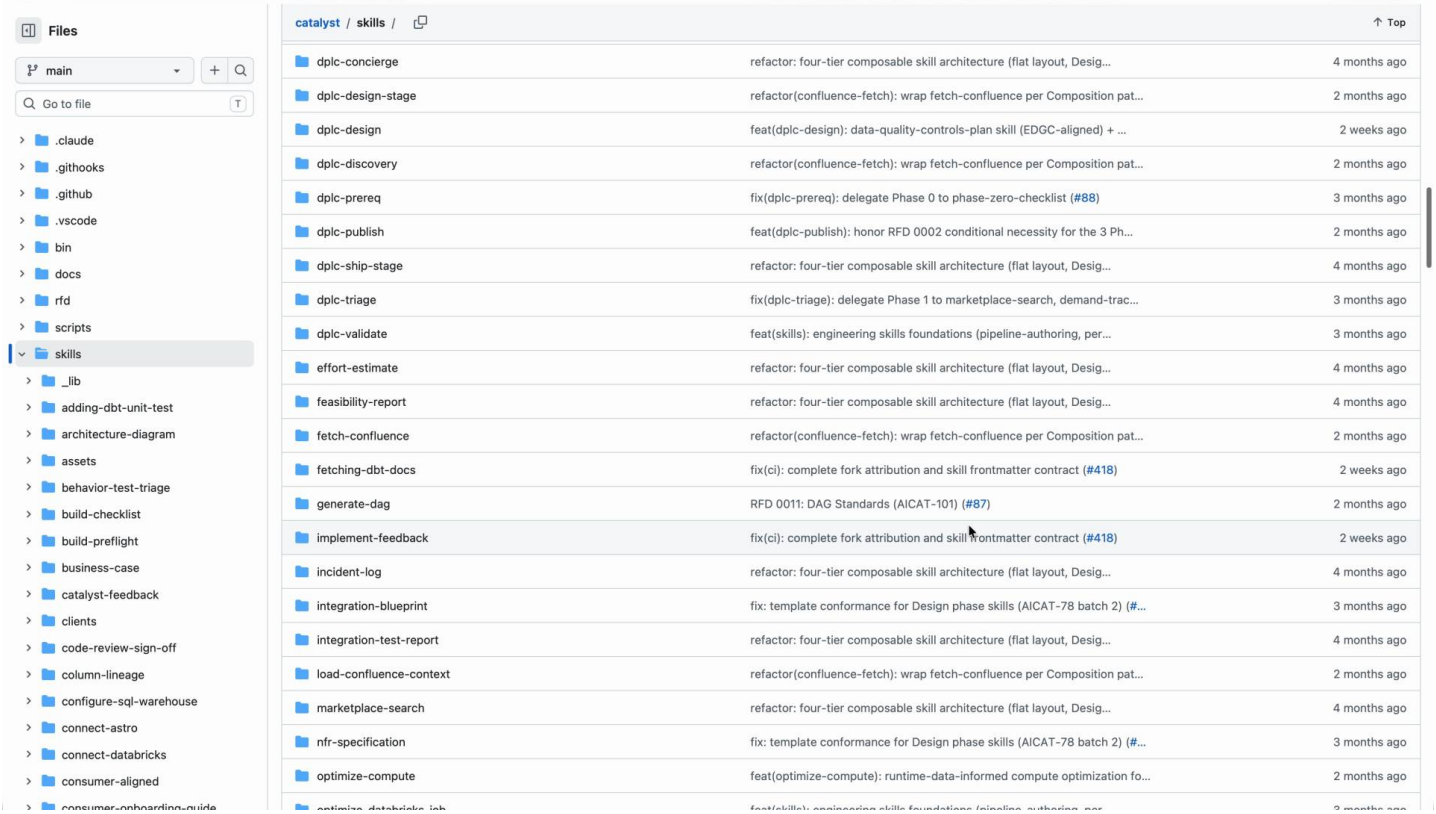
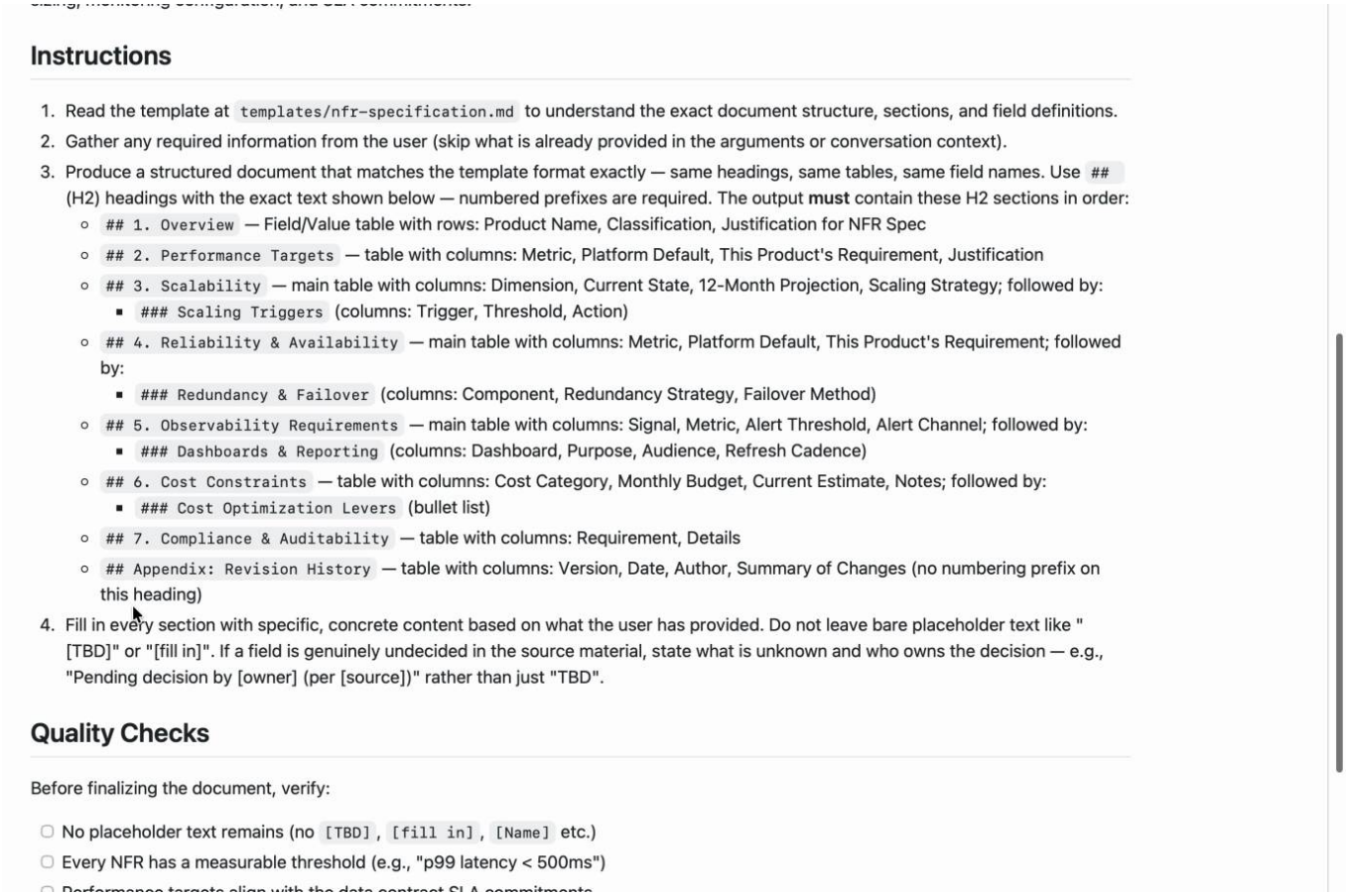


Figure 21: DPLC skills



Instructions

1. Read the template at `templates/nfr-specification.md` to understand the exact document structure, sections, and field definitions.
2. Gather any required information from the user (skip what is already provided in the arguments or conversation context).
3. Produce a structured document that matches the template format exactly — same headings, same tables, same field names. Use `##` (H2) headings with the exact text shown below — numbered prefixes are required. The output **must** contain these H2 sections in order:
   - `## 1. Overview` — Field/Value table with rows: Product Name, Classification, Justification for NFR Spec
   - `## 2. Performance Targets` — table with columns: Metric, Platform Default, This Product's Requirement, Justification
   - `## 3. Scalability` — main table with columns: Dimension, Current State, 12-Month Projection, Scaling Strategy; followed by:
     - `### Scaling Triggers` (columns: Trigger, Threshold, Action)
   - `## 4. Reliability & Availability` — main table with columns: Metric, Platform Default, This Product's Requirement; followed by:
     - `### Redundancy & Failover` (columns: Component, Redundancy Strategy, Failover Method)
   - `## 5. Observability Requirements` — main table with columns: Signal, Metric, Alert Threshold, Alert Channel; followed by:
     - `### Dashboards & Reporting` (columns: Dashboard, Purpose, Audience, Refresh Cadence)
   - `## 6. Cost Constraints` — table with columns: Cost Category, Monthly Budget, Current Estimate, Notes; followed by:
     - `### Cost Optimization Levers` (bullet list)
   - `## 7. Compliance & Auditability` — table with columns: Requirement, Details
   - `## Appendix: Revision History` — table with columns: Version, Date, Author, Summary of Changes (no numbering prefix on this heading)
4. Fill in every section with specific, concrete content based on what the user has provided. Do not leave bare placeholder text like "[TBD]" or "[fill in]". If a field is genuinely undecided in the source material, state what is unknown and who owns the decision — e.g., "Pending decision by [owner] (per [source])" rather than just "TBD".

Quality Checks

Before finalizing the document, verify:

- ☐ No placeholder text remains (no `[TBD]`, `[fill in]`, `[Name]` etc.)
- ☐ Every NFR has a measurable threshold (e.g., "p99 latency < 500ms")

Figure 22: Inside a SKILL.md

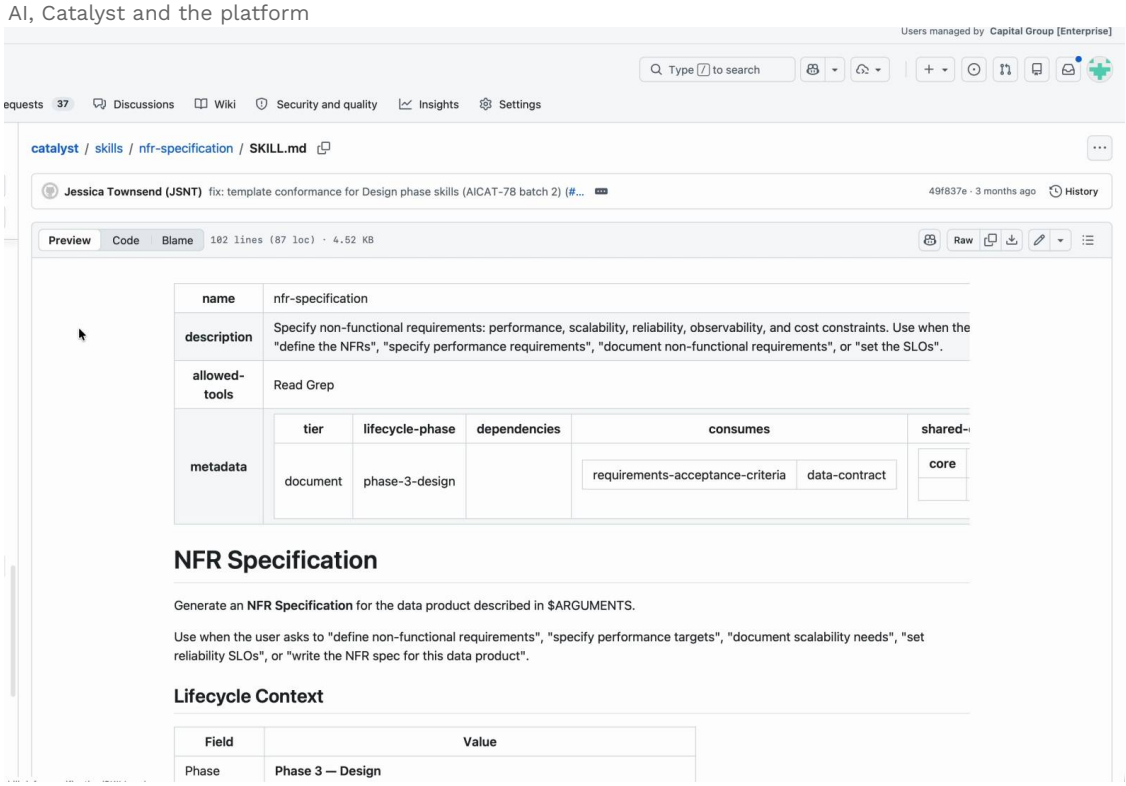


Figure 23: Example: NFR specification

On screen: The DPLC skills map

The lifecycle map shows the stages, the phases inside them and what each phase hands to the next. The stages on screen are Define, Deliver and Sustain. The skill IDs say design, build and ship, so search for those names when you look for a skill.

The later screens show why the design documents still matter after the build. Validate, Publish, Operate and Evolve each need someone responsible and something to check against. The skills follow RFDs, the standards documents, and each RFD records whether it is a draft or published. When a screen or a code comment disagrees with the published RFD, follow the RFD.

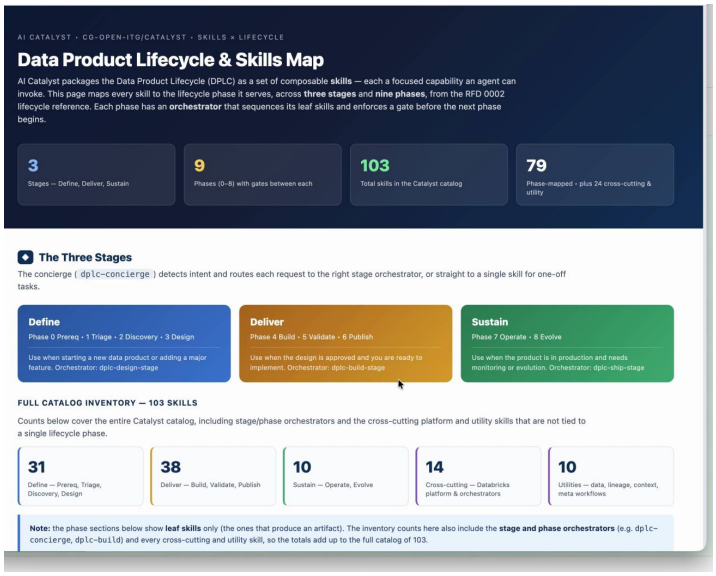


Figure 24: Three stages

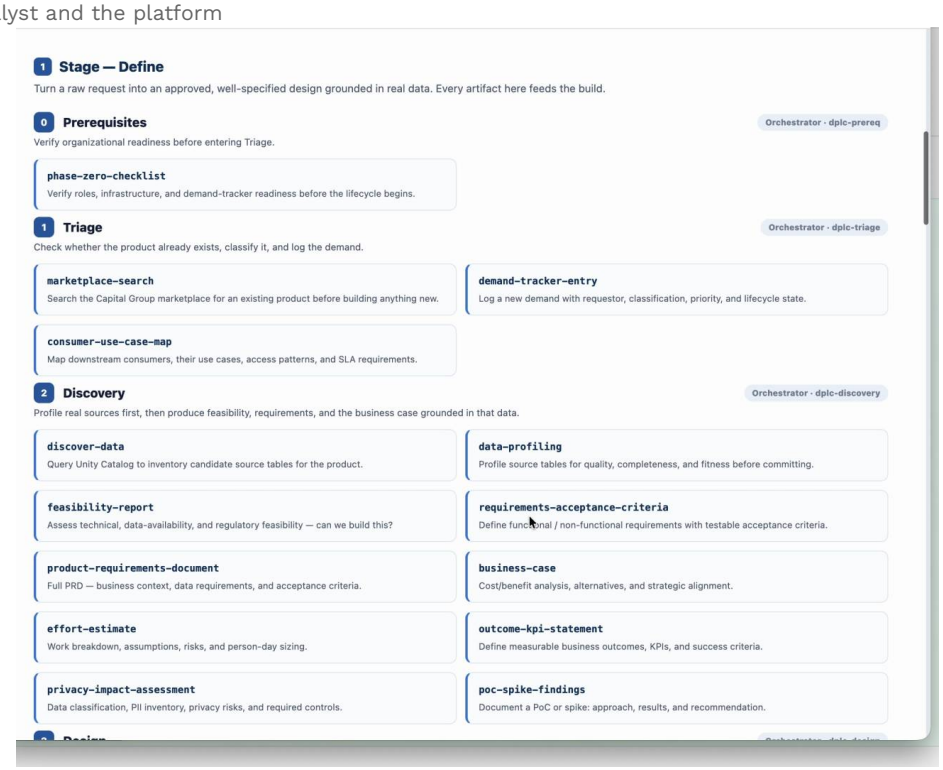


Figure 25: Define

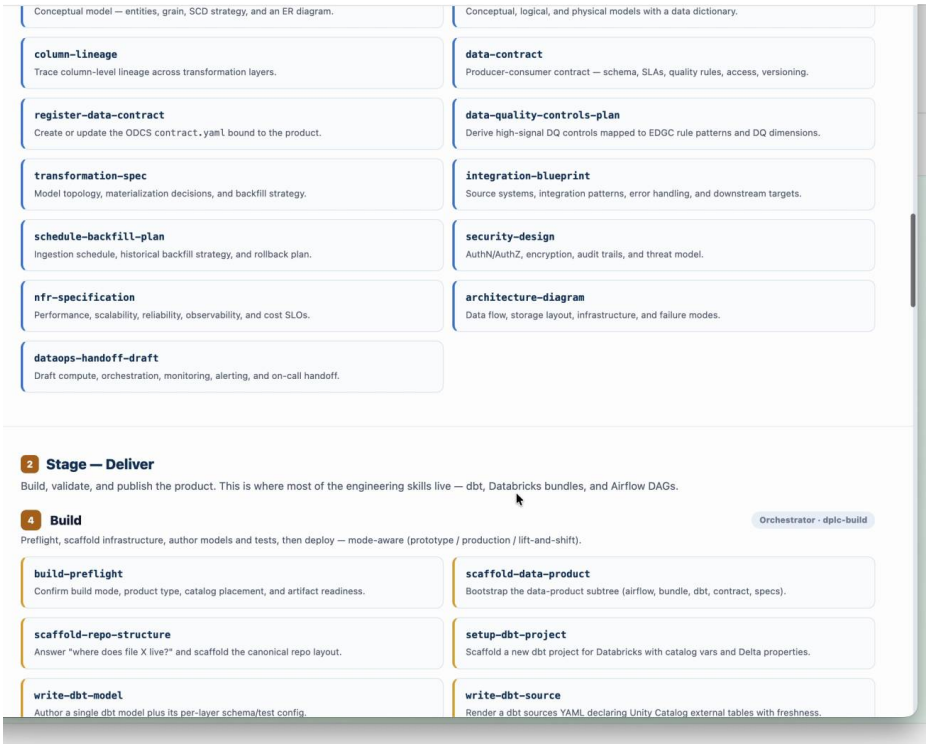


Figure 26: Deliver: build

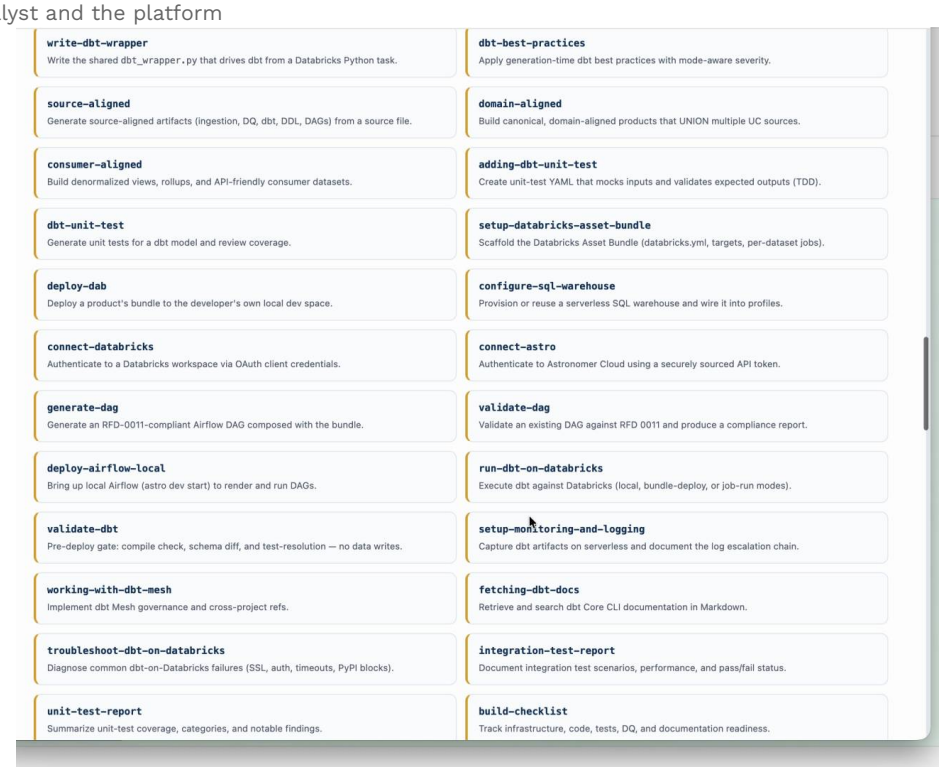


Figure 27: Build skills

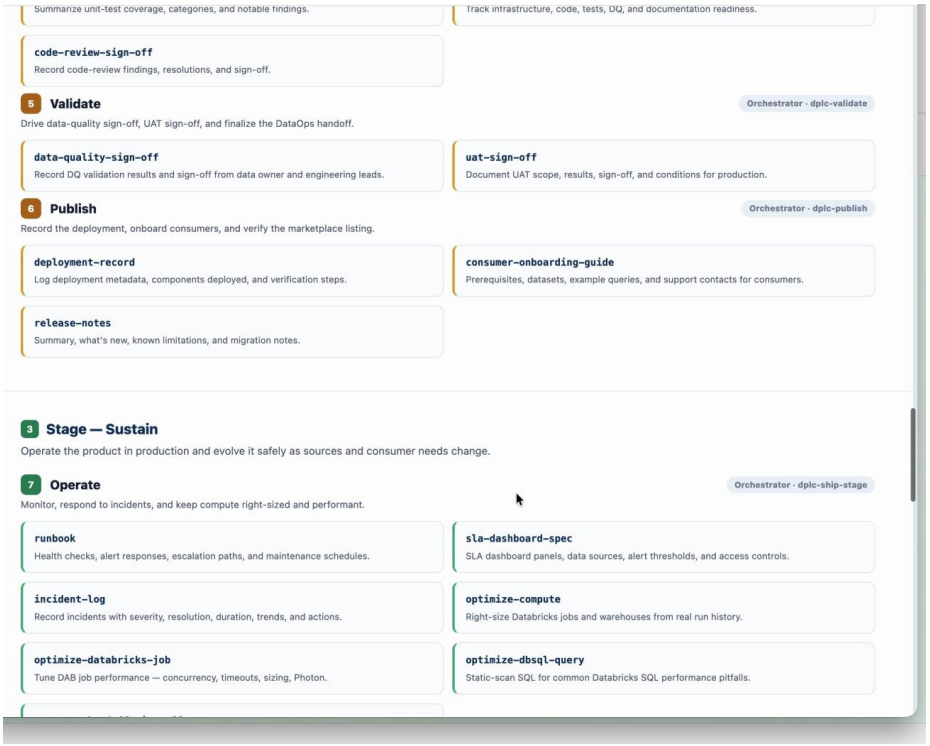


Figure 28: Validate, publish, sustain

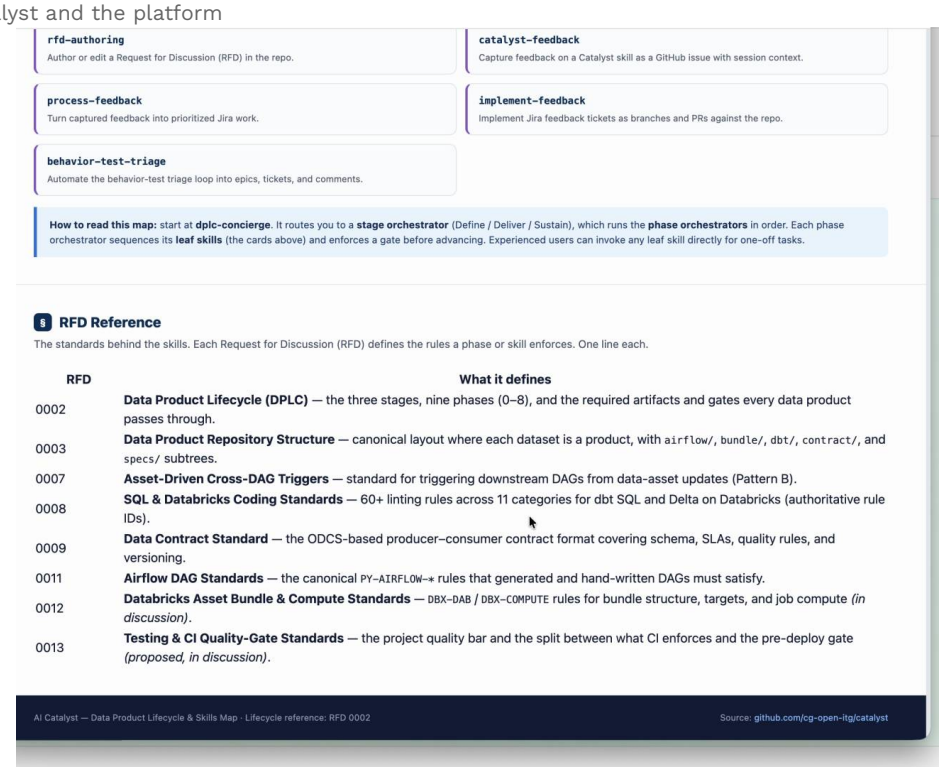


Figure 29: RFD reference

Platform decisions: why the paved path works this way

Close Tuesday by following a dataset through the platforms you have just met. The names are familiar now; the next question is why the platforms were selected and how their boundaries affect your work. By the end, you should be able to explain a cross-team dependency, locate the artefacts that define a run, and show why quality checks belong before shared publication.

THIS SESSION ANSWERS

- Why retain Airflow while changing who operates it?
- What belongs in Unity Catalog, Databricks, dbt and the dataset folder?
- How does checked application data become a shared domain dataset?
- Which promises need evidence or a platform decision before Wednesday's build?

Knowledge check: Before following the pipeline

Your team has a repository, a green deployment and an Airflow DAG. Another team says its source data is ready. Which of those facts establishes that your consumer can read a correct, current output? Name one additional piece of evidence you would request and the person who should provide it.

This architecture case illustrates team-owned orchestration, governed storage and checks before shared publication. Internal service names and example arrangements are not an installation guide. Use the assigned repository and current enablement guidance to establish the supported configuration. The asset-reference names, times and counts below are illustrative.

The 30-minute closing session uses five minutes for platform choices and ownership, five for the repository and runtime, five for publication, twelve for the group review, and three for the handoff. The history, migration and versioning explanations provide reading depth around that discussion.

Start with the operating problem

A shared Airflow estate can contain thousands of DAGs from many teams. Coordinating configuration, upgrades and remediation at that scale creates dependencies beyond any one product.

A team may need to deploy tomorrow while a vulnerability prevents its release. If the fix depends on a central backlog, that team's schedule depends on work it cannot prioritise. An expensive pipeline can also consume capacity needed by other products. Federation addresses ownership and isolation, but there are two different boundaries here. Separating Airflow deployments changes orchestration ownership; protecting Databricks compute also requires appropriate job, capacity and workload configuration.

The paved-path response is to retain common conventions while giving teams responsibility for their deployments. A blueprint can help teams start consistently. It cannot decide whether a source is fit for purpose, whether a release is compatible, or whether a consumer should trust yesterday's result. Those remain product decisions with named owners.

Operating problem	*Design response*	*What the creator still needs to establish*
Many teams depend on one Airflow estate	Team-owned deployments on Astronomer	Who owns upgrades, dependency fixes and incidents for this deployment
A deployment is blocked by a vulnerability	Vended runtime and guidance with team responsibility	Which package is affected, who repairs it, and what evidence permits release
Shared workloads compete for resources	Federation and more local control	Whether the execution resources are isolated and appropriately sized
A new team faces repeated setup work	Vended repository, blueprints and platform connections	Which parts were provisioned, which work, and which access checks remain
Consumers can see data before quality checks finish	Check in application space before domain publication	Which candidate was checked and whether publication of that candidate succeeded

Why Airflow and Astronomer were retained

Airflow already supported ingestion and curation dependencies. Keeping it preserves an orchestration model teams know while changing how the service is operated. Astronomer is described as the choice for vendor support, local development with its CLI, and disaster-recovery arrangements. The decision is about operating Airflow as well as expressing DAGs.

Compare three operating approaches. Terraform modules could give each team an Airflow installation on EKS, but teams would still need an operating and support model. AWS Managed Workflows for Apache Airflow, or MWAA, was considered with concerns about support, executor choices and recovery at the time. Astronomer was selected for the combination the team wanted. Assess each service against current support and recovery requirements.

Approach considered	*Operating consideration*	*Question to ask when assessing the decision*
Share Terraform modules for self-managed Airflow	Repeatable infrastructure and separate installations	Who supports, patches, restores and upgrades each installation?
Use AWS MWAA	A managed AWS Airflow service	Do the supported configuration, recovery and support arrangements meet the requirement?
Use Astronomer	Airflow-focused support, CLI workflow and recovery arrangements	What is covered by the service, and what remains the team's responsibility?

In this example, workspaces group deployments organisationally, and deployments are owned at team level. A project might justify another deployment, but one deployment per dataset is not the default. Ask who should be able to remediate a runtime problem without negotiating another team's backlog. That is a stronger basis for choosing the boundary than counting how many DAGs a product contains.

Disaster recovery also needs a concrete question: what must be restored, how quickly, and how much loss is acceptable? Vendor support or a recovery feature does not establish that a particular product has a tested restoration plan. Thursday develops run and recovery evidence; Tuesday establishes who owns that question.

Cross-team dependencies without shared runtime access

Consider an asset-reference product that depends on another team's instrument data. The consumer needs to know when the agreed input is available. It does not need broad administrative access to that team's Airflow deployment.

In the example architecture, a custom bridge connects Airflow asset events across deployments. The producer emits an update, its bridge sends a readiness message to Kafka, and the consuming deployment's bridge receives the message and updates its local scheduling state. Each team's DAG can then express a dependency through the supplied integration. Kafka carries the notification; the table is consumed through its governed data-access route.

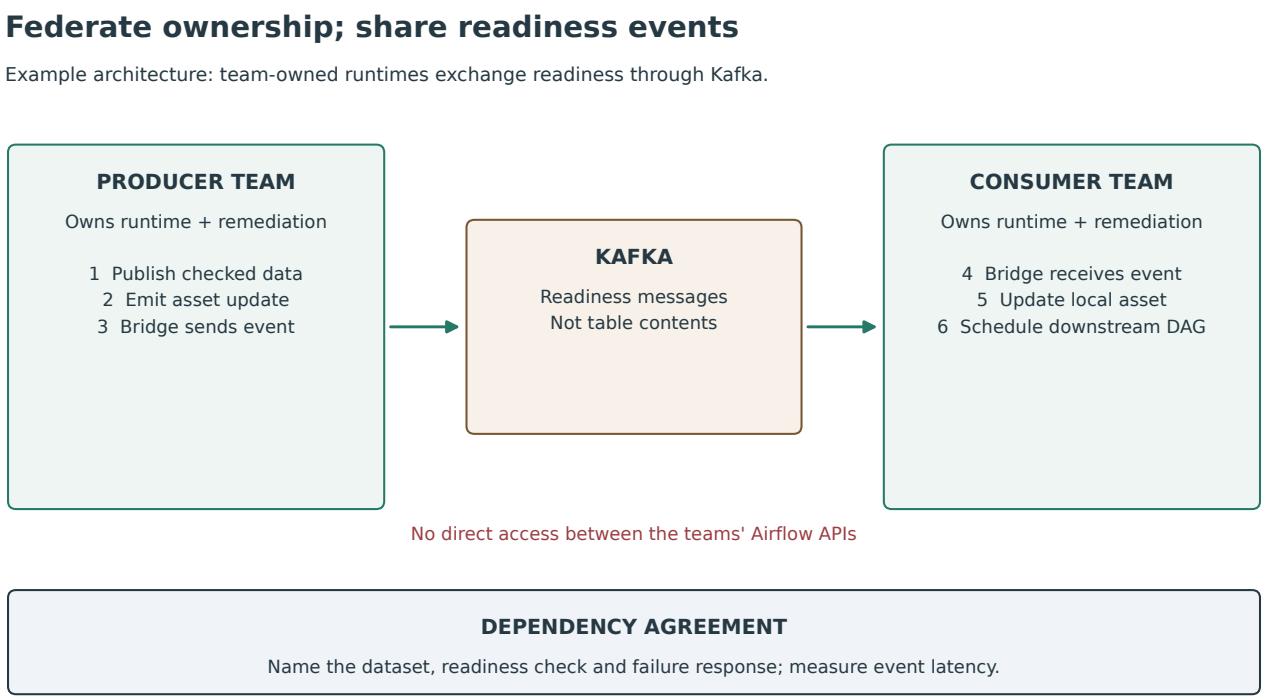


Figure 30: Two team-owned Airflow deployments exchange readiness messages through Kafka: the producer publishes checked data and an asset update; the consumer bridge receives the message, updates a local asset and schedules its DAG. The teams do not call each other's Airflow APIs.

Numbered steps show readiness-message flow. The message identifies an update; consumers read the data through a separate governed route.

The creator's job is to identify the correct dependency and its meaning. Does "ready" mean that an upstream task ended, that a table was written, or that the agreed checks passed and the shared release is available? These are different facts. A usable agreement identifies the dataset, expected partition or

business date, producer, quality boundary and failure response. Access to the event also does not grant access to the data.

Measure event propagation before promising a latency target. The end-to-end deadline includes arrival, upstream execution, checks, publication and downstream execution. For a late input, agree whether to wait, escalate, or serve a previous release with an explicit freshness status.

Airflow's terms vary by runtime version: older versions use "dataset", while newer documentation uses "asset". Native asset-aware scheduling describes scheduling on asset updates. The Kafka bridge and cross-deployment arrangements here are the example platform design; they are not implied by the native feature alone.

Separate governance, execution and physical storage

The Databricks story begins with notebooks and workspace-local metadata. Copying a dataset to another workspace made sharing easier in one sense, but risked divergent access rules and duplicated data. Unity Catalog is introduced as a common governance and metadata layer.

Keep three responsibilities distinct. Databricks compute executes queries and transformations. Unity Catalog describes and governs the registered data objects. S3 holds the underlying files in this architecture. A table's qualified name has three levels: `catalog.schema.table`. For example, `domain_assets.reference.asset_reference` is an illustrative governed name, not a supplied production destination. Databricks' Unity Catalog overview explains the namespace and governance model.

This example uses three catalog roles:

Catalog role	*Purpose*	*Asset-reference example*
Source-aligned	Data organised around the producing source	Instrument records with producer identifiers and source meaning
Application	Internal working area for the application or product	Candidate asset mappings and outputs awaiting quality checks
Domain	Curated shared data aligned to a domain or consumer need	Checked asset-reference output with agreed meaning and access

These are design roles, rather than built-in Unity Catalog object types. Application data is described as restricted from sharing in that arrangement. Domain publication changes who may consume a dataset and what they may reasonably rely on. It needs an agreement about population, grain, units, freshness and support, alongside grants and catalog placement.

A dependency may be in a transitional catalog. Finding useful data there does not prove that it will remain the long-term dependency. Record both the current source identity and the intended migration owner before promising a consumer a stable location.

Delta and UniForm: metadata is different from a publication copy

The example uses Delta tables. A Delta table combines data files with transaction metadata; the metadata enables the engine to identify the relevant table state. UniForm provides a way for other engines to read the data using Iceberg-compatible metadata.

More precisely, UniForm generates Iceberg metadata alongside Delta metadata, using the same underlying Parquet files. Generation can be asynchronous, and supported readers and table features matter. It is not two independently written copies of the data. See Databricks' Iceberg-read documentation.

This distinction matters later: **UniForm adds a compatible metadata representation; deep-clone publication copies data to another table.** A consumer using a different engine still needs an agreed access route, compatible configuration and evidence about the table version it can actually read.

Why dbt sits over Databricks compute

dbt lets creators express transformations in SQL, reuse macros and keep models close to tests and documentation. It reduces some authoring complexity while Databricks remains the execution platform.

dbt defines the transformations and sends work through its configured adapter. On the Databricks route, the selected Databricks compute executes that work. Installing dbt locally is useful for development; it does not make the remote dataset or its permissions local. In this course, offline parse and CSV checks support the assigned Databricks execution; they do not replace it.

Bitemporal data supports being able to ask what was true for a business date and what the system knew at another time. Consider an instrument classification effective on Monday but corrected on Thursday. A report reconstructed with Tuesday's knowledge can differ from one reconstructed after Thursday's correction.

Time question	*What it means in the example*	*Design question for the creator*
Business validity	When was this classification effective?	Which producer field and rule establish that period?
System knowledge	When did our system record this classification or correction?	How are changes captured and previous knowledge retained?
Reconstructed result	What would we have reported with the knowledge available then?	Which business date and knowledge cutoff does the consumer require?

Using dbt or Delta does not automatically create that business model. dbt snapshots capture changes for historical tracking; a complete bitemporal design still needs explicit validity and knowledge rules. Carry those requirements into Wednesday's grain and mapping discussion before choosing history columns.

What the vended repository gives you

Vending is described as supplying an Airflow deployment, Databricks workspace arrangements and a repository that deploys to those platforms. The repository contains conventions as well as code: SQLFluff, pre-commit configuration, tool setup, blueprints and shared components. Their presence helps teams begin consistently; a generated scaffold still needs review against the product requirements.

The CG capstone keeps design, models, orchestration and evidence in one product folder. This abbreviated tree shows the supplied project layout; platform deployment demonstrations may use other conventions.

```
data_products/YOUR_PRODUCT_NAME/
├── dbt_project.yml
├── contracts/product-research.odcs.yaml
├── docs/PRD.md
├── docs/STTM.csv
├── models/phase1/src_cg__product.sql
├── models/phase1/dom_cg__product_cost.sql
├── orchestration/
├── reports/REPORT.md
└── evidence/
```

An **Airflow blueprint** is a reusable implementation selected and parameterised by configuration. YAML makes the declaration easier to inspect; Python and dependencies still implement the behaviour underneath. A supported default avoids repeated plumbing. A custom blueprint introduces code, maintenance and review obligations. Changing a default connection can also change the workspace or identity that runs the job, so it is a meaningful design change.

A **Databricks bundle** defines deployable resources and configuration, including the job that executes the transformation. The supplied repository uses Databricks Asset Bundles, or DABs; current documentation

calls them Declarative Automation Bundles. The supplied repository remains the authority for filenames and supported configuration in this course.

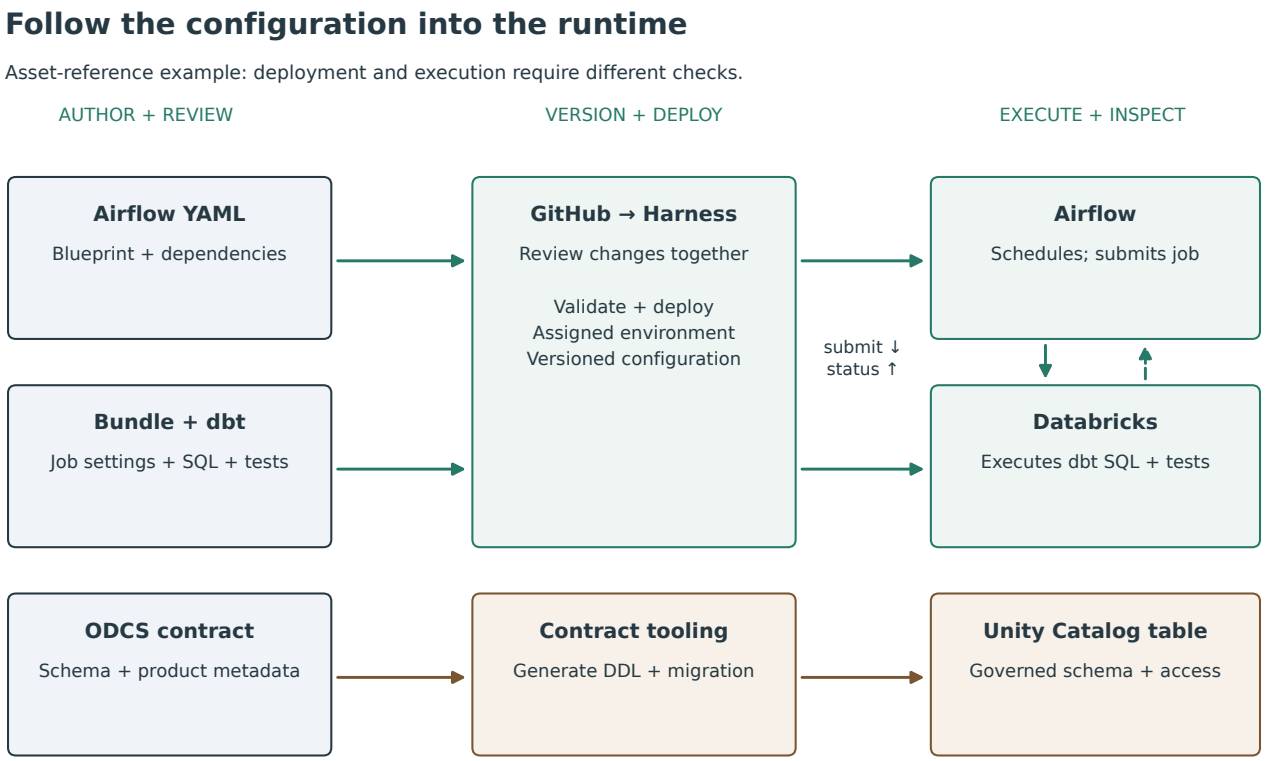


Figure 31: Airflow YAML, bundle and dbt code, and the ODCS contract are authored together. GitHub review and Harness deployment deliver the orchestration and execution configuration. Airflow submits work to Databricks and receives status; contract tooling generates DDL and migrations for Unity Catalog tables.

Figure: a dataset's artefacts have different responsibilities. Teal arrows show deployment and run control; brown arrows show contract-driven schema changes. A dashed return arrow represents job status.

For this job pattern, there are two linked declarations: the DAG specifies dependencies and when to trigger, and the Databricks job specifies what runs. Airflow submits and monitors the job; Databricks executes the transformation. This example does not require every Airflow task to run remotely. Thursday's local practice and other Airflow operators have their own execution behaviour.

Artefact	*Question it should answer*	*Evidence to inspect*
Airflow declaration and selected blueprint	What must be ready, and which job is triggered?	Dependency definition, connection and observed task/run state
Bundle job configuration	Which target and command execute?	Assigned environment, job parameters and actual job result
dbt SQL and model metadata	How are records mapped and checked?	Grain, joins, tests and independent output checks
ODCS contract	What is promised about meaning, schema and support?	Reviewed fields, ownership, quality expectations and version
Generated migration	What changes in the governed table?	Proposed DDL, impact and observed schema
Documentation	How should a consumer use the product or an operator diagnose it?	Meaning, limitations, support route and recovery guidance

Catalyst can help generate these artefacts. Check the output against the source schema, selected blueprint, job configuration and actual run. A polished YAML file does not establish that a referenced connection, job or dependency exists.

Observability gives you somewhere to look

Monitoring must be accessible to the people responsible for the product. Confirm the assigned logging destination and permissions before an incident.

Use that operational view to connect a symptom to a particular deployment, DAG run, Databricks job and candidate. An upstream event received, a job submitted, a transformation completed, a quality check passed and a release published are separate milestones. Logs help explain where progress stopped; the consumer still needs the resulting business date, meaning and freshness. Verify the assigned logging destination and access before relying on it during an incident. The course's local log evidence has its own scope and does not establish a connected Datadog integration.

Contracts connect the design to schema changes

DDL, or data definition language, describes structures such as tables and columns. It cannot by itself express the complete consumer agreement. An ODCS contract holds schema alongside sensitivity, support ownership and other metadata. Use the platform's supported conventions within that standard.

Contract tooling can generate DDL and migrations from a reviewed ODCS schema. Inspect the supplied repository tooling and the proposed migration before applying it; source prose is not a CLI command.

Follow the chain from reviewed contract to generated schema change to the actual governed table. If the contract says a field is required but the deployed schema or transformation permits missing values, you have found a mismatch that a successful deployment may not reveal. Likewise, a support-group name entered in a contract must correspond to an accountable operating arrangement.

A metadata file in Git does not establish that the marketplace can discover its contents. Verify registration and the consumer-visible listing. Product documentation explains meaning and use; platform documentation explains how to build. DAG documentation can provide an additional operational view.

Worked change: carry a currency name through the product

A consumer of the AED listing product requests `currency_name_lower` alongside the existing currency code. The existing interface identifies eligible listings at the agreed listing grain. This request changes the interface without changing that population. Treat it as one reviewed product change spanning requirements, contract, transformation, tests, migration and consumer use. A generated column is useful only when those artefacts describe the same meaning.

Begin with a concrete acceptance example: an eligible listing whose currency code is `AED` should carry `uae dirham` when its authoritative English currency name is `UAE dirham`. Lowercasing `AED` produces `aed`, which is a code, not a name. Neither a plausible expansion invented by an assistant nor an unreviewed lookup can establish the authoritative name. Locate the governed reference source, its owner, language and join key before accepting the derivation.

1. Requirement and consumer agreement. Record the request in the PRD or maintained requirement: an English lower-case display name for the listing's currency, derived from the authorised currency-reference name. Agree whether surrounding spaces are meaningful and what happens when a code has no matched name. In this worked case, retain the eligible listing and publish a null name rather than inventing one. State that consumers must continue to use the currency code for identity and calculations. Name matching is a display concern; it is not a new currency conversion rule. The acceptance examples must include missing reference data and must preserve the existing listing population.

2. Contract. Add `currency_name_lower` to the reviewed ODCS schema with the supported string type, nullable status, description, authoritative origin and lower-case meaning. Keep the existing key, grain, classification, ownership and service promises explicit. Record the reference-data coverage rule agreed with the consumer rather than assuming that a nullable field needs no quality control. Under a policy that treats nullable additions as compatible, this is a candidate minor version change. Check actual consumers before settling that decision: a strict schema validator or positional export can still fail when a column appears. A version label records the agreement; it does not provision a parallel interface.

3. Integration model. Join the listing's currency code to the approved currency reference and derive the field from the reference name. Confirm that the reference contributes at most one eligible row per currency code under the chosen date policy. If the reference is historical, define which business date selects its name; an unrestricted join to all versions can multiply listings. A left join implements the agreed retention of listings without matched names. For the chosen no-trimming rule, the derivation is conceptually `lower(currency_name)`; use the actual project column and supported SQL conventions. Lowercasing must occur after the authoritative name has been selected, not after a guessed mapping from the code.

4. Final model and consumer documentation. Project the new field explicitly into the final interface, alongside the unchanged currency code and listing identifiers. Inspect the output model and published interface: adding a field to an intermediate model does not make it visible to a consumer. Document the null meaning, language and source. Check that the DAG or bundle still selects the intended model graph, and that the candidate-to-domain publication step carries the additional column. This request needs no new schedule merely because its schema changed. Existing access groups and discovery metadata must continue to describe the actual published product.

5. Tests and independent examples. Use a small fixture whose expected results are written before inspecting generated SQL. An active AED listing with the reference name `UAE dirham` produces `uae dirham`; an eligible listing without a reference name retains its key and has null; an inactive AED listing remains excluded. Include an instrument with two distinct eligible listing IDs: both survive, proving that the change preserves listing grain rather than collapsing to instrument grain. Add a reference-key uniqueness check and compare candidate key sets before and after the change. Compilation checks references and syntax; successful execution checks runtime viability; the fixture and key comparison establish the requested meaning and population. A lower-case check alone would accept the erroneous value `aed`.

6. Migration, historical rows and release. In the client repository, commit the reviewed ODCS contract change: CI is the canonical SQL generator, and the Harness bot owns commits under `flyway/migrations/`. Review its generated additive DDL, exact target, type, nullability and dependencies. Local SQL under gitignored `.local/migrations/` is throwaway review output, not an artefact to commit in place of CI-managed migrations. Preserve applied migration history through the supported workflow; do not manually

rewrite deployed history to make it resemble a fresh installation. The client tooling README defines this ownership boundary. Adding a nullable column changes structure but can leave existing rows null. If the consumer needs historical names, specify a bounded backfill, the authoritative name history available for those dates, reconciliation checks and recovery before promotion. Today's reference name must not silently stand in for unavailable historical truth. DDL review and a backfill plan are separate evidence.

Review these changes together on a short-lived branch from `main`. The reviewed commit can move through environments according to the deployment policy, with checks at each target. Local compilation, fixture review and a private candidate do not establish that a production migration or consumer transition has occurred. Production readiness also requires the approved migration, any required backfill, candidate quality evidence, publication and the affected consumers' compatibility checks. The capstone does not require applying these migrations.

REVIEW — finite change assessment. Using only this case and the supplied repository conventions, write six short review comments: one each for requirement, contract, integration, final interface, tests and migration/release. Identify the mistake if the generated integration uses `lower(currency_code)`, the missing evidence if a reference join doubles a listing, and the uncompleted work if DDL adds the column but historical rows remain null. Decide which consumers need a compatibility check before a minor-version release. Your result is a local review note, with no new live commands or deployment required. A complete review connects the expected `uae dirham` value to its source, preserves the listing keys and separates schema readiness from historical-data readiness.

Quality checks belong before domain publication

Publication creates a critical quality boundary. If a pipeline writes to a shared table and checks quality afterwards, a consumer can read the bad data during the gap. A failed check cannot undo a decision already made from it.

The example writes a candidate into the application catalog, runs quality checks there, and publishes to the domain catalog only after those checks pass. Publication uses a deep-clone step. Databricks documents deep cloning as copying data and metadata; a shallow clone references source data files instead. The extra copy here is a deliberate cost of separating private preparation from shared consumption.

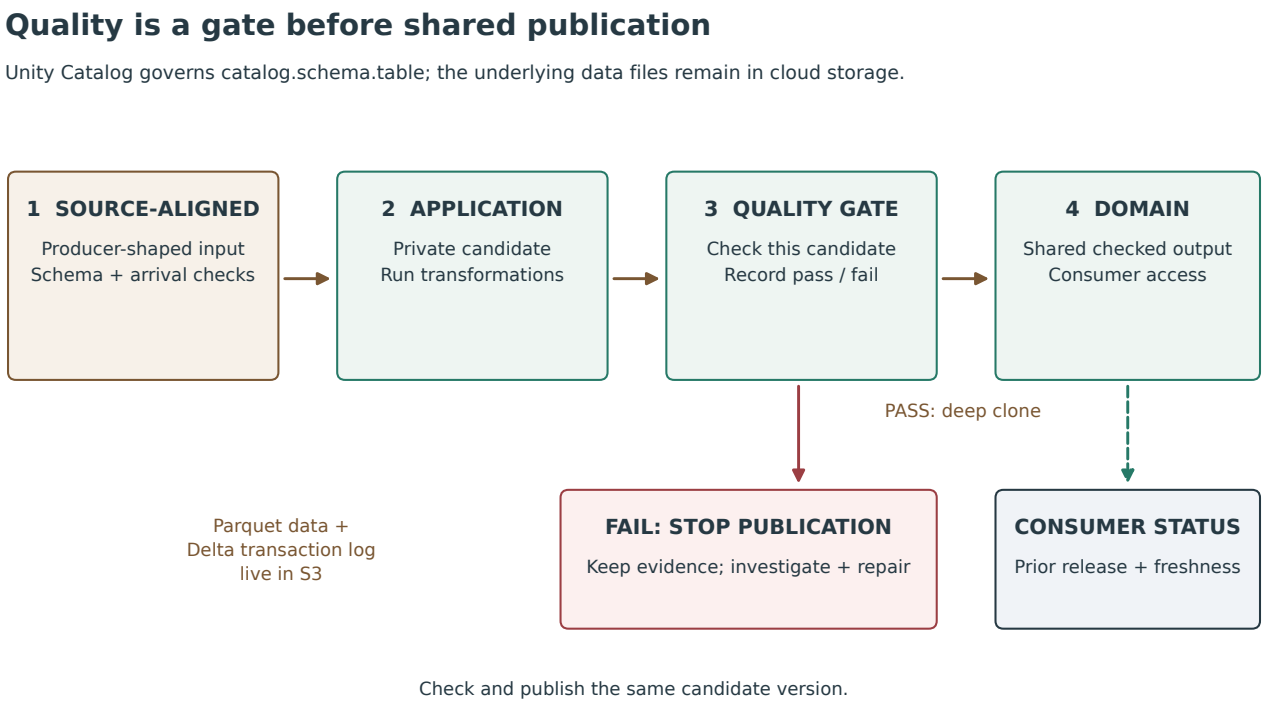


Figure 32: Source-aligned input is transformed into a private application candidate. Quality checks gate publication: a pass allows deep cloning into the shared domain table; a failure stops publication and keeps diagnostic evidence. Consumers need the published release identity and freshness status.

Figure: a quality failure blocks the candidate's publication. Retaining the previous release and communicating its freshness are explicit product decisions.

The gate must protect **the candidate that is actually published**. Checking one table version and cloning a later, unchecked version breaks the reasoning. A creator should be able to identify the candidate, its checks, the publication result and the resulting consumer-visible release. These are requirements to verify with the platform pattern, not additional learner infrastructure tasks.

Use a small asset-reference example. The agreed grain is one record per `instrument_id` for the business date, and `asset_class` must come from an accepted set. The candidate contains 500 records but only 497 distinct identifiers. The write can succeed while the product fails its uniqueness requirement. The quality step must block publication of that candidate.

Stage	*Illustrative observation*	*Decision*
Prepare candidate	500 rows written to application space	Transformation completed; candidate needs checks
Check candidate	497 distinct IDs; three surplus rows violate the grain	Reject this candidate and retain diagnostics
Attempt publication	Required quality check failed	Prevent this candidate from becoming the shared release
Communicate status	Prior release remains; today's deadline may be missed	State the last successful business date and agreed failure response
Repair and rerun	Correct the join, then check the new candidate	Publish only after fresh checks and successful publication

Use the quality framework supported in the assigned environment and verify that a failed required rule blocks publication. A rule library cannot protect consumers when the DAG publishes despite its failure.

Publishing also does not solve all quality questions. A dataset can be unique and complete while using the wrong classification rule or excluding a consumer's intended population. Monday's agreement supplies the acceptance examples; Wednesday's model and tests make them executable; Thursday's orchestration preserves the order.

Migration preserves dependencies and consumer agreements

Plan a migration in observable steps. Ingestion first changes orchestrator while retaining the existing execution stack; later work moves execution and publication towards Databricks and source-aligned catalogs. Separating those changes reduces the number of moving parts to diagnose at once.

An upstream pipeline may remain in Caspian while its downstream product moves to Astronomer. A supported dependency-check blueprint can provide an interim way to inspect legacy DAG completion through an authorised API route. That transitional route has a different access boundary from the federated event design. Neither is an invitation to grant every deployment access to every other one.

A migration may preserve a virtual-dataset interface while changing its underlying physical source. Verify the queries, schema, values and access rather than assuming the unchanged interface makes the change transparent.

Migration boundary	*What might change*	*What needs consumer-facing verification*
Orchestrator	Where the DAG runs and how dependencies arrive	Scheduling, failure behaviour, ownership and readiness timing
Execution engine	Where transformations execute	Equivalent grain, calculations, data types and history
Governed location	Catalog, table identity and access configuration	Permissions, discovery and the promised dataset
Serving reference	Physical source behind an existing virtual interface	Queries, schema, results, freshness and authentication

An unchanged view name is useful, but it does not establish equivalent results or access. Check representative consumer queries and reconcile outputs before calling a migration transparent. An enablement or architecture conversation is also needed when the use case does not fit the supplied dependency, execution or serving patterns. Vending supplies a starting point after onboarding; it does not remove the use-case assessment.

Versioning and roadmap questions need separate answers

Parallel versions require a supported design. A contract version records a change; it does not automatically create two independently usable datasets. Altering a table can still break a consumer using the previous schema or meaning.

Suppose the producer changes `asset_class` from a descriptive label to a code. Four version questions now arise: which code revision ran, which contract described it, which schema was deployed, and which data release the consumer read. Aligning those identities helps explain a result. Incrementing only the contract version does not preserve the earlier interface.

With trunk-based development, short-lived reviewed changes can flow into a shared main line; deployment to each environment and activation for consumers remain separate concerns. Feature flags or parallel branches alone do not establish consumer compatibility. Do not promise dual-version support because a branch or flag exists.

Keep decisions, observed capabilities and unresolved requirements distinct. Latency needs measurements; parallel versions need a supported coexistence, activation and retirement strategy; new service identities need approved permissions and working authentication.

Group review: can this asset-reference dataset be published?

Review the platform handoff

DURATION 12 minutes
OUTPUTS One annotated execution sequence, three rejected claims and an owned handoff
HANDOUT Platform decisions worksheet

Work in pairs with the platform decisions worksheet. One person traces the run; the other checks what each piece of evidence establishes. This is an architecture and product review using the supplied facts.
The instrument-data producer has published a readiness event. Your repository change has deployed successfully. Airflow submitted the Databricks job, and the job wrote 500 candidate records in application space. The uniqueness check reports 497 distinct instrument IDs. A draft release note says: "The deployment is green, so today's domain data is available. Events arrive within ten seconds. Version 2 of the contract lets version 1 consumers keep their existing schema."

1. Draw the path from the upstream event to the consumer, naming where scheduling, execution, checks and publication happen.
2. Correct each of the release note's three claims using the supplied architecture and scenario evidence.
3. Identify what happens after the failed check. State what you would tell the consumer about the last successful release and today's deadline.
4. Assign the next action for the model defect, the unmeasured event promise and the proposed contract change. Explain how Catalyst could help without deciding those facts.

A strong review names the failed candidate and the blocked publication. It also gives the consumer a useful status, rather than treating yesterday's table as evidence that today's promise was met. Use the conceptual answer key after your pair has agreed its reasoning.

End-of-day handoff

Add one page to your reviewed Tuesday artefacts. Identify the dataset and its grain, upstream dependency, owning team, supported execution pattern, quality gate, intended publication location and unresolved questions. Mark each environment result as observed, unavailable or still to check.

Learning Review

Retrieve: explain why federation changes operating ownership, why Airflow and Databricks have different jobs, and why Unity Catalog is separate from S3 storage.
Apply: trace the failed asset-reference candidate through its repository artefacts, execution and blocked publication. Explain why a successful deployment, an event or an incremented contract version does not establish the consumer's promised output.
Self-check: your handoff identifies who fixes the model, who owns the runtime, which candidate must pass checks, and what the consumer hears if the new release is late. Unmeasured latency, migration assumptions and version-coexistence questions have owners rather than invented answers.

Wednesday turns those grain and mapping decisions into dbt models and tests. Thursday examines orchestration, failure and replay. Keep this platform map beside the design: it tells you where each piece of evidence should come from. The dbt guide and Airflow and Astronomer guide provide further technical practice.

Solution: page 97

Lifecycle, roles and warranty

DPLC phases and gates. The DPLC groups its phases into Define (0–3), Deliver (4–6) and Sustain (7–8).

Phase	*Exit gate or hand-off*	*Who decides*
0 Pre-requisite	Ready for Triage? Data Owner, DPM and Creator assigned; real Demand Tracker entry and capacity confirmed.	DPM, or technical-lead fallback.
1 Triage	Worth Pursuing? Classification approved and consumer demand recorded.	Data Owner.
2 Discovery	Accepted, Feasible and Capacity? Sources profiled and requirements feasible.	Data Steward checks completeness; Data Owner decides.
3 Design	Design and Contract Approved? Contract, model, STTM and any conditional architecture or security work accepted.	Technical lead; Data Architect for a new pattern; InfoSec when sensitivity or regulation triggers review.
4 Build	Code Review and CI Passed?	Reviewer through CODEOWNERS.
5 Validate	UAT Passed and Accepted? DQ, UAT, NFR and operating hand-off complete.	Data Owner and engineering lead for DQ; Data Steward for UAT.
6 Publish	Production Validated? Marketplace listing is live, validated, the correct entity type and ancestry, and at least one consumer confirms access.	Domain team lead through Harness Approval.
7 Operate	Warranty hand-off accepted? Development support transfers after day 30.	DataOps accepts; Data Domain Lead arbitrates disputes.
8 Evolve	Deprecate? Continue, improve or deprecate.	Phase 8 review.

Roles.

Role	*Accountable for, and when*
Data Owner	Domain classification, Phase 1 approval and PIA prerequisite sign-off; Phases 0–2.
Data Steward	Request-completeness and UAT sign-off; Phases 2 and 5.
Data Product Manager (DPM)	Framing, requirements, contract, priority and acceptance criteria; Phase 0 onwards.
Data Product Creator (DPC)	Pipeline and model implementation across the alignment chain; Design through Validate.
DataOps team	Operational SLAs and steady-state operation after warranty; Phase 7 onwards.
Data Architect	Approval when a design introduces a new architecture pattern; conditional Phase 3 gate.
Data Domain Lead	Domain governance, role assignment and warranty-dispute arbitration; cross-cutting.
Platform team	Shared libraries, templates and infrastructure; cross-cutting.
Domain team	Domain repository and producer/consumer work; cross-cutting.
Domain team lead	Production promotion through Harness Approval; Phase 6.
InfoSec	Security-design approval when sensitivity or regulation triggers the Phase 3 gate.

Warranty and DataOps hand-off. The first 30 days after launch are called the warranty.

Point	*Owner*	*Required condition*
Days 0–30	Development team	Operate on-call, fix early incidents, tune monitoring and complete the runbook.
Before day 30	Development + DataOps	Prepare `warranty-handoff-sign-off`; DataOps may refuse an inadequate runbook or unresolved gaps.
Day 30 onward	DataOps	Accept steady-state operation and on-call responsibility.
Disputed hand-off	Data Domain Lead	Require remediation or approve a documented two-week extension.

Context use, retrieval and durable decisions

A prompt can include visible user text, higher-level instructions, a skill, retrieved documents and tool output. Some of that context may not be visible in the same chat message. Retrieval-augmented generation supplies selected source material to a generation request; it does not make every generated claim a faithful quotation of that material.

A finite context window imposes a capacity limit. Conversation summarisation or compaction can preserve the broad task while losing a column, exception or unresolved decision. Keep authoritative briefs and correction logs in files. On a new request, identify the current versions and ask the agent to reread the relevant sections.

Position and presentation can matter as well as capacity. Lost in the Middle found sensitivity to where relevant information appeared in its tested long-context tasks. The result is bounded by those models and tasks. For your contract review, make the evidence packet focused, label each source, and test whether the draft used a critical condition instead of assuming a universal ordering rule.

Separate role and purpose from examples. “Equity”, “active”, “real time” and “available” need product-specific definitions. A long thread with several conflicting examples can leave the interpretation unclear. Starting from a short, confirmed brief may produce a more reviewable result than adding another paragraph to the thread.

Why code gives useful but incomplete validation signals

Code offers checks that prose alone may lack: parsing, compilation, tests and comparison with expected results. Those checks help expose defects, but the strength of the signal depends on what they measure. A programme that always returns `4` passes a single “2+2” example without implementing addition. A model that hard-codes one row can make a superficial pipeline check appear successful.

A useful review therefore separates several questions. Does the code load? Do references and declared columns resolve? Does the selected input produce the expected output? Does the output preserve the agreed meaning, grain, units and exclusions? Does the structure support maintenance and review? The final two questions need product and engineering judgment as well as tools.

Automated feedback helps inspect coding output, while software-size anecdotes do not establish a universal line-count threshold for useful AI assistance. Evaluate a bounded model change or contract review in the repository you actually use, with representative dependencies and edge cases. A small change can be consequential; a large file can include much generated or repeated material. Size alone does not settle risk.

A second model’s critique can help generate test ideas, but a shared assumption can survive both generations. Tests that repeat the same generated formula also repeat its error. For the CG product

output, calculate expected expense values from selected source rows separately from the SQL being checked. Include an unknown expense and duplicate source representation.

How can a consumer use AI without Catalyst?

First establish the consumer's approved tools and data access. A workspace may provide a conversational interface such as a Databricks Genie space, an editor extension or another approved assistant. Availability and permissions are environment-specific; access to a dataset does not automatically grant access through every assistant. Ask the platform owner which route is supported and verify the answers against authorised data and definitions.

How should a team investigate an AI-assisted workflow?

Start with a real, authorised example of the finished artefact and a small task that can be evaluated. Identify the required sources, meaning, style and approval criteria. Test one part, inspect the result, and record what changed when the input or instructions changed. Do not begin with a large autonomous system before establishing the work it must perform.

Turn a useful, reviewed iteration into maintained instructions or a skill, then test it on a second example. Transfer to another person, department or product may require different terms and evidence. Reviewer agents can suggest issues, including unclear language or missing terminology, while human and independent checks decide whether the artefact is usable.

Retrieval, an internal application or orchestration may be appropriate once you understand the task, source volume, permissions and evaluation. Retrieval is a way to supply relevant context, not a universal first step or a guarantee of factual output. Follow the sequence of investigation, product design and engineering at the scale the evidence supports. See deliberate prompting.

Solutions: Tuesday — AI, Catalyst and the platform

End-of-day handoff

Back to the section

Platform handoff: conceptual answer key

The upstream readiness message reaches the consumer deployment through the supplied bridge. Airflow schedules and submits the configured job; Databricks executes dbt transformations. The candidate is written to application space. Quality checks evaluate that candidate before a publication step can make it available in the shared domain location.

Claim 1: a green deployment establishes deployment success. The candidate violates uniqueness, so today's publication must remain blocked. Check the actual publication result and consumer-visible release identity; do not infer them from the deployment or write.

Claim 2: the scenario provides no measured ten-second guarantee. Record the platform question and obtain observations and an agreed deadline before promising that latency.

Claim 3: changing a contract version does not create parallel schemas or preserve compatibility. Inspect the proposed migration and consumer impact, and agree coexistence or a coordinated transition with the platform and consumer owners.

The creator investigates the duplicate-producing transformation, repairs the mapping and obtains fresh checks for a new candidate. The team responsible for the event integration establishes latency evidence; the producer, consumer and platform owners agree the schema transition. Catalyst can draft a focused diff, tests or review questions from those facts. It cannot approve quality, invent latency or settle compatibility by producing confident prose.

> If an earlier release remains available, state its business date and freshness, explain that the new candidate failed, and follow the agreed delay/escalation response. Do not label that old release as today's successful delivery. Answers that retain "published" for the failed candidate have missed the central gate.

Wednesday — Data systems and dbt

Architecture and organisational examples in this chapter describe the supplied course cases. Current CG policy, platform responsibility, base images and publication requirements need the relevant internal owner and versioned service documentation. General engineering explanations remain transferable; the local training route is not proof of a live enterprise deployment.

Wednesday's plan

Start with the data systems your product lives in: storage, processing layers, grain and source-to-target mapping. Then place dbt and Databricks in that design before learning how Git and containers support a repeatable development workflow. Writing those rules down in English isn't that complicated. The harder part is turning them into code, and that's dbt, which takes up the afternoon. The dbt tests you write today check Monday's contract, and on Thursday the DAG, recovery and handoff reasoning are built around them. If an agent keeps changing files until a command prints something green, you still don't know what broke. So you need to know enough about Git, Docker, dbt and Databricks to say which one a failure came from.

Working time	*Focus*	*Result*
09:00–09:50	Systems, storage, medallion layers and grain	Distinguish storage, compute, governance and row meaning
09:50–10:00	Morning break	
10:00–10:45	Mapping and dbt/Databricks concepts; prepared demonstration	One contract rule traced through a model and test
10:45–11:30	Git, Docker/dev containers and readiness	A reviewed diff, container boundary and working practice route
11:30–12:15	Lunch	
12:15–15:00	Portfolio Analytics dbt practice and evidence review	165 minutes of protected dbt work
15:00–15:10	Afternoon break	
15:10–15:30	Generated-model review, reconciliation and handoff	What passed and what's still open for Thursday

For the Git practice, use a separate local training repository or the working copy the instructor gives you. Run `git status`, create a branch, make one small change, look at the diff and commit it locally. Don't change a live client repository just to learn a command. Does a branch keep your code away from shared systems? No. Code you run on a branch can still write to the same warehouse or call the same service, because which environment you point at is a separate setting.

Early demonstrations use an instructor-prepared environment. Learner commands begin after the Git/container introduction and readiness checks. Git and Docker full phases are preparation or clinic work; they do not consume the protected afternoon block. Use Appendix H for editor, terminal and container navigation. The dbt project contains the independent Portfolio Analytics checkpoints; question-led foundations appear beside this teaching.

Portfolio Analytics and CG product

Wednesday builds Portfolio Analytics from holdings, prices, dated FX and fund data on Databricks. Thursday runs that model graph through Astro. Friday uses the distinct CG products, fees and returns snapshot. Keep source names, consumer and row grain explicit between products.

The design vocabulary is shared. Monday names the internal analyst and offering grain. Portfolio Analytics demonstrates aggregate grains and independent checks. Friday applies those habits to offering identities, expenses and comparable performance evidence.

A meaningful model review

Suppose an agent proposes a query that joins holdings to a reference table and sums market value. Is the reference key unique at the join? If it has two rows per key, each holding is counted twice, and both the SQL and the row-count test can still pass. So check the unmatched rows and how many matches each key gets, as well as the final total.

Each type of mapping needs its own kind of test, and the table under Mapping types, failure modes and versioning lists them. Decide what a failure means before you write the test. Does it block the batch, flag suspect records, or go down as a known limitation?

When a model is still a stub, don't build a wide downstream selection and then blame the platform for the failures. The client guide uses selectors to build just the step you're writing and skips the tests that depend on unfinished models. Once everything is written, widen the selection and run the whole graph before you hand off.

Change the model, the mapping and the tests together. If you change a business rule but leave the description and the acceptance criterion alone, the code can be right and the product still misleading. For the handoff to Thursday, write down which models you built, which sources they read, and what's still unresolved, such as access or data problems.

On screen today

The supplied screenshots show an observed platform sequence. They support the concept discussion; the early walkthrough uses an instructor-prepared environment before learner setup.

System / sequence	*Why it is shown*	*Walkthrough*
Repository anatomy	Locate product code, environment configuration and deployment files; connect each to its ownership boundary in Git and product versioning.	Sequence
The pull-request workflow	Inspect the proposed diff, review/merge state and model/test evidence described in GitHub and the pull request.	Sequence
S3 data lake	See persistent object storage separately from query compute; relate file/table layout to storage choices and layer, alignment and grain.	Sequence
Databricks	Identify query execution, catalog names and governance separately from stored files; use where dbt and Databricks fit to interpret the prepared output.	Sequence

Today's sessions

- How Wednesday and Thursday run
- Data systems: database, warehouse, lake and lakehouse
- Storage architecture: lakehouse, ACID and data mesh
- Data pipelines, the medallion system and grain
- Source-to-target mapping, transformation rules and traceability
- Worked example: one STTM row into code
- dbt question-led teaching — Portfolio Analytics
- Git and the paved path: a recap
- Collaboration preparation: a reviewed Git change
- Docker: the problem containers solve
- Inside a container

- Anatomy of a Dockerfile
- Kubernetes: infrastructure as configuration
- Docker practice and diagnosis
- Portfolio Analytics setup
- `dbt build` and `--select`
- Review the model and its evidence
- Review challenge: find the defects in a generated model
- Optional enrichment: Catalyst's dbt skills
- Wednesday review: from rule to evidence
- Where should a transformation live?

How Wednesday and Thursday run

> Identify your starting knowledge, then connect the data-system concepts to the development tools and practice route.
>
> **THIS SESSION ANSWERS**
>
> - How do Wednesday and Thursday run?
> - What are Git and GitHub for?
> - Which paved-path tools does a data creator work in?

From here the course changes character. Days 1–2 were group design exercises and open discussion; the remaining days are mostly self-paced technical challenge blocks. Each block runs as a 20–30 minute overview session, then a self-paced challenge block of roughly 45–60 minutes, then regroup and move to something different.

If the core tasks are familiar, complete and explain their required evidence before choosing an advanced clinic. Agree the scope and time with the instructor; the advanced phase has its own prepared baseline.

Choose your starting point

> Knowledge check: Your development starting point
>
> Which tools have you used to make and inspect a real change? For Git, describe a diff you reviewed. For Docker, distinguish an image from a running container. For dbt, explain one model and test. For SQL, sketch a two-table join and a grouped total. Record the tool you need help with before beginning practice.

Use a prepared environment to watch the early concepts. After the tool introduction, complete the readiness checks for your chosen route and ask the facilitator about any unmet prerequisite. Each activity names its outputs and checks; use those to select core work before advanced clinic tasks. Appendix H explains Markdown preview, terminals and the training container.

Data systems: database, warehouse, lake and lakehouse

With Docker done, we turn to the systems the afternoon's dbt work runs in, and to how you write down what a data product has to do.

THIS SESSION ANSWERS

- What are a database, a warehouse, a lake and a lakehouse?
- What is a data pipeline, and what is grain?
- What goes in a source-to-target mapping?
- What do an SLA, an SLI and an SLO promise?

Knowledge check: Storage and processing

Which requirements would make a database, warehouse or lake a suitable home for a product? What would you need to know about schema, query patterns and governance before choosing?

A working glossary

- A database is *structured* storage. It's tabular, with a schema nailed down by the software using the tables. Typically online/transactional systems with daily inserts; the schema is rigid.
- A data store holds raw files: each individual piece may have structure, but the system as a whole has no unified structure. When you hear "data store", you can almost always just think *files and folders*.
- A relational database: what is a *relation*? A relation is a table. It's a mathematical word (a relation is a set of tuples). Relational databases are *not* about relationships; that's a common misunderstanding. Keys and joins express relationships between the tables; their correctness depends on the declared identity and multiplicity.
- OLTP (online transactional processing) — systems optimised for writes/inserts; better thought of as systems of record, where data is written atomically. Why "transaction"? The original OLTP databases were retail databases (Sears-era department stores, 1950s onward), and the transactions were literal financial transactions. Today the word means any write operation that must succeed or fail as a whole. Retail needs this for double-entry: withdrawing from one account and adding to another must succeed or fail together, otherwise you get double spending. (Aside: the one famous solution to double-spending *without* transactions is, arguably, Bitcoin, in 2009.)
- OLAP (online analytical processing) — optimised for reads; analytical databases, often with large numbers of columns. Rule of thumb: write-heavy databases have few columns and many rows; read-heavy analytical databases have many columns. Transactional systems are more sequential (transactions must succeed or fail in order); analytical systems parallelise well. You very rarely see a GROUP BY in a transactional database.
- A data warehouse — an analytical (OLAP) database that also tracks history: tabular, but with a much more flexible schema; tables often have large numbers of columns, typically fitted to reports, and somewhat arbitrary. You keep adding columns to make your report work. Worth knowing: OLTP systems generally *don't* timestamp changes. When a user changes their username, the record is updated in place. If you need a log of changes, that's a separate system, and that log is what gets inserted into the analytical database, which is why the data most of you see always has timestamps everywhere.
- A data lake stores files in varied formats, usually in object storage. It can hold tables, images, free text and contract files without requiring one tabular schema for the entire store. Versioning, ownership, discoverability and access still need explicit controls; flexible storage does not make those guarantees automatic.

Compare a database, warehouse and lake by their storage flexibility, processing needs and governance responsibilities.

In these examples, structured data means fields governed by a declared schema. Tabular storage is a common form. Semi-structured or other file formats can also carry useful structure, so name the actual format and schema requirement rather than treating "structured" as a complete design.

Parquet

To store tabular data in a lake you can simply add a CSV or Excel file, but more commonly a Parquet file (also ORC, Arrow): like a CSV that is compressed and stored column-by-column rather than row-by-row. It's effectively a pivoted CSV.

The lakehouse

The lakehouse sits between the warehouse and the lake on the flexibility scale. It is like a data lake in the storage layer, still just files (Parquet etc.), but the difference is in the querying layer. A database has a querying layer and a storage layer; a data lake has only storage.

A lakehouse combines lake-style file storage with table management and analytical query capabilities. Transactional table formats and metadata add schema, version and concurrency controls over files. The useful design question is which guarantees the chosen format, catalogue and query engine provide, and how those guarantees support the consumer's workload.

Why does it matter? This is the heart of how you'll use Databricks. When dbt queries Databricks, it relies on the fact that Databricks isn't *just* files and folders. There's a SQL layer on top that dbt can use. Even inside Databricks everything is still files underneath; Databricks provides a single management layer (ACID and so on) over them.

ACID: transaction guarantees

ACID describes four transaction guarantees:

- **Atomicity:** a transaction commits as a whole or has no effect.
- **Consistency:** committed data satisfies the constraints enforced by the system; business validity still needs explicit rules.
- **Isolation:** concurrent transactions do not expose an incomplete write to a reader.
- **Durability:** committed changes survive a failure.

For Delta tables, data files and transaction metadata identify committed versions. This is different from claiming that every distributed operation executes in one universal real-time order. Check the supported isolation and transaction scope when choosing a design. Databricks explains its ACID guarantees.

Worked demo: Delta time travel

One command shows the mechanism Databricks built those guarantees on. Against any *table* the project materialises (staging models are views, so pick one from the prepared layer) in your `training_<initials>` schema:

```
DESCRIBE HISTORY training_ab.prepared_orders;
```

Every write to a Delta table produces a version: the table you query is just the latest one, and the earlier versions are kept. Run `dbt run` twice more and run the DESCRIBE again. Two new versions have appeared, one per write, each stamped with when, which operation, and by whom. Then read an earlier state directly:

```
SELECT * FROM training_ab.prepared_orders VERSION AS OF 2;
```

That's how the lakehouse keeps its ACID guarantees on top of plain files. A write either produces a complete new version or it produces nothing, so a reader never sees a half-written table (atomicity), and switching things off between versions loses nothing that was committed (durability).

A separate product case uses Parquet with custom bitemporal logic. Table-version history and business-history requirements are different; choose a design that records the dates and corrections the consumer needs.

On screen: S3 data lake

Buckets and object prefixes are how the files are physically stored. Catalogs, schemas and tables are the governed view of the same data, and they're what you query. A folder called curated is just a name. The quality promise lives in the product's tests and contract. The Iceberg example keeps metadata files beside the data files, and together they make up one table.

> Knowledge check: Read a storage example
>
> What can a file path and storage format tell you? Which product properties would still need evidence outside a screenshot?

You won't browse or configure S3 here. For the client dbt work, reach the data through the tables and views your project and profile point at. You need to tell a path, a table and a product apart, and you can leave administering the storage to the platform team.

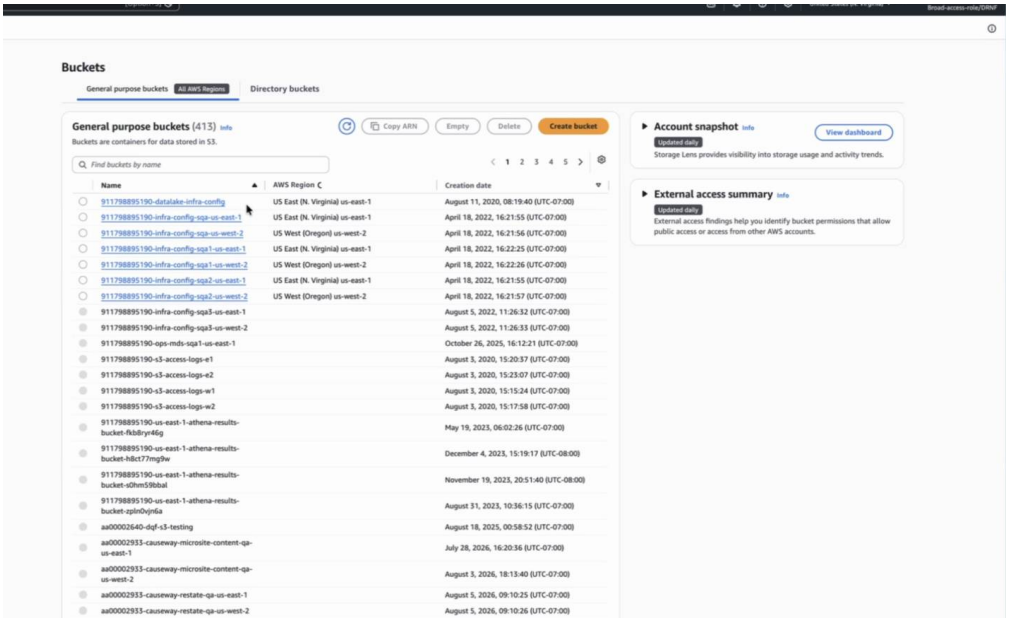


Figure 33: S3 buckets

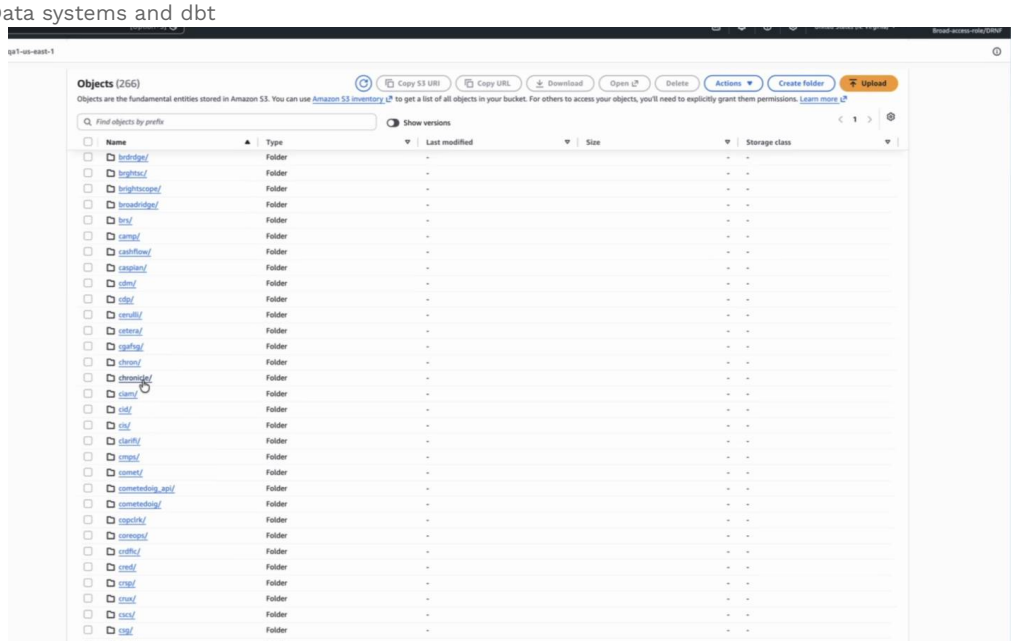


Figure 34: Landing zone

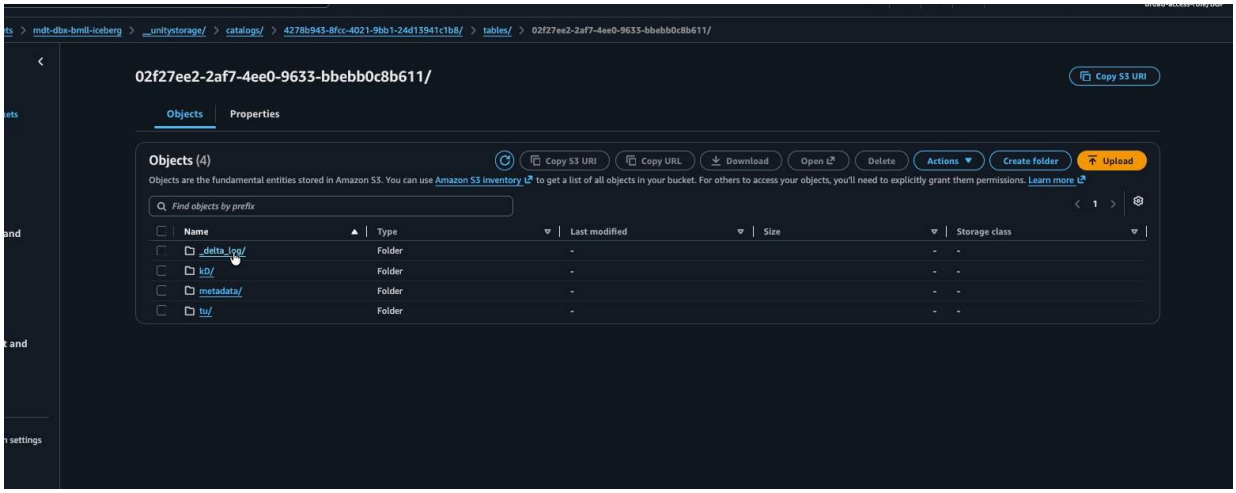


Figure 35: Iceberg table folders

On screen: Databricks

Unity Catalog's tree puts catalogs above schemas, and schemas above tables and views. Compute is separate, and a warehouse runs your query. Seeing a table name in the tree doesn't mean you can SELECT from it, so use the access browser to see who can read what.

> **Knowledge check: Read a platform example**
>
> Where does a query execute, which identity controls access and what names identify its data? What could you check before blaming a missing table on SQL?

In the prepared demonstration, identify the catalog, schema, table and query compute. For a later authorized connected rehearsal, use the instructor-assigned names rather than copying screenshot identifiers. If a table is missing, check the configured namespace and target before changing SQL; if access is denied, record the operation and ask the responsible platform role.

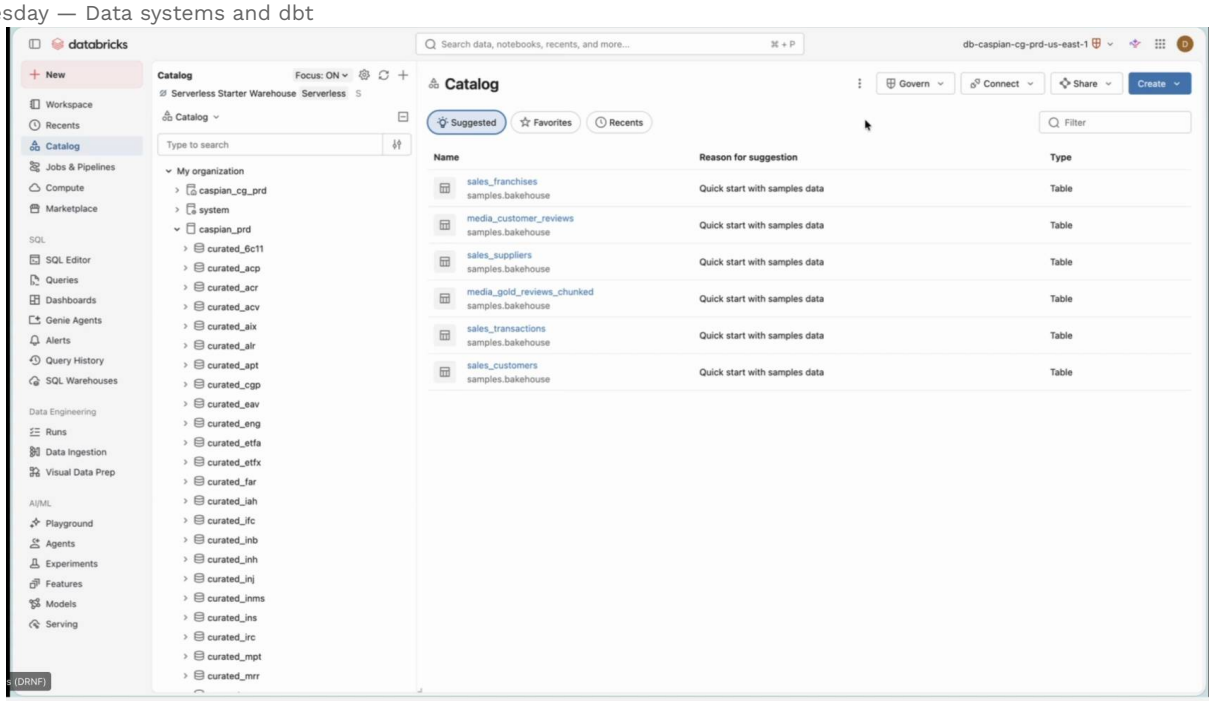


Figure 36: Catalog Explorer

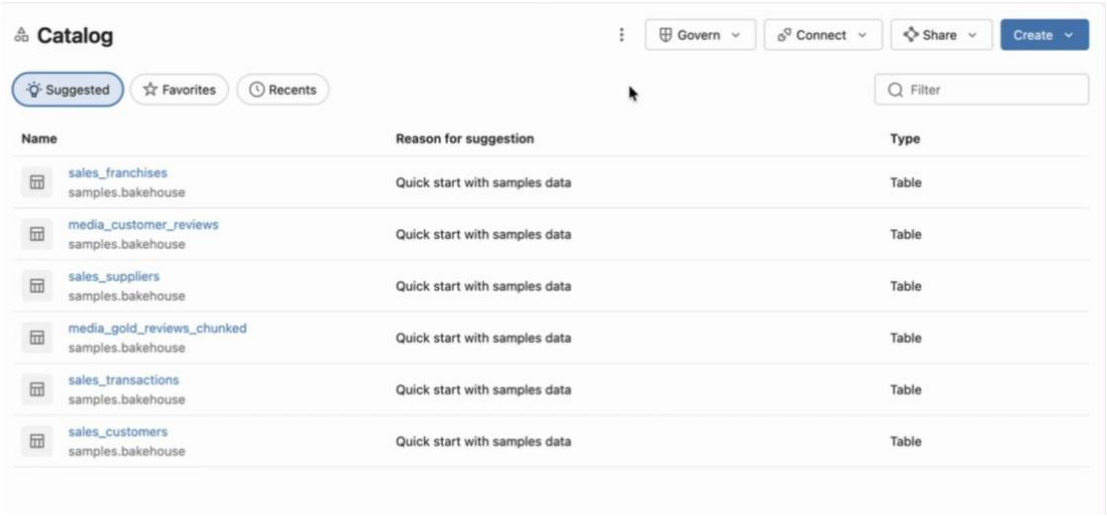


Figure 37: Detail of Catalog Explorer

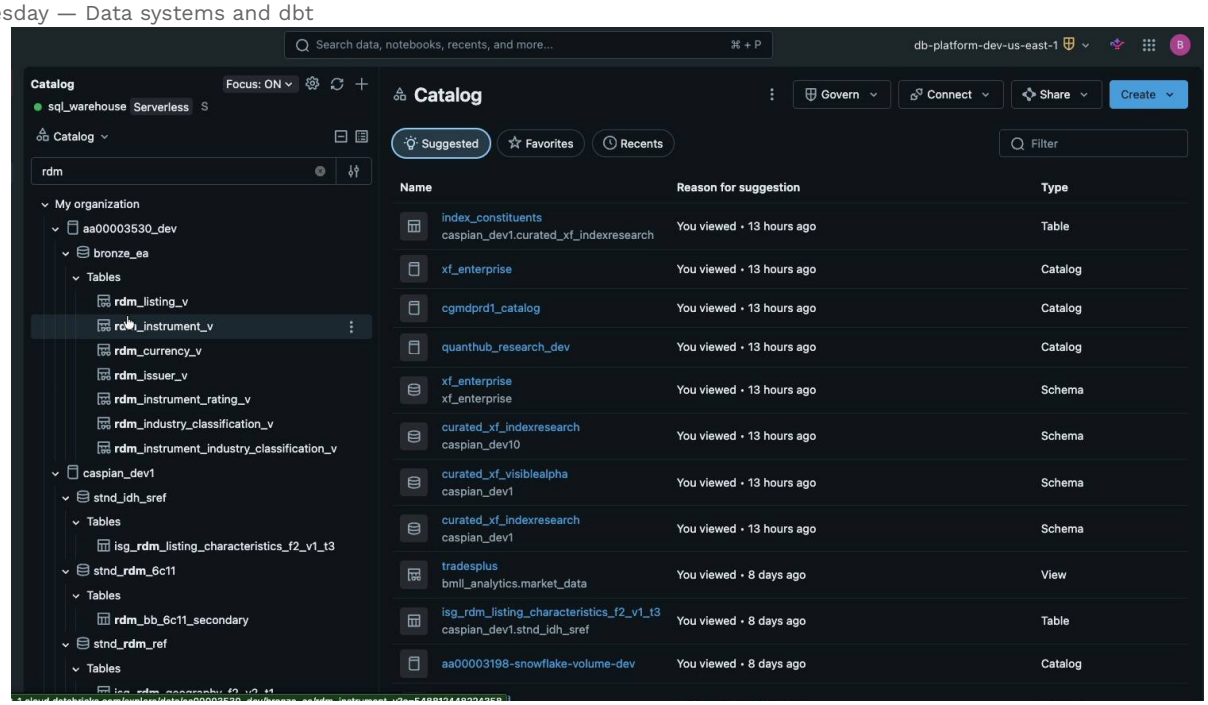


Figure 38: Catalog tree

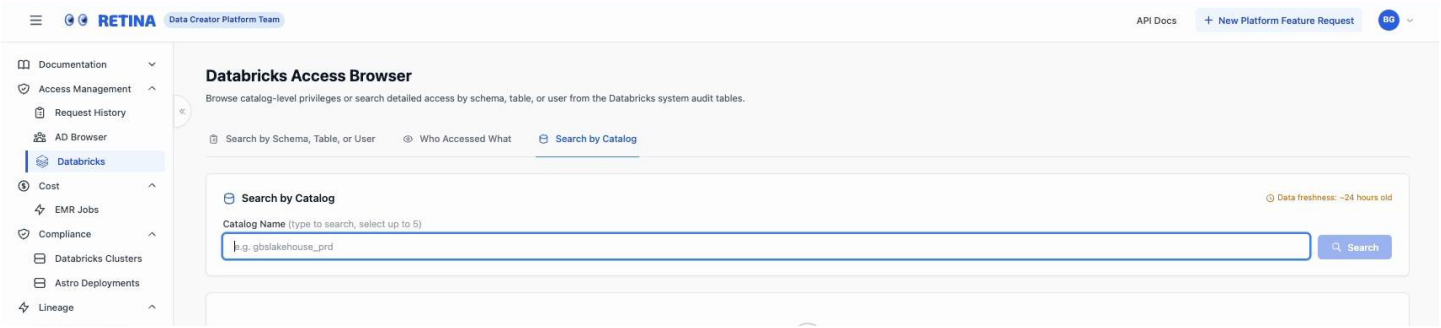


Figure 39: Access browser

Storage architecture: lakehouse, ACID and data mesh

The same storage, this time from the side of the architect choosing a design for a product.

THIS SESSION ANSWERS

- How do you model tables for consumers on a lakehouse?
- How do you keep personal data erasable and access controlled?
- How does a finished product reach its consumers?

Knowledge check: Architecture and product promises

How do storage, retention and access requirements affect the design of a product? Can a useful catalogue description establish that its rows are safe and fit for your decision?

The lakehouse and ACID

A lakehouse adds table management and transaction metadata to file storage. The ACID explanation covers committed writes and concurrent reads; it does not imply that arbitrary files acquire those guarantees.

> Expert
>
> The supplied architecture calls its lakehouse Caspian. This course example does not establish the current enterprise service boundary or support owner; confirm those in the internal service documentation. In that architecture, S3 holds data files and table metadata, while Databricks executes queries and Unity Catalog governs registered objects. Delta uses Parquet data files with a transaction log; it is not a sorted CSV. Parquet-only and Iceberg examples have different table-management arrangements. Delta Lake documents the table format and guarantees.

Dimensional modelling still matters

A lakehouse changes the storage engine, but consumers still need tables organised around the questions they ask. Dimensional modelling separates measurable events into facts and descriptive context into dimensions. In the prepared or curated layer, that gives BI tools predictable joins, gives several products the same definition of a fund or client, and lets a reader find measures without decoding the source system.

This example star uses a territory scenario because its design shows the privacy decision clearly. Its grain is one row per client, per territory, per asset, per date.

Table	*Role*	*Key content*
`fact_territory_assets`	Fact	Foreign keys plus `asset_value`, `position_qty`, `opportunity_score`
`dim_client`	Dimension	Client name, segment and tax identifier
`dim_territory`	Dimension	Territory, region and coverage officer
`dim_asset`	Dimension	Asset name, class and provider
`dim_date`	Dimension	Calendar date, fiscal quarter and business-day flag

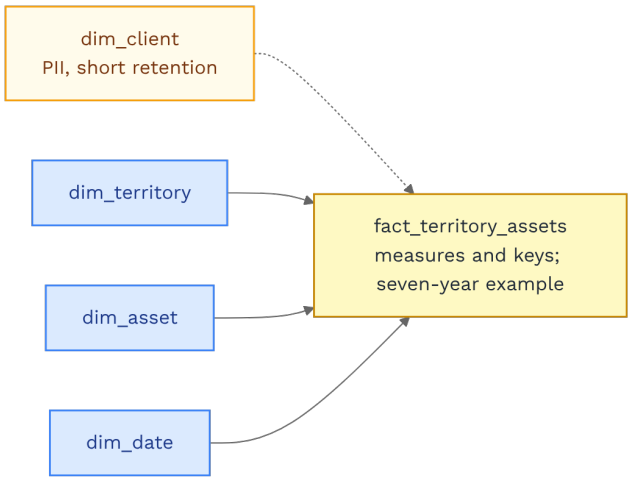


Figure 40: The territory example separates client attributes from long-lived measures and keys. Classification, retention and erasure still require review.

The PII sits in `dim_client` on purpose, away from the long-lived measures in the fact. That choice allows a later erasure request to remove client attributes without deleting the seven-year analytical history held

against a client key. The key treatment still needs privacy and legal review. Make the separation while you're modelling, though, because it's expensive to undo once PII has been copied into every fact row.

Pattern	*Use it when*	*Main constraint*
Star schema	BI and reporting use stable dimensions and repeated slices	Up-front modelling and managed dimension changes
One Big Table	One narrow analysis or feature set needs flat, fast reads	Duplicated dimensions and awkward updates
Data vault	Several changing source systems need an auditable integration history	More joins and slower direct BI queries

For the layer names used in this course, a star belongs in prepared or curated output. A data vault may belong in integration when source structures change often. Keep an OBT for a narrow consumer whose query pattern is known and unlikely to widen.

Privacy engineering and retention

Check the regulations and internal policy with the responsible CG teams for the product and jurisdiction.

Requirement	*Mechanism considered in the design*
GDPR erasure	Delete the PII row, purge deletion vectors, vacuum old files, and rebuild downstream copies
CCPA access and deletion	Locate personal information through the same lineage and apply the approved deletion process
SOX auditability	Retain change history through type-2 records and Delta versions where policy permits
MiFID II record keeping	Keep the required investment-decision records in an append-only governed layer
Internal information barriers	Apply Unity Catalog row filters and column masks defined by the authorised policy owner

> **Erasure removes time-travel history**
>
> On Delta Lake, erasure has four technical steps. First, `DELETE` the subject's PII from the table. Second, `REORG TABLE ... APPLY (PURGE)` where deletion vectors are enabled. Third, run the approved `VACUUM` operation to remove historical files. `VACUUM ... RETAIN 0 HOURS` destroys time-travel history, so it must never be copied from a teaching example into production without the approved retention and safety settings. Fourth, rebuild or delete the same subject from staging, integration, prepared tables, materialised views and streaming outputs. Deleting only the current dimension row leaves historical files or derived copies behind.

Layer	*Retention approach*
Raw landing	Short immutable reprocessing window, then archive under the applicable record policy
Staging	Short rolling window because it can be rebuilt from landing data
Integration	Limited version history for debugging business-rule changes
Prepared / curated	Match the consumer contract and the applicable reporting obligation
PII tables in any layer	Minimum approved retention with an erasure path and controlled vacuum

Time travel keeps old files so you can read a past version of a table, and erasure needs those files gone. So keep PII in short-retention dimensions, like the tax identifier in `dim_client` above, and keep the non-PII facts for the required reporting period.

Architecture decision checklist and anti-patterns

Workflow

Use these two tables before the design is approved. If any decision row is blank, the design review is incomplete.

Decision area	*Question to answer*
Architecture	Why warehouse, lake or lakehouse for these data and workloads?
Modelling	Why star, OBT or data vault for the consumer's queries and source-change rate?
History	Which type-1, type-2 or type-3 behaviour applies to each dimension?
PII separation	Can personal attributes be erased without deleting analytical facts?
Namespace	Do catalog, schema and table names follow the team's domain naming convention?
Access	Which least-privilege grants, row filters and masks apply?
Lineage	Are every upstream source and downstream consumer recorded?
Retention	Does each layer have an approved period for operations, audit and privacy?
Sharing	How will another team or organisation receive the governed product?

Anti-pattern	*Consequence*	*Prevention*
Landing data goes straight to prepared output	Source defects and business rules become mixed in one model	Clean in staging and conform or enrich in integration first
PII appears in facts	Erasure can destroy the analytical history retained for reporting	Put erasable attributes in controlled dimensions
Catalog-wide grants	Users inherit access to unrelated sensitive products	Grant at schema or table level and use governed tags where available
Schema changes ignore lineage	A release breaks consumers that were never contacted	Inspect dbt and catalog lineage before promotion
One retention period covers every layer	The period is either too short for audit or too long for privacy	Set retention by layer, classification and obligation

Fit and lock-in

A warehouse remains a sensible home for structured relational data, stable BI queries and strict governance when the team accepts a proprietary storage and compute model. A plain data lake suits mixed raw formats and low-cost storage when a separate governance system can supply schema, transactions, ownership and access control.

Without owners, metadata and quality controls, teams may create conflicting copies because they cannot find or trust the existing data. Cheap object storage does not remove governance and maintenance costs.

The lakehouse is only partly portable. Delta stores data in an open format that Spark, Trino and DuckDB can read. Unity Catalog's grants, masks, filters and lineage records are a platform commitment; they do not migrate with the Delta files. If an RFD chooses this stack, write down the work needed to recreate the governance somewhere else.

Publishing through two catalog tiers

Teams build and validate in an app-aligned catalog such as `caspian_ig_dev`. After the DQ gate passes, a publish task copies the table into the consumer-facing domain catalog:

```
CREATE OR REPLACE TABLE ig_dev.exposures.fact_fund_exposure
DEEP CLONE caspian_ig_dev.exposures.fact_fund_exposure;
```

Why split it this way?

- Work in progress stays away from consumers.
- The publish step runs only after DQ has passed, so any table you can see in the domain catalog has passed that gate.
- The domain gets its own copy of the table, separate from the build area.
- The same validated product can be copied into more than one authorised domain.

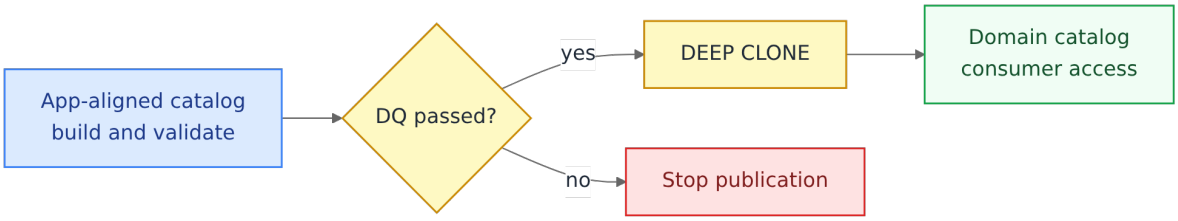


Figure 41: The two-tier publish path keeps work in the application catalog until data-quality checks pass, then clones the validated table into the domain catalog.

The marketplace listing from the consumer's side

Register a product through the supported marketplace process so consumers can find its purpose, owner, schema, glossary and access route. Search the same catalogue before a new build: an existing positions product may meet the need or supply a reusable component. Verify the listing and its definitions instead of inferring registration from a completed pipeline.

What a consumer of the exposures product would see on its listing:

- Sample data and the actual table, plus a glossary tab defining the columns. Some of the data dictionaries are AI-generated, so check them before you rely on them, the same as the glossaries behind an STTM.
- A request-access section if you don't have access, which connects into ServiceNow.
- For highly sensitive data (HR, say): without access you see no sample data and no schema, just the business context.
- A section for how to use it and how not to use it, worth populating when it's your product.

Data meant to be shared across domains, or that anyone might reference, should be discoverable and searchable. Otherwise we're just creating data silos.

Unity Catalog is the Databricks side of the same discoverability. Anything in Caspian (tables, shares) is registered in Unity Catalog, and its namespace has three layers, the same ones your dbt profile named. At the bottom is a table, above it a schema (the database, or container for the data), and above that a catalog. So catalog, schema, table. The marketplace is available to everyone, but Unity Catalog doesn't necessarily let you see everything. Consider the discoverability trade-off. Should everyone be able to browse all schemas and views (without the data) so they can find what exists? Without browse access, the only way to know what's in a catalog is to ask, and discoverability can't really work. The practical request you'll make as a creator is catalog-to-workspace binding. Catalogs are bound to workspaces, and not every catalog is bound to every workspace. "I have this catalog in the Caspian workspace, and I need it in my research workspace. Can you bind it?" For a Caspian catalog, more than likely yes. Binding a domain catalog into another domain's workspace is less likely to be available by default. People often don't realise a catalog needs to be bound to their workspace before they can use it.

Unity Catalog controls at query time

Column masks protect a sensitive value while preserving access to the table. This shortened form shows the mechanism:

```
CREATE FUNCTION mask_tax_id(v STRING) RETURNS STRING
RETURN CASE WHEN is_member('compliance_team') THEN v ELSE 'REDACTED' END;

ALTER TABLE prepared.dim_client
ALTER COLUMN tax_id SET MASK mask_tax_id;
```

Row filters apply the same idea to records:

```
CREATE FUNCTION fund_filter(fund_family STRING) RETURNS BOOLEAN
RETURN is_member('global_team') OR fund_family = current_user_fund_family();

ALTER TABLE prepared.fact_fund_exposure
SET ROW FILTER fund_filter ON (fund_family);
```

Unity Catalog evaluates both controls in the engine, whether the request arrives through SQL, a notebook, a BI tool or an API. Per-table functions become hard to manage across hundreds of products, so governed tags provide the scaling route: tag a column as PII or a table as restricted, then bind a policy to the tag so later objects inherit the control when they are classified. The governance decisions behind these snippets are worked through in Appendix B.

Data mesh

- Data mesh is the opposite of data siloing / a centralised data team: data product creators are spread throughout the business and can create and self-serve their own products. It is an *organisational* concept, not a technology.
- Summary mind-map of the supplied course architecture: data mesh (human/organisational side) + data lakehouse (infrastructure side) + dbt (code/data side) + SLAs (promises).
- Assess a proposed organisational pattern by the ownership, access and support problems it solves. A fashionable label alone is not an architecture decision.

> Learning Review
>
> Retrieve the differences between retention, access control and publication. Apply them to a client dimension linked to a long-retention trade fact: which attributes and relationships require a recorded policy decision? Check that the proposed retention and erasure treatment, masks/filters, owner and catalog publication route agree with the product contract; technical deletion alone does not establish a policy outcome.

Data pipelines, the medallion system and grain

With the lakehouse as our system (files and folders plus SQL, as far as we're concerned), how do we use it to create data pipelines?

> Knowledge check: Layer, alignment and grain
>
> What does one row represent in your source and proposed output? How does that differ from the layer or folder in which the model lives?

A data pipeline is *automated* (prefer that word over "CI/CD") movement of data from one place to another. The data flows, and at every stop an operation is applied. The operations come in a few kinds:

- Ingest / save-and-load — taking data from the source, landing it.
- Transformations (sometimes called *rules*).
- Joining and combining — enriching across data sets (called *curation* in the supplied course examples).
- Exposing to the consumption layer — publishing without changing the data.

The medallion system

The generic industry framing is bronze / silver / gold:

- Bronze — where data is initially dumped from an outside source; raw.
- Silver — the first layer of transformations.
- Gold — the final layer; e.g. report-ready.

These are *relative* terms (each team defines what they mean), but there are often agreements between phases, sometimes enforced automatically on release. The layers might literally be folder names: an agreement might say that by the time a file reaches silver there are no missing values, all currencies are the same, everything is timestamped; gold might mean report-ready.

Nothing in the medallion system is inherently about sources and consumers. Two software systems could ingest from different apps, normalise, and dump back out, system-to-system. A bronze model may preserve source structure and a gold model may serve a consumer, but those names do not determine product alignment. A reusable domain product can contain several processing stages, and another product can consume its output as a source.

Capital Group's stage names

The course examples use these stage names:

- Staging — raw data as landed (e.g. from MFT), plus metadata such as timestamps. Single-source; data still looks like its source.
- Integration — joins, lookups, enrichment; intermediate data artifacts. Still largely single-source and raw-shaped; data quality work happens between staging and integration.
- Prepared / curated (the terms are used for the same thing) — curation logic across multiple sources; this is where the grain changes.

> Other layer names you'll hear
>
> Some teams also described two golds: an application gold and a business gold exposed to the business. Another set is landing / ingest / curate. The industry uses bronze / silver / gold widely, and internal names vary by team.

Keep two ideas apart. Organise by alignment, describe by layer. There are also two sets of layer names, the teaching examples and the platform convention, and the table shows both.

	Alignment	*Layer*
Question	Who is this built for?	How far has it been processed?
Vocabulary	Source-, domain-, consumer-aligned	Teaching examples: staging / integration / prepared-curated. Platform names: raw / staging / intermediate / curated
Where it appears	Product folder, tag, `ref()` chain, ownership	Materialisation, access, contract enforcement, model prefix
Unit described	The product	The model

Use source-, domain- and consumer-aligned to explain whose requirements shape a product. Use the assigned repository's model names to explain processing: the supplied projects include `stg_`, `int_` and prepared/report models. There is no universal one-to-one crosswalk and no claim that every Catalyst version generates the same layout. State both the product boundary and the model stage; a source can be another internal CG team rather than an outside supplier.

For a consumer-aligned mart, the kind of query decides the prefix:

- `rpt_` names a report-shaped query, such as one Morningstar exposure roll-up per fund.
- `agg_` names a measure grouped by a dimension, such as exposure by asset class.
- `ts_` names one row per entity and period, such as daily exposure by fund.
- `api_` names a low-latency single-row lookup, such as the latest fund exposure.

The training project itself uses folders named after the layers.

Grain

Grain states what one row represents: an event, an entity at a specified time, or an aggregate over a defined population. Name its identifying keys and time boundary before joining or grouping. For an aggregated metric, a larger group produces a coarser result, as the following example shows.

Take LA crime data, with crimes scattered unevenly across the city. Compute the mean crime rate per square mile across *all of LA* and you get one number, say 3, which is very non-granular. Replace all the detail on the map with "3" and you can't see anything.

Split LA into districts (Pasadena, Long Beach and so on) and compute the same metric per district, and you get more numbers, each with a smaller group behind it: finer grain. Smaller groups, finer grain; you change the grain whenever you do a group-by.

You could also define grain by level/categorisation (a key change often goes with it). Maximum granularity is generally at the source. In principle joining in additional data sets can add detail later, but mostly a pipeline lowers granularity (makes things more general) as data flows through the stages.

Organisational context has the investment glossary, a worked fact-and-dimension join and two window-function examples.

Grain changes across transformations

An aggregation changes what one row represents, so check the grain on both sides of it. Before summing, check the lookup keys are unique and look at the unmatched rows. After summing, write down the grouping columns and reconcile the total to the eligible inputs. An offering count can be arithmetically correct and still include repeated catalogue/detail records or lose offerings with unknown expenses.

Where dbt and Databricks fit

dbt defines models and tests; the configured Databricks compute executes their SQL over governed tables backed by storage. A model can materialise a view or table rather than move a physical file between folders.

Why not just do it all in Databricks? Databricks does provide SQL (and Python, but PySpark pipelines get complicated fast, so set that aside). Databricks SQL handles reporting queries fine, but as soon as you want to create tables (actually materialise each hop), it gets more complicated. What dbt gives you:

- Simple SQL: you write what looks like a series of `SELECT` statements; behind the scenes dbt does all the table creation and insertion for you. As an analyst you just write selects and dbt does the flow.
- Versioned models, documentation generation, tests, enforced contracts and CI integration. Other transformation approaches need their own equivalent controls.

It remains an architect's trade-off. You *could* use Databricks alone; there's no single right answer.

Source-to-target mapping, transformation rules and traceability

This is the technical half of product design. We have done the non-technical design (requirements, interviewing), and we still need to specify the data product *to the person developing it*: the source-to-target mapping rules, the SLAs and SLIs, what we measure and how. It's quick, because writing the rules isn't the hard part. Implementing them in code is, and that's dbt. dbt is how we *write* the data product. Git is how we version it, and Docker is how we bundle it for release, dependencies included.

> Knowledge check: Rules before SQL
>
> Which facts belong in a source-to-target mapping row? How would you describe a conversion, join or null rule so another creator can implement and review it?

The STTM

The source-to-target mapping (STTM) specifies how source records become target records: source and target names/types, transformation rule and null treatment. Record renames, unit conversions, joins, filters and aggregation explicitly so another creator can implement and test the same meaning.

One row is harder than all the others. Morningstar and FactSet each carry their own security identifiers, and deciding that two of them mean the same asset (and then assigning one internal identifier) is the hardest row in the mapping. For an individual-account product, such as retail or retirement accounts, data may arrive from two to four sources. The sources are joined, duplicates removed, and a unique internal identifier assigned, and that last step is one of the hard areas. The classic case is police databases. Officers fill out forms at crime scenes, names are spelled differently across reports, and it's very hard to know whether any two reports are about the same person. Day 1's identifier-misalignment pain point is this row, and the "No join condition" omission below is what happens when it is left out of the mapping.

The STTM travels with a glossary: the mapping rules don't tell you what the language and values *mean*, and meanings shift in realignment. "Returns" in one column may be risk-neutralised while another consumer wants raw returns. Same word, different operational meanings, so the glossary must pin them down.

Suppose a fund-attribution product ships money-weighted returns when the portfolio managers needed time-weighted returns. Nobody wrote down which "return" was meant, and the fix is weeks of rework. So one glossary entry records term ID, term, definition, synonyms or aliases, authoritative source, example values, related terms and owner. Give each term one owner. Version every changed definition with an effective date and tell the consumers, and review the glossary at launch and at each major version.

The supplied marketplace example lists data sets with glossary sections defining column meanings, as the marketplace listing below shows. But some of those glossaries are currently AI-generated, and they need to be very carefully scrutinised before you rely on them.

Mapping types, failure modes and versioning

Poor source quality complicates the rules and can rule out a transformation without you knowing it, so profile the data before you map it. `discover-data` ranks candidate Unity Catalog tables. For the Morningstar work, run `data-profiling` on the exposures candidates to see null rates, distinct counts, ranges and fitness flags. Do that before `transformation-spec` drafts the STTM, so every source column in the mapping comes from a table you've looked at.

The mapping type tells the builder which kind of test the row needs:

Mapping type	*Test implied by the mapping*
Direct	Compare source and target values after any rename or compatible cast.
Derived	Recalculate the expression independently and compare the result.
Constant	Check that every target value equals the declared constant.
Aggregated	Reconcile sums or counts to the lower-grain input.
Lookup	Check join-key validity and return every unmatched key — country code `GB` looked up to United Kingdom of Great Britain and Northern Ireland, with any code the table lacks returned.
Join	Column 1 from one source, column 2 from another, combined into the target: check the key on both sides and count rows lost or multiplied by the join.
Filter	The rows kept on a condition: check that no kept row fails the condition and reconcile the count dropped.
Union	Rows of two sources joined end to end: check the row count equals the sum of the inputs and the column types agree.
Conditional	A value chosen by an *if*: exercise every branch with a row that reaches it.

Six things get left out of mappings again and again, and each one causes rework:

- No null rule. The builder has to guess whether to reject, default, flag or quarantine a null, and what *empty* means differs by kind of data. State whether null means unknown, unavailable or not applicable.
- An ambiguous transformation. "Calculate return" leaves TWR versus MWR, gross versus net, and the time window unresolved.
- An ambiguous connective. The word *if* has several senses, and *or* can be exclusive or inclusive ("left or right" meaning either, or possibly both). In English, add words rather than relying on *if* and *or*: "either this or that", "both this and that".
- An unscoped term. "Phone number" — US phone numbers only, or worldwide? State the scope of every technical term the rule uses.
- An assumed stable source name. An upstream rename invalidates a mapping that records no source-schema version.
- No join condition. A multi-source mapping needs the join key and a rule for when the sources disagree.

Version the STTM while those details change:

Stage	*Version*	*What the revision records*
Design	`v0.1`	Initial mappings from requirements and profiling.
Build	`v0.2–0.n`	Discovered edge cases, refined null handling and clarified transformations.
Launch and maintenance	`v1.0`, then `v1.1+`	Tested baseline, followed by source changes, additions and deprecations.

Every mapping is relative to a schema, so put the source-schema version in the STTM header from the first draft. When Morningstar changes a column, the header tells you which version of the source the mapping was written against, and the impact analysis starts from there.

The traceability matrix

A traceability matrix and design canvas are structured records for different review questions.

One aspect of traceability is that the promises we make about the data product (in particular the promises about these transformations) are tracked and enforced. A row of the matrix links: the business outcome / consumer promise (e.g. “refreshed by 7 AM, national coverage, complete”), the columns and transformations relevant to that promise, the glossary terms needed to understand them, and the risks, accountabilities, dependencies and findings. In short it is the mapping, the glossary and the risks laid alongside the actual promise and the data consumer. It’s just record keeping.

Take the Morningstar exposures. The consumer wants fund exposures by 07:00. The STTM columns give source and target, the types, the rule and the null handling, and they link to the glossary. The quality commitment covers freshness, completeness and range checks, and the RAID record holds the risk that goes with it.

The design canvas includes everything else (risk register, traceability matrix, reuse assessment and the source-to-target mapping), and the mapping in turn refers to the glossary.

Traceability is not lineage

Lineage records where data came from and which transformations or outputs depend on it. Traceability links a requirement to its implementation, check and acceptance evidence. Use lineage to inspect change impact; use the traceability matrix to show that the consumer promise is covered.

Worked example: one STTM row into code

> Knowledge check: Trace one rule
>
> How would you show that a source value, model expression and test implement the same unit and null treatment? What example would expose a mismatch?

We keep saying dbt is how we implement the STTM. So let’s take one real row and build it end to end.

The row, from the Morningstar exposures mapping:

Source system	*Source column*	*Source type*	*Target column*	*Target type*	*Transformation rule*	*Null handling*
Morningstar	`us_exposure`	float	`exposure_us`	float, 0–1	rename; value must lie between 0 and 1	nulls not permitted — fail the pipeline

Three files implement this schematic rule. The names illustrate the mapping; use the assigned project paths for your own attempt.

First the staging model, which does the rename and nothing else. Staging models are meant to be boring:

```sql
-- models/staging/stg_morningstar_exposures.sql
select
    fund_id,
    us_exposure as exposure_us,
    loaded_at
from {{ source('morningstar', 'exposures') }}
```

Then the integration model. This row's transformation rule contains no arithmetic (a rename plus a validation), so the integration model simply passes `exposure_us` through while joining it onto the fund reference data, which is the job that layer is for:

```sql
-- models/integration/int_fund_exposures.sql
select
    f.fund_id,
    f.fund_name,
    e.exposure_us
from {{ ref('stg_funds') }} f
join {{ ref('stg_morningstar_exposures') }} e
    on f.fund_id = e.fund_id
```

Last, the two right-hand cells of the STTM row become tests. The null-handling cell says "fail the pipeline", and that's what a failing `not_null` test in `schema.yml` does:

```yaml
# models/staging/schema.yml
models:
  - name: stg_morningstar_exposures
    columns:
      - name: exposure_us
        tests:
          - not_null
```

The range rule becomes a test too. It's written today as a one-off SQL file (any row it returns is a defect; the afternoon explains why every dbt test works that way), and can be turned into a reusable generic test; see dbt further study:

```sql
-- tests/exposure_us_in_range.sql
select fund_id, exposure_us
from {{ ref('stg_morningstar_exposures') }}
where exposure_us < 0 or exposure_us > 1
```

> Learning Review
>
> Retrieve the source field, target name, unit transformation and null treatment from one STTM row. Apply them to the example model and its tests without reading the commentary. Check that the SQL performs the stated conversion, the test rejects an out-of-range value, and the required null treatment is present. Show how you would explain one consumer value from that evidence.

dbt question-led teaching — Portfolio Analytics

The dbt project uses the supplied Portfolio Analytics inputs on dbt-databricks/Astro. Use these foundation questions to frame the lesson. Explanations and all thirteen questions are in the project, where you can move directly into the tasks. Follow a question link into the practical task.

1. What problem does dbt solve, and where does it fit in a data pipeline?

Read the explanation and project task.

2. What does a dbt project contain, and how do we connect it to our data?

Read the explanation and project task.

3. How does a SQL query become a dbt model?

Read the explanation and project task.

4. How do we connect models using source() and ref()?

Read the explanation and project task.

5. How should we split a large query into understandable, reusable models?

Read the explanation and project task.

6. When should a model be a view or a table?

Read the explanation and project task.

7. How do we know our models produce trustworthy data?

Read the explanation and project task.

8. How do we make a change confidently and explain the result?

Read the explanation and project task.

Git and the paved path: a recap

The paved path is Capital Group's supported tools and conventions for product development: Git/GitHub, Docker, dbt, Airflow, Databricks and Harness, with Catalyst and documentation alongside.

> Knowledge check: History and execution
>
> What can Git tell a reviewer about a change? What separate configuration decides which service or database the changed code can reach?

Git: versioning the data product

Git is a version-control system for code. It's a bit like a wiki for code, where you can go through the history of edits to a file. Software engineers want three feature sets from it:

1. The log — a history of changes. Engineers make mistakes constantly and need to undo them by walking back through the edit history.
2. Branches — multiple simultaneous versions of a product.
3. Sharing — push and pull: distributing the code between people and machines.

Why use a software-engineering tool for a *data* product? Historically a data product might have been an Excel file, an Access database, or a SQL file. Odd things to put under Git. The supplied data-product

projects use dbt, whose transformations are SQL and configuration under version control. That project choice does not describe every CG data product. So to version data products and test them, we use Git the way software engineers do, on the code of the data product.

If you're comfortable with SQL but new to the software-engineering side, versioning and sharing SQL via Git may feel unusual. It's standard engineering practice, applied to data.

Git vs GitHub, and the pull request

A Git repository holds files, commits and branches. GitHub hosts repositories and adds collaboration workflows, including pull requests and review checks. Git also works locally without GitHub.

You don't have to use GitHub. Git works fine on your own laptop or on CG infrastructure. Why pay for a proprietary hosted system?

- Collaboration — a watering hole: a standard place everyone knows how to use to share code.
- Workflows — GitHub adds workflows that are not part of Git itself, and the key one is the pull request: a GitHub feature (not a Git feature) that puts in front of an approver the request to take one version and make it the released version. Technically it requests a merge; conceptually it is an approval request to promote one version of the software, usually to become the release. That review flow is one reason the supplied development process uses GitHub; integration and environment promotion remain separate decisions.

The line: development vs live

Separate development artefacts from their execution. Git versions the code; Docker packages dependencies. Airflow schedules data tasks and Databricks executes the configured SQL. The branch, deployed version and connected runtime are separate identities.

- Databricks is a platform. From our point of view, it's where the *data* side of the data product lives: where products are discovered and the actual tables can be accessed. Everything above the line is just code that runs and does something, and the something it does is create tables in Databricks. (Caveat for later: the data doesn't literally live on Databricks. It lives in S3 on Amazon. But for working purposes, Databricks is the release target.)
- Airflow — everyone says "orchestration", but what does that mean? Automated sequencing: a task runner and scheduler. If you remember Windows Scheduler from the Windows 98/XP era, it's that idea; if you've used Linux, it's essentially a cron wrapper. It defines, *in code*, when to run a task, on what condition, how often, and what to do if it fails. Airflow is the system that actually triggers the data product's code to run; the code, when run, modifies Databricks.
- Harness is a wrapper around all of this, which we leave until later in the week. For a data creator, detailed knowledge can stop before Harness.

What the data creator actually does

Creators mainly author and review dbt mappings, models and tests. The platform supplies the supported environment; engineers may manage orchestration and deployment. The assigned Airflow phase also asks you to author a small DAG. Retain a reviewed diff and checked output, then use the agreed handoff rather than assuming a push completes the product.

Python appears in Airflow and supporting helpers. The amount you author depends on the selected route: the assigned v2 Airflow activity writes a small Python DAG, while platform examples may use prepared configuration.

On screen: Repository anatomy

The folder path tells you which product and dataset a file belongs to, which a screenshot of one SQL file can't. A product folder holds a related collection of datasets, and a dataset can hold dbt, contract, runtime, Airflow and bundle files. Bundle YAML declares the jobs and resources. Check separately whether those jobs are actually deployed.

> **Knowledge check: Read a repository**
>
> Which files describe the product, execute it and configure the runtime? How would you decide where an unfamiliar file belongs before editing it?

Open the Portfolio Analytics and CG product trees. Follow each project's actual paths and model names. The infrastructure repository contains Terraform, Helm and pipeline definitions managed by the platform team.

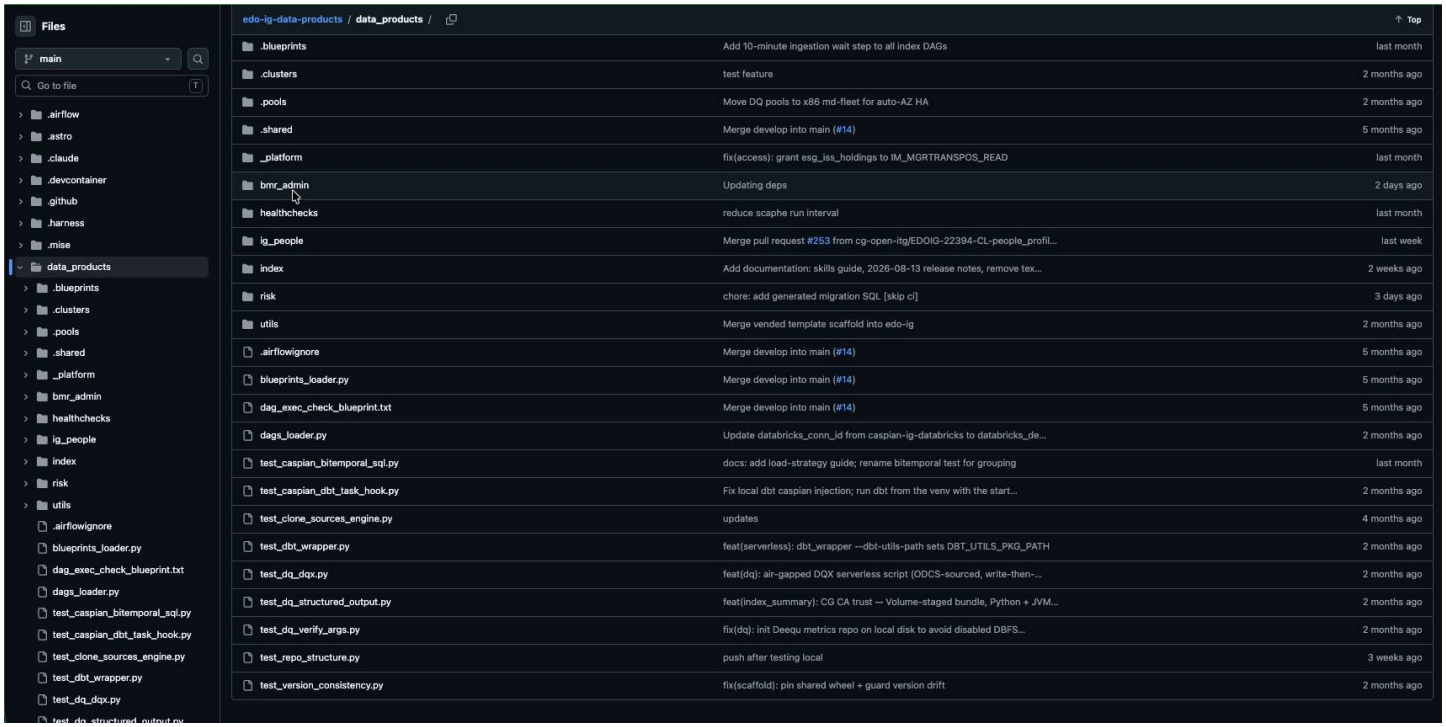


Figure 42: data_products folder

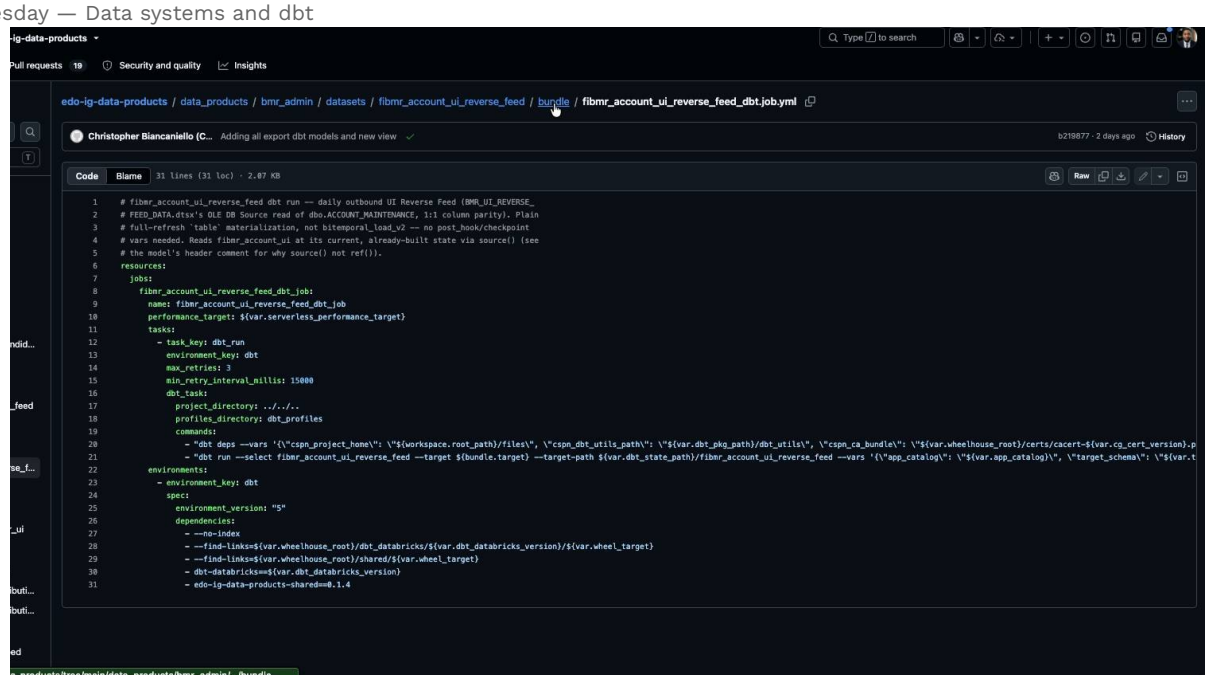


Figure 43: Bundle YAML

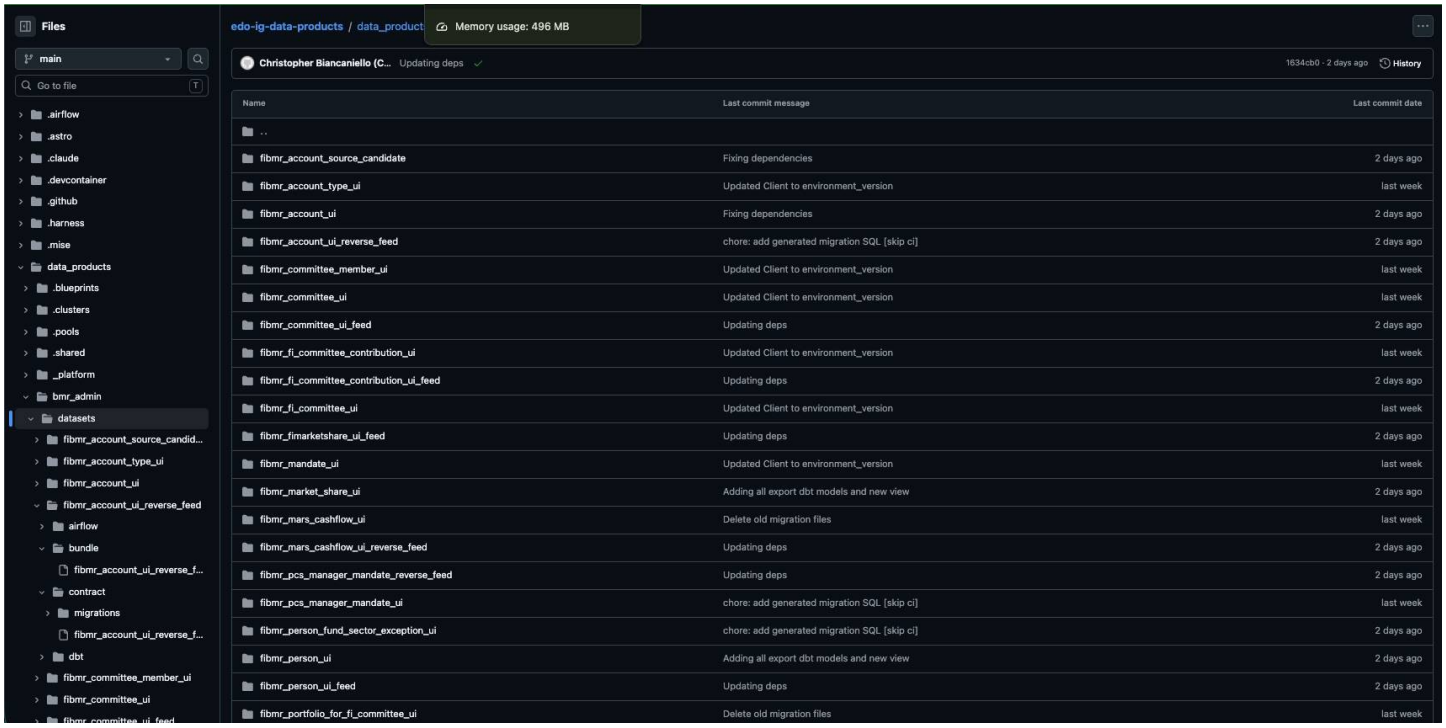


Figure 44: Datasets within a product

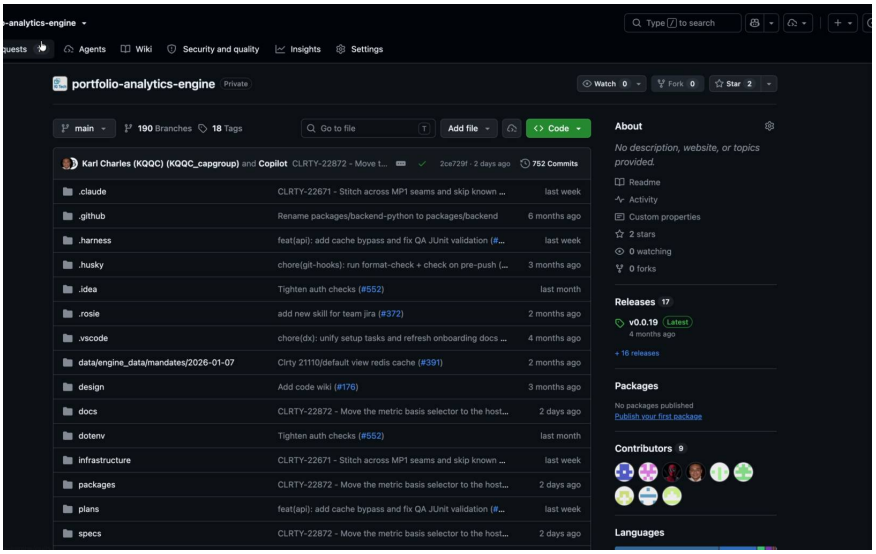


Figure 45: An infrastructure repository

On screen: The pull-request workflow

A pull request proposes a change against one base branch. Pick the wrong base and you can hide relevant work or pull in unrelated commits. The title and description should say what the problem was, what changes and how you checked it, and the Files changed view shows the actual code. A line comment ties your question to a specific line. Replace vague feedback such as “something is wrong here” with the violated requirement and the evidence or correction needed.

> ### Knowledge check: Read a proposed change
>
> What evidence should a reviewer inspect before approving a pull request? What does an approval leave to runtime and product validation?

Comment, Approve and Request changes mean different things in the workflow. Automated checks and the mergeability message cover technical conditions. A green check might only be a syntax check or a narrow test, so a human reviewer still asks whether the changed business rule meets the contract. For an aggregate model, ask for the reconciliation and the grain reasoning.

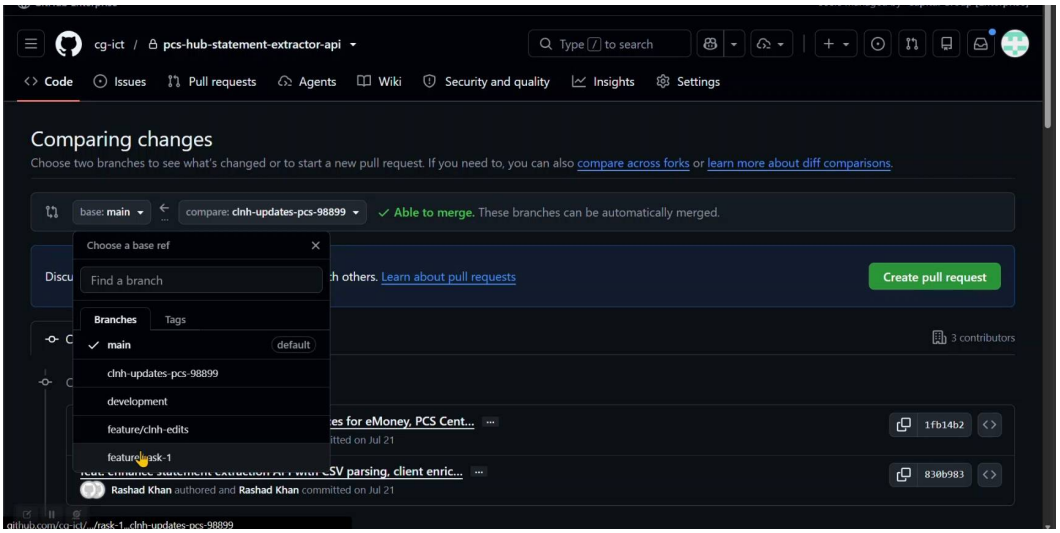


Figure 46: Choosing branches

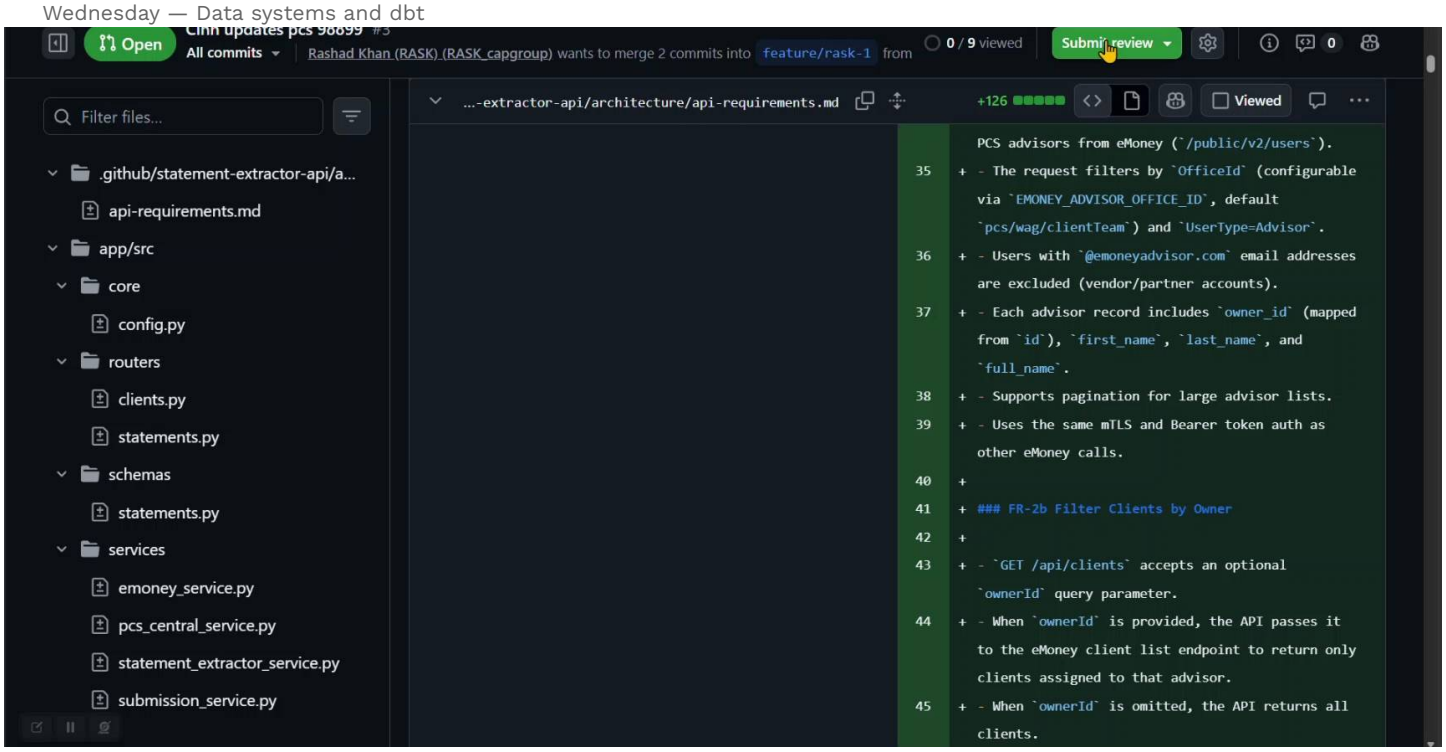


Figure 47: Files changed

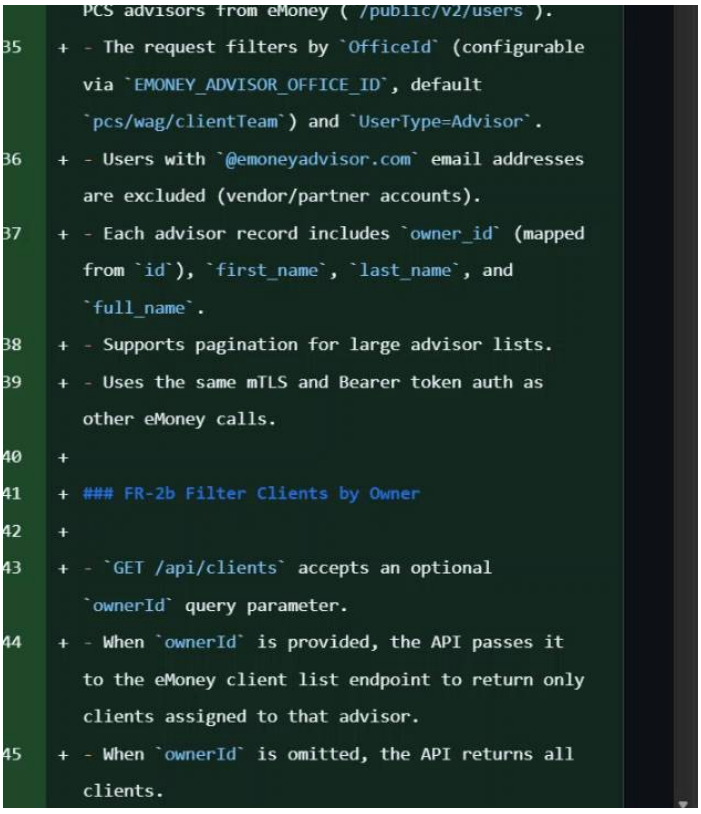


Figure 48: Detail of Files changed

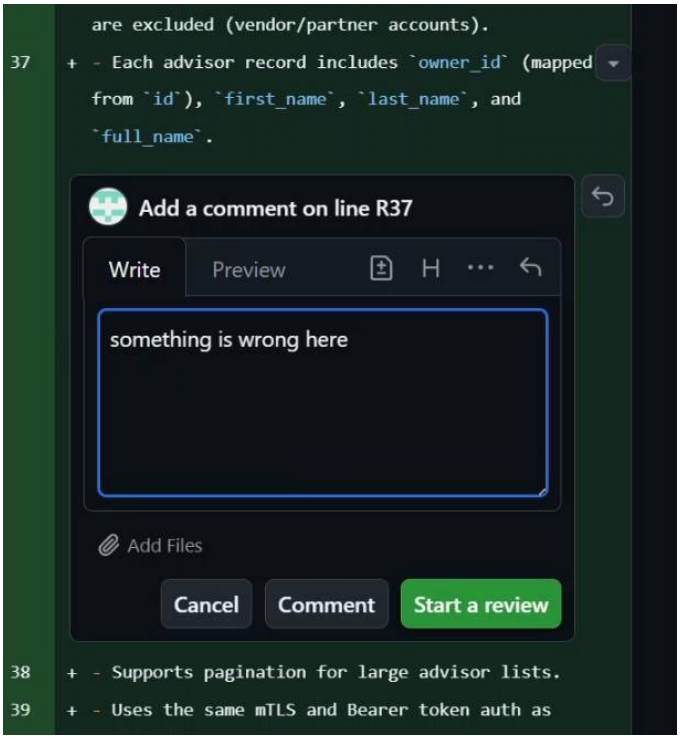


Figure 49: A line-comment draft that needs a specific reason

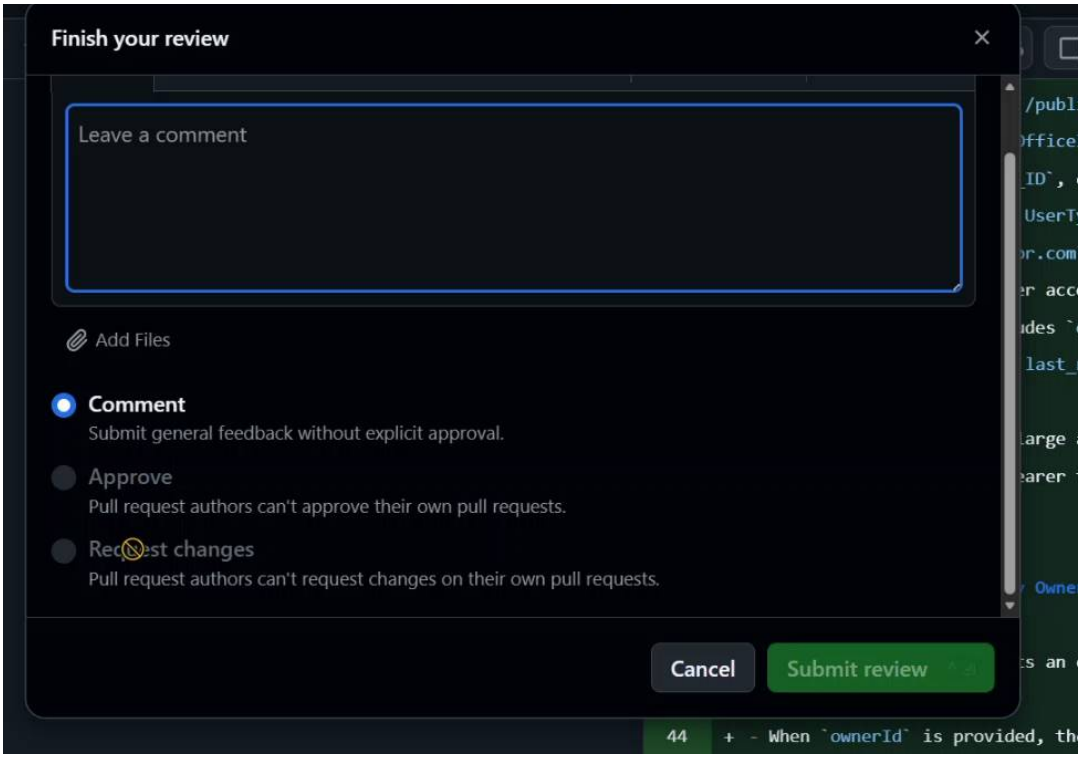


Figure 50: Finishing a review

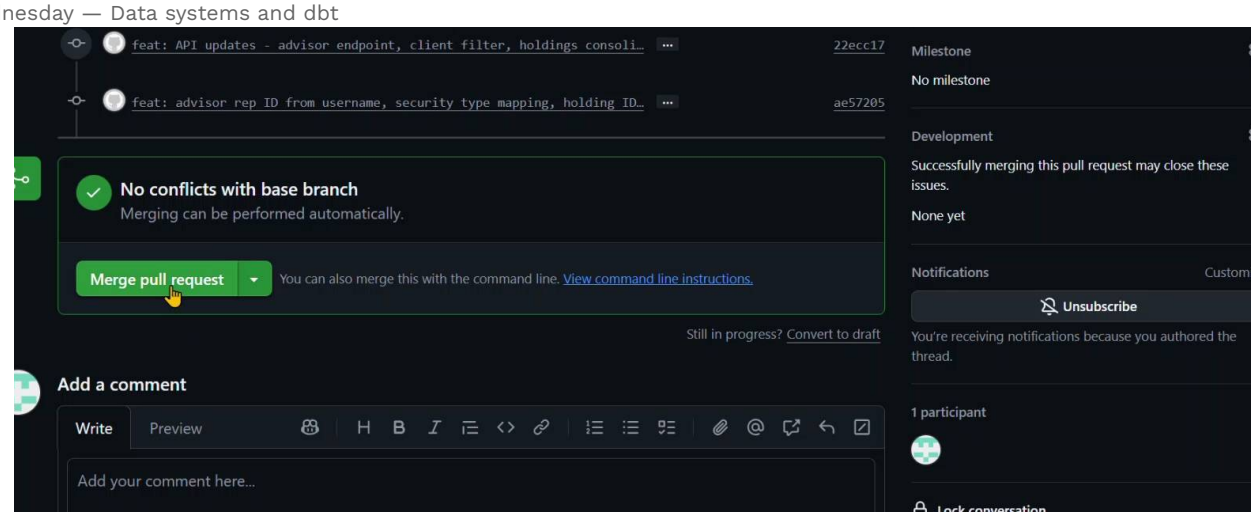


Figure 51: No conflicts with the base branch

Collaboration preparation: a reviewed Git change

Use a separate copied local training repository. Git Phase 1 owns the working-tree, staging, commit, branch, conflict and restore tasks; Phase 2 is a separately timed collaboration clinic with authorized remote access and a reviewer. Both are linked in the task tables below.

Knowledge check: A scoped commit

What is the difference between editing a file, staging it and committing it? How would you leave an unrelated file out of a reviewable change?

Read a small change before committing

OUTPUTS A scoped diff, commit and explanation

After the instructor has prepared the Git workspace, inspect its status and history. Create a local branch, make one intentional change to the supplied contract, inspect the diff and stage only that file. Commit with a message describing the changed promise. Read the committed change and explain why unrelated files were excluded. Use the owning Git Phase 1 instructions for exact commands and evidence.

A branch isolates history, not a database or service. Do not run a change against shared resources just because it is on a new branch. The complete 60–90-minute Git phase belongs to preparation or a clinic, outside the protected dbt block.

Learning Review

Retrieve the roles of the working tree, index and commit. Apply them to a contract edit plus an unrelated notes file: what belongs in the commit? Check the committed diff contains only the intended promise, the remaining status is understood, and you can name the execution target separately from the branch.

Foundation git

Preparation / collaboration clinic. Allow 60–90 minutes (estimate). Open the phase instructions.

Task	*Evidence*
Inspect the three working states	First commit contains only contract.md; notes remain outside that snapshot
Commit an intentional contract change	Commit diffs distinguish 09:00 UTC deadline from escalation policy
Branch and combine compatible work	Both separated facts survive; history explains fast-forward or merge commit
Resolve an ownership conflict	Expected merge failure; no markers; resolution retains owner and escalation decisions
Restore a mistake and explain the final history	Clean lab; committed requirements unchanged; explanation distinguishes restore from revert

Advanced git

Advanced reference / independently prepared, separately timed. Allow 75–110 minutes (estimate). Open the phase instructions.

Task	*Evidence*
Clone and inspect a collaboration boundary	Designated private sandbox, correct clone root, remote and tracking branch
Push and propose a scoped pull request	PR URL/head hash and remote diff contain the intended deadline clarification
Respond to a review with a second commit	Other reviewer/comment, second commit and remotely visible response
Integrate concurrent upstream work	Upstream ancestor and retained classification/deadline/ escalation facts
Read the final review and merge state	Actual final state, current head and next authorized action

Collaboration clinic: a merge conflict

In a separate collaboration clinic, use the disposable v2 Git workspace and an agreed reviewer. Stage conflicting changes to the same lines only after saving the work you intend to keep. This clinic has its own time allocation; it does not interrupt dbt practice.

Distinguish two failure states:

- Machines with uncommitted local edits fail before any merging starts: `error: Your local changes to the following files would be overwritten by merge`. Nothing is conflicted. Git stops early to protect your uncommitted work. The exit is the safe sequence from the top of this block. Run `git status`, then commit or `git restore`, then pull again.
- In the prepared collaboration clinic, competing branches edit the same contract lines, so their merge produces a deliberate conflict. Now both sides have changed the same region, and the file itself gains markers: `<<<<<<< HEAD` above the local lines, `=======` between, `>>>>>>>` below the incoming ones. Git has written both candidate versions into the file and paused the merge, because choosing between them is a human decision.

`git status` is the map throughout. It names the files in conflict and lists the commands available, so read it before touching anything. Then the two exits:

1. Resolve — edit the file to the version you want (usually the incoming fix with your change re-applied on top), delete the three marker lines, `git add`, commit. This is the exit you take on a team repo, where your local change is real work.
2. Discard — only when your local change is scratch work you don't need, and you've saved anything you want to keep. `git merge --abort` returns everything to the moment before the pull. Then drop your local change, pull cleanly and redo the exercise step against the fixed file.

Docker: the problem containers solve

> Code that runs on your laptop often breaks on a colleague's, because their machine has different versions of everything underneath. Docker is how the industry deals with that.
>
> **THIS SESSION ANSWERS**
>
> - What problem does Docker solve?
> - What is a container, and how is it different from a virtual machine?
> - Why run more than one container?
> - How do you describe and build a container?

> Knowledge check: The container boundary
>
> How does an image differ from a running container? Which resources and files come from the host, and what would you inspect when the app stops?

Before reading the definition, distinguish an image, a running container and its host. Describe where the process runs and what filesystem it can see.

A definition must distinguish a container from a zip archive: the image packages files, while the runtime starts processes with configured filesystem, resource and network boundaries.

Start instead with the alternative. Here's a laptop (or any environment, such as a remote machine). It has a set of key resources: RAM (the temporary, ephemeral memory), disk (persistent storage), CPU, and a network connection. Suppose I have my git repository with some code (say my dbt code), and I just run it directly on my laptop. What's the problem?

1. Drift. There are lots of different laptops in the world, with different background environments and different things installed. Even if two environments start from the same position, they eventually drift apart. Code that runs on my machine may not run on yours.
2. Encapsulation (call it security). The code in my git folder can access anything else on my system, such as passwords, configuration and other areas. And it cuts both ways: if the dbt project holds Capital Group-specific keys and passwords, a virus on my machine can access those too.
3. Resource management. The operating system doesn't give very good control over how many resources a piece of software uses. A misbehaving application can consume everything and make the machine unusable. There's also a diagnostic angle: if I declare that an application requires a certain amount of disk and RAM, then if it's using more than that, something is wrong with my software.

So what we want is a uniform environment across many machines, an encapsulated environment where everything inside the bundle can see itself but nothing outside, and carefully managed resources, so we understand what our application actually requires.

The definition

> Expert
>
> Docker is both the container (the bundle) and the runtime, the program that runs containers. Here's the definition we use.
> A container is an isolated process or group of processes running from an image, with its own filesystem view and configured access to host resources. The image supplies user-space files and libraries; it does not contain a running kernel. To *run* a container is to load that simulation into a runtime (the Docker runtime), inside which your application runs.
> An image packages user-space files and libraries. A Linux container shares a Linux host kernel; Docker Desktop provides that host through a VM on Windows or macOS. It is not a complete simulated laptop.
> When we run a container, we tell Docker how many resources to allocate: this container has no network connection, this much RAM, 16 GB of disk, one CPU thread.
> The image is the layered bundle; a container is one running instance of it. Removing that instance leaves the image available to run again, while changes made only inside the removed container do not become part of the image.

Containers vs virtual machines: the syscall explanation

This sounds like a virtual machine. So what's the difference?

- A machine (your laptop, a server) has CPU, RAM, and storage.
- A virtual machine is software running on a machine that *pretends* to have a certain amount of CPU, RAM, and storage, plus a complete copy of an operating system and everything else. A radical level of isolation.
- A container is like a virtual machine, but it does not fully simulate the operating system. It simulates having some CPU, RAM, and disk, but when it needs to do something like open a network connection, it asks the host operating system to do it. It shares a large amount of functionality with the host OS.

Mechanically: Linux provides syscalls, a set of functions the operating system offers (opening a file, for example). A container simply asks the host operating system to run its native syscalls. A virtual machine runs a guest kernel, which provides those system calls to guest applications; the hypervisor virtualises the underlying hardware. Since all Linux systems open files basically the same way regardless of distribution, containers don't need to simulate that layer.

So a container is *less isolated* than a virtual machine, and that's fine, because in most real deployments people run Linux containers on Linux machines, where the container's OS and the host's OS are very similar if not the same. The sharing is reasonable and adds very little overhead. The physical machine is called the host, and the containers running on it are guests. Because each container is lightweight, you can run many containers on one host cheaply.

Two distinctions help interpret the example.

- *Can you run two different versions of Linux as two containers on one host?* Yes, generally. Different distributions share the same syscall interface, so the host can serve both.
- One caveat: if you're on Windows and you want your system to fail the way *Linux* fails, you sometimes want a Linux virtual machine. Running Linux containers on Windows (or macOS) can throw up errors you wouldn't see running Linux containers on a Linux host.

Container networking

Suppose we have multiple applications on the same system and want them to talk to each other. Between two containers on the same laptop we establish a network connection: this container exposes a port (say 8000), that one exposes another (say 9000). Think of a port as an in-tray/out-tray inside each container. The containers are allowed to swap messages via those trays over a pretend network connection between them.

So on one laptop we can have multiple containers, each with a custom network configuration, seeing and communicating with only the specific containers we configure. That gives an entire live environment on a laptop with relatively high security: things inside the environment can't see the laptop, and we have fine-grained control over which containers talk to each other.

Why would a data creator need multiple containers?

Examples: multiple Airflow instances, multiple dbt instances; multiple branches of a feature; multiple product lines, with one team making feature one of a data product, another team making feature two, both live at once on the same server, each team seeing its own version.

More generally, multiple containers are incredibly common. Say you have a login service and a user-profile service, and the login service needs to request something from the profile service. They run as different containers. Why? Often because one team looks after the profile service and another the login service, and they release their containers separately.

The structure of software is decided by institutional factors more than technical factors. Technology tends to follow the organisation and structure of the business. (Databricks is often chosen mostly for institutional reasons too.) With one team, you wouldn't necessarily split things this way.

Development and production benefits

For development:

- Reproducibility — you can copy and paste your entire environment, cheaply. Copying a virtual machine (VirtualBox, VMware) means a very large image that is slow to start and stop; containers are much faster because they share the host OS.
- Everything installed at particular, pinned versions.

For production (live environments):

1. Isolation / security. On one Linux host you might run two data pipelines and a reporting/UI layer. You want them fully isolated, not talking to each other and not seeing each other's files. If an attacker gets into one environment, they don't automatically get into the others.
2. Modularity and encapsulation, which also stops teams tripping over each other. The big problem in software is complexity. If each piece of software runs in its own small environment, teams can't casually see, use or modify another team's code and files. Encapsulation means a capsule around things (barriers and protections), and modularity means reuse and composition.

Then came the Docker revolution. Around 2015–2016, before Docker took hold, developers ran lots of virtual machines locally, which was slow, while the live environment (where performance matters) looked completely different. One of Docker's big wins is similarity between dev and live, because containers are efficient enough to use in both.

Inside a container

Docker plays two roles on the paved path:

> Knowledge check: Editor and runtime
>
> Where do files, shell commands and data processing live in your development environment? Which of those locations would survive recreating the container?

1. Development — the container bundles all dependencies so code runs anywhere (the dev container).
2. Release — to make software live is to deploy the container. We release the bundle itself, so the development environment mirrors the production environment.

What's inside a container

An image is a packaged filesystem built from layers; a container is an instance created from it. The recipe commonly specifies:

1. Operating-system user space — commonly Linux distributions and their libraries. The container uses the kernel of its Linux host; Docker Desktop supplies a Linux VM when necessary on Windows or macOS.
2. System packages
3. Your code (copied in, plus dependencies)
4. The command — the one command that runs when the container runs

From the outside we treat a container *as if it were just that command*. If the command were `2 + 3`, running the container would be an absurdly expensive way to get 5. OS, packages and code, all to run one sum. So the commands are things worth the overhead: a dbt run, an Airflow server.

How the OS layering actually works

Docker itself includes client and daemon components. Docker Desktop normally runs Linux containers in a Linux virtual machine on Windows or macOS. When the containerised code asks the OS to do something, the Linux container runtime makes system calls against its Linux host kernel; Docker Desktop uses a Linux VM on Windows/macOS rather than forwarding Linux system calls directly to a different host kernel, which is why containers are so much cheaper than virtual machines (and why Windows quirks can leak through into your "Linux" container). The system-call distinction explains why an image's user-space files differ from a complete virtual machine; keep that distinction when reading the deployment examples.

Does a data creator need to build containers?

The platform supplies the standard development image. The EDO repository you cloned *is* the standard container, prepared by the EDO team for data product development. It has everything you need, and it's the one container you really deal with. You create dbt code inside it and submit to Git; the engineering process that deploys containers to live systems (Harness) is outside the data creator's scope.

But awareness matters: you'll sit in meetings with dashboards, logs, and errors from these systems, and knowing how they work helps you integrate with the teams deploying your code. (And besides, Docker is one of the tools on the course, so we teach it.)

Runtime controls: the other reason Docker matters

When running a container you can allocate its resources from the host: RAM (temporary memory), storage (long-term), and CPU/compute. Take a laptop with 32 GB of RAM, 8 cores and 1 TB of storage. The container is told which slice of that it may have. At its simplest a container does two things, compute (`2 + 3`) and run a command (`dbt run`), and the limits decide how much of the machine that command gets.

- Locally, resource limits can keep one development workload from consuming the host's available capacity.
- In production it's the main event: physical servers run *many* containers, and resource limits keep applications well-behaved. If a container capped at 2GB of RAM starts demanding more, something is wrong with *it*, and it fails alone instead of starving the host and bringing the system down.
- Networking is the most important control: containers can be restricted to talk only to the containers they're allowed to. Traditional development installs everything on one machine where every programme can see every file. In a live system with high-value data, you assume you *will* be hacked, so a compromised container that's forbidden from communicating with anything else has strictly limited impact. That's defence in depth. Put up as many barriers as you can and keep the blast radius small.

What is a *dev* container?

A dev container is a Docker container plus a VS Code feature (plugin) that runs VS Code inside the container. VS Code starts itself inside the container as if the container were your laptop. Your terminal executes inside the container (no need for `docker run -it` to get a bash prompt), files you create are created inside the container, and every command picks up the pre-wired configuration. It's not touching your host OS.

What's in the EDO dev container:

- Mise (short for *mise en place*) — a cross-language meta package manager: one tool, configured once, that drives system packages, pip, npm, Maven and the rest. The project configuration pins tools and names its setup tasks.
- Astronomer — Airflow wrapped in a container… inside our container. Containers within containers is strange and somewhat fragile; more on why on Thursday.
- Databricks CLI — needed by dbt, and by Catalyst: when Catalyst runs a query for you, it goes through the Databricks CLI.
- Docker (inside Docker) and dbt.

Where did your work go?

One consequence of "your terminal executes inside the container" is worth a minute, because it determines whether a rebuild preserves your work. A container's writable layer is thrown away when that container is removed or replaced. Stopping and restarting the same container normally keeps it. Rebuilding or recreating the container is what usually loses it.

The exception is the workspace mount. The dev container maps your cloned repo folder on the laptop into the container (the `workspaceMount` setting in `.devcontainer/devcontainer.json`), so files in that folder live on the host and survive any number of rebuilds. In raw Docker the same mechanism is a volume flag:

```
docker run -v /Users/you/edo-repo:/workspace my-image
```

That line runs the container and makes my laptop's `edo-repo` folder appear inside it at `/workspace`. Writes under `/workspace` land on the laptop; writes anywhere else land in the container and die with it.

Check where every edited file was saved before removing or rebuilding a container. Files under the mounted repository survive; files in an unmounted home directory or `/tmp` may not. Verify an agent's reported path in the filesystem.

Reading a Dockerfile

A Dockerfile is the build recipe. Read its instructions to distinguish the base image, installed dependencies, copied files, runtime user, working directory and startup command. The example includes these common instructions:

- `FROM` — which image to start from. An image is a blueprint/template for containers (ten containers can share one image). Here: the official Airflow image, so we begin from a template that already contains Linux, Airflow, and its libraries.
- `LABEL` — tagging the container with metadata about what it's for.
- `RUN apt-get update …` — updating/installing system packages (`apt-get` marks this as Debian/Ubuntu Linux).
- `USER airflow` — switching to the airflow user.
- `WORKDIR` — setting the folder we'll copy things into.
- `COPY requirements.txt` + `RUN pip install …` — the Python dependency list, copied in and installed.
- `COPY …` — the application code.
- `CMD` — the one command the container runs: the Airflow server.
- `EXPOSE 8080` — opens the networking: "you're allowed to plug a cable into this container on port 8080." Strictly, `EXPOSE` only records the port. The actual mapping from container port to laptop port happens when the container is run, with `docker run -p 8080:8080`, which is why, on Thursday, the Airflow UI running inside a container appears in your browser at `localhost:8080`.

Anatomy of a Dockerfile

How do we describe and build a container? With a Dockerfile, a file that specifies how the container is built from its layers. It's built up line by line: each line (typically, not always) builds a layer in the bundle. Walking through the example Dockerfile in the challenge material:

> Knowledge check: Build and run
>
> Which Dockerfile instructions affect image contents and which select the running command? Why could a build succeed while the app still fails?

- `FROM python:3.11-slim` — the base image, and a very unusual name when you're new to this. A base image supplies the filesystem layers your image extends. The name reads as if we're installing a Python package; we're not. We're importing an entire Linux system (for a `-slim` image, Debian Linux) with a full set of system packages and a Python 3.11 environment. The image is simply *named after* the one key package it carries, to tell the reader that Python is the point of this container. A less confusing name would be "Linux + Python 3.11". Starting points are named after the thing they're most useful for: there are base images for Airflow, for dbt, for git. `FROM` some git image would mean "load an entire operating system with git installed".
  - *Where do base images come from?* A registry, a store of published images that `docker pull` fetches from, layer by layer. Docker Hub is the public one; large companies typically run an internal mirror of the images they've approved, and the paved-path images come from CG's own infrastructure rather than the open internet (the specifics belong to the platform team). Pulling layer by layer explains two things you may see today: the *first* build on a machine is slow, because the base layers have to come down the network (simultaneous first-time pulls can saturate a shared connection), while every later build is fast, because the layers are cached locally. It also explains a familiar failure: a VPN that intercepts traffic can break pulls, because the connection to the registry doesn't trust the VPN's certificates. Which is the practical argument for the FROM line you'll actually meet at CG: a CG-approved base image

with the SSL certificates already baked in, so everything built on top of it can talk through the corporate network without each team re-solving the certificate problem.

- `WORKDIR /app` — our working folder; all subsequent commands run from `/app` and copy things into it.
- `COPY` — copies everything from the local location into the app folder within the container (there's a slightly strange interaction between the WORKDIR and COPY lines that we won't unpick here).
- `CMD python summarize.py` — the command Docker runs when the container starts: once the operating system and everything else is established, this is what runs at the very end.

That's the whole example file. Three more instructions turn up in almost every production Dockerfile, so learn to recognise them:

- `RUN pip install -r requirements.txt` — a command executed *while building*, its result frozen into a layer. This is where dependencies get installed: RUN happens once, at build time, and everyone who runs the image afterwards gets the already-installed result. (CMD is the opposite case. Nothing happens at build time, and it's executed at run time.)
- `USER appuser` — up to this line, everything in the container runs as root. Switching to an unprivileged user before the application starts means that if the application is compromised, the attacker gets an unprivileged account inside an isolated container. It's the encapsulation argument from earlier again. Production Dockerfiles almost always do this; toy examples usually skip it.
- `EXPOSE 8080` — a documentation line. It records which port the application intends to listen on and opens nothing. As in the first walkthrough, `docker run -p 8080:8080` does the actual mapping at run time. (The in-tray/out-tray from the networking section is created by `-p`; `EXPOSE` only labels the tray.)

Treat the versioned image and its runtime configuration together: the image packages the pipeline dependencies, while mounts, credentials and reachable external services determine what the running process can use.

Will you ever write one? Mostly no, as above. The supplied project includes a configured Dockerfile. For live work, confirm the current base image, certificates and support responsibility with the assigned platform team. A data creator reads Dockerfiles, and dbt is the core skill.

The eight commands: live demo

You've been using Docker Desktop since the environment setup. The command line does the same operations without the buttons. Here are the eight commands, run once on screen against the example container, with the Docker Desktop tab each one matches:

- `docker build -t summarize .` — read the Dockerfile in this folder, build the layers, tag the result `summarize`. This is what fills Docker Desktop's Images tab.
- `docker images` — list every image built or pulled onto this machine: the CLI form of that same Images tab.
- `docker run summarize` — load the bundle into the runtime and execute its CMD. (The example container runs its script and exits, so for the `exec` step below start one that stays alive: `docker run -dt summarize bash`.)
- `docker ps` — what is running right now; add `-a` to include containers that have stopped. The Containers tab.
- `docker logs <name>` — everything the container has printed. When a container dies, this is the first place to look.
- `docker exec -it <name> bash` — a shell *inside* the running container. Run `cat /etc/os-release`: it prints the Linux distribution, and there is the operating system the theory promised. You are on a

Mac or Windows laptop, and inside the container is whichever Linux the base image chose (the file tells you which, so you don't have to guess from the image name). Look around; `exit` to leave.

- `docker stop <name>` — shut the container down.
- `docker rm <name>` — remove the stopped container. The image stays (`docker rmi` removes that), so stopping and removing costs nothing you can't rebuild.

The challenges use only these commands, and tomorrow you'll diagnose the Astronomer environment with `ps`, `logs` and `exec`.

Kubernetes: infrastructure as configuration

Kubernetes coordinates deployed containers through declared desired state. It controls where workloads run, how their instances are grouped and how unhealthy instances are replaced. This explains part of the platform behind the release demonstrations; the assigned local Docker and dbt activities do not require cluster provisioning.

> Knowledge check: Workload orchestration
>
> What does a shared platform need to coordinate beyond one local container? How do workload placement and data-task dependencies differ?

Orchestration coordinates separate work. Airflow orders tasks and observes their execution; Kubernetes places and manages container workloads. The two graphs solve different problems. A model dependency does not tell Kubernetes which nodes to use, and a Kubernetes deployment does not establish a data-quality rule.

A pod is Kubernetes' unit for one or more closely related containers that share specified runtime resources, including a network namespace and configured volumes. Group containers according to their lifecycle and communication requirements; do not put unrelated products in one pod just because the same team owns them. The scheduler places pods on nodes, while controllers reconcile declared configuration with observed state.

For a product platform, configuration can describe a workload's image, replicas, resource requests and limits, environment, storage mounts, service access and recovery behavior. Review that configuration alongside the application's expected files and connections. A container can be healthy as a process while its output is incomplete or late; the data promise still needs separate checks.

A local Docker command starts a scoped instance for development or diagnosis. A shared cluster needs placement, isolation and recovery across many instances and teams. The platform team owns that infrastructure boundary. When you review a deployment demonstration, identify which image/version is selected and what evidence shows that the workload is ready. Do not treat the diagram as permission to deploy.

Versioned configuration makes a change reviewable: inspect the diff, required settings and chosen environment before applying it. It can help reproduce a workload in another prepared environment, but portability depends on external services, identity, storage and infrastructure capabilities. Kubernetes does not by itself provision every cloud account, network or managed service; tools such as Terraform address infrastructure responsibilities shown separately in the platform material.

> Expert
>
> Amazon EKS is a managed Kubernetes service; ECS is a separate AWS container service. The supplied platform examples discuss both and a Capital Group managed-platform direction called Helix. Use current internal platform documentation and the responsible architecture team to choose an approved service; a historical screenshot is not a current approval list. Harness and deployment conventions can vary by team. The creator's

> task is to understand the configured runtime boundary and evidence, while the platform role owns cluster/service provisioning.

Foundation docker

Supervised introduction / runtime clinic. Allow 60–90 minutes (estimate). Open the phase instructions.

Task	*Evidence*
Run an image and inspect the lifecycle	Hello output, actual image/container identity and exit status
Explore an interactive shell	Inside file disappears in a fresh disposable execution; version/OS evidence
Use a bounded bind mount	Actual fixture summary and expected write denial; host input unchanged
Author and run a simple holdings image	Authored image emits independent six-holding result
Prove repeatability and clean scoped resources	Two matching outputs from one image identity; only owned resources removed

Advanced docker

Advanced reference / independently prepared, separately timed. Allow 75–110 minutes (estimate). Open the phase instructions.

Task	*Evidence*
Separate named build targets	Actual target image IDs and expected target outputs
Gate the runtime on a tested artifact	Test failure blocks accepted runtime artifact path; repaired test/app succeeds
Run with a bounded runtime user	Runtime UID 10001; app reads intended files and still prints expected summary
Keep runtime contents focused	Required app/input present; test script absent; expected output preserved
Repair a path assumption and explain the result	Predicted missing-file failure followed by repaired expected output and explanation

Docker practice and diagnosis

The Docker task tables above identify the current copied workspaces, prerequisites and completion evidence. Read the Phase 1 concepts before the supervised introduction. Preparation and full phase practice use their separately scheduled time; Docker advanced work does not replace Wednesday's dbt allocation. Images, the approved Docker runtime and any required corporate access must be prepared before the learner block.

> Knowledge check: Diagnose before changing
>
> What can a build result, exit status, log and filesystem observation each establish? Which would you inspect before changing a path in app code?

Explain the container result

OUTPUTS Observed output and an execution-boundary explanation

Inspect a prepared Docker Phase 1 run. Identify the image, the running or stopped container, the app's input and the location of its output. Explain the difference between the valid starter message and a completed holdings summary. Use the full task instructions for the exact workspace and expected values; a host Python check is supporting evidence, not proof that Docker built and ran the image.

Optional clinic: diagnose a path failure

Docker task Repair a path assumption and explain the result owns this deliberate experiment. It starts from the supplied Phase 2 baseline and changes `WORKDIR` to `/wrong` in a disposable Dockerfile copy. It does not depend on an undocumented broken-container folder.

Read the build result, then inspect the stopped container and its logs. A successful build can still leave the default app command running in a directory where its files are absent. A stopped container cannot accept `docker exec`; inspect its configured directory or start a new diagnostic instance with a shell. Repair the path and repeat both named-target checks. Retain the failed output, repaired result and runtime-user evidence specified by the owning task. This is advanced clinic/reference work and requires Docker execution.

Parallel discussion: map the runtime boundaries

Write six labels: host computer, dev container, separate Astro Airflow runtime, dbt code, remote Databricks and remote storage. In pairs, draw where the editor terminal runs, which process reads the code and where connected SQL executes. Label the optional local follow-up separately from the assigned Databricks route.

The training configuration starts an inner Docker daemon from the dev container; Astro uses it to launch separate Airflow services. The developer shell and Airflow worker have different processes and environments. Remote Databricks and storage are outside the local Docker runtime. Offline file checks do not establish remote execution. Compare your map with Getting started and name one configuration or path that must be available in the process that uses it.

Learning Review

Retrieve the differences between an image, container and mounted workspace. Apply them to a build that succeeds followed by an app that cannot find its input. Check that your proposed diagnosis uses the exit state, logs and configured path before changing app code, and that you distinguish a host test from an actual Docker run.

Portfolio Analytics setup

Follow Readiness and the complete dbt project. The assigned adapter is dbt-databricks; SQL runs on the approved warehouse. Record source loading, versions and namespace before authoring.

Foundation dbt

Assigned Wednesday; selected core within 165 minutes. Allow 75–110 minutes (estimate). Open the phase instructions.

Task	*Evidence*
Inspect the project and source grain	Eight holdings, four funds, two business dates; safe identity and seed inventory
Declare and use sources	Compiled source relations and manifest dependencies agree with all raw seeds
Normalize type, unit and key contracts	Eight unique dated holding keys and native decimal values; a controlled zero-valued holding retains its key; restoration recovers baseline.
Build a useful mart through ref	Eight fund/date rows; independently derived GBP exposure and holdings value per synthetic unit
Add minimum key and relationship proof	Structural/business tests pass; unknown fund fails membership for its intended reason; restored seed/model/output passes again.

Advanced dbt

Advanced reference / independently prepared, separately timed. Allow 90–120 minutes (estimate). Open the phase instructions.

Task	*Evidence*
Diagnose a join fan-out before repair	Eight input keys; date-free price join yields sixteen matches; dated repair retains eight
Repair enrichment at the intended grain	Eight enriched keys; explicit matched-price/FX coverage and decimal GBP value
Write reconciliation that detects the negative case	Coverage and NAV tests fail on copied missing-FX input and pass after restoration
Prove incremental update and replay behavior	Older-date +1,000 GBP correction retains eight unique keys; +500 GBP unseen holding inserts a ninth; both correction and insertion replay without duplication; restore owned eight-key baseline
Explain dependency impact and hand over	Declared fund report, meaningful snapshot intervals, lineage and explicit Astro boundary

`dbt build` and `--select`

> Knowledge check: Selection and failure
>
> Which resources does a selector include? How would an upstream failed test affect a dependency-ordered build?

`dbt run` builds selected models; running tests afterwards leaves a gap in which downstream models may consume unchecked upstream data. `dbt build` runs selected resources and tests in dependency order and can skip dependent work after a required test fails. Inspect the selected resources and test severity before relying on that boundary.

In production, `dbt build` is usually the command the scheduler runs, so the contract's tests run as part of every build.

Use `dbt list` to inspect a dependency selection before building it:

```
dbt build --profiles-dir . --select stg_holdings       # one authored staging model
dbt build --profiles-dir . --select stg_holdings+      # that model and its descendants
dbt build --profiles-dir . --select +fund_exposure    # the authored mart and its parents
dbt list --profiles-dir . --select +fund_exposure     # inspect that dependency slice first
```

Tomorrow: the same pattern for Airflow (an introduction in the same style, then a long self-paced challenge block), plus a look at Databricks and any remaining technologies.

Review the model and its evidence

After Phase 1 practice, connect the authored models to their mapping and checks. Review a generated model against its intended population and grain before moving to orchestration.

THIS SESSION ANSWERS

- What does dbt add to a SQL query?
- Which tests detect lost keys, changed units or duplicated joins?
- What does a dependency edge establish, and what remains a business requirement?

Knowledge check: A result worth accepting

What could be wrong with a model that builds successfully? Which checks would test its intended grain and population rather than repeat its own total?

Case study: A successful build with an incorrect population

Situation. A fund-exposure model builds successfully and produces a plausible total. The lookup table contains two versions of one fund key, so enrichment duplicates a holding. A consumer asks whether the reported exposure can be used for a limit decision.

Decision. Check the declared join relationship and compare raw key membership and per-fund totals before accepting the build. Repair the selected lookup grain, then run the negative case and the independent reconciliation again.

Reasoning. The model's own total and a downstream total can agree because both inherit the same duplicate. A green command reports execution; independent evidence checks the intended business result.

Lesson. Review the population, grain and failure treatment together. This case prepares the generated-model review below; advanced fan-out repair belongs to Diagnose a join fan-out before repair–Write reconciliation that detects the negative case in the separately timed clinic.

Repeatable development and small tests

A useful development loop makes a small change, executes it in an isolated workspace and inspects the result. Git retains the version and diff; dbt selects the model/test scope; the prepared local runtime keeps one learner's experiment from overwriting another's result. Each tool makes the evidence easier to inspect without deciding whether the business rule is correct.

Tests alongside a build

Tests shorten the interval between an incorrect change and its diagnosis. Choose a test for a plausible failure: a required identifier, duplicated row key, invalid category, orphan relationship or wrong total. Keep a known negative case so the test demonstrates that it can fail. Run the selected model while it is unfinished, then widen the build to include all required upstream and downstream checks before handoff.

Layers and reusable transformations

Staging preserves a source's meaning while normalizing names, types and units. Intermediate transformations express reusable joins and calculations. A mart supports the consumer's stated decision and grain. Use the layer and alignment table to explain both technical structure and business purpose; a folder name alone cannot establish either.

Dependency edges and compiled relations

`ref` and `source` resolve configured relations, and references between models declare the dbt dependency graph. They let dbt order a selected build and let a reviewer trace a source-to-consumer path. See the dependency explanation and macro example. A dependency edge identifies what a model reads; it does not establish key uniqueness, freshness or correct units.

Reference question: history and bitemporal requirements

A history requirement must name the time it describes. Business-valid time answers when a fact applied; record/system time answers when the system learned or stored it. A requirement to reproduce both perspectives needs an explicit bitemporal design. A dbt snapshot preserves changing records through validity intervals; do not assume that one snapshot interval automatically implements both timelines. The snapshot and history reference introduces observed-state history. Historical data and bitemporal loading develops two-clock corrections, master and balance grains, supplier-file policies, and recovery. Its temporal-history clinic is optional and separately timed; the protected dbt practice block remains focused on the assigned Portfolio Analytics tasks.

Reference question: reusable SQL macros

A macro turns a named reusable operation into compiled SQL. For example, a shared date conversion can make a legacy field rule explicit in several models. Inspect the compiled query and test representative inputs; hiding repetition in a macro does not validate the transformation. Macros and incremental/history variants are advanced reference topics, outside minimum Phase 1 completion.

Choosing a transformation tool for the workload

dbt is designed around analytical transformations and their dependencies. A low-latency transactional service needs a workload decision covering concurrent updates, application interfaces and latency. Do not treat an analytical model as an application's transactional write path solely because both use SQL. Record the operating requirements and involve the appropriate engineering role.

Review challenge: find the defects in a generated model

> Knowledge check: Review the specification
>
> Which mapping decisions would you trace into the proposed SQL first? What would convince you that its prose summary describes the actual query?

No code is written in this challenge. You get an STTM extract and a model presented as agent output, and you mark every place the model breaks the mapping. Work in pairs, ~30 minutes; you only need to read SQL, so non-technical participants pair with technical ones and interrogate the STTM side.

The brief

The mapping is an illustrative holdings contract, not a verified Morningstar or CG schema: an integration model taking staged holdings and enriching them with security reference data. The target table's grain is one row per fund, per security, per as-of date.

Source table.column	*Target column*	*Transformation rule*	*Null handling*
`holdings.SecId`	`security_id`	rename	reject row to error table — never silently drop
`holdings.FundId`	`fund_id`	rename	reject row to error table
`holdings.Weight`	`weight`	rescale 0–100 to 0–1 (divide by 100); negative weights are valid — they are short positions	reject row to error table
`holdings.AsOfDate`	`as_of_date`	rename	reject row to error table
`security_ref.Sector`	`sector`	left join on `security_id`; unmatched securities get `'UNCLASSIFIED'`	—

The agent produced the model below, with this summary: *"I've implemented the integration model as specified: renamed the columns, rescaled weights to 0–1, handled nulls, and joined the sector classifications. All rows are preserved at the fund/security/date grain."*

```
with holdings as (
    select * from {{ ref('stg_holdings') }}
),

security_ref as (
    select * from {{ ref('stg_security_reference') }}
)

select
    h.fund_id,
    s.sector,
    sum(h.weight / 100)   as weight,
    max(h.as_of_date)     as as_of_date
from holdings h
join security_ref s
    on h.security_name = s.security_name
where h.weight > 0
group by h.fund_id, s.sector
```

Group challenge

OUTPUTS A defect list

Mark every defect against the STTM, and separately mark every claim in the summary you can show to be false. Don't fix anything.

Solution: page 146

Choose a test or review for each defect. Key-membership and representative-row tests can detect lost short positions or wrong joins; reading the SQL against the mapping explains their cause. No single uniqueness or not-null check establishes the whole contract.

Optional enrichment: Catalyst's dbt skills

Use this 35-minute enrichment only when the required Phase 1 evidence is complete and the instructor has reserved that time within the practice block. Otherwise take it as a separately timed clinic or follow-up. The generated-model review and day-ending handoff keep their scheduled allocation.

> Knowledge check: A bounded generated change
>
> What context should a dbt skill receive, and what remains your responsibility after it creates files? How would you check a generated test against the original promise?

Brief

Ask Catalyst which installed skills create or modify dbt projects. Inspect the actual instructions and required outputs in the supplied version before selecting one. Candidate skill names in the reference include the following; availability must be checked in your environment.

- `setup-dbt-project`
- `write-dbt-model`
- `write-dbt-source` (declaring catalogue/source references)
- a dbt troubleshooting skill
- `dbt-on-databricks`

> Group challenge
>
> **OUTPUTS** A reviewed generated change
>
> The challenge takes about 35 minutes. Ideally, use the setup skill to create a new dbt project. If that hasn't worked after 15 minutes or so, fall back to running the more specific skills against your existing dbt project, the one you know works. You need to end up with a new model, and a new project is optional. If you finish early, add tests.

Practical checks before requesting a generated change:

- The setup skill *walks you through* creating a new project rather than silently overwriting an existing one.
- Files Catalyst creates may land in odd places (e.g. under the Catalyst artifacts folder). The Catalyst folder is just a folder of files, so keep your projects somewhere else.
- A selected model may interpret the same skill/context differently across runs. Inspect whether it asks for missing facts, changes the requested scope or chooses an unsupported default. Judge the observed change against your contract and evidence rather than a vendor stereotype.

Amendment: verify with git before you accept

After Catalyst has generated or modified models, and before you accept, commit, or build on anything, run two commands in the project folder.

- `git status` lists every file the agent created, modified, or deleted. Compare that list to the agent's own summary of what it did. The summary is generated text like any other output, so it has the self-reporting problem from Tuesday's AI-agents talk. Trust the file list over the summary.
- `git diff` shows the exact line-by-line changes. Read them. The specific task in this amendment is to find one thing the agent did that its summary did not mention, such as a renamed column, an edited `schema.yml` entry, a deleted test, a changed materialization. In practice there is nearly always at least one.

And when the diff shows a change you don't want: `git restore <file>` puts a modified file back to its last committed state, so rejecting an agent's change costs one command. (Files the agent newly created aren't in git yet, so they are removed by deleting them; `git status` tells you which case you're in.)

Cases: reviewing a generated change

Case study: A skill invocation with incomplete context

Situation. Two creators request a dbt model using the same skill. One supplies the project, source fields and expected grain; the other leaves those decisions unstated. The resulting changes take different paths, including clarification questions or guessed file locations.

Decision and reasoning. Inspect the context, requested output and actual diff before comparing the results. A skill supplies instructions to a probabilistic model; its name does not guarantee identical behavior. Clarify the missing fact and rerun a bounded request.

Lesson. Retain the prompt/context and the reviewed change. Do not equate a skill invocation with a completed requirement.

Case study: Fast generation increases the review workload

Situation. An agent creates a large model set and converts an Excel input into CSV. It also reduces the sample to 1,000 rows. The build succeeds, but the consumer expected complete coverage.

Decision. Confirm which transformations and sampling limits were requested. Review a small representative change first, compare the full expected population and retain excluded-row evidence before expanding the build.

Reasoning and lesson. File conversion, successful execution and plausible summaries do not establish coverage. A small training sample can be useful when it is explicit; silently applying it to a product changes the promise. The creator owns the units, membership, tests and handoff limitation, including review effort when generation is faster than inspection.

Learning Review

Retrieve the distinction between skill instructions, supplied context and generated behavior. Apply it to a model that builds successfully after dropping part of its input. Check `git status` and the complete diff against the requested files, population and tests. Name one independent result that could reject the agent's summary and one unresolved assumption with an owner.

Primary variant: the contract-to-tests round trip

Use the group's own design artefacts instead of a generic new model. Feed Catalyst the data contract your group generated with Catalyst on Tuesday and ask for two things: the `schema.yml` tests that enforce its data-quality section, and the source `freshness` configuration that enforces its arrival SLA (freshness itself is covered on Thursday morning). Then review the output against the contract, sorting every promise in the quality and SLA sections into three piles:

- Promises that became tests. Key rules become `unique` and `not_null`, and categorical rules become `accepted_values`. Range rules (exposures between 0 and 1) become a range test, and the arrival SLA becomes `warn_after` and `error_after` on the source.
- Promises that were mangled in translation. Typical finds: a completeness threshold of 99.5% silently turned into `not_null`, which demands 100%; a range rule attached to the wrong column; a test placed on a staging model when the contract's promise is about the published table. Each one is plausible YAML that enforces something other than what was agreed, the same kind of defect you looked for in the review challenge.

- Promises that cannot be tests at all. Duty to notify, duty to explain, availability, the escalation path. These are agreements between people, and no YAML enforces them. They stay in the contract document, so check the consumer has agreed them.

Review question: which contract promises became executable checks, and which still need a named operational or business owner?

A dbt exposure declares the downstream dashboard or report that consumes a model and extends lineage to what the stakeholder reads:

```yaml
- name: morningstar_exposures_dashboard
  type: dashboard
  depends_on:
    - ref('rpt_morningstar_exposures')
```

It builds no data. `dbt build --select +exposure:morningstar_exposures_dashboard` selects the exposure and everything upstream.

Wednesday review: from rule to evidence

> Learning Review
>
> Retrieve the product's source grain, intended mart grain, unit conversions, model dependencies and test failure treatment without reopening the guide. Explain what Git records, what the dev container supplies and where the assigned Databricks result was computed.
> Apply those concepts to a lookup that suddenly has two rows per fund key. Predict the effect on holdings count and exposure. Choose a key/membership check and an independent per-fund reconciliation, then describe the smallest correction and evidence you would retain.
> Check your answer against your own attempt: the models/tests implement the same promise as the mapping; all eight holdings and eight fund/date report rows across four funds and two dates are accounted for; the zero-value holding retains its stated meaning; a known negative case fails; the repaired output and explanation identify the actual execution runtime. Record incomplete authoring or a blocked connected gate with its next action and owner. A successful baseline or command summary is not the finished consumer handoff.

Where should a transformation live?

Place a transformation where the relevant domain knowledge, governance, reuse and maintenance responsibility can support it. A consumer-private calculation in one report may belong with that consumer; a shared governed calculation needs a documented definition, accountable owner and approved implementation. Moving everything upstream does not automatically improve either arrangement.

Ask who can explain the rule and its edge cases, who must consume it consistently and who will handle a change. Time zones, valuation dates and corrections can become clear only when a consumer sees examples. If a request for many derived fields reduces after the logic and purpose are documented, that is useful discovery rather than an implementation failure.

Optimise the handoff and decision boundaries as well as the SQL. Reproducing a consumer calculation in another team without its context adds work and can change meaning. Conversely, leaving a shared rule in several reports can produce inconsistencies. The final output must serve the consumer, while the owner needs enough pipeline and lineage evidence to support it. See data-mesh responsibilities.

Solutions: Wednesday — Data systems and dbt

Review challenge: find the defects in a generated model Back to the section

Generated-model review — defect key

Defect key and evidence limits

1. Wrong join key, silently dropping rows. The STTM joins on `security_id`; the model joins on `security_name`, a name field, which will not match cleanly between a vendor feed and internal reference data. And it is an *inner* join where the STTM specifies a left join with an `'UNCLASSIFIED'` default, so every unmatched security's holdings vanish from the output with no error, no log, no rejected row.
2. A filter that deletes the most important rows. `where h.weight > 0` removes short positions, which the STTM declares valid. Tuesday's AI-agents talk warned about this kind of script, one that does everything right except delete the rows you need tomorrow.
3. Missing null handling. The same predicate makes null weights disappear silently (`null > 0` is not true, so the row is filtered). The STTM requires nulls to be rejected to an error table; nothing in this model routes anything anywhere. The rows are simply gone.
4. Grain broken by a stray GROUP BY. The target grain is fund × security × date; the `group by h.fund_id, s.sector` collapses it to fund × sector. `security_id` does not even appear in the output, and `max(h.as_of_date)` papers over any mixture of dates in the input.
5. The summary is false in checkable ways. "Handled nulls"? It deleted them. "All rows are preserved"? The inner join and the filter both drop rows. "At the fund/security/date grain"? The GROUP BY changed the grain. One thing it did do correctly: the rescale (`weight / 100`) matches the STTM. A reviewer who checks only that line will pass the model.

Return to the mapping and generated-model brief.

Thursday — Orchestration and operations

Treat internal service names, deployment screenshots, GCC architecture and routing controls here as supplied operating cases. Confirm current topology, access, approvals and operating policy with the responsible platform/product owner. The local Astro exercise does not establish live CG deployment or production readiness.

Thursday's plan

> This morning you draw a pipeline from your own work on flip-chart paper. This afternoon you build the same kind of flow in Airflow, around Wednesday's dbt models.
>
> **THIS SESSION ANSWERS**
>
> - What has to happen first in a pipeline, and what can run at the same time?
> - Where do the checks go, and what happens when a file is late?
> - How does dbt measure whether the data arrived on time?

Use the Airflow project for the complete Portfolio Analytics tasks and checks on scheduled Astro workers with Databricks.

So what does Airflow actually do? It decides when your jobs run and in what order, and it keeps a record of every run, so you can see what failed and run it again. Today you use it around Wednesday's dbt models. The main practice orders source readiness, the same Portfolio dbt graph, quality and publication evidence on scheduled Astro workers.

Working time, 30-minute guest	*Focus*
09:00–09:15	Readiness and guest questions
09:15–09:45	Astronomer guest
09:45–10:30	Dependencies, runs and tasks
10:30–10:45	Morning break
10:45–12:00	Cosmos, freshness and UI walkthrough
12:00–12:45	Lunch
12:45–14:45	120 minutes protected Airflow practice
14:45–15:00	Afternoon break
15:00–15:20	How releases work, and the platform underneath
15:20–15:30	Recovery debrief and Friday starting-state gate

Both guest variants run from 09:00 to 15:30 and preserve the same lunch, breaks, 120-minute Airflow block and closing gate. The longer guest reduces the dependencies discussion and walkthrough; it does not extend the day. Use the prepared example to keep those shorter demonstrations focused.

Working time, 60-minute guest	*Focus*
09:00–09:15	Readiness and guest questions
09:15–10:15	Astronomer guest
10:15–10:45	Dependencies, runs and tasks
10:45–11:00	Morning break
11:00–12:00	Cosmos, freshness and UI walkthrough
12:00–12:45	Lunch
12:45–14:45	120 minutes protected Airflow practice
14:45–15:00	Afternoon break
15:00–15:20	How releases work, and the platform underneath
15:20–15:30	Recovery debrief and Friday starting-state gate

Today you choose the checks for a pipeline, write its order, timing and recovery into Airflow, and run dbt one model at a time. The contract and tests from earlier in the week turn into a DAG, an alert, an on-call response and a runbook.

Today's sessions

- Group challenge: drawing your data pipeline as a flow
- airflow question-led teaching — Portfolio Analytics
- The paved-path scheduling recipe on one page
- Airflow challenges: setup, known issues and walkthrough
- Guest presentation (optional): a production data product end to end
- After the Airflow practice
- Data operations: the tools around a product
- Guest demo: a real paved-path data product, end to end
- How does a data product get released?
- Group exercise: approve or request changes?
- Deployment environments at Capital Group
- Containers in deployment
- Kubernetes and the layers of orchestration
- Harness, indirection and Conway's law
- The cloud
- Reading a release pipeline
- The 3 a.m. failure sequence
- Operations reference
- Thursday review

On screen: Thursday preview

The supplied screenshots cover orchestration, release and platform operations. Their menus and identifiers may differ from your assigned environment. The required learner execution is v2 Airflow Phase 1; cloud provisioning, live releases and production monitoring remain demonstrations. Follow each row to its explanation and walkthrough.

System / sequence	*Purpose and teaching*	*Walkthrough*
GitHub pull requests	Review a change and its merge evidence: Main teaching	Screens
Lambda notifications	Follow an arrival event into a DAG trigger: Main teaching	Screens
Amazon MWAA	Identify managed runtime, DAG location and UI entry: Main teaching	Screens
Airflow DAG discovery	Locate the correct product/environment before diagnosing: Main teaching	Screens
Airflow graph and grid	Read dependencies separately from execution states: Main teaching	Screens
Airflow task instances	Tie logs and retry attempts to one execution: Main teaching	Screens
Airflow fan-out	Distinguish starting child runs from their completion: Main teaching	Screens
Astronomer / Astro	Connect a managed deployment to its Airflow runs: Main teaching	Screens
AWS sign-in and roles	Check account/role/environment access boundaries: Main teaching	Screens
Harness projects and pipelines	Find the release definition and execution history: Main teaching	Screens
Harness Git import	Trace pipeline YAML to repository/branch/path: Main teaching	Screens
Harness YAML	Compare declarative stages with an actual execution: Main teaching	Screens
Harness promotion and approvals	Read environment transitions and paused gates: Main teaching	Screens
Harness execution inputs	Check branch/input set/target before a release: Main teaching	Screens
Harness logs	Locate the failed release stage and its command result: Main teaching	Screens
Terraform configuration	Read infrastructure resources and environment parameters: Main teaching	Screens
Terraform Cloud runs/state	Compare plan/apply history and managed resources: Main teaching	Screens
Helm charts	Map values to Kubernetes deployment configuration: Main teaching	Screens
Datadog and Retina	Separate service health, data freshness and job/cost evidence: Main teaching	Screens
Kubernetes / Lens	Inspect deployed workload identity, state and probes: Main teaching	Screens
Kubernetes pod specification	Compare requested resources/configuration with status: Main teaching	Screens

Group challenge: drawing your data pipeline as a flow

Brief

Form a mixed group and choose a pipeline from your work. Use its datasets, delivery promises and consumers to explain the sequence.

Knowledge check: What does an arrow promise?

Which part of a pipeline diagram describes data movement, and which part describes a prerequisite? Which missing input would make you stop publication?

Group challenge

OUTPUTS A flip-chart flow of your pipeline

As a group, choose one real pipeline that someone at your table works with and draw its flow on flip-chart paper. Use the scheduled morning activity slot; reserve the final part for comparing dependencies and checks. Answer these on the drawing.

- What must happen first? What can happen at the same time? Which steps must wait on others?
- Where should checks happen between steps?
- What happens if something is missing or late? What happens on success vs failure?

Use the drawing to review an Airflow graph later. Explain each dependency so colleagues with different roles can assess its business consequence.

Case study: Source readiness and delivery

Cases: source readiness and delivery

Integrated positions product. Sources: IMAP (positions) and Compass (the portfolio-master system, supporting different roll-ups such as account, segment and equity or fixed-income portions, to measure performance), plus reference data (asset classification, country, security details). Ingestions are daily: positions land ~9–10 PM, reference data ~8 PM. The goal is the final integrated positions table by ~12 AM (midnight) so downstream consumers can start. Daily SLAs were defined for IMAP and Compass availability. Data is staged into IPAC tables (IPAC acting as the centralised calculation engine), positions and portfolios are integrated (active portfolios holding positions as of a date), first/last-trade calculations run on the integrated set, and a file publication is broadcast to Caspian and on to consumers.

Checks are split into **in-motion rules** (duplicate checks, null and primary-key checks and contract conformance, which stop the pipeline if they fail) and **at-rest rules** (value checks, e.g. return percentages must sum to 100%, weight percentages in range). In-motion failures are rare and usually system issues; when they happen, whoever is on call gets paged. On missing/late data, a notification goes to a wide distribution list of consumers of the data. Checksum/file-hash validation detects a changed transfer. In this example it belongs to the NFS transfer boundary, while row-level rules belong to ingestion and curation.

Advisor-level view. Building an integrated advisor-level view (a row = a 360° view of an advisor: AUM, activity involvement, etc.) from individual data sets already standing and QA'd in Caspian, plus benchmark survey data to be added. Mostly daily (some weekly) SLAs; the pipeline waits on all of them before triggering. Staging, then joins/integrations/transformations, with completeness checks in motion and a proposed escalation process with the operations team when an SLA is missed. A useful discovery question is whether consumers need a combined product that the existing stable sources do not yet supply.

Market-close price delivery. After US market close (~1 PM Pacific), a third-party vendor publishes end-of-day prices into a shared Snowflake database. An internal Snowflake notebook runs a batch of queries and cross-references, writes the results to a CSV, and sends it downstream to consuming applications.

Morningstar ingestion contract. A pipeline design identifies the source systems and primary keys for data-quality purposes, then defined the data contract per data set (four Morningstar data sets, e.g. holdings) with DQ rules (e.g. completeness $\geq$ 99.5%, often set to 100%), dependencies between ingestions, arrival SLAs (holdings by 8 AM; prices/FX ingestion from 6:30 using T-1 close; benchmark data needed only for the later benchmark-relative/alpha calculations, so not blocking), schemas, and operational support when files are missing.

Checks run at both the ingestion and curation layers. Critical failures stop processing and alert consumers, and the pipeline is re-run after a fix. Non-critical checks run weekly for statistics without blocking the flow.

Diagram literacy: three kinds of arrow

Review a pipeline diagram by classifying every box as a task, rule or design decision, then label its arrows. Combining three different relationships without a key makes the diagram ambiguous:

- **data-flow arrows** (the dbt kind of arrow),
- **task-order arrows** (the Airflow kind of arrow),
- **dependency arrows** (the SLA/organisational kind).

Give each visual arrow type one meaning. A recovery arrow should return to the failed executable boundary, such as ingestion, rather than to an earlier design activity. Identify the owner, critical gates and intended restart step before writing the DAG.

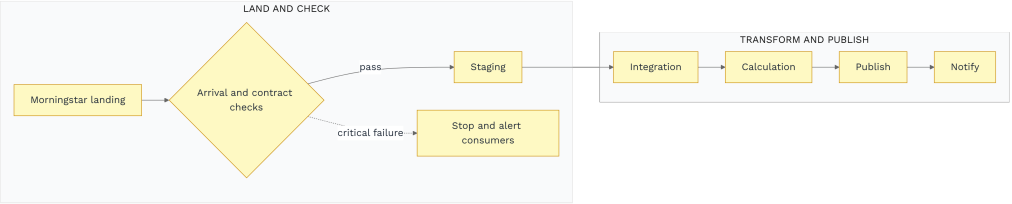


Figure 52: Morningstar task order: arrival and contract checks precede transformation. Critical failure stops the run and alerts consumers; successful checks allow calculation, publication and notification.

Source freshness as an SLI

The Morningstar contract includes a measurable promise: holdings arrive by 8 AM. Wednesday defined the SLI as the indicator you measure to know whether you're hitting the agreement; dbt ships one for arrival SLAs, and it takes four lines of YAML on the source declaration:

```yaml
sources:
  - name: morningstar
    tables:
      - name: holdings
        loaded_at_field: loaded_at
        freshness:
          warn_after: {count: 24, period: hour}
          error_after: {count: 28, period: hour}
```

`loaded_at_field` names the timestamp column written when rows land. Running `dbt source freshness` takes the newest `loaded_at` in the table and compares its age to the thresholds. For this example, a check after 8 AM against a previous delivery older than 24 hours identifies a late arrival. An age threshold is not by itself a proof of the business date: verify the expected delivery identity and timestamp as well. That's the `warn_after` line. Older than 28 hours means it's now badly late and downstream models shouldn't build on stale data. That's `error_after`.

The thresholds come straight from the contract's arrival SLA. That's why Wednesday's contract-to-tests round trip had Catalyst generate this block from the SLA section, and why you check its arithmetic yourself.

This afternoon, Airflow decides when things run. Source freshness measures whether the data arrived when it was promised. Airflow will happily start the 10 PM run on time whether the file arrived or not. Run `dbt source freshness` as the first step, and the pipeline notices the late file before it builds anything on top of it.

Learning Review

Retrieve the three arrow meanings without looking at the diagram: data flow, task order and an external delivery dependency. For a holdings feed due at 8 AM, place a source-arrival check before transformation and explain which tasks must stop when it fails.
Self-check: your drawing has one meaning per arrow style, a named input owner, a critical gate before consumption, an explicit restart step and an evidence check on the recovered output. Explain why a 24-hour freshness threshold alone may not identify the expected business-date delivery.

airflow question-led teaching — Portfolio Analytics

The airflow project uses the supplied Portfolio Analytics inputs on dbt-databricks/Astro. Use these foundation questions to frame the lesson. Explanations and all thirteen questions are in the project, where you can move directly into the tasks. Follow a question link into the practical task.

1. What problem does Airflow solve alongside dbt?

Read the explanation and project task.

2. What are a DAG, task, operator, run and task instance?

Read the explanation and project task.

3. Which components parse, schedule, execute and record the work?

Read the explanation and project task.

4. How do we define dependencies and pass information?

Read the explanation and project task.

5. What do schedules, logical dates, intervals and catchup mean?

Read the explanation and project task.

6. What should happen when a task fails or retries?

Read the explanation and project task.

7. How do reruns and backfills produce correct results?

Read the explanation and project task.

8. How do we inspect a run and prove completion?

Read the explanation and project task.

The paved-path scheduling recipe on one page

The afternoon's Airflow components fit into one repository laid out like this:

```
positions-scheduling/
├── Dockerfile             # FROM an Astro Runtime image (the CG-approved
│                          # base with certificates baked in, once adopted)
├── dags/
│   └── positions_dbt.py   # the DAG file below
├── dbt/
│   └── positions/         # the day-3 DBT project, vendored into the repo
├── requirements.txt       # astronomer-cosmos, dbt-databricks
└── .env                   # tokens and connection details — never committed
```

To run it locally you use the **Astro CLI**. `astro dev init` sets up this layout once. `astro dev start` builds and starts the containers from the `docker ps` exercise. Run `astro dev restart` after you change the Dockerfile or `requirements.txt` (edits to files under `dags/` are picked up without a restart). The UI stays at the address `astro dev start` printed.

The DAG file, in full, is the Cosmos form pointed at Wednesday's project, with the schedule line set from the SLA:

```python
from datetime import datetime
from cosmos import DbtDag, ProjectConfig

positions_dbt = DbtDag(
    dag_id="positions_dbt",
    project_config=ProjectConfig("/usr/local/airflow/dbt/positions"),
    schedule="0 22 * * *",      # the drawn arrival SLA, as one cron line
    start_date=datetime(2026, 1, 1),
    catchup=False,
)
```

Every model in the project becomes its own task that can retry by itself, and the tests run alongside the models they test. The retries and callbacks from earlier go in through `default_args`, as in any other DAG.

Harness releases this repository, and Astro runs what was released. That's two separate cycles, and the release session later today keeps them apart. One is the release: someone changed the code, so it gets reviewed, approved and deployed. The other is the run: 10 PM came round again. The running system only changes when a change to this repo is released. After that, the schedule line runs it every night with no human involved.

Airflow challenges: setup, known issues and walkthrough

Work through the assigned v2 Phase 1 route at a pace that leaves time to inspect its evidence. Catalyst can help explain or draft a change, but you still author or review the result and run the required checks. Experienced learners can work manually; advanced Phase 2 is a separate clinic or reference route.

The Airflow route adds run identity, logs, controlled failure and recovery to the dbt evidence from Wednesday.

> **Knowledge check: Establish the execution gate**
>
> What evidence would convince you the DAG actually executed dbt? What would you retain if local execution succeeds but the optional hosted UI is unavailable?

> **Group challenge**
>
> **DURATION** 120 minutes (12:45 to 14:45)
>
> **OUTPUTS** A running DAG
>
> Work through v2 Airflow Phase 1, retain the executed-run evidence, and check the resulting data. Use remaining time to explain the recovery behavior before choosing an extension. The closing challenge starts you off with your pipeline drawing.

For stretch material, look at the Airflow documentation, and the Cosmos documentation if you're interested in how to adapt Airflow further to run dbt.

The intended flow

- From the training project root, run `mise run challenge:v2:prepare airflow phase-one-certified`; the phase guide identifies the copied workspace.
- Open the v2 Airflow Phase 1 README in VS Code, choose Open Preview, and follow its five tasks in order.
- Execute the assigned DAG on the prepared Astro runtime and inspect its Databricks output. File parsing alone does not complete the phase. Prepare access and dependencies before the timed block.

Challenge zero: the Airflow UI tour

Once `astro dev start` has the local instance running, open the address it prints in a browser. In the challenge repo that's a per-project hostname on port 6563, and a plain Astro project uses localhost:8080. Everyone does this tour before writing or generating anything. Why? Because most data creators will read this screen far more often than they'll ever write a DAG. This is where you find out whether last night's run delivered.

- **Grid view.** Each column is one scheduled run and each cell is one task in that run. Green succeeded, red failed. A missed morning delivery is a red cell with a date on it, and you answer "when did this last work?" by scanning left.
- **Graph view.** This is your flip-chart diagram as the `>>` lines encode it, including which tasks sit in parallel. If the picture doesn't match what you meant to draw, the dependency line is wrong.
- **A task's logs.** Click a task instance, then Logs. That's the output of that one task for that one run. It's the first place to look at a red cell, and where a failed `dbt run` prints which test or model broke.
- **Pausing and unpausing.** The toggle beside the DAG name. A paused DAG doesn't get its scheduled runs. Triggering it by hand is a separate button, and what that does to a paused DAG depends on your Airflow version. New DAGs usually arrive paused, so "my DAG isn't running" usually means "my DAG isn't unpaused".
- **Clearing a failed task to rerun it.** Select the failed task instance and choose Clear. Airflow resets that task so the scheduler runs it again. Downstream tasks are only included if you tick them, so check the run, the task IDs and the options before you confirm. This is the everyday fix once the upstream problem is sorted, and repeat it only after checking the task's replay behavior and publication boundary. Verify that a repeat does not duplicate side effects and that the recovered business output matches the expected state.

If the environment misbehaves

When the prepared runtime fails to start, record the failing command and environment before applying a targeted repair.

- **Read the first error before rebuilding.** Check the configured runtime, dependencies, paths and service logs. Preserve learner files before recreating a container; remove only the identified stale training resources. A separate clean clone can help isolate a setup problem without discarding the existing attempt.
- Working out of OneDrive breaks permissions. Clone to the local file system (Documents or similar) instead.
- **Airflow does not really work natively on Windows.** Many operators use bash, which is a Mac/Linux tool, so on Windows you must use the container, which is what you're doing. One root cause found on Windows laptops was a `.env` file with Windows carriage-return line endings being parsed by bash; hidden characters differ between files written on Mac and Windows, and Tuesday's `core.autocrlf` setting is the fix.
- After a rebuild, rerun the documented settings setup and check the new terminal/worker configuration. Do not copy credentials into review evidence.
- For registry or certificate failures, retain the reported host and failing operation and use the approved image/network setup. Do not change the Astro base image merely because another laptop had a similar symptom.
- **Check where work is saved before stopping or rebuilding a container.** Files under the workspace mount survive a rebuild; anything written elsewhere inside the container does not.

Plan on roughly an hour and a half of self-paced challenge time.

Count the containers: what `astro dev start` actually started

With the local instance running, open a terminal on the host (not inside the dev container) and run `docker ps`. The challenge runs Airflow 3, so you should see five long-running containers. Match each one to its job:

- the **scheduler** — the part that decides what runs and when; the actual orchestrator;
- the **dag-processor** — reads the files in `dags/` and turns them into DAGs the scheduler knows about;
- the **api-server** — serves the UI you just toured;
- the **triggerer** — looks after tasks that are waiting for something (the deferred waits that sensors can hand over to it);
- **postgres** — Airflow's metadata database. Every run, task state and retry count lives here, and the grid view is drawn from it.

An older Airflow 2 project shows a **webserver** in place of the api-server, and no separate dag-processor.

Wednesday separated container roles by teams and product lines. Here's a second answer, running on your own machine: one application split into parts that work together. Astronomer's local runtime is a ready-made multi-container setup, and the development-container and Astro-runtime boundary remains explicit. The Astro CLI writes the container configuration and starts the set, which is the job a Docker Compose file does by hand.

Stretch, for those who did Wednesday's Docker stretch goals: find the Compose configuration the CLI generates and read the service definitions (the CLI's help output lists the export command for your version).

Closing challenge: generate a DAG, then interrogate it

This block ends the way Wednesday's dbt review did: an agent writes something and you check it. In pairs, run the **`generate-dag`** Catalyst skill against your group's flip-chart pipeline from this morning. Describe the tasks, their order, the schedule and what should happen on failure, and let the skill produce the file.

`generate-dag` writes YAML for the DAG's name, schedule, defaults, dependencies and generic tasks. A person or a template then picks the operators that actually get deployed, so open what the skill produced and read it.

DAG modes and output patterns

FROM CATALYST SKILLS

SOURCE skills/generate-dag/SKILL.md @ Catalyst 1.13.0

The supplied source says:

```
## Modes

| Mode | Schedule | MDS tags | Asset triggers | Validation |
|---|---|---|---|---|
| `prototype` | cron OK | optional | optional | lite |
| `production` | Asset-driven required (cron fallback with advisory) | all MDS required |
↪  required | full RFD 0011 |
| `lift-and-shift` | preserve existing | warn-and-add | preserve | full RFD 0011 |

See `references/mode-aware-emission.md` for the full mode matrix.

## Emission patterns

Three patterns. The skill auto-detects between the two YAML carriers; pure
Python is opt-in.

| Pattern | Detection | When chosen |
|---|---|---|
| **dag-factory** | Default | Repo has no `.blueprints/` directory OR user explicitly requests
↪  dag-factory |
| **astro-blueprints** | Repo has `.blueprints/*.py` + a blueprint loader | Team has the
↪  blueprint framework already in place |
```

```
| **python** | `pattern=python` explicit opt-in | Use when the DAG needs features YAML carriers
↪  can't express cleanly (multi-Asset triggers, dynamic task mapping, branching). |

If the YAML patterns are ambiguous, ask the user. See
`references/pattern-dag-factory.md`, `references/pattern-astro-blueprints.md`,
and `references/pattern-python.md` for the full templates.
```

The skill supports several patterns and production controls. Read these before the orchestration request so the offered capability is not mistaken for assigned course scope. The course follows its offered v2/Cosmos route; production callbacks, provisioning and alternative carriers remain outside that assignment.

Before you run anything, check it against this list:

- **Does the `schedule` match the SLA you drew?** Decode the cron line by hand. If the skill reached for `@daily`, remember that means midnight, and check whether midnight is what your drawing says.
- **Are there retries**, with a plausible `retry_delay`? Or does one dropped connection go straight to a failure?
- **Is there an `on_failure_callback`?** The supplied production convention routes a Datadog event to the responsible team. The assigned v2 local callback records the observation in its log; it does not configure Datadog or demonstrate an on-call page.
- **Is `catchup` set**, either way? If it's missing, the replay behaviour is a default nobody chose.
- **Do the `>>` lines reproduce your arrows?** Look for anything you drew in parallel that came out serial, and the reverse.

Correct the review findings and identify the emitted carrier: Python, dag-factory or an existing blueprint framework. The assigned v2 starter uses its provided Python DAG and local execution command. A generated YAML carrier is not executable merely because it looks plausible; keep it as reference unless the prepared runtime explicitly supports it. Optional extensions follow the core evidence gate and use an isolated training copy. Review the schedule, retries and failure behavior before any additional execution.

On the paved path, CI checks every DAG for `<domain>__<dataset>__<frequency>` naming, an `on_failure_callback`, all five MDS tags, no `email_on_failure` and at least one retry. `cimon` runs those checks on each change in Harness, and `scaphe` scans the whole estate for DAGs that have drifted. Check the platform docs for the current list. A DAG can run happily while failing all five. Missing callbacks and tags fail silently, because no routable Datadog event reaches on-call.

Reflective question break

Optional reflection: write down a concept you cannot yet explain, a term you cannot define and a product question from your work. Architecture decision records and the creator's role in requirements discussions are useful examples. Use these questions for the day review or the linked Q&A reference; additional discussion does not consume the protected execution block.

Review your use of AI and Catalyst

Review how Catalyst has affected your product work: which decisions improved, which assumptions needed correction, and which results you can now verify without relying on the agent.

What hints would you share? Write down the two or three you'd give a colleague starting this course next month.

Exploration versus exploitation. Is the request too broad or too narrow? Every session with an agent sits somewhere between exploring (wide questions, to find out what is possible) and exploiting (driving one known approach to completion). Both are right at different moments. The trouble starts when you don't notice which one you're doing. Go too broad and nothing ships. Go too narrow and you commit to the first plausible answer.

Check the review workload. Fast generation can create more assumptions and changes than you have time to inspect. If useful outputs arrive inconsistently, narrow the request and compare several representative results rather than accepting one impressive answer.

How can you correct the pattern? Two remedies. First, ask whether you're getting good results *consistently*. One impressive answer tells you very little. Second, simplify the output and focus on one thing at a time. In practice that means mixing Catalyst with manual work, which the Docker and dbt blocks already asked of you.

Four prompts worth keeping, because each one gets the agent to dig into the assumptions:

- "Ask me some questions about things that have been assumed."
- "Give me a plan to do X."
- "Explain to me what I need to know in order to plan to do X."
- "What are the things I'm missing?"

What is the role of the human in the loop? Tuesday introduced what humans bring that LLMs don't: purpose, judgment and taste. Tuesday's talk, "The Human Role When Working With AI Agents", turned that into a way of reviewing what an agent hands you. Now use the same habit on yourself. Notice what heavy use is doing to your own attention and assumptions, and correct for it.

What the client gate proves

Use `mise run challenge:v2:verify-starter airflow phase-one-certified` to check the supplied starting state. A baseline pass is not completion. The phase's connected verification needs fresh scheduled run/task identities, the controlled failure and repair, and independent Databricks checks.

Start the prepared Astro runtime and use the address it prints. Select your owned DAG, run and task instance before inspecting logs. The final `publication_evidence` task records a checked training handoff; it does not grant access or register a production marketplace listing.

Controlled failure, recovery and replay

Work on a disposable training copy and your own training schema. First run the working path and check the output yourself. Then break it the way the instructor shows you, and keep the original file so you can put it back. Note which task fails, what the log says and which tasks after it don't run. Those later tasks show as `upstream_failed`, which means the failure above them blocked them.

Fix the cause before you retry. Pick the DAG, run and task instances you mean, and read the clear dialog or command options before you confirm. A calendar-day range can include several runs; use the run/task identity to establish which execution the clear action should affect. Don't use an unscoped clear command as a shortcut. If a task writes or publishes data, work out what happens when it runs twice before you include the downstream tasks.

After the fix, check the task state and the data. A task can go green and still run the wrong query, or finish without doing any work. Compare the model outputs with the contract checks, then run it again and show the rerun doesn't duplicate or corrupt anything. Keep the run IDs and model versions so you can show which run produced what.

Tools you watch but don't configure

Some of today is about tools you'll see but won't touch: Datadog, Harness, Kafka, S3 and Lambda. On the paved path, a task that fails after its retries calls its `on_failure_callback`, which sends an event to Datadog, and Datadog routes it to a person. The challenge DAG has its callbacks in the same place, but they only write a line to the task log. The challenge runs Airflow 3, so its DAG has no `sla=` argument either. The two-minute practice timetable demonstrates scheduled execution. A production deadline needs its own business timezone, source-arrival promise and retry allowance.

Foundation airflow

Assigned Thursday; selected core within 120 minutes. Allow 75–110 minutes (estimate). Open the phase instructions.

Task	*Evidence*
Inspect a prepared orchestration baseline	Actual mount/executable identities, owned namespace and supplied-vs-authored inventory
Author a small Python DAG with real dbt tasks	Four meaningful tasks with explicit paths, target and actual warehouse checks
Demonstrate the dependency graph and scheduled run	Readiness → build → quality → evidence; fresh scheduled task identities
Inspect a controlled task failure	Quality failed; evidence upstream_failed; no failed-run receipt; repair succeeds
Explain the run and environment boundaries	Coherent handoff links run/task IDs, artifacts and independent fund/date output

Advanced airflow

Advanced reference / independently prepared, separately timed. Allow 90–120 minutes (estimate). Open the phase instructions.

Task	*Evidence*
Define bounded replay and input identity	Supported May partition, explicit schedule interval and reproducible source readiness
Inject failure and observe useful retry behavior	Known quality/readiness cause, observed attempts and no premature receipt
Prove output correctness after recovery	Eight unique report keys, four partition rows, stable values across repeated input
Use Astro UI to make an operational decision	Exact graph/grid/log identities, causal diagnosis and observed repair decision
Write a short incident handoff	Current success/failure/repair/replay chain, reviewable diff and explicit pending gates

Guest presentation (optional): a production data product end to end

The supplied **Global Client Connect (GCC)** example describes a near-production institutional-client data product. It illustrates Databricks jobs, asset bundles, external tables and Unity Catalog in the supplied case. The course does not establish its current production status, owner or deployment architecture; those require the current internal product record.

> Knowledge check: Read an end-to-end product
>
> Which upstream promise does curation rely on? How would you distinguish resource deployment from the data product's successful delivery?

The product

- **Problem:** reporting teams (INA, and soon others) were working from scattered data in sandbox environments with no single source of truth for institutional client data.
- **Goal:** centralise client, product, assets, client-interaction, opportunity and territory data into a single source of truth for institutional data (retail to follow), consumed initially by two teams and growing.
- **Sources:** data producers already registered in **Caspian** (Capital Group's enterprise data lake) via ingestion DAGs owned by the source teams.

The stack and execution boundaries

- **Airflow** orchestrates the curation pipelines. The territory DAG opens with three or four **dependency-check tasks** that call the Airflow REST API or metadata to verify that upstream ingestion DAGs, such as the CDM team's, ran successfully within a time constraint. Only then does curation start. If a dependency fails, alerts fire to the team and to the business, and processing stops rather than joining against wrong data.
- **Databricks** runs the actual work: Airflow triggers a Databricks *job* (e.g. the territory job), whose tasks are the same dbt commands learners had been running by hand, `dbt seed` and `dbt run` for the models. Jobs are defined in YAML using **Databricks Asset Bundles**, which deploy them into dev/SQA/prod. (A design review asks If orchestration lives in Airflow on purpose, to avoid Databricks lock-in, why is there orchestration-looking YAML on the Databricks side? The answer was that all *dependencies* resolve in Airflow, and the bundle YAML only defines the jobs that run the dbt models.)
- **Harness** provides CI/CD: automating the path from integrated code to released product. Airflow automates scheduled data workflows; Harness automates software releases. Release automation and data orchestration have different owners and evidence requirements. Pipelines exist per environment (dev, SQA, prod), and separate **IaC pipelines** (Terraform) create the infrastructure itself, meaning clusters, warehouses and the S3 buckets for each medallion layer.
- **Storage:** medallion architecture (bronze/silver/gold) on **external tables**. The data sits in the team's own S3 buckets instead of Databricks-managed storage. The architectural reason was disaster recovery. At design time managed tables didn't support cross-region (east/west) replication, and external S3 buckets can be replicated with full control. Databricks is *configured to pretend* those folders are its own; the files actually live on Amazon.
- **Layers of control:** Harness on the outside (it sets up everything, including the S3 layout), then Airflow (schedules, watches, resolves dependencies), then Databricks (runs the jobs), then dbt (does the transformations). The control question is which layer runs the final tasks. Databricks runs the dbt, Airflow wraps Databricks, and Harness wraps the lot. Why Harness *and* Airflow? Specialisation: software-release tasks vs data tasks.
- **Consumption and governance:** gold-layer products are replicated from each team's application catalog into platform-owned **domain catalogs** in **Unity Catalog**. Access is role-driven: raise an access request, get approval, and the domain catalog binds into your workspace. Two consumer types appear here: analytical consumers, such as Power BI through the Databricks connector, and other teams consuming the data into their own curation pipelines. Their source-aligned catalogs bind into their workspaces just as Caspian catalogs bind into GCC's workspace. Unity Catalog holds descriptions, column metadata and read and write grants, so it's where governance happens as well as where you look at data. On the paved path you register a product in the in-house Marketplace at Phase 6, and its metadata goes to Collibra and the Unity Catalog marketplace. Before you build a product, search those catalogues to see whether it already exists.
- Scale for reference: a ~20-table data set, on the order of a billion rows for assets, roughly 100–200 columns per table.

The simplicity argument

ETL transforms data before loading it into the target system; ELT loads first and transforms there. dbt fits the ELT pattern by expressing SQL over already available source tables.

Start an architecture review with the actual input, transformation and consumer output. Then justify the added services through access, scale, recovery, vendor integration, audit and operating requirements. A small prototype can clarify the logic, but it does not establish that a laptop meets production obligations.

Case review: changed values and lineage

- **Upstream logic changes** (schema unchanged, values change): the only defence is data-quality rules with acceptance criteria at your end. For example, a column expected to take three values suddenly produces a fourth and your check fails. But checks encode the assumptions made when they were written. The business changes, so checks need looking after, and with vendor data you often get no warning until something breaks. For change diagnosis, because everything here is code (Terraform, YAML, Python, dbt), a clean git history *is* an audit trail. Nothing changes the system except through versioned code changes that someone has reviewed.
- **Duplication/lineage:** check the domain catalogs for an existing product before building another one, then follow its recorded lineage.
- **Lineage across dbt projects** (each team has its own project, and lineage works within it): inside Airflow, task lineage plus Datadog spans the whole pipeline. Once the dbt tasks sit in the DAG, Datadog gives you observability and logging across the run. The supplied example proposes to co-locate teams' dbt and YAML so lineage can be followed upstream beyond your own team.
- **"Where does the end product actually live?"** In S3, behind the scenes; consumers see it through the domain catalog and don't need to care.

:::

The dependency-check pattern has a name: sensors

A curation pipeline must not start until an upstream team's ingestion has succeeded. The GCC territory DAG hand-built this with tasks calling the Airflow REST API; Airflow has the pattern built in, in two forms.

A **sensor** is a task that waits for a condition instead of doing work:

```
from airflow.sensors.external_task import ExternalTaskSensor

wait_for_cdm = ExternalTaskSensor(
    task_id="wait_for_cdm_ingestion",
    external_dag_id="cdm_ingestion",
    poke_interval=300,          # check every five minutes
    timeout=3 * 60 * 60,        # give up, and fail, after three hours
)
```

Put that first in a DAG (`wait_for_cdm >> stage >> …`) and curation can't begin until the upstream run has succeeded. The timeout gives you GCC's "within a time constraint". After three hours of waiting the sensor fails, the failure callback fires, and nothing downstream joins against missing data.

The trade-off to know is poll versus event. A sensor polls: it re-checks every `poke_interval`, so there is delay of up to one interval between the upstream finishing and downstream starting, and a waiting sensor occupies resources (deferrable sensors mitigate this by handing the wait to the triggerer container from the `docker ps` exercise).

The event-driven alternative is **dataset-aware scheduling**: the upstream task declares the dataset it updates, and the downstream DAG's `schedule` is that dataset instead of a cron line. It runs when the data changed, with no polling at all. The catch is that both DAGs must live where the same Airflow can see them, whereas an authorised API check or a suitable external-system sensor can check a different runtime. The ExternalTaskSensor example above checks runs in its own Airflow environment; it does not

automatically inspect another team's Airflow. A separate instance needs an approved integration and a defined success/time boundary.

From the contract's Access section to Unity Catalog grants

Take the Access section of the data contract your group generated on Monday and Tuesday and ask Catalyst to translate it into Unity Catalog terms:

- which catalog and schema the product's tables would live in;
- who gets SELECT and who gets write. In a sound translation, consumers read and only the product's own pipeline identity writes, so if the output gives a consumer write access, that's a defect to catch;
- what the access-request path is. In the GCC demo, a consumer raises a request, an owner approves, the catalog binds into the consumer's workspace.

For an optional connected demonstration, use only an approved training schema and prepared Databricks access. In that context, inspect its grants with the supplied read-only example:

```
SHOW GRANTS ON SCHEMA <catalog>.training_<initials>;
```

and read the result: which principals hold which privileges on the schema you've been building into all week, including any approved training grant needed during setup. Compare the two descriptions. Did the translation grant anything the contract's Access section never promised? And did any promise in the Access section get lost in translation?

After the Airflow practice

- After the required Airflow evidence is retained, an authorised instructor may show the prepared release/operations systems. Use the supplied screenshot walkthrough when access is unavailable, and distinguish observed images from a live run.
- **Friday's capstone:** use the CG product brief and its individual/pair phases. Retain the consumer contract, authored models, release checks and personal handoff.

> Learning Review
>
> Explain what one successful Airflow run proves, then name two things a parsed DAG alone does not prove. Use the controlled-failure evidence to show why publication stayed blocked and how the repaired attempt restores all eight fund/date rows and the four rows for its named fixture partition.
> Self-check: retain run/task identity, the failed condition, the repair, the new attempt and an independent output query. Keep local log-only callback evidence separate from hosted Astro or Datadog integration evidence.

Trace one commit through release, monitoring and recovery.

Data operations: the tools around a product

> The last session today is the “everything else” session. It covers the tools and systems around a data product that data creators don’t drive directly but should understand end to end, and the umbrella theme is data operations.
>
> **THIS SESSION ANSWERS**
>
> - What does “ops” mean in DataOps?
> - Which tools sit around a paved-path product, and what does each one do?
> - What does a real paved-path product look like in production?

> Knowledge check: Choose the right operational record
>
> Which record tells you a change was released, and which tells you the nightly data task ran? What should an owner record before escalating a failed delivery?

So what does *ops* mean in “DataOps”? Operations means systems (systems, programming and control) and the practices around that: operational practices, CI/CD, release practices, and so on.

The plan:

- **The interfaces of the tools you don’t drive.** GitHub, Harness, Astronomer, the data marketplace, Unity Catalog and Datadog, shown live wherever possible by an authorised instructor with prepared access. Expect a pull request under review, Harness pipeline stages, an approval step and the environment selector, the Astronomer DAG list, a marketplace listing, the Databricks workspace home and Catalog Explorer on a domain catalog, the SQL editor and job logs, and a Datadog monitor.
- **Docker and containers** in more depth; **Kubernetes**, and briefly Terraform and Harness (this is not a Terraform course).
- **The end-to-end infrastructure picture.** How a change becomes a release, where things get deployed, and how all the layers stack up.
- **Mopping up.** Catalyst workflow phases and terminology, plus outstanding questions from the group.

The framing for this session, in terms of Monday’s asset view: a data product is code plus infrastructure plus data plus promises. The week covered data, promises and code, and this session is the infrastructure. It’s a peek behind the curtain at how dbt, Databricks and Airflow actually run, and how everything fits together. This layer is *beneath your feet*. You’ll rarely touch it, but it’s part of the paved path’s context and worth understanding. The stack itself is the list Tuesday gave you: Git and GitHub, Kubernetes, dbt, Airflow with Astronomer and Cosmos, Databricks, S3 and Harness. The supplied platform example also includes Kafka event delivery, Lambda handlers and SQS queues. The event triggers later in the session show the last two.

The course objective is to make everyone capable of producing a data product *with Catalyst*, while understanding what’s going on under the hood. Catalyst can generate all the code for you, but you must be able to read what it produces and judge whether it is right, because data products go into the hands of people making big decisions, and being wrong is expensive. Manual inspection and observed checks let you evaluate generated work and retain ownership of its outcome.

Guest demo: a real paved-path data product, end to end

> Knowledge check: Connect the business purpose to operations
>
> Which consumer decision does this product support? What evidence must accompany a column rename before it is released?

The supplied product example follows **Average Advisor** and **Next-Gen Portfolio Construction**, built on the paved path for institutional/marketing consumers. It connects the week's design, data and operational responsibilities in one deployed product.

The product and its purpose

- The product tracks advisor interaction data from web tools (portfolio construction being one): when an advisor runs a fund comparison, what they compared, portfolio health at run time, whether the same tickers persisted year over year, whether swapping to a custom portfolio would have been healthier.
- The pipeline owner needs the consumer's interpretation even when the dataset's business meaning is outside the owner's specialist role. Use the contract to connect operational checks to that meaning.
- Code management, deployment, quality checks and access remain necessary as tools change. Learn each tool through the responsibility it serves, and retain evidence that the responsibility is met.

Documentation: from requirements to solution delivery

- The product manager maintains Confluence documentation covering the pain point/opportunity, strategy, and measurable, achievable and realistic objectives, using the product decisions introduced earlier in the week.
- A **conceptual architecture diagram** starts the project (inputs, staging, expected outputs). Because it's written by architecture, it's conceptual and not always accurate (arrows don't always match the final build), but it explains data flow to a net-new engineer.
- A **roadmap** precedes committing the team: use cases per data source, objectives and deliverables. These are tasks that are computable, so delivery time can actually be estimated.
- When finished, the team writes a **solution delivery page**: the tables delivered, frequency, owners, the inputs required (Argo, CDM, SFDC, MAI and FinMason, a mix of internal teams and vendors), and the *engineered* data flow as actually built. Every data product gets one, so anyone on the team can triage support issues when the owner is out.
- A current triage guide lets a responder trace failed dependencies when the original engineer is unavailable. Check any AI-drafted guide against the repository and an observed recovery; stale documentation can send support to the wrong step.

The data layers

- ~15 raw files arrive from **Pecan** (the foundation data set of advisor activity/analysis runs) with ~15 correlative **FactSet** files; these land in **staging**, with per-file checks and balances in the dbt models. That's the pattern from the dbt days.
- **Integration** is where transformation happens. Staged sets are merged or unioned (e.g. Pecan fund detail combined with FactSet fund detail).
- **Prepared** (also called curated) is the consumer-facing zone; access controls mean users effectively only ever see prepared tables.
- **Identity enrichment**: activity data carries only an ID; **CDM** maps the party ID to a name, email and role, and from there a whole web journey can be reconstructed. Foundation data vs supporting/metadata data (identity being metadata relative to Pecan).

- Consumers mostly reach the data through **Argo** (an internal API service sitting over the prepared data).

Caspian vs LASR (lakehouse vs legacy)

- The code base lives and runs in **Caspian** (the Databricks-based platform, defined properly in Wednesday's storage section); a simple flag in their Caspian dbt template replicates outputs to **LASR**, the legacy warehouse, because critical mass, user sandboxes included, still lives there. Duplicative data in two places, but the code lives in one, and the replication step can be switched off once LASR is decommissioned. Direction to the team: don't build in LASR only.
- Outputs are written as Parquet into Unity Catalog (schema: Caspian consumption). They aren't Delta by default, because the team has a custom script managing metadata for a bitemporal model. An extra one-time DDL step exposes data through **Dremio**; after that the sync updates automatically each run.

Change-to-production walkthrough (Harness + Airflow)

The supplied demonstration follows a column rename from change to release:

1. New engineer creates a **feature branch** in GitHub, makes the change.
2. In **Harness** (described as "an orchestrator for infrastructure", with GitHub-Actions-style pipelines), select the feature branch as an expression, run the pipeline: it grabs the code and deploys it to the test environment in ~5 minutes.
3. Schedule the Airflow job in the corresponding environment and verify the renamed column appears.
4. Once approved, a prod-level Harness instance builds/releases, and the normal Airflow scheduler (defined in YAML) triggers the job on its nightly schedule.

Airflow specifics from the demo:

- DAGs are chains of **dependency checks** and hand-offs; the example DAG depends on four other Pecan jobs. The Airflow code itself is very thin, "almost just handing off to dbt". Jobs are described in YAML, and business logic lives in the SQL and dbt layer. Some steps (e.g. curation sync to Azure) are macros rather than hand-written.
- Naming conventions are one-to-one between DAGs and repos, so any job can be traced to its code in GitHub (their team repo: Advisor Touch Point).
- Branches of the DAG that are independent (e.g. Dremio sync vs historical sync to Azure) can finish at different times, but all must complete before the team notification is sent. Notifications are success-based to the team; a few high-priority reports notify user distribution groups directly.
- Airflow replaced **Autosys** as the scheduler; the win is that orchestration now lives in the *same repo* as the code (previously two separate repositories, two check-ins per change). Astronomer may be adopted underneath, but "from a data creator point of view it changes nothing".
- **Runtime ownership.** Trace the DAG to its deployed environment, released code and execution logs. In the supplied paved-path example, Harness deploys into Astronomer; another runtime must be established from its own configuration. The release section separates that deployment from each data run.

Case study: Consumer value and service reliability

Case study: value and service reliability

An investment-data product supports analysis and advice, while operational systems make that service available and reliable. A product owner must decide where a proposed change creates value: improved decision data, more reliable delivery, or a better consumer experience.

One view starts with the core business decision: data and advice are the service being sold, and software enables it. Another starts with retention and competitive differentiation: a consumer may leave when the service channel fails even if the underlying advice is sound. The engineering decision is to assess both the analytical benefit and the service promise, rather than infer priority from the software's size.

An engine is necessary for a car, but the journey explains its value. In the same way, operating software successfully is necessary without exhausting the value of the product. The transferable lesson is to name the consumer outcome and the reliability constraint together when prioritising a data-product change.

How does a data product get released?

A release takes a change on GitHub and puts it live in the cloud. We follow one change from pull request to production and look at the layers it runs on.

THIS SESSION ANSWERS

- What does a release involve, and what is CI/CD?
- Why release every day or week instead of every three months?
- Where do things get deployed, and what are containers and Kubernetes for?
- Where do you look when the nightly run fails at 3 a.m.?

Knowledge check: Keep approval and correctness separate

What can an automated test establish that a human approval cannot? What can an approval authorise that a passing test does not?

Take a data product. For our purposes right now it *is* code: some dbt code, some Airflow code, maybe some YAML configuration. On what conditions does this get released? Where does the code live? GitHub.

Release processes are attached to, or driven by, GitHub actions. The actions we can take:

- **Push** and **pull** — pushing and pulling *changes* to code. These are Git actions.
- **Pull request** — a special action that only GitHub-like providers give you, because Git itself has no pull requests. A pull request is literally a *request to pull* your changes, and the reason we request rather than just pull is so the changes can be reviewed before they go in. It also leaves a record. When something breaks you can return to the exact commit that last worked and see what changed.

So the flow: you have code on GitHub, you open a pull request, someone approves it, GitHub pulls the code in. Attached to that pull (or to a push) there can be a **release process**.

What does a release involve?

A data-product release can involve the following changes:

- **Building code** — resolving dependencies, compiling/packaging
- **Testing** — before we release, we test the changes
- **Deployment** — to some remote environment, which may mean creating the infrastructure itself: renting and configuring machines
- **Configuring services** — any services the system uses

CI/CD

- **CI** = continuous integration: continuously merging and building code. **CD** = continuous deployment: automating the infrastructure so code goes live. "Continuous" means you don't wait for a release window. Code goes as and when it passes its quality checks, in practice daily or weekly, where traditional software releases came every 3–6 months.
- A CI/CD process takes you from code in Git to a product running (typically in the cloud) at that frequency. CI/CD is to operations people what data mesh is to data people. It's a set of practices.
- **Triggers:** timing makes no sense for code (developers don't produce hourly changes), so triggers hang off Git, such as a push, a merge or PR, or a tag. A typical pipeline pushes, tests the code, builds a container, pushes the image to a registry, deploys it onto cloud infrastructure and runs it. Between build and deploy the image goes through a **registry** (a store of built container images). The build pushes it there, and the deploy, usually on a different machine from the one that built it, pulls it back out.
- Harness is the configurable system that does this at Capital Group. If you've only ever used Harness to run Airflow jobs, that's just the one task your team configured. Harness will sequence any CI/CD tasks: renting cloud infrastructure, configuring containers, triggering Airflow.

A release keeps the same commit SHA from the Git merge onwards. The code is built and tested, passes the data-quality gate, gets Harness approval and a change record, and is deployed. Then you watch the released product.

Why release continuously? Feedback loops

A long batch release cycle delays evidence about whether a change works for its consumers. Smaller reviewed releases shorten that feedback cycle, provided each change keeps its contract and passes the appropriate quality and delivery gates. The useful trade-off is the size and risk of a change against the cost of review, testing and recovery; speed alone does not establish a sound release.

Releasing something means the customer can see it, the team can see it, the engineers can see it. You get a shared understanding, and you get it often. Continuous integration forces developers to see the latest version of the code; continuous deployment forces the customer, or whoever views the system, to see the latest release. If there's a mistake, an issue, something not happening, you find out *fast*, and your understanding of the system is always current. In a large organisation without it, every three months the whole thing collapses and it's a nightmare to resolve.

Inside Harness: projects, pipelines, input sets and executions

The supplied Harness example follows an application release into Kubernetes. Creators inspect the project, branch, input set, stage results and approval evidence; an authorised platform operator controls the release.

- **Getting a pipeline into your repository.** You have a repository with your code and you want to deploy it. For that you need a Harness account and a **Harness pipeline**. There are templates on DevHub from which you import the Harness files directly into your repository: you tell it which repository you're adding to, and it adds a `.harness` folder containing a YAML file, which holds the structure of your pipeline configuration. Where Airflow had lines pointing between tasks, here you write the stages out in YAML. Once that's committed, the pipeline shows up in Harness.
- **Projects and pipelines.** In Harness you have **projects**, and within a project a list of **pipelines**. You name a pipeline after your repository so you can keep track of all your deployments. Harness's job is to keep track of each deployment and each **execution run**: where it failed, on which path, and the full logs of what happened. A lot of debugging happens here: you deploy and follow the errors.
- **Importing a pipeline from Git.** Choose the Git provider and a **connector** (Capital Group has these enabled already), pick the repository, pick the branch (the maintained pipeline definition lives on

`main`; its execution inputs select the reviewed revision and target environment), then give the YAML path, meaning repository, folder and file. If it checks out, the pipeline is added.

- **Input sets.** A layer inside the pipeline that distinguishes types of deployment: straight to prod, only to QA, only to dev, or the whole package. You set up named groups so you click a button and don't re-enter the inputs each time. When you're deploying to production you want the human-error parts removed, so you get a template you click into that always works the same way.
- **Running a pipeline.** The YAML is text-based and hard to read at a glance; in the UI you choose specific stages from the template ("I only want to build my code and deploy to QA") and choose the branch. The default is `main`, but for authorized isolated development work you can select a feature branch; shared and production targets retain their reviewed promotion and approval gates. Then pick your input set and run.
- **An execution run**, from the history. It builds, then deploys to dev, then deploys to QA, and each stage is listed and displayed as it goes. You can see the repository and branch used, when it ran, and any failures (success, failure, ignored), including security scans. In the supplied execution, the scans hadn't been left long enough and showed warnings, which for a deployment to prod wouldn't be allowed through. The details show which inputs were used, such as the ADM ID and the region, all from the configuration files in the code.
- **Environment-specific configuration.** A `.tfvars` file for the Terraform code, per environment, holds the ADM ID, a **namespace** and values unique to each environment. Going through the environments, some parts change (the AWS account ID, which says where the code deploys and where the S3 buckets live, plus the environment name and the namespace), and that automates the promotion. So you commit your code, work between multiple people on the team without affecting each other's work, deploy, test, in a streamlined way with less human error.
- **Two pipelines.** Asked to walk a full deployment graph stage by stage, the team's answer was the same split the GCC case described: an **IaC** (infrastructure-as-code) pipeline just for infrastructure, which creates any AWS services, Databricks instances included, and a **code** pipeline with code checks and tests, then the Docker build, then the deploy stages into specific Kubernetes pods. The customisation happens inside the YAML in the code.
- **Team practice.** Each developer manages their own application from build through dev to QA. Only three or four people are dedicated to the production pipeline, to restrict usage; code reviews for prod happen between those people.

Two cycles, kept apart

Separate these two cycles before diagnosing a failed delivery.

1. The **data cycle**. Infrastructure is set up once, with Airflow ticking away on its own daily schedule.
2. The **code cycle**. Someone changes the dbt, ETL or YAML in GitHub, and that push (in the guest demo's team, via a manual Harness run in test, then a prod-level Harness release) redeploys the code that the nightly Airflow run will pick up.

In this pattern, Airflow keeps its data schedule and Harness releases the code it uses.

Event triggers: when a vendor file lands

A data-ready event can replace a clock trigger. In the supplied vendor-file pattern, an S3 arrival creates a queue notification, an SQS-fed Lambda submits a trigger to Airflow, and the resulting DAG starts the downstream work. Separate the event acknowledgement, accepted DAG trigger, successful task execution and correct published result; each is a different evidence boundary.

This is the *event* side of the poll-versus-event trade-off from the sensors section, sitting outside Airflow: a sensor polls from inside the DAG, whereas here the cloud tells Airflow the file has arrived and nothing

polls at all. It's also why the GCC case and this demonstration both start from S3. Vendors integrate by dropping files, so the triggers hang off the bucket.

Integration and environment promotion

The paved-path convention uses short-lived branches from `main`, reviewed squash merges, and an explicit promotion record. Integration combines an approved change with the team's main code line. Promotion selects the exact commit or artifact version an environment receives. A merge can trigger Harness without making every environment adopt that revision immediately.

Consider reviewed commit `C104`, which adds a required quality check. Development can use `C104` while SQA and production continue using `C103`. After validation, SQA receives `C104`; production follows only after its required approval and ServiceNow change record. The promotion record, such as `promotions.yml`, identifies those selections. A deployment execution binds the selected revision to its actual target and result. A current `main` checkout alone cannot tell an operator which revision is live in each environment.

A designated live branch or tag is another possible release design. In that arrangement, a merge or tag update can trigger delivery to a named environment. It needs its own approval, desired-state, and deployment reconciliation rules. It is not the standing-branch convention used for this course's paved path, and a Git webhook by itself does not establish a complete GitOps operating model. Follow the repository and promotion conventions rather than combining incompatible branch policies.

In the platform release flow, a developer submits a focused change, addresses review feedback, and integrates the approved pull request. Harness uses the selected environment inputs and pauses at the required ServiceNow change gate before production deployment. The release record identifies the commit, environment, approval, and change request. Keep upstream dependency identities alongside that record so a failed refresh can be routed to the right owner.

Work in progress can remain behind the maintained `include_wip` gate in an isolated development target. Check enabled downstream dependencies and keep promotion intent out of duplicate `_v2` model files. Consumer compatibility is a separate product decision: selecting a code version does not automatically create two compatible consumer interfaces.

Why redeploy the whole product on every change

A partial release can leave uncertainty about which compatible components are running. A repeatable build and a recorded deployment help establish the intended code/configuration identity and expose cross-layer interactions, such as a dbt change affecting an Airflow command. Rebuild or replay only within a defined boundary: verify that the particular tasks are idempotent, that repeated publication does not duplicate external side effects, and that the expected output still holds. The weekend-outage example explains why repeatability needs this evidence rather than an assumption.

Three deployment tools

Three tools deploy different parts of a paved-path product.

Workflow

Tool	*What it controls*	*Artefact it deploys*
Harness	Approval, environment promotion, secrets and the audit trail	The pipeline steps that call the deployment tools

`astro deploy --dags`	The Airflow side	DAG source; image/dependency or dbt-project changes use their separately supported deployment route
DAB (`databricks bundle deploy`)	Databricks resources	Jobs, Lakeflow pipelines, dashboards, models and compute configuration declared in a bundle

Harness holds the approval gate. DAB has no approval gate of its own and doesn't register DAGs with Astronomer, and `astro deploy` doesn't create Databricks resources. That's why you can find orchestration-looking YAML in more than one place, with each tool doing a different job. The supplied platform example illustrates this: a single Harness run deploys the DAGs to Astronomer, the Databricks bundle and the dbt project together, and the whole bundle waits on one approved change request.
Some teams run dbt itself as a Databricks job that Airflow triggers. In that arrangement the bundle can create the Databricks job, while the Airflow DAG triggers it and waits for the result.

Why Airflow orchestrates even when Databricks does the work

An Astronomer-managed Airflow DAG can submit one Databricks notebook job and wait for its result. The Airflow graph then contains one task; detailed execution is in the Databricks job logs. Inspect both records to connect the scheduled run with the transformation it performed.

So why schedule with Airflow rather than as a Databricks job? Standards, basically. Say you have a few data products that are fairly important. If they were just executed as Databricks jobs, the data operations group, which is there to oversee with "eyes on glass" across the important DAGs, wouldn't have the same visibility. The jobs can do the work, but what *orchestrates and triggers* them should be Airflow, which gives the operations side one place to see what's happening. The other argument you'll hear is the vendor lock-in one from Tuesday's Databricks session. Use as much open-source software as possible so vendors can be switched out later. Remember the caveat from Wednesday's lock-in discussion, though. Unity Catalog's grants and lineage stay behind if you move.

The supplied projects invoke dbt from Airflow and do not use a dbt Cloud scheduler. This describes their execution route, not an enterprise-wide CG dbt version, licensing decision or historical adoption sequence.

The three post-build gates

Release gates after the build

A green dbt build means the project compiled, ran and passed its tests. The release still has to get through three more gates.

Gate	*Question*	*How the paved path does it*
Data quality	Is the built data fit to publish?	Caspian DQF runs after dbt and combines every constraint with strict AND logic; one failure returns a non-zero exit code and stops the pipeline
Delivery	Is this exact change allowed into the target environment?	Harness promotes an artefact identified by its commit SHA; production requires both a named human approval and an approved ServiceNow change request (the integration and promotion example above shows where the pipeline pauses for it)

Gate	*Question*	*How the paved path does it*
Coded standards	Does the DAG follow the approved engineering conventions?	The CI check rejects violations such as a missing failure callback or MDS routing tags, forbidden email alerting, or absent retry settings

In an emergency the delivery gate still applies. You raise an Emergency change request instead of the standard one, and it has to link to an active incident. The incident is the reason you're allowed to go faster, and it keeps the record attached to the change. Lower environments could promote automatically, but the supplied paved-path template pauses even at deploy-into-dev for an ADM ID and a change request, because one template serves every environment.

Breaking changes at release time

If you know a release will break your consumers, you declare it in `promotions.yml`. The model's row in that file needs `acknowledge_breaking: true`. The tempting shortcut is a second model alongside the old one, something like `fct_exposures_v2.sql`. Don't do that. You end up with two versions of the same table and the lineage splits across them. Ask the product owner for the current notice period and who signs off the change.

The usual pattern has four steps. Tell registered consumers five working days before the change. Publish the new contract version. Keep compatibility views running alongside for ten working days. Remove the old view once the owners have confirmed they've moved. The flag is only the declaration, so you still owe consumers the notice and the migration window.

Recovery crosses software and data boundaries

A rollback plan must identify what has already changed. Reverting a commit does not reverse every effect of the released implementation. Use the retained release and run identities to select the recovery action.

Changed surface	*What a code revert establishes*	*Additional recovery decision*
DAG or transformation code	An earlier implementation can be released	Confirm the deployed image, packages, configuration, and active-run behavior
Database schema	Earlier source code is available	Verify whether the applied migration is reversible; restore or forward-fix schema and values deliberately
Published dataset	Earlier calculation logic is available	Restore a retained valid release or rebuild the agreed business partitions; check permissions and freshness
Processing checkpoint	Earlier checkpoint code is available	Reconcile durable output with recorded input progress before replay
Exported file or notification	Earlier delivery logic is available	Determine whether the consumer already received it; suppress duplicates or issue an explicit correction

For example, a bad join can pass deployment and then produce a candidate with duplicated keys. Block publication, retain the rejected candidate's diagnostics, and leave the previous permitted release identifiable. Repair the join and validate a new candidate before promotion. If invalid data was already published, also identify affected consumers and the required correction or withdrawal; a later green DAG does not undo their earlier use.

The temporal reference extends this reasoning to historical versions and checkpoint consistency. Recovery is successful when the required business state and consumer interface are restored, not merely when an old commit has been checked out.

Warranty and the launch package

For the first 30 days after a production release, the team that built the product still owns it. That's the warranty period. You tidy up the runbook, the alerts and the known failure modes while the people who built it are still on the hook. DataOps can refuse to take the product over if the runbook is missing or unusable, and the Data Domain Lead settles any dispute.

The hand-off checks two things. The product pillars ask whether the team built the right thing, built it well, tested it properly and documented it well enough. Monday's six quality dimensions check the data itself: completeness, freshness, accuracy, validity, uniqueness and consistency.

Before you announce a product like Morningstar exposures, run it quietly in production for at least three days in a row. The consumer guide has to answer five questions: what the product contains, where to find it, how to connect, what its limitations are, and who to contact. The Marketplace listing is the Phase 6 gate. It has to be live and validated, with the right entity type, domain ancestry and contract reference, and at least one downstream consumer has to have confirmed access. Three roles sign it off:

- the DPM submits it;
- the Data Steward signs off completeness;
- the Data Domain Lead approves visibility across domains.

After launch, watch four numbers: distinct active consumers, query volume, support tickets tagged with the product, and how often it meets its freshness SLA. Remember Monday's Excel workaround? If the product is green while people still refresh their own spreadsheets, find out why before you touch the pipeline. Can they find it? Do the sample queries work for them? Does it use their words?

Group exercise: approve or request changes?

So where does a release start? With a pull request. In this exercise you sit on that gate.

> **Group exercise**
>
> **OUTPUTS** An approve or request-changes decision with its reasons
>
> Review a supplied pull request against the Morningstar-style reference project. It's a plausible dbt change (a rename, an added filter, a touched `schema.yml`) with one planted contract violation. In one variant a null-handling rule from the STTM has been dropped from a staging model, and in the other a test has been quietly deleted. In groups of three, take ten minutes with two artefacts, the rendered GitHub diff and the data contract. Decide as a reviewer would, approve or request changes, and explain the evidence for your decision.

Review criteria:

- Every promise in the contract is kept or broken by what merges. The PR is the last screen where anyone can say no.
- You don't need to be able to write the change to review it. You need to know the contract: which columns may never be null, and which tests encode which promises. That's the data creator's knowledge. Non-engineers can and should review these PRs, especially now that much of the change is generated.
- A deleted test shows up in the diff as a block of red lines, and it's easy to skim past. Read the red lines as carefully as the green.

Use ten minutes for the review and ten to fifteen minutes to compare decisions, retain the evidence and identify the contract correction.

Deployment environments at Capital Group

> Knowledge check: Separate storage and compute
>
> Where would you locate persistent source/consumer data, transformation compute and the orchestration runtime? Which dependency can a green deployment leave unverified?

Where do things get deployed? Distinguish storage, compute and application-runtime destinations:

- **S3** — strip away the Amazon branding: it's a storage service, meaning files and folders.
- **EC2** — virtual-machine compute. An instance supplies configured CPU, memory, storage and networking; the product still needs its own runtime and dependencies.
- **Serverless services** — the provider manages the execution infrastructure. The team supplies the supported code/configuration and checks duration, capacity, access and billing limits.
- **Databricks** — as far as Capital Group is concerned, mostly used as a compute platform: you run things like dbt on it. Databricks *can* store data and run other kinds of compute, but here much of the data sits externally (S3) and Databricks is essentially your SQL engine. S3 is your storage and Databricks is your compute, and together they form your database (mostly, though there are exceptions and some internal stores).
- **Airflow** — the system that triggers events across this infrastructure. Airflow can run in local containers or managed remote infrastructure. The supplied case discusses EKS and Astronomer; it does not establish the current CG deployment topology. Identify the scheduler, worker runtime and platform owner for the assigned environment.
- **Astronomer** — also in the picture, running Airflow on top of EKS. It's the Airflow vendor from earlier today: local containers on your machine, and managed Airflow in its cloud.

For all of this infrastructure there are dev, QA (SQA in some teams' configuration) and prod environments. This part isn't about the data, which is what flows through at the end. This is about *software*: how a piece of software configures this infrastructure. This part of the job is software engineering more than data analysis. The rented infrastructure is duplicated, across regions and within one place, so different processes can be tested in different environments, and for reliability: if one goes down, another picks up.

Containers in deployment

> Knowledge check: Know the packaging boundary
>
> Which dependencies travel in a container image, and which remain external? What would you check when the same image behaves differently in two environments?

One more idea is needed before the deployment picture makes sense: **containers**. This is a recap of Wednesday's container session, now in its deployment context.

Virtual machines first

When we say something "runs on a VM": a virtual machine is an abstract box with an allocation of hard-drive space, an allocation of RAM, and an allocation of CPU time, and we rent all of these. The physical machine has 100% of the CPU, RAM and disk; a virtual machine is a *share* of those resources that we treat as a machine.

What a container is, and what it is not

A container packages an application process with the user-space files and dependencies it needs. It shares the kernel of the host/runtime environment rather than carrying a complete guest operating system. A virtual machine supplies a guest operating system with its own kernel on virtualised hardware. Containers commonly run inside VMs in cloud environments. Docker explains these boundaries.

Packaging cannot include an external database, live credentials or supplier data. Host/runtime configuration and mounted files also remain dependencies. Line endings, paths and permissions can differ between development hosts; check the actual prepared container/runtime instead of assuming the image erases every environmental difference.

Why wrap code in a container at all?

Two responsibilities explain the additional layer around a VM or physical host.

1. **Portability and repeatability.** A versioned image fixes packaged code and dependencies, reducing configuration drift between development and deployment. Repeatability still depends on a compatible runtime, explicit configuration and reachable external inputs. Keep those conditions in the readiness record.
2. **Isolation and interfaces.** Process, filesystem, resource and network boundaries limit interactions and make service interfaces explicit. Their effectiveness depends on the runtime configuration, privileges, mounts and exposed interfaces; a container is not an assurance that any untrusted process is harmless. The platform team supplies the approved configuration.

Docker

Docker provides tools to build images and run containers. A production platform can use a compatible container runtime while retaining the packaged application identity. Separate the image build from the environment and workload configuration that make it run.

Kubernetes and the layers of orchestration

EKS is Amazon's hosted **Kubernetes** (K8s, really "the container configuration service"). Kubernetes is an **orchestrator for containers**. It is to containers what Airflow is to data tasks. It configures, connects and automates the containers on the rented machines. (Kubernetes was originally written by Google.)

Each orchestrator manages a different boundary: Kubernetes places and manages workloads; Airflow orders and observes data tasks. Automation makes these operations repeatable, while the team still defines dependencies, failure responses and ownership.

> Knowledge check: Read the runtime state
>
> What does pod readiness tell you, and what data-product promise does it leave unproved? Which image and namespace identify the workload you intend to inspect?

The whole picture

When we deploy something, the way to think about it:

1. A **container orchestrator (Kubernetes)** drops containers across the infrastructure. That's the outermost orchestration layer.
2. Inside those containers runs **Airflow**, the orchestrator of the data work, moving files around and triggering jobs.
3. Airflow triggers **dbt**, which does the analysis and transformation. You could even see dbt and SQL as the innermost orchestrator, a row-and-column orchestrator.

4. Driving the outermost layer at Capital Group is **Harness**: inside Harness sits Kubernetes and Terraform-style provisioning, the layer that rents infrastructure and configures the container-orchestration system.

The configuration languages underneath:

- **Terraform** (`.tf` files) — the code handed to Harness that asks Amazon/Azure/GCP for machines, S3 buckets, RAM.
- **Kubernetes** configuration — connects and automates the containers on those machines.
- Inside those containers: Airflow, which triggers DAGs, which run dbt, which moves the files.

These platform responsibilities form the **paved path** introduced on Tuesday. A successful data run still needs consumer-facing correctness and delivery evidence.

Harness, indirection and Conway's law

> Knowledge check: Explain a platform boundary
>
> Which responsibility belongs to Harness, Airflow, Databricks and dbt? What coordination problem would remain if their code were combined into one script?

- **Harness at this level:** versioned release control. It can call infrastructure and application deployment steps, while the released Airflow runtime subsequently manages its data runs. Keep the release commit and execution record connected to deployed code and observed runtime behavior; a Git log alone is not proof of the live state.
- This layering is the engineering technique of **indirection**: introducing something that causes something else rather than doing it directly. Each layer localises one concern:
  - **Harness** — infrastructure (GitHub, Docker, cloud, "renting the building")
  - **Airflow** — timing and SLAs
  - **Databricks** — running the code
  - **dbt** — the business logic itself
- **The small-program thought experiment:** a script can illustrate downloading code, scheduling work, deploying it and checking a result. A shared platform separates these concerns to manage environments, identities, resource lifecycles, approvals and operational responsibilities. Review the purpose of each boundary before adding another layer.
- **Conway's Law:** software systems evolve to match the communication structure of the business. Communication and ownership boundaries influence where interfaces are placed. The paved path looks the way it does because of how Capital Group is organised.

Case study: Decoupling teams through file delivery

Case study: decoupling teams with file delivery

A file-delivery platform separates teams whose systems change at different speeds. In the supplied CG example, direct database-to-database integration made replacement costly because consumers depended on the producer's implementation. The decision was to exchange files through stable product boundaries.

Three considerations explain the choice. Organisational ownership makes a published contract preferable to dependence on another team's database internals. Large data volumes also benefit from distributed storage and processing rather than one machine's limits. Finally, domain sharing and supplier feeds already use object storage: a Morningstar file arrival can participate in the existing S3 event ecosystem.

This approach still needs schemas, arrival promises, quality checks and ownership; folders alone supply none of them. The transferable lesson is to improve an established exchange boundary with explicit contracts and observable delivery rather than assume replacing the whole platform will remove its coordination problem.

The cloud

Knowledge check: Name the service before the acronym

What is being supplied as a service: infrastructure, a managed platform, or an application? How would you calculate a continuously running resource's cost from an illustrative hourly rate?

- "Cloud" means three things: **SaaS** (what a businessperson means, like Word or Google Docs), **PaaS**, and **IaaS**. For this course, IaaS matters most: renting machines, or slices of machine time, from a cloud vendor. PaaS = IaaS plus pre-installed, vendor-managed programs (e.g. renting a managed database).
- What you rent: virtual machines (RAM, CPU, disk, network access) plus vendor services for security and networking configuration.
- **Cost arithmetic.** Use the illustrative rates $1, $10 and $100 per hour to practise estimating a continuously running resource: multiply by the billable hours, about 750 in this example month. At $1/hour that is about $750/month. These are arithmetic examples, not current vendor prices or a margin estimate; real decisions use the approved service, region, usage and billing terms.
- **Translate the service name.** EC2 provides virtual-machine compute; S3 provides object storage. Explain the service role before relying on an acronym.
- **Infrastructure as code.** Versioned configuration describes resources, so a reviewed process can plan and apply changes consistently. The platform team owns provisioning and access; creators review the dependencies their product needs.

Getting onto the paved path after the course

After the course, use the platform onboarding route for your application and account. The training material is good for hands-on, but most of you have applications and AWS accounts connected to them, and you'll more likely be building data products off those accounts. The platform team has built templates for onboarding those accounts, meant to be end to end: the paved path needs GitHub, dbt, Databricks, Astronomer and Harness, and the template has all of those elements built in, with a guide to what you get.

The supplied platform example calls its onboarding template **Lake Launch**. Lake Launch onboards the account, and `catalyst-domain-template` (listed in Appendix C) starts the repository.

- **Three prerequisites.** You'll need an ADM ID (the application identifier), an Astronomer workspace and a Databricks workspace or bundle. If you don't have one you get onboarded and given a Databricks connection. The documentation lays all that out; then you traverse the template and fill in the requirements to provision all of it.

- **Where the code lives.** You've done the discovery and triage and you're ready to build: keep all your code in GitHub. You'll be provisioned a repository, either in the shared organisation or your own space. In the shared organisation the existing data product repositories have the same shape: `airflow` and `astro` folders, a data-products folder with the dbt models, Python scripts tying back to the DAGs, and the Astronomer config. It looks very like Wednesday's dev container layout, and that isn't a coincidence. There's a lot of YAML in there, which is easy for tools and AI tools to read, including a large YAML file for the data contract, and a configuration file that pulls the parameters together.
- **Deploying from it.** From there you deploy onto Astronomer using Harness; the Harness components are rendered off the template. You come into Harness, wire it up to your GitHub repository, and get your pipeline built out, with the same steps you saw earlier: detect changes, trigger on PR or merge, run the downstream steps, pause for the change request.

Reading a release pipeline

Read any release pipeline for these four things.

- **The trigger** — what starts a delivery execution. In the integration and promotion pattern, a reviewed merge or approved promotion can trigger Harness; the execution must still select the exact revision and satisfy its target environment's gates.
- **The stages** — provision the infrastructure, deploy the data jobs and validate, in dependency order.
- **The environments** — dev, SQA and prod, and how a release moves from one to the next.
- **The approval gates** — where they sit, and who holds each one (a role is fine where a name isn't known).

The three kinds of arrow from this morning turn up in a release pipeline too. A trigger arrow says this event starts that one. A stage-order arrow says this stage must finish before the next starts. An approval arrow says someone must say yes before the release goes on.

Compare two real examples. The supplied dev-to-prod promotion runs a change in a lower environment first, and the prod release follows approval. The stage graph in Harness shows a real pipeline's stages, with one run paused at an approval step. Use the Harness anatomy (projects, pipelines, input sets, execution history) to name what you see.

> Learning Review
>
> Trace a reviewed change through its commit, release execution, approval, deployed code and the next scheduled data run. For a failed morning delivery after a release, choose the first release log and Airflow task log you would inspect and explain why.
> Self-check: the same intended change is identifiable across the release record; conditional/skipped stages and waiting approvals are explained; the runbook names an owner, recovery decision and consumer update. Successful infrastructure provisioning alone does not establish correct published data.

The 3 a.m. failure sequence

> 3 a.m. dependency failure — SLA AT RISK
>
> The layer stack stays abstract until something fails, so let's run it backwards from an alert. It's 3 a.m. The nightly DAG has failed a dependency check, the same failure as in the support case in the guest example. The consumer's contract promises the data by 7:30, so that SLA is now at risk.
> Where do you look? Each layer answers a different question:

> Knowledge check: Make a recovery decision
>
> A dependency check fails before the consumer deadline. Which evidence would you inspect first, and what must be true before you replay and publish?

- **Airflow task logs** — did the job run at all, and which task failed?
- **Databricks job and query history** — did the SQL itself fail, and on which query?
- **Harness execution history** — did someone deploy last night? A failure the morning after a release points first at the release.
- **Datadog** — the observability platform where the data-operations group keeps **DAG monitors** (did the DQ check pass, did the DAG finish inside its window) and per-product dashboards; who was alerted and when, and which dashboard shows the SLA breach? The routing table from earlier today is what fires from it.

Then someone has to tell the consumer. The message says who is being told, by when, what's known, what's still uncertain and when the next update will come.

The drill leans on three things you already have. The first is the runbook from earlier today, with the full template in Operations: ownership, dependencies, dashboard and log links, alert-to-response steps, rollback choices, communication routes and recovery checks. The second is alert thresholds. A metric that drifts raises an alert without paging anyone until it crosses the agreed threshold, while a failure that threatens the SLA pages the responsible role. A production illustration might compare a run duration with its recent baseline; the assigned v2 phase does not configure that monitor. The third is the difference between monitoring and observability. Monitoring watches the measures we chose in advance. Observability lets the operator dig through logs, metrics, lineage and run history when the failure doesn't match anything we've seen before.

Operations reference

So what happens after your code is merged? Harness releases it, Terraform and Helm set up the infrastructure it runs on, Airflow runs it and Datadog watches it. The platform team drives most of these tools. You still need to know which tool owns which part, so that when something breaks you can tell the right team what failed and where. This reference covers runbooks, alerts, the incident log, scheduling and the deployment tools.

The worked examples use Morningstar exposure data, so swap in the names, thresholds, owners and commands from your own product.

Runbook template

Keep the runbook in the same repository as the DAG. If a pull request changes a task, a dependency, an alert, an owner or a recovery command, it changes the runbook too.

A runbook has seven sections.

1. **Service overview.** The product, what the consumer uses it for, the `dag_id`, repository, owner, timezone, schedule, delivery promise, normal duration and normal row counts.
2. **Architecture.** The path `source → ingestion → transformation → publication → consumer`, who owns each dependency and how you know each input is ready.
3. **Access.** Links to Airflow, Databricks, Datadog and source control, plus a query that checks the product.
4. **Common alerts.** Use the alert table below.
5. **Escalation.** The on-call groups, the business contact, the response window and the P1 route.
6. **Manual procedures.** The retry, restore, full-refresh and backfill commands, written out in full, with how to check each one worked.

7. **Contacts.** The owning team and source teams, consumers, platform support, Teams channels, xMatters, dashboards and maintenance windows.

Name roles or groups in the contacts, because people move teams. Keep credentials out of the file.

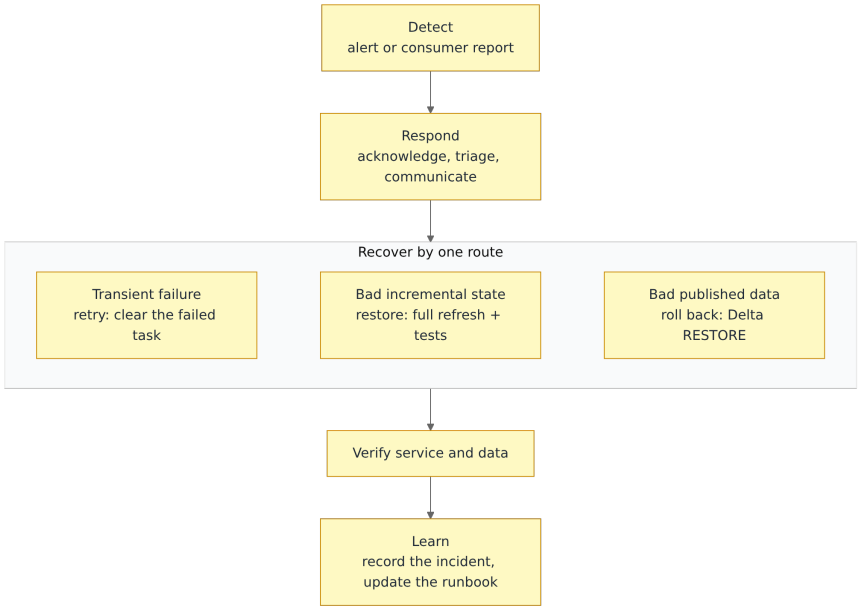


Figure 53: Incident lifecycle. You detect and respond, recover by one route, check the service and the data, and update the runbook when you find a new failure mode.

Common alerts

Alert	*Symptom*	*Diagnosis*	*Remediation*
Source freshness error	Freshness check fails before transformation.	Read the dbt output. Find the stale source and query its latest load timestamp. Check the delivery with the source owner.	When the feed arrives, clear the check and downstream tasks. If the promise will be missed, escalate and notify consumers.
dbt run failure	A model fails, and the models downstream of it don't run.	Find the model and its compiled SQL. Check the warehouse, the source object, permissions and any recent change.	Retry a transient failure. For code, fix, review and release, then rerun from the failed model.
dbt test failure	A quality rule returns failing rows.	Name the test and the failing row count, and look at the rows. Trace the defect back through mart, intermediate, staging and source.	Block publication. Escalate a source-data problem or fix the transformation logic. Restore clean published data if you need to.
SLA miss	The DAG is incomplete at the promised time.	Find the first late task. Compare source delay, retries, queue time and warehouse performance with recent run times.	Update consumers. Fix the cause, then clear the failed task. Change compute or the schedule only when the run history backs it up.

Workflow

Recovery procedures.

- **Clear the failed task and its downstream tasks** after fixing the cause. Check that each expected task runs once.
- **Run `dbt run --full-refresh`, then `dbt test`** when sources can rebuild corrupt incremental state. Check the grain, row counts and tests.

- **Run `RESTORE TABLE … TO VERSION AS OF …`** when published Delta data is bad and a clean version exists. Check the table history and the consumer query.

Incident-log schema

Write one row per incident. These seven fields are enough.

Field	*Content*
`incident-id`	Stable `INC-YYYYMMDD-NN` identifier shared by Datadog, the incident thread, and follow-up
`severity`	P1, P2, or P3, matching the alert tier
`first-seen`	UTC timestamp emitted by the alert
`root-cause`	One-line cause, distinct from the visible symptom
`resolution`	Action that restored service or data quality
`runbook-step`	Entry used, or `none — new failure mode`
`owner`	Responder who resolved it, or the current escalation owner

Write the row at resolution time and keep it beside the runbook. When `runbook-step` is `none`, add the new failure mode and its recovery procedure. When a listed step did not resolve the incident, revise that step using what the log shows.

Keep timestamps in UTC so Datadog, Airflow and the incident thread line up without conversion. For the root cause, write down what started the incident. The alert text usually only describes the symptom. A resolved incident looks like this:

```yaml
- incident-id: INC-20260824-01
  severity: P1
  first-seen: 2026-08-24T10:05:00Z
  root-cause: Morningstar exposure file arrived three hours late
  resolution: source arrived; freshness check and downstream tasks reran
  runbook-step: failure-scenarios/source-freshness-error
  owner: wealth-management-data-oncall
```

Capital Group alerting

In the supplied CG alerting convention, Datadog receives automatic events. The severity and routing tags on the event decide what a person does next.

Priority	*Typical condition*	*Human response from the Datadog event*
P1	SLA breach, or failure after retries	Support manually pages **xMatters**.
P2	Quality or freshness warning	A human posts to **Microsoft Teams**.
P3	Performance or capacity warning	The team creates a **Jira** ticket.

The supplied alert-routing case disallows `email_on_failure`; validate the current policy with the operating owner before applying that restriction to a live product. xMatters, Teams and Jira are human actions from the Datadog payload. Every event carries `l1_domain`, `l1_xmatters_group`, `lifecycle_phase`, `data_classification`, and `criticality`.

> Expert
>
> Group related failures, deduplicate by incident ID, and link P1 and P2 events to the runbook. Audit 30 days of P2s monthly; when more than 20% led to no action, move the condition to P3 or remove it.

MDS tag	*What it supplies to the response*
`l1_domain`	Owning business or data domain
`l1_xmatters_group`	On-call rotation that support uses for a P1 page
`lifecycle_phase`	Whether the product is in build, warranty, or operation
`data_classification`	How sensitive the event payload and linked logs are
`criticality`	P1, P2, or P3 response tier

The event body should carry the failing DAG and task, execution time, Airflow log URL, runbook link and a deduplication key for the execution. The five tags tell support who owns the event and where to send it. Working out what went wrong still needs the log. If an environment has no Datadog credentials, the callback should log and return so the DAG can still parse. Track the missing integration as an environment defect.

So what's a Datadog **monitor**? It's one configured condition. Did the DQ check pass? Did the DAG finish inside its window? The data operations group watches the monitors for the DAGs that need eyes on them, and runs the operating procedure when one goes red or reports no data. **Dashboards** exist per product, and the marketplace has one. Ask the platform team which monitors your product has.

Timezone and scheduling

Work the schedule out from when the business needs the data. Cron expressions are one supported scheduling rule; Airflow also supports timetables and data-ready triggers.

Timing pattern	*Schedule or trigger*	*Use it when*
Weekday batch	`0 5 * * 1-5` UTC	The UTC relationship is intentional and weekend source files do not arrive.
Market-hours refresh	`0 9-16 * * 1-5` in the business timezone	The local trading-hour window must remain stable through daylight-saving changes.
Month-start batch	`0 6 1 * *`	The run is tied to the first of the month, whatever state the source is in.
Data-ready run	Dataset event or sensor	The data arriving should start the run.
Arrival trigger	S3 event, SQS, Lambda, then the DAG and dbt	A Morningstar file landing in S3 should start the run. S3 notifies SQS and a Lambda triggers the DAG.

For a local-time promise, use a timezone-aware start date so daylight-saving changes do not move the local hour. In Python, for example, set `start_date = pendulum.datetime(2026, 4, 7, tz="America/New_York")` after importing `pendulum`.

A fixed `05:00 UTC` schedule runs at midnight during Eastern Standard Time and 01:00 during Eastern Daylight Time. That may be right for a UTC source hand-off. If the business promise is tied to a New York clock time, set the DAG timezone and state the local-time rule in the runbook.

```
usable buffer = SLA deadline - latest planned start - normal runtime - retry allowance
```

For a 07:00 ET delivery, 03:00 start, 25-minute run and 20-minute retry allowance, the buffer is **3 hours 15 minutes**: four hours between start and deadline, less 45 minutes for the run and one retry. Compare that remaining buffer with the source delivery promise. If likely source delay consumes it, move the planned start only when the source can be ready earlier; otherwise use a readiness trigger and renegotiate the downstream promise.

Deployment tools

These are release-control responsibilities, not commands assigned in the local learner task. Current Astro documentation separates project-image, DAG-only and dbt-project deployment payloads; choose the approved route for the released runtime rather than assume `--dags` ships every project dependency. Astro deployment methods.

Tool	*Owns*	*Trigger*	*Shipped artefact*
Harness	Approval, environment promotion, secret references and deployment audit	Git push, pull-request merge, or authorised release	Calls the commands declared by the pipeline
`astro deploy --dags`	Airflow-side delivery	Harness pipeline step	DAG source; image/ dependency changes use a project-image deploy
`databricks bundle deploy` (DAB)	Databricks-side delivery	Harness pipeline step	Jobs, pipelines, notebooks, models, dashboards and workspace configuration

Harness supplies the production gate and records who approved the release. A Databricks bundle has no approval gate of its own. Apart from the target workspace logs, it keeps no record of what was promoted where. When Harness calls `databricks bundle deploy`, the approval and the promotion record stay with Harness.

Each deploy command ships only its own side. A DAG-only deploy releases DAG source to Astronomer, and `databricks bundle deploy` creates the jobs and other resources in the Databricks workspace. Airflow can then trigger a deployed Databricks job and wait for it. The DAG still decides the schedule, the dependencies, the retries and what runs next. The bundle packages the Databricks resources, and the transformations inside them are still dbt, PySpark, SQL or whatever else you wrote.

When a release goes wrong, look in the tool that owns that part. Harness shows the commit, target, approval and command result. Astronomer shows whether the expected DAG version is registered. The Databricks workspace shows whether the named resource exists with the released configuration.

Releasing while a workflow is running

Assume a daily run has completed staging and is waiting to start integration. A release changes the integration output column that its publication task reads. Before deploying, establish which code and dependency versions the pending tasks will use, whether that behavior is configurable, and whether both task interfaces remain compatible. Executor and runtime behavior determine the answer; an active run is not a universal guarantee that every later task stays on its original revision.

Choose an explicit transition: deploy a compatible change, drain the affected work before release, or use a supported version-consistent execution mechanism. Record the run, input partition, code/image identity, remote job, and candidate so the operator can distinguish a mixed-version failure from bad supplier data. Test this boundary in a prepared non-production environment rather than infer it from a DAG-only deployment's speed. Astro operations develops runtime ownership, configuration authority, upgrade checks, capacity, and release consistency.

Harness on screen

Three Harness terms come up all the time.

- **Input sets** are saved groups of parameters, one per environment. A production deploy becomes a click into a template, so nobody retypes the inputs and gets one wrong.

- **Triggers** are GitHub webhooks to Harness. They fire on a pull request, a merge or any code change.
- **Trigger and promotion** are separate decisions. A reviewed merge can trigger delivery, while the selected revision, target environment, human approval and change-request gate determine the actual deployment. A merge webhook alone does not establish a complete GitOps model.

A typical pipeline is set up like this.

- A `.harness` folder in the repository holds the pipeline YAML, which started life as a DevHub template.
- A Harness project holds pipelines named after their repositories.
- Each pipeline has an execution history with one entry per run. Every stage has its own log (build, deploy dev, deploy QA, deploy prod), and failed security scans block the production stage.
- Per-environment `.tfvars` files hold the Terraform values, such as the ADM ID, namespace and AWS account ID.

Thursday review

Use one product journey: source arrival, freshness/quality gate, transformation, publication, consumer delivery and recovery. Connect it to the reviewed commit and the runtime that executed it.

> Learning Review
>
> Retrieve: explain DAG/task/run/operator, schedule/dependency/retry, and release/run/approval. Then identify which promise each control serves.
> Apply: the scheduled run used yesterday's source after a new release. Decide whether to pause publication, inspect the release, repair the input, or replay. Name the evidence needed before each decision; an attractive green dashboard is insufficient.
> Self-check: your handoff states the business date and deadline, source identity, released commit, run/task/attempt, failed gate, recovered output and consumer communication. You separate log-only learner evidence from production alerting and keep infrastructure changes with the platform owner.

On screen: Pull requests in practice

What this sequence controls

A pull request proposes changes for discussion and review before they enter a branch. Its diff and checks supply evidence about the proposed commit; its description connects that change to the product contract. A review decision is a release input, while a later deployment/run provides separate evidence about the live product. GitHub defines this review workflow.

These are pull requests on a real infrastructure repo. The disaster-recovery PR is a documentation proposal. Its timings are unmeasured and its implementation decisions remain open. Passing automated checks does not satisfy the outstanding review gate or demonstrate failover. Read the proposal and approval state together.

When you hand over a change on this course, write the description the same way. Say what changes for the consumer, which files you touched, which tests cover it and what you know is still missing. A dbt fix may need a contract or glossary change too. You'll practise on a small training diff.

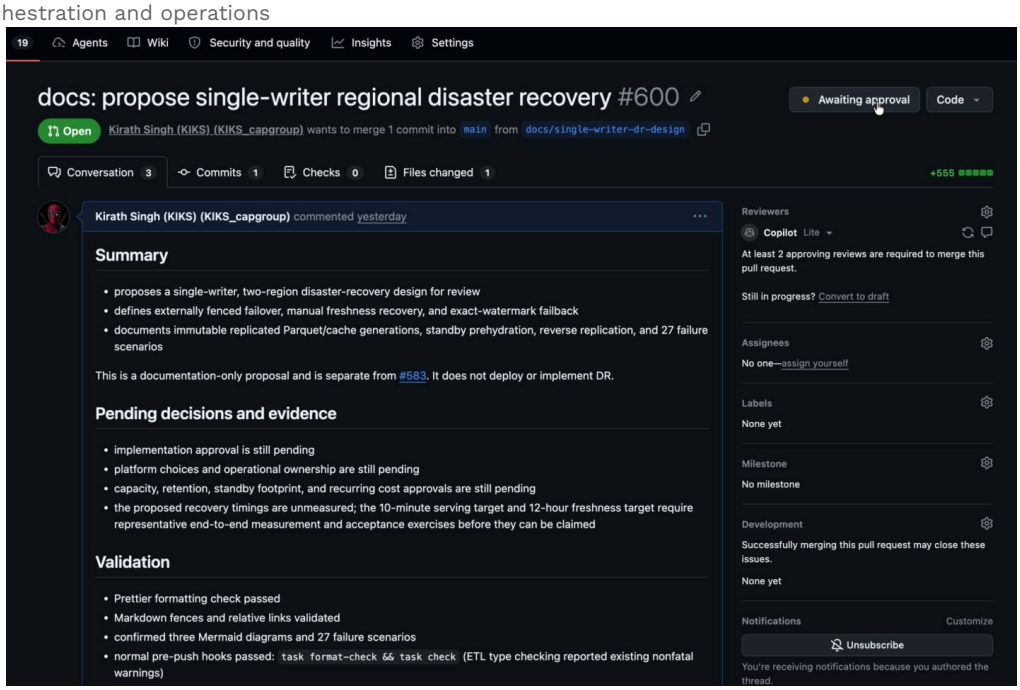


Figure 54: A disaster-recovery documentation proposal

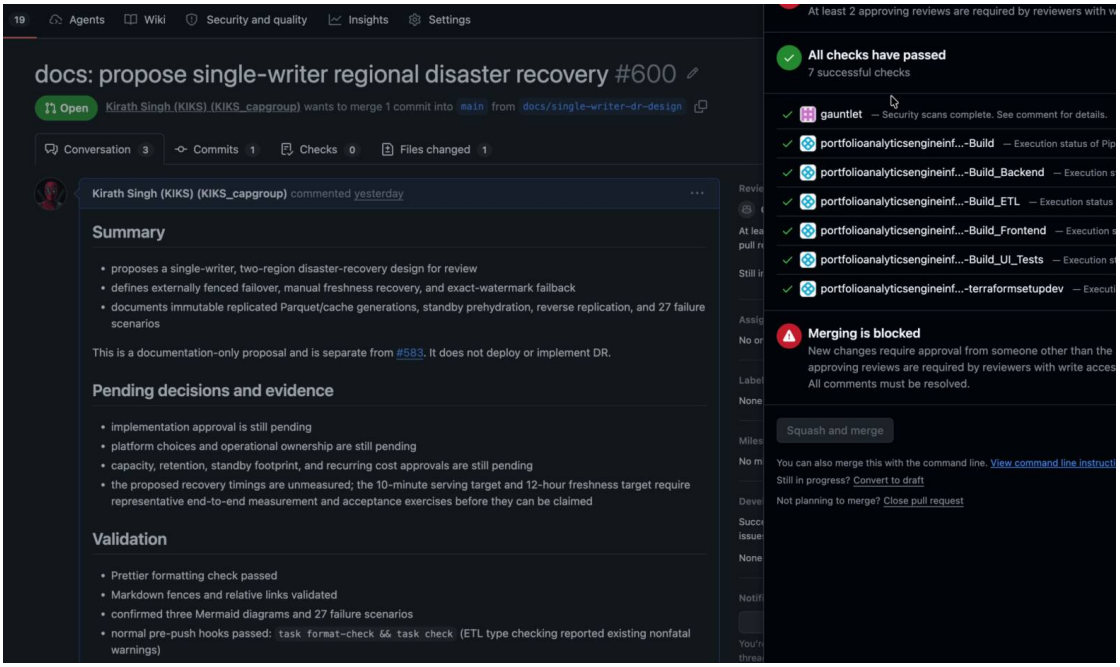


Figure 55: Checks pass while required approval still blocks merge

On screen: The notifications Lambda

What this sequence controls

AWS Lambda runs a handler in response to an event without the product team operating a dedicated server for that handler. Its input is the event payload; its output here is a request to start the appropriate Airflow DAG, together with a recorded trigger result. It does not transform the holdings or prove that a downstream job finished. AWS describes the event-driven handler model.

In the demonstrated notification flow, decode the dataset identifier, select a configured target, submit the DAG trigger and inspect errors. The supplied Kafka, S3/EventBridge and SQS variants illustrate different event boundaries; their acknowledgements and duplicate/retry handling require their own design. Follow

an accepted trigger into the named DAG run and then check its publication evidence. This is a platform demonstration, not a learner Lambda/Kafka/S3 provisioning task.

An arrival event can start a DAG through several routes. Here are three. In the first, a Kafka source invokes a notification Lambda. Its handler decodes the record, works out the target dataset and asks MWAA to trigger a DAG. In the second, S3 events go through EventBridge to a Lambda. In the third, SQS feeds the Lambda. Each one gets different inputs, wraps its events differently and retries differently.

The schema and the target-dataset list show why an event needs more than “data arrived”. The consumer has to work out which dataset it is, ignore events it doesn’t care about, and cope with the same event arriving twice or a trigger that fails. A triggered DAG has only started, so check its run to see whether the transformation finished.

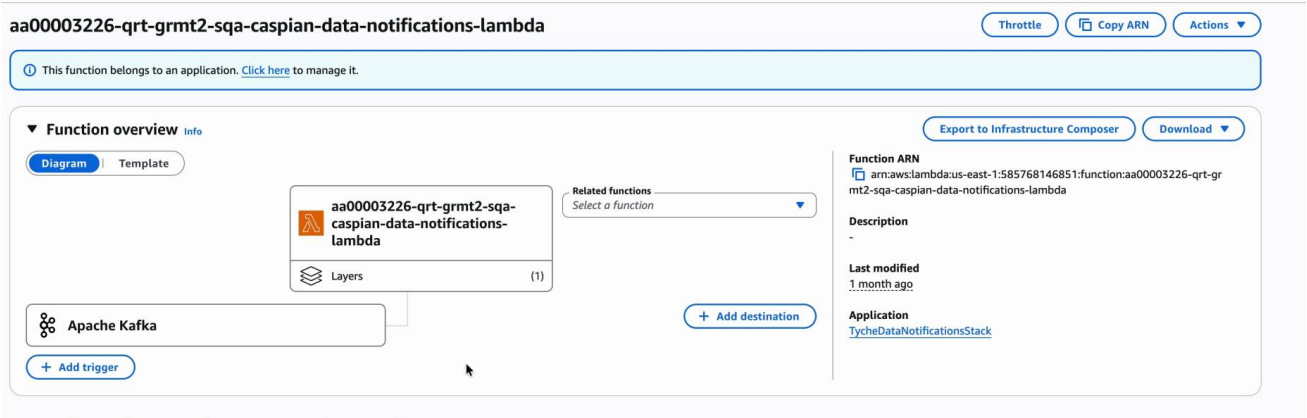


Figure 56: Function overview & trigger

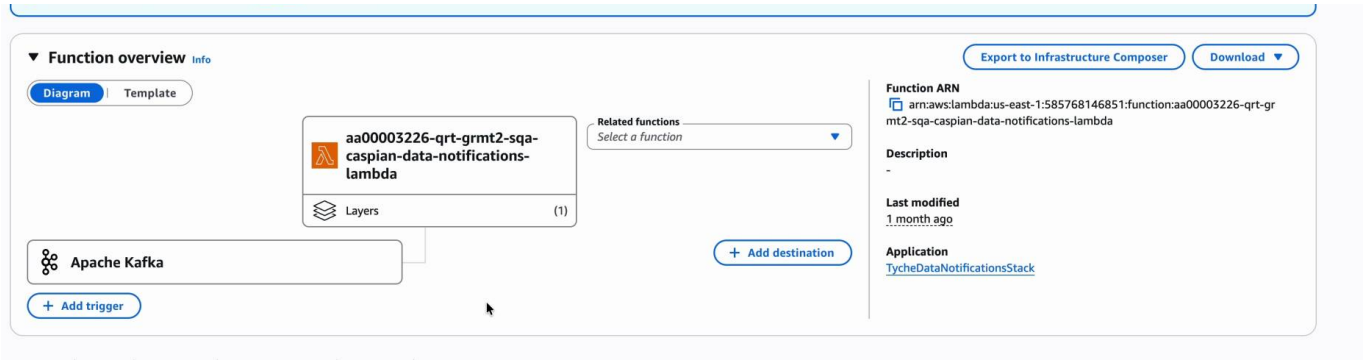


Figure 57: Detail of Function overview & trigger

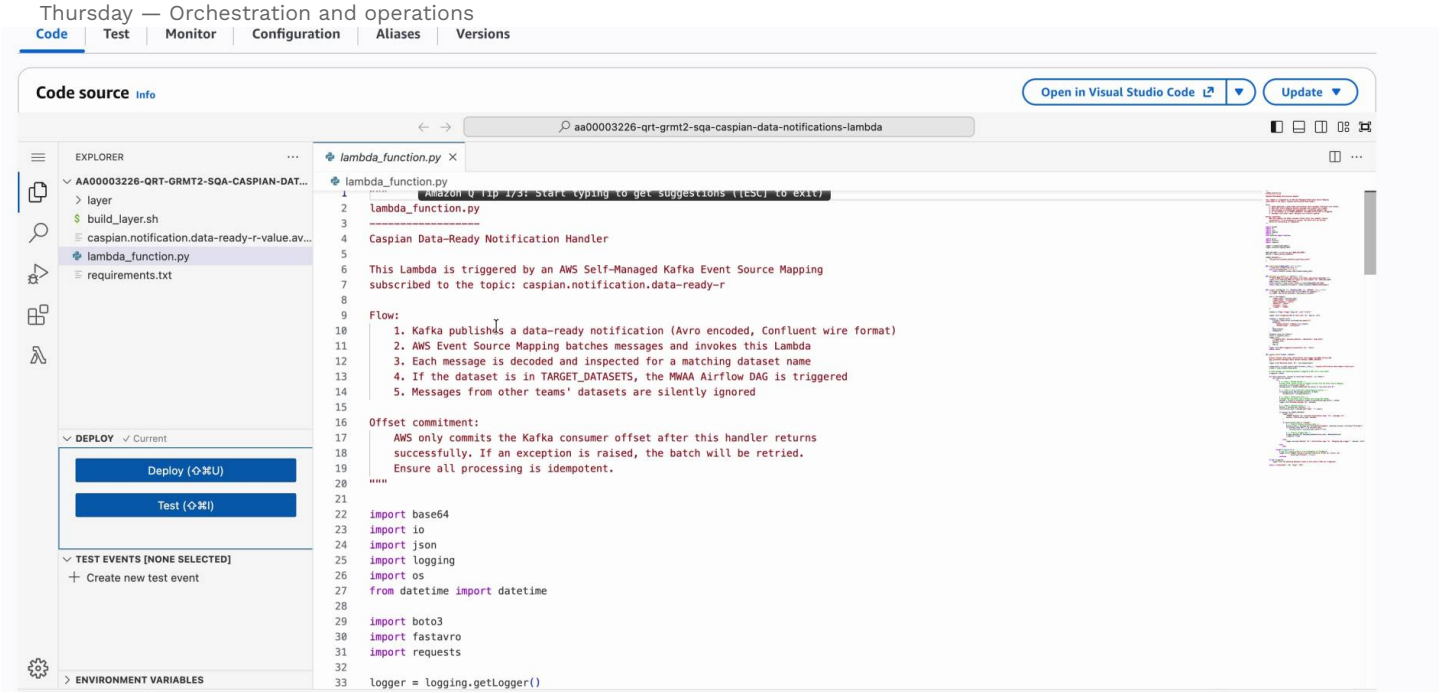


Figure 58: What the function does

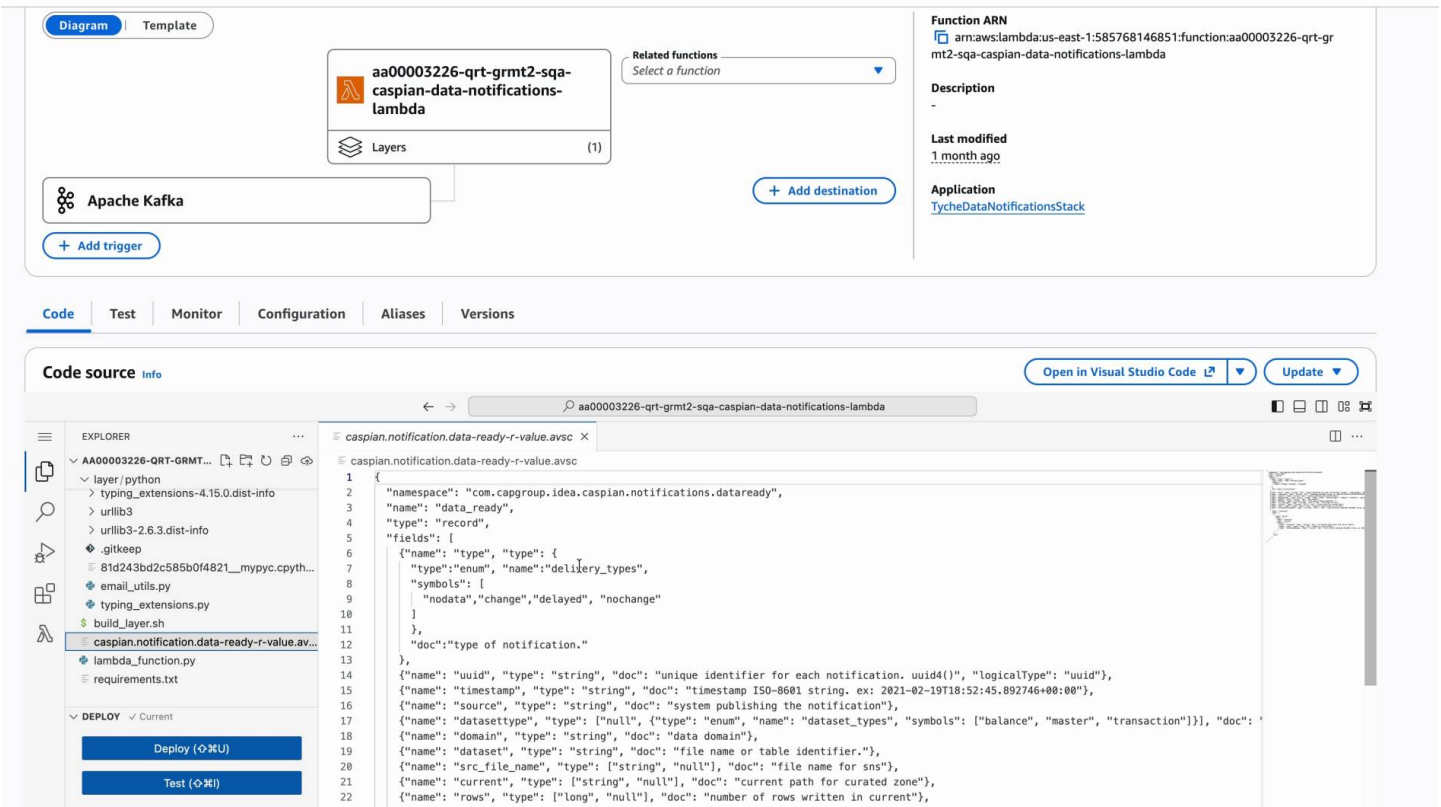


Figure 59: Data-ready Avro schema

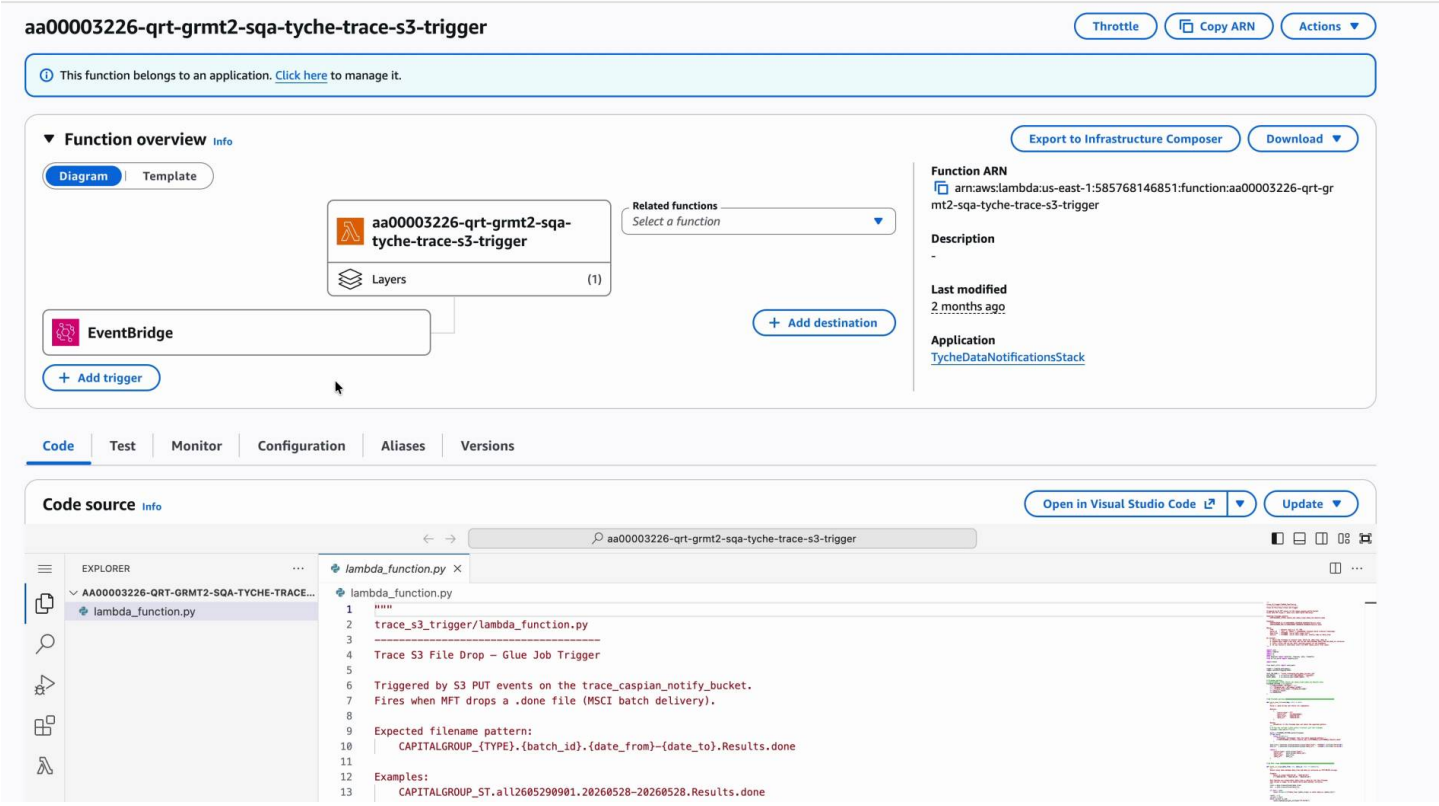


Figure 60: S3 to EventBridge to Lambda

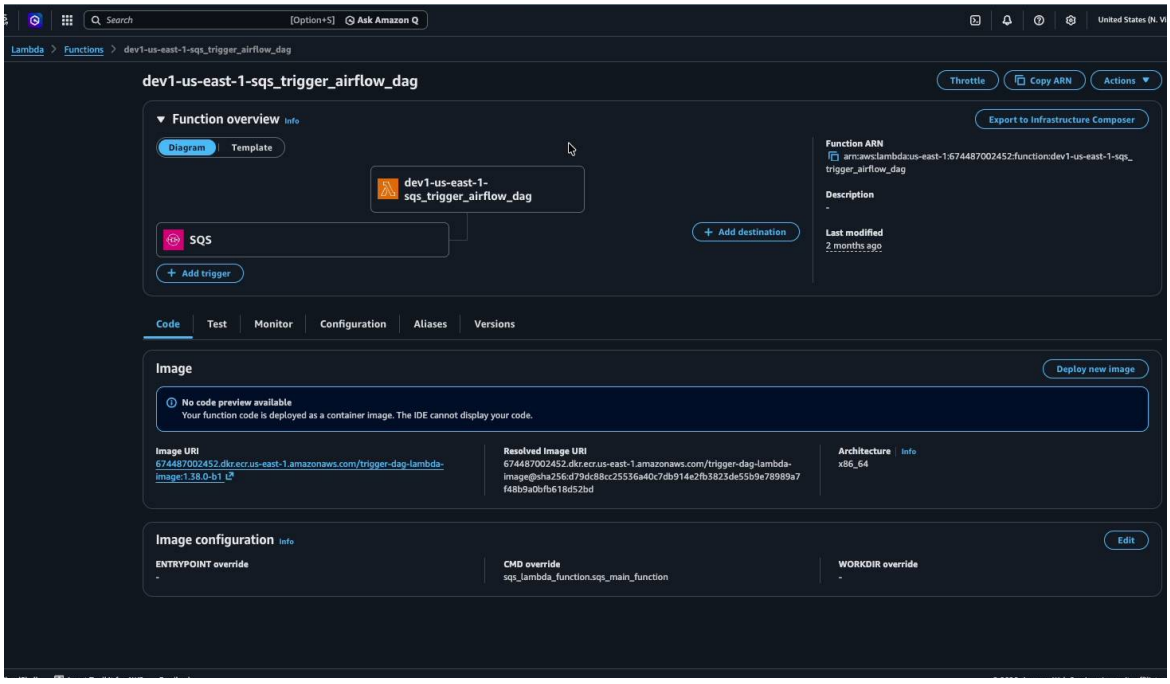


Figure 61: SQS to Lambda to Airflow

On screen: Amazon MWAA

What this sequence controls

Amazon Managed Workflows for Apache Airflow (MWAA) is AWS-managed hosting for Airflow. AWS operates the runtime infrastructure; DAG authors still define tasks, dependencies and scheduling. Environment configuration and access determine which DAG files and resources that runtime can use. AWS describes those managed-service responsibilities.

MWAA and Astronomer are two hosting routes around Airflow, with different environment/access management. The screenshots show MWAA; the assigned v2 Astro runtime has its own version, access and worker checks. Compare the environment/version and the DAG's task/run evidence instead of assuming every managed service has identical menus or credentials.

Figure 62: MWAA environments

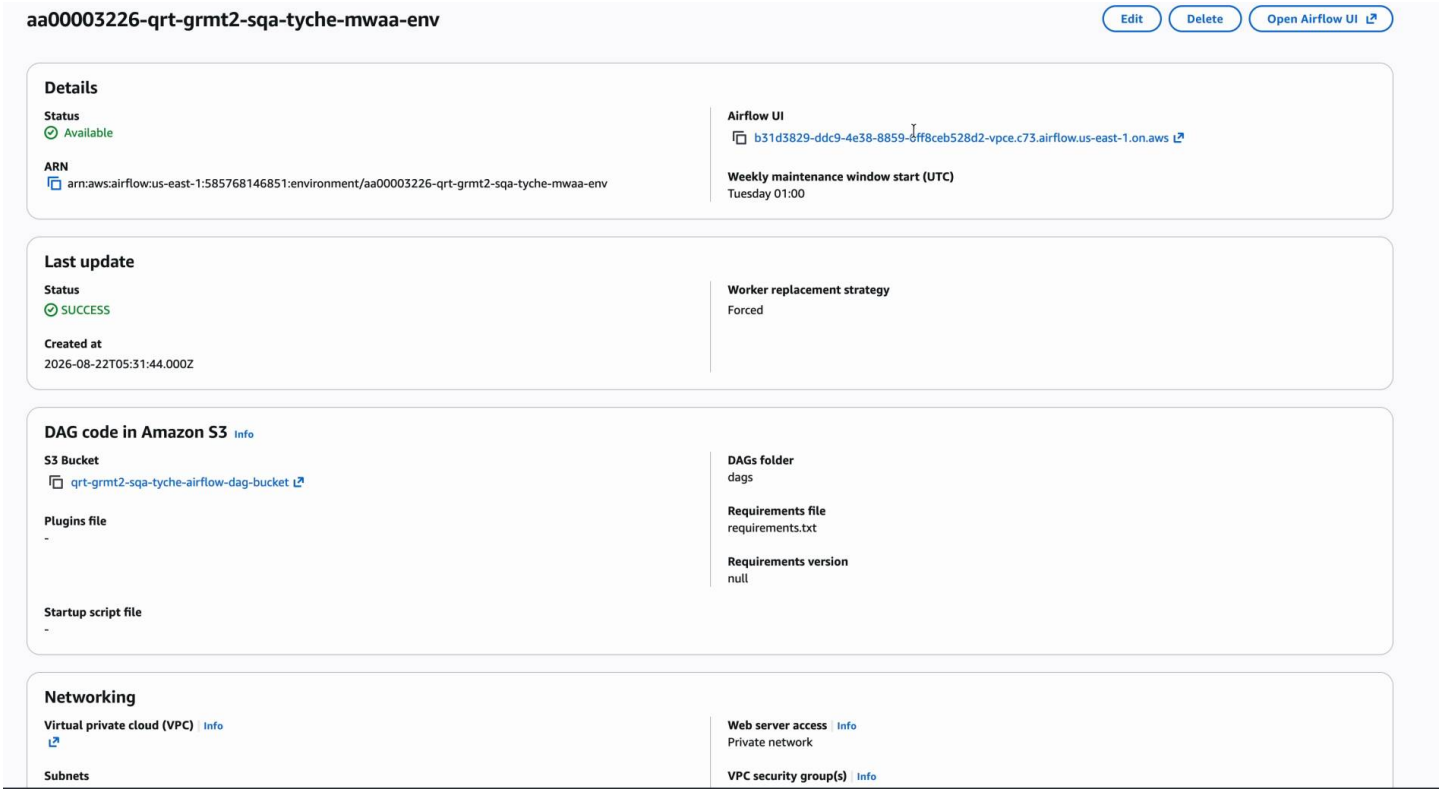


Figure 63: Environment details

On screen: Finding DAGs

What this sequence controls

A DAG definition describes a recurring workflow; a DAG run is one execution of that definition. The list helps locate the workflow, but a schedule or latest-status badge does not establish that the required business-date delivery is correct. Choose the product, environment and run identity first. Airflow explains the run/definition distinction.

The DAG list is where you start looking at runs. Filters narrow it down by product, status or tag. Filtering to the latest failure hides earlier successful runs and other failures, so check which environment and which DAG you're looking at before you read anything into a row.

The DAG list contains daily, monthly, manual and environment-specific workflows. In the Portfolio Analytics phase, find your `<prefix>_portfolio_daily` DAG. The CG capstone has its own instructor-assigned workflow identities; follow its phase-2 orchestration and phase-3 blueprint instructions.

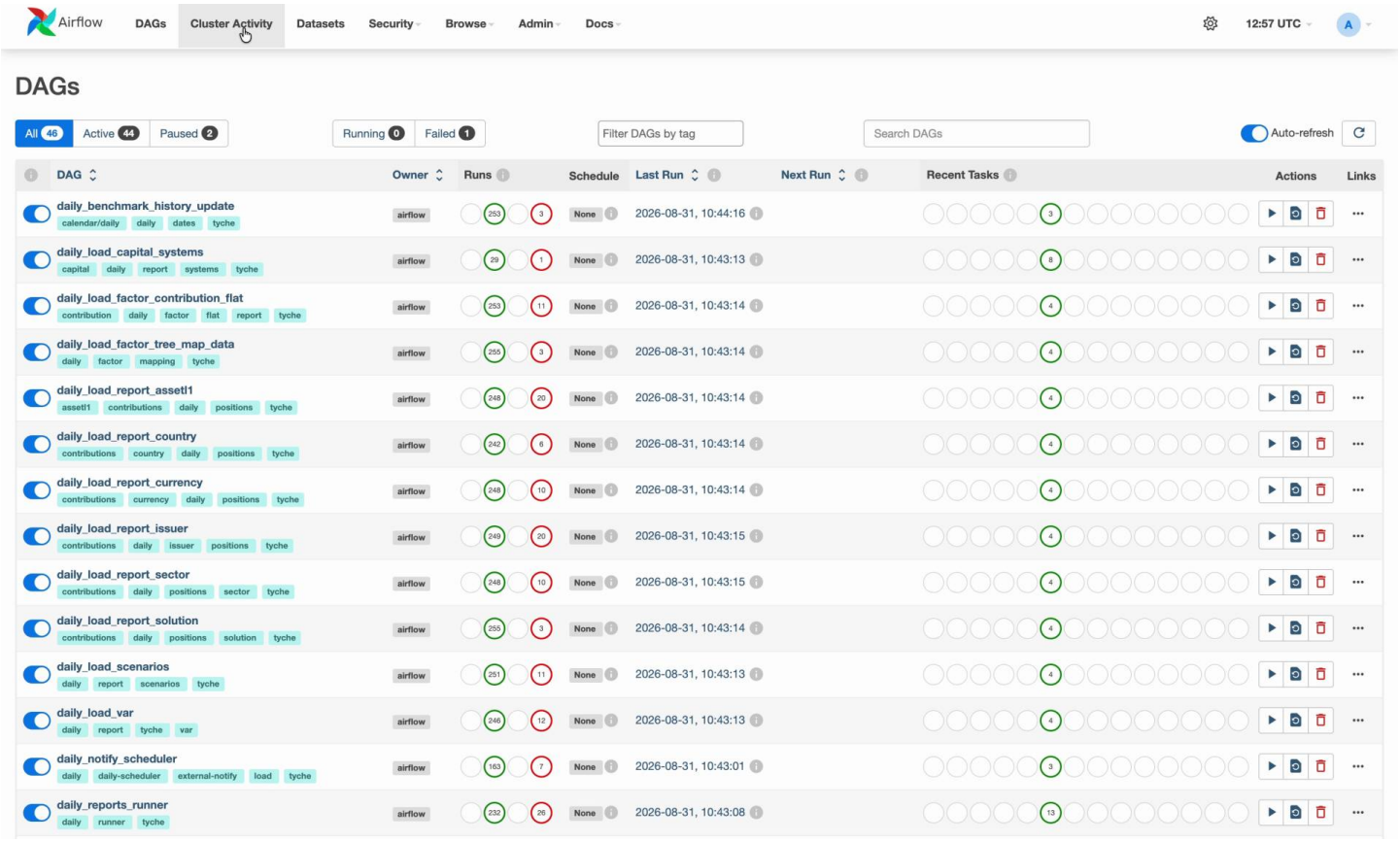


Figure 64: DAG list

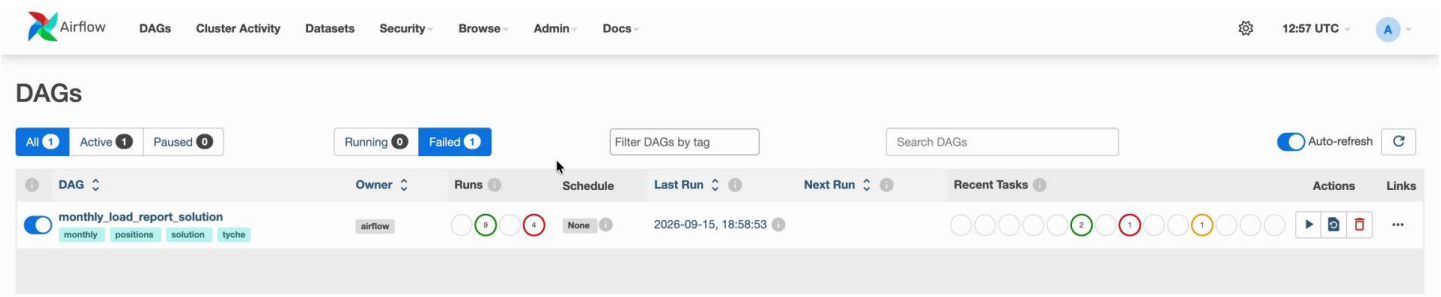


Figure 65: Filtering to failures

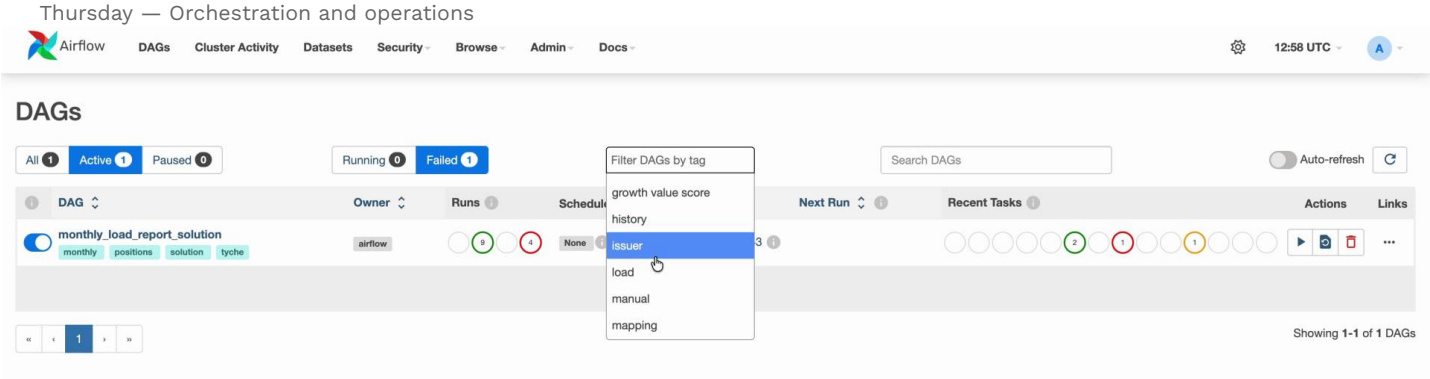


Figure 66: Filtering by tag

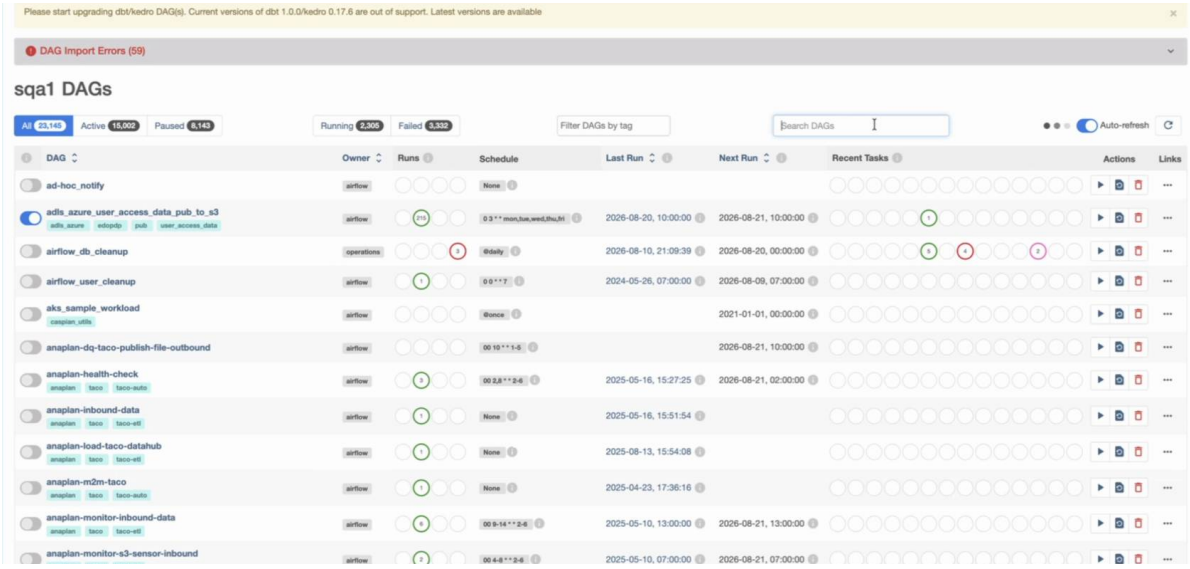


Figure 67: DAGs in SQA

On screen: DAG structure

What this sequence controls

The graph expresses task dependencies; the selected run's grid supplies the observed states. Independent branches can become eligible together, while a downstream task waits according to its dependency and trigger rules. A graph is a design claim; logs and output checks establish whether one execution satisfied it. Use the selected run rather than infer a result from the definition alone.

Compare the graph with the chain you meant to build: freshness check, then transformation, then verification. If a dependency is missing, the downstream task can run before the freshness check has passed. If it's there, a failed freshness gate stops the work after it. Then read the task logs and check the output to see what the selected run actually did.

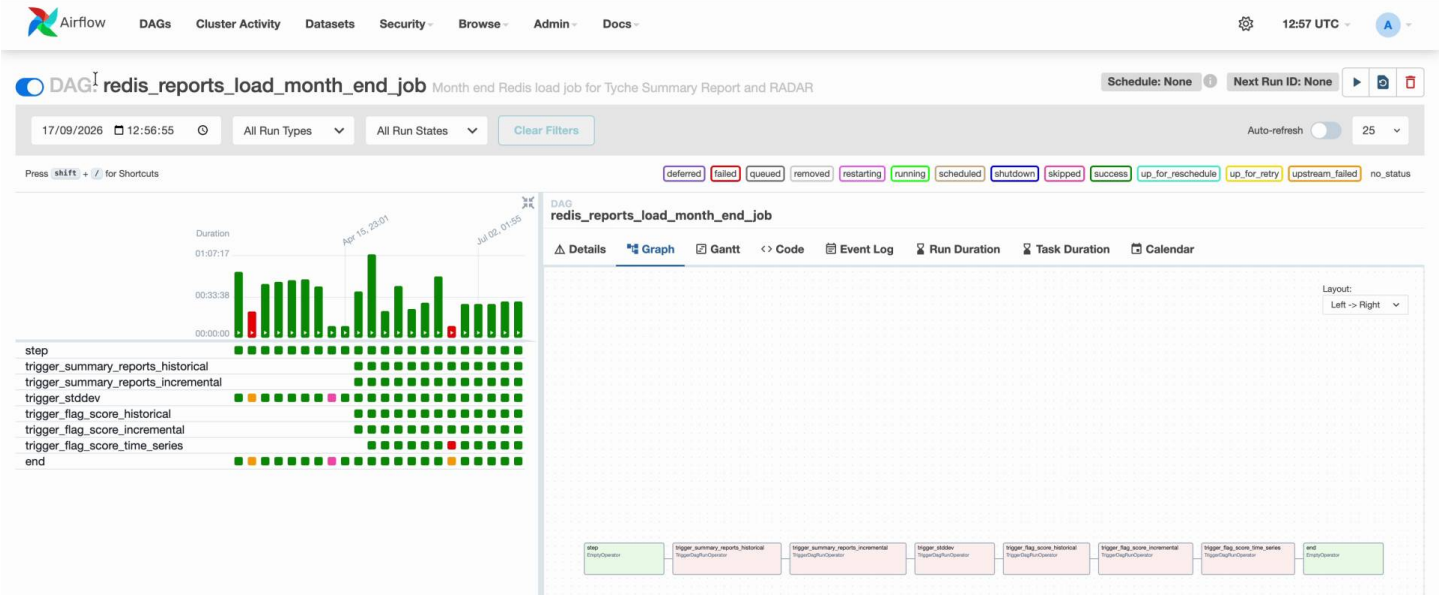


Figure 68: DAG graph

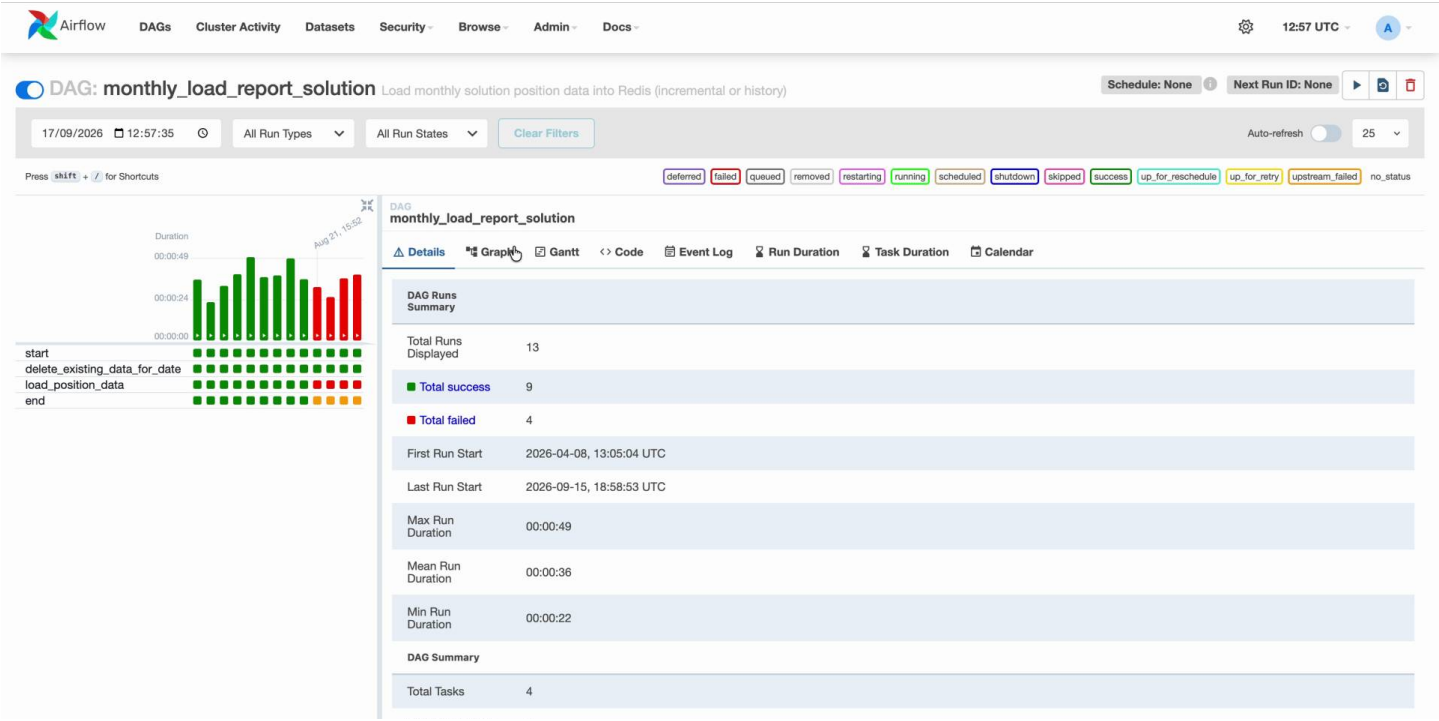


Figure 69: Run history with failures

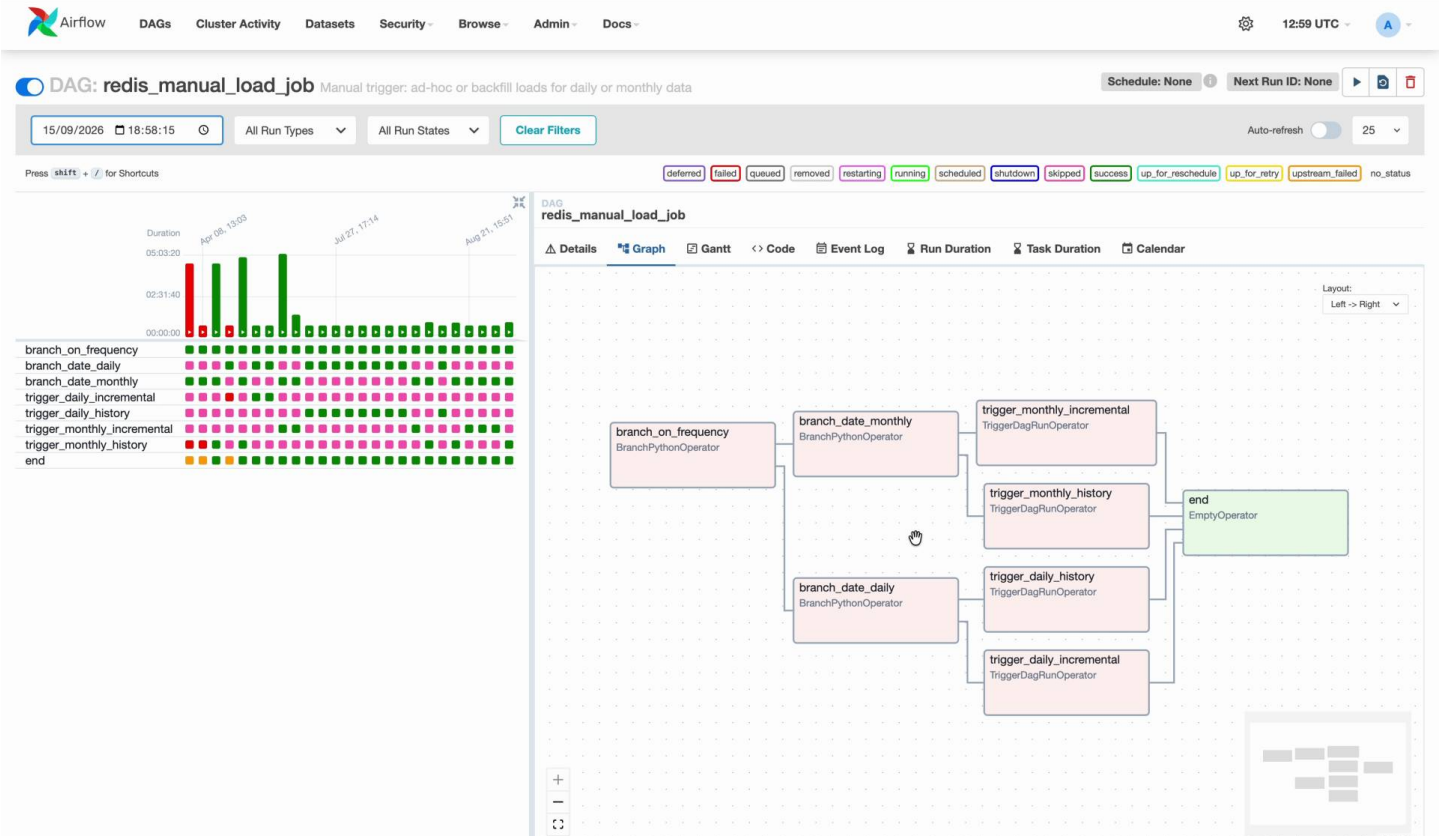


Figure 70: Branching

Figure 71: Ingestion DAG graph

On screen: Runs & task instances

What this sequence controls

A task instance is a particular task in a particular run. Diagnosis needs its run identity, state, attempt, timestamps and log, because the same task definition can have many different results. A blocked downstream state points to an upstream condition; it is not evidence that the downstream transformation ran and failed. Retain those identifiers alongside the independent table/output check.

When you're diagnosing, write down the DAG, run and task before you copy an error. Tell apart a task that failed itself, one marked `upstream_failed` and one skipped on purpose. After a fix, look at the new attempt and at the product output. Menus move between Airflow versions, so find the same controls in the version you're running.

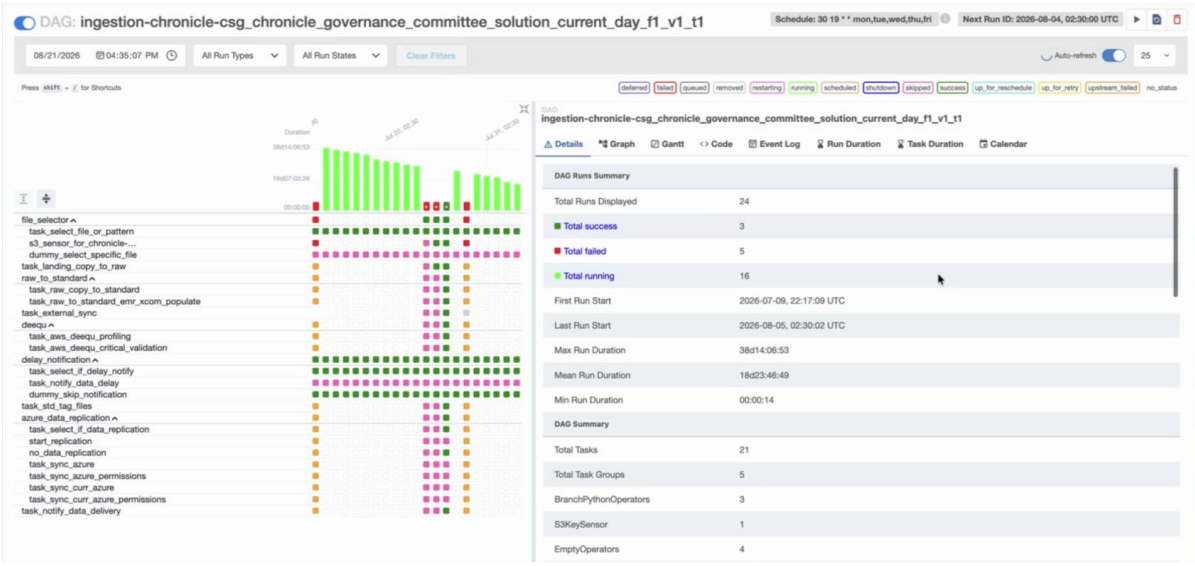


Figure 72: Grid view & run summary

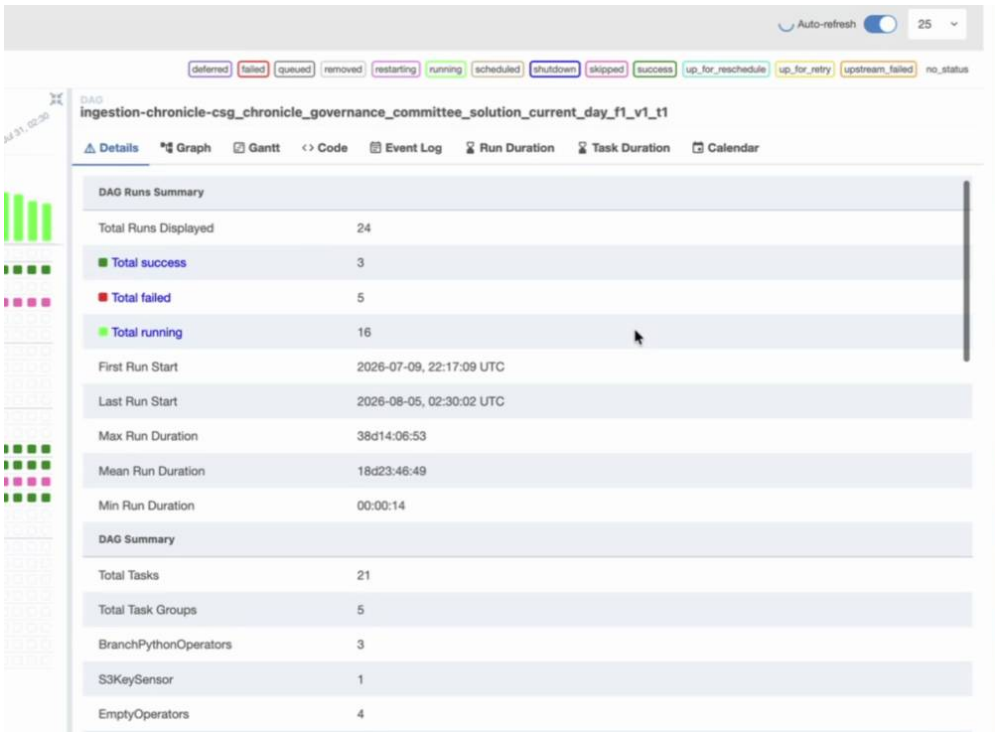


Figure 73: Detail of Grid view & run summary

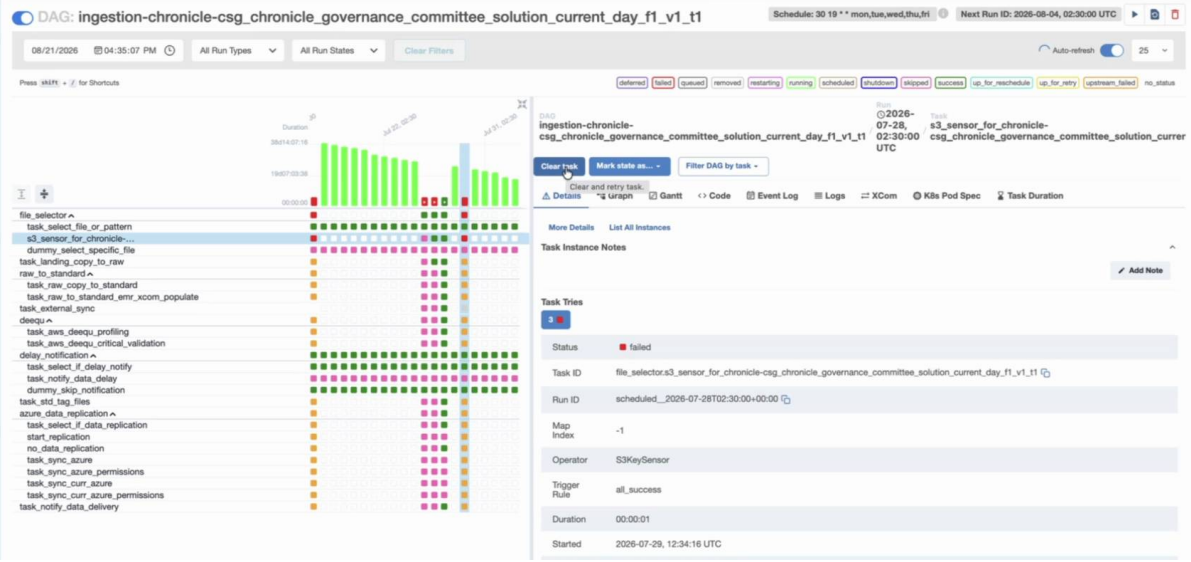


Figure 74: A failed task instance

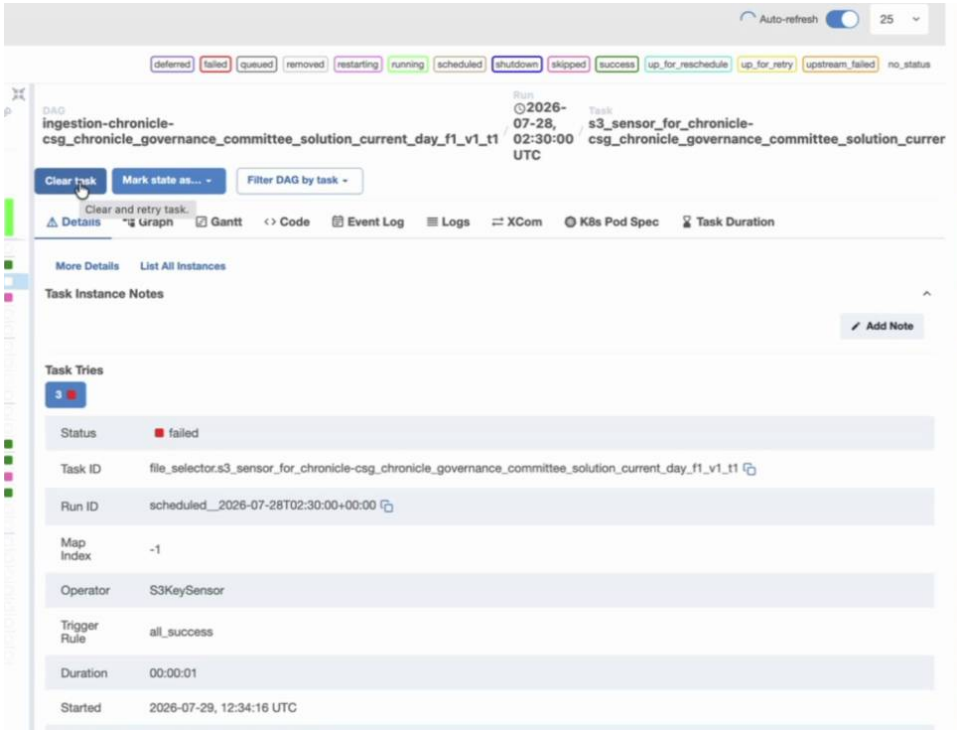


Figure 75: Detail of A failed task instance

Figure 76: A successful task instance

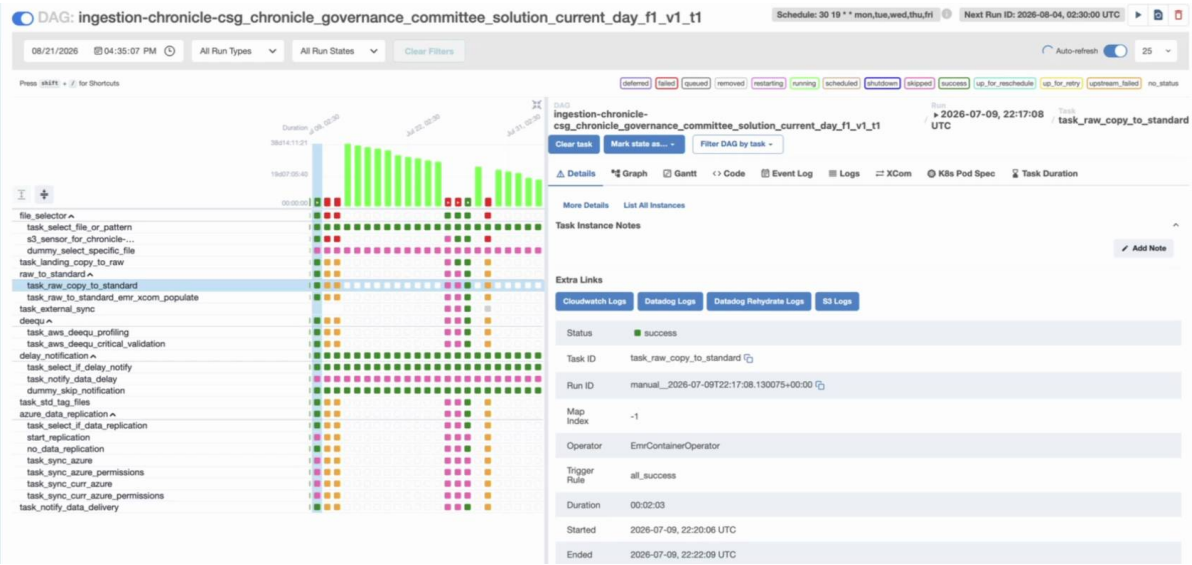


Figure 77: Task details in the grid

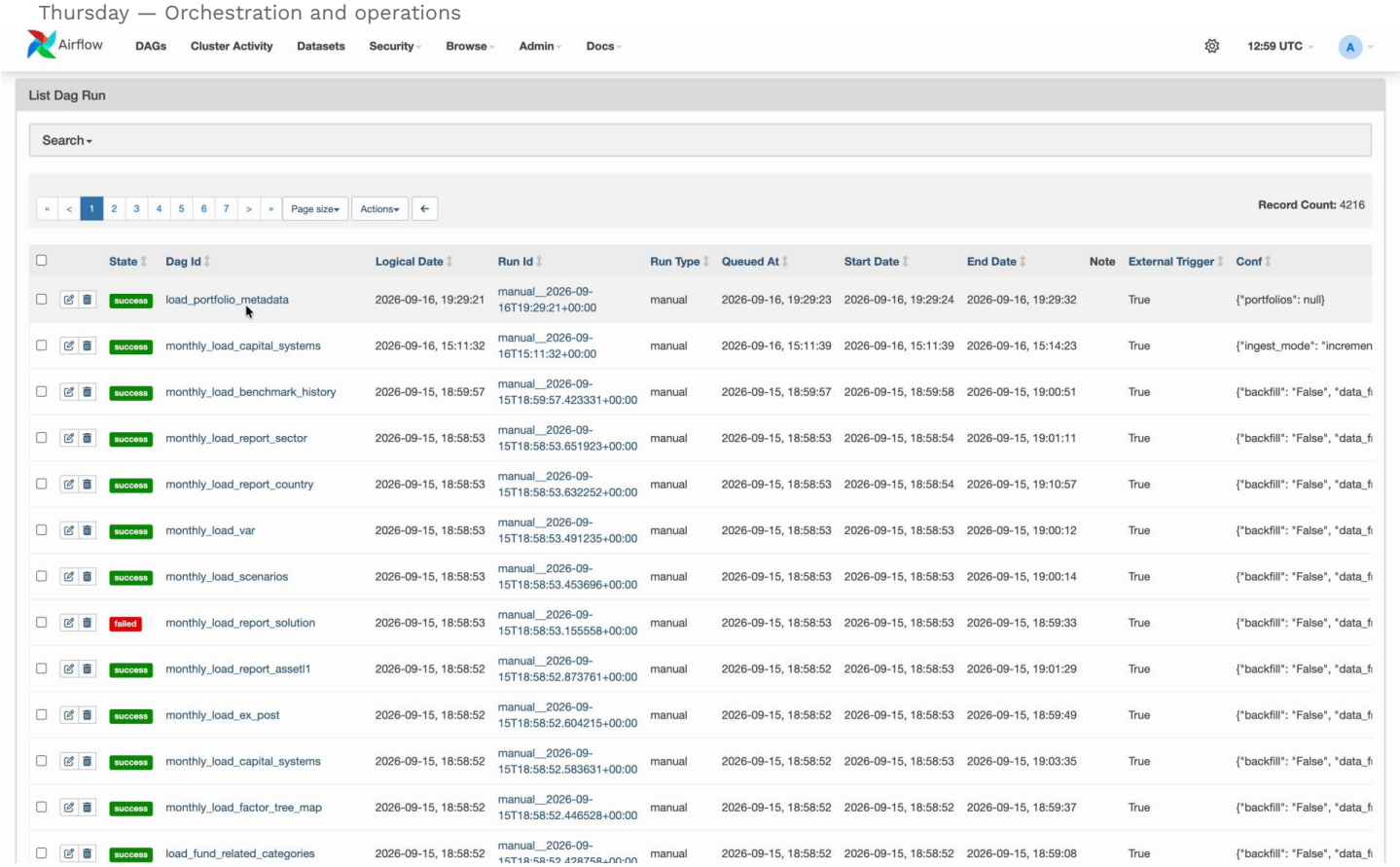


Figure 78: Browsing DAG runs

On screen: DAGs that trigger DAGs

What this sequence controls

A TriggerDagRunOperator requests a run of another DAG. The waiting option determines whether that task also waits for the child run's completion; the default is not to wait. Thus the runner's status must be read with its configuration, and each required child's delivery still needs evidence. The operator reference documents this completion boundary.

The runner starts report DAGs in parallel. The shown code sets `wait_for_completion=True`, so each trigger task waits for its child run. The default without that setting would only confirm the trigger. Inspect each child's run and output; a completed child can still produce the wrong data.

If many reports share one source, a runner like this is a tidy way to start them together. You still have to decide what fresh means for each report and how you'd rerun just one of them.

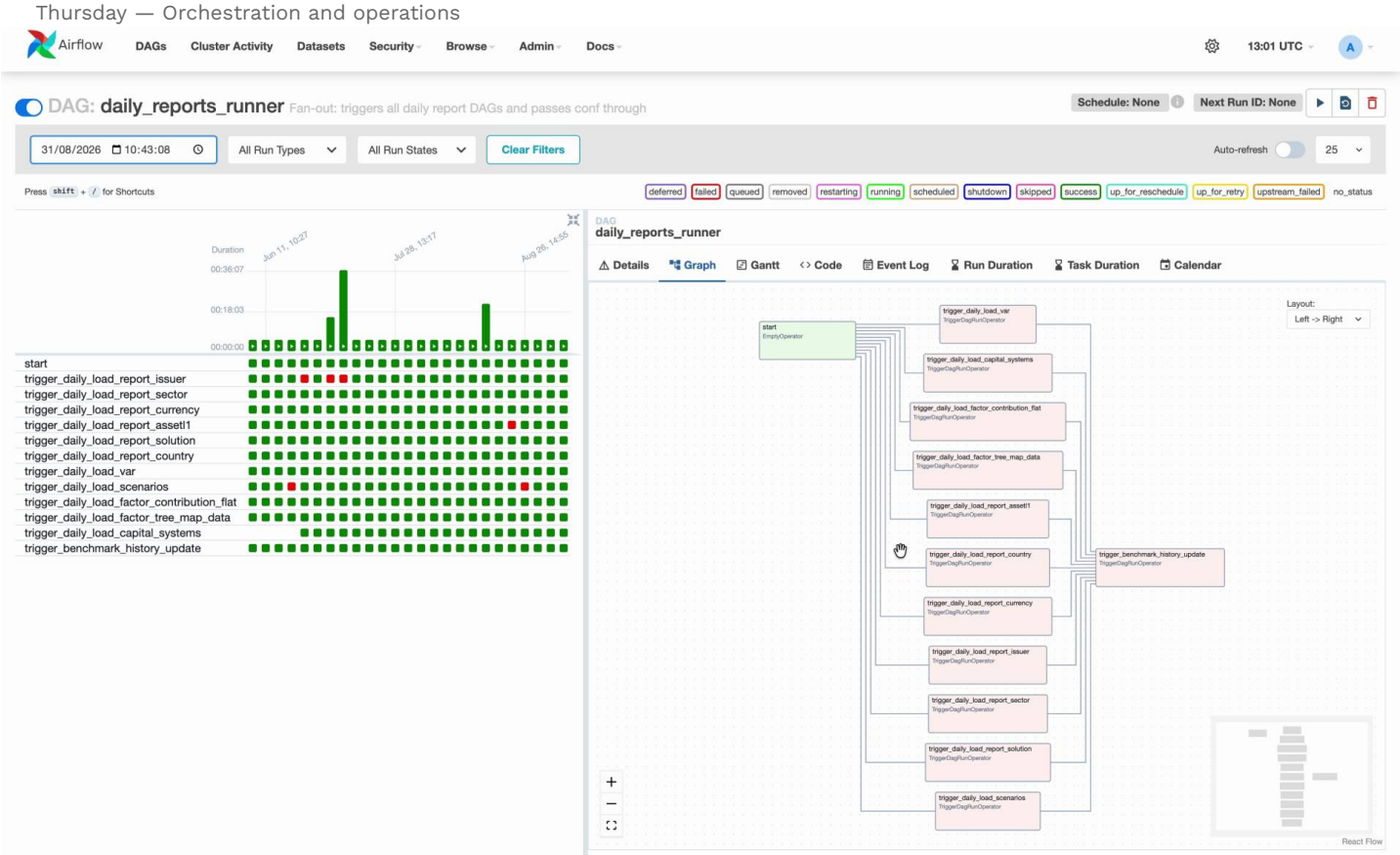


Figure 79: Fan-out runner

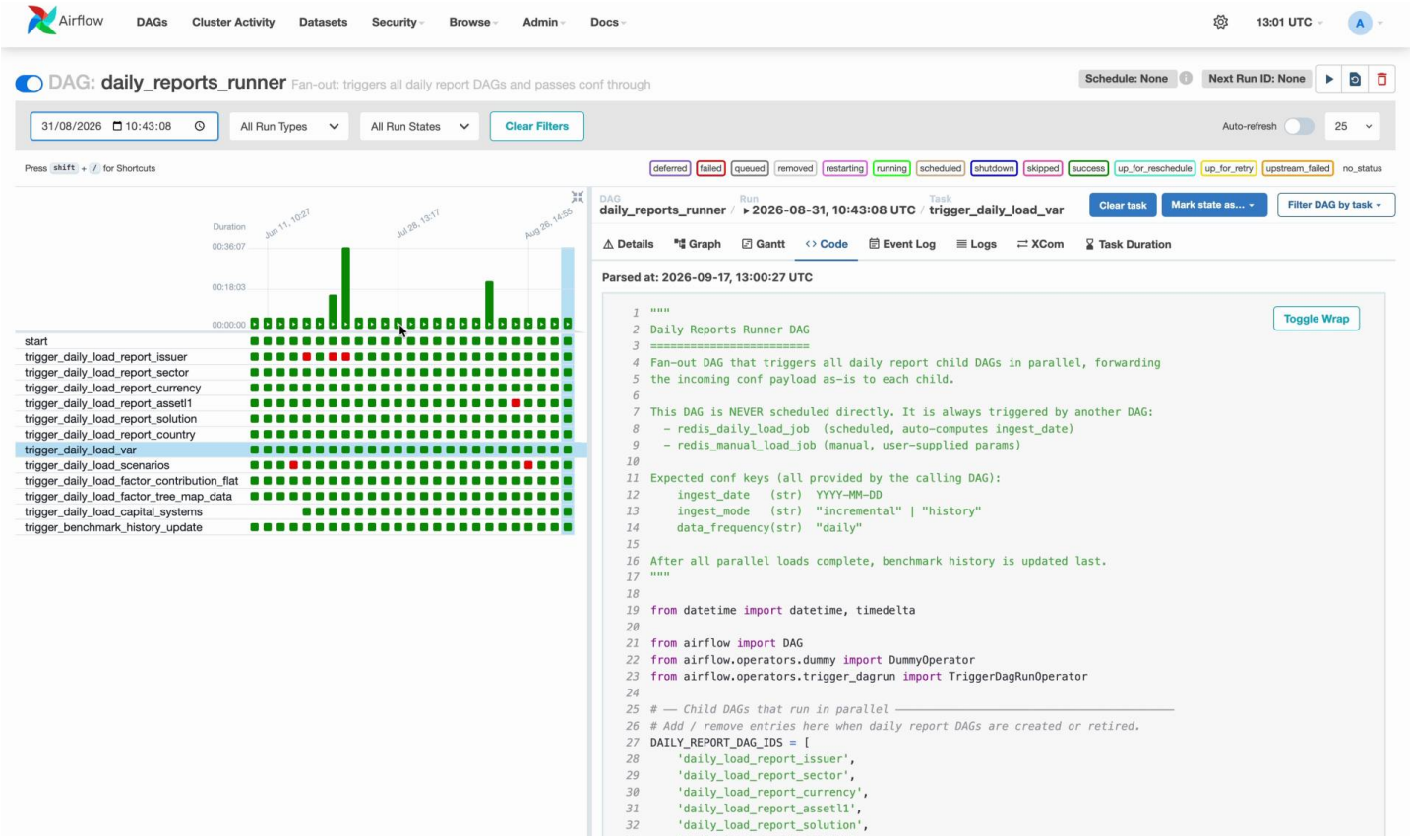


Figure 80: Runner: purpose

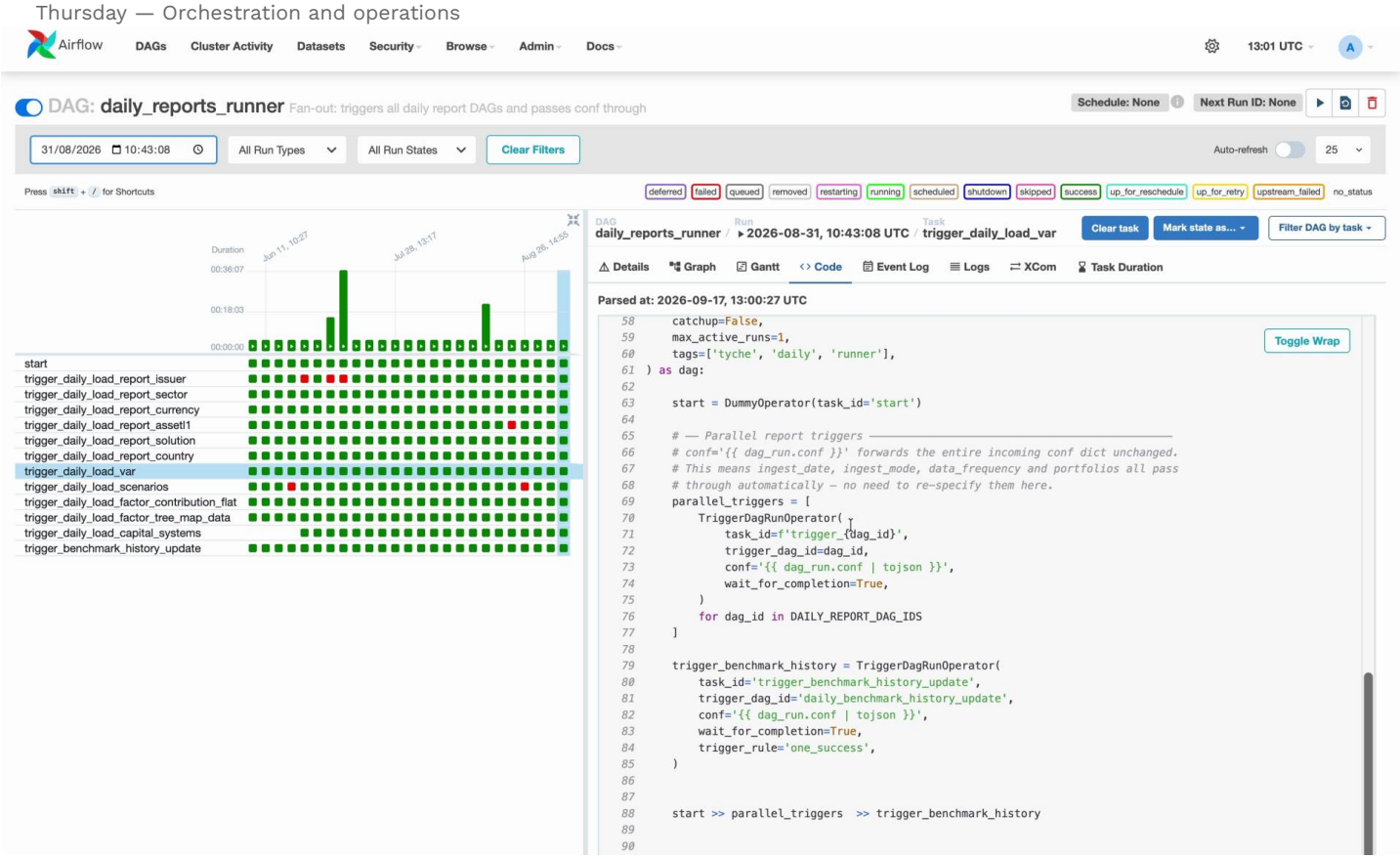


Figure 81: Runner: parallel triggers

On screen: Astronomer

What this sequence controls

Astro is Astronomer's managed Airflow platform. A Deployment is a hosted Airflow instance with its own released project and operational configuration. The platform deployment view and the Airflow task/run view answer different questions: which code/runtime is deployed, and what one scheduled execution did. Astronomer describes Deployments and the local CLI route.

For team-owned hosted deployments, use the optional Astro operations and runtime ownership reference. It distinguishes hosted secrets/configuration authority, maintenance, and capacity decisions from the classroom worker setup.

The local v2 training runtime is not a claim of hosted Astro access. In the supplied images, compare the selected deployment, DAG code and run before interpreting either a successful task or a browser authorisation error. Retain the UI-access failure separately from execution/output evidence.

Astro's home page shows runs across all your deployments, and the Airflow UI inside it shows one DAG's runs and tasks. When you click between them you can land in a different environment, so note which deployment and which run you're looking at when you switch views. The DAG code view links the graph back to the code it came from.

The DAG run graph shows an authorisation error next to a successful task. Both are true. The task ran, and the browser couldn't refresh the page. So when you report a run, record the task result and any UI error separately, and check the output itself.

Figure 82: Astro home

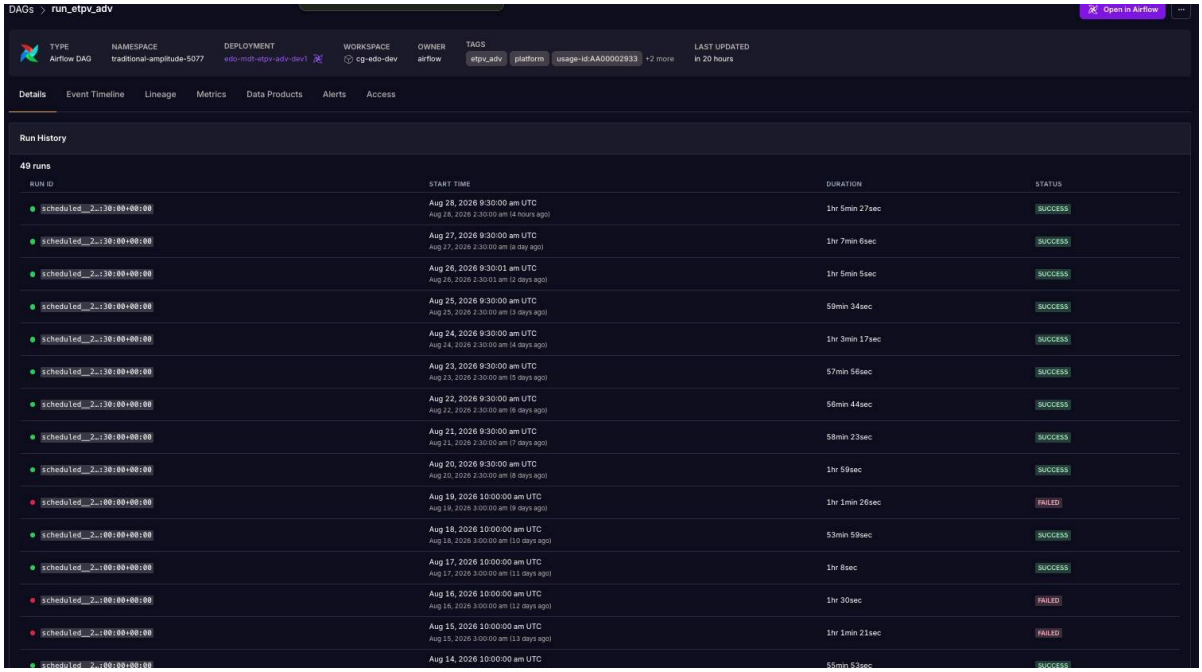


Figure 83: Deployment run history

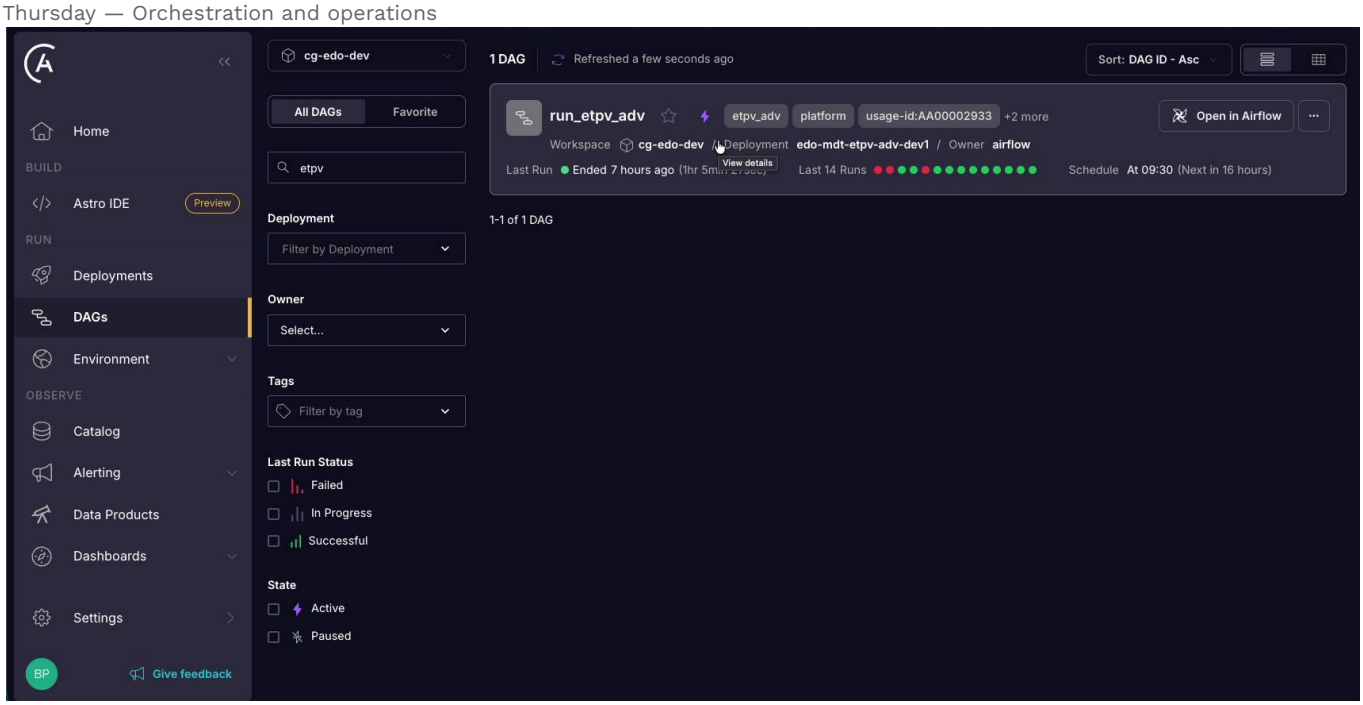


Figure 84: Filtering DAGs

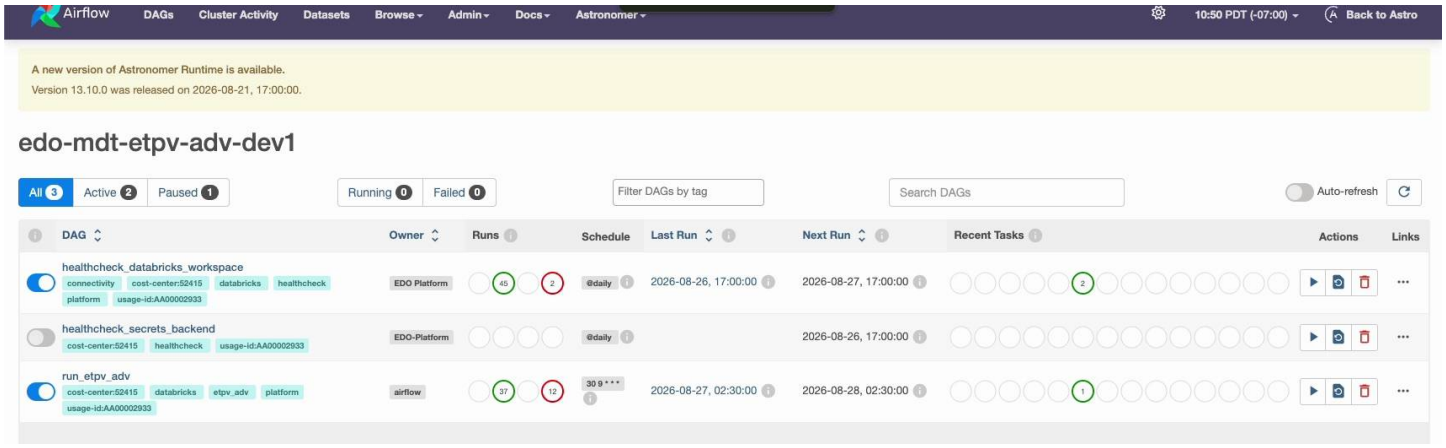


Figure 85: Airflow UI on Astro

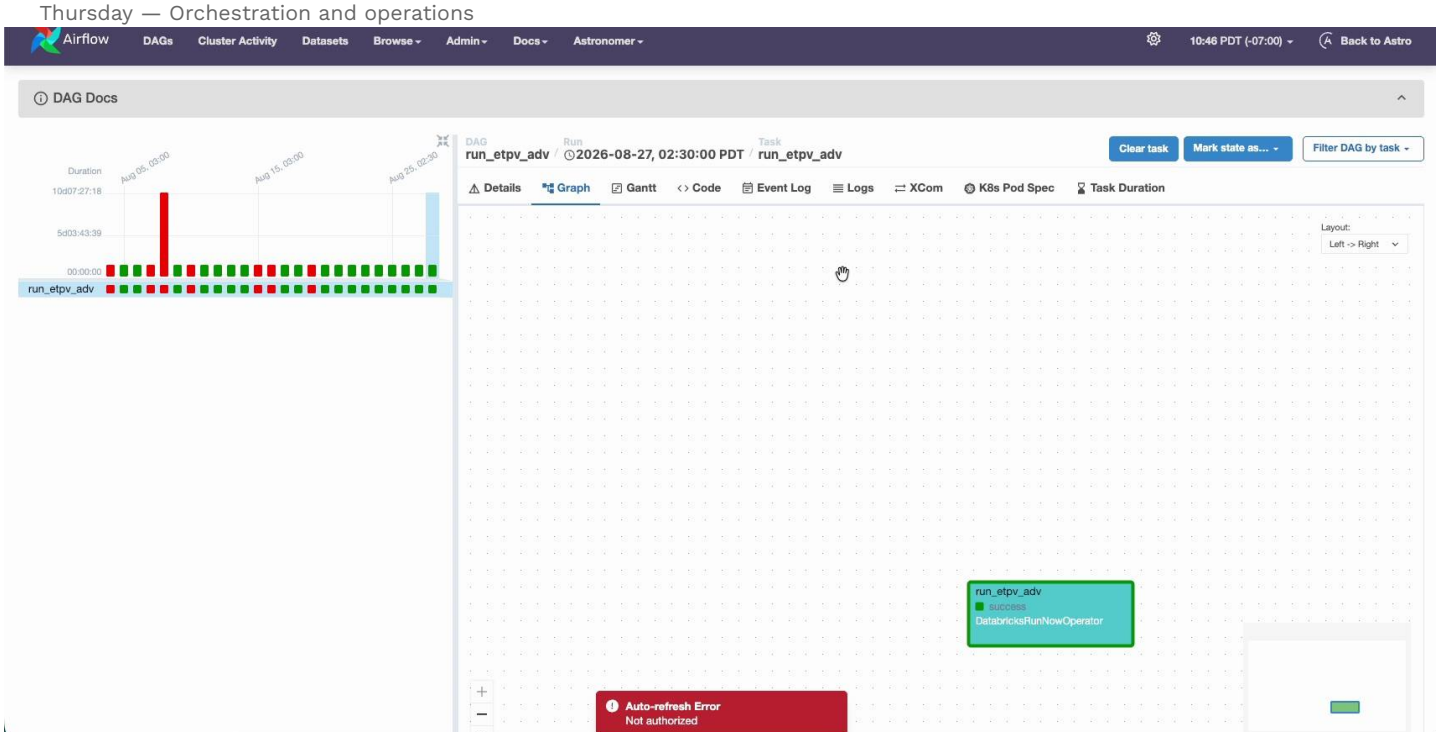


Figure 86: DAG run graph

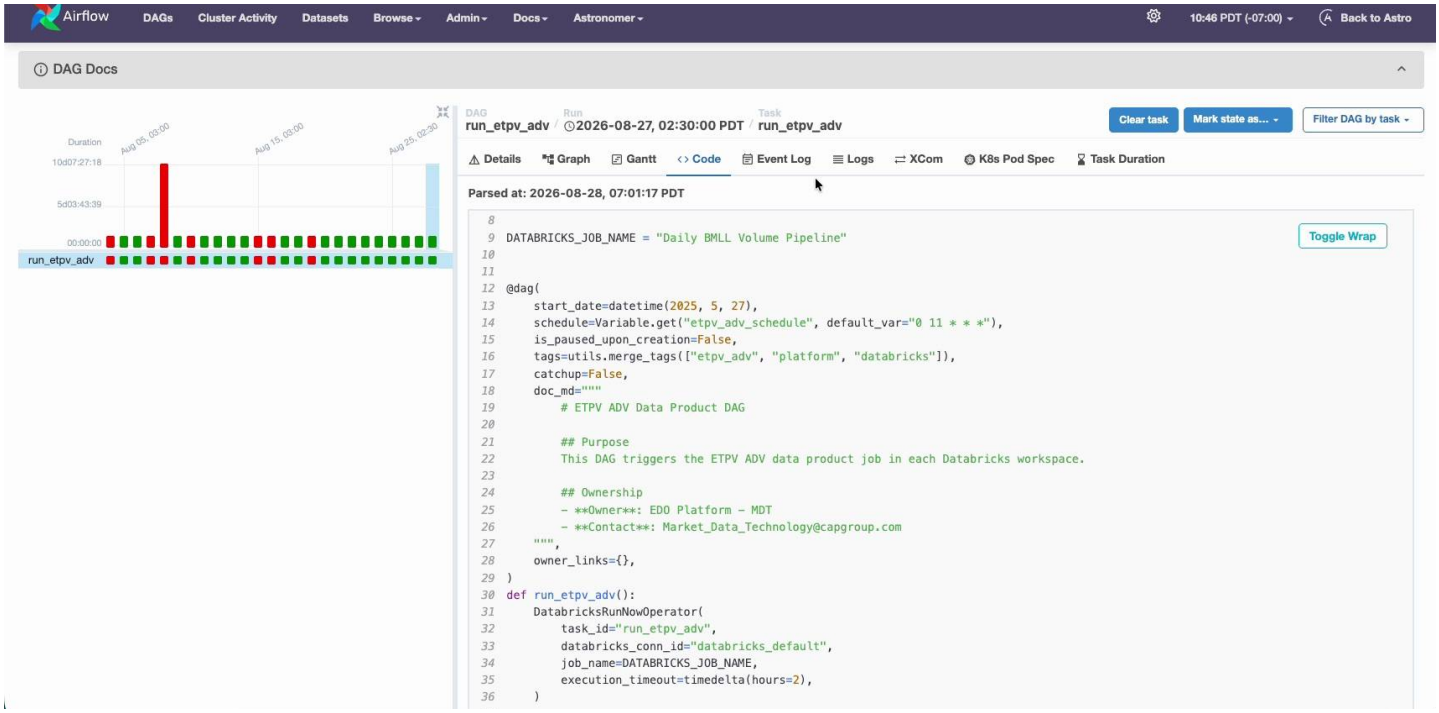


Figure 87: DAG code

On screen: Signing in to AWS

What this sequence controls

AWS sign-in authenticates the identity; an assumed IAM role supplies the permissions for a role session. A federated role can use temporary credentials rather than a personal long-lived key. AWS explains roles and their temporary sessions.

The supplied SAML chooser is an observed access route, not a promise about the current tenant sign-in screen. Check account, role, region and environment before interpreting a console result. Creating accounts, roles, resources or credentials belongs to prepared platform access; sign-in success alone does not prove permission to run a product.

Before you see the AWS console, you pick an account and a role. Pick carefully, because the same service can exist in dev, QA and prod, and your role decides what you're allowed to do in each. Reaching the console home only tells you you're signed in. If you can't get in at all, that's one for platform support.

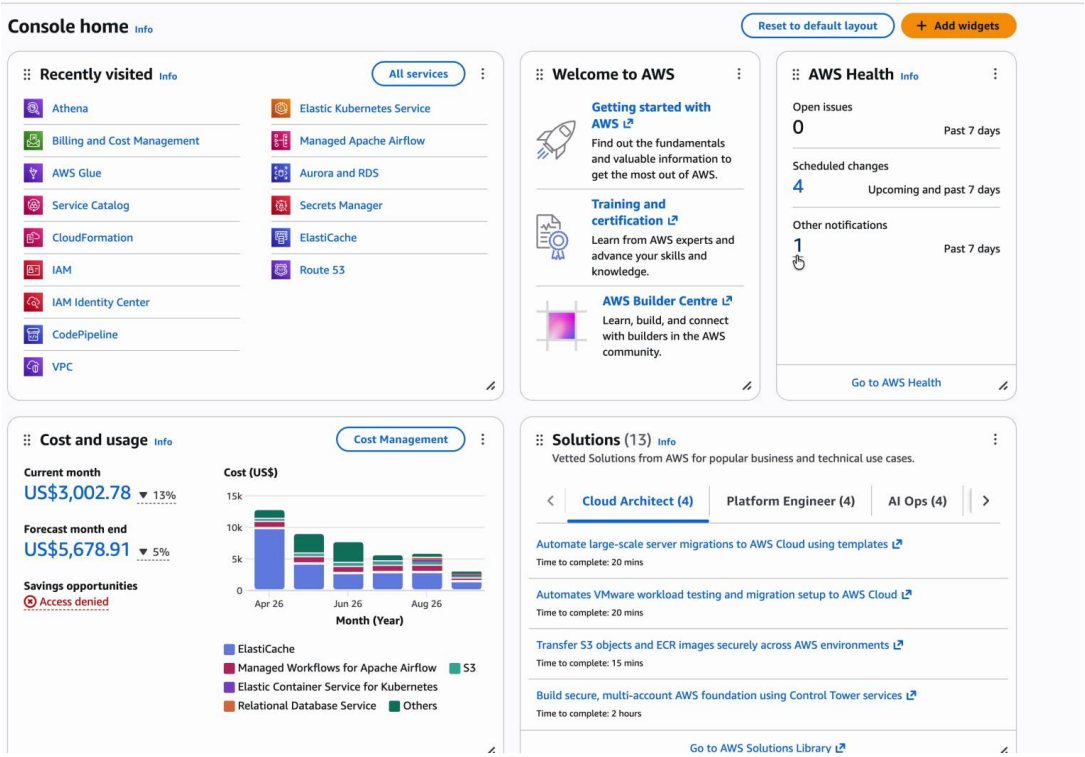


Figure 88: AWS Console home

On screen: Projects & pipelines

What this sequence controls

Harness Continuous Delivery automates a release pipeline composed of stages and steps. A project groups the release resources, while an execution records a particular attempt using selected inputs. The pipeline is the definition; the execution supplies observed release evidence. Harness explains these core objects.

Harness groups release pipelines into projects, usually one per team or application, and shows the recent runs for each pipeline. Each run is called an execution. A green row only tells you something once you know which pipeline, which commit, which environment and which inputs it used. One product can have several pipelines, one for each thing it deploys.

Keep the Harness release that deployed the code apart from the nightly Airflow run that uses it. If a product fails the morning after a release, look at the release first. Then check the commit against the actual error before you blame it.

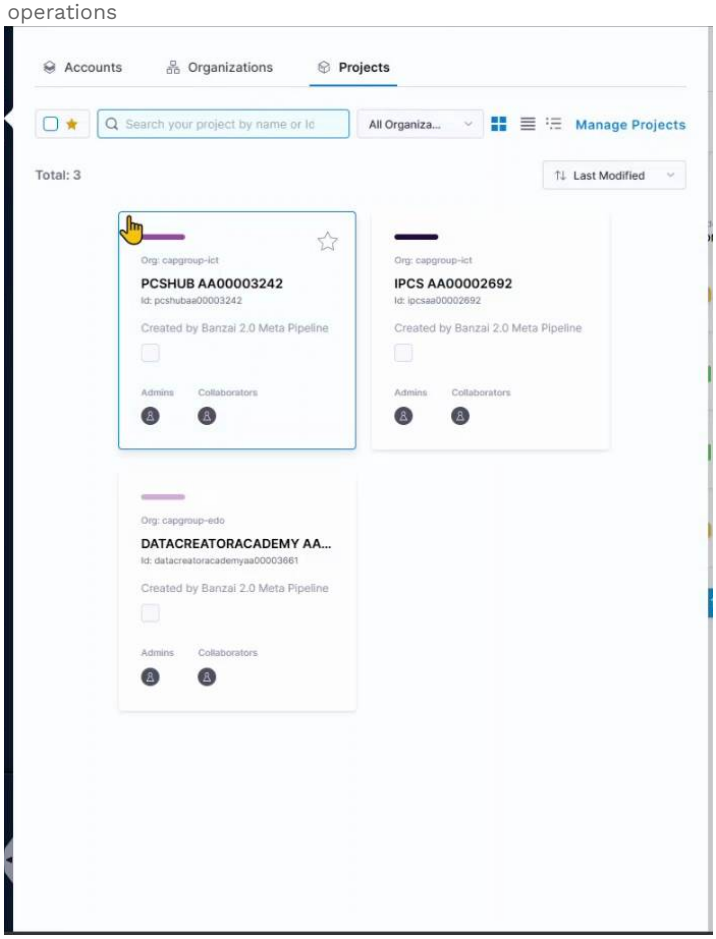


Figure 89: Harness projects

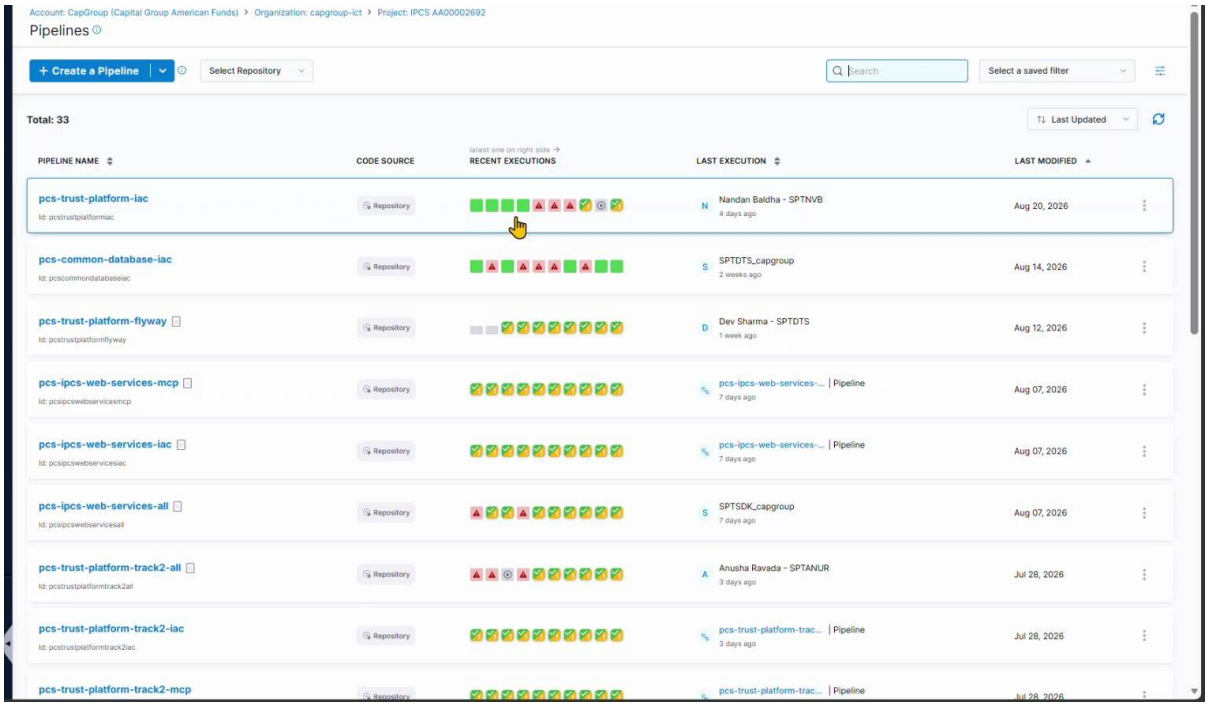


Figure 90: Pipelines list

On screen: Importing a pipeline from Git

What this sequence controls

Pipeline-as-code puts the release definition under the same review/versioning discipline as application changes. Importing it through a Git connector selects a repository, branch and YAML path; it does not by itself approve that definition or prove a deployment. Identify the configuration revision separately from the application commit the pipeline will release.

Select the Git connector, repository, branch and YAML path. When diagnosing a deployment, identify the configuration revision as well as the application commit.

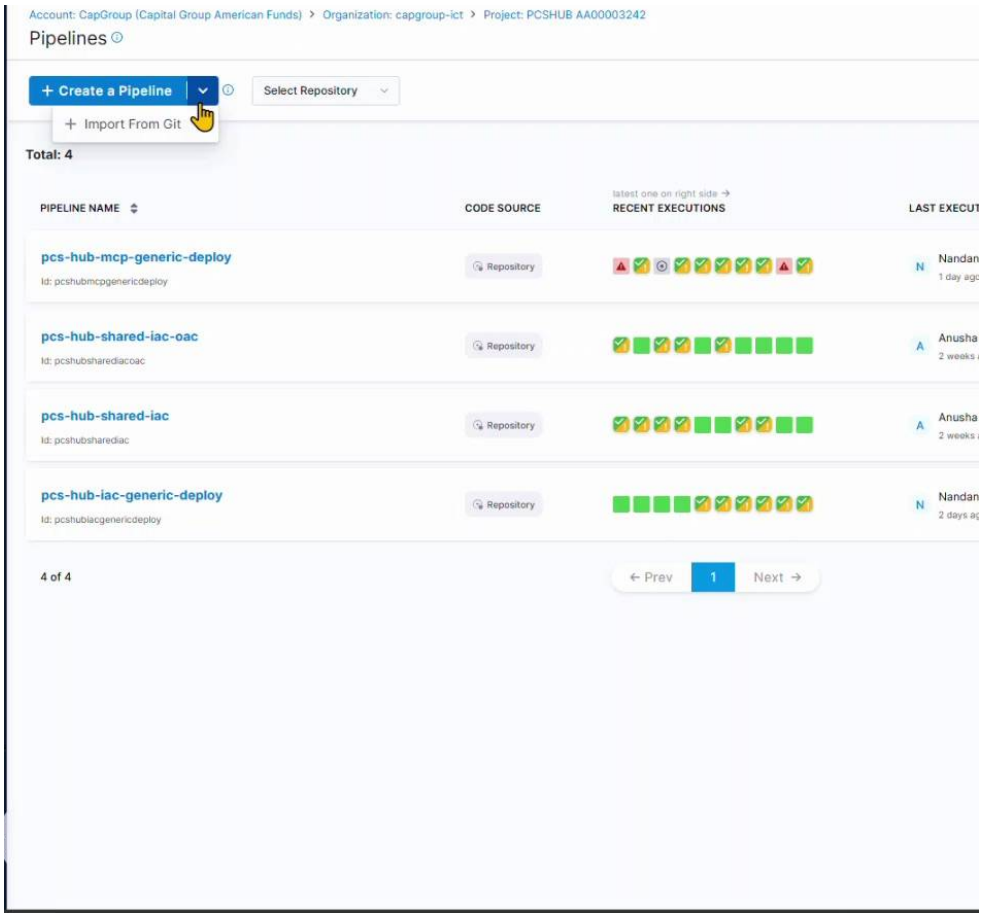


Figure 91: Pipeline list and Create pipeline entry point

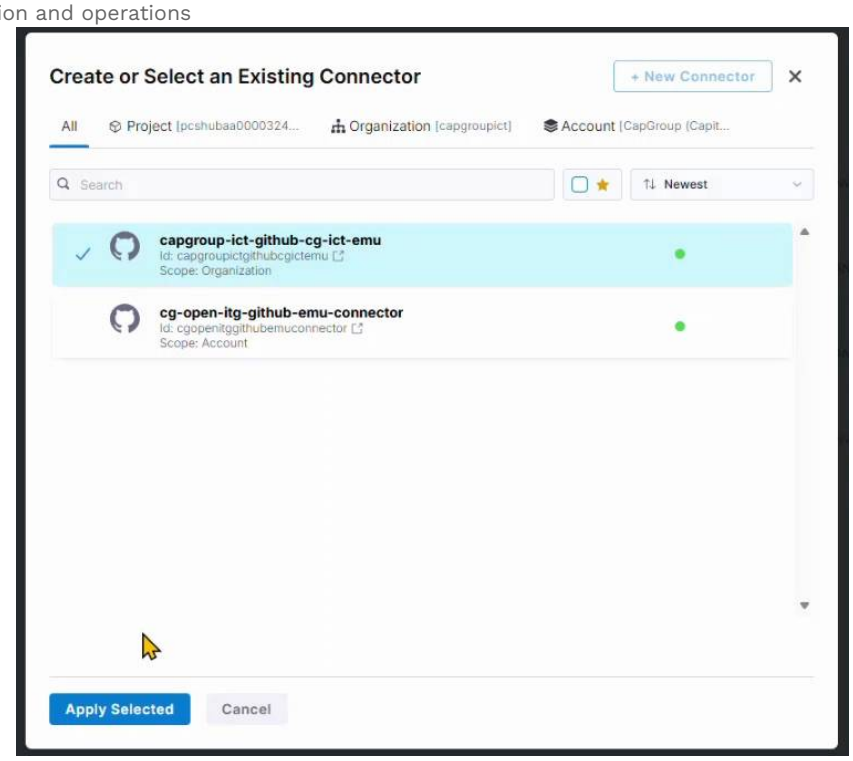


Figure 92: Selecting the Git connector

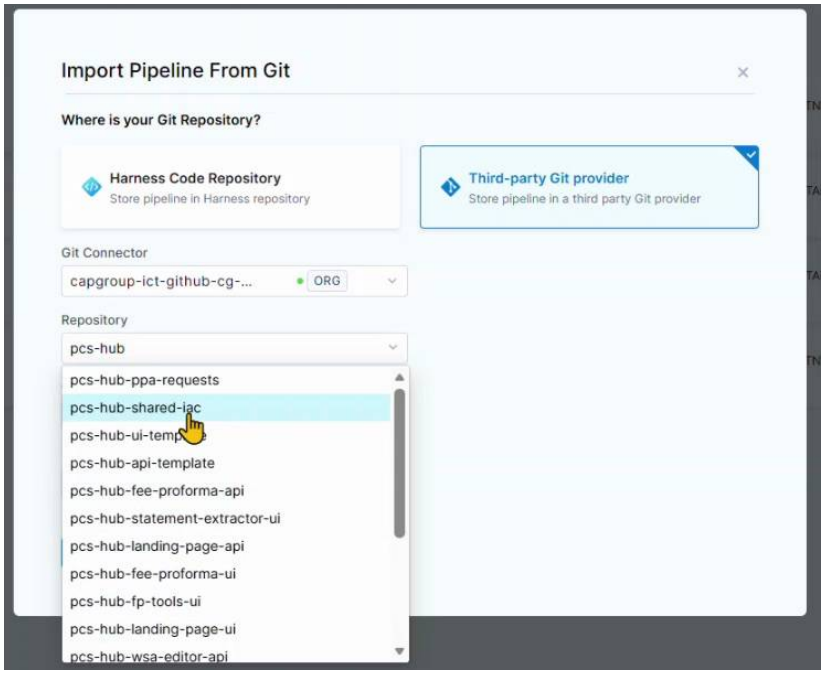


Figure 93: Selecting the repository

On screen: Pipelines as code

What this sequence controls

A release definition names services, environments and ordered operations, with inputs and conditions determining the chosen path. Read the YAML to understand the intended control boundary, then inspect one execution to establish which version, values and stages were used. A stage present in the definition can be skipped in an execution.

Compare the YAML with the stage graph in Harness. Both show the definition. The execution history shows what happened. Before you rely on a stage result, ask which version and inputs it ran with.

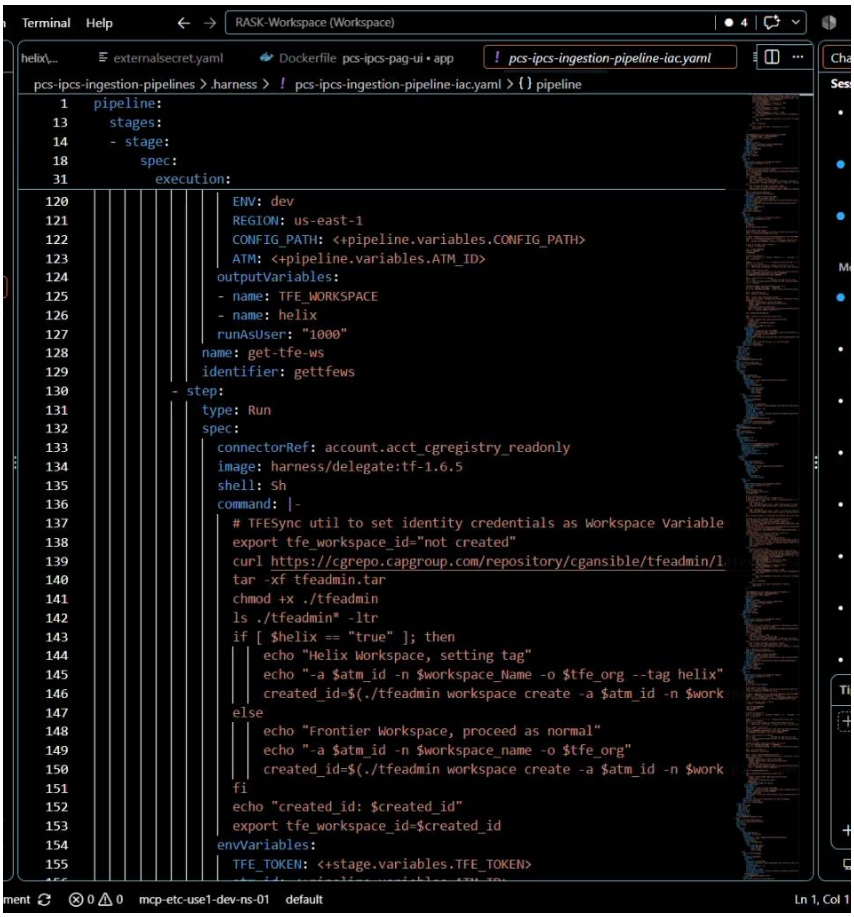


Figure 94: Pipeline YAML in VS Code

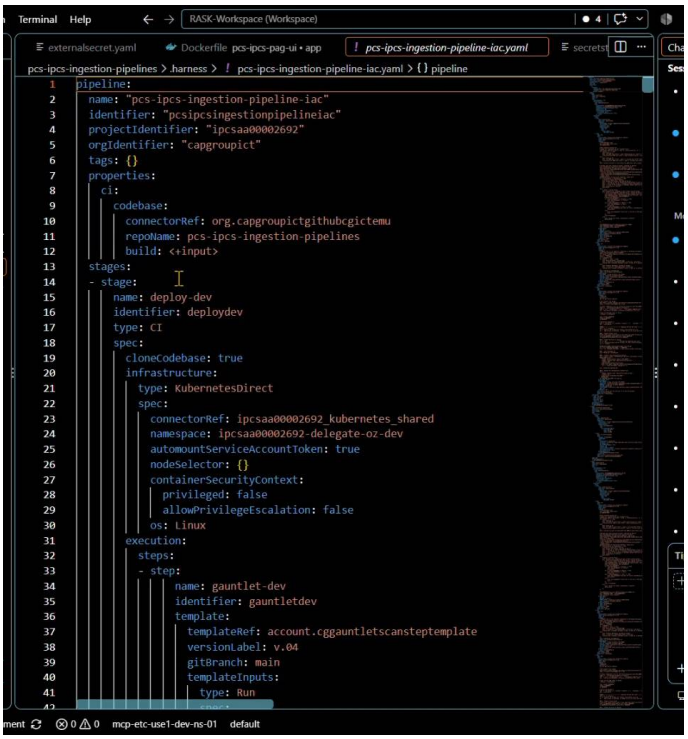


Figure 95: Deploy stage in YAML

On screen: Stages & environment promotion

What this sequence controls

Promotion applies a reviewed change to a target environment; an approval stage records an authorised decision to continue. Automated test evidence and human approval answer different questions. The supplied graphs illustrate product-specific stage ordering and gates, so identify the selected target and any conditional or waiting stage instead of treating the images as one universal CG pipeline.

Read conditional stages and waiting approvals in the selected execution. A stage declared in YAML may be skipped, and a passed test does not replace approval.

These screens come from different products, and they name their environments differently, so don't read them as one standard pipeline. As a creator you hand over the reviewed code, a build anyone can repeat and a note on how the change keeps the contract. The platform team owns the promotion itself. The last screen shows a run paused at an approval step, waiting for that person.

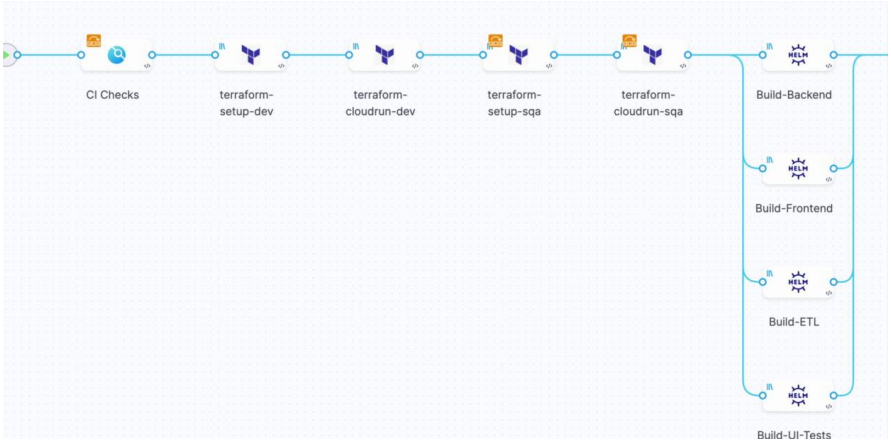


Figure 96: CI, Terraform and build

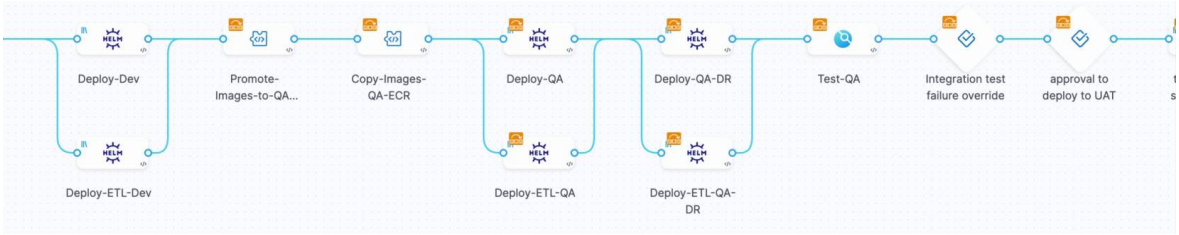


Figure 97: Tests and UAT approval

Figure 98: Detail of Tests and UAT approval

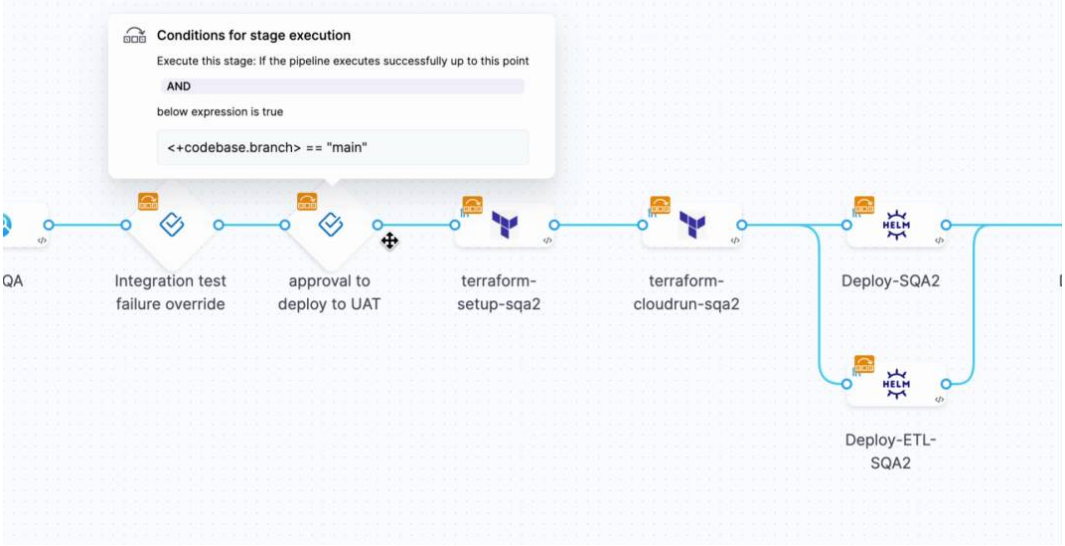


Figure 99: Conditional stages

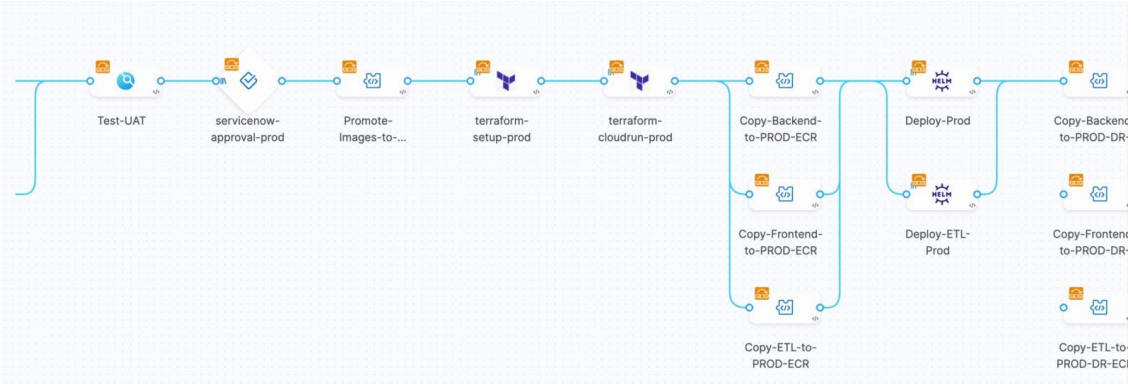


Figure 100: Production stages

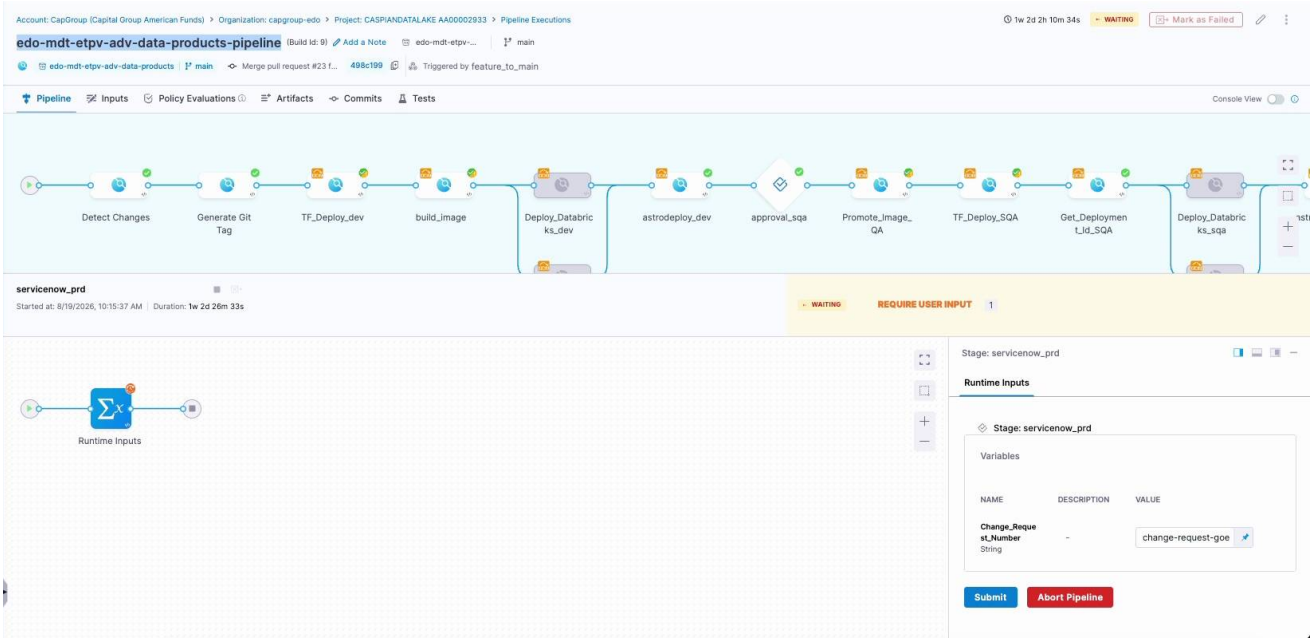


Figure 101: Approval in a data products pipeline

On screen: Running pipelines

What this sequence controls

Running a pipeline binds its definition to input values such as repository branch, application ID and target environment. An input set stores a reusable set of those values; it is not evidence that the selected values are appropriate for this change. Read the chosen inputs and the resulting execution record. Learners review these supplied controls rather than start a production release.

Input sets are how you tell Harness what kind of deployment you want. Only to dev, only to QA, straight to prod, or the whole lot. They're saved, so you click into the same template each time and the run behaves the same way. The branch you pick and the variables matter as much as the pipeline itself. To repeat a run, you need the version of the code and the inputs it used, and the execution inputs screen shows you both. Triggers start runs for you, on a pull request, a merge or a schedule.

Do the same with your dbt and Airflow work. Write down the project, profile, catalog, schema and run parameters you actually used, and leave the credentials out.

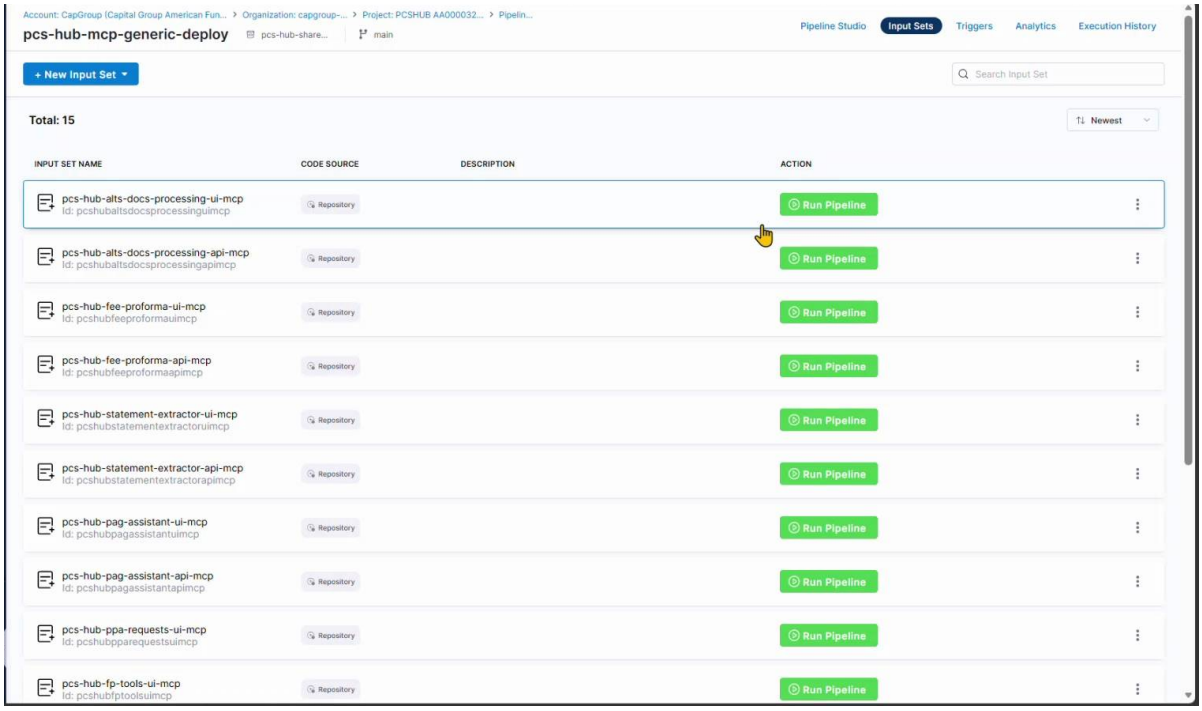


Figure 102: Input sets

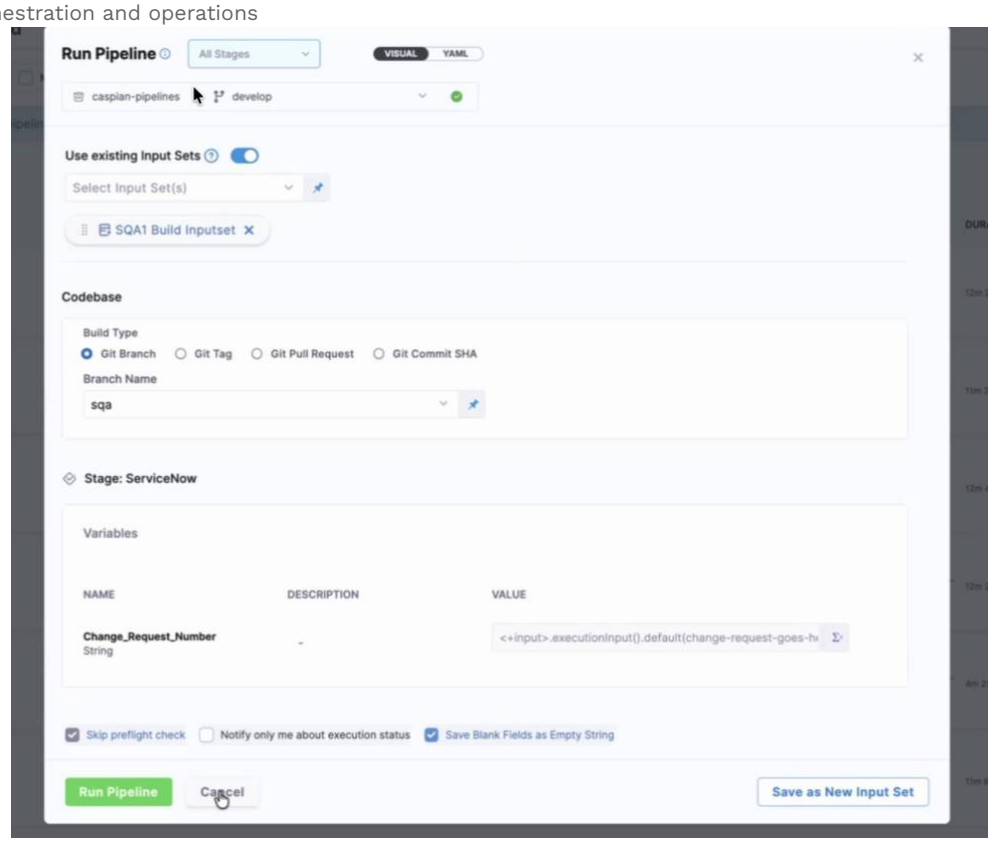


Figure 103: Choosing the branch

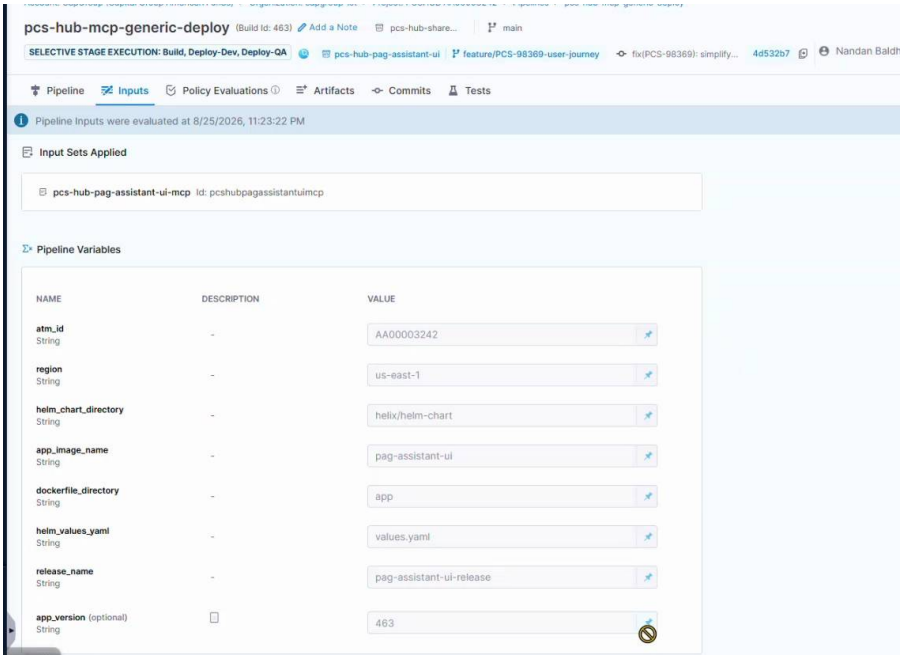


Figure 104: Execution inputs & variables

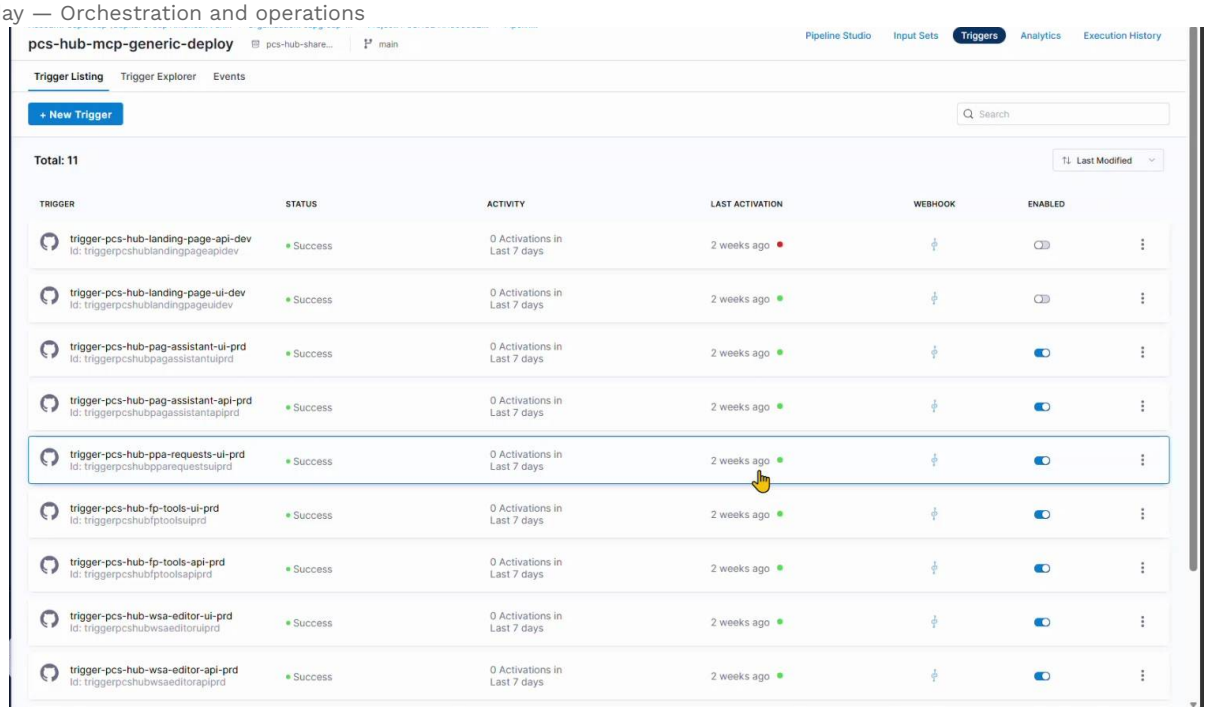


Figure 105: Triggers

On screen: Executions, logs and troubleshooting

What this sequence controls

A release execution is a chain of observed step results, not one undifferentiated green/red signal. Establish the selected commit, inputs and environment, locate the first failed or blocked stage, then read that step's log. Retrying or approving another stage does not correct an unmet gate. Link a deployment incident to both its release execution and the later Airflow/data evidence.

The execution history lists every run with its status, trigger and duration. Click a failed run and the stage graph shows you which stage broke. Click the step and the log gives you the error. Then scroll up from the bottom. The last line often reports what happened as a result, and the first real error earlier in the same run is usually the cause.

When you pass it on, be specific. "The QA deployment step failed for this commit" gets to the right team much faster than "the data pipeline broke".

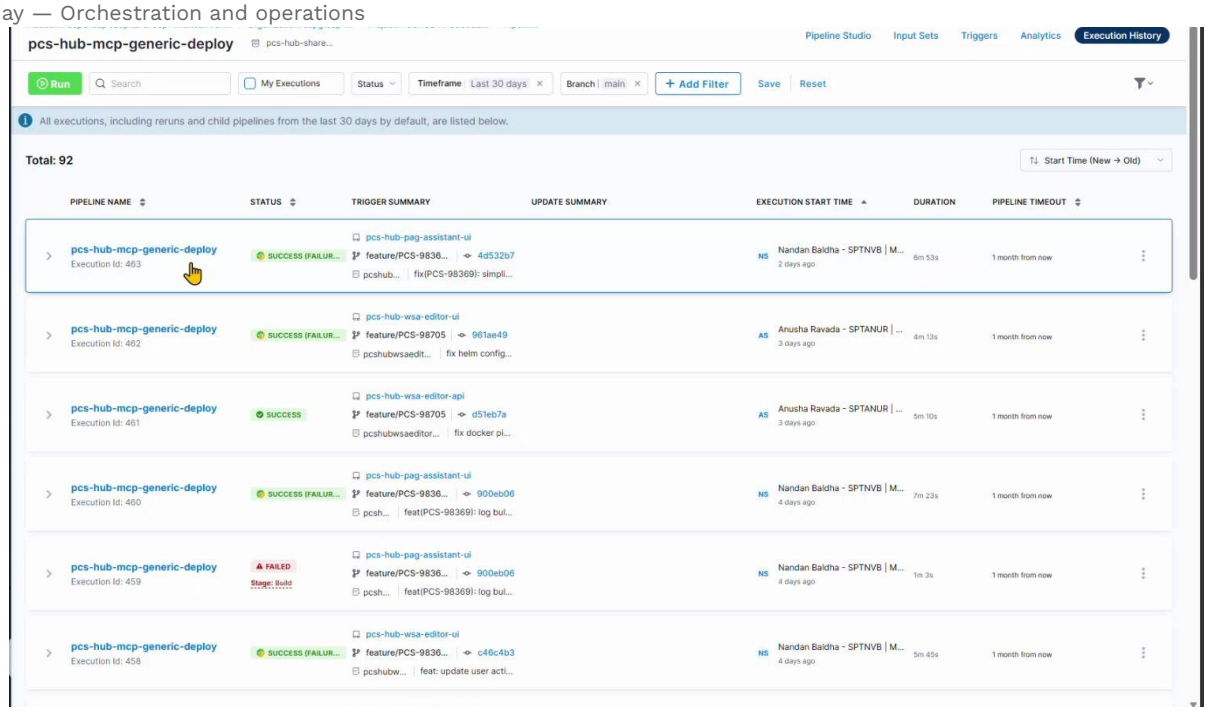


Figure 106: Execution history

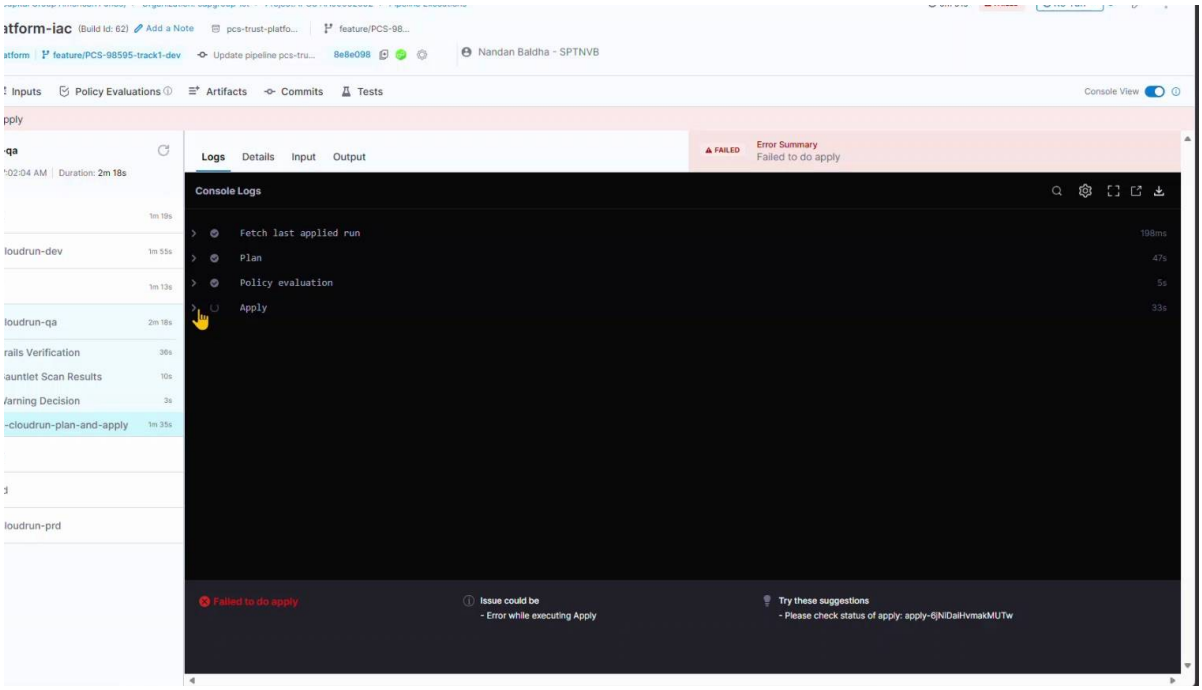


Figure 107: A failed stage

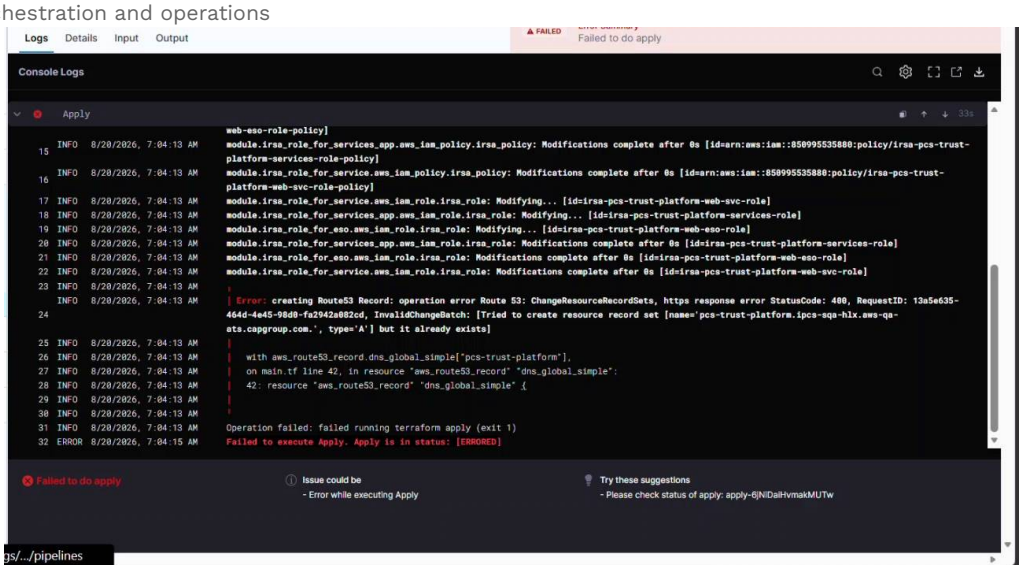


Figure 108: Reading the error

Figure 109: Build stage logs

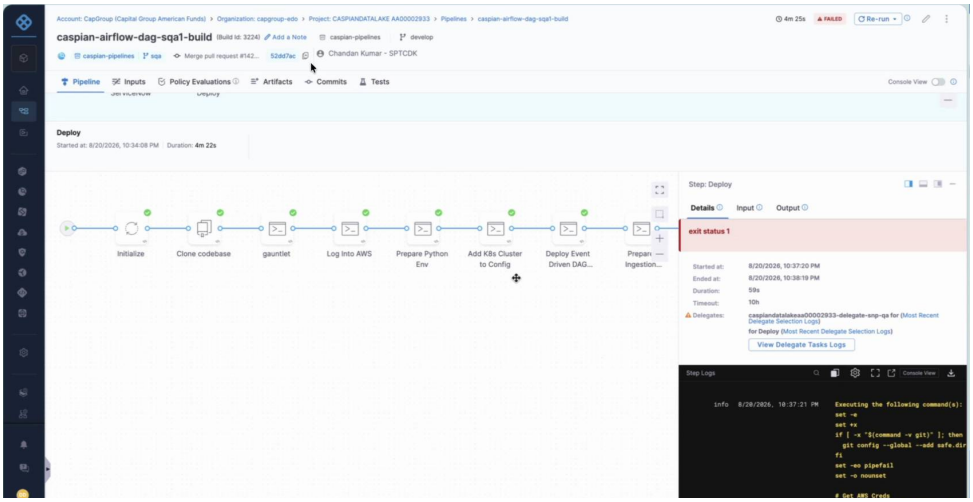


Figure 110: Deploy stage steps

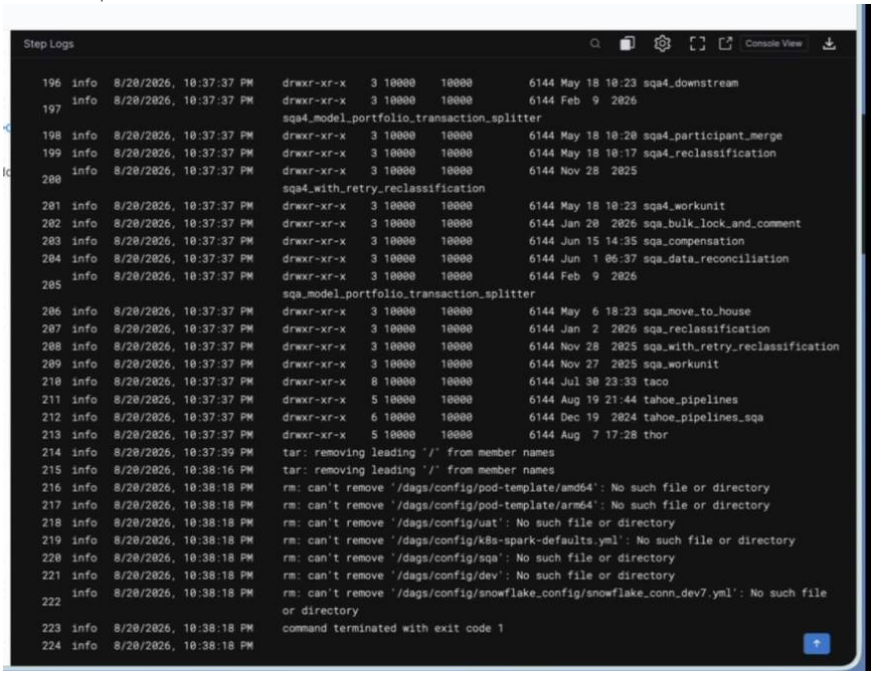


Figure 111: Step failure

On screen: Terraform in the repository

What this sequence controls

Terraform describes infrastructure resources in versioned configuration. Providers translate those declarations into operations against services; inputs parameterise the configuration, and state relates declared resource addresses to managed infrastructure. A plan previews intended changes and apply performs them. HashiCorp explains the configuration/plan/apply workflow.

Terraform controls infrastructure such as the demonstrated ECR registry. It does not express the holdings transformation or prove consumer data quality. Read the recovery/design requirement, resource definitions and environment inputs together; platform owners execute provisioning, while creators explain their product's dependency on those resources.

Terraform is a way to code your infrastructure. The `.tf` files basically say "go to AWS and give me an S3 bucket, give me a machine with this much memory". Here they sit in the repository next to the Helm charts and the documentation. The variables file declares each input with its type, and each environment gets its own `.tfvars` file with values like the ADM ID and namespace. The ECR module creates the container registry that built images are pushed to.

The design document in the same repository says what all this is for, which here is disaster recovery. Read it before you review the Terraform, because the code can be valid and still miss a recovery requirement.

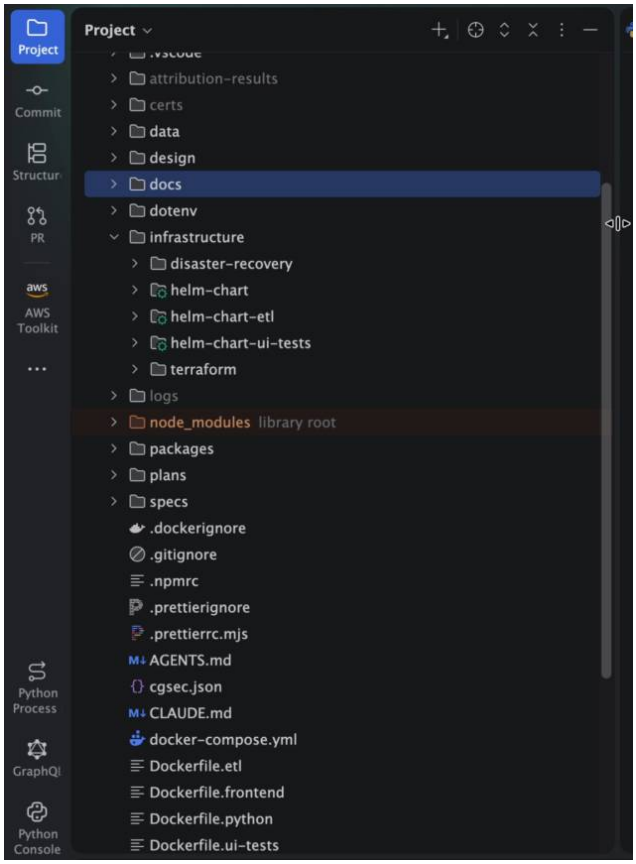


Figure 112: Repository structure

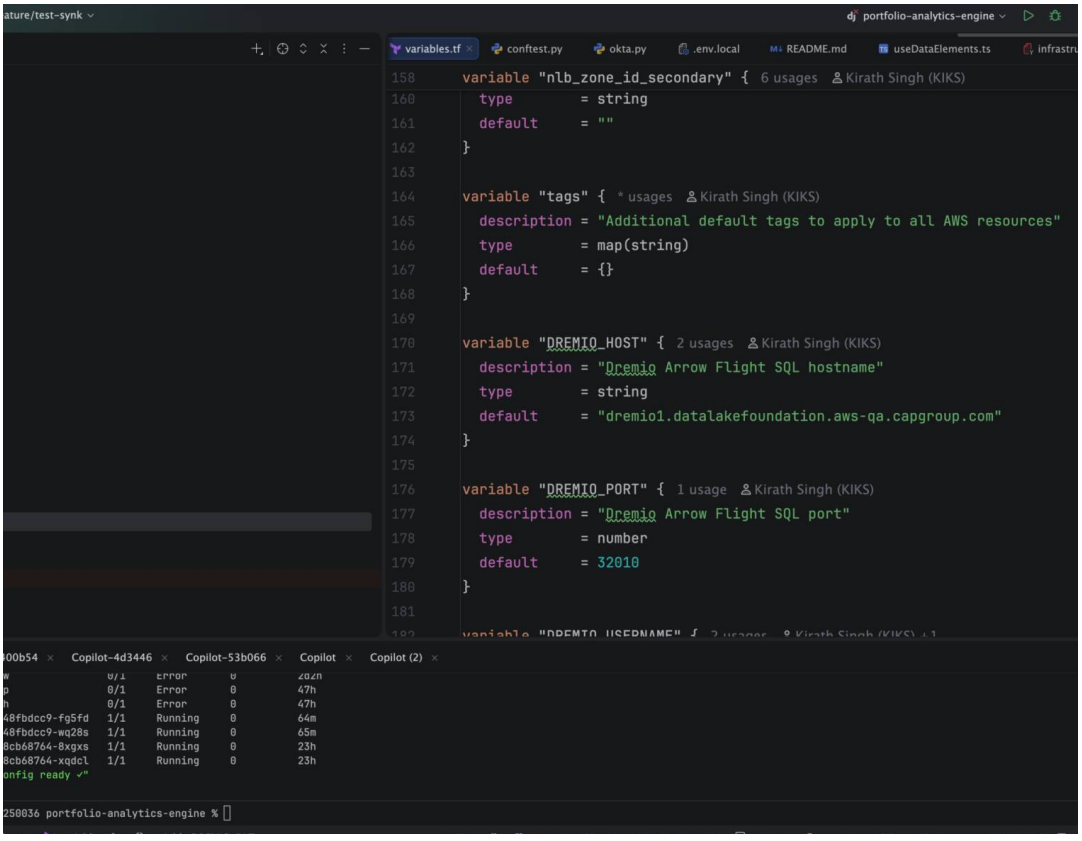


Figure 113: Terraform variables

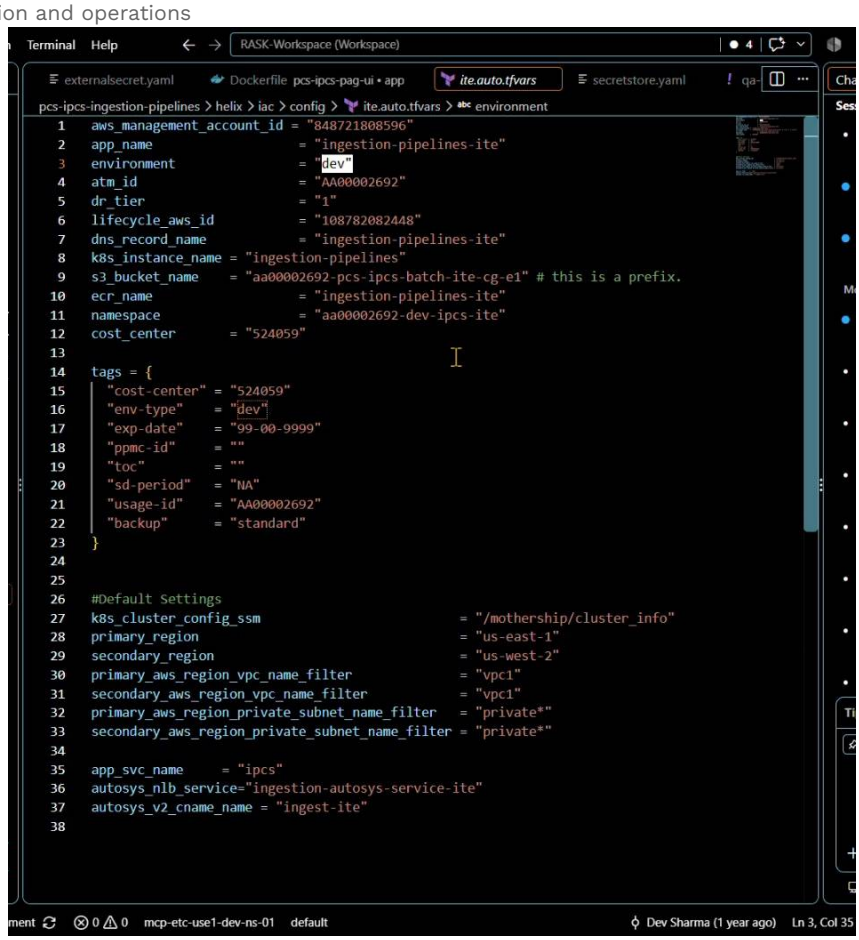


Figure 114: Environment tfvars

On screen: Terraform Cloud

What this sequence controls

The supplied screenshots use the name Terraform Cloud; current HashiCorp documentation calls the hosted product HCP Terraform. Its workspace binds configuration, variables, managed state and a history of runs. A run records the proposed plan and, when performed, the apply result. The official run guide shows commit, status, timeline and command output.

Treat state as Terraform's record used to manage resources, not a measurement of data freshness or a guarantee that application code works. Check workspace/environment identity before reading plan, apply and state history. The demonstrated role-based dynamic credentials are a platform-managed access mechanism; learners do not copy credentials or apply cloud changes.

Terraform Cloud has one workspace per component and environment. Each workspace shows the latest run, the linked repository and the resources it manages. A run has two stages. The plan works out what would change, and the apply makes the change. Successful infrastructure changes are recorded in state history, which is Terraform's record of the infrastructure that exists.

The dynamic-credentials screen shows Terraform borrowing short-lived AWS credentials through a role, so nobody has to store a static key. The platform team looks after the variables and credentials. If a support report mentions a failed plan or a stale state, it's an infrastructure problem. A failed job or a missing row in your table goes to whoever owns the product.

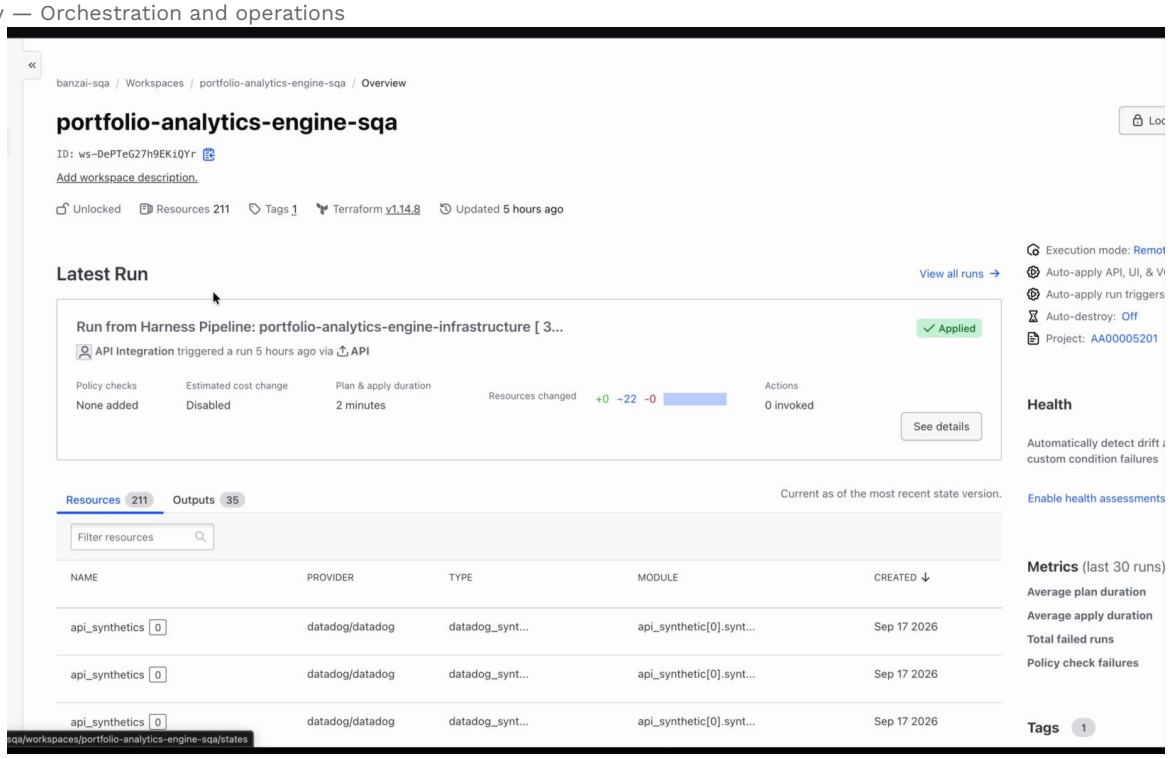


Figure 115: Workspace overview

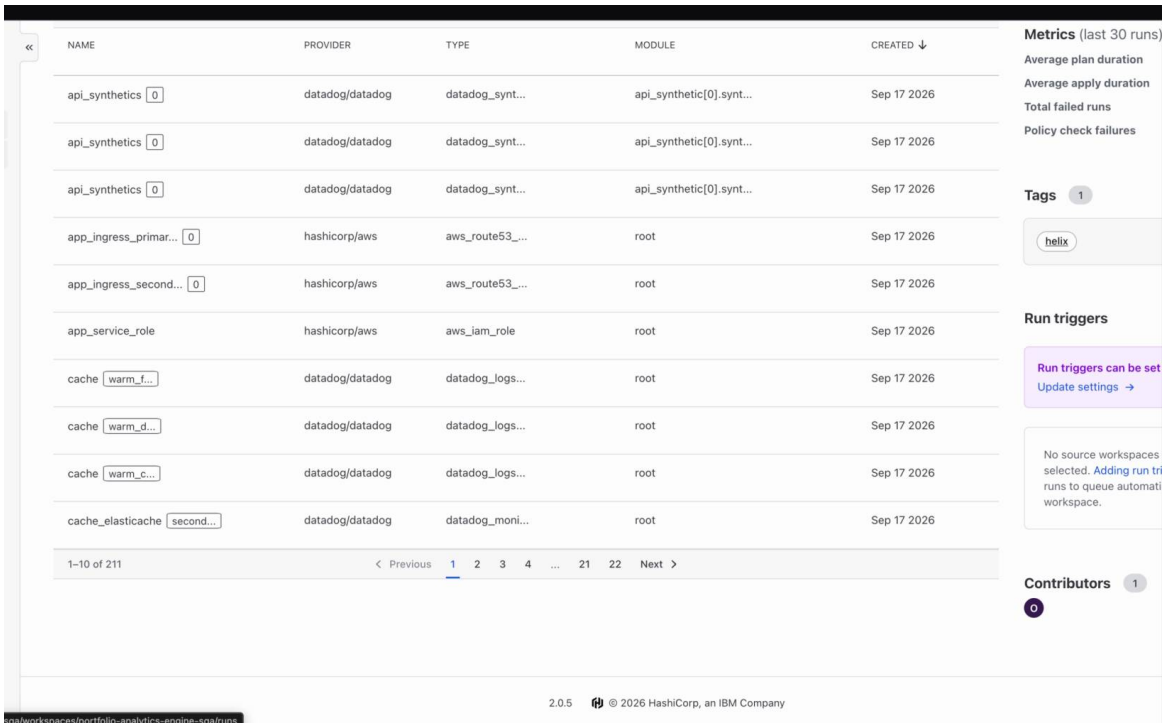


Figure 116: Managed resources

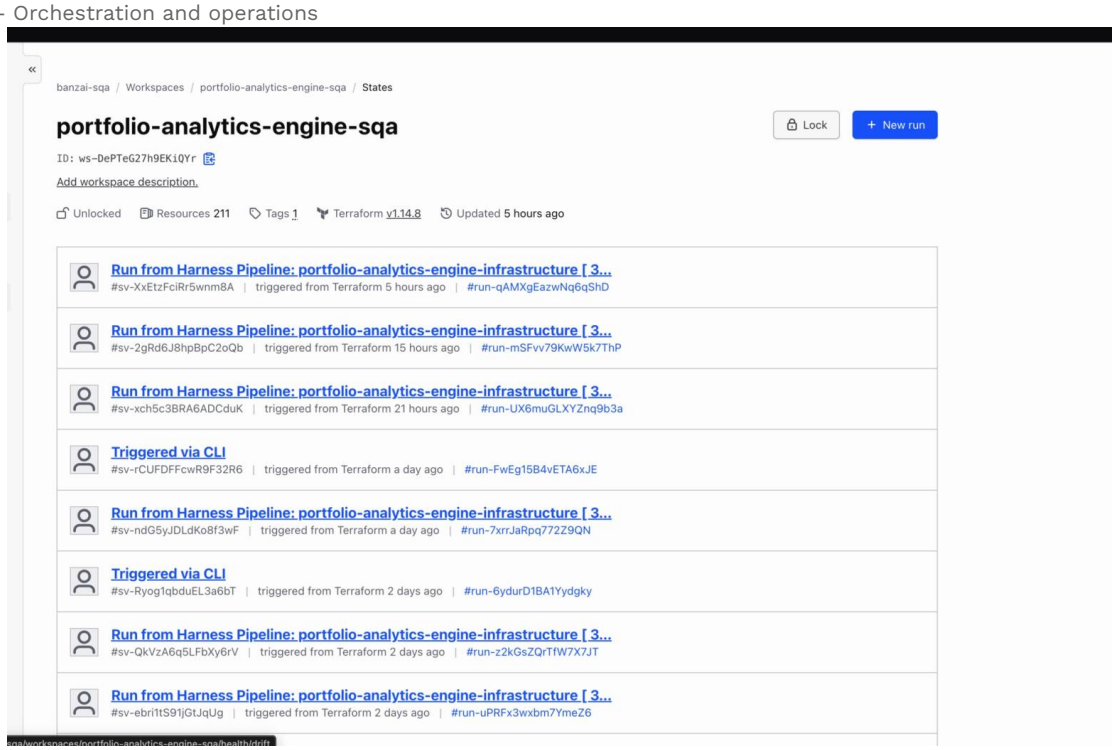


Figure 117: State versions

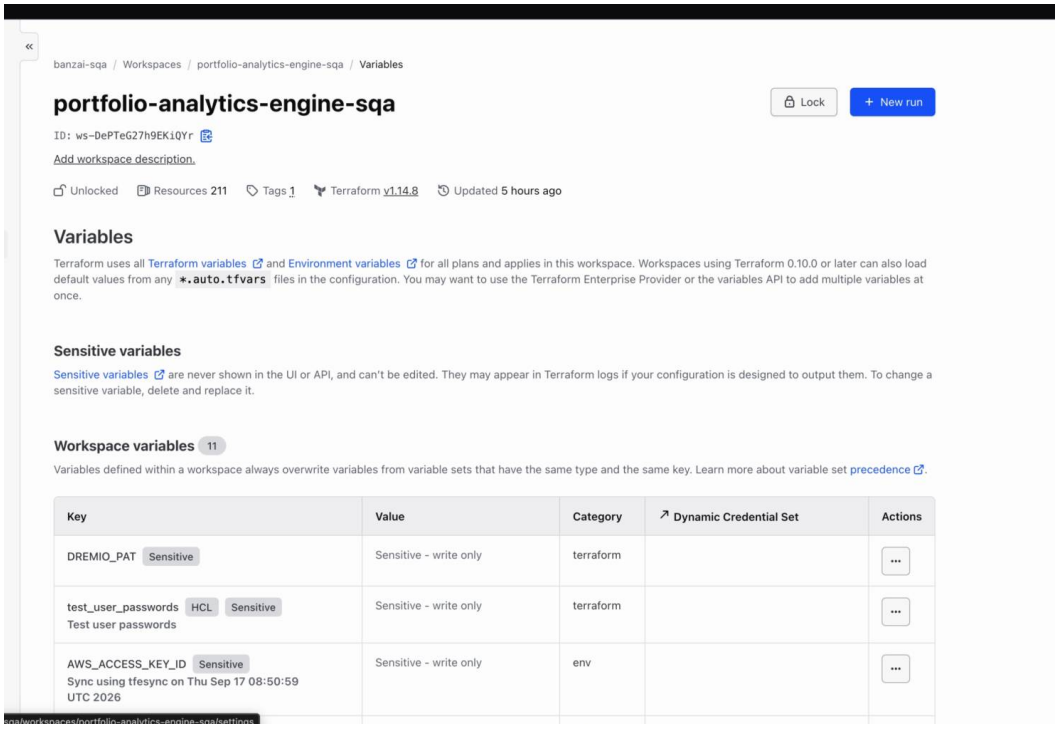


Figure 118: Workspace variables

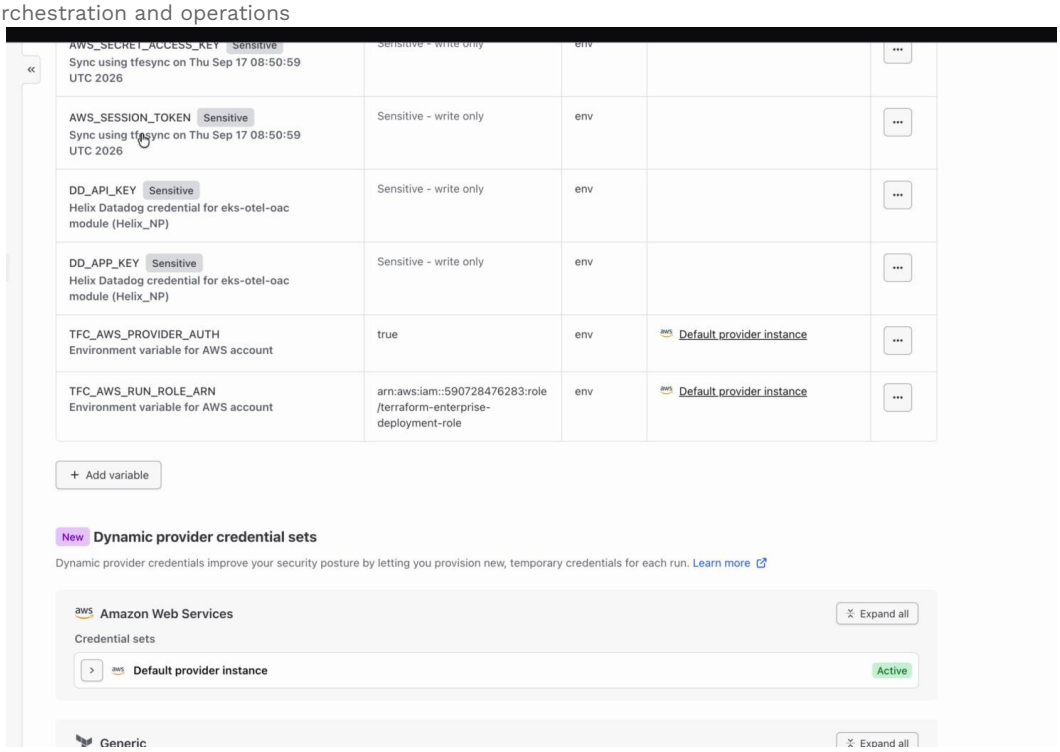


Figure 119: Dynamic credentials

On screen: Helm charts

What this sequence controls

A Helm chart packages Kubernetes resource templates with values. Rendering those templates produces resource definitions such as the demonstrated Deployment; the values select image/configuration/resource settings for an environment. Helm documents chart templates and values.

Terraform and Helm control different boundaries here: Terraform manages infrastructure resources; Helm packages the application's Kubernetes configuration. An edited chart/value is an intended change until an authorised release applies it. Compare the rendered configuration with the cluster's observed pod state; a chart alone cannot establish that the product delivered correct data.

Compare environment values—image, CPU, memory and variables—with the rendered Deployment and observed workload after release.

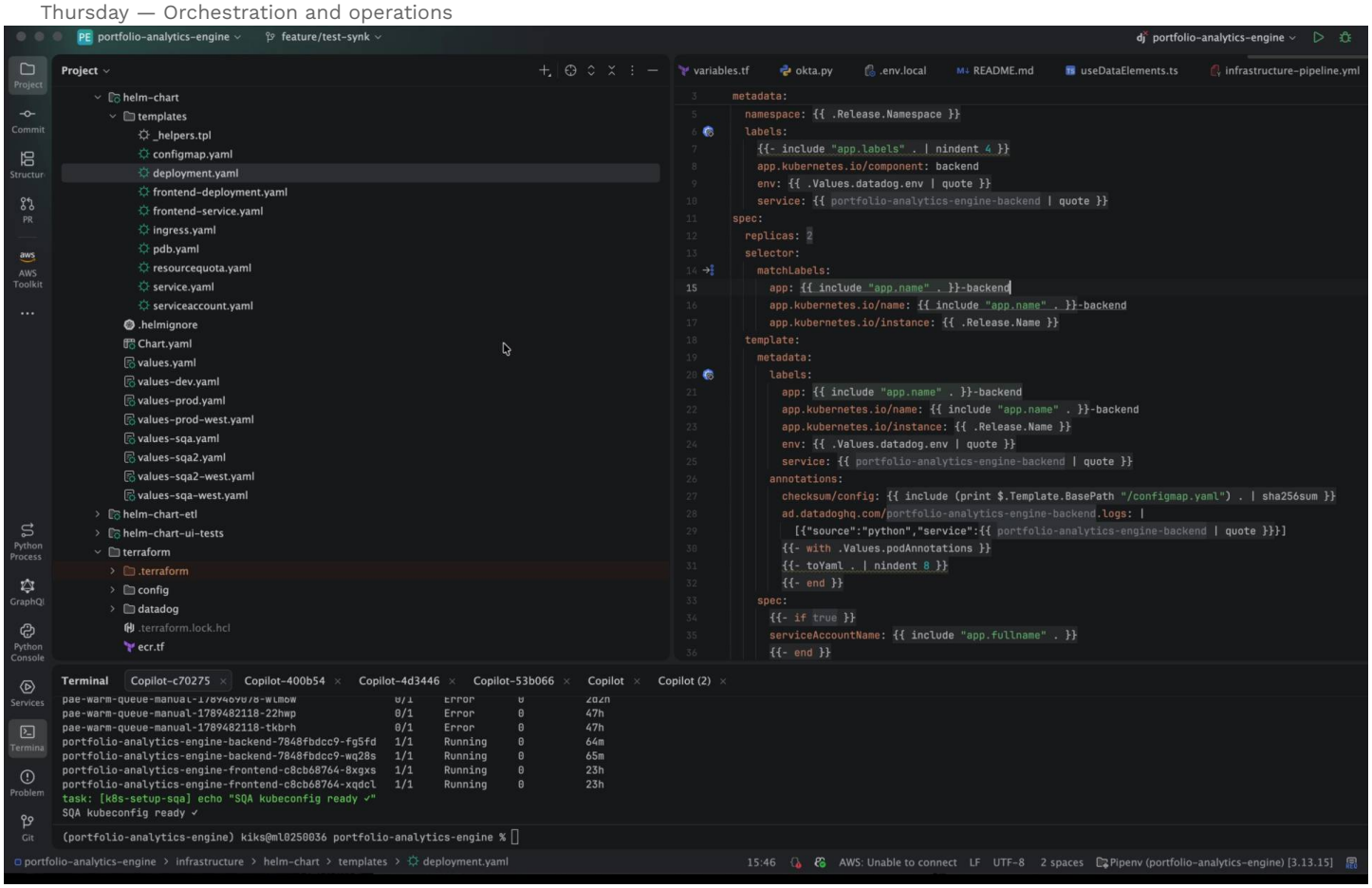


Figure 120: Deployment template: labels and annotations

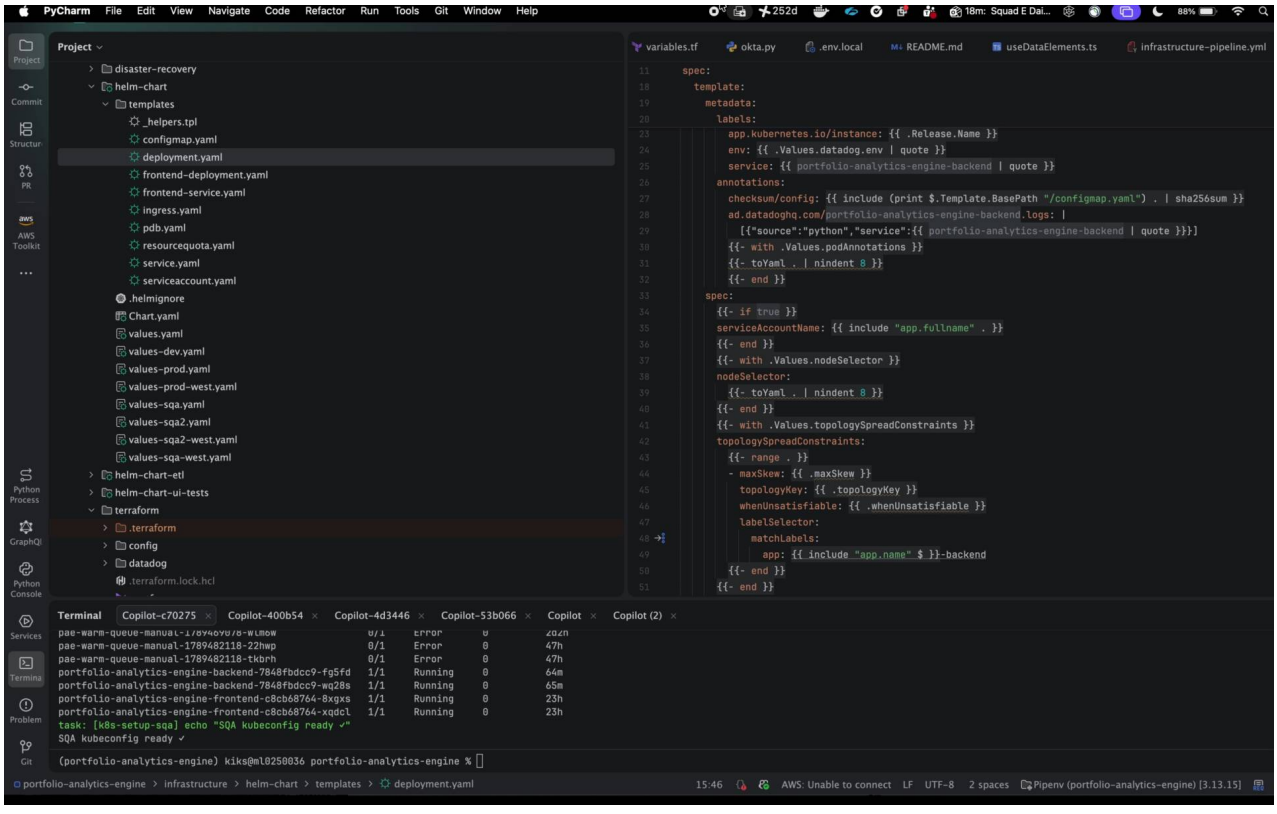


Figure 121: Deployment template: placement settings

On screen: Datadog and Retina

What this sequence controls

A Datadog monitor evaluates a defined query or check against alert conditions; a dashboard groups observations for investigation. The chosen evaluation window, threshold and no-data behavior affect interpretation. Datadog documents those monitor controls.

Retina is the named job/cost view in the supplied platform sequence: the visible evidence is EMR job runs and costs. No broader product capability or current tenant availability is inferred. Keep a healthy backend, a current serving date and a successful business-output check separate. In learner practice, callbacks remain local log evidence; live monitor configuration is a platform demonstration.

Datadog is an observability platform. A product dashboard puts service status, data freshness and monitors on one screen. Look at the serving date as well as the backend health, because a perfectly healthy backend can still be serving yesterday's data. A monitor checks one condition you set up in advance and sends an alert when it trips. Anything you didn't think to set a threshold for, you find by digging through the dashboards and logs. Retina shows EMR job runs and what each one cost.

Your part as a creator is to say which promise needs watching, who responds when it breaks and how you'd know it has recovered. The platform team sets up the monitors.

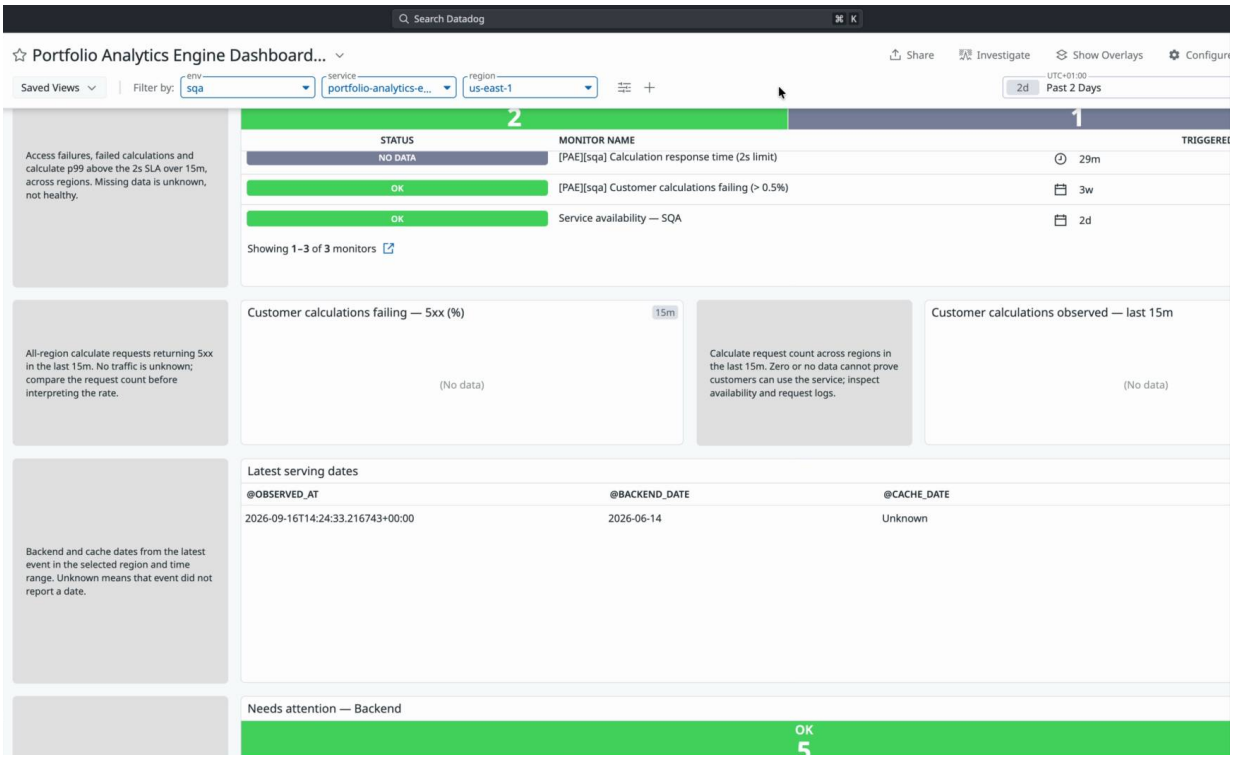


Figure 122: Application dashboard

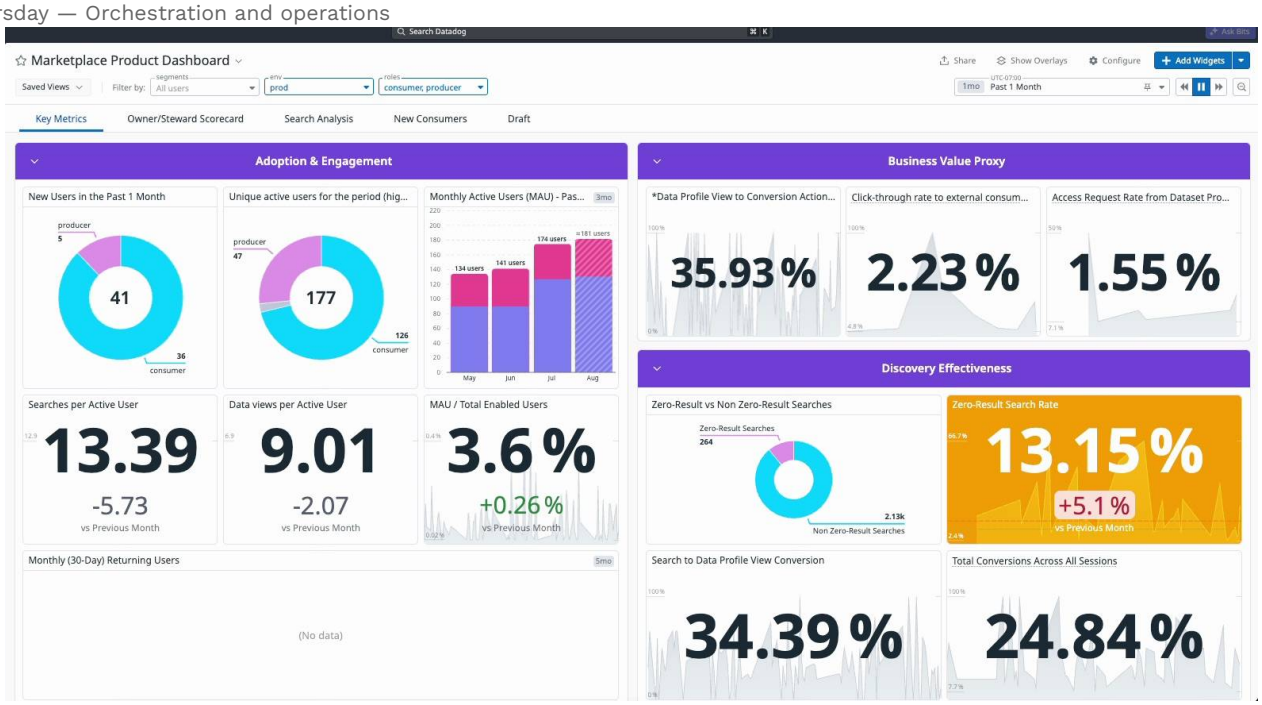


Figure 123: Marketplace dashboard

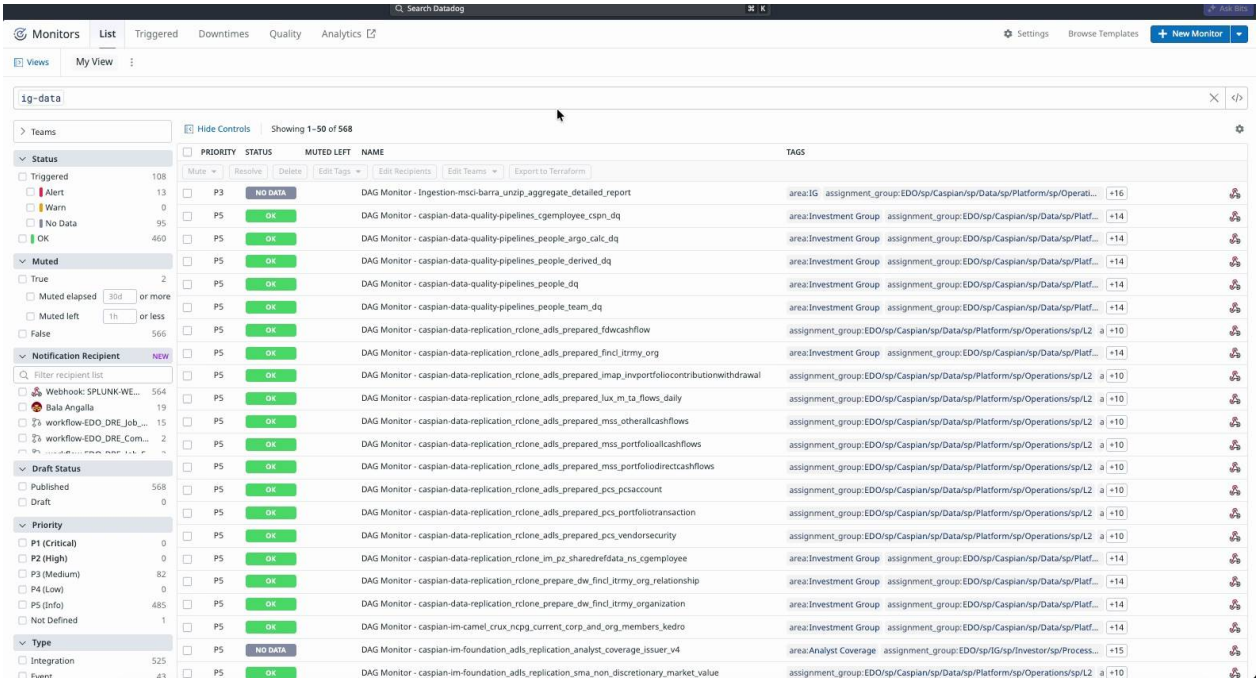


Figure 124: Monitors

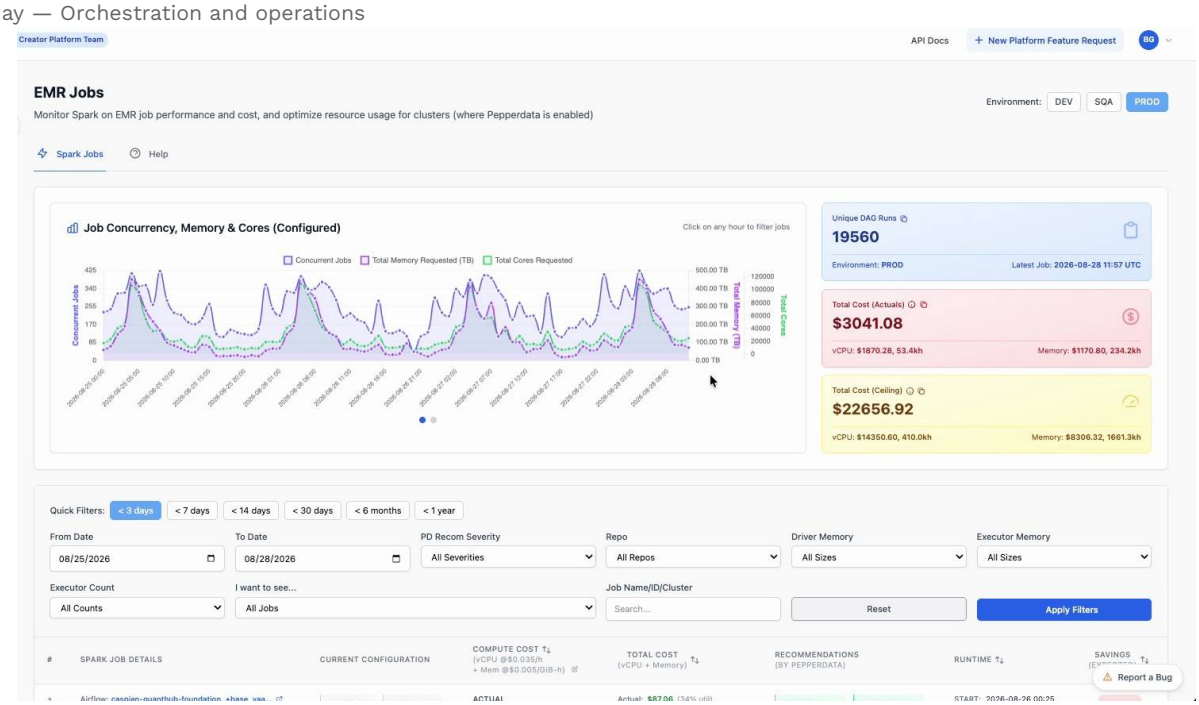


Figure 125: EMR jobs in Retina

On screen: Kubernetes with Lens

What this sequence controls

Kubernetes manages workloads made of pods; a pod contains one or more containers sharing execution resources. Lens presents cluster/workload state and container logs through a graphical interface. Kubernetes defines the pod boundary, and Lens documents selecting a pod and container log.

Read namespace, workload, image and selected pod before interpreting readiness or restart counts. Readiness determines whether a container is ready to serve; a failed liveness probe can restart it. Neither alone proves fresh or correct consumer data. Kubernetes documents these different probes. This sequence is observation, not a learner cluster-management task.

Lens is a desktop app for looking inside a Kubernetes cluster. A deployment runs pods, a pod runs one or more containers, and a CronJob starts jobs on a schedule. The pods screen shows each pod's status and how often it has restarted. Lots of restarts usually means something is unstable, though it won't tell you what that does to your data. The containers screen shows which image is running and its CPU and memory limits, which ties the workload back to a release.

There are two probes. The readiness probe asks whether the pod should get traffic yet. The liveness probe asks whether it's stuck and should be restarted. A CronJob is basically cron in a container. It has no task graph, so you don't get the dependencies between tasks that an Airflow DAG gives you.

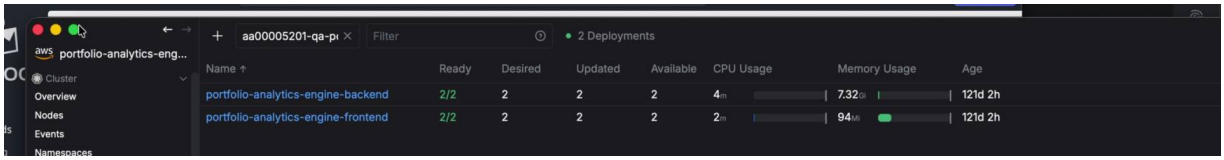


Figure 126: Deployments

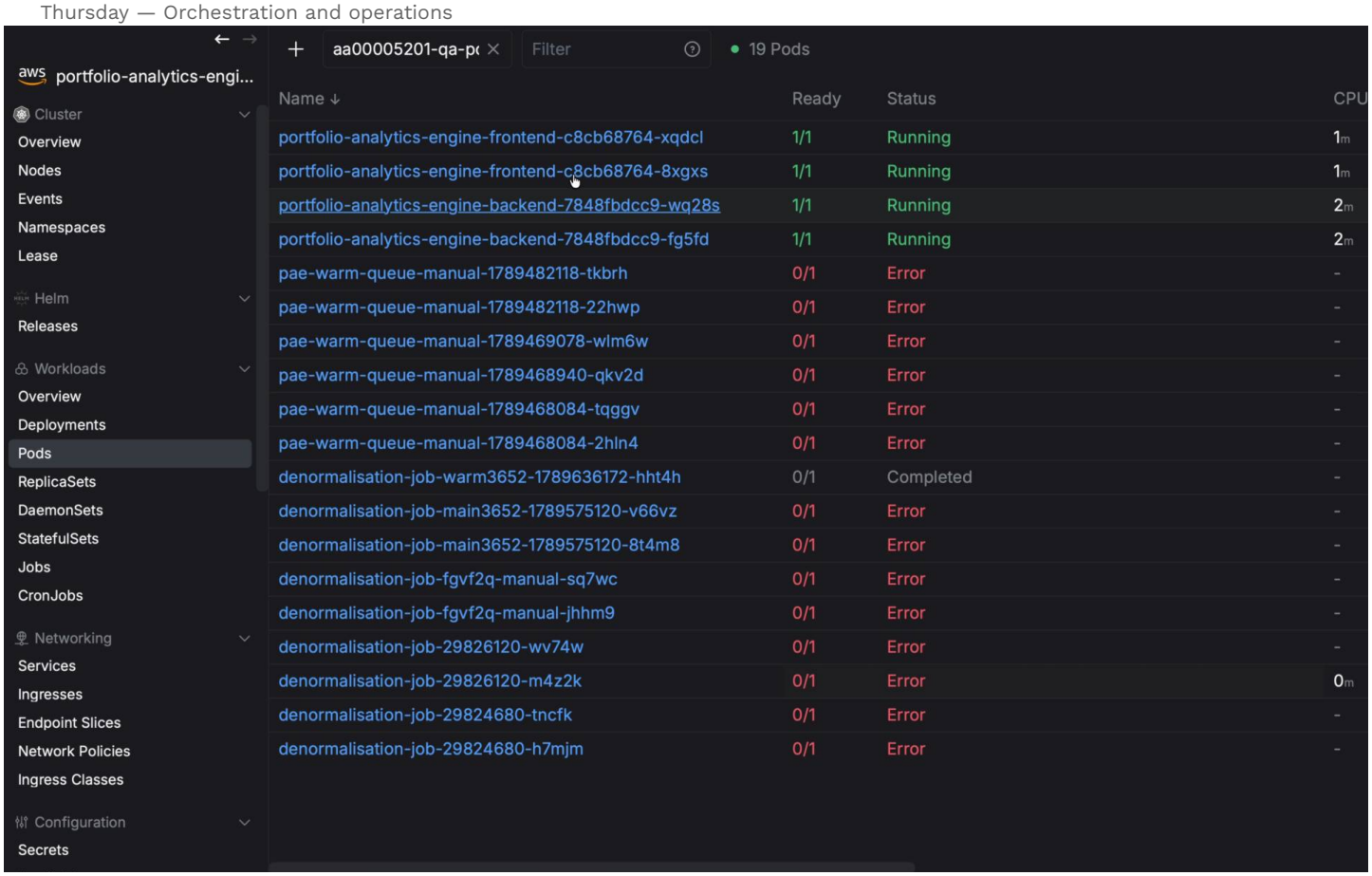


Figure 127: Pods

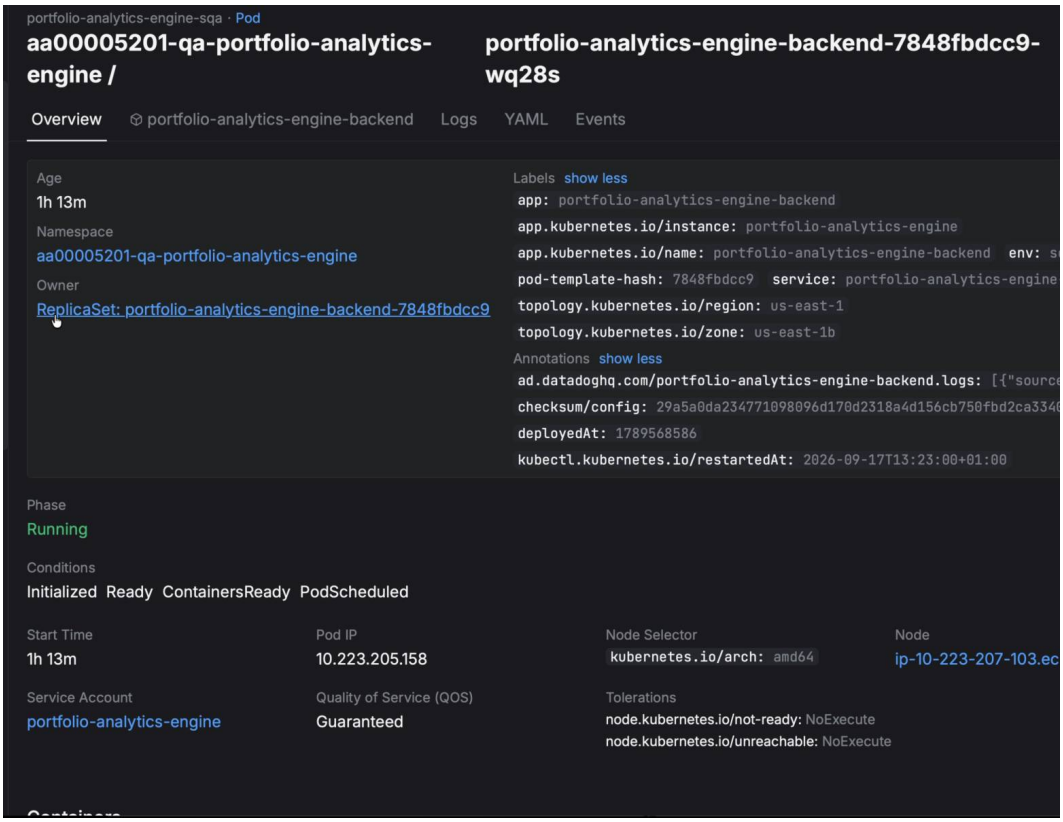


Figure 128: Pod overview

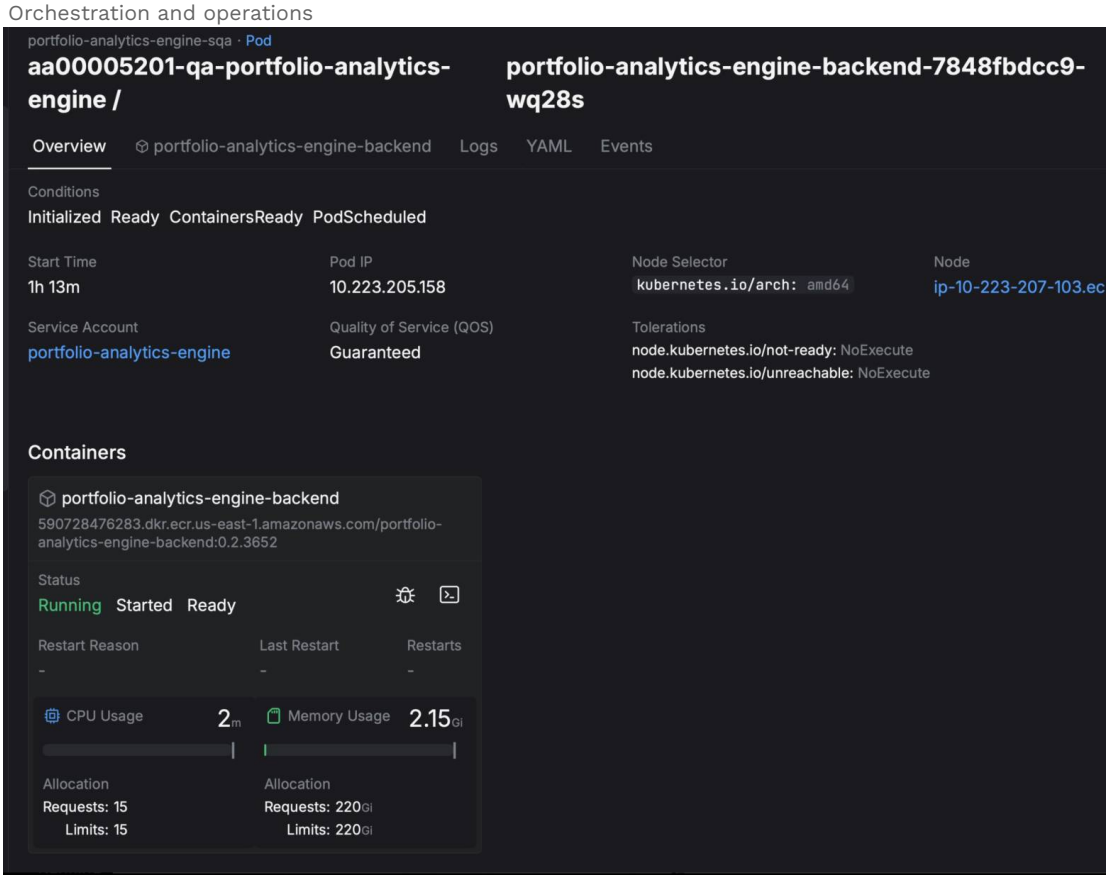


Figure 129: Containers & resources

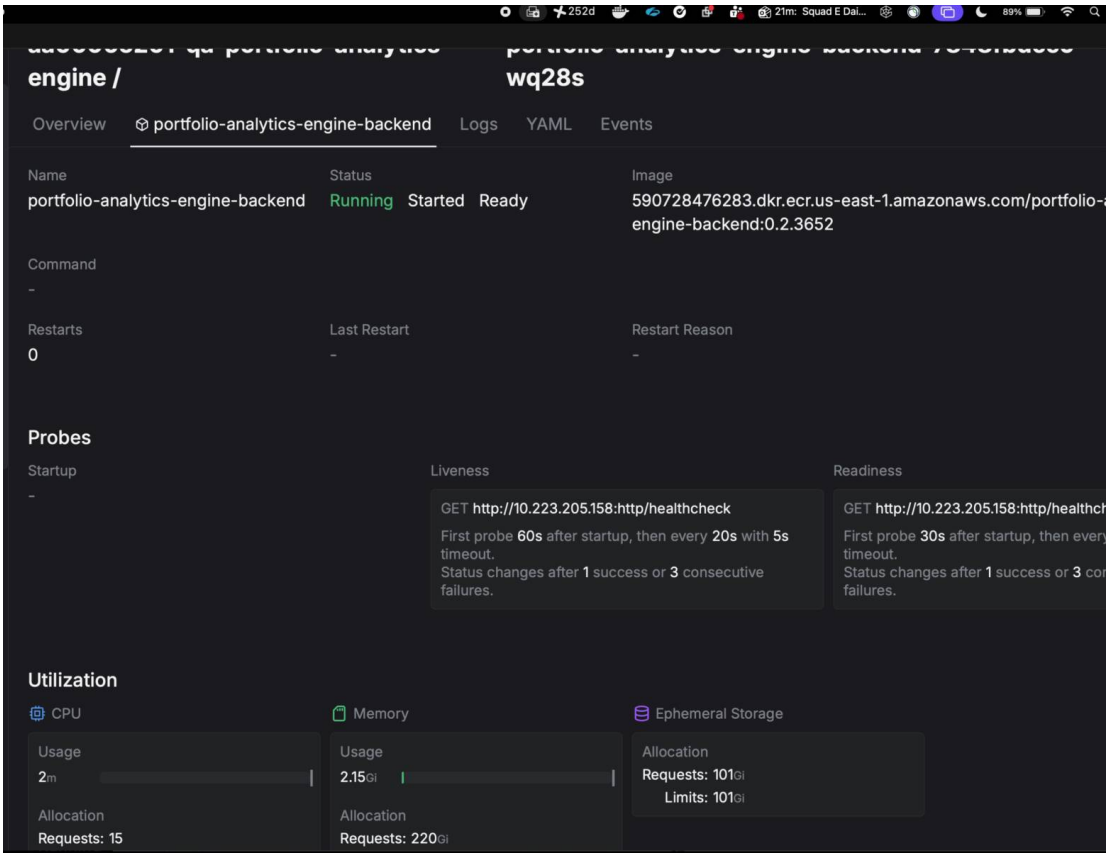


Figure 130: Probes

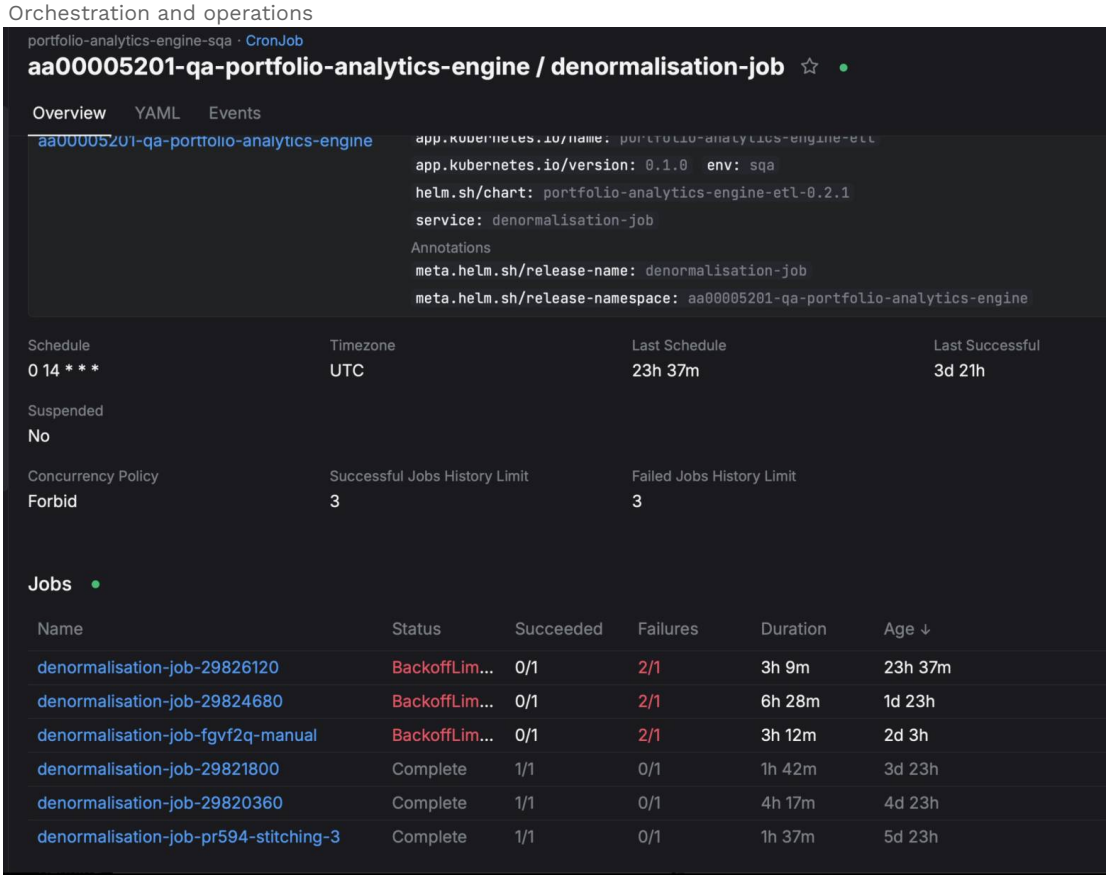


Figure 131: CronJobs

On screen: Reading a pod spec

What this sequence controls

A pod specification declares the requested containers, image, environment, resource settings and lifecycle behavior. Its status reports observed conditions. Workload controllers manage the intended number of pods, so read the owning Deployment as well as an individual pod. Kubernetes describes the Deployment's desired-state role.

In these screenshots, connect the image and environment to the released configuration, then read probes, termination hooks, topology placement and volumes as separate controls. A Datadog environment variable shows intended integration configuration; successful telemetry still needs observed evidence. The pod spec is not the data contract, and editing it is outside learner scope.

The pod YAML declares what you asked Kubernetes for. That's the environment variables, CPU and memory, probes, lifecycle hooks and where the pods are placed. Some of the environment variables wire the pod into Datadog. A preStop hook runs just before a container shuts down. Topology spread constraints spread the copies across machines, and volumes attach storage. The status at the bottom is what Kubernetes actually observed; the spec above is the requested state.

You've met the same split in Airflow. The DAG file says what should happen, and the task states and the data tell you what did, so check both.

portfolio-analytics-engine-sqa · Pod

aa00005201-qa-portfolio-analytics-engine / **portfolio-analytics-engine-frontend-c8cb68764-8xgxs**

Overview | portfolio-analytics-engine-frontend | Logs | YAML | Events

```yaml
apiVersion: v1
kind: Pod
metadata:
  annotations:
    ad.datadoghq.com/portfolio-analytics-engine-frontend.logs: |
      [{"source":"nginx","service":"portfolio-analytics-engine-frontend"}]
    deployedAt: "1789568586"
  creationTimestamp: "2026-09-16T14:23:34Z"
  generateName: portfolio-analytics-engine-frontend-c8cb68764-
  generation: 1
  labels:
    app: portfolio-analytics-engine-frontend
    app.kubernetes.io/instance: portfolio-analytics-engine
    app.kubernetes.io/name: portfolio-analytics-engine-frontend
    env: sqa
    pod-template-hash: c8cb68764
    service: portfolio-analytics-engine-frontend
    topology.kubernetes.io/region: us-east-1
    topology.kubernetes.io/zone: us-east-1d
  name: portfolio-analytics-engine-frontend-c8cb68764-8xgxs
  namespace: aa00005201-qa-portfolio-analytics-engine
  ownerReferences:
    - apiVersion: apps/v1
      blockOwnerDeletion: true
      controller: true
      kind: ReplicaSet
      name: portfolio-analytics-engine-frontend-c8cb68764
```

Figure 132: Pod YAML

portfolio-analytics-engine-sqa · Pod

aa00005201-qa-portfolio-analytics-engine / **portfolio-analytics-engine-frontend-c8cb68764-8xgxs**

Overview | portfolio-analytics-engine-frontend | Logs | YAML | Events

```yaml
spec:
  containers:
    - env:
        - name: AWS_STS_REGIONAL_ENDPOINTS
        - name: AWS_DEFAULT_REGION
          value: us-east-1
        - name: AWS_REGION
          value: us-east-1
        - name: AWS_ROLE_ARN
          value: arn:aws:iam::590728476283:role/portfolio-analytics-engine-qa-service-role
        - name: AWS_WEB_IDENTITY_TOKEN_FILE
          value: /var/run/secrets/eks.amazonaws.com/serviceaccount/token
      image: 590728476283.dkr.ecr.us-east-1.amazonaws.com/portfolio-analytics-engine-frontend:0.2.365
      imagePullPolicy: Always
      livenessProbe:
        failureThreshold: 3
        httpGet:
          path: /nginx-health
          port: http
          scheme: HTTP
        initialDelaySeconds: 5
        periodSeconds: 15
        successThreshold: 1
        timeoutSeconds: 3
      name: portfolio-analytics-engine-frontend
      ports:
        - containerPort: 8080
```

Figure 133: Environment & probes

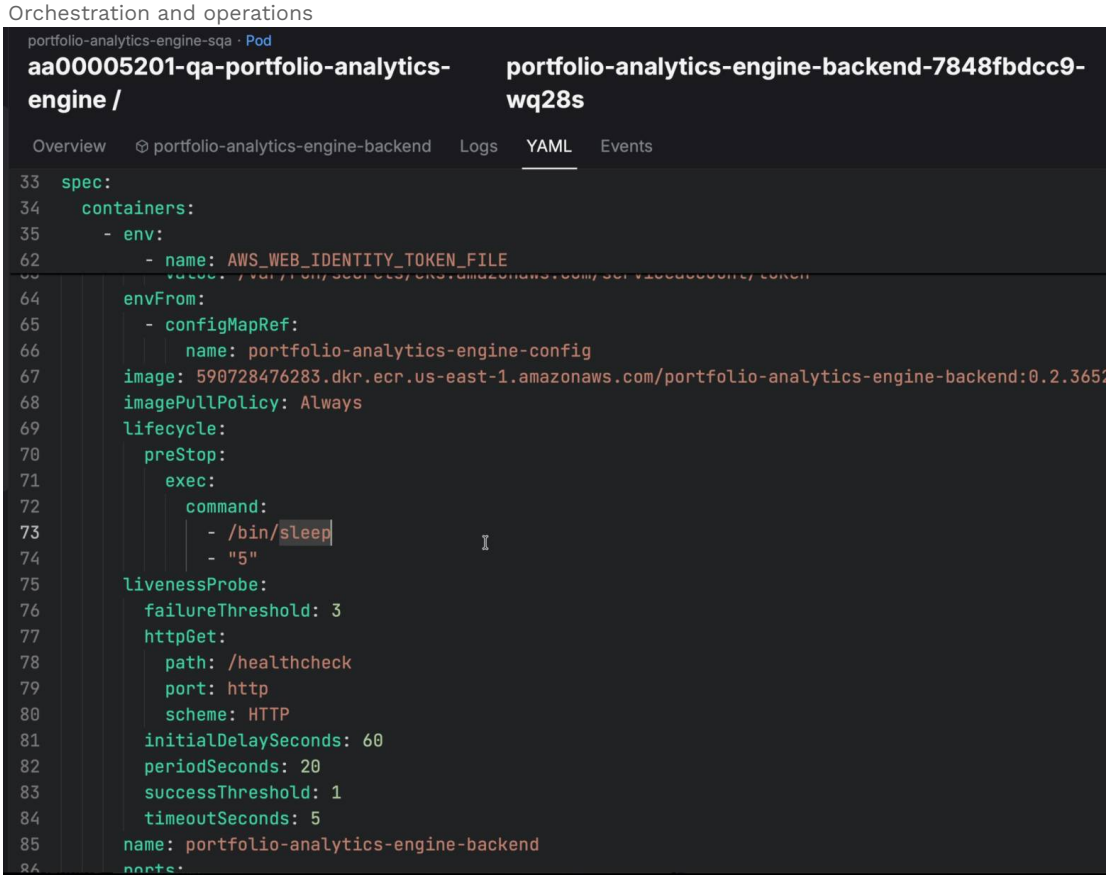


Figure 134: Lifecycle hooks

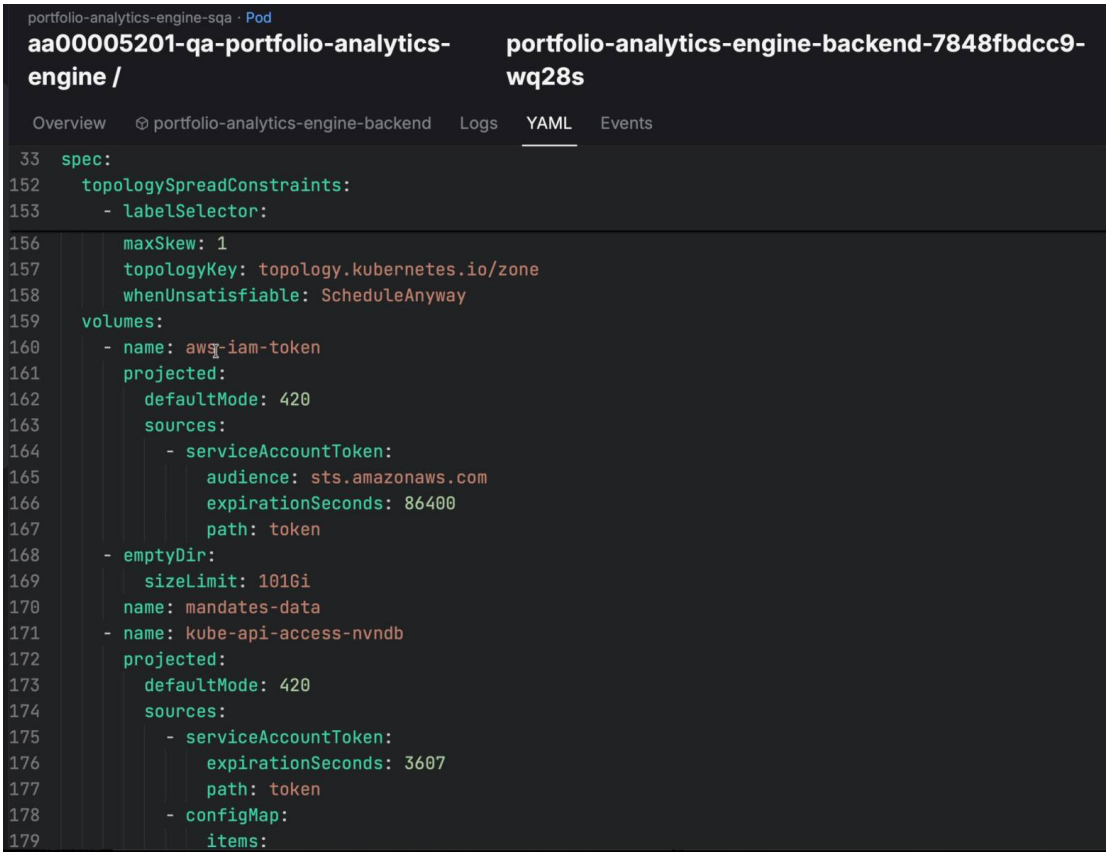


Figure 135: Topology & volumes

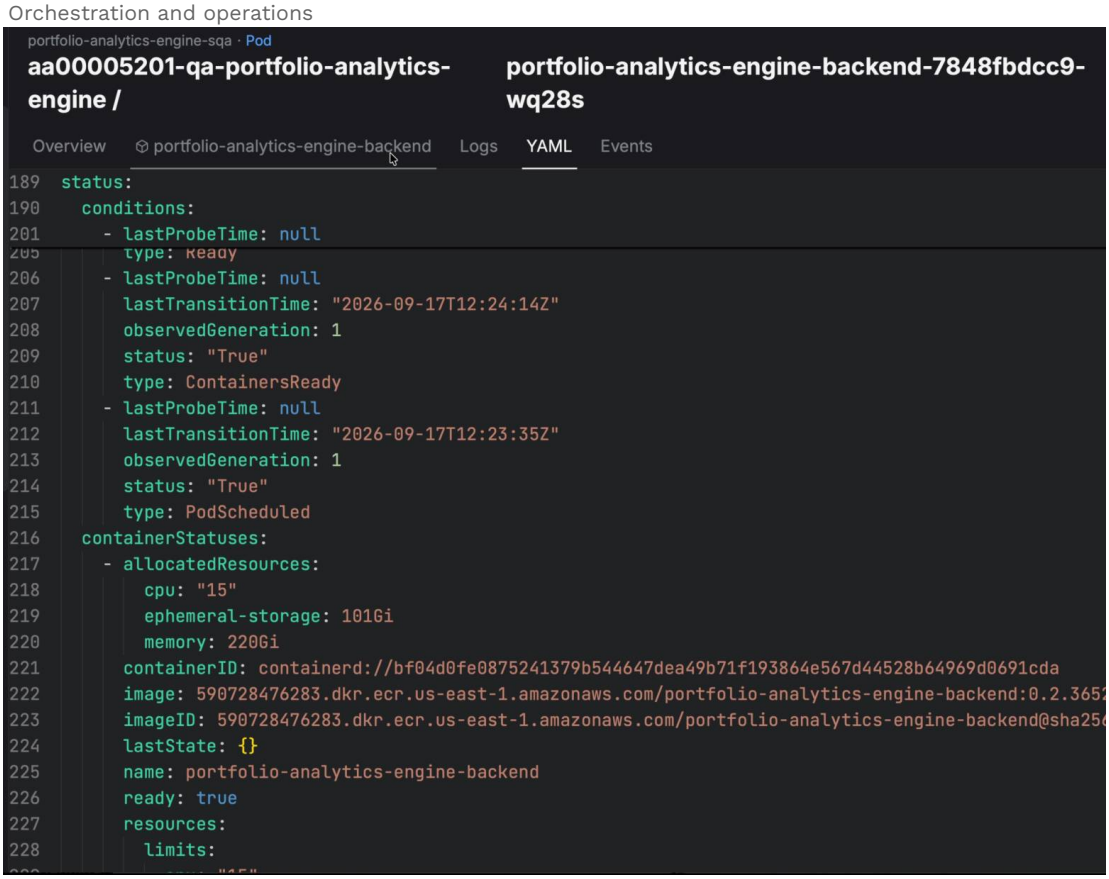


Figure 136: Pod status

Friday briefing and course review

Begin with the CG product-information business brief. Your direct consumer is a fictional internal Product Information Analyst supporting financial-professional enquiries. Build traceable offering identities, published expenses and performance context from the frozen snapshot.

Monday's consumer interview and Tuesday's contract review prepare the PRD, ODCS and STTM. Wednesday's models/tests and Thursday's orchestration/recovery habits support the capstone implementation. The sources and units differ from Portfolio Analytics.

Friday schedule

Time	*Work*
09:00–09:30	Phase 0: readiness, Catalyst PRD and contract
09:30–10:40	Phase 1: personal repository and ETF core
10:40–10:55	Break
10:55–11:25	Continue individual observation, repair and phase-1 checkpoint
11:25–12:00	Phase 2: interfaces, ownership and skill draft
12:00–12:45	Lunch
12:45–14:00	Phase 2: integrate, compare, orchestrate and review
14:00–14:15	Break
14:15–15:15	Phase 3: temporal history, reusable SQL and workflow templates
15:15–15:30	Finish personal report and speaking notes
15:30–16:00	Presentations: every Data Creator speaks; final submission

Today's sessions

- Work and evidence
- Course review
- How do you know when a data product is done?

Work and evidence

Use the current phase guide and complete learner project. Phase 1 and phase 3 are individual; phase 2 is paired with each person's contribution recorded. Every Data Creator submits a personal report and presents at 15:30.

Retain reviewed design drafts, source selection, authored SQL/tests, exact key membership and independent values, controlled failure/repair evidence, observed warehouse and Airflow receipts, and release/temporal results. The evidence checklist identifies the required records. Use the instructor-assigned product folder and target, and run the workspace check whenever switching products.

The phase-1 catalogue contains all 25 ETFs. Phase 2 expands to 109 scoped offering IDs. Missing expenses and reporting dates remain unknown. Explain cost criterion states separately from performance comparison states; gross/net expenses and SMA composite returns have different meanings. Local checks support review but do not establish connected execution.

Course review

Follow one product from a consumer's decision to a checked result. Monday established who needs the data and what it must mean. Tuesday turned interview evidence into a draft specification while keeping a creator responsible for its accuracy. Wednesday gave those promises a technical form: a grain, a mapping, models and tests. Thursday placed the work in an execution sequence with failure and recovery evidence. Friday brings those decisions together in the CG product-information capstone. A successful command is one observation in that chain. The consumer still needs to understand the population, units, limitations and owner of the output.

Use this review in the final consolidation slot. Begin with the product journey and a short team explanation, then choose the application questions that expose your weakest link. The longer scenario and self-check can continue after class. Keep your own contract, mapping, diff, test results and output open; answers backed by those artifacts are more useful than recalling a tool name.

The product journey

Suppose an advisor needs a summary of portfolios with qualifying trading activity. Before choosing a dashboard or model, establish the decision the advisor wants to make. Does the summary explain recent activity, current positions or every portfolio they oversee? Those are different products. A report of active portfolios can correctly omit portfolios with no qualifying trades. A report of all portfolios needs to represent the unknown-cost population as well. The contract makes that choice visible before a query turns it into an accidental filter.

Interview the consumer about their current work, the producer about the source and its limitations, and the creator about feasibility and the checks needed to maintain the promise. Ask for a recent example, the decision made from it and what happens when the information arrives late. Record unresolved questions with an owner. A statement such as "we need better data" is a starting point; an agreed population, unit, freshness rule and acceptance example can guide implementation.

The design canvas collects these choices so that the team can inspect them together. Consumer value, reuse, feasibility and viability matter together: a technically possible product can still be too costly to operate or too difficult for a consumer to adopt. Search the existing catalogue before proposing a new source or output. Explain the gap in what is already available, then decide whether to reuse, adapt or build. The interview activity and canvas are linked in Monday's route; the stakeholder reference helps with interests and adoption when that depth is useful.

A service promise needs an observable measure and a response. "Fresh each morning" leaves several questions unanswered: which business timezone, which qualifying source delivery, which deadline and what happens after a missed deadline? Record what is measured, the threshold and who acts. A quality test can protect a promise only if the model population and failure treatment agree with that promise. Quarantining invalid events, blocking publication and warning a consumer are different decisions.

Specifications, Catalyst and responsibility

Give Catalyst the interview evidence, the intended artifact and the boundaries of the task. Useful context includes the consumer decision, source fields, grain, exclusions, units, known examples and unresolved questions. Request a draft that distinguishes supplied facts from assumptions. Review it against the actual skill instructions and output structure rather than accepting a fluent paragraph as an agreed contract.

A creator owns the resulting decisions and checks. Read the generated contract for invented fields, unstated access, contradictory units and promises that the team cannot measure. When a skill produces a model or test, inspect the files and the diff, identify which source rule each change implements and retain independent evidence. If an output is wrong, supply the specific discrepancy and verify the

correction. Repeating “try again” until a command succeeds can conceal a changed requirement or an omitted test.

Judgment is choosing an appropriate action from the available evidence; taste is recognizing whether the artifact is clear and useful for its reader. Apply both to an acceptance example. A technically valid field list may still fail to tell an advisor which portfolios are included. A clear explanation can still contain an unsupported valuation. Ask a teammate to read the artifact without your spoken explanation, then ask them to identify the population and limitation. Tuesday’s worked review demonstrates that handoff.

For an unfamiliar task, keep the correction small enough to inspect. Check a representative source row, explain the proposed transformation and compare an expected result before widening the build. Model performance can vary across similar tasks; specialist discussion belongs in the AI reference. The practical response is to retain ownership, use bounded changes and choose evidence that could reveal a plausible error.

Grain, mapping and a trustworthy model

State what one row represents at every stage. Distinguish source representations, selected offering rows and return observations. Explain the two authored phase-1 models and the supplied fee-normalization support, then show how phase 2 extends the population to 109 offerings.

Trace one mapping rule from source to consumer. For example, converting a published expense from percentage points to basis points multiplies by 100. Selecting a net expense component and a source representation requires explicit rules. A return comparison needs a declared basis, kind, period and reporting date. Write those rules in the mapping and ensure the SQL, descriptions and tests implement the same meaning.

Before enriching facts, test dimension-key uniqueness and inspect unmatched rows. Joining one fact row to two dimension rows can double a total while producing valid SQL and a successful build. An inner join can lose eligible rows when a lookup is missing. A left join retains rows but introduces null descriptive fields; that does not automatically satisfy the contract. Choose the join from the intended relationship and explain the treatment of unmatched keys.

Reconcile the output independently. Compare exact offering IDs and selected expense values with the frozen source. Reject duplicated representations and unsupported fee components. Missing expense ratios remain unknown; return comparisons require matching basis, kind, period and reporting date.

A useful test checks a plausible failure of the promise. Uniqueness protects row identity. Non-null tests protect required identifiers. Accepted values constrain event meaning. A relationship test checks a declared key connection, and a reconciliation checks the relevant population or amount. A count alone can miss a duplicated row that replaces a lost row. Use representative rows, key membership and totals together when the failure could preserve an aggregate count.

Development and runtime boundaries

Git records the changed files and their history. A branch makes a change reviewable, but code on that branch can still reach a shared database through its configured connection. Read the diff, identify the target runtime and keep learner work in the copied v2 workspace. A merge-conflict clinic practices collaboration after branch/diff concepts; it should not interrupt the protected dbt practice block.

The development container supplies an editor-connected shell and tools. Docker images describe packaged dependencies; a running container is an instance with its own processes, mounts and environment. The separately started Astro environment runs Airflow tasks. A remote Databricks workspace executes connected jobs under an assigned training identity. A path or variable available in the editor terminal is not automatically available to an Airflow worker. Appendix H gives the beginner walkthrough and command/runtime map.

Offline file checks and CSV calculations support review but do not establish a Databricks run. Likewise, installing a CLI does not prove warehouse access. Record the actual runtime, command, output and limitation; when access is unavailable, retain the completed work and name the check that still needs the training environment.

Orchestration, recovery and release

Draw the dependency graph from the data promise. Identify what must complete before a model builds, where quality checks belong and which consumers may use the output after validation. dbt expresses model dependencies; Airflow schedules and observes a wider execution sequence. Running the same SQL through a scheduler does not make it correct. The task graph should preserve the model/test contract established on Wednesday.

For each failure, decide whether retry can change the result. A transient connection failure may recover. An invalid-data test will continue to fail until its input or rule is corrected. Record the failed task and evidence before restarting a whole pipeline. Check whether replay would duplicate data or repeat an external side effect, and use the supplied recovery procedure within the training scope.

Inspect task logs and output after a successful run. A scheduler's green state shows that its tasks reported success; it cannot replace a consumer-level reconciliation. A failure callback in the assigned local route writes log evidence. It does not prove that a remote notification, monitoring platform or production escalation is configured. Thursday's platform demonstrations explain release and operations controls without assigning learners to provision them.

Release control answers a different question from orchestration. A reviewed commit, a promotion record and an approval identify what version is allowed in an environment. Terraform describes infrastructure, Helm packages Kubernetes deployment configuration, and monitoring observes the running system. Read the demonstrated evidence and role boundaries; the course's local checks do not confer production access or approval. Link your runbook to the actual contract, owner, failure treatment and retained evidence.

Common mistakes to challenge

Claim or proposed shortcut	*Better question and evidence*
"The preview looks right, so reuse this product."	Does its documented population, grain, unit, freshness and owner match the decision? Inspect the contract and representative key membership.
"The agent followed the skill, so the output is approved."	Which supplied requirement appears in each changed file, and what independent example could reject the summary? Review the complete diff.
"Both marts have the same total, so the join is correct."	Could they share the same faulty population? Compare raw per-key membership and check multiplicity before aggregating.
"A left join fixes every missing offering."	Was the omission an intended unknown-cost population or a missing reference? Define the consumer population before selecting the join.
"The setup or baseline check passed."	Which environment and state were checked? Identify remaining authoring, actual execution and learner-explanation evidence.
"Retry everything until the scheduler is green."	Is the failure transient, invalid input or a wrong rule? Retain the failed task evidence and prove replay does not duplicate the result.
"A branch makes the experiment safe for shared systems."	Which configured runtime and identity will the command use? Keep the authorized scope separate from Git history.
"A working local output is a released connected product."	Where did execution occur, and which connected, consumer and release gates still need evidence? Name the owner of each remaining gate.

Apply the chain to a change

Review a request to include unknown product costs

DURATION 15–20 minutes

OUTPUTS A change-impact note and a review decision

The analyst asks for all scoped offerings, including those without a published net expense ratio. A proposed patch uses a left join but replaces every missing expense with zero. Review whether that preserves the enquiry and its uncertainty states.

1. Explain the new consumer decision and rewrite the population promise. Decide which fields may legitimately be zero and which represent unknown information.
2. Identify the starting population and the intended output grain. Trace the proposed joins and list the key uniqueness and membership evidence needed.
3. Update one mapping row and two acceptance examples, including an offering with an unknown expense and an offering with a duplicated source representation.
4. Describe the model/test changes you would ask Catalyst to draft, the supplied context and the review checks you would perform yourself.
5. Decide which local checks are available now and which connected or operational checks need the training environment. State how downstream consumers would recognize the changed contract.
6. Give a review decision with evidence: accept, request specific corrections or leave a named prerequisite pending.

A strong answer distinguishes an an unknown expense from a confirmed zero expense. It does not use a join change as the specification. It records any changed units, null meaning or consumer totals and updates the tests with the implementation. If the team cannot establish a connected run, the decision says so while retaining the local results. Ask a peer to challenge one assumption before finalizing the note.

Check your explanation

Learning Review

Retrieve the product journey without reading the day titles: name the consumer decision, interview evidence, population/grain, service promise, mapping rule, model/test, orchestration boundary and handoff limitation. Then use your own CG product-information artifacts to show one link between each adjacent pair.

Apply the change scenario above to your team's output. Explain how you would detect a wrong join, a duplicated key, a late source and an unsupported claim about connected execution.

Check your answer against the criteria below. Where a criterion is missing, write one concrete next action and its owner rather than marking the whole product finished.

Evidence area	*What a reviewer should be able to verify*
Consumer and contract	The named decision, intended population, grain, units, acceptance examples and limitations agree.
Catalyst ownership	Supplied evidence is distinguishable from assumptions; generated files were read and corrected by a named creator.
Mapping and models	A business rule is traceable to authored SQL and a test; supplied baseline stages are identified.
Quality	Representative rows, key membership and an independent reconciliation support the claimed result.
Runtime and recovery	The actual local or connected environment is named; failure, repair and replay evidence match that boundary.
Handoff	The consumer can interpret the output; unresolved checks, responsible owners and next actions are explicit.

For each evidence area, use a simple self-check: 0 means absent or contradicted; 1 means a plausible claim with incomplete evidence; 2 means an artifact, an observed result and an explanation that agree. A teammate should be able to find the evidence without reconstructing your terminal session. Use a named pending gate when a required environment is unavailable; retain the valid local result rather than claiming a connected run. Select the next activity from the weakest link in the journey, not from the number of commands completed.

Continue with a focused next step. Repeat a core task if you cannot yet explain its evidence, choose an advanced clinic when the core is clear, or use the reference for a specific question. The v2 challenge reference preserves the full offered material and optionality. The Questions and answers reference addresses recurring questions; the worked contract reference provides another artifact to inspect. Keep the distinction between learning evidence, runtime readiness and approval when you carry the habits into a real product.

How do you know when a data product is done?

Agree acceptance criteria with the consumer and the accountable product roles. A consumer-aligned table containing rows is necessary for some products, but it does not establish that the required decision, quality, timing, access or support conditions are met. A percentage-complete claim needs a declared denominator and evidence.

Trace each criterion to a test, review or sign-off. Include the relevant consumer classes and unresolved conditions, rather than relying on one enthusiastic user’s approval for every use case. Completion is a decision against agreed criteria; later operation and evolution remain ongoing responsibilities. Tuesday’s worked review shows why a complete-looking contract is still a draft.

Projects and Capstone

Choose your assigned project below. Each phase includes setup, tasks, hints and checks. Advanced phases and optional clinics have separate time budgets. Runtime setup covers Portfolio Analytics. The CG capstone guide supplies its own source, workspace and target instructions.

Project	*Route*
Git collaboration	Preparation / collaboration clinic
Docker and runtime boundaries	Supervised preparation / clinic
dbt — Portfolio Analytics	Wednesday core; further study separately timed
Airflow — Portfolio Analytics	Thursday core; advanced separately timed
Databricks discovery	Prepared access / discovery
CG product-information capstone	Friday individual and pair phases
Design workshops	Optional facilitated practice
Further practice and optional clinics	Historical/small fixtures and optional clinics

Task index

Use the task links to open the instructions in each project. Foundation work is selected for preparation or the taught practice block; advanced work and optional clinics are separately timed.

Git collaboration

Task	*Page*
Inspect the three working states	249
Commit an intentional contract change	250
Branch and combine compatible work	251
Resolve an ownership conflict	252
Restore a mistake and explain the final history	253
Clone and inspect a collaboration boundary	257
Push and propose a scoped pull request	258
Respond to a review with a second commit	259
Integrate concurrent upstream work	260
Read the final review and merge state	261
Compare merge and rebase on disposable local history	263
Recover a lost local branch reference through reflog	264

Docker and runtime boundaries

Task	*Page*
Run an image and inspect the lifecycle	270
Explore an interactive shell	271
Use a bounded bind mount	272
Author and run a simple holdings image	273
Prove repeatability and clean scoped resources	274
Separate named build targets	278
Gate the runtime on a tested artifact	279
Run with a bounded runtime user	280
Keep runtime contents focused	281
Repair a path assumption and explain the result	282
Explain cache reuse and a controlled input rebuild	284
Run the prepared app through bounded Compose configuration	285

dbt — Portfolio Analytics

Task	*Page*
Inspect the project and source grain	293
Declare and use sources	294
Normalize type, unit and key contracts	295
Build a useful mart through ref	296
Add minimum key and relationship proof	298
Diagnose a join fan-out before repair	302
Repair enrichment at the intended grain	303
Write reconciliation that detects the negative case	305
Prove incremental update and replay behavior	306
Explain dependency impact and hand over	307

Databricks discovery

Task	*Page*
Establish the CLI identity boundary	336
Separate workspace and Unity Catalog names	337
Inspect permitted schema and table metadata	338
Filter JSON and explain a limited result	339
Produce a bounded discovery handoff	340

Airflow — Portfolio Analytics

Task	*Page*
Inspect a prepared orchestration baseline	316
Author a small Python DAG with real dbt tasks	317
Demonstrate the dependency graph and scheduled run	318
Inspect a controlled task failure	319
Explain the run and environment boundaries	320
Define bounded replay and input identity	325
Inject failure and observe useful retry behavior	326
Prove output correctness after recovery	327
Use Astro UI to make an operational decision	328
Write a short incident handoff	329

Kafka optional clinic

Task	*Page*
Read the dataset-arrived contract	362
Start the prepared local broker and inspect a topic	363
Produce and follow a uniquely identified event	364
Verify the dbt output belongs to the event	365
Replay and explain duplicate policy	366

CG product-information capstone

Phase	*Objective*	*Page*
Phase 0	Reviewed PRD, ODCS, STTM and readiness	346
Phase 1	Individual traceable 25-ETF core	346
Phase 2	Pair integration, comparisons and checked release	346
Phase 3	Individual temporal history and reusable workflows	347
Submission	Personal report and presentation	347

Route boundaries

Use the Portfolio Analytics projects for assigned dbt and Airflow practice. The small synthetic fixtures are optional local exercises; their results do not establish Databricks or Astro execution.

databricks-cli: Phase two — not offered This tool has a foundation route only. Return to the tool index.

kafka: Phase two — not offered This tool has a foundation route only. Return to the tool index.

Earlier full-book sources, local guides and v1 exercises are retained as dated evidence in the authoring workspace and separate instructor archive. Use the current projects for learner practice.

Git collaboration

Projects contents · Task index

- Foundation core — page 248
- Advanced core — page 256

git challenges

Git makes a data-contract change inspectable through snapshots and ancestry. Foundation practice stays local: distinguish working/index/committed content, make intentional commits, combine branches, resolve a business ownership conflict and restore only an accidental edit. Advanced practice carries a prepared independent contract through an authorized private sandbox PR, real review revision and concurrent upstream change. The result is preserved requirements and an understandable history, not merely a command count.

Before you start

Prepare an isolated attempt through mise and check its repository root before any change. Keep the baseline contract and evidence at the attempt root and local Git work in the inner lab. Foundation requires only delivered Git and a local training identity. Advanced requires an instructor-designated private repository, permitted branch prefix, CLI identity and arranged reviewer. Missing remote or human preparation remains pending; a local simulation can teach mechanics but does not establish peer review. Neither route authorizes publishing the course checkout or modifying a production branch.

Suggested timeline and tasks

Estimates include reading, practice, checks and explanation. Allow additional time for environment setup and optional extensions.

- Phase: Foundation · Task: Inspect the three working states · Minutes: 12–18
- Phase: Foundation · Task: Commit an intentional contract change · Minutes: 12–18
- Phase: Foundation · Task: Branch and combine compatible work · Minutes: 12–18
- Phase: Foundation · Task: Resolve an ownership conflict · Minutes: 12–18
- Phase: Foundation · Task: Restore a mistake and explain the final history · Minutes: 12–18
- Phase: Advanced · Task: Clone and inspect a collaboration boundary · Minutes: 15–22
- Phase: Advanced · Task: Push and propose a scoped pull request · Minutes: 15–22
- Phase: Advanced · Task: Respond to a review with a second commit · Minutes: 15–22
- Phase: Advanced · Task: Integrate concurrent upstream work · Minutes: 15–22
- Phase: Advanced · Task: Read the final review and merge state · Minutes: 15–22

Choose your phase

- Foundation: Foundation core with explicit completion evidence.
- Advanced: Independent advanced baseline and finite core, separately timed.

Finish the core tasks before choosing optional stretch.

Teaching concepts

Snapshots and ancestry protect different aspects of a change. Read working and staged diffs before committing; read branch history and final contract after merging. A clean status cannot prove preserved business facts. Review identity, revision head and actual hosted state extend that reasoning into collaboration. End-of-phase reasoning keys explain decisions after your attempt; full

runnable references remain in the instructor package. Remote writes and reviewer participation are classroom-authorized actions, never automatic verifier behavior.

git: Phase 1 — foundations

Purpose and outcome

Learn to preserve and explain a small data-contract change using local Git. You will distinguish working files from staged content and committed history, make intentional commits, combine branch work, resolve a meaningful ownership conflict and restore an accidental edit. The contract is small so the tool behavior is visible. The point is reviewable changes and preserved requirements, not memorizing a long command sequence.

Dataset and pipeline

The starter is an ordinary holdings contract with owner, grain, currency and daily delivery. It is a documentation fixture, not the later Databricks dataset. You will refine its deadline and escalation promises while keeping the original grain and currency facts. An untracked notes file demonstrates that a file can exist without entering the next commit. Compatible branches edit deliberately separated facts; competing owner edits create a known merge conflict. This local history becomes the evidence for your explanation of how requirements were combined.

Prerequisites and workspace

Use the delivered Git executable and a configured training identity. Prepare an isolated attempt from the training project root through mise, then create a new inner lab repository inside that copy. Confirm the repository root before running change or restore commands. The course repository and another learner's directory are outside the exercise boundary. Keep the copied baseline contract and evidence note at the attempt root, with only the inner lab under its new Git history.

```
mise run challenge:v2:prepare git phase-one-certified
cd .local/challenges-v2/git/phase-one-certified
mkdir lab
cp contract.md lab/contract.md
cd lab
git init -b main
git --version
```

Record your configured author identity without changing another repository's global settings. If Git does not support the initial-branch option in the delivered version, inspect its help and create the main branch explicitly; record that measured variation. Use a new workspace for another attempt instead of resetting existing work. This phase performs local actions only and does not require a hosting account, production branch or public repository.

Starter behavior

The supplied contract is valid text and there is no intentionally broken repository. Initial status shows an untracked contract until you stage it. The first task deliberately creates an additional notes file so you can identify exactly what enters a commit. A later ownership merge is expected to stop with a conflict; the README names that failure before it happens. A clean starter check establishes readable input, not completion. Keep the contract baseline and the new lab separate so the verifier can inspect both without overwriting your work.

Timeline and completion standard

Core reading, authoring, checking and explanation: 60–90 minutes. Allow additional time for setup and optional stretch.

- Task: Inspect the three working states · Minutes: 12–18 · Evidence: First commit contains only contract.md; notes remain outside that snapshot
- Task: Commit an intentional contract change · Minutes: 12–18 · Evidence: Commit diffs distinguish 09:00 UTC deadline from escalation policy

- Task: Branch and combine compatible work · Minutes: 12–18 · Evidence: Both separated facts survive; history explains fast-forward or merge commit
- Task: Resolve an ownership conflict · Minutes: 12–18 · Evidence: Expected merge failure; no markers; resolution retains owner and escalation decisions
- Task: Restore a mistake and explain the final history · Minutes: 12–18 · Evidence: Clean lab; committed requirements unchanged; explanation distinguishes restore from revert

Independent starter files Read the owning phase before editing. Prepare from the training project root with `mise run challenge:v2:prepare git phase-one-certified`; edit the copied attempt.

Authored-file list

Task	*Learner-authored paths*
Inspect the three working states, Commit an intentional contract change, Branch and combine compatible work, Resolve an ownership conflict, Restore a mistake and explain the final history	`lab/contract.md`, `evidence.md`

Declared baseline

The supplied contract is the starting point. Create the inner Git repository under `lab/` and keep `evidence.md` outside it, beside the copied starter. The advanced phase also supplies a review scenario; the training remote and reviewer are arranged separately.

Use a fresh workspace for another attempt; preparation and reset preserve existing work.

Core challenges

Inspect the three working states

Concept

The working tree contains editable files. A commit records a snapshot and its parent history. The index is the proposed next snapshot, not a synonym for every file in the directory. Staging one file selects its current content for the next commit; changing that file again can leave one version staged and another in the working tree. An untracked file has no committed baseline, so its presence in a folder does not imply version control protects it. This distinction matters when a contract edit and an unrelated note share a workspace.

For the first snapshot, the baseline contract should enter history while the scratch notes should not. Inspecting the staged diff and the committed file list gives stronger evidence than a commit command returning success. The repository root is also part of the boundary: operating from the course checkout instead of the disposable lab would select the wrong work. Confirm that root before reasoning about status.

Challenge

Use the prepared inner lab and configure a local training name/email if needed. Confirm its repository root, inspect `git status`, and create an untracked `notes.txt`. Stage only `contract.md`, inspect `git diff --cached`, then make the initial commit. Explain why the note is absent from that snapshot.

Keep the commit hash in `evidence.md` at the parent attempt root. Remove or ignore the scratch file before later clean-state checks. Stage an explicit filename rather than the entire attempt, so runtime files and evidence stay outside this first snapshot.

> Inspect the proposed snapshot before making history
>
> **Hint**
>
> Use git status `--short` and git log `--oneline`. Configure identity in this lab, not globally.
> Compare status with the staged diff before committing, then compare the commit's file list afterward. A note visible in your editor may still be untracked. Use an explicit filename when staging the baseline. If the root probe names the outer course repository, stop and enter or initialize the intended lab before making a commit; the observed status belongs to the wrong boundary.

Check

Inspect the first commit's file list and content, verifying that only the contract was recorded and the scratch note was not. The committed contract must retain the supplied owner, grain, currency and daily-delivery facts. Record the hash and explain the difference between an untracked note, a staged contract and the committed snapshot.

A successful commit with extra files fails this task's intent even if status becomes clean. If an unrelated file was staged accidentally, unstage it without discarding its working content, inspect again and explain the correction before committing. Keep evidence outside the inner repository. The local verifier can inspect final contract/history, while the staged-before-commit evidence explains the first snapshot decision.

```
git rev-parse --show-toplevel
git diff --cached
git show --stat HEAD
```

Phase contents · Reasoning key

Commit an intentional contract change

Concept

A useful commit makes one intention reviewable. A deadline clarification and an escalation rule can be related business work while still benefiting from separate snapshots: a reviewer can see the delivery promise and then the response to a missed promise. A commit message describes that intent; the actual diff proves what changed. Combining unrelated edits makes later diagnosis or reversal harder.

Staging also lets you choose when a change enters history. An accidental extra line in the working file need not be committed with the deadline, and unstaging differs from discarding content. Inspecting both ordinary and cached diffs reveals that boundary. The fixture remains ordinary contract text, so the review should preserve grain and currency while adding a clear time zone rather than silently replacing the original business assumptions.

Challenge

Change Delivery to daily before 09:00 UTC. Review the working diff, stage that edit, inspect the staged diff and commit it with a message explaining the deadline clarification. Add `Escalation: Training support` in a separate commit.

Save both hashes and their scoped diffs in the parent evidence note. Confirm the original grain, currency and owner remain. If an unrelated edit appears in the index, unstage it without discarding its working content. Each commit should let a reviewer assess one contract decision independently.

Make each contract decision independently reviewable

Hint

git diff shows unstaged changes; git diff `--cached` shows staged changes.
Read each staged diff as though reviewing someone else's contract. Does the deadline name a time zone, and does the second change name a usable escalation responsibility? If another edit appears, remove it from the index without deleting its working content. The two hashes should identify independently understandable decisions rather than two arbitrary saves of the same unfinished paragraph.

Check

Inspect both commits and confirm the first clarifies delivery at 09:00 UTC and the second adds the escalation text. Each diff should preserve grain, currency and owner facts except where a later ownership task intentionally changes them. Explain how staged content differed from the working file and why the separate commits help a reviewer assess intent.

Record the parent relationship and scoped content, not only two hash strings. A commit message claiming deadline work does not excuse an unrelated deletion in its diff. If you staged the wrong edit, show the bounded correction without rewriting or deleting another learner's history. The final local phase assessment needs the declared deadline/escalation result and your explanation of the commit boundaries.

```
git diff
git diff --cached
git show --stat HEAD
git log --oneline -3
```

Phase contents · Reasoning key

Branch and combine compatible work

Concept

A branch names a position in commit history; it is not a separate copy of every file you must manage manually. Switching branches changes the checked-out snapshot, while merging combines ancestry and file changes. Compatible edits can be combined without conflict when they touch sufficiently separate content from a shared baseline. Adjacent edits in a tiny file can still conflict, so compatibility is a property to observe rather than assume from different branch names.

Use distinct contract facts for this demonstration, such as a nonadjacent delivery refinement and a newly separated producer note. The goal is to preserve both intentions and explain whether history advanced directly or recorded a merge. Fast-forward means the target simply reached a descendant commit; a merge commit records combined parents when histories diverged. Neither outcome alone tells whether the final text is correct.

Challenge

Create `clarify-grain` and `producer-note` from a common main commit. Clarify the existing grain on the first branch. On the second, append a separate Producer note section with a blank line before it. Commit each change before switching branches.

Inspect each branch's diff, merge both into local main and review the complete contract. Record the common ancestor and whether each merge fast-forwarded or created a merge commit. If the short file produces unintended overlap, record and repair the separated note layout; reserve the deliberate ownership conflict for the next task.

> Verify separated changes before calling them compatible
>
> **Hint**
>
> Use git switch -c and git merge. Commit before switching away from a change.
> Compare each branch with its shared ancestor before merging. Different branch names do not ensure different text regions. Keep this task's compatible demonstration distinct from the later deliberate owner conflict. Use the graph to identify whether main simply advanced or combined divergent parents, and use the final file to prove that both business edits survived.

Check

The final local contract must contain both intended branch facts with the original deadline, escalation and data meaning retained. Inspect branch diffs, merge output and the graph to explain the observed merge mode. A clean merge that dropped a requirement is not success, and an accidental conflict should be recorded and repaired rather than hidden.

Retain the shared baseline and resulting hashes, identifying the branch that carried each fact. Explain why a branch pointer and working-tree snapshot are different concepts. The later ownership task supplies the intentional conflict case; this task should demonstrate a genuinely compatible path through local history without remote activity or broad resets.

```
git log --graph --oneline --all
git diff main...producer-note
```

Phase contents · Reasoning key

Resolve an ownership conflict

Concept

A conflict means Git cannot choose a combined text automatically. It does not mean either branch is necessarily wrong or that a valid business decision follows from accepting one side. The conflict markers show candidate content from two histories; the learner must understand the requirements and produce one coherent contract. For ownership, the resulting named owner and escalation responsibility must be compatible rather than two contradictory labels.

The merge remains unfinished until the resolved file is staged and committed. Removing markers without inspecting surrounding text can still lose the deadline or currency facts. A merge commit's parents record which histories were combined, while the resolution diff records the actual decision. That evidence is useful to a future reviewer who did not watch the editor. The exercise deliberately stops with a nonzero merge result so the expected failure can be distinguished from tool or identity setup problems.

Challenge

From the same main state, create `upstream-owner` with `Owner: Operations team`. On main, set `Owner: Analytics team`. Merge `upstream-owner` and retain the expected conflict state.

Inspect both candidate versions, then resolve to `Owner: Analytics team` with `Escalation: Training support`. Keep the existing deadline, grain and currency. Remove all markers, stage the resolved file, inspect its staged diff and complete the merge commit. Explain the ownership choice in the parent evidence note. A removed marker is only part of the repair; the resulting contract must be coherent.

Resolve the business decision, then complete the merge state

Hint

Read git status after the merge fails. Edit the text itself, remove all conflict markers, then stage it.
Read the entire conflict region and both parent versions before editing. The agreed contract may combine requirements without keeping either side verbatim. Check for markers after resolution and inspect the staged file, not only the visible editor buffer. If the merge failed for an unrelated configuration reason, repair that prerequisite before treating it as the intended ownership exercise.

Check

The initial merge should stop with the named ownership conflict and nonzero status. After resolution, the file must contain no markers, preserve the deadline/grain/currency promises and state the agreed owner/escalation responsibility. Inspect the completed commit's parents and resolution diff so the evidence links both histories to the final decision.

Explain why accepting an entire side could lose another requirement and why staging a marker-free file does not by itself finish the merge. Retain the expected initial failure and successful resolution separately. Do not reset the whole lab to the instructor answer to avoid the conflict. The local verifier's final-content check supplements the required reasoning about how your decision reconciled the two proposals.

```
git status
git show --format=fuller --stat HEAD
git rev-list --parents -n 1 HEAD
```

Phase contents · Reasoning key

Restore a mistake and explain the final history

Concept

Restoring a working file replaces an uncommitted version with a known snapshot. That can safely remove an intentional teaching mistake after all useful work is committed, but it would discard wanted uncommitted edits if used without inspection. Reverting a committed change is different: it creates another historical decision rather than simply changing the current file. Choosing the operation depends on what state contains the mistake.

The final exercise adds accidental text after the completed ownership merge. Inspecting its diff establishes that only this new line is unwanted. Restoring that file should leave every committed deadline, escalation and ownership decision intact. A clean status then means working/index content agrees with history; it does not mean the contract is correct by itself. The committed file and complete prior evidence provide that additional check.

Challenge

After all intended contract work is committed, append `Accidental text` to `contract.md`. Inspect the working diff and confirm no wanted edit is mixed with it. Use `git restore contract.md`, then compare the working file with the current committed snapshot.

Confirm the deadline, escalation and resolved owner survived. Write a short explanation of the working tree, index, branching and ownership decision. Keep the evidence outside the inner repository so the final clean-state check does not require deleting it. Explain why restoring this uncommitted mistake differs from reversing a published commit.

Inspect the accidental diff before restoring a named file

Hint

Do this only inside your disposable lab, after all intended changes were committed.
Identify where the mistake lives before choosing a recovery operation. If a useful edit is still uncommitted, preserve it first rather than using a broad reset. Restore the named contract file, not every file in the attempt. Compare the current committed snapshot afterward; a green status is meaningful only alongside proof that the wanted requirements still exist.

Check

The disposable lab status must be clean and its contract must match the current committed snapshot. Verify that the explicit deadline, escalation and resolved owner remain, while the accidental line is absent. Retain the before/after working diff and explain why this operation did not delete the earlier commits.

Run the root phase verifier on the preserved attempt, then distinguish its behavioral result from the instructor's review of your explanation. A clean empty repository or a copied answer does not satisfy the recorded history sequence. State why restoring an uncommitted mistake differs from reversing an already published commit. No remote action is needed for this foundation outcome, and entering advanced practice should use a fresh independently prepared attempt rather than destroy this evidence.

```
git status --porcelain
git show HEAD:contract.md
```

Phase contents · Reasoning key

Phase completion

Show the final contract, a clean disposable lab repository and a history that explains intentional commits, compatible branch work and the ownership-conflict decision. Your contract must retain the explicit 09:00 UTC delivery promise and escalation text. Keep a record of the staged-only first commit, separate deadline/escalation commits, both branch changes and the conflict resolution's parents. The final restoration should remove only the accidental uncommitted text while preserving every committed requirement.

Store evidence outside the inner lab repository, beside the copied starter, so a clean Git state does not require deleting the explanation. The local verifier inspects your actual copied repository and contract rather than substituting a fresh reference workflow. Its behavioral checks supplement the instructor's review of your explanations; a clean status alone cannot prove all intended edits survived. Phase 1 needs no remote push or peer review. Preserve this attempt when entering the independently prepared collaboration phase, and keep any optional rebase/reflog experiment separate from the minimum outcome.

Optional stretch

The core completion standard is independent of optional work. No additional stretch is required in this phase; the next phase has an independently prepared entry baseline.

git phase reasoning keys

Reasoning keys

Read these explanations after attempting the checks.

Reasoning: Inspect the three working states

The first commit should contain the baseline contract alone. A scratch note can be visible and untracked without being in that snapshot. Status, staged diff and committed file list describe different states. Explicit file staging protects unrelated work, and checking the repository root prevents editing the outer course checkout. Evidence lives outside the lab so the final clean-state check can coexist with a complete explanation.

Return to Inspect the three working states

Reasoning: Commit an intentional contract change

The deadline and escalation changes have distinct purposes and should appear in separate diffs. A message alone cannot establish that scope. Staged inspection protects unrelated working edits, and the final contract retains its original grain/currency facts. Keeping hashes with their content lets a reviewer reconstruct the decisions rather than trusting a terminal success message.

Return to Commit an intentional contract change

Reasoning: Branch and combine compatible work

The branch graph explains ancestry and whether a fast-forward or merge occurred. The final contract proves both intended facts survived. Compatibility must be established from actual separated edits; the tiny original file can produce overlap despite different branch names. Keep accidental overlap distinct from the later planned ownership conflict, and preserve the prior deadline/escalation work.

Return to Branch and combine compatible work

Reasoning: Resolve an ownership conflict

Git exposes competing owner edits but cannot decide the correct responsibility contract. The learner resolves the requirements, retains other facts and completes the staged merge. Both parents explain combined history; the diff explains the actual choice. An unrelated setup failure is not the intended conflict, and removing markers alone does not prove a coherent or committed result.

Return to Resolve an ownership conflict

Reasoning: Restore a mistake and explain the final history

The accidental line is uncommitted, so named-file restoration can remove it after verifying that no useful edit is mixed in. The earlier commit history remains intact. Final cleanliness must be paired with retained contract requirements and before/after evidence. Reverting a committed or shared decision would be a different historical operation. Keep the explanation outside the inner lab so the clean-state check and complete evidence can both hold.

Return to Restore a mistake and explain the final history

git: Phase 2 — advanced practice

Purpose and outcome

Carry a small contract clarification through a collaboration workflow. This phase extends local Git into a designated remote repository, an understandable pull request, an actual review response and synchronization with concurrent upstream work. The challenge is preserving intent and review evidence across participants, not merely making a push command succeed. The advanced route has an independent contract baseline and a supplied review scenario, so you do not need another learner's foundation repository.

Dataset and pipeline

The fixture is the same style of holdings contract used in local practice. A delivery-timezone clarification and escalation responsibility form the proposed change. The review scenario asks for an explanation and revision, while a separate upstream edit adds input classification. These requirements must coexist in the final branch. The hosted PR contains the proposal and review state; local history contains your commits and integrated ancestry. They describe related but distinct collaboration evidence.

Prerequisites and workspace

The instructor designates an authorized private training repository, branch prefix and reviewer. Use the delivered Git and GitHub CLI tools with the intended training identity. Prepare an independent attempt through mise, then clone only the designated sandbox into its inner lab directory. Keep the starter contract and evidence at the attempt root. Confirm remote URLs and repository root before any remote write; the course repository and production remotes are outside this exercise.

```
mise run challenge:v2:prepare git phase-two-proficient
cd .local/challenges-v2/git/phase-two-proficient
gh auth status
```

Account creation, sign-in, reviewer invitation and repository provisioning are arranged preparation, not automated assessment. If a real reviewer or remote is unavailable, use the supplied scenario locally with explicit simulated status and leave the corresponding collaboration gate pending. Do not count a Markdown draft as an actual PR or a self-comment as peer review.

Starter behavior

The contract and review-scenario JSON are readable, benign inputs. There is no real remote state inside the copied starter. Phase verification requires your actual cloned repository and, for live review checks, the designated sandbox/PR identifiers. A local reference workflow can demonstrate Git mechanics but does not satisfy hosted review. The final merge decision follows the instructor's stated policy; an open PR may be the honest final state. Never rewrite the route to imply approval just because a check command exited successfully.

Timeline and completion standard

Core reading, authoring, checking and explanation: 75–110 minutes. Allow additional time for setup and optional stretch.

- Task: Clone and inspect a collaboration boundary · Minutes: 15–22 · Evidence: Designated private sandbox, correct clone root, remote and tracking branch
- Task: Push and propose a scoped pull request · Minutes: 15–22 · Evidence: PR URL/head hash and remote diff contain the intended deadline clarification
- Task: Respond to a review with a second commit · Minutes: 15–22 · Evidence: Other reviewer/comment, second commit and remotely visible response
- Task: Integrate concurrent upstream work · Minutes: 15–22 · Evidence: Upstream ancestor and retained classification/deadline/escalation facts

- Task: Read the final review and merge state · Minutes: 15–22 · Evidence: Actual final state, current head and next authorized action

Independent starter files Read the owning phase before editing. Prepare from the training project root with `mise run challenge:v2:prepare git phase-two-proficient`; edit the copied attempt.

Authored-file list

Task	*Learner-authored paths*
Clone and inspect a collaboration boundary, Push and propose a scoped pull request, Respond to a review with a second commit, Integrate concurrent upstream work, Read the final review and merge state	`lab/contract.md`, `evidence.md`

Declared baseline

The supplied contract is the starting point. Create the inner Git repository under `lab/` and keep `evidence.md` outside it, beside the copied starter. The advanced phase also supplies a review scenario; the training remote and reviewer are arranged separately.

Use a fresh workspace for another attempt; preparation and reset preserve existing work.

Core challenges

Clone and inspect a collaboration boundary

Concept

A remote URL identifies where commits can be fetched or pushed, while a tracking relationship tells Git which remote branch a local branch follows. Neither proves that the signed-in hosting identity has the intended rights. The clone's default branch can also differ from the proposal branch. Those boundaries matter before the first push: a correct contract edit sent to the wrong repository is still the wrong collaboration result.

The independent starter gives content and review context, not remote approval. Inspecting identity, remote URLs, local branch and proposed PR base makes the collaboration destination reviewable. Fetch updates remote-tracking information without automatically rewriting your branch. A private sandbox protects the teaching scope, and a local simulation must retain that label even if its history closely resembles a hosted workflow.

Challenge

Clone the instructor-designated private training repository into a fresh `lab/` directory inside your attempt. Confirm the permitted branch prefix and reviewer before beginning the proposal. If you need a personal training sandbox, arrange that setup with the instructor first.

Inspect the clone root, `git remote -v`, `git branch -vv` and `gh auth status`. Record the training identity, intended PR base and remote destination in the parent evidence note without credential values. Use the supplied contract baseline where needed. If the remote or reviewer is unavailable, practice locally with the scenario labelled simulated.

Verify the destination before writing a remote branch

Hint

Use an existing authorized training identity. Do not copy tokens into evidence. If `gh repo create` reports that the name exists, you already have a sandbox: clone it with `gh repo clone`.

Compare the repository identity shown by the hosting CLI with the clone's remote URL. A valid login can still target another account or repository. Read branch tracking separately from the current branch name. Keep tokens outside evidence; the safe identity, intended base and remote destination are enough to establish the collaboration boundary.

Check

Inspect and record the designated remote identity, inner clone root, local branch, tracking branch and intended PR base. The repository must be the authorized private training sandbox. Explain what fetch can update and what still requires merge or rebase into your local proposal. A successful clone alone does not establish the reviewer's availability or your final branch permissions.

Retain redacted identity/context observations without credentials. Distinguish real hosted state from a local scenario fixture. If no authorized remote exists, mark the live collaboration check pending and continue the local preparation. Never substitute a course or production remote merely because its URL is already configured.

```
git remote -v
git branch -vv
git rev-parse --show-toplevel
```

Phase contents · Reasoning key

Push and propose a scoped pull request

Concept

A push makes branch commits available remotely; a pull request proposes how they should enter another branch. The PR description explains why the contract change matters and how its result was checked, while the diff shows the actual scope. A branch can exist without a PR, and a well-written local description can exist without either. The assessment therefore needs actual remote identity and head commit.

The deadline clarification should name time zone and expected consumer behavior without deleting grain or currency promises. A reviewer who did not attend the terminal session should understand the before/after requirement from the PR. Keeping the proposal on a feature branch preserves the sandbox main baseline and gives the later review response an identifiable place to land.

Challenge

Create a feature branch using the permitted prefix and clarify the delivery deadline/timezone in `contract.md`. Review the diff, commit it, and push the sandbox branch with an upstream. Use `gh pr create` against the intended base.

Describe the before/after consumer expectation and how you checked that grain, currency and owner facts survived. Inspect the hosted PR diff and head commit, then save its URL, branch and hash in the parent evidence note. Keep the proposal limited to this contract change. Compare local HEAD with the PR head before handing it to the reviewer.

> Connect the proposed behavior to the actual hosted diff
>
> **Hint**
> Check gh pr create `--help` and gh pr view `--help` on the installed version.
> Inspect the installed CLI help for proposal and viewing commands, then compare the returned PR base/head with your recorded boundary. A useful description says what changed for the consumer and what stayed valid in the contract. Avoid including every terminal operation or unrelated setup file in the proposal diff.

Check

The hosted PR must reference the intended private sandbox, base branch and proposal head. Its diff contains only the scoped contract clarification and preserves original grain/currency facts. Compare local HEAD with the remote PR head instead of retaining a URL detached from the current revision. Explain the separate roles of commit, push and PR.

A local body file, screenshots of a draft or a successful push without a PR do not satisfy this outcome. If remote access is unavailable, retain a reviewable draft and mark hosted creation pending. Keep all remote work in the designated training sandbox.

```
git show --stat HEAD
gh pr view --json url,baseRefName,headRefName,headRefOid
```

Phase contents · Reasoning key

Respond to a review with a second commit

Concept

Review supplies another participant's reasoning about the proposed behavior. A revision is useful when the new commit addresses a concrete comment and the response explains how. Merely adding any second commit can satisfy a numerical count while leaving the requirement unanswered. The supplied scenario concerns deadline time zone and late-feed responsibility, so the change should make those facts clearer without introducing unrelated policy.

A real peer review differs from a learner playing both sides of a local simulation. Both can teach revision mechanics, but only the former establishes the collaboration boundary required here. The reviewer/comment identity, revision hash and current hosted diff let another person trace that conversation. Approval is also separate from a comment or a pushed response; report it only when the actual review state supports it.

Challenge

Ask the arranged instructor or paired reviewer to apply `review-scenario.json` to the sandbox PR. Read the actual comment and identify the unclear deadline or escalation responsibility. Clarify that requirement in `contract.md`, commit the revision and push the permitted branch.

Explain how the new diff addresses the comment. Retain the comment/reviewer identity, revision hash and remotely visible response in the parent note. Preserve the original proposal so the reviewer can compare it with the revision. A second commit must answer the review requirement; adding an unrelated change does not fulfill this task.

Tie the revision to one actual review requirement

Hint

The reviewer may use the prepared scenario in review-scenario.json. Instructor preparation includes arranging the reviewer.

Quote or summarize the specific requirement in your evidence note and compare it with the revision diff. A second commit should visibly address that requirement. Keep the reviewer arrangement as instructor preparation and do not assume a self-comment establishes another person's review. Inspect the hosted head after the response so an unpushed local edit cannot masquerade as a reviewed revision.

Check

The review evidence must identify another participant, the relevant comment and the remotely visible revision commit. Compare the new hosted diff with the comment's requirement and confirm deadline and escalation facts are now explicit. Retain the original proposal and response identities rather than replacing all evidence with the final text.

Explain why a commit count alone does not establish a useful response or approval. A simulated review needs that label and cannot close the real-human gate. Use the live verifier to inspect the existing PR and review state; record a missing reviewer or access prerequisite.

```
gh pr view --json reviews,commits,headRefOid
git show --stat HEAD
```

Phase contents · Reasoning key

Integrate concurrent upstream work

Concept

An open proposal can become stale while another participant changes its base branch. Fetch updates your information about that upstream history; merge or rebase decides how it enters your proposal. Pull combines these operations according to configuration, which can conceal the decision if used without inspection. The final review should include both your clarification and the shared input-classification requirement.

The supplied concurrent edit adds internal input classification on main. A second clone can demonstrate the mechanics when that role is explicitly simulated, but the handoff should say who actually supplied the edit. A merge conflict requires the same requirement-preservation reasoning as foundation practice. An ancestor check proves history includes the upstream commit, while the final contract proves its content and your proposal both survived.

Challenge

Have the instructor supply the planned main-branch edit, `Input classification: internal`. A second clone may play that role for a labelled local simulation. Fetch origin and compare your feature branch with `origin/main` before integrating.

Follow the instructor's merge/rebase policy, preserving classification, deadline and escalation through any conflict. Inspect the complete contract and diff before pushing the permitted proposal branch. Save the upstream hash, integration result and final head. Explain which operation updated remote-tracking information and which incorporated the change into your proposal. If rebase changes hashes, follow the sandbox's remote-update policy.

Fetch informs the decision; integration changes your proposal

Hint

Use git log `--left-right --oneline` HEAD...origin/main before combining history.
Compare left/right history before integrating so you can identify which commits belong to each side. An updated remote-tracking branch does not automatically place its commit in your local proposal. After conflict repair, inspect the whole contract; an ancestor alone does not prove that a later edit preserved the classification text.

Check

The intended upstream commit must be an ancestor of the final proposal according to the chosen history policy, and the contract must retain the input classification alongside your reviewed deadline/escalation changes. Inspect the hosted PR head after the update and compare it with local HEAD. Explain what fetch changed and what integration changed.

Keep the original upstream identity and final integration evidence. If rebase rewrites proposal hashes, state that effect and follow the classroom remote-update policy rather than applying a broad force push. No course or production branch is part of the exercise. A local second-clone simulation remains clearly labeled, while actual remote concurrency evidence requires the designated sandbox.

```
git log --left-right --oneline HEAD...origin/main
git merge-base --is-ancestor "$DC_UPSTREAM_COMMIT" HEAD
```

Phase contents · Reasoning key

Read the final review and merge state

Concept

A proposal's state reflects review, checks and branch policy, not just the last command exit. A green local workflow can coexist with an open PR awaiting review, and a review comment can coexist with no approval. The learner should identify the exact current state and its next authorized action. Some training sandboxes intentionally stop before merge so the review record remains inspectable.

Synchronizing local main after an authorized merge is another action, distinct from declaring that a proposal is ready. If a gate remains blocked, a useful handoff names it rather than manufacturing a merge hash. The actual head, review outcome, check state and retained contract facts let another participant assess completeness. This classroom evidence does not authorize merging production work or publishing the course repository.

Challenge

Inspect `gh pr view` and `gh pr checks` after the revision and upstream integration. State the current review outcome, checks, head commit and whether the PR is open, blocked or merged. Identify the next action under the instructor's merge policy.

Merge only when that sandbox policy permits it. After an authorized merge, synchronize local main and inspect the resulting contract; otherwise retain the open PR state. Write a short handoff naming the final branch, preserved requirements and remaining decision or access step. A successful local command does not establish another person's approval.

Report observed policy state instead of assuming approval

Hint

A real reviewer and repository permissions are prerequisites; missing access is a blocked live result.
Read the actual review and check fields rather than infer readiness from a green local command. A check command can return nonzero because checks are pending or absent, so inspect its meaning in the installed tool. Compare PR head to your current branch before summarizing. An honest open-state handoff is better evidence than an invented merge or a guessed approval.

Check

The handoff must match the current PR state, review outcome, checks and head commit in the authorized private sandbox. Deadline, escalation and concurrent input classification remain in the proposed or merged contract. Explain any pending gate and the next authorized action. If an actual merge occurred, identify its resulting base history; if it did not, preserve the open state explicitly.

Run read-only live verification with the designated repository/PR environment variables when available. It checks current remote evidence and another account's review; offline local history remains supporting for this phase. Do not claim peer approval from a self-comment or merge eligibility from a generic command success. No remote write is performed by the verifier.

```
gh pr view --json state,reviews,headRefOid
gh pr checks
```

Phase contents · Reasoning key

Phase completion

Present the actual authorized sandbox PR, its scoped contract diff, review/comment identity, response commit, integrated upstream change and honest final review/merge state. Your local branch must include the supplied concurrent input-classification requirement as well as the explicit deadline and escalation clarification. Show the relationship between local HEAD and the current remote PR head. A simulated local workflow can support practice but cannot establish a real peer review or remote collaboration outcome.

Keep the copied attempt and evidence note, using only the designated private training repository for any authorized push or PR. Do not publish the course checkout or assume the exercise authorizes production merging. The verifier reads existing sandbox evidence; it does not create a repository, push, invite a reviewer or merge automatically. A human reviewer and the instructor's merge policy remain real prerequisites. Report an open or blocked PR truthfully instead of inventing approval. Your handoff should let another participant understand which branch contains the final requirements and which decision or access step remains pending.

Optional stretch

The core completion standard is independent of optional work. Choose a separately timed route from the stretch guide.

git phase reasoning keys

Reasoning keys

Read these explanations after attempting the checks.

Reasoning: Clone and inspect a collaboration boundary

The remote identifies where commits go; tracking identifies the remote branch relationship; hosting identity supplies permissions. All must agree with the designated sandbox before a write. Clone and fetch are prerequisites, not evidence of a PR or peer review. Keep a local simulation explicitly separate and record missing human/access preparation rather than silently choosing another repository.

Return to Clone and inspect a collaboration boundary

Reasoning: Push and propose a scoped pull request

The branch commits and hosted PR are related but distinct artifacts. The actual base/head and diff must match the designated sandbox and current proposal. The description explains consumer behavior and verification without replacing the diff. A local draft remains useful supporting material but cannot be presented as a remotely created PR.

Return to Push and propose a scoped pull request

Reasoning: Respond to a review with a second commit

The second commit should explain and implement the requested clarification. Its remote head and diff establish that the response is available for review; another participant's comment establishes peer involvement. A self-simulation remains supporting practice. Comment, response and approval are different states, so none should be inferred simply from two commits.

Return to Respond to a review with a second commit

Reasoning: Integrate concurrent upstream work

Fetch obtains upstream history but does not decide how your proposal combines it. The chosen integration must retain both concurrent classification and the reviewed clarification. An ancestry check and final diff protect different parts of that result. If the concurrent role or remote action was simulated, record it honestly instead of treating local mechanics as actual hosted collaboration.

Return to Integrate concurrent upstream work

Reasoning: Read the final review and merge state

The current hosted review/check/head state controls the handoff. Open or blocked states must remain visible when the sandbox policy stops there. Local mechanics and a second commit do not establish approval. The read-only verifier can inspect existing evidence, while reviewer participation and merge authorization remain separate classroom decisions. Preserve contract facts and identify the next action without inventing a merge.

Return to Read the final review and merge state

git optional stretch

Choose an optional exercise after the advanced core. Allow 20–40 minutes for each and use a fresh copy of your completed baseline. Keep the assessed attempt unchanged.

Optional exercises

Compare merge and rebase on disposable local history

Concept

Merge and rebase can combine related work while presenting different ancestry. A merge retains the divergent histories and can record a commit with both parents. Rebase reapplies commits onto another base, changing their identities even when the resulting contract content is similar. Neither automatically decides the right business requirement, and rebasing already shared work introduces a coordination question beyond this local comparison.

Use a disposable local copy with a known common baseline and two nonconflicting contract edits. The comparison should show final content, parent relationships and rewritten hashes rather than declare one graph universally better. The permitted sandbox review policy still governs the actual advanced collaboration route. Keeping the experiment separate prevents a history-style exercise from rewriting someone else's shared proposal.

Challenge

Create two independent local copies of a completed contract fixture. In one, merge the prepared upstream requirement into the feature branch; in the other, rebase the equivalent feature commits onto

that upstream baseline. Inspect both graphs and final contract projections. Record which hashes changed and which business facts stayed identical. Do not push either comparison or rewrite a shared remote branch. Explain when a team might prefer each history presentation and what additional coordination shared rebasing would require.

Record the shared baseline and the two separate final projections before choosing a preferred explanation of their history.

> Compare content separately from history
>
> **Hint**
> Choose genuinely separated edits so this comparison focuses on ancestry rather than another conflict exercise. Capture commit IDs before rebase; otherwise a claim that identities changed cannot be checked. Compare the final contract content separately from graph shape. If an unexpected conflict appears, record and resolve it intentionally rather than delete one requirement to obtain a tidy graph.

Check

Inspect the merge parents, rebased ancestry, original/new feature hashes and final file content. Both variants must preserve the same contract requirements, while the rebase variant should show rewritten feature identities. Retain commands and observations from the two disposable repositories and explain which conclusion follows from each projection.

A cleaner-looking graph is not a correctness test, and unchanged file content does not mean commit identities were preserved. No remote operation is necessary or authorized by this optional comparison. Keep the actual advanced PR untouched. If your history did not diverge, prepare a meaningful shared baseline before claiming the comparison. This is separately timed depth after core completion, not a substitute for real review evidence.

Optional contents · Reasoning key

Recover a lost local branch reference through reflog

Concept

A branch is a reference into existing commit history. Moving or deleting a local reference can make a useful commit less visible without immediately destroying the underlying object. Reflog records recent reference movements and can help recover the intended local state. Its retention is bounded by repository behavior and garbage collection, so its bounded reference record cannot replace durable commits and a separately preserved backup.

The exercise uses only disposable committed work. A reflog entry can identify an earlier commit, but restoring that pointer does not automatically recover an uncommitted file that was discarded. Keep those recovery cases distinct. The foundation restore task handled an accidental uncommitted edit; this optional experiment concerns a committed change whose branch reference moved.

A recovered pointer still needs its committed file content checked.

Challenge

In a new disposable local copy, create `recovery-demo`, commit a distinctive contract note and record its hash. With that commit at the branch tip and no uncommitted edits, move only `recovery-demo` back one commit using `git reset --keep HEAD~1`. This intentionally removes the committed note from the working file; `--keep` refuses a change that conflicts with uncommitted edits. Inspect `git reflog` to locate the saved commit, then use `git switch -c recovered-note` with that hash to create a recovery branch at it. Compare recovered file content and history with the saved expected hash. Keep the production/course repository and actual advanced sandbox untouched, and preserve the original completed attempt before starting.

Keep the expected commit identity in the parent evidence note and inspect the recovered branch without changing another reference.

> Save the expected commit before moving a reference
>
> **Hint**
> Write down which reference you intend to move and where the useful commit will remain observable. Use the saved hash as an independent recovery expectation rather than picking a plausible-looking reflog line. Reflog records reference activity, not every editor change. Prefer creating a recovery branch over deleting or rewriting more history while diagnosing the lost reference.

Check

The recovery branch must point to the intended committed note and retain its expected content. Inspect reflog entry, saved hash, recovery branch and file projection, explaining which operation moved a reference and which operation made the commit reachable again. The original completed core attempt must remain unchanged.

A branch with the same name but different content fails the recovery outcome. Do not claim that this technique recovers every discarded working edit or provides permanent retention. Keep remote actions out of the experiment and record its separate timing. The result should teach a bounded recovery mechanism after core completion, with any unsupported tool behavior recorded rather than worked around through broad resets.

Optional contents · Reasoning key

git phase reasoning keys

> **Reasoning keys**
> Merge preserves the original feature commits and combines divergent parents. Rebase creates new feature commits on the updated base, so their hashes change. Both histories can preserve the same contract text. Compare ancestry and final requirements separately; shared-branch policy determines whether rewritten history is appropriate for a team.
> Return to Compare merge and rebase on disposable local history
> A recovery branch makes the recorded commit reachable through a name again. The saved hash and committed note verify that you recovered the intended state. Reflog can help locate a moved reference, but its entries expire and it does not record every discarded working edit. Keep a durable backup for work that must survive.
> Return to Recover a lost local branch reference through reflog

Docker and runtime boundaries

Projects contents · Task index

- Foundation core — page 269
- Advanced core — page 277

docker challenges

Docker packages a supplied six-holding summary app so its execution and filesystem boundary can be inspected. Foundation work develops image/container lifecycle, shell isolation, read-only mounts and simple packaging. Advanced work separates named test/runtime targets, verifies actual non-root identity and focused contents, and diagnoses a path assumption. The app is supplied so authoring focuses on runtime behavior. Its small fixture is separate from the later Databricks datasets.

Before you start

Use the delivered client with the prepared training engine and pulled base image. Prepare a copied attempt through mise, choose unique owned names/tags and work from its root so the build context is clear. Mount only the chosen app directory when that task calls for a mount. Engine access and image downloads are preparation, not packaging evidence. The local Python oracle remains supporting when Docker is unavailable. Never use broad pruning, privileged execution or another learner's resource as a shortcut around a path or name collision.

Suggested timeline and tasks

Estimates include reading, practice, checks and explanation. Allow additional time for environment setup and optional extensions.

- Phase: Foundation · Task: Run an image and inspect the lifecycle · Minutes: 12–18
- Phase: Foundation · Task: Explore an interactive shell · Minutes: 12–18
- Phase: Foundation · Task: Use a bounded bind mount · Minutes: 12–18
- Phase: Foundation · Task: Author and run a simple holdings image · Minutes: 12–18
- Phase: Foundation · Task: Prove repeatability and clean scoped resources · Minutes: 12–18
- Phase: Advanced · Task: Separate named build targets · Minutes: 15–22
- Phase: Advanced · Task: Gate the runtime on a tested artifact · Minutes: 15–22
- Phase: Advanced · Task: Run with a bounded runtime user · Minutes: 15–22
- Phase: Advanced · Task: Keep runtime contents focused · Minutes: 15–22
- Phase: Advanced · Task: Repair a path assumption and explain the result · Minutes: 15–22

Choose your phase

- Foundation: Foundation core with explicit completion evidence.
- Advanced: Independent advanced baseline and finite core, separately timed.

Finish the core tasks before choosing optional stretch.

Teaching concepts

Image identity, process lifecycle, writable layer, mount path and working directory each explain a different observation. Build success does not establish runtime output, a USER line does not establish actual process identity, and a stage named test does not establish an enforced oracle. Pair artifact inspection with the independent six-holding result. The phase reasoning keys remain after the work, while full reference Dockerfiles are separate instructor material. Optional cache/size analysis does not replace the finite core.

docker: Phase 1 — foundations

Purpose and outcome

Package and run a tiny portfolio-summary application while learning image/container boundaries. The app is supplied so your authoring focuses on Docker rather than rewriting business logic. You inspect process lifecycle and filesystem isolation, use a controlled mount, write a simple Dockerfile and prove the packaged app's output.

Dataset and pipeline

The fixture under app contains six holdings and a standard-library summary script. Its total market value is 12,011,626.50. This small Docker fixture is distinct from the eight-row supplied Databricks Portfolio Analytics case; do not present its output as that pipeline's completion. The app reads a file relative to its working directory, so runtime path and copied input matter. A supplied Python test independently checks the fixture/application result. A container can exit normally after printing the summary because its main process has finished.

Prerequisites and workspace

Use the delivered Docker client with access to the prepared training engine, Python for the optional local app check, and a pulled base image. Prepare a copied attempt through the version-explicit mise task, then work from its root. Choose a unique container name and image tag so two learners do not collide. The engine and image-pull boundary should be ready before timed practice; source presence alone does not establish either.

```
mise run challenge:v2:prepare docker phase-one-certified
cd .local/challenges-v2/docker/phase-one-certified
export DC_DOCKER_NAME=dc_alex_docker01
export DC_DOCKER_TAG=dc-alex-docker01:practice
docker version
```

Keep commands inside this attempt and mount only its chosen app directory, using an absolute host path. Do not mount the host root, create privileged containers or remove shared images to repair a path mistake. If an interactive terminal is unavailable, use a documented noninteractive inspection command and label that variation. A new attempt gets a fresh workspace and name; reset does not delete existing work.

Starter behavior

The supplied Dockerfile is a benign buildable placeholder that prints its unfinished status and exits successfully. That behavior proves a starter can launch, not that you packaged the app. The Python summary and test work outside Docker from the app directory. Replace the Dockerfile's placeholder with the named authoring outcome before expecting completion verification to pass. Read-only mount writes intentionally fail later; an unavailable engine or image download is a prerequisite failure, not that teaching drill. Keep those failure categories separate in evidence.

Timeline and completion standard

Core reading, authoring, checking and explanation: 60–90 minutes. Allow additional time for setup and optional stretch.

- Task: Run an image and inspect the lifecycle · Minutes: 12–18 · Evidence: Hello output, actual image/container identity and exit status
- Task: Explore an interactive shell · Minutes: 12–18 · Evidence: Inside file disappears in a fresh disposable execution; version/OS evidence
- Task: Use a bounded bind mount · Minutes: 12–18 · Evidence: Actual fixture summary and expected write denial; host input unchanged
- Task: Author and run a simple holdings image · Minutes: 12–18 · Evidence: Authored image emits independent six-holding result

- Task: Prove repeatability and clean scoped resources · Minutes: 12–18 · Evidence: Two matching outputs from one image identity; only owned resources removed

Independent starter files Read the owning phase before editing. Prepare from the training project root with `mise run challenge:v2:prepare docker phase-one-certified`; edit the copied attempt.

Authored-file list

Task	*Learner-authored paths*
Run an image and inspect the lifecycle, Explore an interactive shell, Use a bounded bind mount, Prove repeatability and clean scoped resources	`evidence.md`
Author and run a simple holdings image	`Dockerfile`, `evidence.md`

Declared baseline

The app and independent Python test are supplied. The Dockerfile builds a placeholder image; replace its unfinished configuration to package the app. A successful placeholder build is not completion.

Use a fresh workspace for another attempt; preparation and reset preserve existing work.

Core challenges

Run an image and inspect the lifecycle

Concept

A container's main process determines its lifetime. A short program that prints and exits can be a successful container execution; keeping it alive with an unrelated sleep would change the behavior being inspected. The image is a reusable package, while the container is one process execution with its own writable layer and recorded state. Removing that container does not automatically delete the image.

The hello command makes this distinction visible before app packaging. Inspecting all containers includes stopped executions that a default running-only list omits. Logs and exit status answer whether the process ran as expected, while the image identifier answers which package ran. A familiar tag can point to a different image later, so retain the actual identifier when comparing runs.

Challenge

Run `docker version` and `docker info`, then launch the hello process using your unique container name:

```
docker run --name "$DC_DOCKER_NAME" python:3.11-slim python -c "print('hello holdings')"
```

Inspect its logs, state, exit code and actual image identifier. Explain why successful output leaves a stopped container. Retain those observations, then remove only this named execution. If the engine or base image is unavailable, resolve that preparation issue before interpreting program behavior. Do not keep the hello process alive with a sleep: its normal completion is the lifecycle observation.

A successful short process can leave a stopped container

Hint

Use docker ps -a to include stopped containers. Instructor preparation supplies Docker and a pulled base image.

Use the stopped-container view and inspect the named execution rather than searching for any green-looking process. The expected hello output comes from its main command, not an image build. Keep removal separate from image cleanup so you can explain which state disappears. A name already in use means choose or inspect your owned attempt, not delete an unknown container.

Check

Expect the named execution's logs to contain the declared hello output and its process exit status to be zero. Retain container and image identifiers and explain the stopped state. Removing the execution should leave the reusable image available; inspect that distinction rather than infer it from a command message.

A printed client version alone does not prove engine access or process execution. If a preexisting name prevents launch, use a unique owned name and record the collision instead of removing someone else's container. This lifecycle observation is separate from the final six-holding app result and cannot be counted as completed packaging.

```
docker ps -a
docker logs "$DC_DOCKER_NAME"
docker inspect "$DC_DOCKER_NAME"
```

Phase contents · Reasoning key

Explore an interactive shell

Concept

A container writable layer belongs to that execution. Creating a temporary file inside one disposable container does not place it in the base image or the host application folder. A fresh execution of the same image starts from its packaged filesystem and a new writable layer. This is why an unmounted scratch file disappears after the first container is removed.

An interactive shell changes how you inspect the process, not whether it shares the host kernel. The terminal flags attach input and a pseudo-terminal; a noninteractive command can inspect the same filesystem facts when the classroom interface lacks a terminal. Keep the image's OS/runtime observations distinct from the author's host environment. A container is not automatically a virtual machine with a separate kernel.

Challenge

Start a disposable shell with `docker run --rm -it python:3.11-slim bash`. Inspect `pwd`, Python's version and `/etc/os-release`. Create `/tmp/inside.txt`, exit, then launch a fresh container from the same image and test whether the file exists.

Record both execution identities and observations. If an interactive terminal is unavailable, use a noninteractive command for the same creation/removal experiment and explain that choice. Keep both executions unmounted and leave the shared base image unchanged. A volume or a committed replacement image would introduce persistence and change the question being tested.

> Compare fresh executions without adding persistence
>
> **Hint**
>
> The terminal flags attach standard input and a TTY; `--rm` removes the exited container.
> Compare the two executions' identities and filesystem observations. The first file was created after launch, so it belongs to that writable layer. Do not commit the container into another image or add a volume during this experiment; either would change the persistence contract. A missing interactive terminal should change the inspection method, not the expected filesystem result.

Check

The second fresh execution should not contain the temporary file from the first removed container. Retain the first successful creation and second absence check with Python/OS observations. Explain which filesystem content came from the image and which was created at runtime. An accidentally reused container would not test the same claim.

Record the interactive or noninteractive method and why it was used. Do not describe the outcome as a separate-kernel virtual machine demonstration. The observation establishes disposable writable-layer behavior, not every possible persistence mechanism; the next mount task deliberately changes that boundary.

```
docker run --rm python:3.11-slim python -c "from pathlib import Path;
 ↪  print(Path('/tmp/inside.txt').exists())"
```

Phase contents · Reasoning key

Use a bounded bind mount

Concept

A bind mount exposes a selected host path at a container path. The target can replace what the image would otherwise show there, so both path and access mode matter. Read-only access lets the supplied summary read its fixture while protecting that copied input from container writes. Mounting an entire workspace would broaden the boundary beyond what this task needs.

The app reads its CSV relative to the working directory. A correct mount at one path with execution in another can still fail, even though the file exists somewhere inside the container. Comparing the host and mounted input plus the app output distinguishes this path error from a business-data defect. The deliberate write denial demonstrates the mount promise and should not be repaired by removing read-only protection.

Challenge

From the copied starter, run the supplied summary with `app/` mounted read-only at `/app` and `/app` as the working directory. Use an absolute host path. Compare the copied input with what the container reads.

Attempt a container-side write to the mounted input directory, retain the expected denial and verify the host copy did not change. Explain the source path, target path and access mode separately. If the summary cannot find its CSV, inspect the working directory before changing permissions. Keep the mount limited to the app directory rather than broadening it to the whole workspace.

> Align the mount target and app working directory
>
> **Hint**
>
> Use an absolute host path from pwd and the :ro mount suffix. Do not mount the entire host root.
> Check source path, container target and working directory as three separate values. The read-only suffix belongs to the mount declaration; do not assume the app itself protects the input. If summary cannot find its CSV, inspect the current directory before weakening permissions or mounting more host content. The write should fail while the read remains successful.

Check

The mounted app run must produce six holdings with total market value 12,011,626.50, matching the local fixture oracle. The controlled container-side write must fail for the read-only mount reason, and the host input hash/content must remain unchanged. Retain command, exit categories and output rather than only an image of the mounted folder.

Explain which files come from the image and which from the host mount, and why the current directory affects this app's relative CSV path. An engine or image-pull failure does not count as the intended write-denial drill. Do not remove the protection to make the negative command green.

```
docker run --rm -v "$PWD/app:/app:ro" -w /app python:3.11-slim python summarize.py
```

Phase contents · Reasoning key

Author and run a simple holdings image

Concept

An image build records filesystem content and startup configuration. Copying the supplied app into an image and selecting its working directory makes runtime independent of a host bind mount. The build context limits which files are available to copy, and the startup command determines which process runs. A benign placeholder Dockerfile can build successfully while still lacking the assessed application.

WORKDIR sets the directory for subsequent relative paths, COPY adds build-context files, and CMD selects the default process. The app's path assumption means copied files and working directory must agree. Packaging the test alone or printing the expected text would bypass the application behavior. The independently supplied test and fixture output make that distinction reviewable. Build success establishes that instructions could be executed; only running the resulting image establishes its main process and actual app result.

Challenge

Replace the placeholder Dockerfile with a `python:3.11-slim` image, working directory `/app`, copied app files and default `summarize.py` command. Use your attempt root as the build context and keep the supplied application logic unchanged.

Build with `$DC_DOCKER_TAG`, inspect the actual image identifier, then run without a host app mount. Retain the build log and summary output. Explain how FROM, WORKDIR, COPY and CMD contribute to the packaged workload. The resulting image must contain the script and its CSV at paths the default command can use; a successful build alone does not establish that result.

> Run the packaged app without relying on a host mount
>
> **Hint**
>
> The build context is the copied starter directory; COPY cannot read an arbitrary parent directory.
> Read the app's relative CSV path and arrange the copied filesystem accordingly. Use the copied attempt as build context, not an unrelated parent folder. If output still says valid starter, the placeholder command remains. Inspect the build/run logs and image identity before assuming the engine used your latest Dockerfile.

Check

The authored image must emit the actual summary: six holdings and total market value 12,011,626.50. It must do so without mounting the host app directory, and its main process should exit successfully. Run the supplied local test as the independent fixture/application oracle, then retain the build log, actual image identifier and runtime output.

Explain how working directory, copied files and startup command jointly satisfy the app's path contract. A successful placeholder build, hard-coded echo or local-only Python result does not satisfy image packaging. Keep unavailable engine checks pending; direct source inspection remains useful but cannot establish built-image contents.

```
docker build --progress=plain -t "$DC_DOCKER_TAG" .
docker run --rm "$DC_DOCKER_TAG"
```

Phase contents · Reasoning key

Prove repeatability and clean scoped resources

Concept

Repeatability means a known package and input produce the stated result across executions. A mutable tag alone is weaker identity than the image ID because the tag can be rebuilt between runs. Two disposable runs from the same image should both print the six-holding fixture result and exit normally. Their writable layers are separate even though their package is shared.

Cleanup is also an ownership decision. Removing every container or pruning all images would interfere with other learners and erase useful evidence. The exercise names its own containers/tags and retains logs and identities before removal. A build cache or shared base image need not be deleted to finish the core. Repeatability here concerns this supplied app and fixture, not every external dependency or production deployment scenario.

Challenge

Run the finished image twice with `--rm`, recording the actual image ID and both outputs. Inspect its configured command and working directory. Explain whether changing `holdings.csv` on the host would change an already-built image.

Compare both runs with the independent fixture expectation. List this attempt's leftover containers and tag, retain the needed logs, then remove only those named resources when instructed. Keep the authored Dockerfile and local app test for review. Explain how image, container, writable layer and mount differ, and why rebuilding copied input differs from mounting a changed host file.

Compare actual image identity before cleaning named resources

Hint

Use docker image inspect v2-holdings:p1. Rebuilding is necessary for a copied input change.
Capture image identity before both runs so a rebuilt tag cannot masquerade as one package. Disposable execution removal handles its container but does not necessarily remove the image tag. List owned resources explicitly before cleanup. If a name or tag was shared accidentally, resolve ownership rather than invoking a broad prune.

Check

Both finished runs must produce the same six-holding total and successful exit from the same actual image identity. Retain outputs and identity before cleanup, and show that only your named resources were removed. The final Dockerfile remains reviewable and its local test still establishes the fixture expectation.

Explain why matching tags are weaker evidence than matching image IDs and why container removal differs from image/tag removal. A broad prune or deletion of another learner's resource fails the safe completion contract. The local verifier can check supplied app arithmetic without an engine, but image runtime repeatability stays pending unless the real build/run boundary was exercised.

```
docker image inspect "$DC_DOCKER_TAG"
docker run --rm "$DC_DOCKER_TAG"
docker run --rm "$DC_DOCKER_TAG"
```

Phase contents · Reasoning key

Phase completion

Submit the simple Dockerfile and evidence showing image identity, container lifecycle, isolated filesystem, a bounded read-only bind mount and the actual six-holding application output. The printed summary must be six holdings with total market value 12,011,626.50, matching the supplied app's independent fixture check. A valid starter message is deliberately not that result. Retain repeated runs from the same image and explain why an exiting application can be successful without becoming a long-lived service.

Clean only the named exercise containers and tags, preserving another learner's resources and any evidence needed for review. The local app test can run without Docker and gives supporting expected-output evidence; the build/run/user/filesystem boundaries require a real engine. Mark unavailable engine or image-pull checks pending instead of treating a source Dockerfile as a built image. Foundation work is independent of Databricks access and does not replace the later Portfolio Analytics pipeline. Optional multistage work belongs to the independently prepared advanced phase.

Optional stretch

The core completion standard is independent of optional work. No additional stretch is required in this phase; the next phase has an independently prepared entry baseline.

docker phase reasoning keys

Reasoning keys

Read these explanations after attempting the checks.

Reasoning: Run an image and inspect the lifecycle

A zero-exit hello process is successful even though the container is stopped. Logs, exit status and actual identifiers establish that execution. Container removal and image removal affect different resources. Client installation is only a prerequisite, and this hello result is not evidence that the supplied app was packaged.

Return to Run an image and inspect the lifecycle

Reasoning: Explore an interactive shell

The temporary file belongs to the first execution's writable layer and is absent in a fresh container from the unchanged image. Reusing the same execution or adding a volume would change that experiment. Interactive flags affect inspection, not kernel isolation. Record actual runtime observations separately from the author's host.

Return to Explore an interactive shell

Reasoning: Use a bounded bind mount

The selected host app path is readable in the container but deliberately not writable. The working directory makes its relative CSV lookup correct. Expected summary and expected write denial are complementary observations; a failure before mounting proves neither. Keep the host input unchanged and avoid broad mounts that would obscure the intended boundary.

Return to Use a bounded bind mount

Reasoning: Author and run a simple holdings image

The Dockerfile must package the supplied script and fixture at a working directory that satisfies its relative lookup. A real run without a host mount proves those files are in the image. The placeholder is intentionally buildable but incomplete. Independent app output, image identity and selected logs prevent build success or echoed text from substituting for the authored packaging result.

Return to Author and run a simple holdings image

Reasoning: Prove repeatability and clean scoped resources

Distinct executions can share the same immutable image ID and independent writable layers. Matching expected output from that identity demonstrates this bounded app result. Cleanup preserves evidence and removes only named owned resources, leaving shared bases and other learners intact. Local fixture tests supplement rather than replace the actual image build/run observations.

Return to Prove repeatability and clean scoped resources

docker: Phase 2 — advanced practice

Purpose and outcome

Design a bounded multistage image for the supplied summary app. Advanced work separates test and execution concerns, selects named targets deliberately, runs with a suitable user and diagnoses a real path assumption. You author Docker configuration and evidence; the app and expected fixture result remain supplied so the focus stays on runtime boundaries.

Dataset and pipeline

The advanced copy contains the same six-holding app fixture and independent test as foundation practice. Its expected total is 12,011,626.50. This app dataset is distinct from the assigned Databricks Portfolio Analytics and CG product sources. A build stage supplies files, a named test target executes the oracle, and a runtime target includes only the app/input needed for execution. The exact stage graph determines whether runtime receives a tested artifact and what a target build actually executes.

The app resolves its CSV from the current working directory. A deliberately wrong copy destination or directory can therefore make a perfectly built image fail at runtime. The named targets and actual filesystem/user inspections provide evidence that cannot be replaced by reading FROM, USER or COPY text alone.

Prerequisites and workspace

Prepare an independent advanced attempt through mise from the training project root. Use the delivered engine and prepared base image, then choose distinct test/runtime tags for your attempt. The advanced starter includes app/test inputs without requiring your completed foundation Dockerfile. Work from the copied root so build context and relative source paths are unambiguous.

```
mise run challenge:v2:prepare docker phase-two-proficient
cd .local/challenges-v2/docker/phase-two-proficient
export DC_DOCKER_TEST_TAG=dc-alex-docker02:test
export DC_DOCKER_RUNTIME_TAG=dc-alex-docker02:runtime
```

Record actual engine/base identities before comparing builds. Non-root identity and file permissions should be tested in a real container; source configuration only states intent. Avoid privileged options, broad host mounts and global image pruning. Engine access and image pulls belong to preparation outside the unpiloted task estimate. If unavailable, continue Dockerfile review and the local app oracle while leaving actual target/user/filesystem checks pending.

Starter behavior

The copied Dockerfile still prints a benign placeholder and has no completed multistage outcome. A successful starter build is deliberate but cannot pass the finished target checks. Both app and Python oracle work from their supplied directory. You must author named test/runtime targets, meaningful test execution, required non-root user and focused runtime contents. The path defect is introduced later in a disposable version after a working baseline, so an initial missing engine or image is not its intended failure. Preserve a known working Dockerfile before the drill.

Timeline and completion standard

Core reading, authoring, checking and explanation: 75–110 minutes. Allow additional time for setup and optional stretch.

- Task: Separate named build targets · Minutes: 15–22 · Evidence: Actual target image IDs and expected target outputs
- Task: Gate the runtime on a tested artifact · Minutes: 15–22 · Evidence: Test failure blocks accepted runtime artifact path; repaired test/app succeeds
- Task: Run with a bounded runtime user · Minutes: 15–22 · Evidence: Runtime UID 10001; app reads intended files and still prints expected summary

- Task: Keep runtime contents focused · Minutes: 15–22 · Evidence: Required app/input present; test script absent; expected output preserved
- Task: Repair a path assumption and explain the result · Minutes: 15–22 · Evidence: Predicted missing-file failure followed by repaired expected output and explanation

Independent starter files Read the owning phase before editing. Prepare from the training project root with `mise run challenge:v2:prepare docker phase-two-proficient`; edit the copied attempt.

Authored-file list

Task	*Learner-authored paths*
Separate named build targets, Gate the runtime on a tested artifact, Run with a bounded runtime user, Keep runtime contents focused, Repair a path assumption and explain the result	`Dockerfile`, `evidence.md`

Declared baseline

The app and independent Python test are supplied. The Dockerfile builds a placeholder image; replace its unfinished configuration to package the app. A successful placeholder build is not completion.

Use a fresh workspace for another attempt; preparation and reset preserve existing work.

Core challenges

Separate named build targets

Concept

A multistage file defines a dependency graph of named build stages. Selecting one target chooses the output image and the stages it depends on; it does not automatically publish every intermediate stage as a separate tagged image. The test target and runtime target therefore need deliberate build/tag commands when both are review artifacts.

Named stages also communicate purpose. A runtime target should contain the executable app/input, while a test target can retain the independent test script and its command. The source file alone cannot show which target an actual build selected or whether a cached stage was reused. Record command, target and resulting image identity so a reviewer can connect intended architecture with observed output.

Challenge

Create named `base`, `test` and `runtime` stages in the Dockerfile. The test target runs `test_summary.py`; runtime runs `summarize.py`. State their dependency relationship and how tested app/input reaches runtime.

Build test with `$DC_DOCKER_TEST_TAG` and runtime with `$DC_DOCKER_RUNTIME_TAG`. Inspect both resulting image IDs and run the targets separately. Keep the selected target explicit in each command so a default final-stage build cannot stand in for two outputs. Retain the commands and explain which prerequisite stages each target used. Check that the named stages match the verifier's expected interface.

> Target selection follows the stage graph, not every FROM line
>
> **Hint**
>
> Use FROM ... AS name and COPY `--from`=test for the selected runtime artifacts.
> Draw the small stage dependency graph before writing COPY references. A target flag chooses an output, while a tag names the resulting image. Inspect the actual selected stage command and runtime output to distinguish a test image from an app image. If a stage is unreachable from the selected target, its existence in source is not proof it ran.

Check

The test and runtime target builds must produce identifiable separately tagged images. Running test must emit the supplied oracle success, and running runtime must emit the six-holding summary. Retain both target commands, image IDs and outputs, explaining which stages each depended on.

A default final-stage build alone does not prove the separate test image exists or executed. An echoed success string also does not replace the supplied test behavior. If the engine cannot run here, leave build/target observations pending while preserving a source graph that can be inspected. The finished verifier builds the named targets, not arbitrary intermediate indexes.

```
docker build --target test -t "$DC_DOCKER_TEST_TAG" .
docker build --target runtime -t "$DC_DOCKER_RUNTIME_TAG" .
```

Phase contents · Reasoning key

Gate the runtime on a tested artifact

Concept

A testing stage is useful when it validates the artifact that runtime receives. An unrelated test command in an unreachable stage can look reassuring while providing no gate. The stage graph should connect the tested app/input to the runtime copy path, and a build-time assertion or equivalent explicit workflow should reject an incorrect artifact before acceptance.

The supplied Python test computes the fixture expectation independently of a hard-coded echo. The tiny app's baseline result is fixed, so a controlled copied input or app-path defect should make that assertion fail. Keeping the test's purpose visible prevents a Dockerfile from satisfying advanced work merely by naming a stage test. The runtime output remains a separate observation after the test succeeds.

Challenge

Add `RUN python test_summary.py` to the test stage. Wire runtime's COPY or inheritance path to the same app/input that the test validates. Build the valid target and retain its success.

In a separate copied build context, change one holding amount by a known value and predict the oracle failure. Rebuild, inspect the failed assertion, then restore the fixture and rebuild both targets. Keep the working Dockerfile available throughout. Do not suppress the nonzero test result or alter the expected baseline total. Explain why the tested artifact is the one the runtime receives.

> Trace runtime content back to the actual oracle boundary
>
> **Hint**
>
> Keep the original workspace intact; make the controlled defect in a second copied build context.
> Trace the exact file source of the runtime COPY back to the tested stage. If the test runs only as an optional container command, describe how acceptance enforces it rather than pretending the build itself is gated. Avoid appending an unconditional success echo after a failed assertion. Use the supplied test's expected result to diagnose the copied artifact cause.

Check

The valid test target must execute the real supplied summary test, and the finished runtime must use the corresponding tested app/input. The controlled incorrect artifact must be rejected for the predicted oracle or path reason; restore it and retain a successful test/runtime sequence. Record stage graph, command exit categories and actual identities.

Explain the difference between a stage named test and an enforced test boundary. A compiler/source inspection cannot establish the rejection transition. Keep source fixtures unchanged outside the disposable copy, and do not loosen the expected total merely to accept a broken artifact. Any unavailable engine execution remains pending rather than a source-only pass.

```
docker run --rm "$DC_DOCKER_TEST_TAG"
docker run --rm "$DC_DOCKER_RUNTIME_TAG"
```

Phase contents · Reasoning key

Run with a bounded runtime user

Concept

A runtime user changes the process's permission boundary, not the meaning of the app. The image should provide readable app/input files and an appropriate working directory for the declared non-root identity. A USER line can name an identity that lacks the necessary file access, so inspect actual UID and execute the app rather than trusting the Dockerfile keyword.

The expected training runtime UID is 10001. Its app only needs read access to the packaged fixture; adding broad writable host mounts or privileged flags would change the exercise boundary. An image can build under root while its final process runs non-root, because build and runtime identities are separate. That distinction should be visible in the explanation and actual command output.

Challenge

Create runtime UID 10001, give it the required read access to the packaged script/input, and set `USER 10001`. Use explicit ownership when copying from the test stage. Build runtime, inspect `id -u` inside it and run the default summary.

Retain both observations from the same image ID. If execution fails, distinguish a missing path from a file-permission error and repair only that cause. Confirm the process can read the app and fixture without privileged flags or a writable host mount. Explain which build steps used another identity and why runtime does not need root.

> Prove actual runtime identity together with useful app access
>
> **Hint**
>
> COPY `--chown` can set ownership while copying from a named stage.
> Inspect identity and file permissions as separate observations. The build's default user does not prove the final process's user. Read-only app access is sufficient for this fixture. If a file exists but cannot be read, diagnose its path and mode before changing the whole container security boundary.

Check

The finished runtime must report UID 10001 and still emit six holdings with total market value 12,011,626.50. Retain the identity probe and application run from the same image ID. Explain which build steps ran under another identity and how packaged file access supports the final process.

A USER declaration without successful execution is insufficient, and a successful app run forced back to root fails the stated runtime contract. Do not add broad write permissions or host mounts to conceal a packaging defect. The source may be reviewed locally, but actual UID and access observations require a real container engine.

```
docker run --rm "$DC_DOCKER_RUNTIME_TAG" id -u
docker run --rm "$DC_DOCKER_RUNTIME_TAG"
```

Phase contents · Reasoning key

Keep runtime contents focused

Concept

A focused runtime image contains what the executable needs, not every file available during build. The independent test is useful for acceptance but not required by the summary process. Copying the whole test stage can retain unnecessary scripts or temporary files. Narrow COPY decisions make the runtime boundary easier to inspect and reduce accidental content exposure.

Smaller size alone is not correctness: deleting the input CSV can shrink the image while breaking the app. The appropriate evidence combines required content presence, test-script absence and expected runtime behavior. Build/cache files and runtime files serve different roles. The exercise does not claim a production security audit from this small content check, but it teaches deliberate artifact selection.

Challenge

Copy only `summarize.py` and `holdings.csv` into runtime. Inspect `/app` in both targets and confirm `test_summary.py` is absent from the final runtime image. Compare their configured users, commands and sizes.

Run the summary after any COPY-path change and tie the filesystem inspection to that same image ID. Explain why each retained file is needed and why image size is only supporting information. A smaller image that lacks its CSV has not met the app contract. Inspect the final stage rather than assuming a narrow COPY statement produced the image your tag currently names.

Inspect the final filesystem and keep required behavior intact

Hint

Override the runtime command with ls or python -c when inspecting.
Inspect the final stage rather than a test-stage shell. Copying an entire directory may carry the test script even when no command executes it. Pair absence checks with app output so a missing required file cannot pass as a lean runtime. Use explicit source/destination paths rather than an unexplained broad directory copy.

Check

The runtime filesystem must exclude the supplied test script while retaining the app and holdings input at the expected paths. Its non-root process must still print the independent six-holding summary. Retain filesystem inspection and output tied to the actual runtime image ID.

Explain why an image-size comparison alone cannot establish focused contents or useful behavior. A source COPY statement is insufficient if actual execution uses another image/tag. Keep build/test stage evidence separate from runtime inspection. If an engine is unavailable, record the intended file inventory and leave its actual absence/presence assertions pending.

```
docker run --rm "$DC_DOCKER_RUNTIME_TAG" python -c "from pathlib import Path; assert not
↪  Path('/app/test_summary.py').exists()"
```

Phase contents · Reasoning key

Repair a path assumption and explain the result

Concept

Runtime path failures often come from an inconsistent relationship among build context, COPY destination and working directory. The summary opens its CSV by relative name. Moving its execution directory or copying that input elsewhere can cause a missing-file error even though the image built successfully. The diagnosis should identify that relationship rather than modify business data or install unrelated dependencies.

A controlled defective Dockerfile creates a useful before/after experiment after a known good baseline exists. The build can remain valid while the process fails, demonstrating the distinction between build-time and runtime checks. Repairing the minimal path assumption and rerunning the same fixture establishes recovery. Keep the defect out of the final image and retain evidence before cleaning owned tags.

Challenge

Save the working multistage Dockerfile. In a disposable copy, change runtime's WORKDIR to `/wrong` and predict how the default command will fail. Build and run the defective target, then inspect its configured directory and packaged paths.

Repair the path and repeat both target checks. Retain the failure, minimal Dockerfile diff and repaired summary from identifiable images. Confirm the fix preserves test gating, UID 10001 and the absence of `test_summary.py` from runtime. Write a short handoff explaining why the image built successfully but failed when its default process ran. Keep optional cache analysis separate.

Repair the path contract instead of bypassing the application

Hint

Inspect the image configured working directory and run a listing before editing Python code.
Follow the CSV path from the app source to final COPY destination and process working directory. A missing-file error differs from a read-permission failure. Change one assumption at a time and keep the known-good version available. Do not add a broad host mount or hard-code the expected output to make the defective container appear recovered.

Check

The controlled runtime failure must match the predicted path/entrypoint cause, and the repaired image must again emit the six-holding total with UID 10001 and no packaged test script. Retain failing and repaired command/category, bounded Dockerfile diff and actual image IDs. Explain why build success did not catch this runtime assumption.

Recheck both named targets after repair so a path fix cannot bypass the test boundary. Clean only owned tags/executions after preserving evidence. A missing engine or base-image download is not the intended defect, and local Python success does not prove repaired image behavior. Leave those actual runtime observations pending if they cannot run here.

```
docker build --target runtime -t "$DC_DOCKER_RUNTIME_TAG" .
docker run --rm "$DC_DOCKER_RUNTIME_TAG"
```

Phase contents · Reasoning key

Phase completion

Finish a reviewable multistage Dockerfile with separately buildable test and runtime targets. The test target must run the supplied independent summary test, while the runtime target must emit six holdings and total market value 12,011,626.50. Inspect actual runtime user and filesystem to prove the declared non-root identity and absence of the test script. Retain a controlled working-directory/copy-path defect and its repaired output rather than relying on Dockerfile keyword inspection.

Explain the selected target dependency graph, why a tested artifact reaches runtime, and which files belong to build/test versus execution. Use owned tags/names and clean only this attempt's resources after retaining image identities and logs. A local Python test establishes fixture arithmetic but does not prove image stage, user or filesystem behavior. Mark engine/image-build evidence pending when unavailable. Optional cache/size analysis can follow the fixed core with its own estimate, but it cannot replace the test, runtime, identity and repair outcomes.

Optional stretch

The core completion standard is independent of optional work. Choose a separately timed route from the stretch guide.

docker phase reasoning keys

Reasoning keys

Read these explanations after attempting the checks.

Reasoning: Separate named build targets

Named stages describe dependency and purpose. Selecting runtime does not automatically tag every stage, so retain explicit test/runtime builds and actual image IDs. Their commands produce different expected outputs. Source stage declarations are useful review evidence but cannot substitute for real target execution or an actual oracle run.

Return to Separate named build targets

Reasoning: Gate the runtime on a tested artifact

The meaningful gate is the supplied oracle executed on the same app/input passed to runtime. A disconnected stage or unconditional success message does not establish it. The bounded defective

artifact should fail for its intended reason, and restoration should recover the expected summary. Build-time gating and an explicit separate acceptance test need truthful distinction.

Return to Gate the runtime on a tested artifact

Reasoning: Run with a bounded runtime user

The finished process must actually run as UID 10001 while reading the correctly packaged app/input. Build steps may use another identity. Source USER text states intent, while identity plus successful output proves the runtime boundary. Minimal file access is the proper repair; root or privileged execution would bypass the assessed contract.

Return to Run with a bounded runtime user

Reasoning: Keep runtime contents focused

The runtime does not need the test script, but it does need the summary and input. Actual final-stage inspection plus expected output establishes both sides of that boundary. Size or source COPY text alone can conceal missing required content or accidental test material. Tie the observations to the same runtime identity.

Return to Keep runtime contents focused

Reasoning: Repair a path assumption and explain the result

The app's relative input lookup links COPY path and working directory. A valid build can still fail that runtime contract. The controlled defect and minimal repair must preserve test gating, non-root identity and focused contents. Actual failure/recovered output and image IDs establish the transition; host mounting or echoed output would bypass it.

Return to Repair a path assumption and explain the result

docker optional stretch

Choose an optional exercise after the advanced core. Allow 20–40 minutes for each and use a fresh copy of your completed baseline. Keep the assessed attempt unchanged.

Optional exercises

Explain cache reuse and a controlled input rebuild

Concept

Build cache can reuse a stage whose relevant inputs and instructions have not changed. The observed rebuild log, selected target and actual resulting image determine what was reused; merely running the same command twice does not prove every stage executed again. A mutable tag can hide that distinction because it can point to a new image after a rebuild.

The supplied six-holding input provides a bounded invalidation experiment. Changing one copied CSV value can affect the application layer and expected summary without requiring an unrelated service stack. Keep that variation separate from the fixed core baseline and calculate its expected total independently. Image size alone cannot establish correctness or cache behavior. The useful explanation identifies the input boundary, affected build stage and actual output.

Challenge

Preserve a working completed Dockerfile and perform two builds with unchanged input, retaining plain build logs and image identifiers. In a separate copied variation, change one holding value by a known amount, predict the new total through an independent CSV calculation, and rebuild the selected runtime target. Inspect which stages reused cache and which consumed the changed input. The fixed baseline assertion should now fail. In this optional copy only, replace its expected total with your independently calculated variation total, label that test as variation-specific, and rebuild. Run the resulting image and

compare its actual summary with the predicted variation. Restore the copied baseline afterward without loosening the fixed core oracle.

Record the copied input delta before rebuilding and name the specific layer whose changed content should invalidate reuse.

> Trace the changed input to its COPY boundary
>
> **Hint**
> Compare the stage logs rather than only elapsed time. A fast network or already pulled base can make a build faster without the specific layer reuse you intend to explain. Trace the changed CSV to its COPY boundary. Preserve the original input hash and total so the variation cannot be mistaken for the assessed baseline. A changed tag name alone does not invalidate the relevant content.

Check

Retain unchanged and changed build logs, actual image IDs, input hashes and runtime summaries. The changed result must differ by the independently predicted amount while still containing six holdings; restoration must recover the baseline 12,011,626.50 result. Explain which input/stage boundary produced the observed reuse or invalidation.

Retain the original core test's rejection of the changed fixture and the separate variation-specific test result. Keep the assessed baseline test unchanged. Source inspection or duration alone cannot prove cache reuse. Use the actual engine when available and mark those observations pending otherwise. Clean only the optional experiment tags after preserving evidence, leaving the completed core image and another learner's resources intact.

Optional contents · Reasoning key

Run the prepared app through bounded Compose configuration

Concept

Compose describes a local multi-container application configuration, but a single supplied app can also demonstrate its project/name and command boundaries. The optional experiment should not introduce a database, broker and web service merely to look more advanced. The same packaged summary and its read-only input can establish a useful configuration comparison.

Project identity scopes names and cleanup. A Compose file in source does not prove the selected service image, runtime user or working directory actually ran. The core Dockerfile remains the artifact under test, while Compose selects and starts its local execution. A service that finishes a batch summary can exit successfully; forcing it to stay alive would change the behavior. Retain the expected app result and owned project boundary.

Challenge

Author a small Compose configuration for the completed summary image, with an explicit owned project name and the required runtime path/input contract. Inspect the resolved configuration, execute only that supplied batch service and retain its logs/exit result. Compare it with the direct docker run outcome. If you explore a read-only mount, keep it limited to the copied app directory and explain how it changes packaging assumptions. Remove only this optional Compose project after collecting evidence.

Keep the resolved configuration and its project name beside the actual service result so the observed runtime can be reproduced.

Inspect the resolved service before starting it

Hint

Inspect the installed Compose help and resolved service definition before startup. Compare selected image, working directory, user and mount with the direct runtime contract. A completed batch service need not remain running. Keep project cleanup bounded; a global prune is unrelated to this optional configuration exercise. Do not require an extension-specific renderer to understand the Compose YAML.

Check

The resolved configuration and actual service execution must preserve the expected six-holding total, intended user and bounded filesystem access. Retain project/service identity, selected image ID, exit category and logs. Explain what Compose changed about orchestration/configuration and what remained the same application behavior.

A parsed YAML file alone is supporting evidence. Actual startup and cleanup need the prepared engine, and unavailable runtime stays pending. Do not introduce external infrastructure or another mandatory course dependency. After owned project removal, the completed core Dockerfile and evidence remain intact. The optional route has its own estimate and cannot replace the core multistage/test/user/path checks.

Optional contents · Reasoning key

docker phase reasoning keys

Reasoning keys

The build log identifies cached stages; runtime output checks the result of the changed input. A data-only COPY change should invalidate stages that depend on it while earlier unchanged work can be reused. The fixed core oracle may reject the changed fixture intentionally. Restore the baseline and its expected total before final cleanup.

Return to Explain cache reuse and a controlled input rebuild

Compose selects the image and execution configuration for the same batch app. Its project name scopes resources, while the resolved service shows user, command, path and mounts. A successful summary can exit normally. Compare actual logs and image identity with the direct run to establish that the configuration preserved the app contract.

Return to Run the prepared app through bounded Compose configuration

dbt — Portfolio Analytics

Projects contents · Task index

- Foundation core — page 292
- Advanced core — page 301

The consumer measure is holdings value per synthetic unit: supplied holdings-only GBP exposure divided by 100 million synthetic units, with cash excluded. It is not assets less liabilities or official NAV per outstanding fund share. Compatibility identifiers `fund_nav_report`, `nav_per_unit` and `seed_custodian_nav` remain unchanged. The fund/date reporting grain does not establish manager-sleeve ownership.

dbt — Portfolio Analytics

Build a traceable transformation from supplied custodian feeds to a fund/date report on Databricks. The foundation phase teaches external sources, typed staging, business grain, model references and independent output checks. The advanced phase supplies that baseline and develops dated enrichment diagnosis, reconciliation, controlled corrections, history and a reviewable feature. The consumer question is what each fund's supplied holdings contribute to GBP exposure on each fixture date, with a carefully labelled holdings value per synthetic unit measure.

Before you start

Open the supplied training project in its dev container and complete the delivered Databricks sign-in before starting timed practice. Confirm the installed dbt executable includes the Databricks adapter, identify the approved catalog/schema and reconcile its aliases. Choose a unique relation prefix for each attempt. Your terminal runs dbt while the SQL warehouse executes the models; neither a local target name nor a successful parse establishes a warehouse result.

Prepare through the version-explicit mise interface from the training project root, then run direct dbt commands inside the copied attempt. Keep your original work and use a new workspace for another try. Advanced practice has an independently prepared starter; it does not assume you completed or viewed someone else's foundation answer. Both phases use the same eight supplied input CSVs and disclosed supporting models, with authoring responsibilities listed in the phase guide.

Suggested timeline and tasks

The phase estimates include reading, authoring, checking and explanation. Allow additional time for sign-in, tool setup, source loading and optional extensions. Follow your instructor's selected route for Wednesday's practice block.

- Phase: Foundation · Task: Inspect the project and source grain · Minutes: 15–22
- Phase: Foundation · Task: Declare and use sources · Minutes: 15–22
- Phase: Foundation · Task: Normalize type, unit and key contracts · Minutes: 15–22
- Phase: Foundation · Task: Build a useful mart through ref · Minutes: 15–22
- Phase: Foundation · Task: Add minimum key and relationship proof · Minutes: 15–22
- Phase: Advanced · Task: Diagnose a join fan-out before repair · Minutes: 18–24
- Phase: Advanced · Task: Repair enrichment at the intended grain · Minutes: 18–24
- Phase: Advanced · Task: Write reconciliation that detects the negative case · Minutes: 18–24
- Phase: Advanced · Task: Prove incremental update and replay behavior · Minutes: 18–24
- Phase: Advanced · Task: Explain dependency impact and hand over · Minutes: 18–24

Choose your phase

- Foundation phase: complete the assigned source/staging/report slice and prove its minimum business contract.
- Advanced phase: begin from its supplied baseline and demonstrate diagnosis, replay and a useful handoff.

Finish the core tasks before choosing optional stretch. These phases use the supplied Portfolio Analytics data on Databricks.

Teaching concepts

Trace source declarations to physical owned tables, distinguish source and model dependencies, preserve composite grain, choose deliberate materializations, and separate native valuation from dated GBP conversion. Structural tests and independent business calculations protect different failure modes. The advanced route extends those ideas into controlled Delta replay, snapshot validity, documentation and the Astro worker boundary. Reasoning keys stay at the end of their owning phase; full runnable instructor references are supplied separately.

Question-led foundations and further study

Use questions 1–8 alongside the foundation tasks. Questions 9–13 introduce further study.

Foundations

What problem does dbt solve, and where does it fit in a data pipeline?

Custodian feeds arrive as holdings and cash; prices, dated FX and fund metadata explain those positions. dbt orders SQL transformations and tests. The Databricks warehouse executes SQL and stores views/ Delta tables. Running dbt in a local process does not create a local database. Our consumer output is a fund/date GBP exposure and a holdings value per synthetic unit, with lineage back to the supplied raw tables.

What does a dbt project contain, and how do we connect it to our data?

The prepared Portfolio Analytics attempt contains models, eight seeds, tests, macros, profiles and a local target directory. Read the setup contract before connecting. Profile portfolio_analytics and target local point to the assigned Databricks warehouse. dev and airflow are also Databricks targets. Record the project/profile paths and installed versions; reject missing namespace or conflicting aliases rather than guessing a username.

How does a SQL query become a dbt model?

Author models/staging/stg_holdings.sql. Select fund_cd as fund_id, pos_dt as position_date, isin as instrument_id, qty as decimal quantity, mkt_val_lc as decimal native value and ccy_cd as local_ccy. Cast dates and retain extraction time. Its business key is fund/date/instrument. The baseline contains eight rows: one instrument for each of four funds on each of two dates. A model file becomes an owned view after execution; its filename is the ref name, while its physical alias includes your prefix.

How do we connect models using source() and ref()?

source('custodian','holdings_raw') reads the prefixed raw seed-backed Delta table. ref('stg_holdings') connects transformations and lets dbt establish order. Declare raw source database/schema from the target and identifier from the ownership prefix. A source declaration does not load data. Run the explicit eight-seed preparation, then inspect the compiled relation names and graph. Supplementary NAV/fund seeds alone leave six raw sources missing.

How should we split a large query into understandable, reusable models?

Keep cast/rename in five staging views; dated enrichment belongs in int_holdings_enriched. Join prices by instrument/date and FX by native currency, GBP and date. Fund metadata joins by fund key. Cash positions have fund/date/currency grain and remain separate until aggregated correctly. Validate right-side uniqueness before joining: an undated FX join fans eight holdings out across two dates. refdata_raw is available for a classification extension; it is not a substitute for fund_master_raw.

When should a model be a view or a table?

Views suit the small staging and intermediate transformations. A consumer table persists the report for repeated reads and a stable tested handoff. Inspect information_schema for the prefixed relation and its type; do not infer storage from a successful command. Tables on this adapter use Delta. The richer product has optional optimisation hooks, but the guide omits them to avoid adding managed-permission assumptions to the core exercise.

How do we know our models produce trustworthy data?

Test non-null keys, composite uniqueness, decimal types, dated join coverage and populations. Missing FX is an error, not an implicit rate of one. All supplied holdings happen to be GBP; fund PEF-EM-GROWTH has USD master currency, but this report explicitly values in GBP. Market value is supplied by the custodian and must not be replaced with quantity times price without an agreed contract. The reference synthetic-unit value uses 100 million synthetic units and excludes cash. Compare each of eight fund/date rows independently with seed_custodian_nav.

How do we make a change confidently and explain the result?

Select the changed model and downstream dependencies, retain compiled SQL and run results, then query the actual warehouse. For a negative drill remove a dated GBP FX row in a disposable copy. matched_enrichment must fail; restore, reseed and rebuild before handoff. Documentation must state grain, units, source meaning and the synthetic-unit measure limitation. A green structural test can miss a correct-looking wrong business measure.

Further study

Questions 9–13 are separately timed advanced study.

When does Jinja help, and when should we keep the logic in SQL?

A small native_value(value, rate) macro casts the GBP conversion to a declared decimal precision. Inspect its compiled multiplication and explain the arguments. Keep business joins in readable SQL. Reuse earns its place when the same expression has the same meaning in both the consumer model and the correction/replay model; a macro cannot settle currency or grain disagreements.

How can we process less data without producing incorrect results?

In the separately timed advanced phase, the required holdings_replay model uses Delta merge with fund/date/instrument as its composite key. Read the whole tiny fixture each time so older-date corrections are included. Copy one holding, add 1,000 GBP to its value, reseed, merge and replay. Query the original key after each run: corrected value and exactly one row. A maximum-date watermark would miss this controlled historical correction. This is a labelled variation, not history already present in the source.

How do we preserve history when source records change?

Snapshot a copied fund-master relation before and after one controlled name change using the check strategy and fund_cd key. Retain validity columns and verify the original closed row plus one current row. All snapshots remain in the assigned schema with owned aliases. Snapshots record observed states, not every event between observations. Restoring source CSVs does not delete history; a new attempt/prefix starts clean.

How does a dbt project move into reliable team delivery?

Review SQL, tests, contract and target routing together. Hand off the exact prepared project to Astro with a worker-visible path and inspected executable; resolve the token at task execution. Record versions at both boundaries. Production promotion remains a design discussion. A local compile or prior green run cannot prove a new scheduler/worker/output path.

Can we independently deliver a documented, tested model?

Build fund_nav_report at fund/date grain from the available enrichment. State GBP, the 100 million synthetic units and excluded cash. Include one independent business query, lineage, structural tests, reconciliation, failure/repair and a reviewable diff. Your evidence should let another person distinguish a missing model, stale data, join fan-out and a currency-definition mistake.

dbt: Phase 1 — foundations

Purpose and outcome

Build a small but complete Portfolio Analytics result on Databricks: supplied custodian inputs become typed holdings, dated enrichment and a fund/date consumer report. You write the holdings transformation and report. Other staging and enrichment models are supplied; inspect their dependencies to understand the complete pipeline.

Dataset and pipeline

Eight supplied CSVs enter the project: six raw feeds plus fund and custodian-NAV reference fixtures. Holdings, positions and prices each contain eight rows; FX has six rates, fund master four funds and instrument reference four instruments. Holdings cover four funds on 20 and 21 May 2026. The modeled graph reads holdings, positions, prices, FX and funds. Instrument reference remains available for source inspection. Custodian raw cash is not included in the holdings value per synthetic unit numerator. The report converts supplied native market value to GBP and divides the holdings-only amount by a synthetic one-hundred-million-unit denominator. It is not an official unit-price calculation.

Prerequisites and workspace

Use the supplied training dev container, delivered OAuth setup and a dbt executable with the Databricks adapter. From the training project root prepare a fresh attempt with the version-explicit mise task. Record installed versions instead of assuming a pinned file proves the worker environment. Set the approved catalog/schema aliases consistently and choose an owned relation prefix before loading. Never create an extra schema as an experiment. Your terminal runs dbt; the warehouse executes its SQL. The local target name still selects Databricks, not DuckDB.

```
mise run setup:databricks
mise run challenge:v2:prepare dbt phase-one-certified
cd .local/challenges-v2/dbt/phase-one-certified
export DC_RELATION_PREFIX=dc_alex_dbt01
dbt --version
dbt debug --project-dir . --profiles-dir . --target local
```

Use a new workspace and prefix for a second simultaneous attempt. Work inside the copy, editing only the task's named files. Keep evidence beside the copied project. Do not place tokens in that note, commit local credentials or overwrite the canonical starter. The root mise verification command accepts the absolute or root-relative attempt path; direct dbt commands below run from this prepared directory.

Starter behavior

The foundation starter deliberately contains unfinished holdings staging and report SQL. They are valid files for authoring, but the completion checker rejects their placeholder result. Source declarations, routing macros, remaining staging models and enrichment are supplied. Inspect that boundary before running a broad build: downstream failures from an unfinished model are expected until its task is complete. Seed loading can succeed first; it is not an assessment pass. Freshness uses extraction timestamps separately from the historical business dates. If current freshness fails, record that cause and use the documented timestamp-only preparation rather than changing the May holdings dates.

Timeline and completion standard

Core reading, authoring, checking and explanation: 75–110 minutes. Allow additional time for setup and optional stretch.

- Task: Inspect the project and source grain · Minutes: 15–22 · Evidence: Eight holdings, four funds, two business dates; safe identity and seed inventory
- Task: Declare and use sources · Minutes: 15–22 · Evidence: Compiled source relations and manifest dependencies agree with all raw seeds

- Task: Normalize type, unit and key contracts · Minutes: 15–22 · Evidence: Eight unique holding keys; native decimal values preserved
- Task: Build a useful mart through ref · Minutes: 15–22 · Evidence: Eight fund/date rows; independently derived GBP exposure and holdings value per synthetic unit
- Task: Add minimum key and relationship proof · Minutes: 15–22 · Evidence: Declared tests pass; unknown-fund membership fails, then passes after input restoration

Independent starter files Read the owning phase before editing. Prepare from the training project root with `mise run challenge:v2:prepare dbt phase-one-certified`; edit the copied attempt.

Authored-file list

Task	*Learner-authored paths*
Inspect the project and source grain	`evidence.md`
Declare and use sources	`models/sources.yml`, `models/staging/stg_holdings.sql`, `evidence.md`
Normalize type, unit and key contracts	`models/staging/stg_holdings.sql`, `tests/holdings_grain.sql`, `evidence.md`
Build a useful mart through ref	`models/marts/fund_nav_report.sql`, `evidence.md`
Add minimum key and relationship proof	`models/schema.yml`, `tests/holdings_grain.sql`, `tests/matched_funds.sql`, `tests/matched_enrichment.sql`, `tests/nav_reconciliation.sql`, `evidence.md`

Declared baseline

These focus files are intentionally unfinished: `models/staging/stg_holdings.sql`, `models/marts/fund_nav_report.sql`. Supporting models and profiles are supplied. Complete the listed files, then verify their output in the required Databricks or Astro environment.

Keep `evidence.md` beside the copied project. Create a named file when its task asks for it. Use a fresh workspace for another attempt; preparation and reset preserve existing work.

Core challenges

Inspect the project and source grain

Concept

A dbt project describes transformations and dependencies; it does not become a separate database simply because it lives in a container. The adapter compiles and submits SQL to Databricks, where the approved catalog, schema and warehouse determine the relations you read and write. Seeds load copied training inputs. Sources describe that input boundary, while models describe derived relations. Keeping those roles separate helps diagnose a missing table before changing a transformation.

In this fixture a holding is identified by fund, business date and instrument. The same fund appears on two dates, so fund alone cannot identify a row. A report with four rows after both dates are loaded has probably collapsed the date dimension. Inspecting the eight raw rows makes that mistake visible before SQL is written. Distinguish the position date from the extraction timestamp: one describes the observation being valued, the other describes ingestion freshness. An old business date can be the correct teaching input even when its original extraction timestamp is stale. Likewise, a successful connection says nothing about whether the six raw feeds have actually been loaded.

Challenge

Create an evidence note in the copied project and list each seed's name, row count, key fields and money units. Inspect the project and profile configuration, identifying the adapter, target, approved catalog/schema and relation prefix without copying credential values. Run the version and connection probes,

then load all eight seeds. Find the raw holdings, funds, dated prices and FX sources in the declarations and note which physical prefixed tables they resolve to.

Use a small independent CSV inspection to count distinct funds and dates. Choose one holding and trace its native currency and value to the source field that supplies them. Explain why the product's two supplementary CSVs cannot provision the six raw feeds. Keep a record of any unavailable warehouse prerequisite; continue the CSV inspection locally while leaving the connection and actual loaded-relation checks pending. Do not substitute the optional five-row fixture or call its result this phase's completion.

> Separate input inspection from warehouse readiness
>
> **Hint**
> Start with the headers in the copied seeds and the database/schema expressions in the source YAML. Write the composite holding key as three named fields before counting rows. If connection debug fails, inspect the adapter identity and alias consistency before trying a model build. A namespace mismatch can produce missing relations even when authentication succeeds; a username fallback would conceal that mismatch rather than fix it.

Check

Run the seed command from your attempt and inspect the resulting prefixed input relations. The independent CSV inventory should show eight holdings, four distinct fund identifiers and two business dates. Verify the corresponding warehouse counts when connected; CSV counts alone establish supporting input evidence. The six raw counts are eight holdings, eight positions, eight prices, six FX rates, four fund-master rows and four reference instruments. Supplementary funds and NAV inputs need their own listed counts.

Retain the installed adapter/version, selected target, redacted namespace identity, seed run-results identity and the independently calculated populations. Explain which command loaded inputs and which file only declared their names. If debug reports an authentication or permission error, record that category and leave this check pending rather than treating a local seed inspection as a warehouse pass. Your explanation should predict why removing the position date from a key would merge two otherwise valid observations.

```
dbt seed --project-dir . --profiles-dir . --target local --full-refresh
python runtime/lab.py reference .
```

Phase contents · Reasoning key

Declare and use sources

Concept

The source function gives a model a logical dependency on data loaded outside that model. Its first name identifies the source group and its second identifies the table inside that group. Those logical names need not equal the physical warehouse relation: the source identifier can add the owned training prefix while the SQL remains stable. A reference to another model instead declares a derived dependency that dbt can order and document.

Consider the holdings model reading the custodian holdings source. Databricks ultimately executes a fully qualified relation in your approved namespace; the source declaration explains why that relation is an input rather than a generated mart. A raw seed can be present but wrongly mapped to another learner's table. Conversely, a perfectly written declaration can resolve to a table that was never seeded. Compilation and an actual query expose different halves of this boundary. The instrument-reference input is useful to inspect even though the prepared consumer graph does not depend on it. Do not invent a lineage edge merely to make every loaded feed appear in the final report.

Challenge

Inspect all six raw table declarations in the copied source YAML and verify their identifiers use your relation prefix. Retain the prepared logical source groups for custodian, Bloomberg, FX and product reference. In the holdings staging file replace the unfinished scaffold with a source-based input CTE, then add the transformation required in the next task. Read the supplied fund, FX, price and position staging files to identify their source calls and the prepared enrichment model to identify its model references.

Compile a selected holdings model and trace one rendered relation back to its source declaration, seed file and approved namespace. Record that chain in the evidence note. Keep the supporting model files supplied unless an actual declared task requires a change. If a raw input is missing, fix loading or source mapping before adding a hard-coded qualified name to model SQL. That shortcut would bypass the lineage and ownership contract being learned.

> Trace one logical name to its prefixed input table
>
> **Hint**
> Compare the logical table name in the source call with the identifier expression in YAML; they answer different questions. The source manifest node records database, schema and identifier after configuration is resolved. If compilation fails because the holdings scaffold is incomplete, keep the first error and use the supplied source declarations to complete the input CTE. Avoid changing all six source names at once while diagnosing one bad relation.

Check

Compile the holdings selection and inspect its SQL and manifest dependency entry. The rendered raw relation must belong to your approved catalog/schema and carry your owned prefix. The dependency must point to the custodian holdings source, while the later report depends on models through references. Confirm the source inventory contains the six declared raw inputs; instrument reference may have no consumer-model edge without being an error.

When connected, build the holdings model after its type transformation is complete and inspect the warehouse relation. Record the difference between compilation success and query success. To diagnose a deliberate source-mapping mistake in a disposable copy, point one identifier at a nonexistent owned name: the affected model should fail with a missing-relation category. Restore the declaration and rebuild before retaining completion evidence. Explain why loading the seed alone would not repair a wrong identifier and why bypassing the logical source with a literal table name would weaken lineage.

```
dbt compile --select stg_holdings --project-dir . --profiles-dir . --target local
```

Phase contents · Reasoning key

Normalize type, unit and key contracts

Concept

Staging gives raw data stable names and types while preserving its declared population. Renaming the custodian fund code and position date makes their purpose clearer, but it does not justify dropping a date, replacing a missing key or changing money units. Money stored as native market value must remain native market value until the dated FX join explicitly converts it. A fund's master base currency is a separate attribute, not permission to relabel a holding value.

For example, a holding observed on 20 May and the same instrument observed on 21 May represent different positions. The composite key preserves both; casting the date into a SQL date type makes later dated joins less ambiguous. Decimal conversion preserves exact cents within the stated precision, whereas an uncontrolled floating-point aggregation can introduce comparison noise. Quantity has its own precision and meaning. The supplied valuation is not necessarily quantity multiplied by the supplementary price, so reconstructing it that way would silently change the teaching contract. Filtering a zero or negative amount also changes population semantics unless the contract explicitly requires that rule.

Challenge

The exact staging columns are `fund_id` (string, trimmed uppercase `fund_cd`), `position_date` (date from `pos_dt`), `instrument_id` (string retaining `isin`), `quantity` (decimal(18,4) from `qty`), `mv_local_ccy` (decimal(18,2) from `mkt_val_lc`), `local_ccy` (uppercase string from `ccy_cd`) and `_src_loaded_at` (retained `_extracted_at`). Do not manufacture a missing instrument key; this holdings contract requires it, with cash supplied separately.

Complete the holdings staging SQL in the copied model file. Read through the custodian source, trim and normalize the fund code, cast the position date, retain the instrument identifier, cast quantity and native market value to their stated decimal types, normalize currency, and carry the source extraction timestamp for diagnostics. Keep cleaning separate from enrichment joins. Use the prepared column contract to name the resulting fund, date, instrument, quantity, native value and local-currency fields.

Build the selected staging model as a view, then inspect all eight output rows alongside the original CSV. Check the entire composite key and compare the native value of a chosen holding exactly. Read the supplied grain test and extend it if your transformation introduces an additional normalization assumption. Document any filtering and prove it does not conceal an invalid key in the supplied fixture. Do not erase null-key evidence by substituting a plausible fund identifier or invent a reporting currency during staging.

> Check the three-field grain before changing population
>
> **Hint**
> Write the column mapping before the SQL so casts and renames can be reviewed independently. Compare the source and staging counts before joining any dimensions. For a duplicate-key failure, group on all three business fields; grouping only by fund will report valid repetitions as defects. Inspect Databricks full type metadata when proving decimal precision, because a broad type label may omit precision and scale.

Check

The staging view must return eight rows with eight unique fund/date/instrument keys and native market values matching the copied holdings input. Inspect the SQL type metadata and verify the intended decimal precision and scale, the date type and normalized currency. A count alone is insufficient: an invented key could preserve eight rows while breaking later joins. Compare a complete sorted key/value projection against the input mapping.

Run the grain test and inspect the selected build artifacts. In a separate copied-input variant set one native market value to zero, rebuild the selected staging slice and prove its business key remains among all eight rows with an exact zero value. Restore and reseed before baseline NAV comparison; changing an amount legitimately changes its expected value. In a disposable input copy, duplicate one holding key and show that the grain assertion detects it; restore the supplied CSV and reseed before the final build. Keep that negative result separate from the untouched starter's unfinished-model rejection. Explain why the same fund appearing on two dates should pass and why recomputing market value from quantity and price is outside this staging task. Retain the failed test identity, repaired selected run and one exact native-value comparison.

```
dbt build --select stg_holdings --project-dir . --profiles-dir . --target local
```

Phase contents · Reasoning key

Build a useful mart through ref

Concept

A mart combines modeled inputs into an output a consumer can interpret. Its grain must be declared before aggregation, because an apparently sensible total can conceal loss of dates or duplication from joins. Here the consumer result is one row per fund and position date, containing holding count, GBP exposure and holdings value per synthetic unit. The prepared enrichment provides native market value and the matching dated FX rate; the report sums converted holding values at that grain.

For the global-core fund on 20 May, the independent fixture calculation yields GBP exposure of 1,500,440,000 and holdings value per synthetic unit of 15.004400 after division by one hundred million. The next date is a distinct report row, not an update to that same key. GBP reporting is fixed even when fund master says a fund has another base currency. Cash positions are deliberately excluded from this teaching numerator. Calling the result official custodian NAV would overstate its meaning. A three-column output with a correct grand total can still be wrong for each fund/date, so compare complete keys and values rather than relying on an aggregate smoke check.

Challenge

Return `fund_id` and `position_date` as the report key, `holdings` as the count of enriched holding rows, `exposure_gbp` as decimal(28,2), and `nav_per_unit` as decimal(28,6). The supplied enrichment value field is `mv_base_ccy`; sum it at fund/date grain and divide that sum by 100000000 for the holdings value per synthetic unit. The name of each output is part of the consumer/checker contract.

Author the holdings-value report model through a reference to the supplied enrichment, grouping on fund and position date. Return the declared identifiers, holding count, GBP exposure and holdings value per synthetic unit with the stated decimal types. Keep the synthetic denominator explicit and explain the cash exclusion in the model documentation. Configure the consumer report as a table while leaving the staging and enrichment layers as views.

Before running SQL, calculate one selected fund/date result from the copied holdings and FX files. Then build the report with its upstream dependencies and compare every output key/value with the independent reference command. Inspect the manifest to show why the report waits for enrichment and holdings. If one row has the right exposure but the wrong date, repair the grain instead of relaxing the reconciliation. Preserve the supplementary custodian NAV fixture as a comparison input and state its intended teaching role.

> Declare the reporting grain and units before summing
>
> **Hint**
> Begin by selecting the enrichment columns and checking its row count before aggregation. The supplied native value is already a valuation; use the dated conversion rate rather than reconstructing price times quantity. Keep the denominator outside the sum so its units remain obvious. A missing FX rate should remain visible to a coverage test; replacing it with one would manufacture a convincing but unsupported answer.

Check

Build the report selection with its upstream graph, then run the independent reference over the copied CSVs. Expect eight report keys: four funds on each of two dates. Each holding count is one in this fixture. Check all exposures and NAV values, including global core on 20 May at 1,500,440,000 GBP and 15.004400 holdings value per synthetic unit. The independent output must use your actual copied inputs, not a pasted instructor SQL result.

Inspect warehouse relation metadata to show that the report is a table and holdings staging is a view. Record the selected build/run-results identity and explain the difference between materialization and business grain. A view/table configuration change alone should not change the expected values. If either date disappears or a fund gains two identical report keys, the output fails even when an overall sum is plausible. Explain why excluding cash is a declared limitation and why a different fund-master base currency does not change this report's GBP label.

```
dbt build --select +fund_nav_report --project-dir . --profiles-dir . --target local
python runtime/lab.py reference .
python runtime/lab.py check .
```

Phase contents · Reasoning key

Add minimum key and relationship proof

Concept

A test is useful when it protects a stated failure mode and its result can be interpreted. A uniqueness test checks row identity, a not-null test checks required presence, and a relationship test checks membership in another relation. None independently proves all joins preserve the intended population. A relationship assertion can ignore nulls, and a reconciliation can compare two outputs that already lost the same fund. Combining structural checks with independent business values closes different gaps.

The holding grain includes a date, while fund-master membership is checked by fund identifier. A date-free fund uniqueness test would therefore reject valid holdings. Conversely, a report sum could stay numerically unchanged if an unknown fund's zero-valued holding were silently filtered out; the population and relationship assertions should still catch that loss. For missing dated FX, the correct failure is incomplete conversion coverage, not a guessed one-to-one rate. The teaching drill deliberately changes a copied input so you can observe the failed assertion, repair its cause and prove the restored output. A red test is not automatically a successful drill unless it failed for the intended reason.

Challenge

Review the supplied schema and singular tests, then add or repair assertions that match the authored models: holding composite grain, required fund/date/instrument fields, fund membership, complete dated enrichment and independent NAV reconciliation. Explain which checks protect population, which protect type/units and which protect the consumer calculation. Run a clean seed and full selected build after both authored SQL files are complete.

Create the required relationship negative case in a disposable copy by changing a holdings fund to an unknown identifier; retain the specific membership failure. Also use the runtime helper to inspect the separately declared missing-FX coverage failure when that drill is selected. Capture the specific failed node and its diagnostic output. Restore the input, reseed and rebuild the affected dependencies, then rerun the independent comparison. Keep the original CSVs and canonical starter unchanged. A later correction/replay exercise has a different expected output contract, so do not reuse its intentional baseline reconciliation failure as this phase's final evidence.

> Predict the failed assertion before injecting a defect
>
> **Hint**
> Choose one defect at a time and name the test you expect to fail before running it. Inspect failed rows or compiled SQL to distinguish a genuine coverage failure from an unrelated unfinished model. A relationship check on fund identifiers needs a separately unique fund dimension; the holdings themselves are not unique by fund. Keep the fixture restore and selected reseed in your note so the repaired run can be reproduced.

Check

The final baseline build must pass the structural and business assertions and reproduce the independent eight-key report. The negative run must fail on the promised relationship, grain or missing-enrichment assertion, and the repaired run must pass after restoring and reseeding the copied input. Record all three states with their relevant artifact identities; replacing the failure log with the final green log loses the learning evidence.

Use the runtime helper only on your owned attempt, then select the affected seeds and upstream graph deliberately. Inspect the failing test result rather than inferring its cause from a nonzero command exit. Explain why the report's own total cannot be its only oracle and why not-null and relationships need complementary tests. The root phase verifier may validate finished warehouse output, but your explanation and negative-case evidence remain part of the completion standard. If warehouse access is absent, prepare the drill and its expected assertion locally while leaving execution pending.

```
dbt build --project-dir . --profiles-dir . --target local
```

```
python runtime/lab.py mutate . --drill missing-fx
# Reseed/rebuild and retain the intended failure, then restore the copied input.
python runtime/lab.py restore .
```

Phase contents · Reasoning key

Phase completion

Submit the authored holdings staging model, fund/date report, structural and business tests, and an evidence note that connects the raw feeds to the consumer result. Your explanation must distinguish the raw business key from a model name and the reporting currency from a fund master attribute. Show how you derived one result from the copied CSVs before looking at the warehouse output. Include a selected build, its manifest and run-results identities, the independent comparison, and the intended relationship failure followed by repair.

A successful seed operation establishes input loading; a parsed project establishes that dbt understands the files. Neither establishes this learning outcome. The completion verifier must reject unfinished models and compare actual warehouse values with the independent reference. Keep any unavailable connected check pending. Retain your attempt rather than replacing it with the instructor project, and explain which supporting models were supplied. You may enter the advanced phase through its prepared baseline without copying another learner's work. That baseline is disclosed assistance, not evidence that you authored its components.

Optional stretch

The core completion standard is independent of optional work. No additional stretch is required in this phase; the next phase has an independently prepared entry baseline.

dbt phase reasoning keys

Reasoning keys

Read these explanations after attempting the checks.

Reasoning: Inspect the project and source grain

The expected inventory contains two observations for each of four funds. The composite holding key includes date and instrument, so an eight-row source can be valid even though fund identifiers repeat. Source declarations map logical inputs to already loaded relations; they cannot replace seeding. A target named local describes configuration selection, while its type selects Databricks execution. Good evidence separates CSV inspection, connection readiness and actual warehouse population, retaining an unavailable boundary as pending instead of combining them into one vague pass.

Return to Inspect the project and source grain

Reasoning: Declare and use sources

A source call declares an external input; a model reference declares a derived dependency. The identifier expression resolves the source into an owned physical relation without putting environment-specific names into every model. Missing input and wrong source mapping can produce similar first errors, but their repairs differ. The manifest plus seed inventory distinguishes them. Reference data can be loaded and inspected without forcing it into this bounded consumer graph. The complete chain is evidence of understandable lineage, not proof that a compiled query has executed.

Return to Declare and use sources

Reasoning: Normalize type, unit and key contracts

The expected transformation changes representation without losing either May date. Fund/date/instrument is the row identity; fund alone is a reporting dimension. Correct decimal and date types support later business calculations, but do not prove the join coverage by themselves. The duplicate-key drill should fail because it creates a second observation at the same declared grain.

Restoring the copied raw input repairs that cause. Multiplying quantity by price would create a different valuation measure and must not replace the supplied native value.

Return to Normalize type, unit and key contracts

Reasoning: Build a useful mart through ref

The correct report has eight rows because date is part of the consumer key. Its GBP numerator uses the supplied native valuation and a matching dated FX rate. The synthetic unit count creates a holdings value per synthetic unit measure; the cash exclusion and currency choice must travel with the result. A table materialization changes storage and refresh behavior, not the expected business values. Full per-key comparison detects collapsed dates and shifted fund totals that a single grand-total check could miss.

Return to Build a useful mart through ref

Reasoning: Add minimum key and relationship proof

Structural assertions and the independent reference protect different risks. An unknown fund should fail membership or retained-population checks; a missing FX observation should fail dated enrichment coverage rather than be replaced by a rate of one. The useful evidence connects the controlled change to the actual failed test, then connects restoration and reseeding to the recovered eight-key report. A generic command failure or a final green screenshot cannot establish that sequence. The untouched starter remains unfinished until the authored outcomes and their checks are complete.

Return to Add minimum key and relationship proof

dbt: Phase 2 — advanced practice

Purpose and outcome

Develop the foundation pipeline into a reviewable data-product change. The core begins with dated-join diagnosis and repair, then adds a business reconciliation, controlled incremental replay and an explained feature/history handoff. The starter supplies the supporting models, so a completed foundation attempt is not required.

Dataset and pipeline

The advanced starter uses the same six raw and two supplementary Portfolio Analytics CSVs as the foundation phase. Staging and supporting models are supplied; enrichment, its coverage/reconciliation tests, the reusable value macro, replay model, fund snapshot and final report contain declared unfinished outcomes. The two May business dates provide a concrete date-join risk. Joining a holding to a price by instrument alone can match both price dates, increasing the population while appearing to add useful price information. Missing dated FX creates a different risk: values become unmatched instead of duplicated.

The source valuation remains authoritative native market value. GBP conversion uses the corresponding dated currency rate; the holdings value per synthetic unit denominator stays one hundred million synthetic units and excludes cash. A controlled correction is a copied input change, not evidence that the supplied fixture already contained historical corrections. A controlled fund-name change similarly creates snapshot history for the exercise rather than claiming history already exists.

Prerequisites and workspace

Start in the training project root with the delivered Databricks environment, approved catalog/schema and an owned prefix different from another active attempt. Use the explicit advanced prepare task, then run direct dbt commands from the copied directory. The helper copies supporting files and runtime checks alongside your attempt; no authoring-repository path is needed once prepared. Record the actual adapter version and inspect the profile's local/dev/airflow Databricks targets before choosing one.

```
mise run challenge:v2:prepare dbt phase-two-proficient
cd .local/challenges-v2/dbt/phase-two-proficient
export DC_RELATION_PREFIX=dc_alex_dbt02
dbt debug --project-dir . --profiles-dir . --target local
```

Changing an approved schema variable does not grant permission to create a new schema. All seeds, models and snapshots use the declared relation prefix in the single provisioned namespace. Keep credentials out of evidence and perform sign-in outside any recording. If the warehouse is unavailable, author and inspect locally while leaving merge/history execution pending.

Starter behavior

The supplied staging layer is a prepared successful entry fixture, not your assessed work. Advanced focus models and tests are intentionally unfinished and completion verification rejects them. Seeding is possible before downstream authoring; a broad build can fail on the named unfinished nodes. Diagnose the first failure rather than selecting every dependent model while editing only one. Inspect the checkpoint inventory to distinguish the new task gaps from a wrong source or profile. Negative drills must change only copied data and name their intended failed assertion. Restore source inputs after each drill before using the baseline NAV reconciliation again.

Timeline and completion standard

Core reading, authoring, checking and explanation: 90–120 minutes. Allow additional time for setup and optional stretch.

- Task: Diagnose a join fan-out before repair · Minutes: 18–24 · Evidence: Eight input keys; date-free price join yields sixteen matches; dated repair retains eight

- Task: Repair enrichment at the intended grain · Minutes: 18–24 · Evidence: Eight enriched keys; explicit matched-price/FX coverage and decimal GBP value
- Task: Write reconciliation that detects the negative case · Minutes: 18–24 · Evidence: Coverage and NAV tests fail on copied missing-FX input and pass after restoration
- Task: Prove incremental update and replay behavior · Minutes: 18–24 · Evidence: Older-date correction retains eight keys; a new holding inserts a ninth; replay remains stable
- Task: Explain dependency impact and hand over · Minutes: 18–24 · Evidence: Declared fund report, meaningful snapshot intervals, lineage and explicit Astro boundary

Independent starter files Read the owning phase before editing. Prepare from the training project root with `mise run challenge:v2:prepare dbt phase-two-proficient`; edit the copied attempt.

Authored-file list

Task	*Learner-authored paths*
Diagnose a join fan-out before repair	`models/intermediate/int_holdings_enriched.sql`, `evidence.md`
Repair enrichment at the intended grain	`models/intermediate/int_holdings_enriched.sql`, `macros/native_value.sql`, `evidence.md`
Write reconciliation that detects the negative case	`tests/matched_enrichment.sql`, `tests/nav_reconciliation.sql`, `models/schema.yml`, `evidence.md`
Prove incremental update and replay behavior	`models/marts/holdings_replay.sql`, `evidence.md`
Explain dependency impact and hand over	`models/marts/fund_nav_report.sql`, `snapshots/fund_history.sql`, `models/schema.yml`, `evidence.md`

Declared baseline

These focus files are intentionally unfinished: `macros/native_value.sql`, `models/marts/holdings_replay.sql`, `snapshots/fund_history.sql`, `models/marts/fund_nav_report.sql`, `models/intermediate/int_holdings_enriched.sql`, `tests/matched_enrichment.sql`, `tests/nav_reconciliation.sql`. Supporting models and profiles are supplied. Complete the listed files, then verify their output in the required Databricks or Astro environment.

Keep `evidence.md` beside the copied project. Create a named file when its task asks for it. Use a fresh workspace for another attempt; preparation and reset preserve existing work.

Core challenges

Diagnose a join fan-out before repair

Concept

A join's correctness depends on the relationship between its keys and the grain of each input, not on whether SQL executes. Holdings are at fund/date/instrument grain; prices are at instrument/date grain. An instrument-only price join can attach both May prices to every holding and duplicate the holding's native value. The resulting consumer sum may look plausible because it contains familiar columns and positive amounts, but it measures a fan-out rather than the intended position population.

Count the holding keys before enrichment and the matching price rows per key afterward. With this supplied fixture the date-free price join yields sixteen matches from eight holdings; including the position/price date restores one dated price per holding. An unmatched price or FX observation has a different symptom: a left join retains the holding with null enrichment, while an inner join can hide it by dropping the row. A correct pipeline must detect both duplication and loss. The appropriate diagnosis names which input relationship violated its key contract before choosing a repair. Merely adding distinct to the final report can hide multiplicity without selecting the right dated observation.

Challenge

Use the prepared staging inputs to reproduce a bounded date-free price join in a diagnostic query. Count input holdings, joined rows and the maximum matches per holding key. Inspect a representative instrument on both May dates and explain why the incomplete predicate duplicates its value. Keep this deliberately defective diagnostic separate from the consumer model and never treat it as the final answer.

Write the intended enrichment contract in your evidence note: holding grain remains unchanged, price matches by instrument/date, FX matches by native currency/reporting currency/date, and missing matches must be testable. Inspect the unfinished enrichment file and identify the selected source columns and required output fields before repairing it in the next task. Record the original diagnostic result and one key's two candidate prices. If a dependency is not yet built, build only its supplied upstream staging slice instead of copying an instructor enrichment model.

> Expose the duplicate match before changing the mart
>
> **Hint**
> Start with the smallest join that can demonstrate the defect: holdings plus prices, without FX or aggregation. Compare one holding's date with both price dates. Group the diagnostic by the complete holding key and inspect the match count. A grand total is a later symptom; the duplicated key is the direct cause. Do not repair the join by selecting a latest price when the contract requires the same business date.

Check

Run the prepared upstream staging build, then inspect the defective and corrected diagnostic queries in the warehouse. Expect eight holding keys, sixteen rows from the instrument-only price join, and eight from the dated join. Confirm the discrepancy is actually explained by matching both fixture dates rather than by a duplicate raw key. Retain the query text, execution identity and a representative matching pair.

Explain why distinct over a selected projection might conceal this defect and why dropping unmatched rows would create another one. The negative diagnostic is successful teaching evidence only when it produces the predicted multiplicity. An authentication error or unfinished-model compile failure does not prove fan-out. Restore the intended predicate before any final consumer build, and use the later grain/coverage checks to enforce it. If the warehouse cannot run, retain the inspected fixture keys and diagnostic SQL with execution explicitly pending; a local prediction is not observed warehouse evidence.

```
dbt build --select stg_holdings stg_prices stg_fx_rates stg_funds --project-dir .
 ↪  --profiles-dir . --target local
```

Phase contents · Reasoning key

Repair enrichment at the intended grain

Concept

An intermediate model isolates business relationships that would otherwise be repeated in every consumer query. Its output contract is more important than its position in a folder: here it retains each typed holding and attaches a dated price, dated GBP conversion rate and converted value. Left joins make missing observations visible to a coverage assertion. They do not excuse incomplete data; they preserve evidence needed to reject it.

A reusable expression is valuable when it states an actual shared rule, such as converting a decimal native valuation by an explicitly matched rate with an agreed output precision. That macro should compile into understandable SQL and should not decide the join keys or manufacture a default rate. For example, a missing May 20 GBP rate must remain null and fail coverage even though GBP-to-GBP is normally one. The feed's declared observation is part of the readiness contract. Fund master may describe a USD base currency while this report remains explicitly GBP. Moving environment routing or relationship selection into an opaque macro would make the model harder to review and would not add useful abstraction.

The supplied `int_fund_attributes` and `int_positions_dated` retain v1's two smaller intermediate contracts. The first filters active funds and exposes their reporting attributes; the second adds cash balance and accrued interest once at fund/date/currency grain. All four raw fund-master entries are active here, and the positions helper retains eight dated rows. These are supplied supporting models to inspect rather than additional unfinished SQL assignments. Cash remains outside the declared holdings value per synthetic unit, so an available cash field does not authorize silently adding it to that report.

Challenge

Retain the seven staging fields and append `close_price`, `fx_rate_to_base` and `mv_base_ccy`. Prices provide `instrument_id`, `price_date` and `close_price`; FX provides `from_ccy`, `to_ccy`, `rate_date` and `fx_rate`. Match the holding instrument/date to prices and its local currency/GBP/date to FX. Use `native_value` for the decimal GBP amount and retain the same holding key.

Complete the enrichment model with the declared dated price and FX predicates, preserving the holding fields and appending the expected enrichment columns. Implement the native-value macro with explicit decimal conversion and use it for a meaningful repeated expression. Keep input relationship logic visible in model SQL so a reviewer can see instrument/date and currency/date matching without tracing several macro layers.

Build enrichment as a view and inspect its row/key population before building the consumer report. Document the distinction between the supplied close price, native valuation and converted GBP amount. Show the rendered macro expression in compiled SQL and verify that the resolved source/model references still use your owned namespace. If you choose a different helper signature, preserve the same units and precision and explain that bounded change. Do not replace missing FX with a rate of one or derive native market value from quantity times price.

Build and inspect the two supplied intermediate helpers as well. `int_fund_attributes` exposes `fund_id`, `fund_name`, `inception_date`, `base_ccy` and `strategy_code` for active funds. `int_positions_dated` exposes `fund_id`, `position_date`, `local_ccy` and `cash_plus_accrued`. Trace their refs and explain why filtering or unit arithmetic belongs in one named model rather than being repeated independently by each consumer. Use cautious indirect test selection while downstream focus models are unfinished.

> Keep joins visible and reuse only the value rule
>
> **Hint**
>
> Write the two join predicates separately and check each candidate key against the input grain. Add the conversion expression only after the joined fields make sense. Compile the macro in a selected model and read the emitted SQL as ordinary arithmetic; useful reuse should make the rule easier to inspect. Keep null coverage distinct from numeric reconciliation so a missing match cannot disappear inside an aggregate sum.

Check

The enriched view must retain eight holding keys with exactly one dated price and FX observation per key and decimal GBP values consistent with the independent calculation. Inspect selected rows and the actual relation metadata. The compiled macro expression should use the supplied native value and matched rate without hidden environment names or fallback currency assumptions.

Check four active helper fund rows and eight cash/accrued rows. Compare one helper amount directly with its copied source's cash balance plus accrued interest, and explain why it is excluded from the current NAV. Distinguish a legitimate supporting-model build from a relationships test that prematurely selects an unfinished downstream consumer.

```
dbt build --select int_fund_attributes int_positions_dated --indirect-selection=cautious
 ↪  --project-dir . --profiles-dir . --target local
dbt show --select int_positions_dated --limit 8 --project-dir . --profiles-dir . --target
 ↪  local
```

Temporarily remove the copied May 20 GBP FX observation, reseed that input and rebuild enrichment. The population can remain eight while matched coverage fails for the affected observations; do not interpret

retained row count as a complete conversion. Restore the copied input and rebuild before comparing baseline values. Explain why the relationship predicates remain in the model, why a left join plus an explicit failure check is useful, and why reporting currency is not inferred from the fund master. Retain the compiled expression, missing-match result and repaired enrichment output as separate pieces of evidence.

```
dbt build --select int_holdings_enriched --project-dir . --profiles-dir . --target local
python runtime/lab.py mutate . --drill missing-fx
```

Phase contents · Reasoning key

Write reconciliation that detects the negative case

Concept

A business reconciliation compares a consumer result with an independently specified meaning. It should detect wrong units, key loss, joined duplication and changed values rather than simply compare two sums built from the same faulty model. The supplied custodian NAV fixture and Decimal CSV reference give concrete expected results for the unmodified teaching population. Key membership is as important as numeric closeness: a report missing one date should fail even when another row absorbs its value.

A negative case tests the assertion's sensitivity. Removing a copied FX observation should fail enrichment coverage; feeding that incomplete result into a holdings-only NAV report should not produce accepted output. Another controlled experiment intentionally changes a holding's value. That makes baseline NAV comparison fail for a legitimate reason until the variation's expected output is used. These cases require different interpretations. A test message without the input variant and failed node cannot tell a reviewer whether it detected a real defect, an intended correction or an unrelated connection issue. Keep the assertion's unit, tolerance, key set and applicable fixture state explicit.

Challenge

Author the matched-enrichment and NAV reconciliation SQL tests in the copied project. Preserve the complete holding and report key sets, test non-null matched price/rate fields, and compare the report against the declared baseline expected NAV with an appropriate decimal tolerance. Add descriptions explaining the reporting currency, synthetic unit denominator and cash exclusion. Use structural schema tests as complementary checks rather than replacing the business assertion.

Predict which node will fail when the copied May 20 GBP FX row is removed. Execute that mutation, selected reseed and build/test sequence in your owned attempt, then retain the failed node and diagnostic rows. Restore the source copy, reseed and rerun the same graph. Record the repaired comparison and a brief explanation of why a baseline oracle is inappropriate for the later deliberate value correction. Do not change the expected fixture merely to make a defective model agree with itself.

> Compare membership before tolerating numeric differences
>
> **Hint**
> Compare keys before values and state the tolerance in money or NAV units. A left join from expected keys to actual results can expose missing report rows, while the reverse membership check exposes invented ones. Inspect the first failed assertion and its compiled SQL. If it only compares a grand sum, ask whether two offsetting errors or shared row loss could still pass. Strengthen the contract rather than adding a looser threshold.

Check

Run both authored assertions on an unmodified complete project and compare the report to the independent CSV reference. All eight report keys and their GBP/NAV values must agree. After the missing-FX mutation, the expected coverage assertion must fail because the declared rate is absent; an unrelated unfinished macro or profile failure is not the intended result. Restore and reseed before the final passing run.

Retain the original and mutated source hashes, failed node, observed failed rows, repair diff and final run-results identity. Explain how the test detects a missing date and how its tolerance handles decimal representation without accepting a wrong monetary result. Record that the subsequent corrected-value variant changes the independent oracle rather than the definition of the baseline input. A verifier can inspect values and artifacts, but your explanation must connect the failing contract to the controlled cause. Keep actual warehouse execution pending if only the assertion text and fixture arithmetic have been prepared.

```
dbt test --select matched_enrichment nav_reconciliation --project-dir . --profiles-dir .
 ↪  --target local
python runtime/lab.py restore .
python runtime/lab.py reference .
```

Phase contents · Reasoning key

Prove incremental update and replay behavior

Concept

Incremental materialization changes how a relation is maintained; it does not automatically make a pipeline correct under late data. A merge needs a stable unique key that identifies the business observation being replaced. For holdings that key includes fund, date and instrument. Choosing only a fund would overwrite another date, while appending without key matching would duplicate an existing observation when the same input is replayed.

This exercise creates a correction to the first copied May holding by increasing its native market value by one thousand. The selected observation is GBP-denominated, so its independently expected GBP value changes by the same amount. The date remains the older fixture date. A filter that processes only dates greater than the current maximum would miss this legitimate correction. Replaying the corrected input twice should produce the same eight-key state after both merges, with one changed value and no duplicate. A second controlled input appends a new holding key: PEF-EM-GROWTH, May 20, GB00B3M9JJ78, with native and GBP value 500. Its price and FX already exist, so the result must grow to nine keys with total exposure another 500 higher. Replaying that combined input must retain nine, proving insertion as well as update. The experiment is bounded to your namespace and copied fixture; it does not prove a general production late-arrival policy, transaction isolation or concurrent writer behavior. Those limitations belong in the handoff.

Challenge

The replay relation must expose `fund_id`, `position_date`, `instrument_id` and `exposure_gbp`, with the first three fields forming its unique key. Compute exposure from the matched native value/rate at decimal(28,2) precision; the independently queried checker uses these exact output names.

Implement the holdings replay model with Databricks incremental merge behavior and the composite holding key. Select enough of the current copied input to include an older-date correction rather than blindly filtering above the maximum stored date. Build an initial owned relation and retain its sorted key/value projection. Apply the controlled correction helper to the copied seed, reseed the changed input and run the replay model twice.

After the correction check, apply the new-holding helper to append the specified unseen key. Reseed and run the replay model twice again. Compare both nine-key states with the independently calculated input, including the retained older-date correction.

After each merge, compare the warehouse relation with the independent corrected input calculation. Record the exact changed key, original and corrected amount, unchanged date and overall row count. Keep the consumer baseline NAV comparison separate: its unmodified custodian fixture intentionally no longer agrees with the corrected holdings. Restore the raw copy after collecting the variation evidence,

and state whether the owned replay relation has been rebuilt to baseline. Never reset another learner's prefix or call an empty-table first build proof of incremental replay.

> Prove the older observation participates in both merges
>
> **Hint**
> First show that the selected business date is older than the maximum date already stored. Then inspect the unique-key configuration and the compiled merge predicate. Build from the full tiny copied input for this teaching experiment so a date watermark cannot hide the correction. The decisive check is the second corrected run: identical corrected input should retain eight keys; after the explicit arrival, identical combined input should retain nine. It must not add a tenth row or alter an unrelated key. Keep value-change evidence alongside membership evidence.

Check

Expect eight unique holding keys before the drill, after the corrected merge and after the second replay. Exactly one older-date GBP value should increase by one thousand; all other keys and values remain unchanged. After the new-holding drill, expect nine unique keys and exactly 500 additional GBP exposure; the second replay must retain the same nine-key/value projection. Query actual Databricks output and use the runtime replay checker, which calculates expected values from the copied holdings and FX inputs. A parse or local SQL emulation cannot prove Delta merge execution.

Retain the first and both corrected run-results identities, key/value projections, copied input hashes and the independently identified changed key. Explain why a maximum-date-only filter would reject a valid correction and why replay should not duplicate it. The ordinary baseline NAV assertion is expected to disagree while this variation is active; do not disable it permanently or change the baseline fixture to conceal that distinction. Restore source inputs and disclose the final relation state in your handoff. If connected execution cannot occur here, the implementation and expected transition can be reviewed locally but this check stays pending.

```
python runtime/lab.py mutate . --drill correction
dbt seed --select holdings_raw --project-dir . --profiles-dir . --target local
dbt run --select holdings_replay --project-dir . --profiles-dir . --target local
python runtime/lab.py check-replay .
# Repeat the run/check above, then prove insertion and replay.
python runtime/lab.py mutate . --drill new-holding
dbt seed --select holdings_raw --project-dir . --profiles-dir . --target local
dbt run --select holdings_replay --project-dir . --profiles-dir . --target local
python runtime/lab.py check-replay .
# Repeat the run/check above once more. Restore the copied input and owned table.
python runtime/lab.py restore .
dbt seed --select holdings_raw --project-dir . --profiles-dir . --target local
dbt run --select holdings_replay --full-refresh --project-dir . --profiles-dir . --target
 ↪  local
python runtime/lab.py check-replay .
```

Phase contents · Reasoning key

Explain dependency impact and hand over

Concept

An end-to-end feature needs a consumer question, output grain, units, lineage and evidence that someone can review. Generating another SQL file is insufficient if its result cannot be interpreted. A small fund-level report can answer how the two supplied dates differ in holdings exposure; it must label that observation difference without pretending it is investment return. The fixture lacks the flows and complete unit history needed to establish that stronger measure.

Attribute history is a separate feature. A snapshot can retain successive fund-name versions using a stable fund key and validity intervals. A copied fund-master name change creates the exercise's second observed version; it is not a correction to the holding value and does not change the reporting date. One current interval and one closed interval for the changed key demonstrate the intended

history. Documentation connects these decisions to the next consumer and to Astro's worker boundary: the terminal's successful build does not prove worker executable paths, mounted files or task-time credentials. A useful handoff states what was actually executed and what still needs connected or human review.

Challenge

Use raw `fund_cd` as the snapshot key and track `fund_name`, `base_ccy`, `strategy_code` and `is_active`. Inspect the generated `dbt_valid_from` and `dbt_valid_to` columns for the changed key; the active version has an open validity end under the supplied snapshot configuration. The final report retains the five fund/date fields stated in foundation task Build a useful mart through ref.

Complete the report model and its documentation, keeping a useful bounded feature based on available fund/date inputs. State the consumer question and distinguish exposure difference from financial performance. Add descriptions and tests for the report key, GBP values and holdings value per synthetic unit limitation. Inspect lineage showing sources, staging, enrichment and the new consumer output.

Implement the fund snapshot in the approved namespace with fund identifier as its unique key and the declared changing attributes. Run an initial snapshot, mutate only the copied fund name, reseed and snapshot again. Query current and closed validity intervals for that key, then restore the copied baseline input while preserving captured history evidence. Assemble a reviewable diff and a short Astro handoff listing the project/profile paths, selected target, owned prefix, required worker executable and pending task-time authentication checks. Full runnable instructor references are comparison material after your attempt, not a substitute for your authored feature.

> Separate reporting meaning, attribute history and worker readiness
>
> **Hint**
> Choose a report whose inputs really support its claim. For history, inspect the snapshot key and check-column list before changing a copied fund name; changing a nontracked field should not create the expected new version. Query the changed key rather than assuming a larger snapshot row count proves valid intervals. Keep the source restoration and historical table state distinct so the handoff explains both.

Check

The final baseline report must reproduce all eight independently specified fund/date values and show understandable lineage and documentation. Your additional feature needs an independent calculation for its own declared output. The snapshot experiment must show a closed previous interval and one current interval for the changed fund, with unchanged fund identity. Query the actual owned snapshot relation; compilation cannot establish validity history.

Retain the report/test run-results identities, feature expectation, snapshot before/after projections and bounded diff. Explain the meanings of reporting date, ingestion time and snapshot validity time. Include actual terminal adapter/target details and mark worker mounts, task-time token delivery, managed versions and scheduled run evidence pending unless observed. Restore the copied source data and describe whether the snapshot intentionally retains the teaching variation. Query the Delta and snapshot results in Databricks; include worker connection requirements in the Astro handoff.

```
dbt snapshot --project-dir . --profiles-dir . --target local
python runtime/lab.py mutate . --drill fund-change
dbt seed --select fund_master_raw --project-dir . --profiles-dir . --target local
dbt snapshot --project-dir . --profiles-dir . --target local
dbt docs generate --project-dir . --profiles-dir . --target local
```

Phase contents · Reasoning key

Phase completion

Finish the advanced core with an enrichment that retains every declared holding key, business checks that reject missing matches, a useful reusable decimal expression, and a consumer report whose baseline values agree with the independent CSV calculation. Demonstrate a controlled correction to an older business date, replay it twice through the incremental model, and show that the corrected key appears once. Append the specified new holding and prove nine keys after both insertion and replay, then rebuild the owned replay table to the eight-key baseline after restoring copied inputs. Demonstrate a separate fund-name change through snapshot validity. Keep those experiments distinct: replay updates an observation, while a snapshot retains changing attribute versions.

Your handoff contains the authored files, a bounded diff, failed and repaired assertions, manifests/run results, independent key/value comparisons, and a short explanation of each supplied model. Include a small fund-level reporting feature whose population and units are explicit. Do not manufacture an investment-return measure from two valuation dates without the necessary flow inputs. Preserve baseline fixture bytes and document restoration after variations. Source preparation, parsed SQL and locally calculated expected values remain supporting evidence until the Databricks build, Delta merge and snapshot queries execute. The Astro handoff lists the worker paths and connection checks needed for the Airflow route.

Optional stretch

The core completion standard is independent of optional work. Choose a separately timed route from the stretch guide.

dbt phase reasoning keys

Reasoning keys

Read these explanations after attempting the checks.

Reasoning: Diagnose a join fan-out before repair

Both May price observations share an instrument identifier. The date-free join therefore doubles matching holdings even though raw keys are valid. Matching the business date repairs that particular predicate. A missing match remains a separate coverage concern and should not be solved by dropping the holding. The diagnostic evidence must show input keys, actual multiplicity and the corrected relation, so the reviewer can understand the cause without trusting a final aggregate.

Return to Diagnose a join fan-out before repair

Reasoning: Repair enrichment at the intended grain

The intermediate output should have the same holding grain as staging. Its left joins preserve unmatched observations for diagnosis, and the coverage test rejects them before a consumer result can be accepted. The decimal macro expresses a valuation rule rather than a relationship rule. A rate-of-one fallback would hide a missing declared feed observation; inferring currency from fund master would change the report contract. Both mistakes can yield plausible numbers, so inspect compiled arithmetic and matched keys together.

Return to Repair enrichment at the intended grain

Reasoning: Write reconciliation that detects the negative case

The useful reconciliation owns its expected keys and units independently of the enrichment. Missing FX should be rejected as coverage loss; restoration repairs that input cause. The correction exercise later changes an input value deliberately, so its expected result is recalculated from the copied input rather than accepted against a stale baseline NAV. A test that merely matches the mart to another aggregate of the same broken model would miss shared loss or duplication. Record the variant and failed assertion together.

Return to Write reconciliation that detects the negative case

Reasoning: Prove incremental update and replay behavior

> The merge key identifies the exact dated holding. A copied older-date correction must reach that key even after the later date exists, and running the same correction again must leave the eight-key result unchanged. A new PEF-EM-GROWTH/instrument key then grows the relation to nine, adding 500 GBP without losing the correction; its replay stays at nine. Restoring seed inputs alone cannot delete that inserted target key, so the final owned full-refresh is an explicit restoration step. The independent replay checker uses the modified input, while the custodian NAV fixture still describes baseline data. Delta behavior requires actual warehouse execution and cannot be inferred from compiled SQL. A good report retains before/after state and discloses whether restoration also rebuilt the owned relation.
>
> Return to Prove incremental update and replay behavior
>
> **Reasoning: Explain dependency impact and hand over**
>
> The feature should answer a supported question with a declared key and units. Two exposure observations do not automatically establish return. Snapshot history records a separate changing-attribute contract and needs actual validity queries after a controlled name change. Restoring raw inputs does not erase captured history. Lineage, tests and a reviewable diff explain the terminal result, while the Astro handoff identifies worker paths and authentication that remain unverified until scheduled execution. These distinctions make the product interpretable rather than merely executable.
>
> Return to Explain dependency impact and hand over

Reasoning keys follow their owning phase.

Airflow — Portfolio Analytics

Projects contents · Task index

- Foundation core — page 315
- Advanced core — page 324

The consumer measure is holdings value per synthetic unit: supplied holdings-only GBP exposure divided by 100 million synthetic units, with cash excluded. It is not assets less liabilities or official NAV per outstanding fund share. Compatibility identifiers `fund_nav_report`, `nav_per_unit` and `seed_custodian_nav` remain unchanged. The fund/date reporting grain does not establish manager-sleeve ownership.

airflow challenges

Airflow schedules the same supplied Portfolio Analytics graph used in the dbt projects. The prepared dbt baseline keeps authoring focused on readiness, real worker invocation, quality and run-bound evidence. Foundation work establishes the four-task chain and current scheduled success/failure/repair. Advanced practice develops explicit dates, freshness, bounded retries, replay correctness, worker/auth paths and an incident handoff. A green graph must be connected to independently checked warehouse output.

Before you start

Prepare an owned attempt through mise in the training project, then inspect actual Astro mounts, worker executable and approved Databricks namespace. Choose a unique prefix/DAG identity and stage only through the helper that refuses an occupied destination. Seed the copied supporting project through the terminal and explicitly handle extraction-time freshness without changing May business dates. Worker credentials resolve at execution through the delivered connection; a terminal login alone is insufficient. Pause only the owned DAG after a bounded rehearsal and preserve evidence before scoped cleanup.

Suggested timeline and tasks

Estimates include reading, practice, checks and explanation. Allow additional time for environment setup and optional extensions.

- Phase: Foundation · Task: Inspect a prepared orchestration baseline · Minutes: 15–22
- Phase: Foundation · Task: Author a small Python DAG with real dbt tasks · Minutes: 15–22
- Phase: Foundation · Task: Demonstrate the dependency graph and scheduled run · Minutes: 15–22
- Phase: Foundation · Task: Inspect a controlled task failure · Minutes: 15–22
- Phase: Foundation · Task: Explain the run and environment boundaries · Minutes: 15–22
- Phase: Advanced · Task: Define bounded replay and input identity · Minutes: 18–24
- Phase: Advanced · Task: Inject failure and observe useful retry behavior · Minutes: 18–24
- Phase: Advanced · Task: Prove output correctness after recovery · Minutes: 18–24
- Phase: Advanced · Task: Use Astro UI to make an operational decision · Minutes: 18–24
- Phase: Advanced · Task: Write a short incident handoff · Minutes: 18–24

Choose your phase

- Foundation: Foundation core with explicit completion evidence.
- Advanced: Independent advanced baseline and finite core, separately timed.

Finish the core tasks before choosing optional stretch.

Teaching concepts

Schedule interval, business partition and extraction freshness are independent clocks. The tiny dbt build refreshes the complete fixture, while the receipt selects four fund rows for a named May date. Task ordering protects evidence after quality, but real run/task identity and independent values establish execution. Retries must fit their failure category. End-of-phase keys explain cause and recovery; full runnable DAG references stay instructor-only. Optional Cosmos or hosted monitoring has separate prepared-runtime and timing requirements.

Question-led foundations and further study

Use questions 1–8 alongside the foundation tasks. Questions 9–13 introduce further study.

Foundations

What problem does Airflow solve alongside dbt?

The Portfolio Analytics workflow orders source readiness, transformation/testing and publication evidence. dbt provides the model graph; Airflow controls when and under what conditions it runs. Databricks remains the data execution boundary. A final evidence task records a successful checked handoff and never performs production publication.

What are a DAG, task, operator, run and task instance?

The owned DAG is `<prefix>_portfolio_daily`. Its named tasks are source_ready, dbt_build, quality and publication_evidence. The DAG describes dependencies; each scheduled run has a distinct ID, logical date and instances of those tasks. Record their states from the fresh run rather than presenting an unrelated green grid. An operator is the implementation used to execute one task.

Which components parse, schedule, execute and record the work?

The terminal prepares files and seeds. Astro mounts data_products into the DAG processor/scheduler/worker path. The scheduler creates timed runs; workers execute tasks; the metadata database records orchestration state; Databricks executes dbt SQL. Inspect the actual compose mount and worker executable. Files visible in your terminal outside that mount are not automatically visible to a worker.

How do we define dependencies and pass information?

Wire source_ready » dbt_build » quality » publication_evidence. Pass only a small partition/run identifier, never holdings through XCom. Every task checks the explicit May sample partition while preserving the actual logical date separately. Failure propagates through dependencies, not through a print statement saying the work is ready.

What do schedules, logical dates, intervals and catchup mean?

The owned classroom DAG has an explicit two-minute timetable, a timezone-aware start date, catchup=False and one active run. A scheduler-created interval is not the May business date in these historical CSVs. The fixture_partition parameter selects 2026-05-20 or 2026-05-21 explicitly. Record both the logical interval and selected partition. Unpause only this owned DAG during rehearsal and pause it after collecting evidence. A daily production cadence is a separate design choice. Replay the same business date to demonstrate idempotence rather than assuming each scheduler day creates new data. The prepared tiny dbt graph refreshes both fixture dates; the receipt selects four fund rows for the named partition. Do not describe that selection as date-filtered ingestion.

What should happen when a task fails or retries?

Bound retries and retry delay for transient connection failures. A deliberately missing FX rate or quality-fault marker must fail quality and block publication_evidence. Use logging-only callbacks. Keep the old accepted receipt unchanged on failure and prove that no receipt was created for the failed run. An auth/config error differs from business-quality failure and should be diagnosed from its first log error.

How do reruns and backfills produce correct results?

Replaying a business date reruns the same owned relations and compares their keys/values. Retain run-specific receipts and show which date they represent. A controlled holding correction exercises Delta merge in advanced dbt practice; do not claim a second green task proves idempotence. Restore baseline source data after the drill. Clearing a task is an operational action on a named run, not a repair to its input contract.

How do we inspect a run and prove completion?

Keep the current Astro graph/grid/log, exact run/task/attempt IDs, dbt run results and independent warehouse query. The publication receipt binds the contract, fixture partition, source hashes and checked artifacts. API checks compare scheduler-created identities and task states. Parsing or `dag.test()` checks do not establish scheduled worker execution. Use a fresh run to check the scheduler and worker.

Further study

Questions 9–13 are separately timed advanced study.

How do readiness and freshness differ?

Readiness checks that each supplied raw relation exists with the required columns and population. Freshness checks _extracted_at separately. Historical May timestamps fail a current freshness threshold until an explicit recorded preparation refreshes only ingestion timestamps. A sensor waiting for presence cannot prove freshness. Keep freshness separate from business-date selection and avoid inventing new business rows.

How should Airflow orchestrate dbt, and when does Cosmos help?

The prepared PythonOperator route calls the delivered isolated worker dbt executable with explicit project/profile/airflow target and artifact paths. The same Portfolio graph runs as on Wednesday. An optional Cosmos adapter renders that graph using the installed integration, manifest load mode and task-time token env; its APIs and actual worker behavior must be checked on the measured stack. Do not replace dbt with a scaffold print or declare import success as execution.

What do Astro Runtime and the CLI add?

Astro supplies the project/image lifecycle around Airflow. Read the delivered Dockerfile and compose override, then inspect actual mounts and versions. The intended dbt executable is /home/astro/.venv/cosmos/bin/dbt. Token delivery uses the prepared databricks_default connection at task time; startup, refresh, expiry and restart are separate checks. A running scheduler does not prove the profile paths or warehouse permissions.

How do we test and operate a workflow as a team?

Review source/import checks before scheduling, then run success, intended failure, repair and replay with independent outputs. Keep owned DAG/relation names and cleanup in the handoff. Prove two simultaneous learner attempts do not contaminate one another and removal of one leaves the other intact. Resource provisioning, external alert transports and deployment remain optional/reference work.

Can we deliver a workflow and demonstrate recovery?

Present one scheduled success, one intentional quality failure with no premature evidence, and a repaired replay against the same holdings data. Bind logs, receipt, run results, versions and independent query to the contract. Explain the business partition and why task success alone is insufficient. Retain a restart/auth-expiry rehearsal before acceptance; human pacing and UI walkthrough are still required.

airflow: Phase 1 — foundations

Purpose and outcome

Orchestrate the same Portfolio Analytics product you transformed on Wednesday. Airflow orders source readiness, dbt transformation, quality verification and a publication-evidence step; Databricks still computes and stores the business data. The prepared dbt project prevents this from becoming a second complete modeling exercise. Your authored outcome is a small scheduled workflow whose task relationships and results can be explained.

Dataset and pipeline

The copied project contains six raw feeds and two supplementary Portfolio Analytics CSVs. The modeled result has four funds on each of two May dates. Each dbt build refreshes the complete tiny fixture graph; the run's explicit business partition selects four fund/date rows for its receipt. This distinction is deliberate: changing a receipt label is not a claim that dbt processed only that date. Logical schedule time and the historical fixture business date are separate concepts.

The four tasks are source_ready, dbt_build, quality and publication_evidence. The readiness task checks raw populations and extraction-time freshness. The build invokes the delivered worker dbt executable. Quality compares actual warehouse output with independent copied-CSV calculations. The last step records checked output for an exact run and partition. A failed quality task must prevent that receipt. Data remains in Databricks; only small control information belongs in task messages.

Prerequisites and workspace

Use the prepared Astro project, delivered connection setup and approved Databricks namespace. The terminal must have dbt-databricks access and the worker must receive credentials at task execution. Discover actual mounts and executable versions; the terminal's Python environment is not proof of the worker's environment. Set an owned relation prefix and use the matching DAG identifier, generated from that prefix, so simultaneous learners remain isolated.

```
mise run challenge:v2:prepare airflow phase-one-certified
cd .local/challenges-v2/airflow/phase-one-certified
export DC_RELATION_PREFIX=dc_alex_air01
```

Run direct dbt preparation inside the copied project subdirectory. Restore or record a timestamp-only extraction refresh before seeding when the supplied historical timestamps are stale; the May business dates must remain unchanged. From the attempt root, use the runtime staging helper with the actual Astro project root. It refuses an existing owned destination and writes ownership metadata needed by the DAG. Never stage into another learner's prefix or assume any terminal directory is mounted into the worker.

Starter behavior

The DAG file contains deliberate authoring gaps in dbt invocation, publication evidence or dependency wiring. Static compilation checks Python syntax but completion verification rejects these unfinished outcomes. Importing the unstaged file can fail because ownership metadata is not yet available; first prepare the owned stage as documented, then inspect the actual worker import. The dbt project is a disclosed solved baseline. Use it to understand the product and avoid changing its business logic while diagnosing Airflow. Unavailable scheduler, mount or warehouse access remains a pending prerequisite rather than a reason to substitute a local no-op DAG.

Timeline and completion standard

Core reading, authoring, checking and explanation: 75–110 minutes. Allow additional time for setup and optional stretch.

- Task: Inspect a prepared orchestration baseline · Minutes: 15–22 · Evidence: Actual mount/executable identities, owned namespace and supplied-vs-authored inventory

- Task: Author a small Python DAG with real dbt tasks · Minutes: 15–22 · Evidence: Four meaningful tasks with explicit paths, target and actual warehouse checks
- Task: Demonstrate the dependency graph and scheduled run · Minutes: 15–22 · Evidence: Readiness → build → quality → evidence; fresh scheduled task identities
- Task: Inspect a controlled task failure · Minutes: 15–22 · Evidence: Quality failed; evidence upstream_failed; no failed-run receipt; repair succeeds
- Task: Explain the run and environment boundaries · Minutes: 15–22 · Evidence: Coherent handoff links run/task IDs, artifacts and independent fund/date output

Independent starter files Read the owning phase before editing. Prepare from the training project root with `mise run challenge:v2:prepare airflow phase-one-certified`; edit the copied attempt.

Authored-file list

Task	*Learner-authored paths*
Inspect a prepared orchestration baseline	`evidence.md`
Author a small Python DAG with real dbt tasks, Demonstrate the dependency graph and scheduled run, Inspect a controlled task failure	`dags/portfolio_daily.py`, `evidence.md`
Explain the run and environment boundaries	`evidence.md`, `incident.md`

Declared baseline

These focus files are intentionally unfinished: `dags/portfolio_daily.py`. Supporting models and profiles are supplied. Complete the listed files, then verify their output in the required Databricks or Astro environment.

Keep `evidence.md` beside the copied project. Create a named file when its task asks for it. Use a fresh workspace for another attempt; preparation and reset preserve existing work.

Core challenges

Inspect a prepared orchestration baseline

Concept

A DAG definition describes tasks and dependencies; the scheduler decides when a run is due, and a worker executes its task code. The terminal that edits the file is another process boundary. Databricks is yet another boundary, executing dbt SQL remotely. A file visible to the terminal may not be mounted in the worker, and a dbt executable on the terminal path may not exist inside Astro. Diagnosing those differences before writing operators prevents a successful local command from becoming misleading run evidence.

The supplied project contains a complete Portfolio Analytics baseline and a partly authored orchestration file. Its root is discovered relative to the staged DAG, while ownership metadata binds the namespace and relation prefix. For example, the worker must find the staged project directory, its profiles and an artifact directory it can write, not a path copied from your laptop. The same approved warehouse and schema must be reached by terminal seeding and worker transformations. Source presence and a running scheduler do not prove either mount correctness or database permission. The inspection task records those prerequisites explicitly without pretending that setup alone demonstrates orchestration competence.

Challenge

Inspect the copied DAG and project before filling its gaps. List the four task responsibilities, the supplied dbt models and the exact learner-authored DAG paths. Read the delivered Astro Dockerfile and compose mount configuration, then record the actual running mount when the prepared environment is available.

Identify the intended worker dbt executable, project/profiles directories and artifact locations. Keep measured versions distinct from source-configured expectations.

Prepare the supplied seeds through the terminal's Databricks profile in the project subdirectory and record the approved namespace/prefix used. Stage only this owned attempt into the actual Astro mount using the helper; inspect its ownership metadata without copying token values. Identify what happens if that stage does not exist or uses an already occupied prefix. If Astro or credentials are unavailable, write the source-backed boundary inventory and mark actual mount, worker and warehouse checks pending. Continue reading the prepared graph rather than constructing an unrelated local fixture.

> Follow the staged path across the worker boundary
>
> **Hint**
> Follow the DAG's root-path expression and the stage helper's output together. The ownership file should agree with the terminal source-loading namespace. Inspect the worker executable inside the actual container rather than assuming a shell activation also affects it. Keep configuration probes separate from secrets: names, paths and redacted identities are useful evidence, while a token is not. A missing mount is a preparation problem, not a SQL defect.

Check

Inspect the actual mount, worker executable/version and owned stage when connected. The stage should contain the copied project, profiles, runtime helper, ownership metadata and writable artifact/evidence locations. Terminal and worker namespace identities must agree, and the raw input counts must match the supplied contract. Record which files are solved supporting assets and which orchestration functions remain unfinished.

Run static compilation locally to identify syntax errors without claiming a worker import or scheduled result. The untouched completion verifier should reject declared DAG gaps rather than accept the prepared dbt project as your solution. Explain why a terminal folder and a running scheduler cannot establish worker visibility, and why two learners need distinct prefixes/DAG identities even inside one approved schema. Retain source-backed paths and measured runtime observations in separate fields. If a probe cannot run, leave that individual prerequisite pending; do not fill it with the intended Dockerfile value.

```
python runtime/lab.py static .
# Stage only after the approved namespace and actual Astro root are prepared.
python runtime/lab.py stage-astro . --astro-root "$DC_ASTRO_ROOT"
```

Phase contents · Reasoning key

Author a small Python DAG with real dbt tasks

Concept

An operator executes work; the task identifier is only the name Airflow uses to track that execution. A task called dbt_build that prints a message does not build a data product. The prepared PythonOperator route invokes the delivered dbt executable with explicit project, profile, target and artifact paths. Its exit status and artifacts then connect task state to actual Databricks model and test results. Airflow controls ordering while dbt controls the transformation graph.

The publication-evidence step has a narrower classroom meaning than production release. It checks the consumer output, selects the run's four fund/date rows and writes a receipt tied to run identity, input hashes and the shared contract. A successful no-op marker would be insufficient because it could exist without any correct warehouse result. A small partition identifier can be passed between tasks, but holdings data itself stays in Databricks rather than being copied through XCom. This boundary makes the DAG easier to reason about and prevents a green graph from becoming the only evidence of business correctness.

Challenge

Complete the declared gaps in the copied DAG's worker function. The build branch must run the actual dbt build with the prepared project/profile directories, airflow target and writable artifact path. The evidence branch must independently check warehouse output, select the configured business date, and write a run-specific receipt under the owned stage. Retain the supplied readiness and quality helpers, reviewing their contracts before changing them.

Use the generated owned DAG identity, timezone-aware start date, two-minute classroom schedule, one active run and explicit fixture partition. Keep catchup disabled for this bounded rehearsal. Set task-time connection templating rather than copying a terminal token into source. Compile the Python file and inspect the worker import after staging. Do not replace the prepared dbt graph with shell echoes or create an external publication operation to make the last task sound more realistic. Document each authored branch and the supplied behavior it relies on.

> A task name is not evidence that its work executed
>
> **Hint**
>
> Trace the subprocess command from executable through target and target-path flags. The worker should use its own delivered dbt environment, not the interpreter running the authoring shell. For receipts, use the actual run identifier in the filename and the independently checked partition result in its body. Review the difference between a callback logging a failure and an operator that performs transformation work; they serve different purposes.

Check

Static inspection must show four meaningful task branches and reject any remaining unfinished marker. In the prepared worker environment, verify that the DAG imports with the expected identity and that the dbt command resolves its real executable, project, profiles and artifact directories. Actual execution should produce fresh dbt run results and a warehouse-checked receipt; these runtime outputs remain pending until the scheduler and worker run.

Inspect the receipt contract: exact run identity, fixture partition, shared contract hash, copied input hashes, full eight-key output verification and four selected partition rows. Explain why the whole tiny fixture can be refreshed while the receipt describes one business date, and why that should not be called date-filtered ingestion. Retain a selected command and its output rather than treating a successful Python compilation as dbt execution. The final evidence task must fail instead of writing a receipt when its independent query fails. Keep credentials outside source and logs.

```
python runtime/lab.py static .
# In the actual staged worker, inspect the imported DAG and delivered dbt version.
```

Phase contents · Reasoning key

Demonstrate the dependency graph and scheduled run

Concept

Dependencies state which task must finish successfully before another becomes eligible. They do not themselves move business data or guarantee that the downstream task reads the right relation. The required chain is readiness, dbt build, quality and publication evidence. This protects the receipt from being created before inputs and results have been checked. Leaving a branch disconnected can make a green task appear even when its upstream product work failed.

A manual trigger and a scheduler-created run can execute the same graph, but they establish different things. The manual run can help diagnose code and logs; only the scheduled run proves the configured timetable, scheduler and worker boundary required here. The schedule's logical date/data interval is not automatically the May business partition. Catchup is disabled and the fixture date is explicit. A prior

green UI run may belong to old code or another prefix, so record current DAG/run/task identities and the start of this rehearsal. Both the dependency graph and the actual task state transitions matter.

Challenge

Complete the graph wiring in the copied DAG so each task depends on its declared predecessor. Inspect the rendered worker graph after staging and identify how a failed quality task blocks evidence. Use a controlled manual trigger only as supporting diagnosis, retaining its distinct run identity. Then unpause only the owned DAG for a bounded scheduled rehearsal and record a fresh scheduler-created run after your session start.

Inspect readiness, build, quality and evidence task instances in that run. Follow the build log to dbt artifacts and the final receipt to the selected partition output. Confirm the worker is using the current staged source rather than an old imported copy. After collecting the run, pause the owned DAG so the two-minute timetable does not keep generating classroom evidence indefinitely. Do not clear unrelated DAGs, broad date ranges or another learner's tasks to repair a local wiring error.

> Compare graph edges with the exact current run type
>
> **Hint**
> Read the dependency expression as a sequence of obligations and compare it with the actual graph. An operator can be instantiated without being connected. Check the run type and start timestamp before calling a UI result scheduled evidence. If the expected current task identifiers are missing, inspect staged source and import state before retrying the warehouse. A manual run's success does not answer whether the scheduler produced a new run.

Check

The graph must contain the four required tasks in order, with no evidence task eligible before quality succeeds. For the scheduled run, retain the owned DAG identifier, fresh run identifier, run type, start time and all task states. Its successful dbt artifacts and receipt must correspond to the same run and stage. The full warehouse output contains eight keys; the receipt's selected May partition contains four fund/date rows.

Use the scheduled-evidence command after the controlled failure and repaired runs are available; it checks three distinct scheduler-created identities rather than accepting a historical green graph. Explain the difference between logical schedule interval and explicit fixture business date, and between a manual diagnostic trigger and this acceptance boundary. A missing scheduler/API prerequisite leaves the check pending even if local compilation succeeds. Keep the task-state and product-output evidence linked so a correct SQL query from another attempt cannot satisfy this run's graph evidence.

```
mise run challenge:v2:airflow-evidence "$DC_ASTRO_STAGE" --success "$DC_SUCCESS_RUN"
 ↪  --failure "$DC_FAILURE_RUN" --repair "$DC_REPAIR_RUN" --after "$DC_SESSION_STARTED_AT"
```

Phase contents · Reasoning key

Inspect a controlled task failure

Concept

A task failure is a state with a cause, not merely a red UI tile. A controlled business-quality fault should fail the quality task after dbt has built its input and prevent downstream evidence. A bad executable path or expired token is a different failure category and should not be counted as the intended quality drill. Reading the first useful log error distinguishes these cases and guides the repair.

The supplied fault marker belongs to your owned stage. While it exists, the quality branch raises its declared teaching failure and the evidence task becomes upstream-failed. The last accepted receipt should remain available, but this failed run must create no new accepted receipt. Removing the marker repairs the controlled cause; rerunning without removing it merely repeats the failure. Retries can repeat transient work, but they cannot turn a permanent business-quality violation into correct data. This small

experiment teaches failure propagation and bounded recovery without requiring external alert transports or production deployment.

Challenge

After a successful current scheduled run, place the declared quality-fault marker in your owned staged attempt. Wait for or inspect the next fresh scheduled run, then record the quality task's first useful error, task state and downstream evidence state. Confirm there is no receipt for that failed run and preserve the previous successful receipt rather than overwriting it with an ambiguous latest file.

Remove the marker and retain a repaired fresh scheduled run. Compare its warehouse output and partition receipt with the independent baseline expectation. Explain whether retries occurred and why the marker had to be removed for recovery. Keep changes bounded to the copied stage and do not clear unrelated tasks or modify raw fixtures to manufacture a different failure. If a token, path or permission error prevents this sequence, diagnose it as a prerequisite failure and leave the intended quality drill pending.

> Retain the previous receipt and reject only the failed run
>
> **Hint**
> Name the expected failure message category and task before creating the marker. Inspect the per-run receipt filename instead of relying on whether an evidence directory contains any file. A receipt from the earlier success is legitimate historical evidence for that run; it cannot prove publication for the failed one. Check the upstream-failed downstream state and the quality log together to establish the causal chain.

Check

The failed scheduler-created run must show quality as failed and publication evidence as upstream-failed, with no receipt named for that run. The log should identify the intentional quality marker, not an authentication or missing-file error. After removing the marker, a different fresh scheduled run must succeed and reproduce the expected independent output. Retain the success, failure and repair identities distinctly.

The API evidence checker validates run type, freshness, task states and receipts; inspect the corresponding UI/log readability separately. Explain why repeatedly clearing the quality task without repairing its marker would not recover the workflow, and why a saved successful receipt does not mean the failed run published. Keep source and fixture hashes for the bounded drill so the next reader understands what changed. A no-op callback, a stale green screenshot or a manual local test cannot replace actual scheduled worker failure propagation.

```
# Only within the owned stage, create and later remove this teaching marker.
touch "$DC_ASTRO_STAGE/fail-quality"
# Retain the intended failed scheduled run before removing it.
rm "$DC_ASTRO_STAGE/fail-quality"
```

Phase contents · Reasoning key

Explain the run and environment boundaries

Concept

Completion evidence connects an executed workflow to a business result a reader can assess. Airflow's run and task identities establish which orchestration instance ran; dbt artifacts identify transformations and tests; an independent warehouse query establishes the intended population and values. None can substitute for all the others. A correct report queried from an old prefix says little about whether your current scheduled worker produced it.

The receipt ties a current run to its selected fixture partition and input hashes, while checking the complete small report. It is useful because the May business date differs from the October timetable. The handoff must distinguish those clocks and explain the build's full-fixture scope. Authentication

is another boundary: a terminal login does not automatically make a cached worker connection fresh, and a first successful run does not prove expiry/restart recovery. Describe these unexecuted checks honestly. The classroom evidence step also does not grant consumers access, publish a catalog product or authorize deployment. That narrow meaning avoids overclaiming a successful practice workflow.

Challenge

Write a short evidence and incident handoff for your owned attempt. Link the three current scheduled run identities to their task states, useful logs, dbt manifests/run results, independent report comparison and per-run receipts. State the chosen fixture partition, its four selected fund rows, the complete eight-row build scope and the two-minute schedule interval. Include actual worker paths and versions where observed, and source-backed expectations where not measured.

Describe the controlled failure cause, repair and remaining authentication/restart/UI gates. Identify the code you authored and the prepared dbt/helper baseline you used. Provide the owned cleanup boundary and a reviewable DAG diff without deleting shared resources. Compare against the end-of-phase reasoning keys after your attempt and note any instructor-answer exposure. The handoff should let a colleague understand why this result is trusted and what still requires the prepared runtime or a human.

> Connect identities to business values and remaining checks
>
> **Hint**
> Start with a compact identity chain: owned DAG, current run, task instances, dbt artifacts, input variant and consumer result. Use that chain to find missing evidence rather than listing every terminal command. State the earliest useful failure cause and the precise repair. Keep observed versions and configured paths in different fields; copying a Dockerfile tag into a measured-version field would mislead the next reader.

Check

Inspect the handoff against the actual receipts and API report. Each successful or repaired receipt must name the correct run, contract and fixture partition; the failed run must have none. The report comparison must identify all eight fund/date keys and the selected four-row partition. Check that the DAG was paused after the bounded rehearsal and that cleanup targets only its owned stage and prefixed relations.

Explain why parse, local manual testing, terminal connection and an old green UI do not establish the scheduled worker outcome. State whether token refresh, expiry, restart and two-workspace isolation were actually exercised; leave them pending otherwise. Retain a readable diagnostic log excerpt without secrets and a bounded authored diff. Use the live verifier for API and warehouse checks, then review the corresponding UI and diagnostic excerpt with your instructor.

```
# From the training root, assess the owned staged attempt after fresh evidence exists.
mise run challenge:v2:verify airflow phase-one-certified --workspace "$DC_ASTRO_STAGE" --mode
 ↪  live
```

Phase contents · Reasoning key

Phase completion

Complete the foundation with a readable four-task DAG, the declared dependency chain, explicit worker project/profile paths and a fresh scheduled run in the prepared Astro environment. The independent checks must show the complete eight-row fund/date report and the four fund rows selected for the run's named business partition. Retain the actual run and task-instance identities, useful logs, dbt artifacts and a run-bound receipt. Manual execution and syntax compilation remain supporting evidence; they cannot replace the scheduled worker boundary.

Demonstrate one controlled quality failure that blocks publication evidence, remove its cause and retain the repaired scheduled result. Explain which inputs, models and helper functions were supplied and which DAG gaps you authored. Your handoff distinguishes the terminal, scheduler, worker and Databricks

warehouse, and states what remains pending about authentication refresh, restart and human UI review. Keep the owned DAG paused after rehearsal and preserve its evidence before cleanup. The receipt is classroom evidence, not production publication or a catalog registration. Do not make an external notification, deployment or another learner's complete project a prerequisite for this core.

Optional stretch

The core completion standard is independent of optional work. No additional stretch is required in this phase; the next phase has an independently prepared entry baseline.

airflow phase reasoning keys

Reasoning keys

Read these explanations after attempting the checks.

Reasoning: Inspect a prepared orchestration baseline

The terminal, scheduler, worker and warehouse do different jobs. A useful boundary inventory follows the owned stage into the actual worker mount and verifies executable/profile/artifact paths there. The supplied dbt solution is an input to orchestration practice, not proof of your DAG authoring. An unstaged file may fail import because ownership metadata is absent; a staged file can still fail because its worker executable is wrong. Both are different from a business-quality failure and need distinct observations.

Return to Inspect a prepared orchestration baseline

Reasoning: Author a small Python DAG with real dbt tasks

The build branch executes the real prepared dbt graph on Databricks. Quality and evidence read independently specified warehouse results rather than trusting the task name. The receipt binds the current run to its selected business-date result while explaining that this tiny build refreshes the full fixture. A no-op publication marker or a copied terminal token would break those boundaries. Static and import checks are prerequisites; fresh worker artifacts and output establish the executed behavior.

Return to Author a small Python DAG with real dbt tasks

Reasoning: Demonstrate the dependency graph and scheduled run

The required edges ensure checked readiness precedes transformation and checked quality precedes evidence. A disconnected evidence task defeats that guard. Manual execution is useful diagnosis, but the scheduled run proves the timetable and worker boundary. Its exact task identities, artifacts and receipt must agree with the owned stage and current session. The May business partition is explicit and independent of the schedule interval. Pausing only the owned DAG preserves other learners and ends the bounded rehearsal.

Return to Demonstrate the dependency graph and scheduled run

Reasoning: Inspect a controlled task failure

The quality marker creates a known business-gate failure. It should block only that run's publication evidence while preserving the previous successful receipt. Removing the marker repairs the cause; a fresh repaired run then needs independent output verification. An expired token or wrong path may also fail a task, but does not establish this intended drill. Task state, useful log and receipt identity together show the propagation and recovery rather than merely documenting red then green colors.

Return to Inspect a controlled task failure

Reasoning: Explain the run and environment boundaries

A coherent handoff joins current run/task identities to dbt artifacts and independently checked warehouse values. It states the distinction between fixture partition, complete build scope and schedule interval. The controlled quality failure and repaired run need separate receipts and causal explanation. Worker auth, restart, isolation and human readability are additional

checks, not conclusions inferred from a first success. Classroom publication evidence does not perform production publication. Recording these limits makes the result reviewable without hiding unfinished acceptance work.

Return to Explain the run and environment boundaries

airflow: Phase 2 — advanced practice

Purpose and outcome

Develop the four-task Portfolio workflow into a bounded recovery and handoff exercise. The prepared entry includes the solved dbt product, readiness helper and task scaffold so advanced work can start without another learner's foundation answer. You author the missing gate/recovery/evidence behavior and demonstrate why the repaired result is correct. Check source freshness and worker authentication while investigating recovery.

Dataset and pipeline

The eight supplied Portfolio CSVs remain the only business inputs. Four funds on each of two May dates produce eight report keys. Each dbt build refreshes that full teaching graph; the run's explicit partition determines four selected fund/date rows in its receipt. Supported partitions are 20 and 21 May 2026. Reject another date instead of silently substituting the latest input. Schedule intervals are current operational time; business dates and extraction freshness are independent clocks.

Readiness checks population and extraction timestamp; build invokes the actual worker dbt executable; quality validates the independent report and a controlled teaching fault; evidence writes a run-bound receipt only after success. A stale input or a missing raw relation fails readiness, while a quality marker fails the quality gate. A token or path error belongs to another category. The incident note must explain which category occurred and why the selected repair addresses it.

Prerequisites and workspace

Use the approved training container and Astro project, delivered connection/refresh setup and single provisioned Databricks namespace. Prepare an independent advanced copy and choose a unique owned prefix. Inspect the staged worker paths instead of copying an old terminal directory into an operator. The runtime helper requires ownership metadata and writes only this attempt's stage. Credential values remain task-time inputs, never saved in receipts or source.

```
mise run challenge:v2:prepare airflow phase-two-proficient
cd .local/challenges-v2/airflow/phase-two-proficient
export DC_RELATION_PREFIX=dc_alex_air02
```

Provisioning, image builds and unplanned access delays are instructor preparation outside the core estimate. Record measured managed versions and source-backed expectations separately. Set a rehearsal start timestamp and retain exact fresh scheduled run IDs so old accepted recordings cannot masquerade as evidence for the changed DAG. The actual scheduler/API/UI and warehouse are needed to finish this exercise; unavailable boundaries remain pending while authoring, parsing and fixture calculations proceed locally.

Starter behavior

The advanced starter contains declared unfinished DAG behavior and a complete supporting dbt baseline. Completion verification rejects those gaps. Static Python compilation is useful without credentials, but importing an unstaged file may fail because the owned metadata has not been generated. Stage through the helper after configuring the actual namespace and mount. Historical extraction timestamps need an explicit recorded timestamp-only refresh before a current freshness gate can pass; business dates and values remain unchanged. Do not weaken a stale-source assertion to make the teaching input look current.

Timeline and completion standard

Core reading, authoring, checking and explanation: 90–120 minutes. Allow additional time for setup and optional stretch.

- Task: Define bounded replay and input identity · Minutes: 18–24 · Evidence: Supported May partition, explicit schedule interval and reproducible source readiness

- Task: Inject failure and observe useful retry behavior · Minutes: 18–24 · Evidence: Known quality/readiness cause, observed attempts and no premature receipt
- Task: Prove output correctness after recovery · Minutes: 18–24 · Evidence: Eight unique report keys, four partition rows, stable values across repeated input
- Task: Use Astro UI to make an operational decision · Minutes: 18–24 · Evidence: Exact graph/grid/log identities, causal diagnosis and observed repair decision
- Task: Write a short incident handoff · Minutes: 18–24 · Evidence: Current success/failure/repair/replay chain, reviewable diff and explicit pending gates

Independent starter files Read the owning phase before editing. Prepare from the training project root with `mise run challenge:v2:prepare airflow phase-two-proficient`; edit the copied attempt.

Authored-file list

Task	*Learner-authored paths*
Define bounded replay and input identity, Prove output correctness after recovery	`dags/portfolio_daily.py`, `evidence.md`
Inject failure and observe useful retry behavior	`dags/portfolio_daily.py`, `incident.md`, `evidence.md`
Use Astro UI to make an operational decision	`incident.md`, `evidence.md`
Write a short incident handoff	`incident.md`, `evidence.md`, `dags/portfolio_daily.py`

Declared baseline

These focus files are intentionally unfinished: `dags/portfolio_daily.py`. Supporting models and profiles are supplied. Complete the listed files, then verify their output in the required Databricks or Astro environment.

Keep `evidence.md` beside the copied project. Create a named file when its task asks for it. Use a fresh workspace for another attempt; preparation and reset preserve existing work.

Core challenges

Define bounded replay and input identity

Concept

Replay asks a workflow to repeat work for an identified business input, so correctness depends on more than the new run's timestamp. The scheduler interval describes when Airflow created work; the fixture partition describes which business-date result the receipt presents. The copied graph refreshes both supplied dates, and its output check must still select the declared partition explicitly. A current run can therefore describe historical holdings without claiming that old holdings were newly generated.

Readiness has two separate conditions: the declared raw population exists, and extraction timestamps meet the freshness promise for this rehearsal. A presence sensor alone cannot establish freshness, while a fresh timestamp alone cannot establish complete rows. The supplied May timestamps intentionally reveal that distinction. A documented timestamp-only refresh changes ingestion observation time but keeps fund/date keys and valuations unchanged. The run must reject an unsupported business date rather than silently attach an empty or latest result. Those explicit contracts prevent a green replay from referring to the wrong input or a receipt that only relabels yesterday's evidence.

Challenge

Implement or review the run's supported fixture-partition validation, logical interval recording and readiness checks. Keep the two supported May dates explicit. List the six raw populations and extraction-time fields checked before transformation. Record a controlled timestamp-only preparation with its actual observation time when current freshness needs it, then seed the copied inputs into the owned namespace.

Stage the advanced attempt and execute each supported date in a bounded rehearsal, retaining the selected partition rows in its receipt. Demonstrate an unsupported date being rejected for that input-contract reason. Also demonstrate a stale or missing source failing readiness and repair the actual copied input cause before continuing. The full fixture build scope stays explicit; do not claim per-date ingestion filtering that the supplied models do not implement. Keep date identity, source variant and scheduler-created run identity together in the evidence note.

> Separate business identity from extraction and schedule clocks
>
> **Hint**
> Write the three clocks in separate fields: business position date, extraction observation time and Airflow logical interval. Inspect the readiness function before deciding which one a stale-input warning refers to. The helper's timestamp refresh records what it changed; compare input hashes and business keys afterward. For partition selection, verify all four receipt rows carry the requested May date rather than relying on the receipt's text label.

Check

For each supported partition, expect four fund/date receipt rows and independently correct values from the complete eight-row report. The unsupported date must fail validation, not produce a successful empty receipt. A stale or missing input must fail the source-readiness obligation before the build task is accepted. After the recorded refresh or source restoration, the same raw counts and May business keys should remain and readiness should recover.

Retain the source hashes, preparation timestamp, failed readiness category, repaired result and fresh run identities. Explain why relabeling a receipt cannot prove that the selected data changed and why a timestamp-only refresh does not create new holdings observations. Inspect the actual warehouse sources when connected; local CSV inspection proves only the copied inputs. Leave scheduled/API and freshness execution pending when those boundaries are unavailable. Do not disable catchup or freshness in an undocumented way merely to make a current screen green.

```
# From the copied project subdirectory; use a real recorded ISO8601 observation time.
python ../runtime/lab.py restore . --observed-at "$DC_OBSERVED_AT"
dbt seed --project-dir . --profiles-dir . --target local --full-refresh
```

Phase contents · Reasoning key

Inject failure and observe useful retry behavior

Concept

Retry policies express a belief about which failures might disappear without changing business inputs. A transient connection interruption can merit a bounded retry; a missing required FX observation or a deliberate quality marker requires repair. Retrying the latter only repeats the violated contract. The task log, attempt identity and delay reveal what happened; a final state alone cannot explain whether retry was useful.

The prepared classroom default uses a bounded retry and a short delay, with logging-only failure callbacks. Keep those controls within the rehearsal window rather than introducing unlimited waits. The callback observes a failure; it must not swallow the operator exception or write an accepted publication receipt. With one active run and a two-minute schedule, an unrepaired fault can affect a later scheduled run as

well, so pause or bound the owned rehearsal while collecting evidence. Compare a readiness failure with a quality failure to show where the graph stops. Both should block downstream accepted evidence, but their causes and repair locations differ.

Challenge

Review the retry and delay configuration in the copied DAG and state which failure category it is meant to address. Add the declared controlled quality marker in your owned stage, observe the affected task's attempts and useful error, and retain the downstream blocked state. Implement missing quality/evidence behavior without turning callbacks into hidden transformation or release logic.

Remove the marker and retain a repaired fresh scheduled run. For a separate readiness drill, use only a copied stale/missing input and explain why restoring or refreshing that input repairs readiness. Record whether a retry succeeded, exhausted its bound or simply repeated an unrepaired fault. Keep the DAG and date range bounded to your prefix, and avoid broad task clearing. An expired OAuth token may need connection refresh and worker checks rather than an SQL change; identify that as a prerequisite/auth failure if it occurs.

> Use retries for the failure category they can repair
>
> **Hint**
> Predict the task, error category, expected attempt count and downstream state before injecting the fault. Look for the first meaningful exception in the task log, then compare attempt identifiers and delay rather than reading the final UI color alone. A callback that logs and returns should not change the failed operator's result. Repair the marker or source cause before deciding whether clearing a named task is a useful recovery action.

Check

The quality-marker drill must show the intended quality failure, bounded retry behavior and publication evidence blocked for that run. No receipt may be written for the failed run, even if an earlier accepted receipt remains in the directory. After repairing the marker, a fresh scheduled success must produce the independent expected partition result. For readiness, retain a distinct failure cause and graph stopping point.

Record run/task/attempt identities, useful log excerpts, observed timing, repair and final output checks. Explain why retrying permanent missing data does not produce valid data and why a logging callback cannot be accepted as a release gate. A token failure or malformed path is not evidence of the intended quality drill; separate that diagnosis. Human readability and notification design remain distinct from task-state assertions. If the scheduler cannot execute here, retain the prepared failure contract and leave observed attempts and timing pending rather than inventing a retry story.

```
# Only the owned staged attempt receives this controlled fault.
touch "$DC_ASTRO_STAGE/fail-quality"
# Retain the failed task/attempt evidence, then repair its actual cause.
rm "$DC_ASTRO_STAGE/fail-quality"
```

Phase contents · Reasoning key

Prove output correctness after recovery

Concept

Recovery is successful when the workflow's repaired result satisfies its business contract. A later green run could still read stale relations, duplicate a date or select another learner's output. Independent key/value checking establishes what the consumer actually receives. The shared contract and owned prefix bind the query to the staged attempt; the receipt binds it to a run and business partition.

Replaying identical copied input should preserve the complete eight-key report and the selected four-row partition result. Distinct runs may write distinct evidence files while producing identical business values; that is expected, not duplicate business data. Replaying the other May date should produce a different

selected partition without erasing the first run's receipt. The supplied tiny graph rebuilds the fixture, so this exercise proves repeatable checked output rather than every possible incremental/backfill algorithm. The dbt advanced route demonstrates the separate Delta correction transition. Explain that limit instead of calling two green tasks a general exactly-once guarantee.

Challenge

After repairing the intended failure, retain a fresh scheduled run and independently query its owned report. Compare all fund/date keys and values against copied-CSV calculations, then verify the four selected partition rows in its receipt. Replay the same supported date with unchanged input and compare business results while retaining distinct run identities.

Run the second supported partition and confirm its selected receipt changes date without removing the earlier receipt or the full report's other keys. Preserve source hashes and dbt artifacts so a reviewer can tell whether input was actually identical. Do not compare a warehouse relation from a previous prefix or use a stale target directory as current output evidence. If you also choose the optional controlled holdings correction, use its modified independent oracle and label it separately from baseline replay. Restore the copied fixture after that additional drill.

> Distinct run receipts can describe identical correct data
>
> **Hint**
> Compare sorted business projections rather than receipt filenames: filenames should differ for distinct runs, while unchanged-input data should agree. Check both key membership and values so an offsetting numeric error cannot hide loss. The selected partition field must match every receipt row's date. Inspect source hashes alongside the replay result to establish that the repeated input was actually unchanged.

Check

Expect the baseline report to retain eight unique fund/date keys after repaired execution and identical-input replay. The chosen partition has four fund rows with matching values in each accepted receipt. Distinct run identifiers remain distinct evidence files, while the failed run still has no accepted receipt. The second May partition should select its own four rows without erasing the first run's evidence.

Use the current scheduled API checker and independent warehouse comparison, then retain the before/after projections and input hashes. Explain why a repeated build does not itself prove incremental merge correctness and why an unchanged grand total does not prove unchanged membership. A local compilation or copied instructor output cannot establish the current worker result. Record the actual artifact freshness and run identities, with any unavailable service check pending. This result is a bounded classroom repeatability demonstration, not a claim about concurrent writers, production backfill or distributed exactly-once processing.

```
mise run challenge:v2:airflow-evidence "$DC_ASTRO_STAGE" --success "$DC_SUCCESS_RUN"
 ↪  --failure "$DC_FAILURE_RUN" --repair "$DC_REPAIR_RUN" --after "$DC_SESSION_STARTED_AT"
```

Phase contents · Reasoning key

Use Astro UI to make an operational decision

Concept

An operational screen is useful when it helps answer a decision, not when it merely looks green. The graph shows dependencies, the grid shows run/task state over time, and a task log shows the causal error and attempt. These views need the exact owned DAG and fresh run identity. A bookmarked old run can look convincing while describing different code, inputs or namespace.

For the controlled marker, the decision is to remove the known fault and rerun a bounded owned scope, not clear every task or change warehouse grants. For expired credentials, the decision concerns delivered connection refresh and worker token resolution, not replacing business values. For missing mounted files, inspect the worker path. The API validates machine-readable identities and states; human

review assesses whether the corresponding UI and logs are understandable and safe to present. Neither automatically establishes auth expiry/restart behavior. A short operational note should explain the chosen action and its evidence so another learner can distinguish diagnosis from trial-and-error clicking.

Challenge

Open the prepared Astro UI at the exact owned DAG and current success, failure and repair runs. Inspect graph, grid and the quality failure log, retaining useful run/task/attempt identifiers and a redacted diagnostic excerpt. Compare those views with the API evidence report and receipt files. State what the first useful error implies and choose a bounded repair action that addresses that cause.

Inspect the actual worker project/profile/executable path and task-time connection boundary for any auth or file-location issue. In the prepared environment, rehearse delivered refresh and restart behavior and confirm the worker still reaches the approved namespace. Keep two active learner stages isolated if that check is available. Record any unavailable runtime check and its prerequisite. Do not use an old recording as evidence for newly authored sources.

> Use the exact failed run to select a causal repair
>
> **Hint**
> Begin with the run ID from the API report and navigate to that run rather than the DAG's latest overview. Compare the task log's attempt with the task-instance entry. If the error mentions credentials or an executable path, inspect the corresponding boundary before editing SQL. A useful screenshot or excerpt includes enough identity to locate the current run while excluding secrets and irrelevant browser context.

Check

The graph/grid/log evidence must bind the same fresh owned run identities checked by the API. The quality failure should be explained by the marker and its repaired run by marker removal plus correct warehouse output. Record a bounded recovery action and why it was appropriate, including any task clearing scope actually used. The UI cannot replace independent report values, and an API pass cannot replace human readability review.

For auth/restart or simultaneous-workspace checks, retain measured worker/namespace identity before and after the action; otherwise leave that individual check pending. Explain why a successful terminal seed does not prove worker token freshness and why broad cleanup would risk another learner. If capturing a screen, keep current run identifiers visible and perform authentication outside recording. The instructor can review the incident decision locally from prepared evidence contracts, but actual UI and worker recovery require the approved environment. Do not assign a guessed installed version to a measured field.

```
# Inspect the exact API-verified run in Astro; worker/UI behavior is a live check.
mise run challenge:v2:verify airflow phase-two-proficient --workspace "$DC_ASTRO_STAGE"
 ↪  --mode live
```

Phase contents · Reasoning key

Write a short incident handoff

Concept

An incident handoff explains impact, cause, action and evidence of recovery so someone else can decide what to do next. In this classroom pipeline the impact is blocked publication evidence for a named fixture partition, not a claimed production outage. The cause must be the observed marker, readiness, auth or path condition; vague statements such as Airflow was broken do not guide a successor.

The handoff also states ownership: which DAG, stage and prefixed warehouse relations belong to this attempt and how cleanup preserves others. A prepared Cosmos or hosted observability route can be useful optional depth, but does not become a missing mandatory deployment step. The finite core demonstrates an understandable data result, intended failure and recovery. Distinguish configured

behavior from the recovery you actually observed. A good note names the remaining checks and their exact environment without replacing them with reassuring prose.

Challenge

Write an incident note covering the chosen consumer question and fixture partition, exact current run/task identities, controlled failure cause, downstream blocked receipt, bounded repair and independently verified replay. Link relevant logs, API report, dbt artifacts, copied input hashes and selected partition receipts. Include the authored DAG diff and identify supplied supporting models and helper code.

Document terminal/worker paths, approved namespace/prefix, actual versions, task-time credential delivery and cleanup boundaries. State whether auth expiry/refresh, restart/reconnect, two learners and human UI/timing review ran; keep each absent observation pending. Pause only the owned DAG and restore copied baseline inputs after the drill. Provide a next-step checklist that is specific enough for another instructor to resume without creating extra schemas or deleting shared resources. Compare reasoning keys after your own attempt and disclose any prior answer exposure.

> Write the smallest handoff that proves cause and recovery
>
> **Hint**
> Use a short causal sequence rather than a terminal diary: observed impact, first useful error, targeted repair, independent recovered output and remaining checks. Anchor each claim to its run or artifact. If a sentence says everything passed, replace it with the actual checks and pending boundaries. Keep cleanup as owned resources and explicit reviewable statements instead of an unrestricted delete command.

Check

Inspect the note against the current artifacts: successful and repaired receipts exist for distinct runs, the failed run has none, business values agree independently, and replay preserves the declared keys. The note must explain full-fixture build scope, selected four-row partition evidence and current schedule interval without conflating them. Confirm the owned DAG is paused and cleanup cannot target another prefix.

Retain the current source/contract hashes and separate status for authored source, local static checks, connected execution and human acceptance. List any authentication, restart or isolation check that remains to run. A complete incident narrative cannot compensate for missing actual execution evidence, but it can make prepared local work reviewable. Full runnable solutions belong in the instructor package. No production promotion, external notification, infrastructure provisioning or publication is part of this core handoff.

```
# Inspect owned cleanup statements; do not execute broad deletion.
cd "$DC_ASTRO_STAGE/project"
python ../runtime/lab.py cleanup-sql .
```

Phase contents · Reasoning key

Phase completion

The advanced core ends with a demonstrated operational decision, not merely a running Astro screen. Retain fresh scheduled success, intended quality failure, repaired success and a bounded replay of the same business partition. Link each run to task states, useful logs, current dbt artifacts and independent warehouse results. Show that failed quality created no accepted receipt and that replay did not duplicate or erase the declared product population. Explain how the full tiny fixture refresh differs from the selected four-row partition evidence.

Submit the authored DAG, incident note, source/contract hashes, owned namespace and stage identity, and a short reviewable diff. Record actual worker executable/profile/mount and task-time authentication observations. Exercise restart, token expiry/refresh and simultaneous learners only in the prepared environment; leave each unexecuted check pending here. Restore the copied source variant and pause the owned DAG after rehearsal. Cleanup must preserve another learner's stage, DAG and relations. The

fixed core is complete only when its business evidence and explanation agree; optional Cosmos, hosted alerts, monitoring configuration and deployment remain separately timed reference work.

Optional stretch

The core completion standard is independent of optional work. Choose a separately timed route from the stretch guide.

airflow phase reasoning keys

Reasoning keys

Read these explanations after attempting the checks.

Reasoning: Define bounded replay and input identity

The run's timetable and May input partition are different clocks. Readiness checks both presence/population and extraction freshness, while the selected receipt rows prove which business date the consumer evidence describes. Refreshing copied extraction timestamps is a recorded preparation change, not a new holdings history. Unsupported partitions must fail explicitly. The full tiny fixture refresh remains the build contract; partition evidence cannot be presented as unimplemented date-filtered ingestion.

Return to Define bounded replay and input identity

Reasoning: Inject failure and observe useful retry behavior

The marker deliberately fails quality, so retries repeat the same known error until the cause is removed. Evidence must show attempts and downstream blocking rather than infer them from a final state. Readiness failures occur earlier and have different repairs. A callback can log a failure without converting it into success. Keep failed-run receipt absence separate from the preserved earlier receipt, and use actual scheduled attempts to establish behavior instead of a hypothetical retry configuration.

Return to Inject failure and observe useful retry behavior

Reasoning: Prove output correctness after recovery

A replay creates a distinct orchestration identity but should leave unchanged-input business keys and values stable. The receipt files therefore differ while their selected partition result agrees. Checking all eight report keys detects lost dates or duplication; checking four selected rows proves partition meaning. The supplied rebuild route establishes bounded repeatability, while Delta merge and correction behavior belong to a separately observed dbt exercise. Keep these claims distinct and preserve failed-run receipt absence.

Return to Prove output correctness after recovery

Reasoning: Use Astro UI to make an operational decision

Graph, grid and logs describe different aspects of the same run. Identity binding prevents old or unrelated evidence from passing as current. The marker requires a bounded marker repair, while auth and path failures require their own boundary diagnoses. Machine states and human readability are complementary. Worker refresh/restart and two-workspace isolation need measured before/after observations; neither follows automatically from a green run or a terminal login.

Return to Use Astro UI to make an operational decision

Reasoning: Write a short incident handoff

The incident note should connect observed cause to bounded repair and independently checked recovery. It identifies owned resources, copied input state and current run/artifact hashes. Source authoring, static checks, scheduler/warehouse execution and human review have separate statuses. Pausing the owned DAG ends the rehearsal without disrupting another learner. Optional integrations remain separately scoped, and no classroom evidence receipt implies production promotion or publication.

Return to Write a short incident handoff

Reasoning keys follow their owning phase.

Databricks discovery

Projects contents · Task index

- Foundation core — page 335

databricks-cli challenges

The Databricks command-line interface (CLI) provides a read-only discovery route into the prepared training environment. You establish actual profile/identity, distinguish workspace objects from catalog/schema/table metadata, inspect permitted qualified names, interpret JSON and explain an access limitation. A visible workspace directory is not a queried table, and a valid identity is not proof of every grant. This route contributes context for later warehouse work without provisioning resources.

Before you start

Use the delivered CLI and instructor-provided profile, permitted workspace path and approved namespace/table. Prepare a copied attempt through mise and keep discovery/evidence notes there. Sign in outside recordings and retain safe names/types rather than tokens. Inspect installed help/version instead of treating source pins as measured facts. The metadata JSON is a labelled offline example; its familiar training names do not prove current resources or permissions. No catalog/schema creation, grant changes or deployment is necessary for this read-only core.

Suggested timeline and tasks

Estimates include reading, practice, checks and explanation. Allow additional time for environment setup and optional extensions.

- Phase: Foundation · Task: Establish the CLI identity boundary · Minutes: 9–13
- Phase: Foundation · Task: Separate workspace and Unity Catalog names · Minutes: 9–13
- Phase: Foundation · Task: Inspect permitted schema and table metadata · Minutes: 9–13
- Phase: Foundation · Task: Filter JSON and explain a limited result · Minutes: 9–13
- Phase: Foundation · Task: Produce a bounded discovery handoff · Minutes: 9–13

Choose your phase

- Foundation: Foundation core with explicit completion evidence.
- Advanced: Not offered: no tasks or completion gate.

Finish the core tasks before choosing optional stretch.

Teaching concepts

Profile, identity, workspace path, qualified data name and warehouse execution are separate boundaries. Structured output preserves those distinctions, while empty/missing/denied observations require different explanations. Metadata discovery does not query row values or execute a dbt model. Use the end-of-phase reasoning keys after your attempt and record remaining connected probes individually. Full instructor reference observations belong separately, and no invented advanced assignment is needed to make this route complete.

Keep a discovery trail that another participant can repeat safely: the exact profile, permitted namespace, command category and selected metadata fields. When an object is denied or missing, preserve that limitation with its context instead of replacing it with a more convenient fixture response. This makes the subsequent connection and source-loading work easier to diagnose without pretending discovery granted access.

databricks-cli: Phase 1 — foundations

Purpose and outcome

Use the Databricks CLI to discover a prepared training environment without confusing its object types. You establish identity, distinguish workspace files from Unity Catalog metadata, inspect permitted catalog/schema/table information, interpret JSON and explain a limited result. The core is read-only. It prepares you to ask accurate questions about the later dbt warehouse boundary rather than provisioning resources to make a listing succeed.

Dataset and pipeline

The starter metadata JSON contains a labelled snapshot of one training catalog, schema, table and workspace directory. It is an example response for offline interpretation, not proof that these objects currently exist or are accessible. The workspace path is a filesystem-like object; a catalog/schema/table name identifies governed data metadata. A SQL warehouse is an execution resource rather than another folder in that hierarchy. The local snapshot deliberately uses familiar training names so you can practice those distinctions.

The connected route uses the actual approved namespace and object paths provided by the instructor. Those values may differ from the example. Identity and metadata access remain separate prerequisites: a valid login can still lack permission to enumerate a particular catalog or workspace directory. The report must distinguish such a denial from an empty permitted result or missing object.

Prerequisites and workspace

Prepare an independent copied attempt through mise from the training project root. Use the delivered CLI and identity setup, then inspect the actual installed version/help. Keep your discovery and evidence files inside the attempt. The instructor supplies the profile, permitted workspace path, approved catalog/schema and inspected table; do not guess them from the fixture or create replacement objects.

```
mise run challenge:v2:prepare databricks-cli phase-one-certified
cd .local/challenges-v2/databricks-cli/phase-one-certified
databricks --version
databricks current-user me --profile "$V2_DBX_PROFILE" --output json
```

Sign-in happens outside recordings. Record safe identity and qualified object names without tokens. If no cloud access exists here, inspect the labelled fixture and draft the exact probes with their actual connected observations pending. Source pins are intended versions, not measured installation facts. The class does not require broad grants, new schemas, infrastructure deployment or production job changes.

Connection variables

Before running connected probes, set `V2_DBX_PROFILE` to the delivered profile name, `V2_WORKSPACE_PATH` to the permitted workspace directory, `DBT_CATALOG` and `DBT_SCHEMA` to the approved namespace, and `V2_DBX_TABLE` to the fully qualified inspection table supplied by the instructor. These values describe existing resources; do not create replacements or copy the fixture names into actual-evidence fields. Record the configured names safely and keep authentication values outside the notes.

Starter behavior

The metadata snapshot parses successfully and there is no deliberate broken resource. Offline verification checks that example structure and reports supporting evidence only. Completion requires actual read-only environment observations and an explanation of the distinct object types. A denied path may be a meaningful permitted exercise observation if its boundary is stated; it is not an excuse to label every unavailable probe successful. Phase two is explicitly not offered and has no tasks or invented timing.

Timeline and completion standard

Core reading, authoring, checking and explanation: 45–65 minutes. Allow additional time for setup and optional stretch.

- Task: Establish the CLI identity boundary · Minutes: 9–13 · Evidence: Installed version/help and intended authenticated profile without secrets
- Task: Separate workspace and Unity Catalog names · Minutes: 9–13 · Evidence: Correct object types and separate qualified-name/path examples
- Task: Inspect permitted schema and table metadata · Minutes: 9–13 · Evidence: Approved namespace and metadata/type of one qualified table
- Task: Filter JSON and explain a limited result · Minutes: 9–13 · Evidence: Correct structured projection and diagnostic categories
- Task: Produce a bounded discovery handoff · Minutes: 9–13 · Evidence: Identity/object/type/limitation and next consumer question linked to actual probes

Independent starter files Read the owning phase before editing. Prepare from the training project root with `mise run challenge:v2:prepare databricks-cli phase-one-certified`; edit the copied attempt.

Authored-file list

Task	*Learner-authored paths*
Establish the CLI identity boundary, Separate workspace and Unity Catalog names, Inspect permitted schema and table metadata, Filter JSON and explain a limited result, Produce a bounded discovery handoff	`discovery.md`, `evidence.md`

Declared baseline

`metadata.json` is a saved example for local interpretation. Create `discovery.md` and `evidence.md` for your observations. Real identity and metadata probes use the instructor-provided profile and resources.

Use a fresh workspace for another attempt; preparation and reset preserve existing work.

Core challenges

Establish the CLI identity boundary

Concept

A profile selects connection and authentication context; it is not a business namespace or proof of table permissions. The CLI executable version determines available command groups and output options, while the active identity determines whose rights those commands use. A copied source configuration can describe an intended profile/version without establishing the environment currently loaded in the terminal.

Inspecting current user is a read-only identity probe. It should return a safe account identity in the prepared environment, but its success does not imply access to every catalog or warehouse. Keep tokens out of notes and screenshots. When cloud access is absent, the metadata fixture can teach response interpretation but cannot establish authenticated identity. This distinction prevents a local JSON parse from becoming a false readiness pass for later dbt work.

Challenge

Inspect `databricks --version` and the installed current-user help. Select `$V2_DBX_PROFILE`, supplied by the instructor, and run the read-only current-user probe. Record the returned account identity, profile and CLI version in `discovery.md` without credential values.

Confirm the intended identity before listing resources. Explain how your shell username differs from the remote account and how profile selection affects the request. If authentication is unavailable, record that prerequisite and interpret the local metadata fixture separately. Keep fixture examples labelled; they cannot provide a current-user response or establish the rights of an authenticated training account.

> Separate profile selection from observed authenticated identity
>
> **Hint**
>
> Inspect command-group `--help` for the installed CLI.
>
> Compare the selected profile and account identity; similar host names can belong to different contexts. Use installed help when a documented flag is unavailable. Record a command's failure category without dumping authentication configuration or tokens. Version/source pins describe intent, while the executable response is the measured boundary.

Check

Inspect the current-user response for the intended training identity and retain the measured CLI version/help. The evidence note must avoid secrets and state whether the probe actually reached Databricks. Explain why a successful identity response does not establish later catalog/table privileges.

An offline fixture parse remains labelled supporting, and an auth error remains a prerequisite failure rather than an empty resource list. If multiple profiles exist, identify which one produced the retained observation. Do not switch to another person's credentials or create broad grants to make the exercise pass. The live verifier reads identity and permitted metadata without provisioning.

```
databricks current-user me --profile "$V2_DBX_PROFILE" --output json
```

Phase contents · Reasoning key

Separate workspace and Unity Catalog names

Concept

Workspace objects organize notebooks, directories and other workspace content; Unity Catalog organizes governed data objects through catalog, schema and table names. A path containing training_dev does not mean it is the same object as a catalog named training_dev. The CLI uses different command groups and response fields for those boundaries.

The fixture's workspace directory under Shared and its qualified holdings table illustrate that naming similarity. A SQL warehouse supplies execution for queries, while listing a directory or table metadata does not run model SQL. Misidentifying these objects can send a learner to a filesystem command when they need governed table metadata, or lead them to infer data access from a visible notebook folder. State type, identifier and purpose together rather than retaining only a familiar name.

Challenge

Inspect `metadata.json` and make a short map of its workspace directory, catalog, schema and table. Use the corresponding read-only command groups to inspect `$V2_WORKSPACE_PATH` and the approved catalog in the connected environment.

Record each object's type, qualified path/name and relevant command group. The familiar `training_dev` spelling may appear as a catalog, directory, both or neither; inspect actual responses rather than infer identity from the name. Identify the warehouse separately as an execution resource. Explain why seeing a notebook directory cannot establish SELECT access to a table or execution of a dbt model.

> Record type and qualified identity alongside the familiar name
>
> **Hint**
>
> The starter fixture includes both object families specifically to practice this distinction.
> Look at response fields such as object type, path, catalog name, schema name and full table name. Compare the corresponding command groups in installed help. The separator convention and object type matter as much as the displayed word training. Keep fixture names labelled examples until actual connected probes establish their current counterparts.

Check

The discovery note must correctly distinguish a workspace directory, catalog, schema, table and SQL warehouse, with appropriate identifiers and safe read-only inspection commands. Inspect actual responses when connected and label fixture-only examples otherwise. Explain why workspace visibility does not establish table SELECT rights or warehouse execution.

Retain the two distinct identifiers and their response categories. A listing cannot be described as a full-text search over table data or a SQL query result. If the provided path is unavailable, record whether it was denied, missing or unprepared; do not invent another directory to satisfy the expected shape.

```
databricks workspace list "$V2_WORKSPACE_PATH" --profile "$V2_DBX_PROFILE" --output json
databricks catalogs list --profile "$V2_DBX_PROFILE" --output json
```

Phase contents · Reasoning key

Inspect permitted schema and table metadata

Concept

Metadata discovery answers what objects exist or are visible and how they are described. It does not automatically return their data rows. Catalog/schema/table listings can be filtered by permissions, so a user may see less than an administrator. A table's qualified identity and type help connect a later dbt source declaration to its physical relation.

Use the instructor's approved namespace rather than assuming a training label is globally unique. The same table short name can exist in several schemas, and a source profile can resolve one while a manual CLI command inspects another. Retaining the full qualified name prevents that confusion. Read-only discovery must not become provisioning: creating schemas, changing grants or deleting a denied object would change the environment rather than explain the observed access boundary.

Challenge

List schemas under `$DBT_CATALOG`, then tables under `$DBT_CATALOG` and `$DBT_SCHEMA`, using `$V2_DBX_PROFILE` and JSON output. Inspect the instructor-provided `$V2_DBX_TABLE` with `tables get`.

Confirm the returned full name belongs to the approved catalog/schema. Record its type and one useful available owner/description field in `discovery.md`. Keep the table identifier separate from the workspace path. Compare it with the source-loading namespace used by the dbt route. If you need row values, name the separate warehouse query tool; these listings inspect metadata. Explain any missing or denied operation without creating replacement resources.

Inspect the fully qualified relation in the approved namespace

Hint

The instructor supplies the actual provisioned schema; do not assume training_learner exists remotely. Pass catalog and schema as distinct arguments where installed help requires them, and use a full table name for metadata inspection. If the expected short name appears twice, compare namespace rather than picking the first row. A permission denial needs an access explanation, not a create/grant command. Preserve the returned metadata category and exact permitted scope.

Check

The retained metadata must identify the actual approved catalog/schema and one permitted fully qualified table, with its observed type and available descriptive fields. Explain how this metadata relates to a dbt source identifier and what further SQL/query access would still need verification. Compare namespace identity rather than only a short table name.

If access is limited, record the exact operation and denied/missing category, preserving any successful independent discovery checks. Do not call a partially visible list proof of complete catalog inventory. Local fixture metadata can support interpretation but cannot close these connected probes. No grants or schema creation are part of the completion evidence.

```
databricks schemas list "$DBT_CATALOG" --profile "$V2_DBX_PROFILE" --output json
databricks tables list "$DBT_CATALOG" "$DBT_SCHEMA" --profile "$V2_DBX_PROFILE" --output json
databricks tables get "$V2_DBX_TABLE" --profile "$V2_DBX_PROFILE" --output json
```

Phase contents · Reasoning key

Filter JSON and explain a limited result

Concept

JSON output preserves field names and nested structure, making a small selected projection more reliable than visually scanning a long terminal response. The learner should distinguish a response wrapper from its object list and handle an empty list without inventing an error. A filter selecting one qualified table can also return empty because the name or namespace was wrong.

Missing object, denied access and a permitted empty result have different meanings. The installed CLI's exit category and structured response help distinguish them; a blank-looking terminal alone does not. The supplied snapshot offers a stable local interpretation example, while actual cloud observations may differ. Avoid printing entire auth context as a diagnostic shortcut. A useful discovery note retains the selected safe fields and explains what the observation supports and what it cannot establish.

Challenge

Use Python's `json` module to inspect `metadata.json` and extract safe object types and fully qualified names. Explain whether the top-level response is a list or an envelope containing a list. Retain the projection and its result.

Apply the same approach to a saved, sanitized connected table listing when available. Inspect an instructor-provided empty, missing or denied example, or a safe permitted probe. Record its category and the next question you would ask. An unmatched name filter, a nonexistent schema and a permission denial have different explanations; do not change grants to manufacture a successful result.

> Interpret response structure before interpreting absence
>
> **Hint**
>
> Use json.load and examine whether the CLI output is an envelope object or list before indexing.
> Read the top-level type before indexing a response and check for the actual list field. Select safe names/types, not credential fields. Compare a filter that matches the supplied holdings example with one deliberately unmatched name. Keep an empty successful projection distinct from a command that failed because permission or object lookup was denied.

Check

Show the parsed table name training_dev.training_learner.holdings from the fixture. Explain separately what an empty collection and denied access would mean; do not retry by granting yourself privileges.

The local projection must correctly identify the fixture's directory and fully qualified holdings table without changing the snapshot. Retain the filter and result and explain its wrapper/list shape. For actual discovery, retain the command exit/category and safe projected fields, labeling connected or fixture-only evidence accurately.

Explain three distinct cases: permitted empty list, missing requested object and denied access. A generic nonzero status or empty screen cannot substitute for that interpretation. If a live example is unavailable, use the supplied labelled example and keep actual access behavior pending. Do not create or delete an object just to manufacture a probe result.

```
python -c "import json; d=json.load(open('metadata.json')); print([t['full_name'] for t in
↪  d['tables']])"
```

Phase contents · Reasoning key

Produce a bounded discovery handoff

Concept

A discovery handoff makes the inspected environment useful to another learner. It identifies the actual profile/identity, permitted object types and qualified names, relevant metadata and access limitations. A list of commands without interpretation leaves the next person to repeat every probe. Conversely, a claim that the environment is ready can overstate what metadata alone established.

The later dbt route needs a warehouse, approved namespace, loaded sources and usable credentials. CLI discovery contributes identity and object context but does not prove all those execution steps. State the next consumer question, such as which declared holdings relation the prepared pipeline should read, and which separate query/build check remains. This finite phase is read-only and has no advanced provisioning assignment.

Challenge

Write `discovery.md` with the identity route, approved catalog/schema/table, workspace distinction, filtered JSON result and one interpreted access limitation. Label each observation connected or fixture-only and keep its command beside the safe result.

Add a specific next question about holdings grain, freshness or source loading for the later dbt route. State what row/query/build check is still needed; metadata alone cannot answer it. Review the note for tokens, unsupported readiness claims and accidental provisioning instructions. Share it through the agreed course review workflow so another learner can follow the evidence without repeating every probe.

Turn safe observations into a specific next question

Hint

Use qualified names and omit tokens. A SQL query would require a separately taught API/warehouse boundary. Organize the note by observed identity, object, meaning, limitation and next check. Keep a command beside the safe result it produced. If an entire boundary is unavailable, say so directly rather than filling its fields from the local fixture. The useful conclusion is a bounded discovery result, not a claim that every data product or permission was verified.

Check

Inspect the note for the intended profile/identity, distinct workspace and catalog objects, approved qualified table, interpreted JSON and explicit access limitation. Each claim must identify actual connected or local-fixture evidence and its command category. Explain what additional row/query/build verification the later dbt journey still requires.

Run read-only phase verification in the prepared environment when available and preserve any blocked probes individually. No credential, broad grant, new schema or deployment belongs in the handoff. Phase two remains not offered. Identify any probe you could not run and the access needed to complete it.

```
# From the training root after the supplied profile/object variables are configured.
mise run challenge:v2:verify databricks-cli phase-one-certified --workspace "$DC_CLI_ATTEMPT"
 ↪  --mode live
```

Phase contents · Reasoning key

Phase completion

Submit a bounded discovery note identifying the actual training profile/identity, permitted workspace path, approved catalog/schema and one table's qualified name/type/owner or available metadata. Explain what each object represents and which command/API inspected it. Include a filtered JSON interpretation and one empty, missing or denied example with its correct category. Do not include credential values or imply a metadata listing is a query over table rows.

The supplied metadata fixture supports local practice and expected response interpretation. Actual identity, grants and discovered objects require the prepared Databricks environment; keep them pending when unavailable. This phase offers no advanced assignment and grants no provisioning, deployment or broad access changes. Your handoff gives a consumer or instructor an understandable next question and a safe read-only command rather than a vague statement that everything is accessible. Record actual version/help behavior and source/command hashes so later changes can be rechecked.

Optional stretch

This route has one phase. There is no required stretch or advanced assignment.

databricks-cli phase reasoning keys

Reasoning keys

Read these explanations after attempting the checks.

Reasoning: Establish the CLI identity boundary

The profile selects context, and the current-user response establishes the observed training identity. Installed help/version establishes executable behavior. Neither grants or proves table access. Local fixtures remain supporting interpretation examples, while missing authentication stays pending. Keep safe identity and command category without recording credential values.

Return to Establish the CLI identity boundary

Reasoning: Separate workspace and Unity Catalog names

Workspace path, catalog/schema/table hierarchy and warehouse are different boundaries. Their response types and command groups establish what was inspected. A visible directory cannot

prove data permissions or executed SQL, and a fixture name remains an example until connected discovery confirms the actual environment.

Return to Separate workspace and Unity Catalog names

Reasoning: Inspect permitted schema and table metadata

Catalog/schema/full table identity anchors the observed relation. Metadata type/description helps later source mapping but does not establish row access or SQL execution. Permission-filtered visibility is a limitation to explain, not a reason to provision new objects. Keep actual namespace and response category rather than a familiar short name.

Return to Inspect permitted schema and table metadata

Reasoning: Filter JSON and explain a limited result

A structured filter reveals selected safe fields and can legitimately return an empty list. Missing or denied operations have different categories. Interpret the response wrapper and intended namespace before inferring that no data exists. Fixture projections teach that logic but do not establish actual cloud access.

Return to Filter JSON and explain a limited result

Reasoning: Produce a bounded discovery handoff

The handoff joins safe identity and qualified object meaning with interpreted limitations and a next consumer question. Metadata is not executed dbt SQL or complete access proof. Local fixtures stay labelled, missing live probes remain pending and the route stays read-only. No invented advanced phase or provisioning action is needed to finish the discovery core.

Return to Produce a bounded discovery handoff

CG product-information capstone

Build a product-information data product for a fictional internal Product Information Analyst using the current CG capstone. The full learner project, instructions and templates are in the CG product-information project guide.

CG product-information business brief

Capital Group is an investment manager. Its public offerings include American Funds mutual funds, Capital Group ETFs and separately managed account (SMA) strategies. A share class is a particular way of owning a mutual fund, with its own identifiers and published costs. An exchange-traded fund (ETF) trades on an exchange. An SMA implements a strategy in a separately managed account arrangement, often through a sponsor platform. These are different offering types; their costs and published results are not automatically interchangeable.

Capital Group describes The Capital System™ as an investment approach combining distinct investment perspectives with a long-term focus. Today's assignment concerns published product information: identities, expenses and performance context.

You are building a product-information data product for an internal analyst who supports sales and relationship colleagues answering financial-professional enquiries. The role name and requests below are fictional classroom scenarios, not statements about an actual CG department. The analyst needs a dependable research view: product identity, stated fund expenses, performance figures with their calculation method and reporting date, and reasons to seek more information.

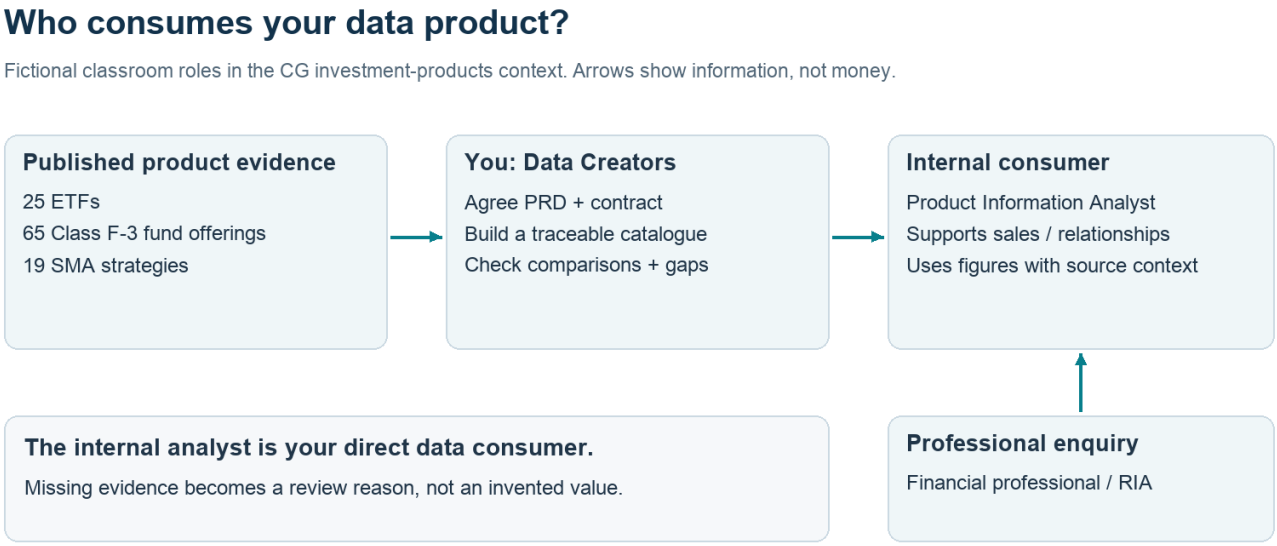


Figure 137: Published evidence flows through Data Creators to the internal analyst; the external professional enquiry is a separate relationship.

Open this diagram at full size

The analyst's current information comes from different public representations. A catalogue and detail page may describe the same ETF. A mutual fund has multiple share classes. SMA performance is reported for a strategy composite, not a particular client's account. Reporting dates differ; some extracted fields are blank. A plausible-looking joined table can be wrong even when its SQL runs.

Your task is to make the scope, meaning of each row, measurement units and source evidence explicit; implement and check them; then show the analyst what can be answered and what remains uncertain.

An expense ratio measures annual fund operating expenses. A net expense ratio reflects applicable waivers or reimbursements; it does not necessarily include every cost paid by an investor. One basis point (bp, plural bps) is 0.01 percentage point, so 50 bps means 0.50%. NAV means net asset value per share. An annualized return summarizes a multi-year result as a compound annual rate.

An SMA composite reports a group of portfolios under a stated methodology. It is different from an individual client's account return. These definitions matter when choosing what to combine in your report.

Three enquiries to answer

Enquiry	*Internal consumer request*	*What your product supplies*
E1 — Product lookup	"What does our snapshot say about CGGR's vehicle and published net expense ratio? Where did that answer come from?"	One product row with its expense ratio, source link, correct units and any missing reporting date
E2 — Research shortlist	"Which scoped fund offerings have a published net expense ratio no greater than 50 bps? Keep products with unknown costs visible."	A result for the 50-bps filter and a list of products needing follow-up; this covers published fund expenses only
E3 — Performance evidence	"Show three-year annualized returns calculated from net asset value (NAV), reported at September 30, 2026. Explain which figures meet our comparison rules and which do not."	Figures with matching calculation basis, time period and reporting date; products without those figures remain listed with a reason

Later, the analyst asks an operational question: "Did a failed candidate leave the last accepted output available, and can we reconstruct earlier knowledge after a correction?" You answer using a checked release workflow and a synthetic temporal exercise.

Your source scope

Domain	*Included records*	*Important source complication*
ETFs	25 ETF identifiers from detailed product records	Repeated catalogue/detail representations; all selected fee reporting dates are unknown
Mutual funds	65 Class F-3 identifiers from detailed mutual-fund records	Source class codes `F3` versus catalogue `F-3`; fund-class history/disclosures matter
SMAs	19 strategy IDs	No quoted expense-ratio table; some fields in the extracted CSVs are blank; returns cover only 12 strategies
Combined catalogue	109 unique scoped IDs	Different kinds of offerings, not 109 distinct underlying portfolios or every CG product worldwide

Use the frozen files under `data/`; do not scrape during the event. Read data conventions, disclosures and glossary.

Evidence boundaries

Use each figure within its published meaning. Gross/net expenses and gross/net strategy returns are different measures. A source reporting date differs from the download date. Matching return dates and periods does not establish equal objectives, risk or suitability. Blank CSV fields can reflect information the collector did not extract; check the source disclosure before drawing a broader conclusion.

Use three criterion states: `matches`, `does_not_match`, `needs_review`. Separately use comparison states: `comparable_on_declared_dimensions`, `not_comparable`, `needs_review`. Explain the reason; do not call a product “best,” “eligible” or “cheapest all-in.”

Phases and completion evidence

Phase	*Work*	*Required evidence*
Phase 0	Design and readiness	Reviewed PRD, ODCS contract, STTM and source-selection decision
Phase 1	Individual ETF core	Personal repository, two authored models, all 25 ETFs, failure/repair and individual observation
Phase 2	Pair integration	109 scoped IDs, comparison reasons, checked release workflow, callback and personal AI skill
Phase 3	Individual advanced work	Temporal history, reusable SQL and workflow templates, with checked run evidence
Submission	Report and presentation	Personal report, speaking notes and linked evidence; every Data Creator presents

Phase 0: Design and readiness

09:00–09:30. Review the three enquiries and frozen source fields with Catalyst. Author `docs/PRD.md`, `contracts/product-research.odcs.yaml`, `docs/STTM.csv` and decision notes. Keep source reporting dates separate from retrieval and load times. Record one unsupported generated suggestion you corrected. Check the instructor-assigned workspace and local/connected readiness using the project instructions.

Phase 1: Certified data creator

09:30–11:25, including the 10:40–10:55 break. Work individually in your assigned personal product folder. Author `models/phase1/src_cg__product.sql` and `models/phase1/dom_cg__product_cost.sql`. Select the declared detailed ETF representation, preserve provenance and retain all 25 ETFs. Select the matching net expense component; percentage points convert to bps by multiplying by 100. Unknown fee reporting dates stay null. Retain independent key/value checks, a controlled failure and repair, your code commit and individual observation. Follow the phase-1 guide.

Phase 2: Technical practitioner

11:25–14:00, with lunch 12:00–12:45. Work in pairs and record each person’s contribution. Integrate 25 ETFs, 65 Class F-3 mutual-fund offerings and 19 SMA strategies: 109 scoped IDs. Keep missing costs visible. Answer the 50-bps enquiry with `matches`, `does_not_match` or `needs_review`. For performance, retain the calculation basis, return kind, period and reporting date; explain comparison states separately from cost criteria. Build a custom orchestration and callback, demonstrate failed-candidate preservation of the last accepted output, and author your personal AI skill. Follow the phase-2 guide.

Phase 3: Advanced practitioner

14:15–15:15, individually after the 14:00–14:15 break. Complete both the synthetic temporal-history component and reusable SQL/workflow-template component. Distinguish business-effective time from knowledge time and show earlier knowledge after a correction. Restore your personal product's assigned target before execution. Use the phase-3 guide for the exact authored files and checks.

Report, presentation and submission

15:15–16:00. Finish the personal report and speaking notes by 15:30; every Data Creator speaks during 15:30–16:00. Link commits, source decisions, check results and actual runtime receipts from the evidence checklist, report and submission checklist. Local checks do not establish Databricks or Airflow execution. State unresolved checks and the owner of the next action.

Design workshops

The organisational names and reported incidents in these workshop cases are supplied teaching examples. They do not establish current CG structures, approved definitions, production status or enterprise service commitments. Use them to practise discovery; confirm any live business assertion with its accountable owner and current evidence.

Route: optional, separately timed. Purpose: extend consumer interviews, adoption and reviewable design decisions. Dataset: CG product stakeholder context and your own contract. Prerequisites: Monday design teaching. Runtime: documents and facilitated conversation. Deliverables: annotated contract, decision record and communication plan. Projects contents.

Write one from your build

Choose one design decision from the product you built this week. Write only Context, Decision and Consequences. Use numbers where you have them, name at least one realistic alternative in the Context, and include one negative consequence with its mitigation. A partner should be able to read the three sections and implement the same choice without asking what you meant.

Optional activity: stakeholder interviews and a Catalyst contract

This optional extension connects the stakeholder interview with Tuesday's contract revision. Use it when adoption or a working-practice concern needs more investigation. Tuesday's required contract exercise does not depend on an empathy map.

Optional interview-to-contract extension

DURATION 30–45 minutes outside the core route
OUTPUTS Evidence-labelled empathy map, revised draft and correction log

1. Choose an authorised business example that a participant understands, such as CG product-information or a product from Monday. In threes, assign consumer and two creator/reviewer roles; use a producer brief or invite producer validation for source questions.
2. Inspect an example local contract and identify the relevant decision, source and promises. Do not treat a generated schema or approval as a confirmed fact.
3. Interview the consumer about a real pain point and current workaround. Record observations separately from interpretations. Map says and does from evidence; mark thinks and feels as hypotheses and ask for confirmation or correction.
4. Feed the notes and confirmed requirements into Catalyst. Ask it to revise the affected contract sections and name unsupported decisions with their responsible roles. Use synthetic training inputs when needed, labelled as synthetic and distinct from source facts.
5. Open the revised file, inspect the changes and compare it with the interview, source and original draft. Check that an adoption action addresses the stated concern and that no invented permission or approval appeared.

Check: a requirement is traceable from the consumer statement to the contract; an adoption concern and a technical/source defect are not confused; source gaps remain visible. The result is a reviewed draft with outstanding decisions, not a live approval.

Case study: Interview evidence changes the contract review

Asset order-entry data. Timeliness and exchange-rate inputs span countries and providers, with different identifiers. A generated contract combines fields from unrelated sources. Decision: supply the exact source schema and mapping, then remove or raise unsupported fields. Reasoning: more topical context alone can

broaden the draft beyond the intended product. Lesson: ground the output in the specific source and consumer boundary.

Expense budgeting. Two draft generations include extra optional fields while omitting required ones. Decision: compare each draft with a must-have requirement list and a traceable schema. Reasoning: model agreement or a preferred writing style does not establish coverage. Lesson: evaluate both missing requirements and unsupported additions.

Scheduled business publication. A draft contains useful change-handling and slowly changing dimension questions but marks itself approved. Decision: retain the useful design questions and reset approval status to outstanding. Reasoning: a template can remind the creator about a responsibility without performing it. Lesson: generated approval text is not evidence of a sign-off.

Positions. Specific consumer context produces a more relevant draft, then a second review identifies questions still to inspect. Decision: keep the concrete problem, source constraints and review results with the contract. Reasoning: a strong prompt can improve relevance while still leaving errors. Lesson: use a second model as a source of questions and preserve independent validation.

Multi-currency equity positions. A domain reviewer spots missing currency sources and an incomplete schema. Decision: add the source requirements, currency basis and explicit conversion rule before proposing a derived value. Reasoning: a missing FX table can leave structurally valid SQL with misleading meaning. Lesson: domain evidence remains necessary when the draft looks complete.

When a proposed response makes the concern worse

A response aimed at one concern can worsen another. Repeated assurances that an expert is trustworthy may increase a consumer's fear of losing control. Demonstrating that a calculation is correct may leave their actual workflow problem unresolved. In this sense, a response can backfire: the intended improvement produces less willingness to adopt.

Test the interpretation before prescribing a response. Ask which part of the work the person must explain, control or approve, and what evidence would change their assessment. Conflicting interests may need negotiated responsibilities or a transition plan rather than more persuasion. If information does not change behaviour, investigate the remaining concern rather than repeating the same explanation.

Addressing the concern

For the parallel-spreadsheet user, first distinguish a source defect from a need for explanation, control or training. A volatile asset may be behaving correctly but appear implausible without context; a note explaining that behaviour can help, while a genuine defect needs a technical correction.

Involve the consumer early in design and review, make progress and decisions visible, and offer a real channel for feedback. If they need to verify a result, agree a reconciliation view or comparison they can inspect. Recognition can support adoption when it reflects an actual contribution, but visibility and credit do not replace access, quality or useful decision rights. Avoid simulated controls that give an impression of influence without a real effect.

When the quadrants don't align

Worked example: a salesperson using a portfolio-simulation report. *Says*: "I recommend this portfolio to a customer" (though they don't like the report). *Does*: observed using the report correctly in a sale, and the metrics look fine. *Thinks/Feels*: ambivalent, both frustrated (given a report that doesn't work the way they want) and accomplished (they made the sale). A successful sale can coexist with frustration about the report.

When the quadrants do not align, investigate the difference. A difference may reflect uncertainty, context, constraints or reluctance to criticise a tool they still need. Ask for an example and confirm the interpretation rather than inferring a motive.

Optional activity: interviewing with an empathy map

Optional stakeholder interview

DURATION 45–60 minutes outside the core route

OUTPUTS Interview notes and an evidence-labelled empathy map

Choose a business data product that one participant understands well. In threes, assign consumer, producer and creator roles. The creator interviews both roles using Monday's question prompts; the participant closest to the consumer's actual work should represent that role.

Before the interview, record what you know about the current workflow, previous changes and existing difficulties. If a colleague knows the consumer's context better, ask for it and label it as indirect evidence. Avoid assuming that a concern is irrational or that a person will react in a particular way.

During the interview, follow "what decisions depend on this data?" with "how are you making those decisions now?". Map one pain point into says, does, thinks and feels. Quote observations in the first two quadrants; label the latter two as hypotheses and ask the consumer to confirm, correct or reject them. If their report and observed workflow differ, investigate the conditions before attributing a motive.

Produce one requirement, one adoption action and one unresolved question from the map. Check that the action responds to a stated concern, preserves a real feedback or decision channel, and does not disguise a technical or contractual defect as a stakeholder problem.

For a pair, use the source brief for producer evidence and record the missing validation. A third observer may check leading questions and unsupported inferences. Review the interview's purpose and outputs at the end so an absorbing conversation still produces a usable handoff.

Case study: Interview findings and product decisions

Cases: consumer decisions and working constraints

- Client analytics. Ingesting client engagement data (website visits, email engagement), grouped by user role rather than individual users. The interview raised the question of how to define an outlier (compare to yesterday? to prior periods? to an external baseline?) and clarified that outliers should be *reported on*, not removed, because they're informative. Decision: report informative outliers rather than deleting them. Reasoning: unusual engagement may be the very signal the consumer needs. Lesson: agree the comparison baseline and keep the interview focused on a recorded requirement.
- **Exposures:** security-level exposure informs portfolio rebalancing. Missing data can prevent the decision and breach detailed contractual commitments. With no equivalent fallback, agree the failure impact, source dependencies, escalation and consumer update before setting a deadline.
- CG investor data. Data that influences investors' financial decision-making; if it's late or worked around, the risk is the investor goes to a competitor, with frustration and loss of trust in the service. Proposed focus: real-time delivery. Useful clarification: "real time" here means *market close*. Always pin down what real time means to the consumer. Decision and reasoning: Define "real time" using the stated business event, such as market close. A named event is more usable than an ambiguous latency term. Verify the event with this consumer.
- **Global results:** a two-day SLA has a manual fallback for the first six months, followed by stricter commitments. Consumers want automation but need evidence that it matches their existing result. Agree a parallel comparison, transition criteria and the later service promise; upstream checks alone do not establish adoption.
- HR promotion pipeline. Data integrated across several stages of the HR process, with different data needed at different points; combines internal data (employees) with external data (competitor pay ranges). Empathy finding: risk and anxiety around promotions. Decision and reasoning: Identify which stage uses each field and who may access it. Different stages and external pay sources create distinct meaning and sensitivity questions. Validate the staged workflow with its owner.
- Company data. The reference data representing companies, foundational for the entire enterprise, critical to compliance, ESG, stock trading, and many other functions. Decision and reasoning: Declare the

reference identity and accountable source before designing consumers' views. Multiple downstream compliance, ESG and trading uses raise change impact. Trace each consumer dependency.

- **Legal invoices:** budgeting external-counsel spend depends on vendor cooperation and the ability to estimate recurring costs. Separate the budgeting decision from automation constraints. Record the missing tool, source or agreement with its owner.

Case study: Interview findings and product decisions

Cases: definitions, responsibility and trust

- Salesforce data (external source). An initial problem is assigning the roles. Who is actually the consumer and who the producer? Issues were data duplication and getting data in a timely manner, ultimately leaving the consumer feeling they couldn't do their job, with concern about explaining the result to others (which makes esteem relevant). Decision and reasoning: Resolve producer and consumer responsibilities before specifying freshness and deduplication. A source label does not identify who can promise delivery. Name the accountable contact.
- Investor detail and content data (combined product, broken down by region). Delays in refreshing the published pages. A useful finding: the issues persist mainly through lack of prioritisation, and answering *why* it was never prioritised is what stops you falling into the same trap again.
- Capital market assumptions (received annually; drives forecasting). Impacts of delayed data collection and errors: reputational risk. That risk has actually been realised: a consumer of the documents came back and said "your numbers are wrong" before the team caught it themselves.
- **Marketing publication:** "real time" means the moment a document becomes legally effective. An hourly job may miss that business trigger. Agree the effective event, producer notification, permitted publication time and handling of a delayed notification.
- Sales data. Technical participants need a business representative to validate the consumer perspective. There's a voice of the business, of the consumer, of the producer, and of the technical team, and balancing them is hard. Decision and reasoning: Have a business representative validate the interpretation before treating a technical discussion as consumer evidence. Multiple voices serve different responsibilities. Mark any missing validation in the handoff.
- Monthly data feed with missing or inconsistent data. The consuming team applies a band-aid fix: when data is missing, they reuse the last reported data for their calculations. The frustration is that this happens *monthly*; the provider is not proactive about following the contract or even communicating when data is missing, and SLAs are violated. The expectation is a fix within an hour or so, and it sometimes takes over a day. Decision and reasoning: Agree how missing values, temporary reuse of prior values and correction delays are communicated. A silent carried-forward value changes the meaning of the result. Label the fallback and ask the consumer whether it is acceptable.
- Factsheet publication. At a high level, agreeing SLAs is easy; the pain is in the details, e.g. week-of-African-market data missing at the time of publication when the contract requires interest data available without gaps by a certain time. Upstream may only commit to "available by second business day" when the downstream deadline is earlier. Pin these things down at the start of the contract. The timelines are contractual first (multiple parties sit between producer and end consumer), with some regulatory obligations layered on top. Decision and reasoning: Reconcile upstream second-business-day availability with the earlier downstream publication deadline. A promise cannot ignore a producer dependency. Record the mismatch and its owner before agreeing the contract.
- **Dependency ownership:** a downstream team may bear the impact of a third-party failure without controlling its cause. Record the dependency owner, escalation route and consumer communication. Investigate frustration through the workflow rather than assigning a motive to the affected team.

Cases applying the eight scoping questions:

- **AFSG semantic layer:** a lakehouse names an implementation, while the consumer needs reports or discoverable service data for analysis. State the business decision, supported vocabulary and output before choosing the storage platform. Operational APIs and analytical reporting may need different interfaces.

- Product reference data for portfolio simulation. Wholesalers build hypothetical portfolios for advisors and need product reference data behind a search bar. Value discussion: fast search is a non-functional requirement; the real value is improved decision-making for sales conversations. Hardest question: who is the consumer, the internal salesperson or the end advisor? Lots of products have this ambiguity. The missing SLA is a design gap to resolve with the consumer and producer. Decision and reasoning: Identify the salesperson and advisor roles separately, then distinguish search speed from the sales decision enabled. Performance is one requirement; consumer value is another. Add the missing service promise.
- Holdings data. A flattened holdings/asset-reference record with multiple consumer categories: regulatory reporters ("keep us out of litigation"), shareholders, research analysts, sales. Value and ROI require further evidence. Holdings are scattered today, so the product is a consolidation view, and the value is abstracting complexity away from downstream consumers. Design implication: option value (creating the ability for other systems to develop around something) is intuitively valuable but the hardest kind of value to justify, and often where the most value lies. Decision and reasoning: List the different consumer decisions and what complexity the consolidation removes for each. Option value may matter even without a current monetary estimate. Record the assumptions rather than claiming a measured return.
- Capital market assumptions (CSG). Parts of the product already exist, but they always need a few extra data points from external vendors, and each request is a slow procurement process, so the value is a data product that includes everything they need. Legal and compliance questions include whether vendor terms permit use by an AI workflow. Check the actual contract and renewal terms; a technical capability does not establish permitted use. Decision and reasoning: Include procurement delay and permitted vendor use in feasibility and cost. Extra external fields can change both timing and rights. Ask the authorised owner to confirm the current terms.

Learning Review

Retrieve the distinction between an observation, an inferred motivation and a product requirement. Apply it to a consumer who reproduces a report in Excel: what would you ask before concluding that the problem is trust? Check that the proposed response addresses the actual workflow, names the producer/approver dependency and retains evidence for the chosen decision. Return to the interview when those facts are missing.

Optional activity: planning stakeholder communication

Optional adoption planning

OUTPUTS A communication plan and decision-rights register

In groups, choose an existing data product in your organisation. Who are its stakeholders? Is it successful, meaning used by its consumers and meeting its SLAs?
Now suppose you want to build a version 2. Who would block it, and why? Who would it affect negatively? Positively? Who should be involved in each design decision, who can represent a role, and why? Who should be kept *informed but not involved*? Finally, suppose you do all this analysis and it still fails. Why? What did you miss? Choose a product with a real change in responsibilities or working practice.

Case study: Adoption decisions and consequences

Client and party master data. A heavily used enterprise product has late nightly completion and merge/unmerge problems in unique client profiles. A proposed second version adds critical-event updates during the day and improves integration. Decision: use an architect or engineering manager to represent feasibility during initial discovery, then involve the implementation engineers at a declared scope checkpoint. Reasoning: discovery needs time to define the desired outcome, while delivery needs technical challenge before promises are fixed. Lesson: document which decisions require engineering input and share the context; "we will involve them later" is not a decision rule.

Secure-lending application. A vendor product replaces an internal interface, leaving the old application as a back-end feed. Users outside the main group must learn a new interface; the maintaining team loses some control and institutional influence. Decision: agree the changed operational responsibilities, vendor SLA and transition support before migration. Reasoning: responsibility may shift to the vendor, but internal consumers still need a clear accountable contact. Lesson: a marginal user may value simplicity while a power user values specialist capabilities. Evaluate both workflows and preserve critical requirements rather than treating all resistance as unwillingness to change.

Disclosure system. A dynamic, text-based disclosure replaces static tags. Existing users know the tag system, and an incorrect disclosure has serious business consequences. Decision: investigate reliability objections, confirm approval criteria with the accountable role, and compare the systems in parallel for a defined transition period. Reasoning: validated calculations alone do not address uncertainty about a new workflow or lost review control. Lesson: distinguish a real reliability defect from an adoption concern through evidence and interview questions; avoid dismissing a concern because the new system appears technically sound.

Web analytics migration. An old internal system requires increasing support, while the migration has substantial cost and limited immediate reporting benefit. Decision: involve the budget owner with a clear explanation of support risk, maintainability and service promises. Reasoning: uncertain benefits should remain uncertain rather than being given invented metrics. Lesson: use a measurable commitment when evidence supports it, and explain qualitative benefits in plain language when it does not. “Available every day except a stated maintenance allowance” can be easier to review than a percentage with no assumptions attached.

Further practice and optional clinics

Choose an optional clinic or reference after completing your assigned work. The event clinic uses its own local fixture; dated tutorials are available through the reference links. Projects contents.

Version 2 challenge route

Practice building and reviewing data products through seven tools. Foundation and advanced phases have separate starters, so you can begin either route with the required prerequisites. Read the phase's purpose, dataset, starter behavior and completion standard before editing.

Prepare your environment

Open the supplied training project in the dev container and inspect the delivered tool versions. Git foundation and local fixture inspection can proceed without cloud access. Docker needs its prepared engine and images; connected dbt and capstone need the approved Databricks identity/warehouse/namespace; Airflow needs the actual Astro scheduler and worker boundary. Sign in through the delivered setup outside recordings and keep credentials out of source, evidence and package files. Check installed versions with each tool's version command.

Use an isolated copied workspace for each attempt and inspect its execution directory. The version-explicit mise prepare task refuses an existing destination; reset also preserves existing attempts by requiring a fresh path. Choose owned relation prefixes, DAG names, container tags and optional event topic/groups so simultaneous learners do not collide. Never create extra schemas or delete broad resource sets to repair a training issue.

Choose the work

Tool	*Foundation*	*Advanced*	*Classroom role*
Git	Local snapshots, branches and conflict	Authorized sandbox review and synchronization	Preparation/ collaboration clinic
Docker	Lifecycle, mount and app packaging	Tested targets, user and runtime contents	Supervised runtime practice
dbt	Supplied Portfolio sources/ staging/report	Dated joins, reconciliation, correction and history	Wednesday selected core
Databricks CLI	Read-only identity and metadata discovery	Not offered	Environment preparation
Airflow	Same Portfolio product on Astro	Readiness, failure/repair/replay and handoff	Thursday selected core
Kafka	Optional prepared event clinic	Not offered	Optional further practice
CG product-information capstone	Individual ETF core	Pair integration and individual temporal/workflow components	Friday product event

Advanced phases have disclosed successful entry baselines and do not require a completed peer answer. Optional stretch follows the core completion gate with separate prerequisites and timing. Kafka cannot become a hidden requirement of mandatory work.

Run and assess an attempt

From the training project root prepare and verify through mise; inside the copy use direct tool commands so you can inspect what the tool does. Explicit live mode reaches the declared connected boundary.

Offline support, parsed SQL, local JSON and compilation cannot establish Databricks or Astro completion. An untouched starter may pass its baseline check while being rejected as completed learner work. Expected instructional failures are named before the drill, and unrelated prerequisite failures stay distinct.

```
mise run challenge:v2:prepare dbt phase-one-certified
mise run challenge:v2:verify-starter dbt phase-one-certified --mode offline
mise run challenge:v2:verify dbt phase-one-certified --workspace
 ↪  .local/challenges-v2/dbt/phase-one-certified --mode live
```

Read the guides effectively

Each task presents Concept, Challenge, Hint and Check, with required work visible and optional hints in native details disclosures. Horizontal rules separate task boundaries. Use phase contents and task/reasoning return links; compare end-of-phase reasoning after your own attempt. Full runnable reference assets are supplied separately to instructors. Keep the commands, results and business explanation in your attempt's evidence note. Mark any check you could not run.

Portfolio Analytics uses its runtime setup. The CG capstone uses its own workspace instructions and assigned target. Follow each guide's commands and retain previous attempts.

Portfolio Analytics runtime

Read SETUP.md for the terminal and Astro worker setup. `lab.py` prepares independent checkpoints, validates configuration, computes CSV reference outputs and checks actual Databricks results. Use CSV reference outputs to predict results, then check the actual Databricks relations. `dataset-runtime-contract.json` binds the supplied data and routes.

Historical dbt extension

The earlier extension is retained in the separate instructor archive and authoring workspace. Its setup and claims require review before reuse; it is outside the current learner assignment.

Earlier examples assume missing FX can default to 1.0. The current project requires a matching dated rate; missing enrichment must fail its checks. Storage tuning also needs measured query evidence.

Use the current advanced phase for the supported tasks, independently prepared baseline and evidence checks. Ask the instructor to select any additional historical exercise.

Historical airflow extension

The earlier extension is retained in the separate instructor archive and authoring workspace. Its setup and claims require review before reuse; it is outside the current learner assignment.

Earlier SLA and readiness statements describe a different runtime. A static check does not establish scheduled success, worker access, publication or production readiness.

Use the current advanced phase for the supported tasks, independently prepared baseline and evidence checks. Ask the instructor to select any additional historical exercise.

kafka challenges

Kafka is an optional event-bridge clinic with a prepared broker, consumer and processing runtime. It teaches event meaning, partition/offset progress, run identity, checked output and a bounded replay policy. The retained small A/B/C fixture and local processing chain are distinct from assigned Databricks Portfolio work. Completing this clinic cannot replace the required dbt/Airflow evidence, and skipping it cannot block the mandatory capstone.

Before you start

The instructor supplies the allocated bootstrap address, topic/group, bridge interpreter and isolated processing runtime. Prepare a copied optional attempt through mise and inspect actual versions and resource ownership. Do not point experimental records at a production broker or share another learner's consumer group. Broker/image preparation stays outside the timed core. A port collision or unavailable engine is a prerequisite condition, not the intended contract rejection. The valid event JSON and allowed immutable fixture location can be inspected locally while actual broker/run checks remain pending.

Suggested timeline and tasks

Estimates include reading, practice, checks and explanation. Allow additional time for environment setup and optional extensions.

- Phase: Foundation · Task: Read the dataset-arrived contract · Minutes: 12–18
- Phase: Foundation · Task: Start the prepared local broker and inspect a topic · Minutes: 12–18
- Phase: Foundation · Task: Produce and follow a uniquely identified event · Minutes: 12–18
- Phase: Foundation · Task: Verify the dbt output belongs to the event · Minutes: 12–18
- Phase: Foundation · Task: Replay and explain duplicate policy · Minutes: 12–18

Choose your phase

- Foundation: Optional clinic; mandatory course work does not depend on it.
- Advanced: Not offered: no tasks or completion gate.

Finish the core tasks before choosing optional stretch.

Teaching concepts

Producer acknowledgement, consumer offset and downstream receipt describe different progress. Event identity/version/location must agree across broker, bridge and actual run/output. An identical completed replay can reuse a run under the supplied bounded policy; same identity with changed content is a conflict. This is not a general distributed exactly-once guarantee. Reasoning keys follow the owning phase, full runnable bridge references remain separate, and the optional clinic's runtime boundary is stated without borrowing assigned Databricks acceptance.

kafka: Phase 1 — foundations

Purpose and outcome

Explore how a dataset-arrived event can trigger a prepared processing workflow without making the event the dataset itself. This is an optional event clinic. You inspect a contract, use a supplied broker/consumer bridge, follow event identity into an actual run/output and reason about replay. The bridge and processing baseline are prepared so learners do not need to build Kafka infrastructure, author a complete consumer and repeat dbt/Airflow labs in one phase.

Dataset and pipeline

The event contains event identifier, type, dataset identifier/version, controlled fixture location and arrival timestamp. The small immutable holdings fixture produces exposure A 150, B 100 and C zero. This retained event fixture is intentionally separate from the assigned eight-seed Portfolio Analytics pipeline. Its prepared local Airflow/DuckDB chain demonstrates event identity and offset semantics; it cannot establish Databricks/Astro completion for the assigned guides.

Kafka retains records in an ordered partition log, and a consumer group tracks progress through offsets. The supplied bridge validates allowed input identity/location, triggers the prepared DAG and records partition/offset/run evidence. A receipt and actual data check establish downstream completion separately from producer acknowledgement. Identical completed replay can reuse the same run, while a conflicting payload with the same event identity is rejected according to the declared bounded policy.

Prerequisites and workspace

The instructor supplies the prepared broker, unique topic/group, bridge interpreter and isolated Airflow/dbt runtime. Inspect actual installed versions rather than copying source pins into measured evidence. Use version-explicit mise preparation from the training project root and keep event/evidence edits inside the copied attempt. Image pulls, broker setup and port allocation belong to preparation outside the unpiloted clinic estimate.

```
mise run challenge:v2:prepare kafka phase-one-certified
cd .local/challenges-v2/kafka/phase-one-certified
python -m json.tool event.json
```

Record the approved bootstrap address and owned topic/group. Do not point an experiment at a production broker or consume a shared group's offsets. The prepared local port may already be occupied; diagnose that ownership/preparation condition rather than stop an unknown service. The runtime path and bridge flags should be inspected through delivered help. No hosting account, infrastructure deployment or mandatory capstone dependency is introduced.

Starter behavior

The event JSON is valid and the supporting processing graph is supplied. A starter parse/contract validation can succeed before any broker or workflow runs. It is not a completion pass. Keep learner event edits separate from the immutable baseline and never execute an arbitrary producer-provided filesystem path. Broker absence, port collision or missing runtime is a prerequisite failure, while malformed or conflicting event identity is an intentional contract rejection described by its task.

Timeline and completion standard

Core reading, authoring, checking and explanation: 60–90 minutes. Allow additional time for setup and optional stretch.

- Task: Read the dataset-arrived contract · Minutes: 12–18 · Evidence: Valid unique event and predicted small-fixture output without arbitrary path execution
- Task: Start the prepared local broker and inspect a topic · Minutes: 12–18 · Evidence: Current intended topic/partition metadata in prepared isolated broker

- Task: Produce and follow a uniquely identified event · Minutes: 12–18 · Evidence: Event ID, broker partition/offset and current Airflow run identity
- Task: Verify the dbt output belongs to the event · Minutes: 12–18 · Evidence: Matching event/run receipt and independent A 150, B 100, C 0 output
- Task: Replay and explain duplicate policy · Minutes: 12–18 · Evidence: Completed run reused, output unchanged; conflicting payload rejected without accepted duplicate

Independent starter files Read the owning phase before editing. Prepare from the training project root with `mise run challenge:v2:prepare kafka phase-one-certified`; edit the copied attempt.

Authored-file list

Task	*Learner-authored paths*
Read the dataset-arrived contract, Start the prepared local broker and inspect a topic, Produce and follow a uniquely identified event, Verify the dbt output belongs to the event, Replay and explain duplicate policy	`learner-event.json`, `evidence.md`

Declared baseline

`event.json` is the valid baseline. Create `learner-event.json` with a unique event ID and keep your explanation in `evidence.md`. The bridge generates `event-evidence.json` and the completed workflow generates `receipt.json`; inspect these outputs without editing them.

Use a fresh workspace for another attempt; preparation and reset preserve existing work.

Core challenges

Read the dataset-arrived contract

Concept

An event describes an occurrence and the input it refers to; it does not carry every row of that dataset. Identity supports tracing and replay decisions, version identifies the expected fixture semantics, and location is a controlled reference rather than permission to execute an arbitrary path. The arrival timestamp describes event context, not automatic proof of processing completion.

The bridge validates the small fixture contract before it triggers work. A valid record can therefore be accepted as input while its downstream run remains pending. Reusing an event identifier with a different payload changes its meaning and must be rejected under the declared policy. Kafka's retained log and consumer offsets track broker progress; they are separate from a receipt that says the data result was checked. Keeping those boundaries explicit prevents a syntactically valid JSON event from becoming a false end-to-end pass.

Challenge

Inspect `event.json` and create `learner-event.json`, changing only `event_id` to your unique value. Retain `dataset_id` holdings, version `fixture-v1` and location `fixture://holdings-v1`. Explain what each field identifies and predict the A/B/C exposure output.

Validate your copy using the supplied bridge and retain the validator result in `evidence.md`. Keep the baseline event unchanged. A location is a controlled input reference, not permission to execute a host path. Explain how arrival time differs from processing completion and why another payload under an accepted event ID tests a conflict rule rather than a fresh trace.

Validate input meaning before following its execution

Hint

The supplied pipeline accepts exactly this prepared dataset contract. Kafka is optional and does not gate mandatory Airflow/capstone.

Compare your event with the validator's allowed dataset/version/location contract. A new event ID is appropriate for a new trace; changing a payload under an already accepted identity tests a different rule. Inspect validation output without assuming its run identifier means a run actually exists. Arrival time and processing state should remain separate evidence fields.

Check

The learner-event copy must parse and pass the declared contract validator, with a unique identity and allowed fixture reference. Retain the predicted A/B/C exposure values and explain why the event is not the holdings dataset itself. A malformed location or unsupported version must be rejected for that contract reason rather than executed.

Validation output is supporting input evidence only. It cannot prove producer delivery, consumer progress, Airflow execution or dbt output. Record those downstream boundaries as pending until their tasks execute. Keep canonical input bytes and any deliberate invalid-event copy separate so the valid baseline can be reproduced.

```
python -m json.tool learner-event.json
# From the training root, inspect the delivered bridge interface.
python3 scripts/v2_kafka_bridge.py --help
```

Phase contents · Reasoning key

Start the prepared local broker and inspect a topic

Concept

A Kafka topic contains partition logs: ordered lanes of retained records. Each record has an offset, its position in that lane. A consumer group maintains its own progress. Two groups can consume retained records independently, but two learners accidentally using the same group can affect each other's observed offset progression. Topic/group ownership is therefore part of the classroom setup rather than an optional naming preference.

A broker metadata response establishes that the intended topic/partition is reachable. It does not mean an event was produced or a workflow completed. The prepared local runtime keeps infrastructure work outside this short clinic. A port collision or missing container engine is a setup condition; it should not be repaired by deleting unrelated services. The learner's task is to inspect the actual allocated boundary and explain its role.

Challenge

Use the instructor-prepared broker, bootstrap address and allocated topic/group. From the copied starter, start only this clinic's Compose project when needed, then run the bridge's topic preparation and metadata inspection through the supplied runtime interpreter.

Retain the topic, partition and group identity before producing a record. Confirm the Compose project and local port belong to your attempt. Wait for broker readiness and distinguish an unreachable broker from an absent allocated topic. Resolve collisions with the instructor instead of stopping an unknown service. Explain what the metadata response establishes and which later producer/consumer observations are still needed.

> Inspect the allocated log and group boundary before producing
>
> **Hint**
> The compose broker binds localhost:19092. Instructor preparation includes image pull and the isolated Airflow deployment.
> Compare the bootstrap port and Compose project ownership before diagnosing a connection failure. A metadata response should identify the allocated topic, not merely any topic in the broker. Keep group identity separate from topic name. The bridge's preparation action may create the owned topic but does not publish a dataset-arrived record.

Check

The current broker metadata must return the intended owned topic and partition without an error. Retain bootstrap/topic/group identity and measured runtime context. Explain what remains unproven: record delivery, consumer processing and data output. A source Compose file or successful JSON parse is not broker readiness.

If the local port is occupied or engine unavailable, record that exact prerequisite and leave broker observations pending rather than removing an unknown service. Topic inspection must not use a shared production consumer group. The finite optional clinic assumes prepared infrastructure and does not assess broad provisioning.

```
# Inspect actual bridge/broker metadata using the allocated topic and group.
# Use the prepared runtime interpreter and supplied --bootstrap/--topic flags.
```

Phase contents · Reasoning key

Produce and follow a uniquely identified event

Concept

Producer acknowledgement means the broker accepted a record at a partition/offset. It does not mean a consumer processed it or a downstream data job succeeded. The supplied bridge reads the event, validates its input contract and triggers the prepared Airflow run with a deterministic identity. Its evidence records broker position and the corresponding run so those boundaries can be traced.

Consumer offset progress also has a policy: acknowledging or committing a consumed record can occur before downstream success, depending on the bridge contract. The learner must inspect that behavior rather than call every committed offset completed data. A uniquely identified event avoids confusing a current trace with a prior record already retained in the topic. Actual task states and receipts provide later execution evidence.

Challenge

Use the supplied bridge to produce `learner-event.json` to the allocated topic, then consume one record with your assigned group. Retain the producer acknowledgement's partition/offset and the bridge's decision and Airflow run ID.

Follow that exact run in the prepared processing runtime until it completes or reaches the supplied wait bound. Keep event JSON, bridge evidence and run identity together. Compare the run configuration with the consumed payload rather than inspecting any latest green DAG. Explain whether consumer progress was committed before downstream completion, and what a pending or failed run means for recovery. Do not execute producer-provided paths.

Link broker position to the exact downstream run

Hint

The bridge uses the instructor-configured Airflow CLI inside the runtime boundary; keep broker/Airflow reachability explicit.

Read the producer acknowledgement and consumer evidence as two different records. Follow the event identifier and deterministic run ID into the actual prepared Airflow runtime. A producer's successful exit says only what its acknowledgement reports. If the run is pending, retain that state and wait within the supplied bound instead of calling the chain complete.

Check

Retain the produced event identity, broker partition/offset, consumer decision and actual matching Airflow run. The event fields must agree across those artifacts. Inspect eventual task state and mark it pending until execution finishes; a successful producer command alone cannot close this task's chain observation.

Explain whether the supplied bridge commits consumer progress before or after downstream acceptance, and what a pending/failed run means for recovery. Use the owned topic/group and bounded wait. Missing broker or processing runtime remains a blocked prerequisite, while a malformed event rejection should retain its specific cause. Do not fabricate a receipt to make the trace look complete.

```
# Inspect the exact event-evidence.json and its identified prepared Airflow run.
python -m json.tool event-evidence.json
```

Phase contents · Reasoning key

Verify the dbt output belongs to the event

Concept

An event-driven result must be associated with the input event that selected it. Correct-looking fund totals from a previous run or another fixture are not sufficient. The receipt should bind event payload and run identity, while independent dbt output checking confirms the controlled fixture's per-fund values and population. A shared grand total cannot reveal values shifted between funds.

This optional clinic deliberately uses the small A/B/C fixture and local prepared chain. Its evidence must not be presented as the assigned Databricks Portfolio product. The independent reference protects both fund membership and native fixture values. A zero-valued C holding still belongs to the expected population, so dropping it because it contributes nothing to a sum would change the output contract. Trace identity and business values together.

Challenge

After the event-triggered run completes, inspect `receipt.json` and compare its event ID, version, location and run ID with the produced record and `event-evidence.json`. Query the copied processing database or use the supplied output checker.

Verify all three fund rows: A 150, B 100 and C zero, with total exposure 250. Keep C in the population even though it adds nothing to the sum. Retain the per-fund comparison and explain which artifact identifies input, execution and checked output. Use this run's project/output path so a database from an earlier attempt cannot supply misleadingly correct values.

Verify event identity and each fund value independently

Hint

Use the supplied expected-output oracle and the event metadata, not just a successful task state. Compare the complete event payload, not only its ID, with the receipt. Then compare per-fund output rather than an overall 250 total. The C row's zero value is a membership obligation. Inspect the exact project's output path and run identity so a retained database from a prior attempt cannot provide apparently correct evidence.

Check

The actual receipt and bridge evidence must agree on the event payload and run ID, and the current prepared dbt output must show A 150, B 100 and C zero with all three keys retained. Retain independent per-fund checks and actual run/task completion. A producer acknowledgement or a manually written JSON receipt does not satisfy the chain.

Explain why the zero-valued fund remains part of the expected result and why a grand sum alone is weaker. State the local optional fixture execution boundary honestly; it cannot close assigned Databricks/Astro acceptance. Keep missing runtime or pending run states explicit, and preserve baseline source bytes throughout the check.

```
# Inspect the prepared run receipt and invoke the delivered verify-chain interface.
python -m json.tool receipt.json
```

Phase contents · Reasoning key

Replay and explain duplicate policy

Concept

Replay policy defines how retained records are interpreted when the same event is seen again. The supplied bridge can reuse an identical completed run when its receipt matches the event payload. A record with the same event identity but changed payload is a conflict, not a new legitimate input under that policy. The broker can assign a later offset while the bridge reuses the prior run.

This is a bounded teaching strategy based on a matching completed receipt, not a general exactly-once guarantee. An existing run without a matching completed receipt needs explicit recovery inspection, and concurrent events or a broader deduplication store introduce additional design questions. The learner should explain those limits rather than claiming that a successful replay solved all distributed failure modes. Business output stability and run reuse are separate checks.

Challenge

Produce the identical valid event again and consume it through your allocated group. Inspect completed-run reuse, the new broker offset and unchanged per-fund output. Retain the accepted receipt before the replay.

In a separate event copy, change a payload field while retaining `event_id`. Predict and inspect the conflicting-payload rejection and its offset behavior. Keep the accepted receipt and baseline event unchanged. Explain what recovery requires if an earlier run exists without a matching completed receipt. After saving both experiments, remove only the clinic's named Compose project when instructed; broad pruning would affect unrelated resources.

Replay depends on matching completed meaning, not ID alone

Hint

The bridge commits offsets only after an accepted/new-or-identical run. A conflict remains uncommitted for operator diagnosis.

Inspect the complete accepted receipt before replay so the reuse precondition is real. A later broker offset does not require another data run when the policy accepts identical completed replay. For the conflict, change only a declared payload field under the same identity and predict rejection. An incomplete prior run is a separate recovery state, not permission to fabricate a matching receipt.

Check

Identical completed replay must retain the same run identity and unchanged A/B/C output while recording the new broker position and reuse decision. The conflicting-payload copy must be rejected for the intended identity/content reason according to the documented offset policy. No accepted conflicting run or overwritten baseline receipt may appear.

Explain what happens if the earlier run exists without a matching completed receipt and why this bounded strategy is not distributed exactly-once processing. Retain input/receipt hashes and both decisions. Clean only owned topic/group/Compose resources after preserving evidence. Offline event validation remains supporting; actual replay and rejection through broker/bridge/runtime stay pending without the prepared environment.

```
# Inspect replay evidence and compare it with the retained accepted receipt.
python -m json.tool event-evidence.json
```

Phase contents · Reasoning key

Phase completion

This optional clinic completes with a uniquely identified dataset-arrived record, observed broker partition/offset, the actual prepared Airflow run, a matching receipt and independently checked fixture output. The local event fixture expects fund exposure A 150, B 100 and C 0; it is a separate event-bridge clinic, not the assigned Portfolio Analytics Databricks product. Retain an identical-event replay that reuses the completed run and preserves output, plus a conflicting-payload rejection with the stated offset policy.

Explain producer acknowledgement, consumer offset and downstream completion as separate observations. The supplied bridge is bounded and its completed-receipt policy does not establish distributed exactly-once behavior or arbitrary concurrent event deduplication. Use the instructor-provided topic/group/runtime and clean only the owned clinic resources after preserving evidence. Offline JSON validation is supporting input evidence; the broker, actual Airflow and dbt chain remain pending without their prepared runtime. Kafka is optional everywhere and phase two is not offered. Mandatory dbt/Airflow/CG product-information completion never depends on this clinic.

Optional stretch

This route has one phase. There is no required stretch or advanced assignment.

kafka phase reasoning keys

Reasoning keys

Read these explanations after attempting the checks.

Reasoning: Read the dataset-arrived contract

Identity and controlled dataset/version/location establish which input the event means. The validator must reject arbitrary locations and conflicting semantics. A computed run ID is a trace identifier, not proof that a run exists. Broker delivery, consumer offset and independently checked output are later separate observations.

Return to Read the dataset-arrived contract

Reasoning: Start the prepared local broker and inspect a topic

Topic metadata establishes the allocated broker/log boundary. Group identity controls this learner's progress and should not be shared accidentally. It does not establish production or downstream completion. Missing engine/port/topic preparation stays a specific prerequisite condition, and only owned clinic resources may be started or cleaned.

Return to Start the prepared local broker and inspect a topic

Reasoning: Produce and follow a uniquely identified event

The broker's partition/offset proves accepted record position. The bridge maps that record to an actual run, while task state indicates execution progress. Offset commit policy does not automatically imply checked data completion. Retain matching event fields and current identities, leaving pending runs visible rather than turning producer success into an end-to-end claim.

Return to Produce and follow a uniquely identified event

Reasoning: Verify the dbt output belongs to the event

The receipt ties the full event to its run, and independent per-fund output confirms the selected fixture. C remains a valid zero-valued member. Correct totals from another project or manually authored receipts do not establish the chain. This optional local clinic has its own evidence boundary and cannot substitute for the main Databricks route.

Return to Verify the dbt output belongs to the event

Reasoning: Replay and explain duplicate policy

An identical payload with a matching completed receipt can reuse the prior run despite a later broker offset. Same identity with different content is rejected. Existing incomplete work needs another recovery decision, and this simple receipt strategy is not a general exactly-once solution. Compare business values as well as run reuse and preserve the accepted baseline evidence before owned cleanup.

Return to Replay and explain duplicate policy

CG capstone optional practice

Complete the current capstone phases and personal submission first. Select additional work from the project's optional practice, with its own scope and time budget. Keep any synthetic temporal exercise separate from the frozen public product snapshot.

Optional reference examples

Choose a follow-up after completing your assigned project:

- Small synthetic exercises for additional design and query practice.
- dbt advanced practice for reconciliation, corrections and replay. Earlier tutorials are retained separately as dated instructor reference.
- SQL readiness check to practise the query patterns used in dbt.

The archived mini-lakehouse example compares storage, transformations and publication. It is a conceptual reference; its MinIO/Spark/Kubernetes setup is separate from Databricks/Astro practice.

SQL readiness check: five patterns

> **Knowledge check: Read the SQL meaning**
>
> Before running a query, can you name its grain, join relationship and treatment of unmatched or null records? Which small fixture would distinguish two plausible interpretations?

Predict the grain, join behaviour and returned rows for each query before running it. Then vary one condition: use a left join, add a grouping column, stack another CTE, introduce a cast, or change the filter.

1. A staging rename.

```sql
select id as customer_id, name as customer_name from customers
```

Same rows, two renamed columns, which is what a staging model does.

2. A two-table join.

```sql
select o.order_id, c.customer_name
from orders o
join customers c on o.customer_id = c.customer_id
```

With unique customer keys, one row per order that has a matching customer; orders without a match disappear. That disappearance is the difference between `join` and `left join`, and it matters in the intermediate models.

3. A GROUP BY with a filter.

```sql
select customer_id, count(*) as order_count
from orders
where status = 'complete'
group by customer_id
```

One row per customer: grouping changes the grain from orders to customers. The filter runs first, then the grouping collapses orders into customers.

4. One CTE.

```sql
with completed as (
    select * from orders where status = 'complete'
)
select customer_id, count(*) as order_count
from completed
group by customer_id
```

Identical output to snippet 3: a CTE is a named sub-query, and the intermediate models are built out of them.

5. The negative amount.

```sql
select order_id, amount from payments where amount < 0
```

If negative payments violate the contract, this query returns the offending rows. It returns no rows when that rule holds. A dbt data test uses the same pattern.

Explain each result before using the pattern in your project. If a prediction surprises you, try a small input with an unmatched key, a null or a duplicate and compare the result.

Temporal-history clinic

After completing the assigned transformation work, use the optional temporal-history clinic to reason about business-valid time, system knowledge, retrospective corrections, balance observations, retries, and checkpoint/output consistency. Allow 45–60 minutes, separately from Wednesday's 165-minute dbt block and Thursday's 120-minute Airflow block. Read Historical data and bitemporal loading first.

The clinic uses a small independent history and an owned local attempt. It does not modify the Portfolio Analytics or CG product-information fixtures, require a hosted deployment, or establish that a production Caspian macro has been validated. Its output is a reviewed history, two-clock query results, and a recovery explanation. The facilitator keeps the full answer key separately.

Reference appendices

Use these appendices to prepare for the course and look up business terms, platform conventions and practical instructions while you work.

- Appendix A — Readiness
- Appendix B — Data contracts and decision records, worked
- Appendix C — Capital Group conventions
- Appendix D — dbt and tooling reference
- Appendix E — Environment and troubleshooting
- Appendix F — Managing stakeholders
- Appendix G — AI reference
- Appendix H — Using VS Code and the training dev container
- Appendix I — Historical data and bitemporal loading
- Appendix J — Astro operations and runtime ownership

Appendix A — Readiness

Use this appendix to prepare for the course. Read the business explanation before the design exercises, or go straight to the practical instructions to open the workspace, check your setup, or find a definition.

Investment and organisational context

Understanding the business comes before choosing a transformation. A SQL query can run correctly and still answer the wrong question if it treats a client's account as a fund, a trade as new money entering the account, or a transaction fee as an advisor's income. The sections below explain the distinctions you will use when designing the course's data products. If you need to prepare your computer first, follow Open the correct project and Setup and readiness, then return here before the design exercises.

Capital Group and the Capital System

Capital Group describes the Capital System as an investment approach in which several portfolio managers contribute to one portfolio. They share research and ideas while making their own investment decisions. The published explanation also describes investment groups that make decisions independently. The Capital System.

This helps explain why "portfolio" can mean different things in different conversations. It can mean the whole collection of investments, or a particular manager's part of that collection. A manager's part is often called a portion or sleeve. When you work with a real CG dataset, ask its owner which level the portfolio identifier represents. Do not assume that a field called `portfolio_id` always identifies the same kind of thing.

An account answers a different question: which investments are recorded for this investor? A fund brings investors' money together to invest under a shared objective. The investor owns shares in the fund, while the fund holds the underlying investments. A client-facing advisor helps serve the investor's relationship and needs. The portfolio manager decides how to invest within the authority they have been given. These roles and levels are related, but they are not interchangeable.

Perspective	*What it describes*
Investor and account	An investor has an account with recorded positions. Those positions may include shares in funds.
Investment management	A fund or investment mandate has a portfolio of investments. Managers may be responsible for different portions of it.
Client relationship	An advisor serves clients and uses reports about their accounts. The advisor is not necessarily the manager choosing a fund's investments.

These are ways of understanding the business, rather than a diagram of CG's databases. The exact relationships and names depend on the service and dataset you are working with.

A share class is a particular class of shares in a fund. Classes can have different charges and arrangements for paying financial professionals, even when they belong to the same fund. Capital Group's share-class explanation gives examples of these differences. The CG capstone selects Class F-3 offerings and published fund expense ratios; these figures do not describe every investor cost.

A model portfolio describes a proposed mix of investments. A target allocation says how much is intended to be invested in each part. Actual holdings describe what is held on a particular date. A mandate sets the investment objective, authority and restrictions. A list of securities and percentages could appear in any of these, so ask what the percentages mean before using them.

Ownership, look-through and double counting

Suppose a client account owns fund shares worth 1,000 GBP. A look-through report explains that investment as 600 GBP of equity exposure and 400 GBP of bond exposure. The client still has 1,000 GBP invested in the fund. Adding the fund shares and the two underlying exposures would count the same money twice and produce 2,000 GBP. Use the underlying exposures to explain the investment, not to add another layer of owned assets. The same problem occurs if you add a whole portfolio's value to the values of the manager portions that make it up.

Two accounts and several funds can all hold the same security. Finding that shared security does not show that either account owns shares in those funds. To establish fund ownership, you need a record of the account's fund shares or another clearly defined ownership relationship. A join on a common instrument can repeat one trade across several funds and inflate a total. Being able to join two tables is not enough to show that the result makes business sense.

People, products and accountable roles

Investors provide the money being invested. Financial advisors support client relationships. Portfolio managers make investment decisions, and analysts contribute research. Building a data product introduces another set of responsibilities: someone supplies the source data, someone builds the product, someone agrees its business meaning, someone uses it, and someone supports the platform on which it runs.

For example, a trading-operations team might supply executions in CSV files. That team is the producer; the CSV is how the data is delivered. A creator checks the input and builds an activity report. The business owner agrees what the report's fees and totals mean. An advisor or reporting analyst consumes the result, while a platform operator keeps the scheduled job running. This example illustrates the responsibilities; it is not an organisational chart for CG.

One person may carry out several of these jobs, but you should still be able to say who answers each question. Who can explain or correct the source? Who agrees the product's meaning? Who approves its use? Who responds when it fails? The person funding a project may be different from its data owner, and the person approving a release may be different from its consumer. Titles such as DPM or steward help only when their responsibilities have been explained for the product.

Events, states and financial measures

A trade execution records a completed purchase or sale of an instrument. A fund subscription means buying participation in a fund, and a redemption means selling that participation back. A contribution or withdrawal moves money into or out of an account. Selling an investment inside an account usually replaces the investment with cash; it does not necessarily mean the client has withdrawn money.

A position describes how much is held at a particular time. To calculate a complete closing position, start with the opening position, add purchases, subtract sales and include other changes such as transfers or share splits. If you only have a list of trades, you can calculate the change represented by those trades. You cannot know the complete closing balance without knowing what was held beforehand and whether anything else changed. A negative result could represent a short position, but it could also arise because the opening holding or another event is missing.

An execution price is the price paid or received in a trade. A valuation price is the price chosen to value a holding at a particular time. An independent daily price feed can provide a valuation price even when the account has not traded that day. The latest execution price can be a useful classroom substitute, but it may be old and is not necessarily the market's closing price. Check the price's date, currency and units before multiplying it by quantity. A bond quote per 100 of face value needs different handling from a price per share.

Market value estimates what a position is worth using an appropriate price. Assets under management, or AUM, describes the value of assets managed within a stated collection of accounts or investments. You must explain what is included, when it is valued and which currency is used. Net asset value, or NAV, is assets less liabilities. NAV per share divides that amount by the number of outstanding shares. Averaging underlying security prices does not calculate either measure. Investor.gov's NAV definition.

Cost basis is the acquisition cost assigned to an investment under the relevant accounting rules. When part of a holding is sold, those rules determine which acquisition costs belong to the sale and which remain. Realised gain concerns sold investments; unrealised gain concerns investments still held. Purchases less sales do not automatically give you the remaining holding's cost basis. Calculating an investment return also requires a method for handling valuations, income, time and money entering or leaving the account. Trading activity and transaction fees alone cannot give you that return.

Transaction fees are charges connected with executions. Management fees pay for management services under a separate arrangement. A fee paid by a client account does not tell you who received it. Keep these meanings separate in the report.

For every financial total, explain which records it includes, the date or period, the currency, the units and how missing values are handled. Adding USD and JPY numbers produces a number, but not a comparable measure of wealth. Checks must also protect the final report: valid trades can still be duplicated by a join, assigned to the wrong advisor or omitted from an aggregate.

The two supplied business cases

Portfolio Analytics is the fund-reporting case used on Wednesday and Thursday. It contains historical holdings with supplied market values. Preserve those values rather than replacing them with quantity multiplied by a quoted price. Dated exchange rates convert the values to GBP, and cash is supplied separately. The report divides holdings-only GBP value by 100 million synthetic units to demonstrate a calculation. That result is a teaching value per unit, not an official fund or share-class NAV.

Friday uses the CG product-information brief. The source snapshot describes ETFs, Class F-3 mutual-fund offerings and SMA strategies. It contains product identities, expense components, published returns and provenance. The direct internal consumer is a fictional Product Information Analyst.

CG source relationships

Source field	*Relationship / selection*	*Meaning*
`cg_products.product_id`	Select a declared `source_layer` before deduplicating	Offering identity; a share class has its own identifier
`cg_fees.product_id`	Join the matching offering and selected representation/component	Gross and net expense ratios are alternatives
`cg_returns.product_id`	Match declared basis, return kind, period and reporting date	A product can have several different published return observations
`source_id`, `source_url`	Retain the source evidence for each selected observation	A consumer can trace the published figure

Check key uniqueness and membership before joining. A catalogue/detail duplication can multiply observations; `DISTINCT` can conceal an unsupported source choice. Keep missing reporting dates null and distinguish them from retrieval/load dates. The phase-2 scope is 109 offering IDs; it does not represent account holdings or the full worldwide catalogue.

From source to consumer, and from code to execution

A data product often starts with operational events and reference data. The creator checks those inputs, gives them useful business definitions and prepares an output for a consumer. “Source-aligned” means organised around an originating source; “domain-aligned” means organised around reusable business meaning; “consumer-aligned” means designed for a particular use. These names describe the product’s purpose. Staging, intermediate and curated describe processing stages. A single product can use several processing stages, and bronze/silver/gold is another set of names for those stages.

Use the project instructions and naming conventions provided for your exercise. Other teams may use different repositories or deployment arrangements. The development container, Airflow workers, Databricks warehouse and deployment pipeline each have their own jobs and permissions. Running a local check is not the same as running the product on the warehouse, and passing a quality check is not the same as obtaining approval to publish it. For real work, ask the responsible team which procedures and approvals apply.

Why the course uses a dev container

VS Code is the editor: it shows project files, previews Markdown and opens terminals. A workspace is the project folder you have opened. The training dev container is the prepared Linux environment connected to that workspace. Its configuration supplies compatible tools, certificate settings and setup tasks so the class can run the same commands without installing every dependency separately on each computer.

Keep track of the execution location. A terminal opened in the connected VS Code window runs inside the dev container. A terminal you open from your computer’s desktop runs on the host. Docker also launches a separate Airflow runtime, and connected dbt work sends SQL to a remote Databricks warehouse. The assigned v2 route uses copied projects with dbt-databricks. Offline file checks and CSV calculations support the work but do not establish warehouse execution.

Place	*What happens there*	*How you recognise it*
Your computer / host	VS Code window, browser and Docker Desktop or the approved Docker host	Desktop applications; a host terminal has the host’s folders
Training dev container	Edit mounted project files; run mise, Git, Python, dbt and Astro CLI	VS Code remote indicator names the container; terminal `pwd` starts under `/workspaces/`
Separate Airflow runtime	Scheduler, API server and task execution started by Astro	`astro dev start` reports services and the UI address; Docker lists their containers
Remote Databricks	Authorized workspace/catalog metadata and warehouse SQL processing	Remote identity and connected result evidence; an installed CLI alone does not establish access

Closing an editor window does not mean you have stopped every Docker service. Stop a running practice Airflow runtime with the documented Astro command when you finish.

Open the correct project

You need the supplied training repository, VS Code with the Dev Containers extension enabled, an approved Docker environment, corporate network/certificate access for the supplied image registries, and the training access assigned by your instructor. Docker Desktop access may require the organisation’s `CPZ_INF_BB00000003_Docker_Desktop_User` role. Ask the instructor for the approved access route; do not create a substitute cloud environment.

If the Reopen command is missing, open **View** → **Extensions** in the host VS Code window, search for **Dev Containers**, and select the extension published by Microsoft. Choose **Install** or **Enable** as applicable. Where host extensions are centrally managed, ask the facilitator for the approved installation route before continuing.

1. Start Docker using the approved host procedure. In VS Code choose **File** → **Open Folder** and select the training project folder containing `.devcontainer/`, `.mise.toml`, `challenges/` and `data_products/`. In the authoring repository it is `learner/practice/client/`; in the complete candidate bundle it is `practice/client/`. After extracting `client-practice.zip`, open the extracted project root containing those four entries. An approved training checkout may be named `edo-training-data-products`; use its contents to identify the root rather than assuming the folder name.
2. Open **View** → **Command Palette**. Choose **Dev Containers: Reopen in Container**. The project already supplies the container configuration. Allow the first build and setup to finish; follow the progress notification to its startup log if it stalls.
3. Check the remote indicator at the bottom left of the VS Code window. It should name the connected development container. Open **Terminal** → **New Terminal** and check your directory using the commands below.
4. On a later visit, open the same project and use **Dev Containers: Reopen in Container** to reuse it. Use **Dev Containers: Rebuild Container** when the instructor changes the image/configuration or asks you to repair a failed build. Save files first; a rebuild recreates the environment and may need setup again.

The Command Palette shortcuts are Ctrl+Shift+P on Windows/Linux and Shift+Command+P on macOS. Menu/action names are the primary route. For more detail, see the official VS Code container guide.

Find files and previews

Use **Explorer** to browse the workspace and select a file. Save edited files with **File** → **Save** before running a command that reads them. A changed editor buffer and the file on disk can differ. For a Markdown file, use **Markdown: Open Preview to the Side** from the Command Palette to see its headings, tables and code blocks. Source Control shows changed files and diffs; review the actual change before staging or committing it. The **Ports** view shows forwarded services when the container starts them. These interface areas are described in the VS Code interface guide.

Project location	*Purpose*
`.devcontainer/`	Image definition, lifecycle hooks, setup scripts and readiness helper
`.mise.toml`	Tool versions, environment configuration and named project tasks
`.mise/env.sh`	Session environment and Databricks/dbt variable wiring
`challenges/v2/`	Canonical learner instructions and untouched starter fixtures
`.local/challenges-v2/<tool>/<phase>/`	Copied learner attempts created by the preparation command
`data_products/`	Prepared connected product examples; follow the phase guide for the copied attempt

Open a terminal and navigate

Choose **Terminal** → **New Terminal** in the connected window. The terminal is a shell: it interprets commands and arguments. Paste one command at a time and press Enter. Copy the contents of a code block without adding a prompt character. An option such as `--help` changes a command's behavior;

an argument such as `dbt` selects a tool route. The VS Code terminal guide explains opening additional terminals and managing terminal sessions.

```
pwd
ls
```

`pwd` prints your current directory; `ls` lists its entries. At the project root you should see `challenges`, `scripts` and `data_products`; hidden files such as `.mise.toml` need `ls -a`. If you are in a copied attempt and want to return, use the actual root shown when you first opened the terminal. For a checkout with this folder name:

```
cd /workspaces/edo-training-data-products
```

Replace that path with your workspace's actual root if it has a different folder name. `cd` changes directory; it produces no success message. Run `pwd` again to check. `cd ..` moves up one directory. Ctrl+C interrupts a command running in the current terminal. Open a second terminal if you need to inspect files while logs keep streaming in the first one.

Python and SQL are file contents, not shell commands. A `python3 ...` or `dbt ...` line belongs in the shell; a `select ...` SQL query belongs in the indicated SQL file or query editor. Read the expected output after each command. Stop at the first unexplained error and retain its non-secret message with the command and location.

Setup and readiness, in order

All commands in this section run from the training project root inside the dev container. The configuration's post-create hook runs `mise trust .mise.toml`, `mise install` and `mise run setup` automatically. Setup installs/configures the local shell and Python dependencies, bootstraps GitHub CLI from existing credentials, and installs the configured training-project dependencies. It does not run interactive Databricks login. The post-start hook starts/checks the inner Docker daemon and invokes `dpc-check`; its tolerated startup failure means you must inspect the report yourself.

Command / action	*Prerequisite and purpose*	*Expected evidence / next step*
`mise trust .mise.toml`	Recovery only if trust/setup was interrupted; approve the supplied project configuration	Trusted config; investigate a missing/wrong project file before retrying
`mise install`	Corporate network and configured tools; recover an interrupted install	Declared tools available; retain the first registry/network error
`mise run setup`	Recover failed local post-create setup	Named dependency tasks finish; follow the task's first failure before rerunning
`mise run setup:databricks`	Instructor-provided host/ warehouse and authorized learner identity; interactive OAuth-first setup	Complete the browser login; script configures profile and environment files
Open a new terminal	After authentication or shell-hook changes	New session loads generated environment; return to project root
`dpc-check`	Check the current container and configured access	Read every FAIL/WARN and its `fix:` hint; authenticate/run required services as indicated

Command / action	*Prerequisite and purpose*	*Expected evidence / next step*
`mise run setup:airflow-settings`	Connected Airflow exercise only	Generates Airflow settings; this task depends on `setup:databricks` and may prompt for auth again

The interactive Databricks helper also calls the Airflow settings helper. If you explicitly run the named settings recovery task, mise first runs its authentication dependency and may prompt again.

Use the PAT fallback, `mise run setup:databricks --pat`, only when your instructor supplies that route. Enter credentials interactively; never place them in learner evidence. Shell/session refresh matters: `.mise/env.sh` is cached by mise within a session, and installed shell hooks handle short-lived OAuth delivery. After setup changes, a fresh terminal is the first way to load the updated configuration. Do not dump environment variables or print tokens to diagnose it.

`dpc-check` checks tool availability, Python/dbt dependencies, inner Docker, certificates, prepared project/profile files, local parse, optional Airflow services and Databricks identity. It distinguishes hard failures from warnings. Exit zero with WARN results means the hard checks passed; it does not mean every connected gate passed. A successful `dbt parse` validates project structure without proving SQL ran on Databricks. If identity works but dbt cannot connect, use environment troubleshooting and retain the failing operation rather than repeatedly reauthenticating.

The broader `mise run verify:local-dev` runs `dpc-check`, offline pytest checks, optional live pytest checks when `CERT_INTEGRATION_TESTS=1`, and `scripts/verify-with-databricks.sh`. It includes a connected Databricks gate, so it needs prepared access even when the optional pytest switch is unset. Use it when the instructor requests the connected route, not as the first offline command.

Environment readiness check

This challenge verifies that the devcontainer already ships the required toolchain. You only need to confirm authentication and service reachability.

Goal

- Confirm the repo devcontainer has the required CLI tooling.
- Confirm you can authenticate to Databricks.
- Confirm you can run the Astro CLI.

Toolchain

- `dpc-check`: repo-provided Databricks preflight check
- `databricks` CLI: OAuth profile configured via `databricks auth describe --profile DEFAULT`
- `astro` CLI: local Airflow / Astronomer tooling

How to use

1. From inside the devcontainer, run:

   ```
   dpc-check
   ```

2. Inspect every FAIL and WARN. Hard-check success with authentication or service warnings does not establish connected readiness.
3. If auth is missing, run:

```
mise run setup:databricks
```

Notes

- This challenge assumes the repo devcontainer workflow. The toolchain should already be available inside the container.
- If something is missing, prefer the documented devcontainer setup and follow `GETTING_STARTED.md` and `.devcontainer/README.md`.

Assigned dataset and runtime readiness

From the extracted learner bundle root, prepare your first independent attempt after reading the shared setup:

```
export DC_RUNTIME="$PWD/practice/portfolio-runtime"
python "$DC_RUNTIME/lab.py" prepare .local/dbt-first --checkpoint foundations
cd .local/dbt-first
```

The incomplete starter is a starting point; author its owned outcomes before connected completion. Use the prepared runtime for readiness, seed/build and independent warehouse checks.

Wednesday and Thursday share Portfolio Analytics: holdings, prices, dated FX and fund references. Friday uses the distinct CG product-information project. Portfolio practice uses shared setup; the capstone uses its own assigned product workspace and readiness checks. Databricks and Astro execution need their own observed evidence.

Platform conventions and vocabulary

In the platform examples, Caspian is the lakehouse queried through Databricks, `caspian-core` supplies reusable macros, and DQF performs quality checks. LASR appears as an older destination for outputs. The course material does not establish LASR's exact system definition, so ask the responsible team before relying on that name in real work.

> Knowledge check: Names and boundaries
>
> What do source-aligned, domain-aligned and consumer-aligned describe? Which examples distinguish a business purpose from an implementation layer?

Convention	*How it is used in the examples*
`caspian-core`	Platform dbt package, pinned by version tag and commit SHA and obtained through the approved internal source. It supplies routing macros, tests, surrogate-key and audit-column macros, and checkpoint hooks.
`bitemporal_load_v2`	Incremental strategy for bi-temporal SCD loads, paired with `dl_*` audit columns and declared surrogate-key fields. Use a bespoke `MERGE` only when the maintained strategy cannot express the case. Each row records both the period it's valid for in the business and the time the system loaded it. dbt-core has no bitemporal mode of its own, so the platform supplies one.
`contract: {enforced: true}`	Schema enforcement on each curated model. Change the `SELECT` and its declared columns and types together.
`catalyst-domain-template`	Starting repository with contract, dbt, `promotions.yml`, CODEOWNERS, Harness pipeline, dev container and branch protection.

Convention	*How it is used in the examples*
`promotions.yml`	Per-model promotion intent for `dev`, `sqa` and `prod`, tied to a commit. A known break needs `acknowledge_breaking: true` and an approved impact assessment.
Marketplace pin	Search before building a duplicate. The classroom exercise uses `search_rank` thresholds: STOP > 0.5, INVESTIGATE 0.1–0.5, PROCEED ≤ 0.1. Check the applicable search rules for real work.
Curated prefixes	`fct_` for facts, `dim_` for dimensions, `stg_<source>__<entity>` for staging and `int_` for intermediate models. Grain and entity type choose the prefix.
Consumer prefixes	`rpt_` report shape, `agg_` grouped measures, `ts_` time series and `api_` single-row API lookup. The consumer query chooses the prefix.
`process_domain` and `dc_tag`	System code and classification tag are separate. Known pairs are `blmbrg`→`blmbrg`, `rdm`→`ref`, `moodys`→`rat`, `refinitiv`→`ref`, `factset`→`fds`, `msci`→`msci`, `rapid`→`raem`, `comet`→`mor`.
Source-aligned folder	One feed directory with `ingestion/`, `curation-emr/` and `curation-databricks/`, routed by deployment target.
Six-artefact ingestion bundle	`<process>.yml`, `dq.yml`, `pipeline.py`, `sttm.csv`, `source.yml` and `ddl.sql` version together.
Full `bitemporal_load_v2` config	Incremental Delta config declares `dl_row_uid`, load type, surrogate keys, audit columns, partitioning and platform post-hooks. Copy the maintained target-repository form.

The examples below explain how the named services fit together. Ask the platform team which options and approvals apply to your project before deploying anything.

Topic	*Notes*
Platform direction	The example includes ECS, EKS and Helix, described as managed EKS. The architecture and platform teams can explain which service is available and appropriate for a real project.
Lake Launch	The example uses this DevHub Harness template to prepare an AWS account in Helix and connect its development services. It asks for an ADM ID and the relevant Astronomer and Databricks workspaces. Lake Launch prepares the account; `catalyst-domain-template` starts the repository.
Also in play	Some teams use Kafka as a plain message queue driven from Python, plus Lambda functions and SQS.

The following sequence shows how the tools work together in the platform example. Your project's instructions determine the exact versions, repository and approvals.

Station	*Tool or artefact*	*Output*
1 Contract	ODCS YAML	Schema, quality rules and governance tags.
2 Schema	Flyway through `datacounsel`	Versioned DDL in the build catalogue.
3 Build	`dbt_wrapper.py` in a four-task Databricks Asset Bundle job	Models and tests run through the platform entry point.
4 Orchestrate	dag-factory or astro-blueprints YAML	Schedule, task order and routing.
5 Quality	Caspian DQF after dbt	Deequ-backed and API-supplied rules evaluated.
6 Ship	Harness `quality_gate`	Commit SHA, approval and change-request record.
7 Publish and register	`DEEP CLONE`, data-ready event, Collibra and Unity Catalog marketplace	Validated domain data and a discoverable listing. Registration is guided by the Catalyst skill; `dplc-triage` searches Unity Catalog and the marketplace for near-duplicates before a build starts.

Wednesday uses staging, integration and prepared, also called curated. The platform example uses raw, staging, intermediate and curated. Read the project's own folder names when following its pipeline.

Platform example	*Wednesday's term*	*Mapping*
Raw	Raw landing discussed with staging	This case names untouched landing data as its own layer. The teaching route folds landing and load metadata into staging.
Staging	Staging	Source-shaped renames, casts and load metadata.
Intermediate	Integration	Joins, enrichment and reusable calculations. Keep the repository's own term.
Curated	Prepared / curated	Consumer-ready output where grain and shape may change.

Product alignment is a separate question. Calling a product source-aligned, domain-aligned or consumer-aligned says whose needs it serves. Layer terms say how far a model has been processed, and one aligned product may contain several layers.

So which of Wednesday's two vocabularies do you use? When you document or talk about a product, to consumers, to non-technical colleagues or in the marketplace listing, call it source-, domain- or consumer-aligned. When you build, follow the supplied project's folders and prefixes, including `stg_` and `int_` where provided. Do not assume every Catalyst version or team uses identical naming. If a repository uses bronze/silver/gold, explain those processing labels separately from product alignment. A layer label such as "gold" does not explain whose decision the table supports.

For editor, terminal and container navigation, use Appendix H — Using VS Code and the training dev container. The lifecycle and roles reference explains the approval boundaries.

Organisational context

> Knowledge check: Place the product in context
>
> Who owns the source, the product promise and the platform boundary? What would you need to know about each role before accepting a consumer request?

The investment context explains how these terms fit together. The glossary below gives their general meaning and shows how they are used in the course. For a real CG dataset, ask its owner about the exact entity definitions and relationships.

Glossary

Further background is available in The Capital System, Capital Group's share-class explanation and Investor.gov's NAV definition.

Term	*Meaning and course use*
Fund	A pooled investment vehicle that invests for its shareholders.
Fund share	An investor's unit of ownership in a fund. Owning the share is different from directly owning the fund's underlying securities.
Share class	A class of shares in a fund with its own charges and arrangements.
Account	A record of investments associated with an investor.
Mandate	The investment objective, authority and restrictions governing a management arrangement.
Portfolio	A collection of investments. It can describe a whole investment portfolio or a manager's portion.
Sleeve / manager portion	A separately managed part of an investment portfolio.
Model portfolio	A proposed mix of investments, rather than a record of what is actually owned.
Target allocation	The intended proportion invested in each part of a portfolio.
Actual holding	An investment quantity held by a stated owner on a stated date. Portfolio Analytics supplies dated holdings.
Instrument	An investment object such as a share, bond or ETF, identified by its own key.
Client / investor	The person or organisation whose investment relationship is represented.
Advisor	A person or team serving the client relationship.
Portfolio manager	A person who makes investment decisions within assigned authority.
Analyst	A person who researches investments and contributes information to investment decisions.
Book	An advisor's defined set of clients or accounts.
Position	The quantity held at a time.
Execution / trade	A completed purchase or sale.
Subscription	An investor's purchase of participation in a fund.
Redemption	An investor's sale of participation back to a fund.
Cash flow	A movement of cash. External contributions differ from trade settlement.
Execution price	The price paid or received in a trade.
Valuation price	The price chosen to value a holding at a time.
Market value	A position's value using an appropriate price and units. Portfolio Analytics preserves supplied values.
AUM	Assets under management. Explain which assets are included, their valuation date and currency.

Term	*Meaning and course use*
NAV	Assets less liabilities. A weighted average of underlying security prices does not calculate NAV.
NAV per share	Net assets divided by outstanding shares. Portfolio Analytics's holdings-only value per synthetic unit is not official NAV per share.
Cost basis	The acquisition cost assigned under the applicable lot and accounting rules.
Realised / unrealised gain	Realised gain concerns sold investments; unrealised gain concerns investments still held. Both need an appropriate cost basis.
Performance / return	An investment result calculated with a stated method for time, valuations, income and flows.
Transaction fee	A charge connected with a trade.
Management fee	A charge for investment-management services under a separate agreement.
Currency	The monetary unit, such as GBP or USD.
Quotation unit	The basis of a price, such as per share or per 100 of face value.
Valuation cutoff	The time for which a valuation is prepared.
Basis point	One hundredth of one per cent: 1 bp equals 0.01%. Multiply a fee fraction by 10,000 to express it in basis points.
Grain	What one row represents. A trade, an account/instrument group and an account summary have different grains.

Facts, dimensions and a three-table join

Table type	*What one row records*	*How it changes*	*Question it answers*
Fact	An event or measurement, with keys and additive numbers	Appends or accumulates over time	What happened, when, and how much?
Dimension	Descriptive context for an entity named by a fact key	Changes slowly and may need history	Who or what did the event concern?

This example uses an imaginary account-holdings table to explain a three-table join. It is not a Morningstar or CG schema. Each holdings row carries an account ID and instrument ID; the other two tables add the client, advisor and instrument descriptions:

```
select
  h.as_of_date, h.portfolio_id,
  p.client_id, p.advisor_id,
  h.instrument_id, i.ticker, i.asset_class,
  h.weight, h.market_value
from example_account_holdings h
join portfolios p using (portfolio_id)
join instruments i using (instrument_id);
```

The holdings table supplies `weight` and `market_value`, while the dimensions explain who and what the row concerns. Before summing market value, check that the rows share a currency or join an FX rate. Before summing weight, check whether the intended total is per portfolio, date, asset class, or the whole result set.

Six checks in the CG product data

Use the frozen snapshot and declared source rules. Each check supports a specific enquiry; it does not establish suitability or equal investment risk.

Check	*Measure and comparison*	*Interpretation*
Offering coverage	25 ETFs, 65 F-3 offerings, 19 SMA strategies	Compare exact key membership, rather than only 109 total rows
Source representation	Catalogue versus detail observations	Declare the selection before joining
Expense components	Gross versus net expenses	Select the requested component; do not add them
Expense units	Percentage points × 100 = bps	0.50% is 50 bps; missing expense stays unknown
Date evidence	Reporting date versus retrieval/load time	Unknown fee reporting dates remain null
Return comparison	Basis, kind, period and reporting date	Keep incompatible or absent figures visible with reasons

Window functions

> Extra
>
> A `group by` collapses many rows to one row per group. A window function keeps each row and computes over related rows selected by `over ( ... )`. Ranking uses an ordered window; rolling measures add a frame.
>
> | *Function* | *Tie behaviour for values 100, 90, 90, 80* |
> |---|---|
> | `row_number()` | `1, 2, 3, 4`: every row differs; the order inside a tie needs a second sort key. |
> | `rank()` | `1, 2, 2, 4`: ties share rank and leave a gap. |
> | `dense_rank()` | `1, 2, 2, 3`: ties share rank and the next distinct value follows without a gap. |

For an illustrative expense ranking, equal published expense ratios can share a dense rank. This orders a stated measure; it does not establish suitability or all-in cost. Exclude unknown values from the numeric ranking while retaining them in the enquiry's follow-up output.

```
select
  product_id,
  net_expense_ratio_bps,
  dense_rank() over (order by net_expense_ratio_bps) as expense_rank
from product_cost_example
where net_expense_ratio_bps is not null;
```

`product_cost_example` is an illustrative relation; use your capstone's actual output and field names.
For a rolling measure, `partition by asset_class` gives each class its own history, `order by trade_date` defines sequence, and `rows between 6 preceding and current row` includes the current row plus six earlier rows. It's a seven-row window. That's seven calendar days only when each class has a row for every date.

```
with daily as (
  select
    t.trade_date,
    i.asset_class,
    sum(t.gross_value) as traded_value,
    sum(t.fee) as fees,
    count(*) as trades
  from dom_trading__trade t
  join dom_instrument__instrument i using (instrument_id)
  group by t.trade_date, i.asset_class
)
select
  trade_date,
  asset_class,
  traded_value,
  fees,
  trades,
  sum(traded_value) over (
    partition by asset_class
    order by trade_date
    rows between 6 preceding and current row
  ) as rolling_7d_value
from daily;
```

The CTE builds one row per asset class per trading date first. The outer query then adds up those daily totals, so each row in the window is one day for one asset class.

Acronyms

If someone uses an acronym you cannot find here, ask them what it means. Some internal names appear in the course examples without a confirmed expansion or complete description. The entries below say what you need for the example; the relevant team can explain the name and responsibilities in real work.

Capital Group teams, systems and process

Acronym	*Meaning*
ADM ID	Application identifier. A Harness pipeline and the onboarding template both ask for one.
ADR	Architecture decision record. One decision written down with its context and consequences.
BTS	This name appears in an Investment Operations example about improving manual processes. Ask the team how the name is expanded and what work it covers.
CDM	The example uses this name for client and party data. Ask the source owner which people and organisations its records represent.
CG	Capital Group.

Acronym	*Meaning*
CSG	This name appears in examples about multi-asset investment construction and risk. Its full name and responsibilities need to be explained by the team.
DPC	Data Product Creator, the person or team that builds the product.
DPLC	Data Product Lifecycle. The course organises nine phases, numbered 0–8, into Define, Deliver and Sustain.
DPM	Data Product Manager. Owns the product and its version.
DQF	Data-quality framework. In the Caspian example, a required check must pass before downstream work continues.
EA	Enterprise architecture. An architecture tool can connect applications with the business capabilities and data classifications they support.
EDO	This name appears in the enterprise-data examples. The relevant team can explain its full name and responsibilities; the acronym alone does not identify who owns a tool.
GBS	This name appears in a business-reporting support example. Ask the team about its full name and responsibilities.
GCC	Global Client Connect, the product discussed in Thursday's institutional-client example. The example does not give a complete description of the real product.
IO	Investment Operations.
LASR	The older output destination named in the platform examples. Ask its owner what system it is and which outputs it receives.
MDS tags	Five tags help route support in the DAG example: `l1_domain`, `l1_xmatters_group`, `lifecycle_phase`, `data_classification` and `criticality`. Your service's support instructions explain which tags it needs.
P1, P2, P3	Alert priorities. The example routes them to xMatters, Teams and Jira respectively; use your service's support instructions for actual alerts.
PIA	Privacy impact assessment. Follow the exercise's review steps, and ask the privacy team which assessment is required for real data.
RFD	Request for Discussion, used in the architecture-review example. The responsible team supplies its repository and review procedure.
SQA	Some teams' name for the QA environment, the one between dev and prod.
WIP	Work in progress. A WIP model is switched off by default with `enabled = var('include_wip', false)`.

Designing data products

Acronym	*Meaning*
BI	Business intelligence. Reporting tools such as Power BI.
CRM	Customer relationship management. The CG capstone uses synthetic enquiries; it does not connect to a live CRM.
DQ	Data quality.
ERD	Entity-relationship diagram. A drawing of a system's tables and how their keys link.
KPI	Key performance indicator.
LTV	Lifetime value. What a product is worth over its whole life. It belongs in the business case.
MoSCoW	Must, should, could, won't. A way of sorting requirements, or SLIs, by how much they matter.
MVP	Minimum viable product. The CG phase-1 core supplies a traceable 25-ETF output.
NFR	Non-functional requirement. An operating constraint such as response time or availability.

Acronym	*Meaning*
ODCS	Open Data Contract Standard. The course examples use this YAML format to describe data and its obligations.
PI	Program increment. A fixed planning period. Assess RAID before each one starts.
PII	Personally identifiable information.
PRD	Product requirements document. A Catalyst skill of that name drafts one.
RACI	Responsible, accountable, consulted, informed. A way of writing down who does what.
RAID	Risks, assumptions, issues and dependencies.
RICE	Reach, impact, confidence and effort. Score a promise as reach × impact × confidence ÷ effort.
ROI	Return on investment.
SLA	Service-level agreement. The promise you make to consumers about an indicator, and what happens if you miss it. For delivery delay it might be “no more than two days late”.
SLI	Service-level indicator. The thing you measure, for example the delay in delivering a dataset.
SLO	Service-level objective. The agreed target or range for an SLI, measured over a stated window; for example, at least 95% of deliveries arrive by the deadline.
SME	Subject-matter expert.
STTM	Source-to-target mapping.
UAT	User acceptance testing. The consumer and accountable business roles agree whether the result meets its intended use.

Data, SQL and dbt

Acronym	*Meaning*
ACID	Atomic, consistent, isolated, durable. The guarantees of a database transaction. A lakehouse is a lake plus ACID.
CSV	Comma-separated values. A plain-text table. The challenge seeds are CSV files.
CTE	Common table expression. A named step inside one SQL query (`with … as`).
DAG	Directed acyclic graph. Usually an Airflow DAG, though dbt’s model graph is one too.
DDL	Data definition language. The part of SQL that creates things, such as `CREATE TABLE` and views.
ELT, ETL	Extract, load, transform (load raw data into the warehouse and transform it there, which is dbt’s world) and extract, transform, load (transform before loading).
JSON	JavaScript Object Notation. A text format for structured data. dbt writes its results to `run_results.json`.
MFT	Managed file transfer. How many source files arrive before staging.
OBT	One big table. A single wide table for a narrow consumer whose queries are known.
OLAP	Online analytical processing. Read-heavy querying over large tables, which is what dbt and Databricks are for.
OLTP	Online transaction processing. Lots of small writes, as in an order system. dbt is the wrong tool for it.
SCD, SCD2	Slowly changing dimension. Type 2 keeps every old version of a row with valid-from and valid-to dates.
SQL	Structured Query Language.
UC	Unity Catalog. Databricks’ catalog of tables, grants and lineage.

Acronym	*Meaning*
YAML	A plain-text configuration format. dbt properties, ODCS contracts, bundles and Harness pipelines are all YAML.

Platform, cloud and release

Acronym	*Meaning*
API	Application programming interface. How one program asks another for data or actions.
AWS	Amazon Web Services. The supplied platform case uses AWS; this does not inventory every CG service.
CI/CD	Continuous integration checks integrated changes. Continuous delivery prepares releasable artifacts; continuous deployment automatically releases qualifying changes. A merge does not itself prove production deployment.
CLI	Command-line interface, such as the `dbt`, `databricks` and `astro` commands.
CPU, RAM	Central processing unit and random-access memory, the processor and the memory. Raise Docker Desktop's memory allowance to 8 GB before you open the dev container.
DAB	Databricks Asset Bundles. YAML that declares Databricks jobs and resources, deployed with `databricks bundle deploy`.
DR	Disaster recovery.
EC2	Amazon Elastic Compute Cloud. In practice, a virtual machine.
ECR	Amazon Elastic Container Registry, where built images are pushed.
ECS	Amazon Elastic Container Service. It is one of the container services discussed in the platform example.
EKS	Amazon Elastic Kubernetes Service, Amazon's managed Kubernetes.
EMR	Amazon EMR, short for Elastic MapReduce, is a managed service for large Spark jobs. The Retina example shows job runs and costs.
GitOps	Version-controlled desired state reconciled to an environment. The target branch/configuration and promotion controls determine deployment; merging to main need not release to production.
IaC	Infrastructure as code. The pipeline that creates the AWS resources, separate from the one that ships the application.
K8s	Kubernetes (eight letters between the K and the s). Runs containers from a configuration file.
M2M	Machine-to-machine. OAuth credentials associated with a service identity. The supplied shared-job case uses them; confirm target identity policy.
MWAA	Amazon Managed Workflows for Apache Airflow, Amazon's managed Airflow.
OAuth	The sign-in standard the dev container uses for Databricks. `mise run setup:databricks` uses it by default.
OS	Operating system.
PAT	Personal access token. Use the documented training fallback only when assigned; follow the authentication instructions for the tool you are using.
PR	Pull request. A request to merge one branch into another once someone has reviewed it.
REST	Representational state transfer, a common style of web API. GCC's dependency checks call the Airflow REST API.
S3	Amazon Simple Storage Service. Object storage. Caspian's Delta files live in S3.
SAML	Security Assertion Markup Language. A sign-in standard; the supplied AWS access case uses account/role selection.

Acronym	*Meaning*
SHA	Secure Hash Algorithm. A commit SHA is the hash that identifies a Git commit. The supplied Harness case identifies the build being promoted by its commit SHA.
SQS	Amazon Simple Queue Service. The queue in the supplied S3/Lambda/Airflow event-trigger example.
SRE	Site reliability engineering. Error budgets and burn rates come from SRE practice.
SSO	Single sign-on. One corporate login for many tools.
UI	User interface.
VM	Virtual machine. It virtualises the hardware and runs a full operating system.

Networks and troubleshooting

Acronym	*Meaning*
CA	Certificate authority. Behind the corporate proxy your tools need the approved root CA.
CRLF, LF	Carriage return plus line feed, the Windows line ending, and line feed, the Linux and Mac one. A script or `.env` file saved with CRLF fails inside the container.
DNS	Domain Name System. Turns host names into addresses.
HTTP, HTTPS	Hypertext Transfer Protocol, the protocol of the web. HTTPS is HTTP with TLS encryption.
SSH	Secure Shell. One way Git signs in to GitHub.
TLS, SSL	Transport Layer Security and its older name, Secure Sockets Layer. Corporate SSL inspection causes the `CERTIFICATE_VERIFY_FAILED` error.
VPN	Virtual private network, the corporate network route.
WSL2	Windows Subsystem for Linux, version 2.

Investment data

Acronym	*Meaning*
AUM	Assets under management. Explain which assets are included, their valuation date and currency.
bp, bps	Basis point, one hundredth of one per cent.
ESG	Environmental, social and governance data.
ET, UTC	US Eastern Time and Coordinated Universal Time. Keep logs in UTC, and state a local-time promise in the business timezone.
ETF	Exchange-traded fund.
FX	Foreign exchange. Adding up across currencies needs an FX rate and a stated base currency.
GBP, USD, EUR, CHF, HKD, JPY	Currency codes for pounds, US dollars, euros, Swiss francs, Hong Kong dollars and yen.
ISIN	International Securities Identification Number, the code for one instrument.
MV	Market value.
NAV	Net asset value: assets less liabilities. NAV per share additionally divides by outstanding shares; weighted security prices are neither.
RM	Relationship manager.
TWR, MWR	Time-weighted and money-weighted return. “Calculate return” is ambiguous until you say which one.

Acronym	*Meaning*
UHNW	Ultra-high-net-worth clients.

AI

Acronym	*Meaning*
LLM	Large language model, the kind of model behind Copilot. It predicts the next word.
RAG	Retrieval-augmented generation. Parts of your files are put into the prompt before the model answers.
RLHF	Reinforcement learning from human feedback. The training step that teaches a model which answers people prefer.

Regulations named on Tuesday

These regulations appear in the privacy-review example. The privacy and legal teams determine which apply to a real product and what review it needs.

Acronym	*Meaning*
CCPA	California Consumer Privacy Act.
DORA	Digital Operational Resilience Act, the EU rules on financial firms' IT resilience.
GDPR	General Data Protection Regulation, the EU and UK data-protection law.
GLBA	Gramm-Leach-Bliley Act, the US financial-privacy law.
Reg S-P	The SEC's Regulation S-P, on the privacy of consumers' financial information.
SOX	Sarbanes-Oxley Act, the US law on controls over financial reporting.

Appendix B — Data contracts and decision records, worked

Recording decisions: the RFD

A migration plan records what will change; a decision record explains why. For example, a team choosing a lakehouse should record why Delta Lake and a star schema fit its constraints, and which alternatives it rejected. A future reviewer can then assess the decision without reconstructing its rationale.

ADR and RFD vocabulary

An Architecture Decision Record (ADR) captures a design choice and its rationale. The supplied course materials use a Request for Discussion (RFD), with repository and review conventions illustrated below. Confirm the current internal process with the product's architecture owner before applying it to live work.

The reasoning begins with three sections:

Section	*What goes in it*
Context	Measured technical, business, compliance, organisational and cost constraints, stated without arguing for one option
Decision	One choice in active voice, specific enough for an engineer to implement
Consequences	Positive, negative and neutral effects, including mitigations for accepted risks

One RFD records one decision. If two independent choices have appeared in the Decision section, split them so that either can later be replaced without rewriting the history of the other.

The Capital Group wrapper

The supplied RFD example numbers decisions with four digits and stores each one at `rfd/NNNN/README.md`. Work happens on a matching `rfd/NNNN` branch. Review comments and alternative proposals stay on the pull request, so the whole discussion stays attached to the merged RFD.

The lifecycle is a five-stage flow with two possible outcomes at the final stage:

Stage	*State*
Form the question	`ideation`
Open a draft PR	`prediscussion`
Invite review, revise on the PR and get approval before Phase 4	`discussion`
Merge the agreed record	`published`
Record the outcome	`committed` when acted on, or `abandoned` when the published decision is dropped

Numbers are not reused. When a committed decision is superseded, the old RFD stays in the repository and links to the replacement, because deleting it would remove the constraints the next team needs to understand.

When the RFD gate applies

In the supplied governance case, the paved path is an approved-pattern registry maintained by the Data Architect community. These gate criteria are training-case rules; confirm the current internal registry, classification thresholds and authorised reviewers before applying them to a live product. A design that cites an approved pattern can proceed without formal architecture review. An off-pattern design triggers

a Phase 3 Data Architect review and uses the RFD to describe the deviation, alternatives and trade-offs. A separate Phase 3 InfoSec gate applies to products classified Restricted or above, or to products covered by named regulatory requirements. Some designs trigger both gates, with each reviewer signing the part they own.

Measure before you optimise

> Expert
>
> Start an optimisation record from a measurement. “The query feels slow” isn’t one. Take a 14.2-million-row holdings table with 3,847 files and a 4.1 MB median file size, where the p95 account query took 12.3 seconds against a 5-second target. The query plan showed a full scan. Those numbers are what justify comparing compaction, Z-ordering, liquid clustering, pruning and data skipping, and Appendix C has the full table. Put the baseline in the RFD Context. In Consequences, say how you’ll re-measure after the change and what the chosen technique costs you.

Classification and the PIA gate

PIA trigger	*Response*
Any field has `PII = Yes`	PIA
Any field has `PII = Unknown`	PIA, resolving Yes or No
Sensitivity is `Confidential`	PIA
Sensitivity is `Restricted`	PIA plus InfoSec review
A named regulation applies	Phase-2 PIA plus the Phase-3 Security Design and InfoSec gate

PII is `Yes` or `No`. Sensitivity is `Public`, `Internal`, `Confidential` or `Restricted`. Criticality runs from `Tier-1` to `Tier-4`. Judge every §2 column on what it actually holds. The named regulations are SOX, DORA, GLBA, Reg S-P, GDPR and CCPA. The PIA settles minimisation, masking, access and retention before design starts.

Tier-1 and Tier-2 controls

Decision	*Tier-1*	*Tier-2*
Miss handling	A freshness miss may block regulator-facing output; accuracy uses tight tolerance and rapid escalation.	Flag and deliver usable late data; quarantine suspect rows and publish unaffected rows.
Access and fixed controls	Use narrow roles and a formal security review.	Wider named internal groups may fit. Required keys, uniqueness and semantic-change notice apply at both tiers.

One filled contract

This illustrative contract publishes one vendor exposure row per portfolio, business date and exposure type. Its Morningstar label, domain, roles, dates, service levels and classification below are supplied case assumptions, not evidence of a provider schema or a current CG product registration. The case’s portfolio key identifies the contracted reporting aggregate; it does not establish an investor account or manager sleeve. It excludes direct personal data, so no column holds PII.

§1 Identity. Morningstar Portfolio Exposures publishes to `investment-exposures`. The Investment Exposures DPM owns version `1.0.0`; its Data Steward oversees quality. The effective date is 26 August 2026, review is quarterly or after material change, and classification is Internal, Tier-2 with the Tier-1 columns below.

Name	*Type*	*Description*	*Sensitivity*	*Criticality*	*Example*
`portfolio_id`	`STRING`	Internal portfolio identifier	Internal	Tier-2	`PF-1042`
`as_of_date`	`DATE`	Exposure business date	Internal	Tier-1	`2026-08-25`
`exposure_type`	`STRING`	Geography, style or asset-class family	Internal	Tier-2	`GEOGRAPHY_US`
`exposure_value`	`DECIMAL(12,8)`	Portfolio share from 0 to 1	Restricted	Tier-1	`0.28450000`
`source_file_date`	`DATE`	Date on the provider file	Internal	Tier-2	`2026-08-25`
`quality_status`	`STRING`	`CURRENT`, `STALE`, `MISSING` or `QUARANTINED`	Internal	Tier-2	`CURRENT`
`updated_at`	`TIMESTAMP`	UTC publication time	Internal	Tier-3	`2026-08-26 11:24Z`

Every published row requires non-null `portfolio_id`, `as_of_date`, `exposure_type`, `quality_status` and `updated_at`. The composite key is `portfolio_id` + `as_of_date` + `exposure_type`. `exposure_value` and `source_file_date` are conditionally nullable: they describe an accepted observation, and a missing delivery supplies neither. The business date remains the date being requested, even when no observation exists.

§3 Service-level agreements.

§3 target	*Promise and measurement*
Freshness	Accepted prior-business-day observations by 07:30 ET on at least 95% of business days; assess the accepted batch publication time, excluding status-only rows, quarterly over 63 business days.
Completeness	At least 99% of expected portfolio/type pairs; compare with the active-pair registry per batch.
Accuracy	At least 99.5% of values within the agreed provider tolerance; reconcile over 21 business days.

Severity	*Response*	*Resolution*
Critical	30 minutes	Four business hours
High	Two hours	One business day
Medium	One business day	Three business days
Low	Five business days	Ten business days

Escalation goes from the Data Steward or on-call engineer to the engineering lead and the product owner. The §4 rows below use the same severity words, so the rule that fails tells you which response applies.

§4 Data quality.

Edge condition	*Rule*	*Severity / action*
Late arrival	Mark usable output `STALE`, notify and publish.	Medium / Flag and deliver
Missing source	Publish `MISSING`; do not impute.	High / Flag and deliver
Null / status inconsistency	Required identity, status and publication time must exist; payload must obey the status rules below.	Critical / Block delivery
Schema mismatch	Required columns and types must match §2.	Critical / Block delivery
Duplicate	Composite key must be unique.	Critical / Block delivery
Out of range	An observed `exposure_value` must be from 0 to 1.	Medium / Quarantine the observation; publish only its status

Make failure states executable

The same relation carries accepted observations and explicit delivery-status rows. Consumers select `CURRENT` or `STALE` when they need usable values; a row's presence alone does not establish completeness. Completeness counts expected pairs with usable observations, so emitting `MISSING` rows cannot improve the 99% target artificially. A recent `updated_at` on a status row likewise does not satisfy the freshness promise: measure the deadline against accepted prior-business-day observations and retain the quarterly 63-day assessment.

Status	*Accepted payload*	*Publication meaning*
`CURRENT`	Non-null value in range and non-null provider-file date	Accepted observation for the requested business date, available within the agreed deadline.
`STALE`	Non-null value in range and non-null provider-file date	Usable observation delivered late for the requested business date; record the miss and notify.
`MISSING`	Both payload fields null	No accepted source observation for this expected key; preserve the requested date and do not impute.
`QUARANTINED`	Both payload fields null	A source observation was rejected; retain the original value, file identity and failed rule in restricted diagnostic evidence.

A `STALE` row never silently copies a previous day's value into today's key. Such a carry-forward would require a separately agreed meaning and origin date. A rejected observation's source-file date belongs in its diagnostic record, rather than presenting it as an accepted payload. Corrected arrivals can replace status rows only after the new candidate passes the publication gate.

The following illustrative SQL returns offending rows; it is a review example, not an additional classroom execution task. Its null handling is deliberate because a SQL comparison with null is unknown.

```sql
select *
from exposure_candidate
where portfolio_id is null or as_of_date is null
   or exposure_type is null or updated_at is null
   or quality_status is null
   or quality_status not in ('CURRENT', 'STALE', 'MISSING', 'QUARANTINED')
   or (quality_status in ('CURRENT', 'STALE') and
       (exposure_value is null or source_file_date is null
        or exposure_value < 0 or exposure_value > 1))
   or (quality_status in ('MISSING', 'QUARANTINED') and
       (exposure_value is not null or source_file_date is not null));
```

Separately compare accepted publication times with the requested date's agreed deadline in `America/New_York`: `CURRENT` must be on time and `STALE` late. Verify `source_file_date` against the accepted provider file and its documented business-date mapping; do not assume file date equals `as_of_date`. These checks need delivery provenance and a business calendar beyond the row-shape SQL. Also group by the three key columns and reject any count greater than one. Schema, key, duplicate or status-rule failures block the entire candidate at Critical severity; keep the previous accepted release and escalate. A missing expected source produces a High-severity `MISSING` row. An out-of-range observation produces a Medium-severity quarantine record and `QUARANTINED` status while unaffected accepted observations may publish. Late accepted observations produce Medium-severity `STALE` output. These actions use the existing response and resolution times; status publication records a miss rather than erasing it.

§§5–8 Completion.

Section	*Filled decision*
§5 Access and security	Named Unity Catalog consumer groups receive `SELECT`; producers receive write access. Direct PII is excluded. Apply fund or portfolio filters where entitlements require them, use platform encryption, and set retention from approved record-keeping and privacy decisions.
§6 Change	Removal, narrower precision, required-field addition and changed meaning are breaking. Nullable additions and clearer descriptions are compatible. Use major/minor/patch semantic versions and support the current plus one prior version during migration.
§7 Consumers	The registry below names each consumer, its access route, a contact role and the onboarding date.
§8 Approvals	The DPM signs scope and version. The Data Steward signs schema, classification and quality. Security or privacy reviewers sign when their review is triggered. Each consumer signs the SLA and access route. Real names and dates replace the role labels before launch.

§7 Consumer Registry (Client Portfolio Analytics reference scenario).

All three groups have SELECT access and an example onboarding date of 4 April 2026. Their contact is the named consumer representative for each team.

Consumer	*Use case*
Wealth Management RMs (45 named accounts)	Quarterly portfolio reviews from one view instead of five source systems.
Portfolio Managers (12)	Review `return_ytd`, `return_1yr`, `risk_score` and holdings; *Return* means TWR.
Compliance (3)	Read the authorised detail defined in §5.

Change management worked through

Change	*Example*	*Migration support*	*Consumer effect*
Removal	Morningstar removes `exposure_type`.	Deprecation warning, then removal	Consumers cannot group exposure families.
Precision	`exposure_value` narrows from `(12,8)` to `(8,4)`.	Parallel column, then cutover	Rounding changes portfolio totals.
Meaning	`GEOGRAPHY_US` moves from issuer domicile to revenue source while name and type stay fixed.	Documentation plus consumer review	CI passes, but comparisons answer a different question.

Use semantic versions: major for breaking changes, minor for additions and patch for description fixes or compatible improvements. Support the current and one prior version.

Each breaking change gets 30 days' notice. Tell consumers within five business days of classifying the change, and support both versions for at least ten business days, or longer if anyone still hasn't migrated.

Step	*Action*	*Owner or gate*
Detect	CI fails; the producer raises semantic changes.	Producer
Assess	Classify and version.	Producer + DPM + Data Domain Lead for major bumps
Communicate	Notify every §7 consumer with migration steps.	DPM via the maintained contact
Iterate	Run both versions until all consumers confirm migration.	Producer + DPM

Failure actions and the Pólya question

Workflow

Action	*Use*
Block delivery	A schema, required-field or key failure makes the whole output unsafe.
Flag and deliver	Consumers can use data that is clearly marked late or partial.

Quarantine row	An accuracy or range failure may be an error or a real Pólya-type tail event, where a few rare, large observations hold much of the information. Oil futures going negative in 2020 was one. Quarantine lets a person correct the error or release the event without silently trusting or deleting it.

Each artefact in the contract package answers one question.

Artefact	*Question answered*
Contract	What are we promising?
SLA definitions	How good, how soon, and what follows a miss?
NFR specification	Which operating constraints apply?
Backlog	In what order will we build the promise?
Feasibility report	Can this team build and operate it?

The feasibility report rates each dimension. Green means implementable as written, Amber requires a named modification, and Red names a blocker.

Feasibility dimension	*Rating and condition*
Source availability	Amber: design the late and missing-file path.
Schema implementability	Green: all target fields and keys are derivable.
Quality achievability	Amber: profile borderline values and retain review.
SLA achievability	Amber: leave retry buffer or agree labelled stale data.
Operational sustainability	Green: owner, steward, alert route and review queue are named.

Each Amber becomes a modification the DPM and the creator agree.

DPM need	*DPC constraint*	*Agreed modification*
07:30 delivery	Provider delays leave no retry window.	Add buffer and agree when `STALE` may publish.
Tight exposure tolerance	Rounding differs by exposure family.	Keep source precision and set tolerance by family.
Five checks at launch	Two need profiling.	Launch three critical checks; schedule two after profiling.

The backlog then turns contract sections into work items.

Backlog item	*Contract source*	*Priority and check*
Build table and composite key	§2	Must; enforced schema builds
Implement six edge checks	§4	Must; one check per rule
Build freshness alert	§3	Should; forced delay alerts and marks `STALE`
Provision consumer groups	§5, §7	Must; approved scopes verified
Real-time streaming	Outside v1	Won't this time; reconsider at v2

The following example links the contract package to Global Balanced Growth Fund analytics for about 45 relationship managers.

Stage	*Output*
Product framing	RMs need fund performance against a benchmark to prepare a review in under 30 minutes; no complete reusable product exists.
Risk	An unannounced custodian format change scores probability 4 × impact 4 = 16, so the team mitigates it before build.
Design	`fund.nav_per_share` maps to `nav`; null rejects the row. `portfolio_return_twr` uses the agreed time-weighted definition.
Contract	§2 declares `nav DECIMAL(16,4) NOT NULL`; §4 reconciles it; §3 sets freshness and breach response.
Package	The backlog funds the table, tests and monitor; source availability stays Amber until retry is designed.
Enforcement	A dbt model contract checks columns and types; dbt tests check nulls, uniqueness and allowed values.

Reuse decision

Reference scenario: Client Portfolio Analytics. The team finds a Marketplace asset called Client Summary and checks it before depending on its name.

Criterion	*Finding*	*Rating*
Freshness	Eight months old; daily data needed.	Red
Coverage	Holdings, performance, benchmarks and risk are undocumented.	Red
Certification	No domain certification.	Red
Lineage	Sources and transformations unknown.	Red
Consumers	Ownership and usage unknown.	Red
Deprecation	Not marked, but stale and ownerless.	Amber

Source	*Decision*
Holdings	Build a versioned integration.
Performance	Formalise access to existing internal views.
Benchmarks	Reuse the certified daily vendor product.
Risk	Build a scheduled output with the Risk team.
CRM	Build a minimised extract and exclude unneeded PII.

The result is a partial build. Benchmark reuse adds three dependencies to the contract package: compatible schema, refresh before the product deadline, and an active vendor agreement.

Unity Catalog governance decision

Reference scenario: Investment Risk Analytics.

Decision	*Answer and mechanism*
Namespace	Three levels: catalog, then schema, then table. Consumers read `risk_analytics.daily`; `risk_analytics.staging` receives no consumer grants, preventing access to intermediate data.
Workspace binding	A catalog is bound to the workspaces that may use it, and not every catalog is bound to every workspace. Before a consumer in another workspace can query `risk_analytics`, the platform team has to bind it there. A Caspian catalog can usually be bound. Another domain's catalog isn't bound by default.
Grants	Risk officers read the daily schema; portfolio managers read exposure, fund, security and date tables; Compliance also receives authorised counterparty detail.
Masks	Non-compliance roles see counterparty name and address as `REDACTED`, and only the last four tax-ID characters.
Row filters	`fund_family` must appear in the user's authorisation map. Shared fund-of-funds positions are mapped explicitly to each authorised constituent family.
Governed tags and lineage	Sensitivity tags apply masks, audit and row-filter policy. Unity Catalog lineage identifies dependent tables, reports and models before a migration.

Appendix C — Capital Group conventions

Reference catalogue

This appendix records conventions in the supplied training materials. It is a course reference, not a current enterprise policy register. Before applying its lifecycle gates, branch controls, package routes, release approvals or risk thresholds to a live product, the product owner and responsible platform/ governance team must confirm the current internal version and scope. The classroom examples use these conventions to practise evidence and decision boundaries; no public CG source establishes the detailed rules below.

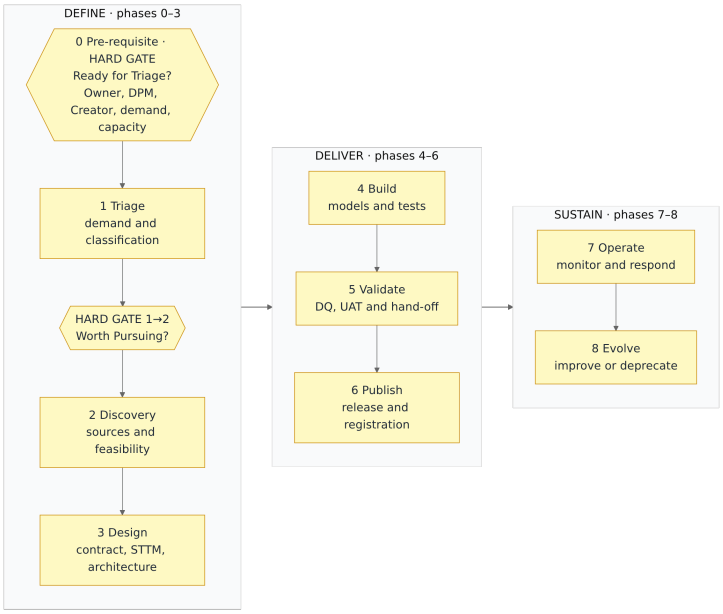


Figure 138: The DPLC lifecycle: nine phases grouped into Define, Deliver and Sustain, with hard gates at Phase 0 and the Phase 1-to-2 decision.

Platform vocabulary and build-stack conventions are introduced in Before you start. The lifecycle and control reference below expands that context.

Repository, release and risk controls

Control	*Convention*
Branching	Short-lived branches from `main`; no standing `develop`, `release` or `staging` branch. Target life is 1–5 days.
Names	`rfd/NNNN`, `feat/<description>`, `fix/<description>`, `hotfix/<description>`.
Merge	Squash only; force-push disabled.
Ownership	CODEOWNERS uses team identities with at least two members and protects relevant paths.
PR scope	Apply the no-and test: split unrelated concerns joined by “and”.
PR data	Use `pr_<number>_<username>` and drop the schema when the PR closes.
WIP model	Keep the normal name and set `enabled = var('include_wip', false)`.
Promotion	`promotions.yml` records the commit or version each environment receives.
Breaking promotion	Production needs `acknowledge_breaking: true` and an approved downstream-impact assessment.
Production and hotfix	Normal production requires human approval and a ServiceNow change request as its legal and audit record, even when a pipeline creates the request. An emergency release also needs an incident link.

Anti-pattern tripwires.

Tripwire	*Failure*	*Redirect*
Shadow `_v2` model	Filenames carry promotion intent and two files claim the product.	Keep one model, use the WIP gate and record promotion in `promotions.yml`.
Public-index package install	Internal packages fail and the approved audit path is bypassed.	Use the repository’s internal Nexus configuration.
Bronze/silver/gold as product types	Storage state and product alignment are mixed.	Use layer terms for processing state, and source, domain or consumer alignment to say whose needs the product serves.
Separate ADR series	A second decision process appears beside the supplied case’s RFD process.	Put Context, Decision and Consequences in `rfd/NNNN/README.md`; discuss them on the PR.
`backlog.md` as live Demand Tracker	The organisational record and sign-off trail are absent.	Use Demand Tracker; Markdown may serve as a portable reference or export.

Probability × impact risk card. Score each factor from 1 to 5 and multiply them. For impact, use the higher of delivery delay or live-consumer harm.

Score	*Probability*	*Impact anchor*
1	Very unlikely	Negligible delay, or no consumer notices.
2	Unlikely	Under one week, or a brief lapse consumers absorb.
3	Possible	One to two weeks, or recurring degradation covered by a documented fallback.
4	Likely	Over two weeks, or a breach consumers cannot work around.
5	Almost certain	The product cannot ship or serve its purpose.

`Risk score = probability × impact.` 16 or above needs mitigation straight away; 4 or below can be monitored. Scores between them still need an owner and a recorded treatment decision.

Delta Lake optimisation techniques

Measure query time, file count, scan volume or storage growth first, then choose the operation that changes that measurement.

Technique	*Mechanism*	*Use when*
Compaction (`OPTIMIZE`)	Merges small files.	Small writes or `MERGE` created excessive files.
Z-ordering	Co-locates values for named columns.	A stable, high-cardinality filter dominates reads.
Liquid clustering	Maintains clustering without fixed partition layout.	A new or rebuilt table serves changing query dimensions.
Data skipping	Uses file statistics to avoid unrelated files.	Filters align with columns covered by statistics.
Partition pruning	Avoids whole directory partitions.	Access follows a stable, coarse date or region key.
`VACUUM`	Deletes obsolete files outside retained state.	Storage growth is measured and the retention decision preserves recovery and audit history.

Repeated Z-order work consumes compute, so schedule it only when measurements support that choice. A short `VACUUM` retention can delete files needed for time travel and audit; the product owner and governance contacts must agree the retained history.

Appendix D — dbt and tooling reference

For assigned Portfolio Analytics workspaces, tasks and checks, use the dbt project. This appendix provides command and concept reference. Historical code examples require version-specific adaptation; use a separately prepared project for follow-up.

Slowly changing dimensions with `dbt snapshot`

Say the portfolio-manager coverage table has four columns, `fund_id`, `manager_id`, `manager_name` and `updated_at`, and it's updated in place. When a fund changes manager, the old name is overwritten. You know who runs the fund now, but you've lost who ran it last quarter, and attribution needs that.

A dbt snapshot compares the current source rows with the copy it stored last time and keeps the history:

```
{% snapshot pm_coverage_snapshot %}

{{
    config(
        target_schema='snapshots',
        unique_key='fund_id',
        strategy='timestamp',
        updated_at='updated_at'
    )
}}

select fund_id, manager_id, manager_name, updated_at
from {{ source('reference', 'pm_coverage') }}

{% endsnapshot %}
```

`unique_key` sets the tracked grain. The `timestamp` strategy versions a row when `updated_at` advances; `check` compares named values when the source has no reliable timestamp.

Run `dbt snapshot` on an Airflow schedule and each version gains `dbt_valid_from` and `dbt_valid_to`; the current row has a null end. A quarter-end query selects the row whose validity range contains 30 June.

SCD type	*Storage rule*	*Choose it for*
1	Overwrite	Current state only; no lookback.
2	Add a date-ranged row	Full point-in-time history.
3	Keep a previous-value column	Current and one prior value.

Delta `MERGE` closes the old type-2 row and inserts the new one. A dbt snapshot or a compatible configured loading strategy writes the required SQL; inspect the actual implementation and warehouse behavior.

For facts, dimensions and a worked Morningstar join, see Organisational context.

A bitemporal load tracks two perspectives: business time, when the value was true, and system time, when it was recorded. A correction adds a version while preserving the earlier observation. dbt has no bitemporal mode as such. A configured snapshot tracks changes through validity intervals; an incremental update alone does not preserve every overwritten version. Define both time perspectives explicitly when the consumer requires them. `bitemporal_load_v2` is the supplied Capital Group packaged example; inspect its actual audit fields below and the dbt snapshot behavior.

The maintained local `v2` audit map uses `dl_business_start_date`, `dl_business_end_date`, `dl_system_start_timestamp`, `dl_system_end_timestamp` and `dl_current_indicator`. Older examples use `*_from_date`/`*_to_date` names. Match the model to the assigned macro rather than assuming those naming families are interchangeable.

An as-of join reproducing historical knowledge must filter both the business interval and the system interval. Define the meaning of a current flag before testing its cardinality:

- For a reference entity at a specified business date and knowledge timestamp, return at most one applicable version; require exactly one only where the contract promises coverage.
- At a fixed knowledge timestamp, business intervals for the same reference key must not overlap. Business intervals from different system versions may overlap legitimately.
- For a dated balance, require at most one system-valid version per complete observation key, including its business as-of date.
- A closed system-history row stays closed. A corrected business timeline may contain several business intervals that all belong to the latest system understanding.

The historical data chapter works through corrections, queries using both clocks, delivery policies, deletion and replay. Its optional clinic tests a finite sequence of arrivals. Keep the snapshot example above as a single-validity-range exercise; it does not automatically provide independent business and system history.

dbt and Airflow reference

dbt commands

Use the owning phase guide's working directory and profile options with these commands.

Command	*Purpose and boundary*
`dbt deps`	Install packages declared in `packages.yml`.
`dbt debug`	Resolves the profile and connection; ends `All checks passed!`.
`dbt parse`	The cheapest check that the YAML and Jinja are well-formed; writes the manifest without touching the warehouse.
`dbt compile`	`dbt compile --select <model>` renders SQL into `target/compiled`; it can connect for metadata or macro queries. Use `dbt parse` for local structure validation.
`dbt ls`	Alias of `dbt list`; `dbt list --select <selector>` shows scope before a build or test.
`dbt seed`	Load small project-owned CSV fixtures or lookups. Production vendor ingestion belongs in the approved source pipeline.
`dbt run`	Build selected models without tests.
`dbt run --select <model>`	Build the named model; selector forms are listed below.
`dbt test`	Run data tests; passing test SQL returns no invalid rows.
`dbt test --select <model>`	Run tests for the selected model.
`dbt build`	Build selected models, tests, snapshots and seeds in dependency order; failures can block downstream work.
`dbt docs generate`	Generate resource descriptions and lineage artefacts under `target/`.
`dbt docs serve`	Serve the generated local documentation site.
`dbt show --select <model> --limit 10`	Execute compiled SQL and preview a small result.
`dbt source freshness`	Compare source timestamps with configured warning and error thresholds.

See the compile documentation for connection and introspection behaviour. Inspect the executed output separately from generated SQL.

Selectors

Form	*Scope*
`model_name`	One named resource.
`+model_name`	The resource and all upstream ancestors.
`model_name+`	The resource and all downstream descendants.
`+model_name+`	Both directions.
`tag:pii`	Every resource carrying the tag.
`path:models/marts`	Resources below a path.
`source:morningstar`	Resources selected through a source group.
`state:modified`	Resources changed against a prior `manifest.json`.
`--exclude <selector>`	Removes a slice from the current selection.

dbt terms

Term	*Meaning*
Model	A SQL `SELECT` materialised as a view, table, incremental table or inline CTE.
Source	YAML for an upstream table, read with `source()`.
`ref()`	Declares a dependency and compiles its qualified relation.
Grain	What one row represents.
Staging	Source-shaped renames, casts and light checks.
Intermediate / integration	Reusable joins, pivots and calculations before publication.
Mart / prepared / curated	A consumer-facing fact, dimension, report or API shape; record the team's term.
Generic test	A reusable YAML rule such as `unique` or `relationships`.
Singular test	SQL under `tests/`; zero rows means pass.
Macro	A reusable Jinja and SQL function, for example a CG-specific macro that rewrites a legacy system's date format on the way in.
Snapshot	dbt's built-in observed-state history. The supplied course's two-clock example is the bitemporal load.
Manifest	`target/manifest.json`: resources, tests, sources and graph edges for lineage and `state:` selection.
CG model layout	Source CTEs first, joins next and the final select last. Check the repository's documented policy; a generated explanation is not proof of compliance.
Cosmos	The adaptor between Airflow and dbt. It's an Astronomer package that provides Airflow operators for dbt, `DbtDag` and `DbtTaskGroup` below.
Astronomer	Managed Airflow. `astro dev start` runs Airflow locally in Docker; `astro deploy --dags` ships DAGs to the managed side (Operations).
dbt-core 1.x	The Python dbt implementation used by the supplied project. dbt Cloud is a hosted offering; this fixture does not establish an enterprise-wide CG licensing or platform policy.

Airflow terms

Use these terms to separate the workflow definition, executable work and scheduling process.

Term	*Meaning*
DAG	A structured group of tasks plus its schedule; Airflow's primary object, as the model is dbt's.
Task	One program to run, built from an operator instance (a dbt run, for example).
Operator	The adaptor to the system a task talks to: `BashOperator` for the shell, a SQL operator for Postgres, Cosmos for dbt. Operators arrive from `pip install`ed provider libraries.
Scheduler	Evaluates schedules and dependencies and makes eligible work available for execution.
Dependency operators	`t1 >> t2` ("and then"), `t2 << t1` (the same edge from the right), `t1.set_downstream(t2)` is the same as `t2.set_upstream(t1)`; a list on either side fans out or in (`t1 >> [t2, t3]`), and `t1 >> t2 >> t3` chains. Reading a DAG file on Day 4 goes through a full example.

Python you'll meet in Airflow

The first seven forms support DAG authoring. Recognise the remaining forms when reviewing task execution and dependencies.

Form	*Practice*	*Example*
Function	Write	`def validate_exposure(rows, minimum=1): ...` defines a check and default.
Dictionary	Write	`{"owner": "exposures", "retries": 2}` stores keyed settings.
Decorator	Write	`@dag( ... )` declares a DAG factory, and `@task` declares task nodes.
f-string	Write	`f"fund={fund_id} rows={row_count}"` inserts values.
`datetime` and `timedelta`	Write	Timezone-aware start, and `timedelta(minutes=5)` for the retry delay.
Import	Write	Light imports at the top of the file. Put heavy libraries inside the `@task` so the scheduler parses the DAG quickly.
`raise`	Write	`raise ValueError("exposure row count is zero")` fails with a cause.
`subprocess.run`	Read	Runs dbt as a child process. A non-zero exit code must fail the task.
`>>`	Read	Airflow reuses Python's shift operator so `extract >> validate` records a dependency edge.
Type hint	Read	`def count(rows: list[dict]) -> int` documents the types. Python doesn't check them when it runs.

When you call an `@task` function inside a DAG, it adds a task and hands back a placeholder for the result. Airflow fills in the placeholder when that task runs, after its upstream tasks finish.

Cosmos operators

Operator	*Use it when*
`DbtDag`	A dbt-only workflow; supply project, profile, execution and scheduling configuration.
`DbtTaskGroup`	dbt is one step in a DAG that also runs other tasks, such as a notebook task.

Catalyst skills by DPLC stage

Use `dplc-concierge` to discover the skills available in your supplied version.

Stage	*Phase map and named entry points*
Define (0–3)	0 `dplc-prereq`; 1 `dplc-triage` + `marketplace-search`; 2 `dplc-discovery` for discovery, profiling, feasibility and spikes; 3 `dplc-design` for model, contract, STTM, NFRs and handoff draft
Define — targeted skills	The product requirements document skill (Phase 3). Give it a data product, then keep iterating in the same conversation to get a sample STTM and glossary. The PRD and data contract skills fill in a template. The `dplc-` skills ask you questions.
Deliver (4–6)	`dplc-build-stage` spans all three; Phase 4 has no separate orchestrator and uses alignment, the dbt and DAG generators and a checklist; 5 `dplc-validate`; 6 `dplc-publish`
Sustain (7–8)	7 `runbook`, `sla-dashboard-spec`, `incident-log`; 8 usage, impact, deprecation and `column-lineage`
Any phase	`dplc-concierge`, connections, queries, SQL validation and lineage

What `generate-dag` writes for you

`generate-dag` writes the DAG's name, schedule, defaults, generic tasks and dependencies. A short result looks like `dag_id: morningstar_exposures_daily`, a weekday `schedule_interval: "0 7 * * 1-5"`, `start_date: "2026-01-01"`, owner and retry defaults, then a freshness check, `dbt run` and `dbt test` in that order.

You, or an approved template, add the rest:

- the operators;
- callbacks;
- routing tags and the owner;
- the runbook link.

For the course, use the Cosmos recipe from Day 4, and check the generated DAG against the SLA and runbook.

> For current workspaces and phase commands, use Projects and Capstone. Adapt historical examples to the supported runtime.

Reference recap: dbt concepts and evidence

Use this recap when reviewing a model result or diagnosing a failure.

Case study: A toolchain error is not a model defect

Situation. A dbt command fails before the model executes because the selected shell cannot find the expected profile or dependency environment. Rebuilding model SQL would not address that failure.
Decision and reasoning. Identify the executable, profile, target and actual error before changing code. Inspect one configuration boundary at a time, then rerun the same bounded command. Configuration and dependency interactions make a multi-tool environment harder to diagnose than one SQL query.
Lesson. Classify setup/identity errors separately from model behavior, and retain the operation that failed. Use the current environment appendix for recovery.

- SQL-based transformations: a model query is materialized as a configured relation or inlined as an ephemeral model.
- Model selection: `--select` names the resources to run or inspect; check dependency scope before execution.
- Adapter boundary: connection and adapter settings select compute. SQL dialects and platform-specific behavior may need changes when porting a model; reuse does not guarantee portability.
- Tests: a data test returns failing records. Schema tests and purpose-built SQL checks protect different parts of the declared contract.
- Test packages: inspect the installed package and version for its available tests. Package availability does not replace a requirement-specific acceptance example.
- `ref` and `source`: configured relations and model references produce a dependency graph ordered for execution. A topological ordering respects those edges; it does not establish correct joins or population.
- YAML configuration: profiles, project settings, tests, descriptions and tags have distinct purposes. Read the effective configuration before interpreting a result.
- Documentation: `dbt docs generate` produces resource descriptions and lineage artifacts. The generated graph helps review dependencies, while a human reviews whether descriptions and rules are correct.

Learning Review

Retrieve materialization, selection and dependency resolution. Apply them to a model with two parents and one failed test: which selected build can proceed? Check the effective configuration, dependency scope, failing records and the consumer rule before explaining the result. This reference self-check supplements the day-ending review.

Connected dbt profile fields and targets

The assigned Portfolio Analytics route uses dbt-databricks. The instructor supplies the approved workspace, warehouse, catalog/schema and identity setup. Use the copied attempt's profile and the shared setup; historical local DuckDB exercises are a separate optional route.

Profile field	*Meaning and diagnostic question*
host	The remote workspace address: which workspace receives the request?
http_path	The SQL warehouse within that workspace: which compute executes the query?
catalog	The top-level Unity Catalog namespace for the assigned tables.
schema	The approved isolated write namespace within that catalog; its name is supplied by the instructor.
identity/token delivery	The principal making the request and the configured OAuth/session delivery; an authenticated identity may still lack object permissions.

These fields identify distinct boundaries. A correct host does not prove that the warehouse or schema is available to your identity. Record the failing operation for a `403`, then ask the owner for the assigned access; do not create schemas or grant yourself privileges. If the CLI identity works but dbt cannot use the generated environment, open a fresh dev-container terminal and follow the documented token-delivery troubleshooting before repeating login.

A project may define several targets, such as development and test. Selecting `--target` changes configured execution settings, not model intent. Read the profile and identify the exact catalog/schema before executing a write. Keep credential-bearing configuration out of Git, inspect the diff after setup and never paste secrets into evidence. A successful parse is local structure evidence; connected completion requires an actual authorized model/test run and its output.

Orchestration reference. The following examples explain scheduling and recovery. For the assigned scheduled Astro project, use Airflow Portfolio Analytics; adapt historical APIs before executing an example.

Airflow: DAGs, schedules, retries and alerts

> Read the vocabulary and a conceptual DAG file, then use the supplied v2 Airflow Phase 1 runtime. The Astro UI is a separate prepared route.
>
> THIS SESSION ANSWERS
>
> - What is orchestration, and what is a DAG?
> - What are a task, an operator and the scheduler?
> - How do the schedule, retries and callbacks put your SLA into code?
> - How does Cosmos run a dbt project as Airflow tasks?

> Knowledge check: Explain a run before reading its code
>
> How would you distinguish a workflow definition from one execution? Which checks should finish before a consumer-facing table is published?

Use the delivered Astro/Airflow environment and its isolated dbt-databricks executable. Check installed versions at the worker boundary before practice.

What Airflow is: orchestration

Airflow schedules tasks, evaluates dependencies and tracks execution and failure. DAGs are defined in Python; tasks can invoke shell commands, Python functions or remote systems. In this course, dbt defines transformations, Databricks executes their SQL, and Airflow coordinates the scheduled workflow.

DAGs (directed acyclic graphs)

A **directed acyclic graph (DAG)** defines task dependencies without a directed cycle. Independent branches can run concurrently when resources permit. A run can retry a task, and later runs can repeat the graph.

A successful task can make downstream work eligible; a failed task can block it. Failure callbacks report or record incidents separately from those dependency rules. In Airflow, the DAG object also carries scheduling configuration. dbt's DAG records model dependencies instead: keep data-flow, task-order and organisational SLA relationships distinct.

The four key terms: DAG, task, operator and scheduler

- **DAG:** the workflow definition and its scheduling configuration.
- **Task:** a unit of executable work, such as a Python function or CLI command.
- **Operator:** the template for a task, such as `BashOperator` or an operator supplied by a provider library.
- **Scheduler:** evaluates scheduling rules and dependencies, then makes eligible tasks available to the runtime.

Cosmos supplies the dbt integration described below.

Reading a DAG file

This historical Airflow 2-style example checks two inputs before staging, integration, calculation, publication and notification. Use the assigned Airflow guide for supported imports, scheduling arguments and the current starter.

```
from datetime import datetime, timedelta

from airflow import DAG
from airflow.operators.bash import BashOperator

default_args = {
    "owner": "positions-team",
    "retries": 2,
    "retry_delay": timedelta(minutes=10),
}

with DAG(
    dag_id="positions_daily",
    description="Integrated positions table, ready by midnight",
    start_date=datetime(2026, 1, 1),
    schedule="0 22 * * *",        # 10 PM daily, after both feeds land
    catchup=False,
    default_args=default_args,
    tags=["positions", "training"],
) as dag:

    check_imap = BashOperator(
        task_id="check_imap_landed",
        bash_command="python checks/imap_landed.py",
    )

    check_reference = BashOperator(
        task_id="check_reference_landed",
        bash_command="python checks/reference_landed.py",
    )

    stage = BashOperator(
```

```
        task_id="stage_positions",
        bash_command="dbt run --select staging",
    )

    integrate = BashOperator(
        task_id="integrate_positions",
        bash_command="dbt run --select integration",
    )

    calculate = BashOperator(
        task_id="run_trade_calculations",
        bash_command="dbt run --select calculations",
    )

    publish = BashOperator(
        task_id="publish_to_caspian",
        bash_command="python publish/broadcast_file.py",
    )

    notify = BashOperator(
        task_id="notify_consumers",
        bash_command="python notify/distribution_list.py",
    )

    [check_imap, check_reference] >> stage >> integrate >> calculate >> publish >> notify
```

Read the definition in order:

- **The imports.** Ordinary Python. `DAG` is the container object and `BashOperator` is one kind of task. `datetime` and `timedelta` are there because scheduling is all about time arithmetic.
- **`default_args`.** Settings applied to every task in the DAG unless a task overrides them. `retries=2` with `retry_delay=timedelta(minutes=10)` means a failed task is attempted up to two more times, ten minutes apart, before Airflow declares it failed. Your SLA, in code explains why those two lines matter for the SLA.
- **The `schedule` line.** `"0 22 * * *"` uses five cron fields: minute, hour, day-of-month, month and day-of-week. It schedules 22:00 daily, after the example's input arrivals. `@daily` corresponds to midnight (`"0 0 * * *"`), which would leave no processing buffer before the example's midnight deadline.
- **`start_date` and `catchup`.** The date from which schedules exist, and whether missed ones are replayed. The weekend outage covers both.
- **The operators.** Each task is an operator instance, and a BashOperator is the simplest adaptor. The system it talks to is the shell, so it's simply a terminal command to run. The `task_id` is the name you will see in the UI. Three of the seven commands here are Wednesday's dbt project being run layer by layer with `--select`; the pipeline's middle *is* the dbt project. Whether you author or generate a DAG, review its task IDs, commands, dependencies and failure behavior; syntax you cannot explain needs further inspection.
- **The dependency line.** `[check_imap, check_reference] >> stage >> integrate >> calculate >> publish >> notify`. The two input checks can run in parallel; staging waits for both. Each remaining task waits for its predecessor.

> Scheduling warning
>
> When the promise is tied to local time, use a timezone-aware start date such as `pendulum.datetime(2026, 4, 7, tz="America/New_York")`. Timezone and scheduling has the scheduling patterns and works out how much retry time you have.

The arrow has more than one spelling, and generated DAGs and the Airflow documentation use all of them. They all have the same effect:

```
t1 >> [t2, t3]           # a list fans out: t2 and t3 both wait for t1
[t2, t3] << t1           # the same edges, written from the right
t1.set_downstream(t2)    # is the same as t2.set_upstream(t1) is the same as t1 >> t2 is the
 ↪  same as t2 << t1
t1 >> t2 >> t3           # chaining: a straight line of three
```

`>>` and `<<` express dependency direction; the methods record the same edges. A list creates fan-out or fan-in dependencies, allowing concurrent execution when other requirements are met. Once the file is loaded, the web interface is where you act on it: from there you can manually trigger a DAG, inspect a task's logs, and monitor task status. The Airflow UI tour goes through each one.

Start a business review with the schedule, source checks, transformation commands, publication and notification. Explain which contract promise each step serves before inspecting its implementation. A reviewer can challenge a missing dependency or unrealistic deadline without first becoming the DAG author.

Your SLA, in code

Translate the contract's timing and failure promises into retry settings, failure callbacks and deadline checks.

When	*What happens*
22:00	The scheduled start, in the business timezone you declared.
First run fails	Wait 10 minutes and retry.
Retry 1 fails	Wait 10 minutes and retry again.
After retry 2	Check the delivery deadline. If the run is complete, the promise is met. If it's late or failed, a Datadog event goes out and is routed to a person by priority.

`retries` and `retry_delay`. Some failures are transient: a connection drops, a file arrives ninety seconds after the check ran. Two retries ten minutes apart can absorb a temporary failure while retaining its attempts in the run record. They also consume deadline buffer; account for runtime and retries along the whole critical path. In the positions-product example, critical failures require an on-call response; the retry settings establish how many attempts occur before escalation. Choose retries against the SLA: too few can escalate transient faults, while too many can delay intervention.

`on_failure_callback`. Monday's RICE example ranked three SLAs: timing, quality and the duty to explain. In Airflow, the duty to explain starts with a function:

```
def post_to_team_channel(context):
    ti = context["task_instance"]
    send_message(
        channel="#positions-pipeline",
        text=f"{ti.dag_id}.{ti.task_id} failed for {context['ds']}. "
             f"Logs: {ti.log_url}",
    )

default_args = {
    "retries": 2,
    "retry_delay": timedelta(minutes=10),
    "on_failure_callback": post_to_team_channel,
}
```

Airflow calls the function when a task has used up its retries and failed. It passes a `context` dictionary with the task, the run date and the log link. `send_message` stands in for whatever the team posts with, and on the paved path that's a Datadog event. Configure a tested notification route for missing or late data rather than relying on manual discovery.

> Alert routing on the paved path
>
> On the paved path, Datadog is the only automatic destination for alerts. People pick it up from there, depending on priority.
>
Priority	*Human response from the Datadog event*
P1	Support pages the on-call rotation through **xMatters**.
P2	A human posts to **Microsoft Teams** for discussion.
P3	The team creates a **Jira** ticket.
>
> Every event carries `l1_domain`, `l1_xmatters_group`, `lifecycle_phase`, `data_classification`, and `criticality`; The supplied alert-routing case disallows `email_on_failure`; confirm the current operating policy and routing owner before using that restriction outside the case. Once a month, look back over 30 days of P2 alerts. If more than 20% led to no action, move that condition to P3 or remove it. Capital Group alerting has the full routing model.

`sla` and `sla_miss_callback`. Failure callbacks fire when something breaks. But a pipeline can miss its deadline without failing at all. Every task succeeds, slowly, and the 7:30 delivery from the RICE example lands at 9. Airflow 2 handles this with an `sla` on the task plus an `sla_miss_callback` on the DAG:

```
with DAG(
    ...,
    sla_miss_callback=post_late_delivery_notice,
) as dag:

    publish = BashOperator(
        task_id="publish_to_caspian",
        bash_command="python publish/broadcast_file.py",
        sla=timedelta(hours=9, minutes=30),
    )
```

In this Airflow 2 example, a 07:30 promise after a 22:00 scheduled start allows nine and a half hours. Airflow 3 removed `sla`; Airflow 3.1 introduces deadline alerts. Check the supplied version's syntax before adapting the example.

Runbook and incident log

Workflow

Keep a runbook beside the DAG and change it in the same pull request as the code. It has seven sections:

- Service overview: purpose, owner, schedule and SLA.
- Architecture: the path from source to consumer, and its dependencies.
- Access: Airflow, Databricks, logs and dashboards.
- Common alerts: symptom, diagnosis and fix for each one.
- Escalation: who's on call, in what order, and how fast they respond.
- Manual procedures: backfill, retry, rollback and how to check the result.
- Contacts: team roles, channels and xMatters schedules.

For the Morningstar exposures pipeline, two alert rows look like this.

Alert	*Symptom*	*Diagnosis*	*Remediation*
Morningstar freshness	Freshness task fails.	Check latest load and owner status.	On arrival, clear and rerun; notify if the SLA is at risk.
dbt test	Test task fails.	Trace failed rows to source or model.	Escalate source or release a model fix; log the outcome.

At resolution, log what happened and the runbook step used. No matching step means a new failure mode, so add an entry before closing the follow-up.

- **dbt full refresh:** rebuild bad incremental state from source.
- **Delta time travel:** `RESTORE TABLE ... TO VERSION AS OF ...` for a known good table version.
- **Airflow task clear:** after fixing the cause, rerun the failed task and its downstream chain.

Operations has the full runbook template, a four-row alert table and the incident-log schema. The incident drill applies it to a failure.

Backfill, catchup, and idempotency: the weekend outage

Suppose an expired certificate prevents the weekend runs and is repaired on Monday. Decide which missed business dates require replay.

- **`catchup`** is the setting that decides automatically. `catchup=True` allows scheduled runs for missed intervals; `catchup=False` does not automatically create that full backlog. The example DAG sets it to `False`, because for this pipeline consumers want *tonight's* positions, and rebuilding Saturday's file on Monday serves nobody. A pipeline producing daily history files would choose the opposite. Either can be right. What goes wrong is leaving it to the default, which is why "is `catchup` set?" is on the closing challenge's checklist.
- **Backfill** is the manual form of the same replay, for when you want specific dates: `airflow dags backfill positions_daily -s 2026-08-22 -e 2026-08-24` re-runs the DAG for that date range, whatever `catchup` says. That's the Airflow 2 command. In Airflow 3, which the challenge runs, it's `airflow backfill create --dag-id positions_daily --from-date 2026-08-22 --to-date 2026-08-24`.
- **Idempotency** is the property that makes either replay safe: running a task twice for the same date must leave the same state as running it once. The counter-example is a task that *appends*. Replay the weekend and Saturday's rows land twice. A table rebuilt from fixed inputs can reproduce its result; an incremental model needs a key and update policy that prevent duplicate replay. Volatile expressions such as `current_timestamp()` can change values between runs. When you audit a generated DAG, "can every task run twice safely?" is the question behind both the catchup setting and the clear-to-rerun button in the UI.

Astronomer

The supplied stack combines Airflow, Astronomer and Cosmos. Astronomer provides tooling for local development and managed Airflow hosting:

1. Local, automatic running of Airflow jobs in containers on your own system, which is what makes Airflow pipelines reproducible and is how the dev-container challenges run Airflow on this course.
2. **Managed Airflow hosting.** The supplied platform routes deployment through Harness. Learner practice uses local Astro services; deployment is a separate authorised operation.

Questions for choosing a managed runtime:

- *Where does dbt run?* The assigned route runs dbt on an Astro worker and sends SQL to Databricks. Databricks also offers scheduling; choose the orchestrator according to the supported execution and dependency requirements.
- *Vendor dependence*: assess platform-specific configuration, operating costs and migration effort. In the supplied scenario, per-team Airflow instances reduce the coupling created by one shared instance.

Cosmos: one Airflow task per dbt model

Something has to run Wednesday's dbt project inside Airflow, and the blunt option is the one already visible in the walkthrough DAG, a BashOperator per layer, or even a single one for the whole project:

```
run_dbt = BashOperator(
    task_id="run_dbt",
    bash_command="dbt build --project-dir /usr/local/airflow/dbt/positions",
)
```

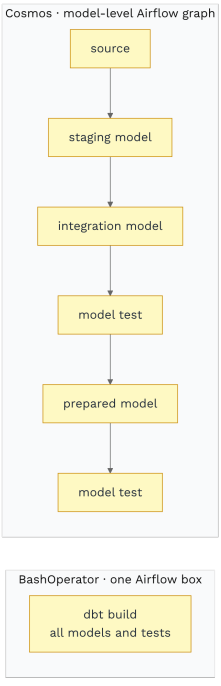


Figure 139: The same dbt project at two Airflow resolutions: one BashOperator task, or the model-and-test graph rendered by Cosmos.

A single BashOperator provides task-level logs and retries for the entire dbt command. Airflow cannot separately retry a model within that task, though the dbt log identifies the failed resource.

Cosmos renders the dbt dependency graph as Airflow tasks, with model-level logs and retries. Its test configuration determines how tests appear and gate downstream work. The following sketch shows the project configuration; a runnable DAG also needs the appropriate profile and execution settings:

```
from cosmos import DbtDag, ProjectConfig

positions_dbt = DbtDag(
    dag_id="positions_dbt",
    project_config=ProjectConfig("/usr/local/airflow/dbt/positions"),
    schedule="0 22 * * *",
    start_date=datetime(2026, 1, 1),
)
```

`DbtTaskGroup` embeds the rendered dbt tasks within a larger DAG, so input checks, publication and notification can surround the transformation graph:

```
from cosmos import DbtTaskGroup, ProjectConfig, ProfileConfig

with DAG(dag_id="positions_daily", schedule="0 22 * * *", ...) as dag:

    # check_imap, check_reference, publish and notify as in the walkthrough

    positions_models = DbtTaskGroup(
        group_id="positions_dbt",
        project_config=ProjectConfig("/usr/local/airflow/dbt/positions"),
        profile_config=ProfileConfig(profile_name="positions", target_name="dev"),
    )

    [check_imap, check_reference] >> positions_models >> publish >> notify
```

The transformation logic stays in dbt. The three layer-level BashOperators become a group of model-level tasks; the surrounding checks, publication and notification remain ordinary Airflow tasks. Use `DbtDag` for a dbt-only workflow and `DbtTaskGroup` when dbt sits within a larger workflow.

> Expert
>
> A production deployment also supplies a profile configuration pointing at the connection details created by Wednesday's `.env` setup. The paved path chooses this over the bash one-liner because each model becomes independently retryable and observable, at the cost of a dependency on the Cosmos library.

Data mesh and localised development

Data mesh places data-product ownership and development in the relevant business domains rather than one central delivery team. Shared platform tools support self-service while domains remain responsible for meaning, consumers and service promises.

Self-service does not remove coordination needs. Teams still need contracts, access rules, dependency responsibilities and clear ownership. Per-team Airflow instances can reduce runtime coupling, but folder or instance boundaries alone do not establish a governed product.

Appendix E — Environment and troubleshooting

Begin with Using VS Code and the training dev container for workspace navigation, terminal use and the supported v2 setup sequence. The tables here help diagnose a failed step; they do not establish a second setup route.

Troubleshooting runbook

Use this whenever a command stops working in the dev container. Find your symptom in the table, change one thing, then run the check command for that layer. A `401` means your login failed. A `403` means you logged in but don't have permission.

> **Credentials and packages**
>
> Learner setup uses interactive OAuth by default. Use the explicit PAT fallback only when the instructor supplies it. Shared service identity is a separate platform responsibility. Install dependencies through the configured corporate package route.

Run the first check that matches the failure boundary.

Layer	*Sign*	*First command*
dbt profile, identity or warehouse	`401`, profile error or unreachable warehouse	`dbt debug`
Astro CLI	`astro` is not found	`astro version`
Public network, VPN, proxy or TLS	The public route fails	`curl -sI https://github.com \{}| head -1`
Private Git	The private Git route fails	`gh auth status`, then `git fetch`

Common symptoms, causes and checks:

Symptom and cause	*Fix and check*
Copilot Chat or Catalyst gives no useful response. Copilot is signed out or the chat is reading a different project/context from the intended workspace.	Sign in, open the approved project with File → Open Folder and open Copilot Chat from its UI. Verify that chat can read a named repository file.
`dbt debug` returns `401`, "token expired", "invalid host" or "invalid token". The credential or OAuth session expired, an environment variable is empty, or the host or HTTP path points at another warehouse.	Use the project's OAuth-first `mise run setup:databricks`, then open a fresh container terminal and compare the assigned host/warehouse with the generated profile. Inspect identity and the failing operation; do not print or commit secrets.
The profile is missing, the schema is absent, or Databricks returns `403`. dbt cannot find the profile, the target is wrong, or your login works but lacks `USE` or `CREATE` on `training_<initials>`.	For assigned v2 work, use the copied workspace's profile with `--profiles-dir .` and inspect its assigned target. Ask the owner to resolve missing rights; verify the documented command without creating an extra schema or object.
A Dev Container task fails on `.sh` or `.env` with an odd parse or "no such file". Git or the editor wrote CRLF; bash expects LF.	Save as LF, run `git config core.autocrlf input`, and add `* text=auto eol=lf` to `.gitattributes`. Verify `file script.sh` no longer reports CRLF and rerun the task.

Symptom and cause	*Fix and check*
pip, git or npm reports `CERTIFICATE_VERIFY_FAILED`. Corporate SSL inspection presents a root CA the tool does not trust.	Get the approved CA, then set `PIP_CERT`, `REQUESTS_CA_BUNDLE` and `NODE_EXTRA_CA_CERTS` to it. Git uses `http.sslCAInfo`. Repeat the command without disabling TLS checks.
Git HTTPS returns 403, SSH returns `publickey`, or a private repository returns 404. The credential expired, the SSH key is absent, the remote is wrong, or access is missing.	Refresh the approved GitHub credential, or add `~/.ssh/id_ed25519.pub` to GitHub. For 404, inspect `git remote -v` and confirm team access. Verify `gh auth status` or `ssh -T git@github.com`, then `git fetch`.
Downloads time out or cannot resolve a host. VPN is disconnected, proxy variables are absent, or DNS is stale after a VPN change.	Reconnect VPN. Set `HTTP_PROXY`, `HTTPS_PROXY` and `NO_PROXY`. Flush DNS with the OS command and use the approved package index. Verify `curl -I https://github.com` and retry the original host.

Don't disable TLS verification, and never commit a secret.

Problems seen in the client dev container

If authentication fails, don't fix it by installing packages into some other environment. Check which executable you're running and which target it points at.

Common failure boundaries in the prepared client environment:

- If you start Astro before the setup exports the warehouse HTTP path, its runtime variables can be empty. Fix the dev-shell setup, then restart the Astro stack so its processes pick up the new values.
- Logging in again with OAuth in your own shell doesn't refresh a worker's cached connection. Refresh the token or connection through the repository's own setup, then look at the run you selected.
- Windows profile links and corporate path expansion can point at host folders the container can't see. Find out which process reads the path before you change it.
- A dbt selector can pick up tests on models that are still stubs. Select only your own model while you're working, then run the full selector before you finish.
- The current v2 preparation/reset interface creates a new attempt and refuses an existing destination. Preserve the previous workspace and choose a fresh `--workspace` path; verify that same path explicitly.
- Some retained legacy verifiers resolve dependencies or restore lock files. Current v2 checks use a temporary copy and hash the supplied workspace. Inspect the command's boundary and the diff before accepting evidence.
- Before rebuilding an image, identify the failing executable, target and error.

Screen index

Screens	*Explained in*
DevHub: LaunchLake guide	Day 2
Choosing an AWS role	Operations
Paved path: from Git to Harness	Day 2
The OpenITG organisation	Day 3
The Pull requests tab	Day 3
PRs on a real project	Day 4
Harness projects	Operations

Screens	*Explained in*
Create pipeline and import from Git	Operations
Pipeline YAML in VS Code	Operations
Pipeline Studio	Operations
Input sets	Operations
Execution history	Operations
Design doc & SSO task	Operations
Workspaces	Operations
Helm values	Operations
Lambda functions	Day 4
MWAA environments	Day 4
DAG list	Day 4
DAG details	Day 4
Grid view & run summary	Day 4
Fan-out runner	Day 4
Astro home	Day 4
S3 buckets	Day 3
Catalog Explorer	Day 3
Marketplace home	Day 1
Data apps overview	Operations
Deployments	Operations
Pod YAML	Operations
Business capabilities	Day 1
Contract folder	Day 2
Skills in GitHub	Day 2
Three stages	Day 2

Appendix F — Managing stakeholders

Use this reference when an interview, adoption decision or organisational change needs more investigation than Monday's core product brief. It expands Monday's interviews and adoption bridge. The activities are optional and do not replace the required producer/consumer/creator interview or design canvas.

Empathy maps

Use an empathy map to investigate a workflow or adoption concern. Its four quadrants separate what people **say** and **do** from what you infer they **think** and **feel**. Prepare with a business colleague and treat the map as evidence and questions, rather than a personality diagnosis.

Empathy map: what you observe and what you infer

Record observations in the Says and Does quadrants; check your Thinks and Feels hypotheses during the interview.

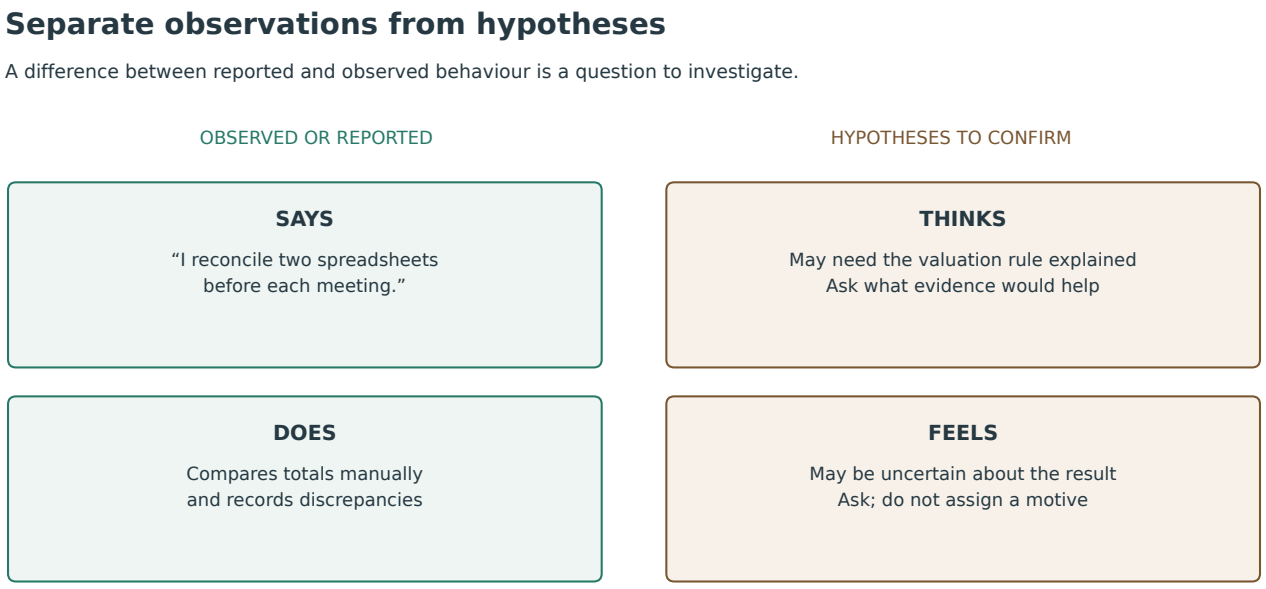


Figure 140: Observed statements and actions are separate from hypotheses about thoughts and feelings; ask the consumer to confirm the interpretation.

Worked example: the Excel workaround

Scenario: consumers reproduce a delivered report in Excel or a private script instead of using the data product. Investigate the workaround before proposing a replacement.

Interview statement	*Question to investigate*
"We have always done it this way."	Which familiar steps or presentation features does the current workflow support?
"I needed a small change."	Is a capability missing, or does the consumer need to correct or approve a specific step?
"I want better control."	Which decision or responsibility requires visibility and feedback?
"I did not know there was another way."	Can the consumer discover and access the product?
"I need to check the numbers myself."	Is the concern accuracy, explainability, skill or accountability?

Possible feelings include doubt, loss of control, concern about changing roles and frustration at duplicate work. Check these interpretations with the consumer. Criticising the workaround can obscure a genuine product defect.

If validation evidence resolves an accuracy concern but the consumer still wants to run the numbers, ask which step they need to understand, reproduce or approve. Reliability, competence, ownership and familiarity require different responses.

Basic human motivations

The categories below are prompts for investigating a concern, not conclusions about someone else's motives. Ask for examples and check your interpretation. Safety, purpose, mastery and ownership can connect several categories at once.[3]

Motivation	*Observable stakeholder concern*	*Useful interview response*
Power	"Who will decide changes to this process?"	Establish decision rights, the accountable owner and how an affected role can challenge a change.[4]
Control	"I still need to run the numbers myself."	Ask which step needs visibility, correction or approval; agree a real feedback channel and its response time.[5]
Competence	"I cannot reproduce or explain this result."	Ask for the difficult step; provide a worked comparison, training or documentation and check that the person can use it independently.
Trust	"The source was wrong last time."	Investigate the failure, agree evidence and validation, and demonstrate a reliable response when the promise is missed.[6]
Familiarity	"The spreadsheet fits the way I work."	Observe the workflow; preserve useful affordances or plan an explicit transition with support.

[3] Safety, purpose, competence and mastery can affect adoption. Use them to frame questions, not to diagnose a person or reduce every objection to a psychological cause.

[4] Power concerns the ability to affect an environment; money, authority, responsibilities and recognition can affect it. A change may shift institutional influence. Make those consequences visible and negotiate with the appropriate owner rather than assuming compensation resolves them.

[5] Control concerns whether actions affect an outcome; competence concerns doing so reliably. A person may trust a product but still need an auditable way to correct or approve it. Repeated lack of effective feedback can lead to withdrawal; investigate the process before attributing the behaviour to the individual.

[6] Delegating a check depends on confidence that another party acts reliably and in the consumer's interest. A source-quality defect, an unexplained volatile value and a history of missed commitments require different responses.

Motivation	*Observable stakeholder concern*	*Useful interview response*
Esteem	"I am held responsible for results I cannot explain."	Clarify responsibility, involve the person in review and recognise their contribution without substituting recognition for a working product.[7]

Stakeholder roles, influence and adoption

Stakeholder roles

Identify the sponsor (commissions or champions work), approver (authorises a decision), consumer (uses the output), blocker (can prevent progress) and influencer (affects others' decisions). Check these roles against actual responsibilities and decisions, as well as interview answers.

The supplied course uses the operating roles below. These are a different dimension from sponsor, approver, consumer and influencer; they are not one-to-one replacements. A data owner may sponsor one decision and consume another product. Record who approves access, business meaning and release separately, using the current product's responsibility matrix rather than inferring authority from a title.

- Data owner — accountable for the data and may sponsor or approve related work.
- Data product manager — owns product priorities and represents consumer needs.
- Data steward — maintains SLAs, standards, quality and rules. Identify the named person; stewardship may be a responsibility rather than a job title.
- Data product creator — designs, builds and checks the product.

Stakeholder mapping, and finding your leaders

Map stakeholders by influence and interest, then add their responsibilities for the specific decision.

Influence / interest	*Communication and involvement*
High / high	Review critical decisions with the approver or champion; address a blocker's concerns directly.
Low / high	Consult users and specialists where their knowledge is needed. Align at the start and end of a phase, and provide progress summaries.
High / low	Provide concise updates on strategic decisions and request approval when required.
Low / low	Share a relevant release brief and monitor for changed involvement.

Recheck the map: a trusted specialist may influence an approver without formal authority. Interest suggests how often to communicate; responsibility and influence identify which decisions need someone's involvement. Avoid replacing a required approval with repeated discussion among interested colleagues.

Agree communication responsibilities with the product manager. As a creator, identify the decisions you own, evidence you supply and approvals you need.

[7] Esteem concerns respect and recognition within a relevant group. Public embarrassment, unclear accountability and losing a valued contribution can obstruct adoption. Credit useful work and provide meaningful involvement; an award cannot compensate for inaccurate or inaccessible data.

> **Case study: A decision with the wrong participants**
>
> Situation. A creator repeatedly discusses a proposed product with interested colleagues, but the access or delivery decision remains blocked. The person with formal responsibility has not reviewed the evidence.
> Decision. Identify the required approver, the people affected and the domain reviewer who can test the proposal. Send the relevant evidence to each role and record a decision point instead of adding another broad meeting.
> Reasoning. Interest helps explain needs and adoption, while formal responsibility determines who can approve the particular change. Neither is a reason to dismiss a valid concern. Match the discussion to the decision and retain the affected consumer's requirements.
> Lesson. Use the stakeholder map to arrange a review, then verify the assumptions behind the roles. Update it when responsibility or the product scope changes.

Match each review to the person's responsibility and evidence needs. An informed sceptic may expose a valid defect; use evidence and explicit decision criteria rather than dismissing the concern because of a label.

Stakeholder attitudes and influence

The diagram separates how much formal influence someone has from how ready they are to support the change. An influential sceptic may know things about the domain that you need, and an enthusiastic colleague with little authority may make the best early tester. For each person on the map, write down one thing you'll ask of them, such as reviewing a valuation rule, letting you watch their current workflow or sponsoring an access decision.

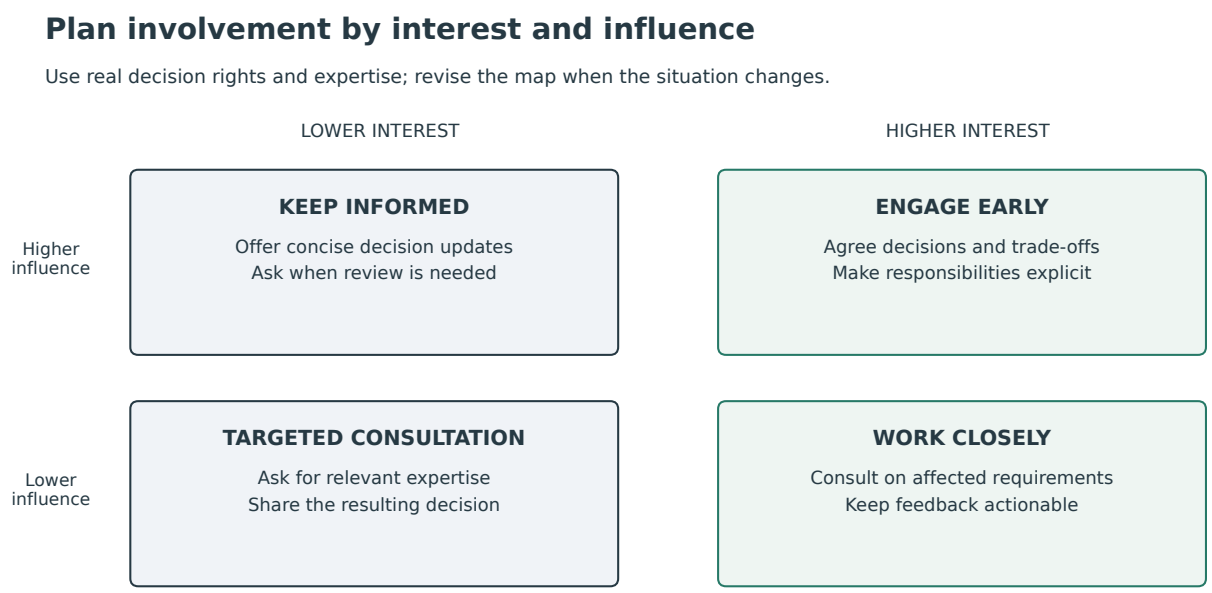


Figure 141: Interest and influence guide involvement: keep informed, engage early, consult for expertise and work closely on affected requirements.

Why good data products stall

A technically sound product can stall because it lacks priority, access, discovery, training or a fit with the consumer's workflow. Check those practical objections before interpreting resistance as a question of esteem, power or control.

- Esteem: recognise useful contributions and provide meaningful involvement.

- Control: make decisions, correction routes and feedback visible.
- Power: discuss changes to authority and responsibility with the appropriate owner; recognition or rewards may not resolve them.

Change specialists can help maintain empathy maps, communication plans and adoption decisions.

Adoption readiness

Identify adoption barriers before investing in implementation. Record access, discovery, training and workflow decisions alongside the technical design, with an owner and a check that the intended consumer can use the product.

Appendix G — AI reference

For a generated diagram or source-to-target mapping, substantiate each edge with supplied keys and each measure with its definition. In the CG product brief, a shared product ID alone does not make two return observations comparable: check basis, kind, period and reporting date. Mark unsupported relationships and assumptions explicitly.

Use this reference alongside Tuesday's product-to-Catalyst workflow to understand model training, review risks and AI costs. The supplied Catalyst version is 1.13.0.

Model training and post-training

Pre-training learns patterns from data to predict or generate text. The learned parameters are not a recoverable archive of training documents, and an answer is not necessarily a lookup of an existing passage.

Instruction fine-tuning uses examples of responses to requests. Preference training uses comparisons or ratings to shape preferred outputs. These activities can improve usefulness while leaving factual errors and incomplete instruction-following. The InstructGPT paper describes one combination of demonstrations and human comparisons; training methods vary by model.

A request to "write a data contract" leaves many product facts unspecified. A fluent draft can use the wrong consumer, invented columns or an unsupported delivery promise. Supply the actual product evidence and inspect the result. The GPT-3 paper illustrates why performance should be assessed for the particular task.

Generated claims and the word "hallucination"

Unsupported output can read as confidently as a correct answer. A claim about a table, approval or successful command needs evidence: check the cited source or relevant tool result. The term "hallucination" describes an output problem, rather than an intention to deceive. Correct the authoritative artefact so later work does not inherit the error.

Imitation and simulation as teaching analogies

Recognising a familiar pattern differs from validating the process you need. A proposed schema may look plausible while missing correction behaviour or unusual macro arguments. Ask what was tested: inspect generated SQL, compile it in the supported environment and compare executed output with an independently specified example. Parsing validates structure; it does not validate the business rule.

Prompt and context improvements should be evaluated on the task they serve. Repeat the relevant checks when grain, data type, correction policy or consumer changes.

Repeated actions, correlated mistakes and responsibility

> Expert
>
> If independent actions each have a 1% failure probability and any failure makes the run unsuccessful, the chance of at least one failure in 100 actions is `1 - 0.99^100`, about 63.4%. This example is not a measured Catalyst rate: real errors can be correlated, repaired and different in consequence.

Distinguish error frequency from impact. Many correct fields cannot compensate for a wrong currency basis, missing regulatory field or unapproved access rule. Rare, high-impact errors can dominate the cost of a process; measure both their likelihood and consequence.

Shared prompts, skills and glossaries can create correlated mistakes. Ten contracts using one incorrect glossary are not ten independent confirmations. Check against independent evidence and correct the authoritative glossary, template or skill.

"De-responsibilisation" describes reduced personal responsibility when a system appears to decide. The creator remains accountable for the reviewed artefact and its use. An agent transcript records a proposal; it does not grant approval.

Choose review points according to the cost of a wrong assumption. Verify facts before they become implementation inputs and verify executed results before treating an implementation as successful. Separate observed tool results from the model's summary.

The economics of AI: a dated framework

Compare the cost of producing an accepted result, including infrastructure, integration and review. The following framework distinguishes cost categories; it is not a current price list or forecast.

Broad training as an investment

Training requires data, compute, engineering and evaluation. Its up-front cost differs from the cost of another request and from your team's implementation and support costs. Open-weight releases can widen access to a capability; running them still requires resources, rights and governance.

Instruction and preference data

Reviewed examples can capture useful organisational knowledge: consumer questions, faithful contracts and tested corrections. A team can maintain these examples in prompts, templates and skills without retraining a model. Fine-tuning changes the trained model; editing a prompt changes its supplied context. Select an approach according to the problem and evaluate the result.

Use the organisation's approved service and data-handling rules. Check the applicable agreement for retention and training rights rather than assuming that providers or account types handle information identically.

Inference, reasoning and repeated requests

Inference consumes resources during use. A conversation or agent workflow can involve several model requests and tool operations, with additional computation in some systems. Measure the whole task's latency, recurring resource use and review effort against the accepted result. A subscription allowance or editor credit multiplier is not a direct measure of the provider's underlying cost.

Compare approved models and effort levels on the same representative task, using correctness, completeness, duration and inspection effort. Record the configuration and version; price alone does not establish suitability.

Agent workflows and specialised organisational value

Agent workflows coordinate operations using context, tools and review. Catalyst supplies organisation-specific instructions and artefact structures; assess whether those help someone perform a real responsibility reliably.

The "flooring effect" describes a possible consequence of widely available baseline capabilities: additional value may come from specialised data meaning, integration, useful interfaces and trusted decisions. Test

that hypothesis for your workflow. A team-specific skill needs maintenance and a check that it transfers to another product or department.

Models, harnesses and applications

A model supplies generation; a harness provides tools and context; an application connects them to a workflow and user. Costs and value can arise at each layer. A faster draft may shift effort into review and correction, so compare time to an accepted artefact rather than the number of pages generated.

Model choice, resource budgets and deployment options

Check the current approved models, allowances and reasoning controls in your environment. Evaluate them with a representative task. Hosted APIs, managed cloud services and self-hosted stacks have different data-handling and operational boundaries; local hosting also requires hardware, maintenance and governance.

A project-facts file can reduce repeated context questions. Keep it accurate, exclude secrets and retain the required approvals and output review.

Appendix H — Using VS Code and the training dev container

What's inside the environment

Check these tools in the approved training environment. Different Python environments can select different executables; confirm the terminal and Astro worker use their assigned dbt environments.

Tool	*Purpose and course use*	*Representative command / runtime*
mise	Pins tool versions, loads project environment, runs tasks with dependencies	`mise --version`; training dev container
Python / virtual environments	Run helpers with their own dependency sets	`python --version`; selected terminal or worker environment
uv	Creates environments and installs Python dependencies	`uv --version`; dev container / instructor v2 setup
Git / GitHub CLI (`gh`)	Local history, diffs, branches; authorized remote discovery/review	`git status`, `gh --version`; dev container; real PR work needs sandbox/reviewer
Databricks CLI	OAuth identity and permitted workspace/ catalog metadata	`databricks --version`; dev container, requests sent remotely
dbt Core / Databricks adapter	Transform/test models; send connected SQL to the warehouse	`dbt --version`; selected dev-container Python environment
Docker / Compose	Build images; run scoped containers and the inner Airflow runtime	`docker version`, `docker compose version`; CLI inside dev container, inner daemon
Astro CLI	Starts/logs/stops the separate local Airflow services	`astro version`; dev-container CLI launches Docker services
Airflow / Cosmos	Schedules task dependencies; Cosmos connects prepared dbt graph to tasks	Airflow executes in the separate Astro runtime
jq / ripgrep / curl	Inspect JSON, search files, request resources	`jq --version`, `rg --version`, `curl --version`; development helpers
Flyway / Java	Schema-migration tooling shown in the prepared platform	`flyway --version`; declared image tools, outside assigned provisioning
datacounsel	Contract helpers, including migration generation	`datacounsel --help`; declared image helper, not assigned migration/deployment work
Editor extensions	Python/Pylance/formatting, YAML, GitLens, Copilot, Databricks, Prettier and Makefile editing	Installed into the connected VS Code environment by configuration; access-dependent AI features need their own entitlement

The project's `.mise.toml` pins tools and defines setup and challenge tasks. Run version checks and `dpc-check` before practice. Ask the instructor to resolve missing dependencies in the assigned environment; avoid recreating an environment over active work.

Check your understanding

> **Knowledge check: Editor, container and command boundaries**
>
> Where did your last command run? What does mise manage? What does a WARN in `dpc-check` leave unresolved? How does your dev container differ from the Airflow runtime and remote Databricks? Why does verifying a starter differ from checking your attempt?
> A useful answer identifies the VS Code connection and directory; separates versions/environment/tasks; names the specific unresolved readiness gate; distinguishes separate processes and remote execution; and uses `--workspace` for learner verification. Keep the command, task ID, non-secret output and your explanation in the copied workspace's evidence file. Follow troubleshooting when a prerequisite is unavailable.

For authentication, namespace variables and worker setup, follow Readiness and the shared setup.

First v2 command and everyday reference

From the project root, discover the supported command interface:

```
python3 scripts/challenge-v2.py --help
python3 scripts/challenge-v2.py list
```

Prepare the assigned dbt Phase 1 attempt, then enter it:

```
mise run challenge:v2:prepare dbt phase-one-certified
cd .local/challenges-v2/dbt/phase-one-certified
```

The preparation command copies the supplied starter into a new directory and prints that path. It refuses a directory that already exists, preserving your earlier attempt. Choose a fresh `--workspace` path if you need another attempt; reset also refuses overwrite. `cd` enters the copied attempt; confirm its location with `pwd` and check that `dbt --version` identifies the assigned Databricks adapter. Read the dbt Phase 1 guide. Run direct dbt commands there so you learn the actual tool. The prepared starter has meaningful initial limits; a passing starter check is not your completed solution.

Command	*Working directory / result*
`dbt seed --profiles-dir .`, then `dbt build --profiles-dir .`	Copied dbt workspace; build the authored models and tests described in the owning task
`dbt show --profiles-dir . --select fund_nav_report`	Copied dbt workspace after authoring its mart; inspect its actual rows
`python3 scripts/challenge-v2.py verify-starter dbt phase-one-certified --mode offline`	Project root; checks a fresh baseline, not your work
`mise run challenge:v2:verify dbt phase-one-certified --workspace .local/challenges-v2/dbt/phase-one-certified --mode offline`	Project root; checks authoring structure in a temporary copy. Offline results leave connected completion pending.
`pytest tests/local_env/offline`	Project root after Python setup; tests local contracts, not live access
`astro dev start`	Project root, after connected authentication/settings and only for the prepared Astro route; launches separate services
`astro dev logs`	Same project; inspect logs, stop streaming with Ctrl+C
`astro dev stop`	Same project; stop its Airflow services when finished

Astro reports the actual UI address. Use the address and login supplied by the running training environment; the **Ports** view may provide a forwarded browser address. Do not infer a successful DAG execution from an open UI. If Astro started before authentication/environment changes, finish setup,

open a fresh terminal and use `astro dev restart --wait 5m` as conditional recovery. Restart is not a routine step for a healthy runtime.

Appendix I — Historical data and bitemporal loading

A financial report depends on more than the values available today. It may need the values that applied on a particular business date, the values that were available when the report was produced, or today's corrected understanding of that earlier date. These are different questions. A reliable historical model records enough information to answer the question the consumer actually intends.

By the end of this chapter, you should be able to distinguish business time from system time, design a temporal grain for reference and balance data, explain a retrospective correction, choose an appropriate incoming-file policy, and test historical queries and repeated loads.

Three kinds of history

Suppose a portfolio changes manager. An ordinary update replaces the old manager's name with the new one. This is useful for a current directory, but it loses the information needed to attribute last quarter's performance to the right manager.

A slowly changing dimension of type 2 keeps separate versions and associates each version with a validity interval. A query can then select the manager whose interval contains the reporting date. A type-1 dimension overwrites the old value, while a type-3 design retains a limited previous-value attribute. The choice depends on the consumer's need for history.

There is a further complication. The manager change may take effect on October 1 but reach the platform on October 8. A report produced on October 5 could reasonably contain the old manager because the new information had not arrived. Reproducing that report and producing a corrected report are both legitimate operations, but they should return different answers.

A bitemporal model records two clocks:

Clock	*Question*	*Meaning of the interval*
Business-valid time	When did this fact apply?	The period during which an attribute or relationship is true in the business domain
System-record time	When did the platform know this version?	The period during which this version was part of the platform's recorded understanding

A business change and a data correction can have the same arrival date but different effective dates. Using the arrival timestamp as the business start date collapses the two meanings and prevents accurate reconstruction.

Table-version history is a third concept. Delta's transaction history supports storage-level recovery and inspection of table states. Its availability depends on retained logs and data files. It does not automatically encode the effective dates supplied by a business source or provide a durable business-history interface. Retained table versions, type-2 dimensions, and bitemporal records solve related but different problems.

Define time and grain before writing the merge

The grain tells us what one business observation represents. A master/reference record might identify an instrument. A positions record might identify a portfolio, instrument, account, currency, and business date. Temporal versions add another layer of identity: the same business observation may have several recorded versions.

For reference data, define the natural key and the attributes being versioned. For balances, include the as-of date in the observation grain. A portfolio's balance on October 4 is a different observation from its

balance on October 5, even if the amount is unchanged. A correction to the October 4 balance is a new system version of that same observation.

Choose the time types deliberately. A business-valid date may have day precision, while a system timestamp needs sufficient precision to order multiple arrivals on one day. Define the time zone for timestamps and the convention for simultaneous events. The database column type and the source's effective-date meaning must agree.

This chapter uses half-open intervals:

```
start ⩽ requested_time < end
```

The start is included and the end is excluded. A version ending at midnight on October 8 and another starting at that instant do not overlap. When a source supplies an inclusive end date, normalize it carefully into the chosen representation rather than mixing interval conventions.

An open interval may use a null end or a distant-future sentinel such as `9999-12-31`. A low business-date sentinel such as `1900-01-01` means an unspecified or conventionally unbounded start; it does not establish that the entity existed in 1900. Query predicates must match the representation chosen by the product.

A reference-data correction across both clocks

Consider instrument I1. The platform receives its reference record on October 4. The source says that the instrument has been named **Orion Holdings** since September 1. On October 8, the source supplies a correction: the name should be **Orion Asset Management** from October 1.

Before the correction, the platform knows one version:

Instrument	*Name*	*Business from*	*Business to*	*System from*	*System to*
I1	Orion Holdings	Sep 1	Open	Oct 4	Open

After processing the correction, a canonical interval representation contains three rows:

Instrument	*Name*	*Business from*	*Business to*	*System from*	*System to*
I1	Orion Holdings	Sep 1	Open	Oct 4	Oct 8
I1	Orion Holdings	Sep 1	Oct 1	Oct 8	Open
I1	Orion Asset Management	Oct 1	Open	Oct 8	Open

The first row preserves the earlier understanding. It describes what was known between October 4 and October 8. The second and third rows express the corrected business timeline known from October 8 onward. They split the earlier business interval at the correction's effective date.

Now ask three questions:

Business date	*Knowledge date*	*Answer*
Oct 2	Oct 6	Orion Holdings
Oct 2	Oct 9	Orion Asset Management
Sep 30	Oct 9	Orion Holdings

The first result reproduces earlier knowledge. The second uses the correction for the same business date. The third shows that the correction does not change the name before October 1.

For this example, the two temporal predicates can be written as follows. The parameters identify the requested business date and knowledge timestamp; they are supplied by the report or application.

```
select instrument_id, instrument_name
from instrument_history
where business_from ⩽ :business_date
  and (business_to is null or :business_date < business_to)
  and system_from ⩽ :knowledge_timestamp
  and (system_to is null or :knowledge_timestamp < system_to)
```

The query uses both clocks. Filtering only on a current flag cannot reproduce all three answers. Filtering only on business time could return the old and corrected names together because both are valid representations of different knowledge states.

A product's loading macro may store the history differently, but the observable query behavior must satisfy its contract. Define expected results from concrete histories before selecting an implementation.

Late-arriving dimensions

A position can arrive before the descriptive record for its instrument. The position is a fact; the instrument description is a dimension. This is a late-arriving dimension problem.

Suppose a position dated October 1 arrives that day, but its instrument definition arrives on October 5. A business-date join made with today's knowledge may enrich the earlier position successfully. A report reproducing knowledge on October 2 must reflect the absence of that definition at that time.

Choose how the product handles the gap. It may retain the fact with an explicit unknown classification, hold publication until the reference record arrives, or expose a separately documented incomplete state. An inner join that silently drops the position makes the problem harder to detect. The intended policy should be visible in tests, reconciliation, and consumer documentation.

There are two different quality questions: whether every required reference record is available now, and whether the historical report accurately reflects what was available then. A reconstruction can be historically accurate while containing an unknown value. A current corrected view can resolve that value without pretending it was known earlier.

Balance and positions history

Balances are observations at a business date, rather than entity attributes valid across a continuous business interval. Consider a portfolio position whose supplied market value is corrected:

Portfolio	*Instrument*	*As-of date*	*Market value*	*System from*	*System to*
P1	I1	Oct 4	100.00	Oct 4	Oct 9
P1	I1	Oct 4	101.50	Oct 9	Open

The two rows refer to the same observation: portfolio P1, instrument I1, as of October 4. The second row changes the platform's knowledge of the amount; it does not represent the next day's position.

A historically known balance query selects the business as-of date and the system interval:

```
select portfolio_id, instrument_id, market_value
from position_history
where as_of_date = :business_date
  and system_from ⩽ :knowledge_timestamp
  and (system_to is null or :knowledge_timestamp < system_to)
```

A reference/master strategy and a balance strategy therefore use different business semantics. Applying the reference strategy to daily observations can accidentally stretch one day's balance across several days. Applying a balance strategy to continuously valid reference attributes can force consumers to manufacture daily snapshots they do not need.

The unique key must include all columns needed to distinguish the actual observation. Portfolio and date alone are insufficient when the portfolio contains several instruments or accounts. A generated surrogate key is useful for joins and storage, but it does not repair an incorrectly defined business grain.

Source delivery policy and loading policy

Before temporal merging, decide which arrivals should be processed. A pipeline may normally receive one file per day and then find several files waiting after an outage.

Incoming-file policy	*Suitable source meaning*	*Example*
Latest complete file	Each file fully replaces the same population, and intermediate states are unnecessary	A current currency reference snapshot
Chronological processing	Successive changes must be applied in order	Instrument rating updates with effective dates
Combine batches	Each batch contributes distinct incremental records	Several disjoint portfolio transaction files

Latest-file processing is safe only when the newer delivery really supersedes the older ones for the same scope. If each file contains different incremental additions, taking the latest loses data. If consumers need to reproduce every arrival, discarding intermediate full snapshots may also violate the history requirement.

Combining batches requires a duplicate and overlap policy. `UNION ALL` preserves every incoming row; `UNION` removes identical rows. Neither decides which of two different corrections to the same business observation should win. Establish batch identity, source sequence, effective dates, and tie-breaking before merging.

These choices are separate from three other decisions:

- **dbt materialization:** how a model is persisted and updated.
- **Temporal strategy:** how business and system versions are maintained.
- **Publication policy:** when validated results become visible to consumers.

An incremental source delivery may feed an incremental dbt model using a balance strategy, with publication held until quality checks pass. The repeated word "incremental" does not make these decisions interchangeable.

Prepared models and custom loading macros

In Caspian-style prepared models, the custom `bitemporal_load_v2` strategy works within dbt's incremental materialization. Configuration selects the load type and business-key metadata. Balance loads also identify the business as-of column.

A prepared balance model has the following configuration shape:

```
{{ config(
    materialized='incremental',
    incremental_strategy='bitemporal_load_v2',
    load_type='balance',
    as_of_col='position_date',
    detect_change_flag='Y',
    file_format='delta',
    unique_key='dl_row_uid',
    meta={
        'surrogate_key_cols': [
            'portfolio_id', 'instrument_id', 'position_date'
        ]
    }
) }}
```

This example assumes those columns are the observation grain and that the project's macro supplies the matching row identifier and audit fields. The complete model must produce the business values, checksum or change-detection inputs, keys, and audit columns expected by that macro. Configuration alone does not implement the load.

Temporal column names are part of the project interface. The maintained local `get_bitemporal_column_names('v2')` map uses `dl_business_start_date`, `dl_business_end_date`, `dl_system_start_timestamp`, `dl_system_end_timestamp` and `dl_current_indicator`; its checksum name is `dl_checksum_value`. Older examples use `*_from_date` and `*_to_date` names. Do not substitute one family into a model expecting the other. Inspect the assigned macro, its expected audit-column list and its boundary behavior together.

The course's maintained local strategy delegates through `get_incremental_bitemporal_load_v2_sql` to the shared strategy generator. It distinguishes `incremental`, `full`, `balance` and `balance_full` branches. The source-delivery catch-up policy is a separate configuration: the supplied curation enum calls its alternatives `latest`, `chronological` and `combined`. These names establish the configuration boundary; they do not prove that every late correction or recovery requirement has been implemented. Acceptance histories must verify that behavior in the assigned environment.

The interval tables in this chapter define a consumer requirement, rather than promise that every packaged macro produces exactly this physical representation. Before adopting a macro, compare its compiled statements and observed results with the expected histories. A loading strategy that cannot split a retrospective business interval needs adaptation or a different documented product interface.

A current indicator also needs a precise definition. It may represent the current business state, the latest known system version, or a specific combination. In a fully reconstructed business timeline, several historical business intervals can be part of the latest system understanding at once. "One current row per key" is meaningful only after defining which kind of current the flag denotes.

dbt snapshots provide type-2 history with `dbt_valid_from` and `dbt_valid_to`. They can be appropriate when that validity history meets the requirement. A single snapshot interval does not automatically provide independent business and system clocks. Similarly, an ordinary incremental `MERGE` that overwrites a match does not preserve the previous recorded value merely because it runs incrementally.

Corrections, replacements, and deletions

A full snapshot, an incremental change stream, and a bounded replacement have different meanings when a key is absent.

In a full snapshot, absence may mean that an entity is no longer present. In an incremental file, absence usually means that the batch says nothing about that entity. A bounded replacement may supersede only a declared portfolio, date range, or other block. Deleting everything not present in a partial delivery can remove valid data outside that block.

Explicit deletion indicators require equally clear semantics. A source may withdraw an incorrect return calculation or remove an entity. Decide whether consumers should see a tombstone, an ended validity interval, an excluded current result, or a physical removal under an agreed retention policy. A logical withdrawal and erasure of all historical evidence are different operations.

Corrections can arrive out of order. A later received message may have an earlier effective date than a previously processed message. The product must reconstruct the intended business intervals rather than assume that arrival order is business order.

Checkpoints, replay, and retained history

A checkpoint records which inputs a load has successfully processed. Advancing it before the output is durable can skip data after a failure. Failing to advance it after success can trigger duplicate processing. Design the checkpoint and output commit behavior together, and test the interruption points between them.

A replay of the same identified input should not create another business observation or an unjustified historical version. This is idempotency: repetition should preserve the correct final state. System-time history may record genuine new observations or source corrections, but a retry of the same batch should not masquerade as a new correction.

A full refresh deserves special attention. Rebuilding only from a current-state source cannot recover versions that existed solely in the historical target. Before rebuilding, establish which retained inputs can reconstruct the required history. A valid recovery plan identifies the source batches, transformation version, temporal rules, and historical output to restore.

Storage maintenance must respect this plan. Removing obsolete physical files affects table-version recovery, while deleting logical historical rows affects the consumer's business-history interface. Define the retention period for each and assess the impact on audit, reproducibility, and recovery.

Verify behavior with small histories

Temporal tests should exercise a sequence, not just a final row count. Start with a small set of keys and record the expected answer after each arrival.

Case	*Expected property*
First arrival	The business observation is available from the correct knowledge time
Unchanged retry	No duplicate observation or false correction is introduced
Genuine effective-date change	Business intervals change at the specified boundary
Retrospective correction	Historical and current-knowledge queries return their different intended answers
Late reference arrival	Enrichment follows the documented incomplete-data policy
Several waiting files	All required changes are processed in the correct scope and sequence
Explicit withdrawal	Current consumption and historical reconstruction follow the deletion policy
Failed load and recovery	Output and checkpoint remain consistent

Check that a query at a business/knowledge pair returns the expected cardinality. Non-overlap means that contradictory versions should not both cover the same business point and system point for the same key. Business intervals can legitimately overlap across different system-history versions; the old and corrected understandings in the reference-data correction above demonstrate why.

Review questions:

1. Why does selecting only the maximum load timestamp fail to reconstruct some historical reports?
2. In the Orion example, which rows answer a business date of October 2 with knowledge dates of October 6 and October 9?
3. Why is the business date part of a balance grain?
4. When does selecting only the latest source file lose valid information?
5. What does your current indicator mean, and which query cannot be expressed with it alone?
6. What evidence shows that a retry neither skipped an input nor created a false version?

Use the dbt and tooling reference for the snapshot example and the dbt project for assigned practice. The separately timed temporal-history clinic applies these principles to frozen arrivals, retries and recovery without changing a live platform.

Appendix J — Astro operations and runtime ownership

A managed orchestration service reduces the infrastructure work required to run Airflow. Reliable data products still require explicit ownership of workflows, dependencies, credentials and delivery outcomes. The platform team establishes common infrastructure and access conventions; the product team maintains the applications running within that boundary.

This optional reference extends the orchestration material with operational decisions for a hosted deployment. The assigned classroom project uses local Astro orchestration and remote Databricks computation. Its successful execution establishes evidence about that prepared environment; hosted access, resilience and support arrangements need separate validation.

Federation and the service hierarchy

A central Airflow installation couples teams to one dependency environment, scheduler, release process and upgrade timetable. Federation gives a team a deployment whose products can follow a common maintenance lifecycle. It reduces some shared failure modes while placing responsibility for that lifecycle with an identifiable owner.

Choose deployment boundaries around compatible operating needs. Products sharing dependencies, availability requirements and an upgrade owner can reasonably share a deployment. A separate deployment for each small task increases administration and resource consumption without necessarily improving delivery.

Component	*Operational meaning*
Organization	Top-level service administration and organizational access boundary
Workspace	Logical grouping of deployments and user access
Deployment	Configured Airflow environment receiving released code and execution settings
Cluster	Hosting infrastructure associated with deployments in the applicable platform arrangement
Astro Runtime	Packaged runtime defining Airflow and supporting software

A workspace groups access; it does not execute SQL. A deployment schedules work, maintains execution state and provides the relevant worker and UI/API services. The warehouse or remote job service may perform most data processing. Establish the actual execution mode, executor and infrastructure arrangement before assigning responsibilities.

Environment names also need evidence. For each development, SQA and production deployment, record its catalog/schema targets, credential identity, network route and intended consumer population. Two deployments named differently can still write to the same database. Isolation requires distinct, correctly configured boundaries.

Provisioning and configuration authority

Provisioning establishes a reproducible environment before regular application releases begin. A vended repository can provide runtime definitions, dependency files, DAG layout, delivery templates and environment configuration. Managed groups supply access through the approved onboarding route.

The platform IAC reference describes Terraform Enterprise state, environment-specific values, AWS roles and secrets, and monitoring integration. It sits outside assigned learner practice. Platform owners supply environment values and review infrastructure plans; ordinary DAG development uses the resulting environment.

Decide which configuration system owns each setting. A maintained Terraform value and an emergency UI adjustment can disagree. A later infrastructure apply may restore the declared value, unexpectedly undoing the incident response. Record exceptional changes, their reason and their expiry, then reconcile the maintained configuration before the next application of that state.

A release record should identify the application commit, dependency image, deployment and configuration revision. A successful pipeline proves that its steps completed for those inputs. Inspect the target environment to confirm the expected code and settings arrived, then obtain scheduled-run and output evidence separately.

Access controls follow boundaries. Deployment inspection, production administration, warehouse queries, secrets retrieval and consumer table access are different permissions. When a task fails, establish which identity crossed which boundary instead of requesting broad access to every layer.

Connections across hosted and classroom environments

In an AWS Secrets Manager-backed hosted pattern, the configured backend supplies approved Airflow connection and variable values. The deployment identity needs the corresponding retrieval permission. Adopt one maintained source for each value and avoid conflicting copies in the Airflow metadata database, environment variables and external backend. The actual lookup configuration determines which value execution receives. Astronomer's secrets backend guide describes the supported integration routes.

Plain variables, JSON-valued variables and connections have different uses. Supported operator template fields can retrieve configuration through expressions such as `var.value.product_setting`, `var.json.product_settings.target_name` and `conn.product_api.host`. Only designated fields are rendered; arbitrary strings do not become templates automatically. Hooks and task code can retrieve values through the runtime APIs appropriate to their execution path.

The classroom follows its delivered OAuth and worker connection route. The Portfolio setup requires terminal sign-in through `mise run setup:databricks`, then worker refresh through `mise run databricks:oauth-refresh` after starting Astro. Scheduled tasks resolve host and token from `databricks_default` at execution time. A newly authenticated terminal does not automatically refresh that worker connection.

Check the running worker's executable, versions, mounted project, writable artifact directories and selected catalog/schema/warehouse. Reconcile its namespace with the terminal's seeded relations. A terminal build can succeed while a worker lacks the executable; both can authenticate while selecting different schemas. The setup's owned prefixes and staging metadata make these differences diagnosable without overwriting another learner's attempt.

Record host, connection name and target identity where appropriate, with credentials redacted. Keep tokens, passwords and local environment files out of evidence and source control. Rehearse expiry, refresh and restart, then demonstrate a fresh scheduled task with the intended identity. Authentication is a maintained execution dependency.

Runtime inventory and upgrade evidence

Maintain an inventory connecting each deployment to its runtime image, Airflow version, executor, providers, dbt adapter, Cosmos integration and customized shared templates. Include the owning team, upgrade decision date and applicable support policy. Check current release notes and maintenance milestones when planning an upgrade; a runtime tag copied from a training example is not a long-term production policy.

An image that builds successfully may still change graph generation, connection resolution or operator behavior. Use a non-production environment with representative inputs and comparable configuration.

Define expected outcomes before upgrading so a changed result cannot be accepted merely because every task is green.

Regression area	*Evidence to retain*
Definition loading	All intended DAGs parse; expected tasks and dependencies remain present
dbt integration	Manifest, generated task graph and adapter behavior match the intended project
Authentication	Worker resolves the intended connection after refresh and restart
Execution	Scheduled runs finish against the correct partition and namespace
Failure and retry	Deliberate failure blocks publication; retry preserves input and candidate identity
Callbacks and alerts	Responsible team receives actionable, correctly routed information
Replay and recovery	Same-date replay and correction produce reconciled results without duplication
Release compatibility	Active-run handling follows the documented deployment strategy

For Portfolio, use the controlled quality failure, repair and replay sequence as a small regression workload. Preserve the historical business partition explicitly; the scheduler's logical date does not create newer holdings. The final classroom evidence task records checks and delivery evidence rather than granting consumers access.

Separate the code release decision from the runtime upgrade decision when that makes failures easier to isolate. Keep the previous validated artifact and a recovery procedure, while recognizing that restoring an image does not undo database changes or external notifications.

Vulnerability ownership and shared templates

A managed service maintains its service boundary. Teams remain responsible for their application dependencies, configured integrations and adopted template copies. Record who fixes each vulnerability and how the update reaches the deployed product. A centrally maintained template provides value only when its consuming repositories have an effective update path.

When a security gate blocks promotion, identify the affected package, installed version and available remediation. Evaluate compatibility, rebuild the artifact and run the relevant regression checks. Removing a gate without resolving the risk does not establish a maintained deployment. If remediation requires a temporary exception, use the organization's existing risk process and record its owner and expiry.

An inventory also prevents indefinite postponement. Agree maintenance capacity before a support deadline forces an urgent upgrade alongside business changes. Security remediation and runtime compatibility need evidence from the actual execution environment.

Releases while work is active

A release changes the software available to execute; an Airflow run performs work using that software. Daily processing can use the same release repeatedly. The release record and the product delivery record answer different questions.

DAG-only changes and image changes have different deployment paths. Dependency, plugin or runtime changes require image validation even when the visible DAG change is small. Deploy the complete intended DAG set for the selected deployment boundary, including unchanged workflows. A partial artifact can unintentionally remove definitions used by other products. Astronomer's deployment overview describes the supported deployment options.

Establish how the installed runtime, executor and bundle configuration associate code with active runs. Do running tasks retain old code? Which code do later tasks and retries receive? Are dependencies and

shared files versioned with the DAG? A claim about one deployment mode does not establish another mode's behavior.

Suppose a NAV run has already produced a candidate using one column name when a release renames the downstream task's expected column. Later tasks must consume a compatible interface or remain bound to the validated version. One strategy is to stop new admission, allow active work to finish, deploy and validate, then resume scheduling. Pausing a schedule alone does not prove that active tasks have drained.

Another strategy preserves compatibility across the transition, such as supporting both representations until all earlier work finishes. Rehearse that strategy, including retries. Preserve the business date, immutable input selection, candidate identity and release identity throughout; a task retry must not silently pick up a different supplier file or verify a replacement candidate under earlier evidence.

Capacity, remote compute and cost

Scheduler capacity supports parsing and timely scheduling. Worker capacity supports task execution within memory, CPU and concurrency limits. Remote Databricks compute supports the query or job itself. Increasing resources at one boundary cannot repair an unrelated constraint at another.

Observed symptom	*Investigation before changing capacity*
Slow definition loading or scheduling	Parse duration, scheduler utilization and DAG complexity
Tasks waiting to execute	Queue delay, worker availability, pools and concurrency limits
Worker termination	Peak memory, resource errors and task-local processing
Remote query taking longer	Warehouse/job metrics, contention and query plan
Submission rejected	Credentials, permissions, request limits and target availability

For example, a worker that submits a large warehouse query may use little CPU while waiting. A longer warehouse execution time does not alone justify a larger worker. Conversely, a task that loads results into memory may need worker capacity even when the remote query is fast. Gather representative periods and peak behavior; average utilization can conceal a memory limit.

Use supported queues or routing where task classes need different resources. Form a hypothesis, change the relevant setting and compare scheduling delay, task duration and delivery time. Include cost alongside those results. Consumer deadlines and isolation requirements matter more than maximizing utilization of every component.

Hibernation can reduce non-production resource use outside required periods. Verify what scheduling, queued work and service startup do in the configured arrangement. Rehearse credential recovery, dependency availability and readiness after waking. Include a responsible owner for restarting before planned tests; dormant development infrastructure cannot supply evidence until it is ready again.

Availability, disaster recovery and support

High availability keeps service functioning through failures within an operating environment. Disaster recovery establishes how service resumes after a broader disruption. A deployment's availability arrangement does not automatically recover source data, warehouse state, credentials or delivery evidence.

Define recovery objectives from the product agreement, then identify the state needed to achieve them: code and configuration, execution metadata, input identities, durable candidates, published relations and receipts. Exercise restoration and replay. Verify the recovered product independently before reopening its consumer interface.

Operational monitoring must connect infrastructure health with business delivery. Logs and metrics help locate a failing boundary; freshness, completeness and publication evidence establish whether the consumer received the agreed product. Ensure the owning team can access monitoring and incident systems before an incident.

Route support by the failing boundary. Product owners diagnose logic, quality and delivery impact. Internal platform teams address onboarding, shared configuration and managed patterns. Managed-service support addresses service issues through the applicable agreement. Keep current escalation contacts and internal coverage accessible, select severity from business impact and send a concise packet: environment, deployment, runtime, release, run/task identities, redacted error and affected delivery.

An upstream readiness event supplies a producer milestone. The consumer still needs evidence of its own transformation, checks and publication. Carry dataset identity, business date and release or candidate identity across the dependency. Restoration of orchestration alone does not establish restored delivery.

Review questions

1. Which runtime, application and delivery responsibilities belong to the product team, platform team and managed service?
2. How would you detect that two apparently separate environments write to the same warehouse relations?
3. Why can terminal OAuth sign-in succeed while a fresh scheduled worker task fails?
4. Which regression evidence would you require before changing the runtime and dbt adapter together?
5. A NAV run is active during a column rename. How would you preserve compatible execution, input identity and candidate evidence?
6. Which observations distinguish scheduler delay, worker saturation and remote warehouse contention?
7. What state and checks must be restored before declaring recovery of a published product?